PROYECTO IA

De la Neurona al LLM

Un viaje al corazón de la Inteligencia Artificial

Construye tu propia IA desde cero, con Python

CRÉDITOS

Carlos Soria Martínez
Autor de la obra, director del proyecto, responsable de la concepción, estructura, supervisión académica y revisión final de todos los contenidos.

Herramientas de Inteligencia Artificial utilizadas

Durante el proceso de investigación, redacción, revisión y desarrollo de esta obra se han empleado diversas herramientas de inteligencia artificial como apoyo técnico y creativo. Su utilización ha contribuido a agilizar determinadas tareas, sin sustituir en ningún caso la supervisión, el criterio ni la responsabilidad intelectual del autor.

- **Claude (Anthropic)**: asistencia en la redacción de contenidos, diseño didáctico y desarrollo de los programas en Python incluidos en el libro.
- **ChatGPT (OpenAI)**: apoyo en la generación, revisión y mejora de textos, explicación de conceptos, elaboración de ejemplos, asistencia editorial y generación de los Diagramas 11.2 y 12.1.
- **Gemini (Google)**: generación y apoyo en la creación de ilustraciones e imágenes.
- **DeepSeek**: asistencia en el análisis de software, revisión técnica y evaluación de código.
- **NotebookLM (Google)**: organización, síntesis y consulta de documentación de referencia durante el desarrollo del proyecto.

Nota editorial

> Esta obra ha sido desarrollada mediante un modelo de inteligencia aumentada, en el que la experiencia, el criterio y la supervisión del autor se combinan con las capacidades de diversas herramientas de inteligencia artificial. La responsabilidad intelectual, académica y editorial de la selección, organización, validación y presentación de los contenidos corresponde exclusivamente al autor.

AGRADECIMIENTOS

Mi más profundo agradecimiento es para Gustavo López Rubio, catedrático de la Universidad Pontificia de Salamanca en Madrid, quien confió en mí mucho más allá de lo razonable y en más de una ocasión. De él no solo aprendí ingeniería, sino también una forma de afrontar las adversidades de la vida que siempre me acompañó.

Quiero expresar también mi gratitud a mis compañeros Enrique Fernández Rasero y Alfonso Alejandre Casado, por compartir conmigo tantos años de trabajo, aprendizaje y amistad.

Por último, deseo recordar de manera muy especial a la persona que despertó y alimentó mi interés por estos temas: Carlos Galán Pascual. Fue hace ya muchos años, en la cafetería de la universidad. Él estaba terminando la carrera y yo apenas comenzaba. Aún conservo el recuerdo de aquella gruesa carpeta de cartón repleta de papeles garabateados, servilletas llenas de esquemas e ideas aparentemente descabelladas. Tarjetas de cartulina perforada, listados impresos en el inconfundible «papel pijama» —como llamábamos entonces al papel continuo, por sus finas líneas horizontales de color azul claro— (el del SISSI 2, que todavía conservo) y programas en COBOL completaban aquel fascinante panorama.

Entonces jamás habría imaginado que, a lo largo de nuestras vidas, aquellas conversaciones y aquellas ideas llegarían tan lejos.

Edición digital, edición impresa, colaboración y licencia de uso

Este libro nace con la vocación de facilitar el acceso al conocimiento y alcanzar la mayor difusión posible.

La versión digital oficial de esta obra puede descargarse gratuitamente escaneando el código QR de la última página del libro o accediendo al repositorio oficial de GitHub, cuya dirección figura también allí. En dicho repositorio se publicarán las versiones oficiales del libro, las correcciones de erratas y, cuando proceda, material complementario o nuevas ediciones.

Quienes prefieran disponer de una copia física podrán adquirir la edición impresa bajo demanda a través de Amazon.

Comentarios, sugerencias y erratas

Las sugerencias, propuestas de mejora o comunicaciones de erratas pueden remitirse a:

`de.la.neurona.al.llm@gmail.com`

El autor agradece sinceramente todas las aportaciones recibidas. No obstante, debido a limitaciones de tiempo, no podrá mantener correspondencia ni responder individualmente a los mensajes. Todas las comunicaciones serán leídas y, cuando proceda, serán tenidas en cuenta en futuras ediciones del libro.

Donaciones

La versión digital de este libro seguirá estando disponible gratuitamente para cualquier lector.

Si considera que esta obra le ha resultado útil y desea contribuir a su mantenimiento y mejora, puede realizar una donación completamente voluntaria mediante el enlace o código QR indicado en el repositorio oficial.

Su apoyo ayudará a sufragar los costes de edición, revisión, publicación y mantenimiento de futuras versiones. La realización o no de una donación no condiciona en modo alguno el acceso al contenido del libro.

Licencia de uso

© 2026 Carlos Soria Martínez

ISBN de la edición impresa: 9798193023975

Esta obra se distribuye bajo la licencia **Creative Commons Atribución–NoComercial–SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0)**

Usted es libre de:

- Copiar, descargar y compartir esta obra por cualquier medio o formato.

Siempre que se respeten las siguientes condiciones:

- **Atribución (BY):** deberá reconocer adecuadamente la autoría e indicar la fuente.
- **No Comercial (NC):** no podrá utilizar esta obra con fines comerciales.
- **Sin Obras Derivadas (ND):** no podrá distribuir versiones modificadas, adaptadas o transformadas de esta obra.

El autor anima expresamente a la máxima difusión de la versión oficial de esta obra, respetando su integridad y las condiciones establecidas por esta licencia.

El texto completo de la licencia puede consultarse en:

`https://creativecommons.org/licenses/by-nc-nd/4.0/deed.es`

Código fuente

Los programas de este libro y de su repositorio oficial se distribuyen bajo la licencia MIT (© 2026 Carlos Soria Martínez): pueden usarse, modificarse y redistribuirse libremente, también con fines comerciales, siempre que se conserve el aviso de copyright y de licencia. La licencia CC BY-NC-ND 4.0 se aplica al texto y a las ilustraciones del libro.

«El conocimiento crece cuando se comparte.

Gracias por leer, difundir y contribuir a mejorar esta obra.»

ÍNDICE

Prólogo: Una carta al futuro programador.....6

PARTE I — FUNDAMENTOS: DE LA NEURONA A LA RED

Capítulo 1: ¿Qué es la Inteligencia Artificial?.....(*) 10
Capítulo 2: La Neurona — Del Cerebro al Código.....(*) 16
Capítulo 3: Programando Nuestra Primera Neurona.....(*) 23
Capítulo 4: El Aprendizaje — Gradientes y Backpropagation.....(**) 31
Capítulo 5: Los Optimizadores — Cómo Aprende Realmente la Red.....(***) 41
Capítulo 6: Red Neuronal Multicapa (MLP).....(**) 57
Capítulo 7: Sobreajuste y Regularización.....(**) 65

PARTE II — DEL TEXTO AL TRANSFORMER

Capítulo 8: Del Número a la Palabra — Texto como Vectores.....(**) 90
Capítulo 9: Tokenización BPE — Del Texto al Token.....(**) 97
Capítulo 10: Redes Recurrentes — El Camino hacia el Transformer.....(***) 110
Capítulo 11: El Mecanismo de Atención.....(***) 124
Capítulo 12: El Transformer — La Arquitectura que lo Cambió Todo.....(***) 139

PARTE III — CONSTRUYE TU GPT-2

Capítulo 13: El Nano-LLM — Construyendo un Modelo de Lenguaje.....(***) 149
Capítulo 14: Estrategias de Muestreo — Top-k y Top-p.....(**) 202
Capítulo 15: ¿Memoriza o razona? — Del loro estadístico al razonador.....(***) 215
Capítulo 16: Tu código mueve GPT-2.....(***) 251

PARTE IV — DEL LABORATORIO AL MUNDO REAL

Capítulo 17: Del Nano-LLM a los Gigantes.....(*) 279
Capítulo 18: Evaluación de Modelos de Lenguaje.....(**) 288
Capítulo 19: Ética y Sesgos en Inteligencia Artificial.....(*) 301
Capítulo 20: Herramientas y próximos pasos.....(*) 311

APÉNDICES

Apéndice A: Resumen de Todos los Programas.....321
Apéndice B: Glosario (141 términos).....326
Apéndice C: Referencias y Recursos.....336
Apéndice D: Los Mismos Programas con PyTorch.....340
Apéndice E: Guía de Instalación (requirements.txt).....352
Apéndice F: De la Academia a la Industria.....354
Apéndice G: Visión por Computador y Redes Convolucionales.....358
Índice alfabético de materias.....373

Nivel de dificultad: * Introductorio ** Intermedio *** Avanzado

Los números de página corresponden al documento completo con este índice.

Prólogo: Una carta al futuro programador

Querido alumno:

Hay momentos en la historia en los que una tecnología cambia todo. La imprenta democratizó el conocimiento. La electricidad transformó la industria. Internet conectó el mundo. Y ahora, la Inteligencia Artificial está redefiniendo lo que significa pensar, crear y resolver problemas.

Este libro nació de una convicción: la mejor manera de entender la IA no es usarla, sino construirla. No como magia, sino como ingeniería. Cada componente que verás aquí —la neurona, la red, la atención, el transformer— tiene una razón matemática y computacional clara, y puedes programarlo tú mismo con pocas líneas de Python.

No te prometemos que al terminar este libro vayas a replicar GPT o Claude. Eso requiere miles de millones de parámetros, meses de entrenamiento y miles de GPUs. Lo que sí te prometemos es algo más valioso: vas a entender por qué funciona. Y esa comprensión es el cimiento sobre el que se construyen los grandes modelos.

El camino que seguiremos es ascendente. Empezaremos por el elemento más pequeño —una sola neurona artificial— y, paso a paso, iremos añadiendo capas, profundidad, y finalmente el mecanismo de atención que da vida a los LLMs (Large Language Models) modernos. Cada concepto nuevo se apoya en el anterior. Cada programa que escribirás hace, a escala didáctica, lo mismo que hacen los grandes modelos.

¿Por qué Python? Porque es el lenguaje de la ciencia de datos y el aprendizaje automático. Casi toda la investigación en IA se publica con código en Python. Aprenderlo aquí te abre la puerta al mundo real. Los programas principales de este libro usan solo NumPy —sin frameworks de alto nivel como TensorFlow o PyTorch— para que veas cada operación matemática sin ningún velo; PyTorch y Hugging Face aparecen al final, cuando ya sabes qué hacen por dentro.

> 💡 Este libro no requiere conocimientos previos de IA, pero sí de Python básico (variables, funciones, bucles) y algo de matemáticas de secundaria (funciones, potencias y logaritmos). Las matrices y las derivadas que hacen falta se explican en el propio libro (secciones 1.5 y 4.3), así que no te preocupes si no las recuerdas.

Cómo usar este libro

Cada capítulo tiene tres partes: una explicación teórica con analogías y diagramas en texto, los programas de Python comentados en detalle, y ejercicios propuestos para que experimentes por tu cuenta. No te limites a copiar el código: modifícalo, rómpelo, y vuelve a arreglarlo. Ahí es donde se produce el aprendizaje real.

Los programas están diseñados para ejecutarse directamente desde la terminal con:

📄 Ejecución de un programa

```
python programa.py
```

O desde un entorno como Jupyter Notebook, VS Code o PyCharm. En la sección 1.4 ("Preparando el entorno de trabajo") encontrarás un script que instala Python, NumPy y todo lo necesario en un entorno virtual aislado (entorno_ia) — es la vía que sigue este libro; no hace falta instalar Anaconda ni ninguna otra distribución de Python.

Cómo está organizado el libro

Los 20 capítulos se agrupan en cuatro partes. La Parte I construye una red neuronal desde una sola neurona; la Parte II convierte el texto en números y llega al Transformer; la Parte III programa un GPT-2 en miniatura y termina ejecutando con ese mismo código el GPT-2 real de OpenAI; la Parte IV sale del laboratorio: los grandes modelos, su evaluación, la ética y las herramientas para seguir. Siete apéndices completan el libro; el último, la visión por computador, presenta la otra gran familia del Deep Learning. Los asteriscos indican la dificultad de cada capítulo.

No hace falta leerlo todo en orden. Elige tu ruta:

- **Ruta del curioso** (sin programar): los capítulos marcados en la primera columna. Explican qué es un LLM, de dónde viene y qué consecuencias tiene, sin necesidad de ejecutar código.
- **Ruta del programador** (de la neurona a GPT-2): el camino más corto para construir un LLM y entenderlo por dentro. Termina en el Capítulo 16, con tu código moviendo GPT-2.
- **Ruta completa**: todos los capítulos en orden. Los que no están en la ruta del programador (optimizadores, sobreajuste, redes recurrentes, ¿memoriza o razona?) son los que convierten el saber hacer en saber por qué.

Cap.	Título	Nivel	Curioso	Programador	Completa
1	¿Qué es la Inteligencia Artificial?	*	●	●	●
2	La Neurona — Del Cerebro al Código	*	●	●	●
3	Programando Nuestra Primera Neurona	*		●	●
4	El Aprendizaje — Gradientes y Backpropagation	**		●	●
5	Los Optimizadores — Cómo Aprende Realmente la Red	***			●
6	Red Neuronal Multicapa (MLP)	**		●	●
7	Sobreajuste y Regularización	**			●
8	Del Número a la Palabra — Texto como Vectores	**		●	●
9	Tokenización BPE — Del Texto al Token	**		●	●
10	Redes Recurrentes — El Camino hacia el Transformer	***			●
11	El Mecanismo de Atención	***		●	●
12	El Transformer — La Arquitectura que lo Cambió Todo	***		●	●
13	El Nano-LLM — Construyendo un Modelo de Lenguaje	***		●	●
14	Estrategias de Muestreo — Top-k y Top-p	**		●	●
15	¿Memoriza o razona? — Del loro estadístico al razonador	***			●
16	Tu código mueve GPT-2	***		●	●
17	Del Nano-LLM a los Gigantes	*	●		●
18	Evaluación de Modelos de Lenguaje	**			●
19	Ética y Sesgos en Inteligencia Artificial	*	●		●
20	Herramientas y próximos pasos	*	●		●

Los programas del libro

A lo largo del texto encontrarás 45 programas completos organizados por capítulo que se construyen unos sobre otros. Por capítulos, son estos:

* Programas 1A, 1B: Funciones de activación y la neurona artificial
* Programa 2: Una neurona que aprende (puerta lógica AND)
* Programas 3, 4: Red neuronal multicapa y resolución de XOR
* Programas O-1 a O-4: Los optimizadores — SGD, Momentum, Adam, AdamW
* Programas R-1 a R-4: Sobreajuste, regularización L2, Dropout y Early Stopping
* Programa 5: Tokenizador y tabla de embeddings

* Programas BPE-1, BPE-2: Entrenador y codificador Byte Pair Encoding
* Programas RNN-1, RNN-2: Redes recurrentes y el gradiente que desaparece
* Programa 6: Mecanismo de self-attention paso a paso
* Programa 7: Bloque Transformer completo
* Programa 8: El Nano-LLM — de los datos al texto generado
* Programas S-1, S-2: Estrategias de muestreo Top-k y Top-p
* Programas RZ-1 a RZ-4: ¿Memoriza o razona? — de la memorización al razonamiento paso a paso
* Programas 8 bis y G-1 a G-4: Tu código mueve GPT-2 — los pesos reales de OpenAI en tu forward(), mirar dentro, KV-cache y ajuste fino
* Programas EVAL-1, EVAL-2: Perplejidad, BLEU y benchmarks de evaluación
* Programas 9A, 9B: Convolución manual y CNN entrenada
* Programas E-1, E-2: Detección de sesgos y métricas de equidad
* Programas PT-1 a PT-7: Los programas principales, reimplementados con PyTorch

El Apéndice A contiene una ficha completa de cada programa. El Apéndice D muestra las versiones con PyTorch de los programas principales.

¡Comencemos el viaje!

PARTE I

Fundamentos: de la neurona a la red

La neurona artificial, cómo aprende con el descenso por gradiente y la retropropagación, los optimizadores que usan los LLM y las redes multicapa, con el sobreajuste como primer gran enemigo.

Capítulo 1: ¿Qué es la Inteligencia Artificial? (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Distinguir con precisión entre Inteligencia Artificial, Machine Learning y Deep Learning.
> ✓ Explicar en qué consiste, fundamentalmente, un LLM moderno.
> ✓ Tener el entorno Python y NumPy funcionando y verificado.
> ✓ Realizar tu primera operación matricial con NumPy, la base de todo el libro.

1.1 De la ciencia ficción a tu bolsillo

La idea de construir máquinas que piensen es muy antigua, pero la Inteligencia Artificial como disciplina nació en el verano de 1956, en un taller de la Universidad de Dartmouth (Estados Unidos) para el que se acuñó el término. Dos años después, Frank Rosenblatt presentó el perceptrón, la primera neurona artificial capaz de aprender de los datos (Capítulo 2). Siguieron décadas de entusiasmo y de decepción: los llamados inviernos de la IA, cuando las promesas no se cumplían y la financiación desaparecía.

Todo cambió en poco más de diez años. En 2012, una red neuronal profunda, AlexNet, ganó por un margen enorme el concurso de reconocimiento de imágenes ImageNet. En 2017, investigadores de Google presentaron el Transformer, la arquitectura que construirás en este libro. Y en noviembre de 2022, ChatGPT puso un modelo de lenguaje al alcance de cualquiera: según las estimaciones de la época, en dos meses tenía cien millones de usuarios.

¿Qué cambió? Tres factores se alinearon en la segunda década del siglo XXI:

* Datos: Internet generó cantidades masivas de texto, imágenes y audio. Más datos significan modelos más capaces.
* Potencia de cómputo: Las GPUs (unidades de procesamiento gráfico), diseñadas para videojuegos, resultaron perfectas para las operaciones matriciales de las redes neuronales.
* Algoritmos: Investigadores refinaron técnicas de entrenamiento, arquitecturas de redes y funciones de optimización que hacen el aprendizaje mucho más eficiente.

1.2 La jerarquía: IA, ML y Deep Learning

Muchas veces estos términos se usan como sinónimos, pero tienen significados distintos y forman una jerarquía:

> ℹ️ Inteligencia Artificial ⊃ Aprendizaje Automático ⊃ Aprendizaje Profundo

La Inteligencia Artificial (IA) es el campo más amplio: cualquier técnica que permita a una máquina realizar tareas que normalmente requerirían inteligencia humana. Incluye sistemas expertos con reglas escritas a mano, algoritmos de búsqueda, lógica difusa, y muchos otros enfoques.

El Aprendizaje Automático (Machine Learning, ML) es un subconjunto de la IA donde la máquina aprende patrones a partir de datos, sin que el programador escriba reglas explícitas. El programador diseña el modelo y el algoritmo de aprendizaje; los datos hacen el resto.

El Aprendizaje Profundo (Deep Learning, DL) es un subconjunto del ML que usa redes neuronales con muchas capas ('profundas'). Es la tecnología detrás de los reconocedores de voz, los traductores automáticos, los generadores de imágenes y, por supuesto, los LLMs.

En este libro nos centraremos en el Deep Learning, específicamente en las arquitecturas que dan lugar a los LLMs. Pero iremos despacio, empezando desde el principio absoluto: una sola neurona.

1.3 ¿Qué es un LLM?

Un LLM (Large Language Model, Modelo de Lenguaje Grande) es una red neuronal con millones o miles de millones de parámetros entrenada sobre enormes volúmenes de texto. Sus capacidades emergentes son sorprendentes: puede escribir código, traducir idiomas, responder preguntas, resumir documentos o mantener una conversación coherente.

Ejemplos actuales son GPT (OpenAI), Claude (Anthropic), Gemini (Google), Llama (Meta) o los modelos abiertos Mistral, Qwen y DeepSeek. Todos comparten la misma arquitectura base: el Transformer, propuesto en 2017 por investigadores de Google en el artículo 'Attention is All You Need'. A lo largo de este libro desmontaremos ese transformer pieza a pieza.

> ℹ️ Un LLM, en esencia, hace una sola cosa: dado un trozo de texto, predice cuál es el siguiente token (un trozo de texto: una palabra, parte de una palabra o un carácter) más probable. Esa tarea aparentemente simple, aprendida por un modelo de miles de millones de parámetros sobre billones de tokens de texto, produce capacidades sorprendentemente ricas.

1.4 Preparando el entorno de trabajo

Antes de escribir una sola línea de código necesitas preparar tu entorno de trabajo: el conjunto de herramientas que van a permitirte ejecutar todos los programas del libro de forma organizada y sin conflictos.

Hemos preparado un script de instalación automática para cada sistema operativo. Elige el tuyo, ejecuta el script y en 15-25 minutos tendrás todo listo. No necesitas tomar ninguna decisión durante el proceso.

📦 **Qué instala el script:**

- **Python 3.11 o posterior —** el lenguaje de programación del libro.
- **Entorno virtual** — carpeta aislada que contiene solo las librerías de este proyecto.
- **NumPy** — cálculo numérico. Base de todos los programas (Caps. 1-20).
- **Matplotlib** — gráficas y visualización de resultados.
- **PyTorch CPU** — aprendizaje profundo (Apéndice D).
- **Transformers + Requests — Hugging Face, para el Capítulo 20.**
- **Visual Studio Code** — editor de código con soporte completo de Python.

🪟 WINDOWS 10 / 11 — instalar_windows.bat

1. Descarga el archivo instalar_windows.bat del repositorio del libro.
2. Haz clic derecho sobre él → «Ejecutar como administrador».
3. El script te irá mostrando el progreso paso a paso. Acepta cualquier ventana de permisos que aparezca.
4. Cuando veas «INSTALACION COMPLETADA», el entorno está listo.

Verificación rápida — abre CMD (Símbolo del sistema; en PowerShell la orden de activación es otra) y escribe:

```
%USERPROFILE%\entorno_ia\Scripts\activate.bat
python -c "import numpy; print(numpy.__version__)"
```

🐧 LINUX (Ubuntu / Debian) — instalar_linux.sh

1. Descarga el archivo instalar_linux.sh del repositorio.
2. Abre una terminal en la carpeta donde lo guardaste.
3. Hazlo ejecutable y lánzalo:

```
chmod +x instalar_linux.sh
./instalar_linux.sh
```

4. Cuando aparezca «Script completado correctamente», el entorno está listo.

Verificación rápida:

```
source ~/entorno_ia/bin/activate
python -c "import numpy; print(numpy.__version__)"
```

🍎 macOS (versión 12 o posterior) — instalar_mac.sh

1. Descarga el archivo instalar_mac.sh del repositorio.
2. Abre la aplicación Terminal y navega hasta la carpeta donde lo guardaste (por ejemplo: cd ~/Downloads).
3. Hazlo ejecutable y lánzalo:

```
chmod +x instalar_mac.sh
./instalar_mac.sh
```

4. En Mac con Apple Silicon (M1/M2/M3/M4) PyTorch se instala con aceleración de chip integrada automáticamente. En Mac con procesador Intel, PyTorch solo llega a la versión 2.2, que necesita NumPy 1: el script la instala en un segundo entorno, entorno_ia_pytorch, que solo se usa en el Apéndice D.
5. Cuando aparezca «Script completado correctamente», el entorno está listo.

Verificación rápida:

```
source ~/entorno_ia/bin/activate
python -c "import numpy; print(numpy.__version__)"
```

Uso diario del entorno

El entorno virtual solo se activa una vez por sesión de trabajo. Cuando termines, cierra la terminal o escribe deactivate. La próxima vez que quieras trabajar con el libro, actívalo de nuevo:

Windows (CMD):

```
%USERPROFILE%\entorno_ia\Scripts\activate.bat
```

Linux y macOS:

```
source ~/entorno_ia/bin/activate
```

> 💡 **¿Tienes tarjeta gráfica NVIDIA?** Los scripts instalan PyTorch en modo CPU, que funciona perfectamente para todos los ejercicios del libro. Si quieres activar la GPU al terminar, desinstala la versión CPU y sigue las instrucciones de la sección D.3 del Apéndice D.

1.5 NumPy en cinco minutos

Casi todo lo que hace una red neuronal se reduce a operaciones con listas y tablas de números. En matemáticas se llaman vectores y matrices, y NumPy es la librería de Python que las maneja de forma rápida. Para empezar bastan cuatro ideas.

Un vector es una lista ordenada de números, como [1, 2, 3]; en NumPy, np.array([1, 2, 3]). Una matriz es una tabla de números con filas y columnas. Su forma (shape) dice cuántas tiene: una matriz de 2 filas y 3 columnas tiene forma (2, 3), y un vector de 3 números, forma (3,).

Las operaciones +, −, * y / actúan elemento a elemento: [1, 2, 3] * [4, 5, 6] = [4, 10, 18]. El producto punto (o producto escalar) multiplica elemento a elemento y suma el resultado: [1, 2, 3] · [4, 5, 6] = 4 + 10 + 18 = 32. Es exactamente la suma ponderada de una neurona, que verás en el Capítulo 2. En el texto escribimos la multiplicación con un punto centrado, a · b, para no confundirla con el punto decimal.

El producto de matrices, que en NumPy se escribe con @, hace muchos productos punto a la vez: el elemento (i, j) del resultado es el producto punto de la fila i de la primera matriz por la columna j de la segunda. Por eso las dimensiones interiores tienen que coincidir: una matriz (2, 3) por una (3, 2) da una (2, 2).

📄 Ejemplo — NumPy en cinco minutos

```
import numpy as np

v = np.array([1, 2, 3])
w = np.array([4, 5, 6])
print(v.shape)           # forma: un vector de 3 números
print(v * w)             # elemento a elemento
```

```
print(v @ w)             # producto punto: 1*4 + 2*5 + 3*6

A = np.array([[1, 2, 3],
              [4, 5, 6]])            # 2 filas y 3 columnas
B = np.ones((3, 2))                  # 3 filas y 2 columnas, todo unos
print(A.shape, B.shape, (A @ B).shape)
print(A @ B)
print(A.T.shape)         # .T, la traspuesta: filas <-> columnas
print(A.sum(axis=0))     # suma de cada columna
```

> Salida del programa:

```
(3,)
[ 4 10 18]
32
(2, 3) (3, 2) (2, 2)
[[ 6.  6.]
 [15. 15.]]
(3, 2)
[5 7 9]
```

Las dos últimas operaciones aparecerán a partir del Capítulo 6: .T intercambia filas y columnas (la traspuesta), y sum(axis=0) suma cada columna. Con esto tienes lo necesario para el Ejercicio 1.1.

✏️ EJERCICIOS — Capítulo 1 — ¿Qué es la IA?

Antes de pasar al capítulo siguiente, intenta resolver estos ejercicios por tu cuenta. Compara luego tu respuesta con la solución propuesta: verás qué ideas has asentado y cuáles conviene repasar.

Ejercicio 1.1 Vectores y matrices con NumPy

Crea dos vectores de 4 componentes. Calcula su suma, diferencia, producto elemento a elemento y producto punto. Luego crea dos matrices (2×3 y 3×2) y multiplícalas (sección 1.5): ¿cuál es el tamaño del resultado?

> 💡 💡 Pista: El operador @ multiplica matrices. El producto punto de vectores produce un escalar; la multiplicación de matrices produce otra matriz.

✅ Solución:

📄 ejercicio_1_1.py

```
import numpy as np

v1 = np.array([1.0, 2.0, 3.0, 4.0])
v2 = np.array([4.0, 3.0, 2.0, 1.0])

print('Suma:                    ', v1 + v2)
print('Diferencia:              ', v1 - v2)
print('Producto elem. a elem.: ', v1 * v2)
print('Producto punto (dot):    ', np.dot(v1, v2))  # 1*4+2*3+3*2+4*1 = 20

# Multiplicación de matrices
A = np.array([[1, 2, 3],
              [4, 5, 6]])       # (2, 3)
B = np.array([[7, 8],
              [9, 10],
              [11, 12]])        # (3, 2)

C = A @ B                      # (2, 3) @ (3, 2) = (2, 2)
print('\nA (2x3):\n', A)
print('B (3x2):\n', B)
print('A @ B (2x2):\n', C)
print('Regla: (mxn) @ (nxp) = (mxp)')
```

> ℹ️ La regla de las dimensiones: para multiplicar A(m×n) @ B(n×p), las dimensiones internas (n) deben coincidir. El resultado tiene dimensión (m×p). Esta regla es fundamental en Deep Learning.

Ejercicio 1.2 Softmax y la propiedad de suma = 1

Softmax convierte una lista de números cualesquiera en una distribución de probabilidad: valores entre 0 y 1 que suman exactamente 1. Su fórmula es softmax(z)_i = e^{z_i} / Σ_j e^{z_j} — cada elemento se exponencia y se divide entre la suma de todos los exponenciales. La usaremos constantemente a partir de aquí, para convertir las puntuaciones internas de una red neuronal en probabilidades sobre qué token generar a continuación.

Implementa la función softmax desde cero y verifica que, para cualquier vector de entrada (incluso con valores muy grandes como [100, 200, 300]), la suma de las salidas es siempre 1 (salvo errores de redondeo del orden de 10^{-16}, inevitables con decimales en coma flotante; para comparar dos números así, usa np.isclose).

> 💡 💡 Pista: El truco de restar el máximo antes de calcular los exponenciales (np.exp(z - z.max())) evita el desbordamiento numérico sin cambiar el resultado matemático.

✅ **Solución:**

📄 ejercicio_1_2.py

```
import numpy as np

def softmax(z):
    """Implementación estable de softmax"""
    e = np.exp(z - z.max())   # restar el máximo evita overflow
    return e / e.sum()

# Vectores de prueba, incluyendo casos extremos
vectores = [
    np.array([1.0, 2.0, 3.0]),
    np.array([100.0, 200.0, 300.0]),   # valores muy grandes
    np.array([-5.0, 0.0, 5.0]),
    np.array([0.0, 0.0, 0.0]),          # todos iguales -> uniforme
]

print('Verificación: softmax siempre suma 1.0')
print(f"{'Vector':>28}  {'Softmax':>26}  {'Suma':>10}")
print('-' * 68)
for v in vectores:
    s = softmax(v)
    print(f'{str(v):>28}  {str(s.round(4)):>26}  {s.sum():.8f}')

# Sin el truco del máximo, np.exp(300) ≈ 2e130 aún cabe en float64, pero
# np.exp(1000) -> inf (overflow): z - z.max() lo evita en cualquier caso
# Con el truco: exp([100-300, 200-300, 300-300]) = exp([-200,-100,0])
```

> ℹ️ El vector [0, 0, 0] produce [0.333, 0.333, 0.333]: si todas las puntuaciones son iguales, la distribución es uniforme. Es lo que pasa, aproximadamente, al principio del entrenamiento, cuando los pesos son pequeños y las puntuaciones muy parecidas.

Ejercicio 1.3 El efecto de la temperatura en softmax

La temperatura τ modifica el softmax dividiendo los logits: softmax(z/τ). Prueba con τ = 0.1, 0.5, 1.0, 2.0 y 10.0 sobre el vector [2.0, 1.5, 0.5] y describe qué ocurre con la distribución de probabilidad en cada caso.

> 💡 💡 Pista: Con τ -> 0, la distribución se concentra en el máximo (determinista). Con τ -> ∞, la distribución se vuelve uniforme (aleatoria). Este es el mismo parámetro que usaremos en la generación del Nano-LLM.

✅ **Solución:**

📄 ejercicio_1_3.py

```
import numpy as np

def softmax_temp(z, tau=1.0):
    z_t = z / tau
    e   = np.exp(z_t - z_t.max())
    return e / e.sum()

z = np.array([2.0, 1.5, 0.5])
print(f'Vector: {z}')
print(f"{'Tau':>8}  {'P(A)':>8}  {'P(B)':>8}  {'P(C)':>8}  Comportamiento")
print('-' * 58)

for tau, desc in [
    (0.1, 'muy determinista'),
    (0.5, 'algo concentrada'),
    (1.0, 'neutro (default)'),
    (2.0, 'algo uniforme'),
    (10.0,'casi uniforme'),
]:
    p = softmax_temp(z, tau)
    print(f'{tau:>8.1f}  {p[0]:>8.4f}  {p[1]:>8.4f}  {p[2]:>8.4f}  {desc}')

print()
print('Con tau=0.1: casi todo el peso en el mayor (2.0)')
print('Con tau=10:  los tres tokens tienen probabilidad similar')
print('En generacion de LLMs: tau=0.8 da buen balance')
```

Capítulo 2: La Neurona — Del Cerebro al Código (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Describir la analogía entre la neurona biológica y el modelo matemático.
> ✓ Escribir la ecuación completa de una neurona artificial: suma ponderada + activación.
> ✓ Diferenciar sigmoid, ReLU, tanh y softmax, y saber cuándo usar cada una.
> ✓ Explicar por qué la no-linealidad es imprescindible para el aprendizaje profundo.

2.1 La neurona biológica

Nuestro cerebro contiene aproximadamente 86 mil millones de neuronas interconectadas. Cada neurona es una célula especializada en recibir, procesar y transmitir señales eléctricas. Aunque la IA no pretende copiar exactamente la biología, la neurona artificial se inspira en los principios básicos del funcionamiento neurológico.

Una neurona biológica tiene tres partes principales:

* Dendritas: ramificaciones que reciben señales de otras neuronas. Pueden recibir miles de señales simultáneas.
* Soma (cuerpo celular): integra todas las señales recibidas. Si la suma supera cierto umbral, la neurona se 'activa'.
* Axón: fibra que transmite la señal de salida hacia otras neuronas a través de las sinapsis.

Simplificando mucho, lo que hace el soma es una suma ponderada: cada señal entrante tiene un 'peso sináptico' (la fuerza de la conexión), y el soma suma todas esas señales multiplicadas por sus pesos. Si el total supera el umbral de activación, la neurona dispara una señal. Una neurona real es mucho más compleja (emite impulsos a lo largo del tiempo y su respuesta depende de procesos químicos), pero el modelo artificial se queda solo con esta idea. El Diagrama 2.1 resume visualmente todo este proceso.

> ℹ️ La analogía es directa: entradas -> dendritas, pesos sinápticos -> pesos en el modelo, suma ponderada -> cuerpo celular, función de activación -> umbral de disparo, salida -> axón. Lo que la biología hace con química y electricidad, nosotros lo hacemos con aritmética.

2.2 El modelo matemático de la neurona artificial

En 1943, Warren McCulloch y Walter Pitts propusieron el primer modelo matemático de neurona artificial. En 1958, Frank Rosenblatt presentó el Perceptrón, el predecesor directo de todas las neuronas artificiales modernas.

El modelo matemático de una neurona con N entradas es el siguiente:

📄 **Ecuación de la neurona (en pseudocódigo matemático)**

```
# Dadas:
#   x1, x2, ..., xN  -> entradas (los datos)
#   w1, w2, ..., wN  -> pesos (parámetros aprendidos)
#   b                -> sesgo (bias, un parámetro extra)
#   f(.)             -> función de activación

# Paso 1: Suma ponderada (pre-activación)
z = x1*w1 + x2*w2 + ... + xN*wN + b
  = sum(xi * wi) + b
  = x · w + b        (producto punto en notación vectorial)

# Paso 2: Activación
salida = f(z)
```

El sesgo b desplaza el umbral de la neurona: sin él, z valdría 0 siempre que todas las entradas fueran 0; con un b positivo la neurona se «enciende» con más facilidad, y con uno negativo le cuesta más, con las mismas entradas.

Este modelo tiene una elegancia sorprendente: con solo dos pasos (suma ponderada + activación), una neurona puede aprender a separar dos clases de datos (siempre que una recta, o un plano, pueda separarlas: en el Capítulo 6 verás qué pasa cuando no), detectar patrones simples o actuar como una puerta lógica. La magia emerge cuando conectamos miles o millones de neuronas en capas.

2.3 Funciones de activación: por qué son imprescindibles

Sin la función de activación, $z = x \cdot w + b$ es simplemente una función afín (lineal más un desplazamiento; en redes neuronales se suele llamar «lineal» a secas). El problema es que, sin importar cuántas capas añadas, la composición de funciones afines sigue siendo afín. Por ejemplo, dos capas lineales seguidas, $W_2 \cdot (W_1 \cdot x + b_1) + b_2$, equivalen a una sola: $(W_2 \cdot W_1) \cdot x + (W_2 \cdot b_1 + b_2)$, una matriz por x más un vector. Esto limita enormemente la capacidad del modelo para aprender relaciones complejas.

Las funciones de activación introducen no-linealidad, que es la clave de la expresividad de las redes neuronales. Las más utilizadas actualmente son:

Sigmoid (σ)

Comprime cualquier número real al intervalo (0, 1). Históricamente fue la más popular, y sigue usándose en la capa de salida de clasificadores binarios.

📄 Función sigmoid

```
σ(z) = 1 / (1 + e^(-z))

# Propiedades:
# * Salida siempre entre 0 y 1
# * Interpretable como probabilidad
# * Derivada: σ'(z) = σ(z) · (1 - σ(z))
# * Problema: saturación (gradientes muy pequeños para z grande)
```

ReLU (Rectified Linear Unit)

La más usada hoy en capas internas (ocultas). Es computacionalmente eficiente y evita la saturación de sigmoid para entradas positivas.

📄 Función ReLU

```
ReLU(z) = max(0, z)

# Propiedades:
# * Si z > 0, la salida es z (idéntica a la entrada)
# * Si z ≤ 0, la salida es 0 (la neurona se 'apaga')
# * Derivada: 1 si z > 0, 0 si z ≤ 0
# * Muy rápida de calcular
# * Para z > 0 su derivada vale 1: el gradiente no se encoge
# * Pega: si z queda siempre <= 0, la neurona 'muere'
#   (gradiente 0). En z = 0 la derivada se fija por convenio
```

Tanh (tangente hiperbólica)

Similar a sigmoid pero con rango (-1, 1). Útil cuando queremos que la salida pueda ser negativa.

📄 Función tanh

```
tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z))

# Propiedades:
# * Salida entre -1 y 1
# * Simétrica respecto al 0
# * Útil en RNNs y LSTM (Capítulo 10)
```

Softmax

Se usa en la capa de salida de los clasificadores multiclase y, como veremos en el Capítulo 11, dentro del mecanismo de atención. Convierte un vector de números en una distribución de probabilidad (todos positivos, suman 1).

📄 **Función softmax**

```
softmax(z)_i = e^(z_i) / sum(e^(z_j) para todo j)

# Ejemplo: z = [2.0, 1.0, 0.5]
# e^2.0 = 7.39,  e^1.0 = 2.72,  e^0.5 = 1.65
# suma = 11.76
# softmax = [0.628, 0.231, 0.140]  (suman 1.0)

# Es la función clave en los LLMs para elegir el siguiente token
```

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - Una neurona artificial calcula una suma ponderada ($z = x \cdot w + b$) y aplica una función de activación no lineal.
> - Sigmoid, ReLU, tanh y softmax tienen propiedades distintas: rango de salida, saturación y casos de uso.
> - Sin no-linealidad, ninguna cantidad de capas superaría la expresividad de una sola capa lineal.

DIAGRAMA 2.1 — LA NEURONA ARTIFICIAL

De la información a la decisión

La neurona artificial es la unidad básica de procesamiento en las redes neuronales. Recibe entradas, las combina con pesos aprendidos, aplica una activación no lineal y produce una salida o predicción.

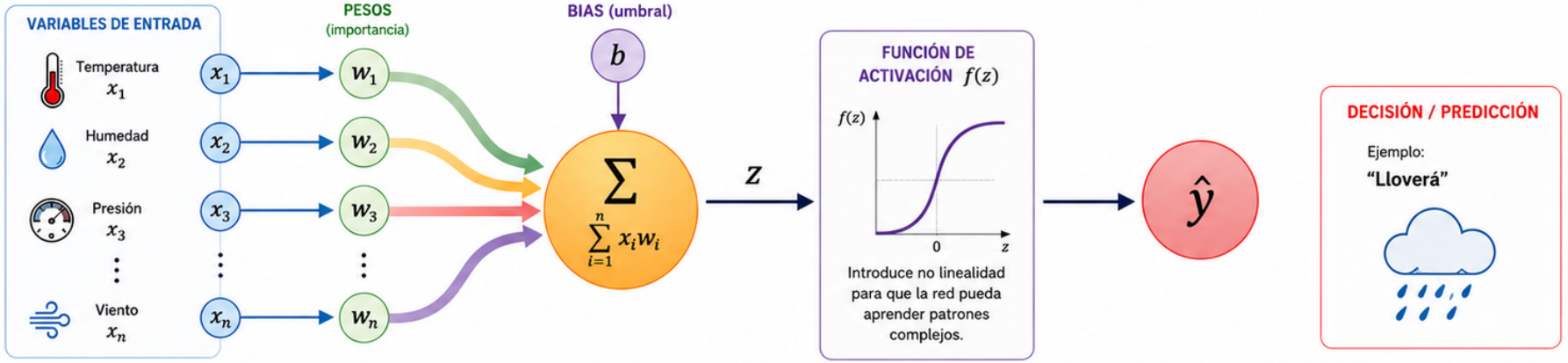
1 ENTRADAS — Lo que percibimos del entorno.

2 EVIDENCIA PONDERADA — Cada entrada tiene una importancia (peso).

3 COMBINACIÓN — Integramos la evidencia y ajustamos el umbral (bias).

4 ACTIVACIÓN — Introducimos no linealidad para tomar la decisión.

5 SALIDA — Nuestra predicción o decisión.



Entradas (x) — Características del problema o del entorno.

Pesos (w) — Parámetros aprendidos que indican la importancia de cada entrada.

Bias (b) — Ajuste adicional que desplaza el umbral de activación.

Combinación lineal — Suma ponderada de todas las entradas más el bias.

$$z = \sum_{i=1}^{n} x_i w_i + b$$

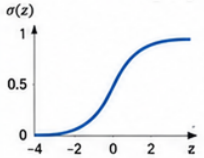
Función de activación — Aplica no linealidad: $f(z)$ transforma z en la salida.

Salida (ŷ) — Resultado final: predicción, probabilidad o decisión.

FUNCIONES DE ACTIVACIÓN MÁS COMUNES

1 SIGMOID (σ)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



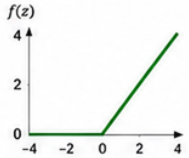
Rango: (0, 1)

Uso: Clasificación binaria

✓ Devuelve probabilidades.
✗ Saturación para valores extremos (gradientes pequeños).

2 ReLU

$$f(z) = \max(0, z)$$



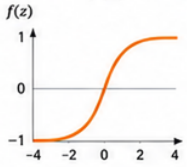
Rango: [0, +∞)

Uso: Capas ocultas profundas

✓ Muy eficiente de calcular.
✗ Puede producir "neuronas muertas" ($z \leq 0$).

3 TANH

$$f(z) = \tanh(z)$$



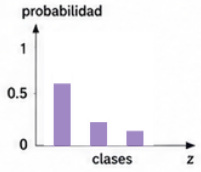
Rango: (-1, 1)

Uso: RNN y modelos que requieren salidas centradas en cero

✓ Salidas centradas en cero.
✗ Saturación para valores extremos.

4 SOFTMAX

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



Rango: Probabilidades (0, 1) que suman 1

Uso: Clasificación multiclase

✓ Convierte en distribución de probabilidad.
✗ No se usa en capas ocultas (solo salida).

¿CÓMO ELEGIR?

- Problema binario → Sigmoid
- Capas ocultas profundas → ReLU
- Redes recurrentes / Datos secuenciales → Tanh
- Clasificación multiclase → Softmax

EN RESUMEN

Cada neurona responde a una pregunta simple:
"¿La evidencia combinada es suficiente para activar esta característica?" → Si la respuesta es sí, la activación lo permite y la señal continúa hacia adelante en la red.

✏️ EJERCICIOS — Capítulo 2 — La Neurona Artificial

Dedica un rato a estos ejercicios antes de seguir leyendo. Al comparar tu solución con la propuesta descubrirás detalles de la neurona artificial que en una primera lectura pasan inadvertidos.

Ejercicio 2.1 Cálculo manual de una neurona

Sin usar código, calcula paso a paso la salida de una neurona con: entradas = [0.5, 0.8], pesos = [0.3, -0.4], sesgo = 0.2. Muestra todos los pasos: (1) cada producto $x_i \cdot w_i$, (2) la suma ponderada z, (3) la salida sigmoid(z). Luego verifica con Python.

> 💡 💡 Pista: z = 0.5 · 0.3 + 0.8 · (−0.4) + 0.2. La clave está en los signos: el segundo peso es negativo, así que esa entrada resta. El sigmoid de un número positivo pequeño estará ligeramente por encima de 0.5.

✅ **Solución:**

📄 **ejercicio_2_1.py**

```
import numpy as np

entradas = np.array([0.5,  0.8])
pesos    = np.array([0.3, -0.4])
sesgo    = 0.2

def sigmoid(z): return 1 / (1 + np.exp(-z))

# --- Cálculo paso a paso ---
p1 = entradas[0] * pesos[0]    # 0.5 x 0.30 =  0.15
p2 = entradas[1] * pesos[1]    # 0.8 x (-0.4) = -0.32
z  = p1 + p2 + sesgo           # 0.15 + (-0.32) + 0.20 = 0.03
sal = sigmoid(z)

print('=== CÁLCULO DETALLADO ===')
print(f'  x1.w1 = {entradas[0]} x {pesos[0]}  = {p1:.4f}')
print(f'  x2.w2 = {entradas[1]} x {pesos[1]} = {p2:.4f}')
print(f'  sesgo =                    {sesgo:.4f}')
print(f'  z     = {p1:.4f} + ({p2:.4f}) + {sesgo:.4f} = {z:.4f}')
print(f'  σ({z:.4f}) = 1/(1+e^{-z:.4f}) = {sal:.6f}')
print()
print(f'Como z={z:.4f} > 0, la salida ({sal:.4f}) > 0.5 ✓')
```

> ℹ️ Hacer este cálculo a mano al menos una vez es importante: deja claro que una neurona es solo aritmética básica. Todo el poder de las redes neuronales emerge de repetir esta sencilla operación millones de veces.

Ejercicio 2.2 Neurona con ReLU y comparación

Escribe una clase NeuronaFlex (en el Capítulo 3 construirás la clase Neurona completa) que pueda usar tanto sigmoid como ReLU, seleccionable al crear la neurona. Crea dos neuronas con los mismos pesos y sesgo (usa la misma semilla) y compara sus salidas para las entradas [1, 1, 1], [-1, -1, -1] y [0, 0, 0].

> 💡 💡 Pista: Con estos pesos, las entradas negativas dan una z negativa (−0.40): sigmoid produce un valor por debajo de 0.5, pero positivo, y ReLU, exactamente 0. Con otros pesos el signo de z podría cambiar. Para entradas positivas, sigmoid está acotada en (0, 1), mientras que ReLU puede superar 1.

✅ **Solución:**

📄 **ejercicio_2_2.py**

```
import numpy as np

class NeuronaFlex:
```

```
    def __init__(self, n, activacion='sigmoid', semilla=42):
        np.random.seed(semilla)
        self.W = np.random.randn(n) * 0.5
        self.b = 0.1
        self.activacion = activacion

    def calcular(self, x):
        z = np.dot(np.array(x), self.W) + self.b
        if self.activacion == 'sigmoid':
            return 1/(1+np.exp(-z)), z
        elif self.activacion == 'relu':
            return max(0.0, z), z
        elif self.activacion == 'tanh':
            return np.tanh(z), z

n_sig = NeuronaFlex(3, 'sigmoid')
n_rel = NeuronaFlex(3, 'relu')

casos = [[1.0,1.0,1.0], [-1.0,-1.0,-1.0], [0.0,0.0,0.0]]
print(f"{'Entradas':>22}  {'Sigmoid':>10}  {'ReLU':>10}  {'z':>8}")
print('-' * 57)
for x in casos:
    s, z = n_sig.calcular(x)
    r, _ = n_rel.calcular(x)
    print(f'{str(x):>22}  {s:>10.4f}  {r:>10.4f}  {z:>8.4f}')

print()
print('Diferencias clave:')
print('* Sigmoid: acotada en (0,1), nunca exactamente 0')
print('* ReLU:    puede ser 0 exacto (neurona apagada)')
print('* ReLU:    sin acotación superior')
```

Ejercicio 2.3 El papel del sesgo como umbral

Crea una neurona con 1 sola entrada y peso = 1.0. Mantén la entrada fija en x = 0.0 y varía el sesgo de -5 a +5 en pasos de 1. Observa y explica cómo el sesgo actúa como umbral de activación.

> 💡 💡 Pista: Cuando x=0, la neurona calcula simplemente sigmoid(b). El sesgo desplaza el punto de operación de la función sigmoid. Un sesgo muy negativo 'apaga' la neurona incluso para entradas positivas moderadas.

✅ Solución:

📄 **ejercicio_2_3.py**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))

peso   = 1.0
x_fijo = 0.0

print('Neurona con 1 entrada: w=1.0, x=0 (fija)')
print(f"{'Sesgo b':>10}  {'z=x*w+b':>10}  {'σ(z)':>10}  Estado")
print('-' * 50)

for b in range(-5, 6):
    z = x_fijo * peso + b
    s = sigmoid(z)
    if s < 0.12:   estado = 'prácticamente apagada (≈0)'
    elif s < 0.40: estado = 'inhibida'
    elif s < 0.60: estado = 'neutral (≈0.5)'
    elif s < 0.88: estado = 'activa'
    else:          estado = 'prácticamente encendida (≈1)'
    print(f'{b:>10.0f}  {z:>10.1f}  {s:>10.4f}  {estado}')

print()
print('Conclusión: el sesgo controla el umbral de activación.')
print('Sesgo negativo -> la neurona necesita entradas más fuertes')
print('Sesgo positivo -> la neurona se activa incluso con entradas débiles')
```

> ℹ️ Esta propiedad del sesgo es análoga al 'potencial de reposo' de una neurona biológica: el nivel de excitabilidad basal sin ningún estímulo externo.

Capítulo 3: Programando Nuestra Primera Neurona (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Implementar desde cero las cuatro funciones de activación fundamentales.
> ✓ Construir la clase Neurona completa en Python con NumPy.
> ✓ Entender por qué la inicialización aleatoria de pesos es imprescindible en una red.
> ✓ Interpretar la salida de una neurona no entrenada y explicar su comportamiento.

3.1 De la ecuación al código

Ahora vamos a implementar la neurona desde cero en Python. La idea es que cada línea de código corresponda directamente a un concepto matemático que ya conocemos. No habrá nada misterioso.

Crearemos una clase llamada Neurona que encapsula los pesos, el sesgo y la lógica de cálculo. Usaremos NumPy para las operaciones matemáticas.

PROGRAMA 1A — Funciones de activación

📄 **programa_1a_activaciones.py**

```
import numpy as np

# ============================================================
# FUNCIONES DE ACTIVACIÓN
# Cada función transforma un número de entrada en una salida
# ============================================================

def sigmoid(z):
    """Comprime cualquier número al rango (0, 1)"""
    return 1.0 / (1.0 + np.exp(-z))

def derivada_sigmoid(z):
    """Derivada de sigmoid — necesaria para el aprendizaje"""
    s = sigmoid(z)
    return s * (1.0 - s)

def relu(z):
    """Rectified Linear Unit — la más usada en capas ocultas"""
    return np.maximum(0.0, z)

def derivada_relu(z):
    """Derivada de ReLU: 1 donde z>0, 0 donde z≤0"""
    return (np.asarray(z) > 0).astype(float)   # asarray: funciona también con un
número suelto

def tanh_act(z):
    """Tangente hiperbólica — rango (-1, 1)"""
    return np.tanh(z)

def softmax(z):
    """Convierte un vector en distribución de probabilidad"""
    # Restamos el máximo para evitar desbordamiento numérico
    e_z = np.exp(z - np.max(z))
    return e_z / e_z.sum()

# -- Demostración ----------------------------------------
print('=== FUNCIONES DE ACTIVACIÓN ===')
print(f'{"Entrada":>8}  {"Sigmoid":>9}  {"ReLU":>6}  {"Tanh":>8}')
print('-' * 40)
for z in [-3.0, -1.0, -0.5, 0.0, 0.5, 1.0, 3.0]:
    print(f'{z:8.1f}  {sigmoid(z):9.4f}  {relu(z):6.2f}  {tanh_act(z):8.4f}')

# Softmax con un ejemplo
logits = np.array([2.0, 1.0, 0.5, -0.5])
probs  = softmax(logits)
print(f'\nSoftmax([2.0, 1.0, 0.5, -0.5]):')
```

```
print(f'  {probs.round(4)}')
print(f'  Suman: {probs.sum():.6f}')
```

```
  > Salida del programa:
=== FUNCIONES DE ACTIVACIÓN ===
  Entrada    Sigmoid    ReLU      Tanh
----------------------------------------
     -3.0     0.0474    0.00   -0.9951
     -1.0     0.2689    0.00   -0.7616
     -0.5     0.3775    0.00   -0.4621
      0.0     0.5000    0.00    0.0000
      0.5     0.6225    0.50    0.4621
      1.0     0.7311    1.00    0.7616
      3.0     0.9526    3.00    0.9951

Softmax([2.0, 1.0, 0.5, -0.5]):
  [0.5977 0.2199 0.1334 0.0491]
  Suman: 1.000000
```

> 💡 Observa cómo sigmoid aplana los extremos hacia 0 y 1 (saturación), mientras que ReLU simplemente 'corta' los negativos y deja pasar los positivos sin cambio. Esa diferencia tiene grandes consecuencias en el entrenamiento de redes profundas.

3.2 La neurona como clase de Python

En el código, la suma ponderada $z = x_1 \cdot w_1 + \ldots + x_n \cdot w_n + b$ se escribe en una sola línea: z = x @ w + b (o np.dot(x, w) + b), el producto punto de la sección 1.5. Si en lugar de un ejemplo pasamos una matriz X con un ejemplo por fila, de forma (N, n), X @ w calcula de una vez la z de los N ejemplos. Por eso las redes se programan con matrices.

PROGRAMA 1B — La neurona artificial completa

📄 **programa_1b_neurona.py**

```
import numpy as np

# ============================================================
# CLASE NEURONA ARTIFICIAL
# Implementa el modelo: salida = sigmoid(X . W + b)
# ============================================================

class Neurona:
    """
    Una neurona artificial con función de activación sigmoid.

    Parámetros aprendibles:
      - pesos: un valor por cada entrada
      - sesgo: un valor adicional (permite desplazar la activación)
    """

    def __init__(self, n_entradas, semilla=42):
        np.random.seed(semilla)
        # Inicializamos con valores pequeños aleatorios
        # Si fueran todos cero, la red nunca aprendería diferencias
        self.pesos = np.random.randn(n_entradas) * 0.3
        self.sesgo = 0.0
        self.n_entradas = n_entradas

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-z))

    def calcular(self, entradas):
        """
        Realiza el cálculo forward (hacia adelante).
        Retorna la salida y la pre-activación z.
        """
        entradas = np.array(entradas, dtype=float)
        if entradas.shape != (self.n_entradas,):
            raise ValueError(f'Se esperaban {self.n_entradas} entradas')
```

```
            # Paso 1: suma ponderada
            z = np.dot(entradas, self.pesos) + self.sesgo
            # Paso 2: función de activación
            salida = self._sigmoid(z)
            return salida, z

        def __repr__(self):
            return (f'Neurona(n={self.n_entradas}, '
                    f'pesos={self.pesos.round(4)}, '
                    f'sesgo={self.sesgo:.4f})')


    # -- Demostración ----------------------------------------
    print('=== NUESTRA PRIMERA NEURONA ARTIFICIAL ===\n')

    # Creamos una neurona que acepta 3 entradas
    neurona = Neurona(n_entradas=3)
    print(f'Neurona creada:\n  {neurona}\n')

    # La probamos con distintos conjuntos de entradas
    casos = [
        ([0.0, 0.0, 0.0], 'Entradas todas a cero'),
        ([1.0, 1.0, 1.0], 'Entradas todas a uno'),
        ([0.8, 0.2, 0.6], 'Entradas mixtas'),
        ([-1.0, 2.0, -0.5], 'Entradas con negativos'),
    ]

    print(f'{"Descripción":<30} {"z (pre-act.)":>14} {"Salida":>10}')
    print('-' * 58)
    for entradas, desc in casos:
        salida, z = neurona.calcular(entradas)
        print(f'{desc:<30} {z:14.4f} {salida:10.4f}')

    # Interpretación de la salida
    print('\nInterpretación:')
    print('  Salida ≈ 0.5 -> la neurona no tiene preferencia')
    print('  Salida -> 1.0 -> la neurona se activa fuertemente')
    print('  Salida -> 0.0 -> la neurona se inhibe fuertemente')
```

> Salida del programa:

```
=== NUESTRA PRIMERA NEURONA ARTIFICIAL ===

Neurona creada:
  Neurona(n=3, pesos=[ 0.149  -0.0415  0.1943], sesgo=0.0000)

Descripción                    z (pre-act.)     Salida
----------------------------------------------------------
Entradas todas a cero                0.0000     0.5000
Entradas todas a uno                 0.3018     0.5749
Entradas mixtas                      0.2275     0.5566
Entradas con negativos              -0.3291     0.4185

Interpretación:
  Salida ≈ 0.5 -> la neurona no tiene preferencia
  Salida -> 1.0 -> la neurona se activa fuertemente
  Salida -> 0.0 -> la neurona se inhibe fuertemente
```

3.3 Anatomía del código

Analicemos las decisiones de diseño más importantes del código anterior:

La inicialización aleatoria de pesos es crucial. Para una neurona sola, empezar con los pesos a cero no sería grave: el gradiente de cada peso depende de su entrada, y los pesos se separarían en cuanto empezara a aprender. El problema aparece en una red: si todas las neuronas de una capa empiezan con los mismos pesos, calculan lo mismo, reciben el mismo gradiente y siguen siendo idénticas para siempre (el problema de la simetría). Empezar al azar rompe esa simetría. Los valores pequeños aleatorios (aquí multiplicamos por 0.3) permiten que el aprendizaje empiece en distintas 'posiciones' del espacio de parámetros.

El sesgo (bias) es igual de importante que los pesos. Permite que la función de activación se desplace hacia la izquierda o la derecha, dando mucha más flexibilidad al modelo. Sin

sesgo, una neurona con activación sigmoid siempre produciría 0.5 cuando todas las entradas son cero, independientemente de lo que haya aprendido.

La función calcular retorna tanto la salida como z (la pre-activación). Necesitaremos z en el capítulo siguiente para calcular los gradientes durante el entrenamiento.

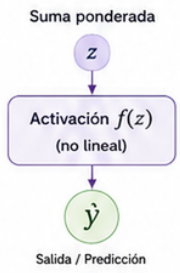
> ℹ️ Una neurona con pesos aleatorios todavía no resuelve ninguna tarea concreta. Es como una persona recién nacida: tiene el hardware (neuronas) pero no sabe nada. El aprendizaje viene en el siguiente capítulo, donde veremos cómo ajustar los pesos usando el error.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - La clase Neurona encapsula pesos, sesgo y la lógica de forward en unas pocas líneas de NumPy.
> - La inicialización aleatoria de pesos es imprescindible: sin ella, las neuronas de una misma capa serían copias idénticas y aprenderían lo mismo.
> - Una neurona con pesos aleatorios no sabe nada todavía — el aprendizaje llega con el gradiente descendente.

DIAGRAMA 3.1 — COMPARATIVA DE FUNCIONES DE ACTIVACIÓN

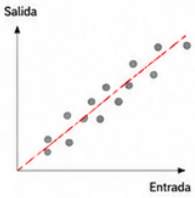
Transforman la suma ponderada en una salida no lineal que permite aprender patrones complejos.

A. ¿QUÉ PROBLEMA RESUELVEN?



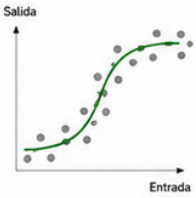
SIN ACTIVACIÓN

Solo relaciones lineales



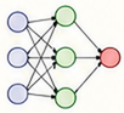
CON ACTIVACIÓN

Relaciones no lineales



¿POR QUÉ SON IMPORTANTES?

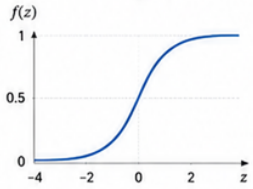
- ✓ Introducen no linealidad.
- ✓ Permiten modelar fenómenos complejos.
- ✓ Afectan al flujo del gradiente y al entrenamiento.
- ✓ Su elección impacta directamente en el rendimiento del modelo.



B. FORMAS DE LAS FUNCIONES

SIGMOID (σ)

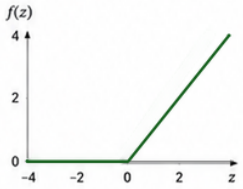
$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Rango: $(0, 1)$
Salida siempre entre 0 y 1

RELU

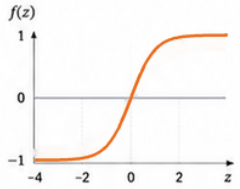
$$f(z) = \max(0, z)$$



Rango: $[0, +\infty)$
Salida 0 para $z \leq 0$, lineal para $z > 0$

TANH

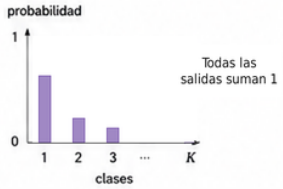
$$f(z) = \tanh(z)$$



Rango: $(-1, 1)$
Salida centrada en 0

SOFTMAX (salida)

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

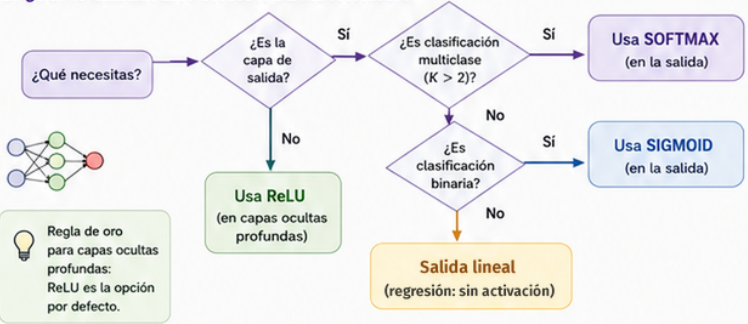


Rango: $(0, 1)$
Distribución de probabilidad sobre K clases

C. COMPARACIÓN PRÁCTICA

Función	Sigmoid (σ)	ReLU	Tanh	Softmax (salida)
Forma				
Rango	$(0, 1)$	$[0, +\infty)$	$(-1, 1)$	$(0, 1)$ $(\text{suma} = 1)$
Ventaja principal	Salida interpretable como probabilidad	Muy eficiente y simple de calcular	Centrada en 0, mejor para el gradiente	Ideal para clasificación multiclase
Problema principal	Saturación (gradientes muy pequeños en los extremos)	Neuronas muertas ($z \leq 0$ siempre)	Saturación (gradientes pequeños en $\|z\|$ grandes)	No se usa en capas ocultas (solo en salida)
Uso típico actual	Clasificación binaria, capas de salida	Capas ocultas en redes profundas	RNN, algunas redes antiguas o específicas	Capa de salida en clasificación multiclase (ej. LLMs)
Derivada (para el training)	$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$	$f'(z) = \begin{cases} 0 & \text{si } z \leq 0 \\ 1 & \text{si } z > 0 \end{cases}$	$f'(z) = 1 - \tanh^2(z)$	$\frac{\partial y_i}{\partial z_j} = y_i\,(\delta_{ij} - y_j)$
Notas	Puede perder gradiente en redes profundas	Estándar en la mayoría de redes modernas	Mejor que sigmoid para gradientes	Convierte logits en probabilidades

D. ¿CÓMO ELEGIR LA FUNCIÓN DE ACTIVACIÓN?



Regla de oro para capas ocultas profundas: ReLU es la opción por defecto.

E. EN LA PRÁCTICA MODERNA

Modelos actuales (CNNs, Transformers, LLMs) usan principalmente ReLU o variantes: Leaky ReLU, GELU, SiLU.

Sigmoid y Tanh son menos comunes, pero siguen útiles en ciertos escenarios (RNNs, probabilidades, etc.).

La elección correcta mejora la convergencia, estabilidad y capacidad de generalización del modelo.

Elegir bien la función de activación es tan importante como elegir el modelo.
Es el puente entre la matemática lineal y la inteligencia no lineal.

✏️ EJERCICIOS — Capítulo 3 — Programando la Neurona

Programar es la mejor forma de comprobar que se ha entendido algo. Escribe tu propia solución a cada ejercicio antes de mirar la propuesta y fíjate en qué se diferencian las dos versiones.

Ejercicio 3.1 Implementar Leaky ReLU

La variante Leaky ReLU corrige un problema de ReLU: cuando z < 0, en lugar de devolver 0, devuelve α · z (con α = 0.01). Esto evita que las neuronas queden 'muertas' permanentemente durante el entrenamiento. Impleméntala y compara su comportamiento con ReLU para valores entre −3 y +3.

> 💡 💡 Pista: Leaky ReLU(z) = z si z > 0, α · z si z ≤ 0. Cuando z = -3, ReLU devuelve 0, pero Leaky ReLU devuelve -0.03. Esa pequeña diferencia mantiene un gradiente no nulo.

✅ **Solución:**

📄 **ejercicio_3_1.py**

```
import numpy as np

def relu(z):                      return np.maximum(0, z)
def leaky_relu(z, alpha=0.01):    return np.where(z > 0, z, alpha * z)
def elu(z, alpha=1.0):            return np.where(z > 0, z, alpha*(np.exp(z)-1))

valores = np.array([-3, -1, -0.5, 0, 0.5, 1, 3], dtype=float)

print(f"{'z':>6}  {'ReLU':>8}  {'LeakyReLU':>12}  {'ELU':>8}  Gradiente Leaky")
print('-' * 56)
for z in valores:
    r  = relu(z)
    lr = leaky_relu(z)
    e  = elu(z)
    # Derivada de Leaky ReLU: 1 si z>0, 0.01 si z<=0
    grad_lr = 1.0 if z > 0 else 0.01
    print(f'{z:>6.1f}  {r:>8.3f}  {lr:>12.4f}  {e:>8.4f}  {grad_lr:>14.2f}')

print()
print('ReLU: gradiente = 0 para z < 0 (neurona muerta)')
print('Leaky: gradiente = 0.01 para z < 0 (siempre aprende algo)')
```

> ℹ️ ELU (Exponential Linear Unit) es otra variante que produce valores negativos suaves. En la práctica, Leaky ReLU y sus variantes se usan cuando se detecta el problema de 'neuronas muertas' durante el entrenamiento.

Ejercicio 3.2 Simular una puerta OR con parámetros a mano

Sin entrenar nada, encuentra unos valores de pesos (w1, w2) y sesgo (b) que hagan que la neurona se comporte como una puerta OR. Recuerda: OR devuelve 1 cuando al menos una entrada es 1. Verifica que tu neurona produce salidas > 0.5 para (0, 1), (1, 0), (1, 1) y < 0.5 para (0, 0).

> 💡 💡 Pista: OR es más fácil que AND: el umbral es más bajo (basta con que UNA entrada sea 1). Intenta con pesos grandes positivos (p.ej. 8.0) y un sesgo moderadamente negativo (p.ej. -4.0) para que la neurona solo se inhiba cuando AMBAS entradas sean 0.

✅ **Solución:**

📄 **ejercicio_3_2.py**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))

# Parámetros escogidos a mano para OR
# Lógica: queremos sigmoid(w+b) > 0.5 cuando solo hay 1 entrada activa
# sigmoid(z) > 0.5  <=>  z > 0
# Para OR: necesitamos  w1*1 + w2*0 + b > 0  y  w1*0 + w2*0 + b < 0
# Es decir:  w1 + b > 0  y  b < 0
# Con w1=w2=8, b=-4:  8-4=4>0 ✓  y  -4<0 ✓

w1, w2, b = 8.0, 8.0, -4.0

X = [[0,0],[0,1],[1,0],[1,1]]
y = [0,   1,   1,   1  ]

print('Puerta OR — parámetros ajustados manualmente')
print(f'w1={w1}, w2={w2}, sesgo={b}')
print()
print(f"{'Entradas':^12}  {'z':>8}  {'σ(z)':>10}  {'Esperado':>8}  OK")
print('-' * 50)
correcto = 0
for xi, yi in zip(X, y):
    z   = xi[0]*w1 + xi[1]*w2 + b
    sal = sigmoid(z)
    ok  = round(sal) == yi
    correcto += ok
    print(f'{str(xi):^12}  {z:>8.1f}  {sal:>10.4f}  {yi:>8}  {"✓" if ok else "✗"}')
print(f'\nPrecisión: {correcto}/4')
```

Ejercicio 3.3 Visualizar el espacio de decisión

Una neurona con sigmoid traza un 'hiperplano' en el espacio de entrada. Para dos entradas x1 y x2, usa los pesos que obtendrá la neurona AND entrenada del Programa 2, en el próximo capítulo, para calcular la salida en una cuadrícula de puntos y visualizarla en ASCII. ¿Dónde está la frontera de decisión (salida = 0.5)?

> 💡💡 Pista: La frontera de decisión ocurre donde z = 0, es decir, donde w1 · x1 + w2 · x2 + b = 0. Para la neurona AND entrenada (w1 ≈ w2 ≈ 7.28; b ≈ −11.01), esa recta pasa entre el punto (1, 1) y los demás.

✅ Solución:

📄 **ejercicio_3_3.py**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))

# Pesos de la neurona AND que entrena el Programa 2 (Capítulo 4)
w1, w2, b = 7.28, 7.28, -11.01

print('Mapa de activación de la neurona AND')
print('(█ > 0.7,  ▓ entre 0.4 y 0.7,  ░ entre 0.2 y 0.4,  . < 0.2)')
print()

# Cuadrícula de -0.4 a 1.4 en pasos de 0.2 (incluye el 0 y el 1)
x_vals = np.arange(-0.4, 1.5, 0.2)
y_vals = np.arange(1.4, -0.5, -0.2)   # invertido para que (0,0) esté abajo

# Filas: x2 de arriba abajo; columnas: x1 de izquierda a derecha
print('x2 ^')
for x2 in y_vals:
    fila = f'{x2:>4.1f} |'
    for x1 in x_vals:
        z   = w1*x1 + w2*x2 + b
        sal = sigmoid(z)
        if sal > 0.7:   fila += '█'
        elif sal > 0.4: fila += '▓'
        elif sal > 0.2: fila += '░'
```

```
            else:           fila += '.'
        print(fila)
    print('     +' + '-'*len(x_vals) + '-> x1')
    print('      ' + ''.join(f'{v:>1.0f}' if abs(v-round(v))<0.01 else ' '
                             for v in x_vals))
    print()
    print('La frontera (▓) separa el punto (1,1) de los demás.')
    print('Es una línea recta: la neurona es un clasificador lineal.')
```

ℹ️ Esta visualización revela por qué AND sí puede resolverse con una sola neurona (los puntos son linealmente separables) pero XOR no (ninguna línea recta los separa).

Capítulo 4: El Aprendizaje — Gradientes y Backpropagation (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar qué es una función de pérdida y por qué la necesitamos.
> ✓ Describir el gradiente como la 'brújula' que guía el ajuste de los pesos.
> ✓ Implementar gradiente descendente manual para entrenar una neurona.
> ✓ Entrenar con éxito una neurona para resolver la puerta lógica AND.

4.1 ¿Qué significa aprender?

Hasta ahora, nuestra neurona tiene pesos asignados aleatoriamente. Para que sea útil, necesitamos que sus pesos se ajusten automáticamente para hacer predicciones correctas. A ese proceso lo llamamos entrenamiento o aprendizaje.

La idea central es sorprendentemente simple: comparamos lo que predice la neurona con lo que debería predecir, calculamos el error, y ajustamos los pesos en la dirección que reduciría ese error. Si repetimos esto miles de veces con muchos ejemplos, los pesos convergen hacia valores que hacen buenas predicciones.

Para que esto funcione necesitamos tres ingredientes:

* Una función de pérdida (loss): mide numéricamente cuán equivocada es la predicción.
* Un gradiente: indica en qué dirección y cuánto cambiar cada peso para reducir la pérdida.
* Una regla de actualización: cómo modificar los pesos usando el gradiente.

4.2 La función de pérdida

La función de pérdida asigna un número a la distancia entre la predicción y el valor real. Queremos que ese número sea lo más pequeño posible.

Empezaremos con la más sencilla, el Error Cuadrático Medio (MSE). En clasificación lo habitual es la entropía cruzada, que conocerás al construir el Nano-LLM (Capítulo 13); para una sola neurona, el MSE basta y se entiende mejor:

📄 **Función de pérdida MSE**

```
# Error Cuadrático Medio (Mean Squared Error)
# prediccion: lo que dice nuestra neurona
# objetivo:   lo que debería decir

L = (prediccion - objetivo) ** 2

# Si la neurona predice 0.9 y el objetivo es 1.0:
L = (0.9 - 1.0)**2 = 0.01  # Pequeño error

# Si la neurona predice 0.1 y el objetivo es 1.0:
L = (0.1 - 1.0)**2 = 0.81  # Error grande

# Para N ejemplos, promediamos:
L_total = promedio de (prediccion_i - objetivo_i)^2
```

4.3 El gradiente — la brújula del aprendizaje

La derivada de la función de pérdida respecto a un parámetro nos dice: 'si aumentas este peso en una pequeña cantidad, la pérdida sube o baja, y cuánto'. Como la red tiene muchos parámetros, hay una derivada para cada uno (se llama derivada parcial, porque mueve ese parámetro y deja fijos todos los demás). El gradiente es el vector que las reúne todas.

¿Qué es una derivada? Si no la has estudiado todavía, basta con esta idea: la derivada de una función en un punto es su pendiente allí, es decir, cuánto cambia el resultado por cada

unidad que se mueve la entrada, cuando el movimiento es muy pequeño. Se puede calcular tanteando, con un empujoncito h: derivada ≈ (f(w + h) − f(w)) / h. Por ejemplo, para la pérdida L(w) = (w − 3)², que es mínima en w = 3, en el punto w = 1:

📄 **Ejemplo — la derivada como pendiente**

```
def L(w):
    return (w - 3) ** 2          # una pérdida sencilla: mínima en w = 3

w = 1.0
for h in [0.1, 0.01, 0.0001]:
    pendiente = (L(w + h) - L(w)) / h
    print(f'h = {h:<7} pendiente = {pendiente:.4f}')
print('Fórmula exacta, 2*(w - 3) =', 2 * (w - 3))
```

> Salida del programa:

```
h = 0.1     pendiente = -3.9000
h = 0.01    pendiente = -3.9900
h = 0.0001  pendiente = -3.9999
Fórmula exacta, 2*(w - 3) = -4.0
```

Cuanto más pequeño es h, más se acerca el tanteo a −4. Que la pendiente sea negativa significa que aumentar w un poco hace bajar la pérdida (unas 4 veces lo que se mueve w): es justo la información que necesita el gradiente descendente, que se mueve en contra del signo de la derivada. Las reglas de derivación (la derivada de w^2 es 2w; la de la sigmoide, p · (1 − p)) son atajos para obtener ese mismo número sin tantear.

La intuición geométrica es la de una bola rodando por una colina: el gradiente apunta cuesta arriba. Si queremos minimizar la pérdida, debemos movernos en la dirección opuesta al gradiente (cuesta abajo). A eso se le llama gradiente descendente. El Diagrama 4.1 ilustra esta idea paso a paso.

📄 **Gradiente descendente — regla de actualización**

```
# Regla de actualización para cada peso w:

w_nuevo = w_viejo - lr * gradiente

# Donde:
#   lr  = tasa de aprendizaje (learning rate)
#         valor típico: 0.001 a 0.1 (con una sola neurona sigmoide se puede más: el
Programa 2 usa 0.5)
#         demasiado grande: el modelo diverge (oscila)
#         demasiado pequeño: el aprendizaje es muy lento
#   gradiente = derivada de L respecto a w

# Para una neurona con sigmoid y pérdida MSE:
# gradiente_w = -2 * error * sigmoid'(z) * x_i
# gradiente_b = -2 * error * sigmoid'(z)

# Donde: error = objetivo - prediccion
#        sigmoid'(z) = sigmoid(z) * (1 - sigmoid(z))
```

4.4 Backpropagation — la regla de la cadena

Backpropagation (propagación hacia atrás) es el algoritmo que calcula eficientemente los gradientes de todos los pesos de una red. Aplica la regla de la cadena del cálculo diferencial: si tenemos una composición de funciones, la derivada total es el producto de las derivadas individuales.

Para una sola neurona el cálculo es sencillo. Para redes con muchas capas, backprop propaga el error desde la última capa hacia la primera, distribuyendo la 'culpa' del error a cada peso según su contribución.

Veámoslo con números. Tomemos la neurona del Ejercicio 2.1 (entradas [0.5, 0.8], pesos [0.3, −0.4], sesgo 0.2) y supongamos que la respuesta correcta es 1. La pérdida depende de w_1 a través de una cadena de tres pasos: $w_1 \to z \to p \to L$. La regla de la cadena dice que la derivada total es el producto de las derivadas de cada eslabón: $dL/dw_1 = dL/dp \cdot dp/dz \cdot dz/dw_1$, donde $dL/dp = -2 \cdot (y - p)$ (la derivada del MSE), $dp/dz = p \cdot (1 - p)$ (la de la sigmoide, Capítulo 2) y $dz/dw_1 = x_1$ (porque $z = x_1 \cdot w_1 + x_2 \cdot w_2 + b$).

📄 **Ejemplo — la regla de la cadena con números**

```
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))

x, w, b, y = np.array([0.5, 0.8]), np.array([0.3, -0.4]), 0.2, 1.0   # la neurona del
Ejercicio 2.1
z = x @ w + b                  # 1) suma ponderada
p = sigmoid(z)                 # 2) activación
L = (y - p) ** 2               # 3) pérdida
dL_dp = -2 * (y - p)           # derivada de la pérdida respecto a p
dp_dz = p * (1 - p)            # derivada de la sigmoide (Capítulo 2)
dL_dw = dL_dp * dp_dz * x      # regla de la cadena; dz/dw = x
dL_db = dL_dp * dp_dz          # dz/db = 1
print(f'z = {z:.4f}   p = {p:.4f}   L = {L:.4f}')
print(f'dL/dp = {dL_dp:.4f}   dp/dz = {dp_dz:.4f}')
print(f'dL/dw1 = {dL_dw[0]:.4f}   dL/dw2 = {dL_dw[1]:.4f}   dL/db = {dL_db:.4f}')
lr = 0.5
w, b = w - lr * dL_dw, b - lr * dL_db
p = sigmoid(x @ w + b)
print(f'Tras un paso con lr = {lr}: p = {p:.4f}   L = {(y - p) ** 2:.4f}')
```

> Salida del programa:

```
z = 0.0300   p = 0.5075   L = 0.2426
dL/dp = -0.9850   dp/dz = 0.2499
dL/dw1 = -0.1231   dL/dw2 = -0.1970   dL/db = -0.2462
Tras un paso con lr = 0.5: p = 0.5653   L = 0.1890
```

Los tres gradientes son negativos: subir w_1, w_2 o b acercaría p a 1. Un solo paso de gradiente descendente con lr = 0.5 lleva p de 0.5075 a 0.5653 y la pérdida de 0.2426 a 0.1890. El Programa 2 hace exactamente esto, miles de veces y con los cuatro ejemplos de la tabla AND; la única diferencia es que omite el factor 2 de dL/dp, que queda absorbido en la tasa de aprendizaje.

> 💡 No te preocupes si la matemática de backpropagation te parece compleja. Lo importante es entender la idea: medimos el error, calculamos cuánto contribuyó cada peso a ese error, y los ajustamos proporcionalmente. Los frameworks modernos como PyTorch lo hacen automáticamente.

4.5 Programa 2 — Una neurona que aprende

Vamos a entrenar una neurona para que aprenda la función lógica AND. La puerta AND devuelve 1 solo cuando ambas entradas son 1:

PROGRAMA 2 — Neurona que aprende (puerta AND)

📄 **programa_2_neurona_aprende.py**

```
import numpy as np

# ============================================================
# NEURONA QUE APRENDE MEDIANTE GRADIENTE DESCENDENTE
# Problema: función lógica AND
# ============================================================

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def derivada_sigmoid(z):
    s = sigmoid(z)
    return s * (1.0 - s)

# -- Datos de entrenamiento: tabla verdad del AND --------
X = np.array([[0, 0],   # 0 AND 0 = 0
              [0, 1],   # 0 AND 1 = 0
              [1, 0],   # 1 AND 0 = 0
              [1, 1]])  # 1 AND 1 = 1
y = np.array([0, 0, 0, 1], dtype=float)

# -- Inicialización de parámetros ------------------------
np.random.seed(0)
pesos = np.random.randn(2) * 0.3   # 2 pesos (uno por entrada)
sesgo = 0.0
```

```
lr    = 0.5                         # tasa de aprendizaje

print('=== ENTRENANDO NEURONA PARA APRENDER AND ===\n')
print(f'Pesos iniciales: {pesos.round(4)}  Sesgo: {sesgo:.4f}\n')

# -- Bucle de entrenamiento ----------------------------
for epoca in range(10_000):
    error_total = 0.0

    for xi, yi in zip(X, y):         # un ejemplo a la vez
        # === FORWARD PASS ===
        z = np.dot(xi, pesos) + sesgo
        pred = sigmoid(z)

        # === CÁLCULO DEL ERROR ===
        error = yi - pred            # cuánto nos hemos equivocado
        error_total += error**2

        # === BACKWARD PASS (gradientes) ===
        # Regla de la cadena aplicada a una neurona
        delta = error * derivada_sigmoid(z)

        # === ACTUALIZACIÓN DE PARÁMETROS ===
        pesos += lr * delta * xi    # actualizar cada peso
        sesgo += lr * delta         # actualizar sesgo

    # Mostrar progreso cada 1000 épocas
    if epoca % 1000 == 0:
        perdida_media = error_total / len(X)
        print(f'Época {epoca:5d} | Pérdida: {perdida_media:.6f}')

# -- Resultados ----------------------------------------
print(f'\nPesos finales: {pesos.round(4)}  Sesgo: {sesgo:.4f}')
print('\n=== PRUEBA FINAL ===')
print(f'{"Entradas":^20} {"Predicción":>12} {"Esperado":>10} {"Correcto":>10}')
print('-' * 55)
for xi, yi in zip(X, y):
    z    = np.dot(xi, pesos) + sesgo
    pred = sigmoid(z)
    ok   = '✓' if round(pred) == yi else '✗'
    print(f'{str(xi.tolist()):^20} {pred:12.4f} {int(yi):10d} {ok:>10}')
```

> Salida del programa:

```
=== ENTRENANDO NEURONA PARA APRENDER AND ===

Pesos iniciales: [0.5292 0.12  ]  Sesgo: 0.0000

Época     0 | Pérdida: 0.262964
Época  1000 | Pérdida: 0.005612
Época  2000 | Pérdida: 0.002616
Época  3000 | Pérdida: 0.001684
Época  4000 | Pérdida: 0.001236
Época  5000 | Pérdida: 0.000974
Época  6000 | Pérdida: 0.000802
Época  7000 | Pérdida: 0.000681
Época  8000 | Pérdida: 0.000592
Época  9000 | Pérdida: 0.000523

Pesos finales: [7.2846 7.2842]  Sesgo: -11.0135

=== PRUEBA FINAL ===
      Entradas       Predicción    Esperado    Correcto
-------------------------------------------------------
      [0, 0]             0.0000          0           ✓
      [0, 1]             0.0234          0           ✓
      [1, 0]             0.0235          0           ✓
      [1, 1]             0.9722          1           ✓
```

La neurona aprendió la función AND con éxito. Observa cómo la pérdida decrece monótonamente con las épocas —señal de que el gradiente descendente funciona correctamente— y cómo los pesos finales son significativamente mayores que los iniciales, con un sesgo negativo grande que actúa como umbral.

> ℹ️ Los pesos iguales para las dos entradas tienen sentido: en AND, ambas entradas tienen el mismo papel. El sesgo negativo grande (−11.01) asegura que la neurona solo se active cuando ambas entradas estén activas simultáneamente, compensando la suma de los dos pesos positivos.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - El gradiente indica la dirección de mayor crecimiento del error; nos movemos en la dirección opuesta.
> - Backpropagation aplica la regla de la cadena para calcular gradientes de forma eficiente capa por capa.
> - Con gradiente descendente manual, una neurona puede aprender una función lógica como AND desde cero.

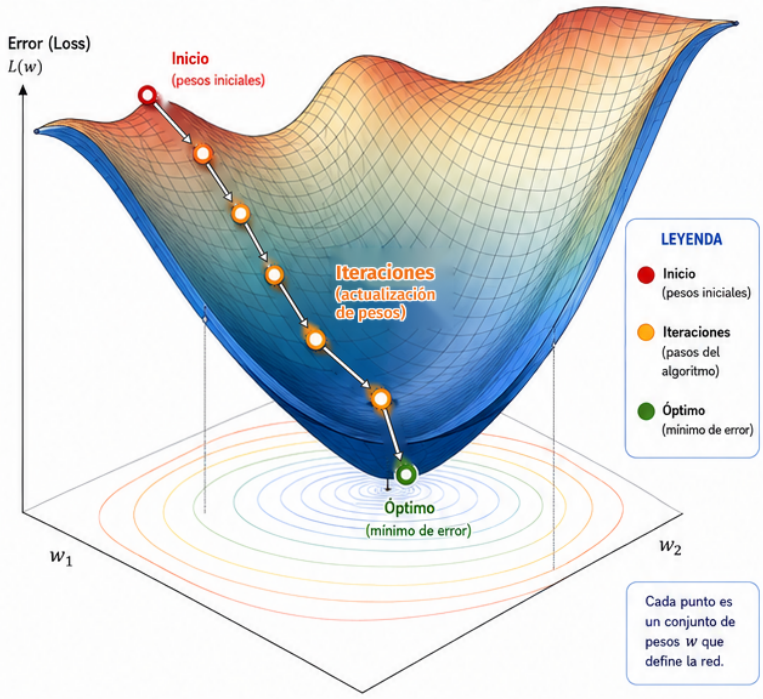
DIAGRAMA 4.1 — GRADIENTE DESCENDENTE

¿CÓMO APRENDE UNA RED NEURONAL?

Descenso por gradiente hacia el mínimo de error.

1. EL PAISAJE DEL ERROR (FUNCIÓN DE PÉRDIDA)

La red neuronal busca los valores de los pesos que minimizan el error (pérdida).



2. ¿QUÉ OCURRE EN CADA PASO?

1 REALIZAR PREDICCIÓN
La red genera una salida $\hat{y}$ con los pesos actuales w.

2 CALCULAR ERROR (PÉRDIDA)
Se mide cuánto se equivoca la predicción respecto al valor real y.

3 CALCULAR GRADIENTE
Se calcula la dirección de máxima subida del error $\nabla L(w)$.

4 ACTUALIZAR PESOS
Se mueven los pesos en la dirección opuesta (hacia abajo) para reducir el error.

5 REPETIR
Se itera el proceso hasta converger a un mínimo (óptimo local o global).

Objetivo: encontrar w^* tal que $L(w^*)$ sea mínima.

3. ACTUALIZACIÓN DE PESOS

Regla de actualización (gradiente descendente)

$$w^{(t+1)} = w^{(t)} - \eta \, \nabla L\left(w^{(t)}\right)$$

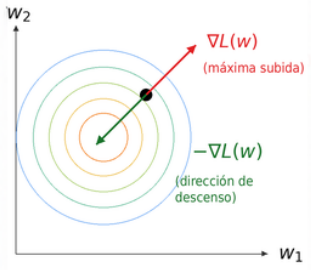
Donde:

- $w^{(t)}$ — Pesos en la iteración t
- η — Learning rate (tasa de aprendizaje) Controla el tamaño del paso.
- $\nabla L\left(w^{(t)}\right)$ — Gradiente de la función de pérdida respecto a los pesos.

Interpretación:
Caminamos en la dirección opuesta al gradiente para reducir lo más posible el error.

4. ¿QUÉ ES EL GRADIENTE?

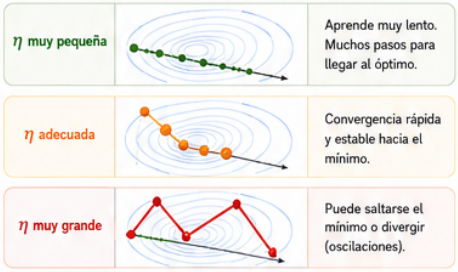
El gradiente apunta hacia donde el error aumenta más rápidamente.



Nos movemos en $-\nabla L(w)$ para bajar la colina.

5. EL PAPEL DEL LEARNING RATE (η)

El tamaño del paso afecta la velocidad y estabilidad del aprendizaje.



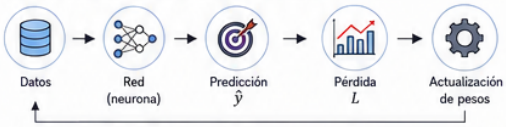
Elegir un buen η es clave para un entrenamiento eficiente.

6. ANALOGÍA: EL MONTAÑISTA EN LA NIEBLA

No puede ver todo el valle, solo el terreno inmediato.

El gradiente descendente no busca adivinar la solución.
La encuentra mediante miles o millones de pequeños ajustes que reducen progresivamente el error del modelo.

7. PROCESO COMPLETO DE APRENDIZAJE



Evolución del error durante el entrenamiento

Convergencia: el error se estabiliza cerca de un mínimo.

IDEA CLAVE: El gradiente descendente es el motor de aprendizaje de las redes neuronales. Transforma el error en dirección y esa dirección en mejores predicciones.

DIAGRAMA 4.2 — BACKPROPAGATION

¿CÓMO ENCUENTRA LA RED CÓMO AJUSTAR CADA PESO?

El algoritmo que propaga el error hacia atrás para calcular los gradientes.

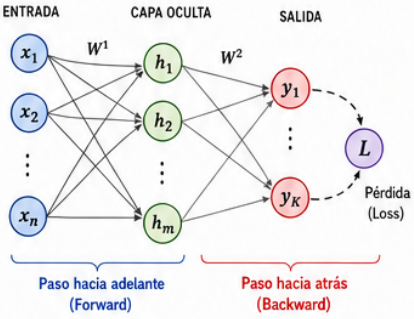
1. IDEA PRINCIPAL

Backpropagation calcula cuánto contribuye cada peso al error final para poder actualizarlo.
El error se propaga desde la salida hacia las capas anteriores, usando la regla de la cadena.

INTUICIÓN

Si empujar aquí ↑ aumenta mucho el error, entonces el gradiente dirá: "mueve menos en esa dirección".

La información viaja hacia atrás para decidir a cada peso qué hacer.

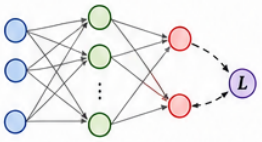


- **Entrada** — Datos del problema
- **Neurona oculta** — Detecta características intermedias
- **Salida** — Predicción de la red
- **Pérdida (L)** — Mide qué tan mala fue la predicción

2. ¿CÓMO FLUYE LA INFORMACIÓN?

FORWARD (hacia adelante)

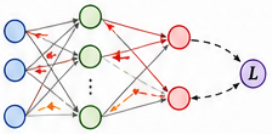
1. Las entradas x pasan por la red usando los pesos $(W^1, W^2, \ldots)$.
2. Se obtiene la predicción $\hat{y}$.
3. Se calcula la pérdida L comparando $\hat{y}$ con el valor real y.



Dirección del cálculo

BACKWARD (hacia atrás)

4. Se calcula el gradiente de la pérdida respecto a la salida: $\partial L/\partial \hat{y}$.
5. Ese error se propaga hacia atrás usando la regla de la cadena.
6. Se obtienen los gradientes respecto a cada peso $(\partial L/\partial W^1, \partial L/\partial W^2, \ldots)$.
7. Con esos gradientes, se actualizan los pesos.



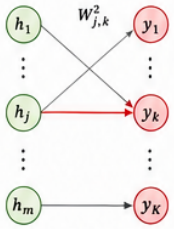
Dirección de la propagación del error

3. CÁLCULO DE GRADIENTES: REGLA DE LA CADENA

CAPA DE SALIDA

Para cada neurona de salida k:

1. Error en la salida k:
$$\delta_k = \frac{\partial L}{\partial z_k}$$
2. Gradiente de los pesos que vienen de la neurona oculta j:
$$\frac{\partial L}{\partial W^2_{j,k}} = \delta_k \cdot h_j$$
3. Gradiente del sesgo de la neurona k:
$$\frac{\partial L}{\partial b^2_k} = \delta_k$$



CAPA OCULTA (capa l)

Para cada neurona oculta j en la capa l:

1. Error propagado a la neurona j:
$$\delta^{(l)}_j = f'(z^{(l)}_j) \sum_k W^{(l+1)}_{j,k}\, \delta^{(l+1)}_k$$
2. Gradiente de los pesos que vienen de la capa anterior i:
$$\frac{\partial L}{\partial W^{(l)}_{i,j}} = \delta^{(l)}_j \cdot a^{(l-1)}_i$$
3. Gradiente del sesgo:
$$\frac{\partial L}{\partial b^{(l)}_j} = \delta^{(l)}_j$$

NOTACIÓN

- z: entrada neta a una neurona (antes de la activación)
- $a = f(z)$: activación (salida de la neurona)
- L: pérdida total (loss)
- δ: término de error de una neurona
- $W^{(l)}$: matriz de pesos de la capa l
- $b^{(l)}$: vector de sesgos de la capa l
- $f'(z)$: derivada de la función de activación

IDEA CLAVE

Cada δ mide "cuánto le importa" a esa neurona reducir el error. Backpropagation usa la regla de la cadena para repartir ese error hacia atrás, capa por capa.

4. ACTUALIZACIÓN DE PARÁMETROS

Con los gradientes calculados, actualizamos cada peso y sesgo mediante descenso por gradiente.

Regla de actualización

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} \qquad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

η: learning rate (tasa de aprendizaje)
Controla el tamaño del paso.

5. EJEMPLO NUMÉRICO SIMPLE (1–2–1)

Una entrada, dos neuronas ocultas y una salida.

Datos

$x = 1.0$, y (real) $= 0.8$

Pesos iniciales:

$$W^1 = \begin{bmatrix} 0.2 \\ -0.4 \end{bmatrix} \quad W^2 = \begin{bmatrix} 0.7 & -0.3 \end{bmatrix} \quad b^1 = \begin{bmatrix} 0.0 & 0.0 \end{bmatrix}^T \quad b^2 = \begin{bmatrix} 0.1 \end{bmatrix}$$

Forward

$$z^{(1)} = W^1 x + b^1 = \begin{bmatrix} 0.2 \\ -0.4 \end{bmatrix}$$
$$a^{(1)} = f(z^{(1)}) = \begin{bmatrix} 0.5498 \\ 0.4013 \end{bmatrix}$$
$$z^{(2)} = W^2 a^{(1)} + b^2 = 0.3645$$
$$\hat{y} = f(z^{(2)}) = 0.5901$$
$$L = \frac{1}{2}(\hat{y} - y)^2 = 0.0220$$

Backward (resumen)

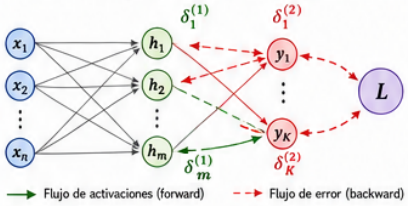
$$\delta^{(2)} = (\hat{y} - y) f'(z^{(2)}) = -0.0508$$
$$\delta^{(1)} = (W^2)^T \delta^{(2)} \odot f'(z^{(1)}) = \begin{bmatrix} -0.0088 \\ 0.0037 \end{bmatrix}$$

Gradientes de pesos y sesgos (se calculan con las fórmulas anteriores)

Con estos gradientes, actualizamos W^1, W^2, b^1, b^2 y repetimos.

6. VISUALIZACIÓN DE LA PROPAGACIÓN DEL ERROR

El error (δ) se origina en la salida y se distribuye hacia atrás.



¿QUÉ INDICA δ?

Señala en qué dirección cambiar la neurona (o sus entradas) para reducir el error.

Es la "culpa" asignada a cada componente.

7. BACKPROPAGATION EN RESUMEN (ALGORITMO)

1. Inicializar pesos y sesgos aleatoriamente.
2. Forward: calcular la salida de la red y la pérdida L.
3. Calcular δ en la capa de salida.
4. Propagar δ hacia atrás, capa por capa (regla de la cadena).
5. Calcular gradientes $\partial L/\partial W^{(l)}$ y $\partial L/\partial b^{(l)}$.
6. Actualizar pesos y sesgos (descenso por gradiente).
7. Repetir para muchas muestras (épocas) hasta converger.

¿POR QUÉ FUNCIONA?

Porque evalúa la influencia de cada peso en el error final a través de todas las rutas posibles en la red.

MENSAJE CLAVE: Backpropagation no prueba millones de combinaciones. Usa cálculo eficiente (regla de la cadena) para saber exactamente cómo cambiar cada peso y reducir el error.

EN UNA FRASE: Backpropagation hace que el error viaje hacia atrás, para que el aprendizaje avance hacia adelante.

✏️ EJERCICIOS — Capítulo 4 — El Aprendizaje

Haz estos ejercicios con lápiz y papel antes de ejecutar nada: calcular a mano un par de gradientes es la mejor manera de entender la retropropagación. Después, contrasta con la solución.

Ejercicio 4.1 Entrenar AND, OR y NAND con la misma neurona

Reutiliza el código del Programa 2 para entrenar una neurona en tres funciones lógicas distintas: AND, OR y NAND (NOT AND: inverso de AND). ¿Qué diferencias observas en los pesos finales? ¿Tiene sentido intuitivo que NAND tenga pesos negativos?

> 💡 💡 Pista: NAND es el inverso de AND: devuelve 0 solo cuando ambas entradas son 1. Sus pesos serán negativos (el efecto de las entradas está 'invertido') y su sesgo, positivo.

✅ Solución:

📄 ejercicio_4_1.py

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))
def d_sig(z): s=sigmoid(z); return s*(1-s)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)

tablas = {
    'AND':  np.array([0, 0, 0, 1], dtype=float),
    'OR':   np.array([0, 1, 1, 1], dtype=float),
    'NAND': np.array([1, 1, 1, 0], dtype=float),
}

for nombre, y in tablas.items():
    np.random.seed(1)
    w = np.random.randn(2) * 0.3
    b = 0.0
    for _ in range(8000):
        for xi, yi in zip(X, y):
            z = np.dot(xi, w) + b
            d = (yi - sigmoid(z)) * d_sig(z)
            w += 0.5 * d * xi
            b += 0.5 * d
    print(f'{nombre}: w=[{w[0]:.2f}, {w[1]:.2f}]  b={b:.2f}', end='   ')
    pred = [round(float(sigmoid(np.dot(xi,w)+b))) for xi in X]
    print(f'preds={pred}  {"✓" if pred == list(y.astype(int)) else "✗"}')

print()
print('Observación: NAND tiene pesos negativos y sesgo positivo.')
print('Esto tiene sentido: NAND activa la salida por defecto')
print('y la inhibe cuando recibe entradas positivas fuertes.')
```

> ℹ️ La simetría es hermosa: AND y NAND tienen pesos de signo opuesto. Esto refleja que NAND = NOT AND. Si invirtiéramos el signo de los pesos de AND y ajustáramos el sesgo, obtendríamos NAND.

Ejercicio 4.2 Efecto de la tasa de aprendizaje

Entrena la neurona en la puerta AND con seis tasas de aprendizaje distintas: 0.001, 0.01, 0.1, 0.5, 1.0 y 5.0. Para cada una, registra cuántas épocas tarda en converger (pérdida < 0.005) o declara que no converge. ¿Qué valor de lr es el más eficiente?

> 💡 💡 Pista: Una lr muy pequeña converge pero lentamente. En redes grandes, una lr demasiado alta hace oscilar o divergir el entrenamiento. Aquí verás una

> sorpresa: la más alta, 5.0, es la más rápida (93 épocas). La sigmoide limita cada paso, porque su derivada nunca pasa de 0.25, y el problema es tan sencillo que no hay nada que se desestabilice.

✅ **Solución:**

📄 **ejercicio_4_2.py**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))
def d_sig(z): s=sigmoid(z); return s*(1-s)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0, 0, 0, 1], dtype=float)

def perdida(w, b):
    return sum((y[i]-sigmoid(np.dot(X[i],w)+b))**2 for i in range(4))/4

print(f"{'lr':>8}  {'Épocas hasta conv.':>20}  {'Pérdida final':>15}  Estado")
print('-' * 60)

for lr in [0.001, 0.01, 0.1, 0.5, 1.0, 5.0]:
    np.random.seed(42)
    w = np.random.randn(2) * 0.3; b = 0.0
    ep_conv = None
    for ep in range(15000):
        for xi, yi in zip(X, y):
            z = np.dot(xi, w) + b
            d = (yi - sigmoid(z)) * d_sig(z)
            w += lr * d * xi; b += lr * d
        if perdida(w, b) < 0.005 and ep_conv is None:
            ep_conv = ep + 1
    L = perdida(w, b)
    ep_str  = str(ep_conv) if ep_conv else '>15000'
    estado  = 'converge' if L < 0.01 else 'NO converge'
    print(f'{lr:>8.3f}  {ep_str:>20}  {L:>15.6f}  {estado}')
```

Ejercicio 4.3 Curva de aprendizaje y tasa de mejora

Entrena la neurona en AND durante 5000 épocas y guarda la pérdida cada 100 épocas. Calcula y muestra la 'tasa de mejora' entre cada intervalo: ¿la mejora es constante o se ralentiza con el tiempo? ¿Por qué?

> 💡 💡 Pista: La mejora suele ser rápida al principio (cuando hay mucho error que corregir) y progresivamente más lenta a medida que la pérdida se acerca a cero. Este fenómeno se llama 'rendimientos decrecientes' y es universal en el entrenamiento de modelos.

✅ **Solución:**

📄 **ejercicio_4_3.py**

```
import numpy as np

def sigmoid(z): return 1/(1+np.exp(-z))
def d_sig(z): s=sigmoid(z); return s*(1-s)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0, 0, 0, 1], dtype=float)

np.random.seed(42)
w = np.random.randn(2)*0.3; b = 0.0
curva = []

for ep in range(5000):
    L = sum((y[i]-sigmoid(np.dot(X[i],w)+b))**2 for i in range(4))/4
    if ep % 100 == 0: curva.append((ep, L))
    for xi, yi in zip(X, y):
        z = np.dot(xi, w)+b; d = (yi-sigmoid(z))*d_sig(z)
        w += 0.5*d*xi; b += 0.5*d
```

```
print('Curva de aprendizaje AND (lr=0.5)\n')
print(f"{'Época':>8}  {'Pérdida':>12}  {'Mejora abs.':>13}  {'Mejora rel.':>13}")
print('-' * 52)
for i, (ep, L) in enumerate(curva[:15]):
    if i == 0:
        mejora_abs = mejora_rel = 0
    else:
        mejora_abs = curva[i-1][1] - L
        mejora_rel = mejora_abs / curva[i-1][1] * 100
    print(f'{ep:>8}  {L:>12.6f}  {mejora_abs:>13.6f}  {mejora_rel:>12.2f}%')

print()
print('Observación: la mejora absoluta decrece con el tiempo.')
print('Las primeras 100 épocas eliminan la mayor parte del error.')
print('El resto del entrenamiento ajusta finamente los pesos.')
```

Capítulo 5: Los Optimizadores — Cómo Aprende Realmente la Red (***)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar por qué el SGD básico falla en paisajes de pérdida difíciles.
> ✓ Implementar desde cero Momentum, RMSprop, Adam y AdamW.
> ✓ Justificar por qué AdamW es el estándar para entrenar Transformers.
> ✓ Diseñar una estrategia de Learning Rate Scheduling con warmup y coseno.

5.1 El gradiente es una brújula, no un GPS

En el Capítulo 4 aprendimos que el gradiente descendente actualiza los pesos moviéndose en la dirección opuesta al gradiente: w := w - lr × ∂L/∂w. La idea es correcta, pero la implementación básica tiene problemas serios que se hacen evidentes en cuanto los modelos tienen más de unas pocas capas.

El gradiente nos dice la dirección cuesta abajo en el punto exacto donde estamos. Es una brújula: nos indica hacia dónde ir. Pero no nos dice cuánto avanzar, ni si el camino se va a torcer, ni si hay un atajo más eficiente. Los optimizadores son los algoritmos que convierten esa información local en una estrategia de movimiento inteligente.

Para entender por qué necesitamos algo más que SGD básico, necesitamos visualizar el tipo de paisajes de pérdida que aparecen en la práctica.

5.2 SGD — El punto de partida

El SGD (Stochastic Gradient Descent, Gradiente Descendente Estocástico) es el algoritmo base, el que hemos usado, en su forma más sencilla, en todos los programas anteriores. 'Estocástico' porque en lugar de calcular el gradiente sobre todos los datos (lo que sería computacionalmente prohibitivo), lo calculamos sobre un lote (batch) aleatorio pequeño en cada paso. Una pasada completa por todos los datos de entrenamiento se llama época. Los programas de los Capítulos 2 a 4 tienen solo cuatro ejemplos. El Programa 2 actualiza los pesos tras cada ejemplo (lotes de tamaño 1, recorridos siempre en el mismo orden); el MLP del XOR usa los cuatro en cada paso: es el descenso por gradiente «por lotes completos», sin la parte estocástica.

📄 **SGD — Regla de actualización**

```
# En cada paso t:
grad_t = calcular_gradiente(batch_aleatorio, pesos)
pesos  = pesos - lr * grad_t

# Parametros:
#   lr  = tasa de aprendizaje (el unico hiperparametro de SGD)

# Problemas del SGD puro:
#  1. Zig-zagging en valles estrechos: el gradiente apunta a las
#     paredes del valle, no al fondo. Se avanza muy despacio.
#  2. Sensibilidad extrema al lr: muy grande -> diverge,
#     muy pequeno -> convergencia desesperantemente lenta.
#  3. Puntos de silla: el gradiente es cero pero no es un minimo.
#     Cerca de ellos SGD avanza muy despacio; el ruido
#     del mini-lote le ayuda a escapar.
```

A pesar de sus limitaciones, SGD sigue siendo relevante. Para algunos problemas de visión por computador entrenados durante muchas épocas con un buen learning rate scheduling, SGD+Momentum puede superar a Adam. Pero para la mayoría de los problemas modernos, especialmente los LLMs, necesitamos algo más sofisticado.

5.3 SGD con Momentum — La bola que rueda

Imagina que en lugar de colocar una piedrecita sobre el paisaje de pérdida y moverla un pequeño paso, colocas una bola de bolos. La bola acumula velocidad mientras desciende. En un valle estrecho, los componentes laterales del gradiente (que causan el zigzagueo) se cancelan entre sí porque alternan de dirección, mientras que el componente que apunta hacia el mínimo se acumula consistentemente.

El Momentum implementa exactamente esta intuición: mantiene un vector de velocidad v que es una suma ponderada de los gradientes anteriores, en la que los más antiguos pesan cada vez menos. Los gradientes que fluctúan se anulan; los que apuntan consistentemente en una dirección se acumulan.

📄 **SGD con Momentum — Regla de actualización**

```
# Inicializacion:
v = ceros (misma forma que los pesos)

# En cada paso t:
grad_t = calcular_gradiente(batch_aleatorio, pesos)

v     = beta * v + grad_t          # acumular velocidad
pesos = pesos - lr * v             # mover en direccion de la velocidad

# Parametros:
#   lr   = tasa de aprendizaje (tipicamente 0.01 a 0.1)
#   beta = factor de inercia (tipicamente 0.9 = recordamos 90% del pasado)

# Interpretacion de beta=0.9:
#   El gradiente actual pesa 1.0
#   El gradiente del paso anterior pesa 0.9
#   El de dos pasos atras: 0.9^2 = 0.81
#   El de diez pasos atras: 0.9^10 = 0.35
#   -> memoria exponencialmente decreciente del pasado
```

PROGRAMA O-1 — SGD vs SGD+Momentum en un paisaje difícil

📄 **programa_01_momentum.py**

```
import numpy as np

# ============================================================
# FUNCION DE ROSENBROCK: el test clasico para optimizadores.
# Es un valle largo y curvo con un unico minimo en (1, 1).
# f(x,y) = (1-x)^2 + 100*(y-x^2)^2
# El gradiente es muy diferente en las dos direcciones,
# lo que hace que SGD basico zigzaguee terriblemente.
# ============================================================

def f(p):      return (1-p[0])**2 + 100*(p[1]-p[0]**2)**2
def grad_f(p):
    dx = -2*(1-p[0]) - 400*p[0]*(p[1]-p[0]**2)
    dy = 200*(p[1]-p[0]**2)
    return np.array([dx, dy])

P0     = np.array([-0.5, 0.8])   # punto de partida
N      = 3000                     # pasos
LR     = 0.001
BETA   = 0.9

# --- SGD puro ---
p_sgd  = P0.copy()
hist_sgd = [float(f(p_sgd))]
for _ in range(N):
    p_sgd -= LR * grad_f(p_sgd)
    hist_sgd.append(float(f(p_sgd)))

# --- SGD + Momentum ---
p_mom  = P0.copy()
v      = np.zeros(2)
hist_mom = [float(f(p_mom))]
for _ in range(N):
    g  = grad_f(p_mom)
```

```
        v  = BETA * v + g
        p_mom -= LR * v
        hist_mom.append(float(f(p_mom)))

    # ============================================================
    # MOSTRAR CONVERGENCIA
    # ============================================================
    print('=== SGD vs SGD+MOMENTUM — Funcion de Rosenbrock ===')
    print(f'Minimo en (1.0, 1.0), f=0.  Partimos de {P0.tolist()}\n')

    print(f"{'Paso':>6}  {'SGD':>12}  {'SGD+Momentum':>14}  Pérdida del Momentum")
    print('-' * 48)
    for paso in [0, 50, 200, 500, 1000, 2000, 3000]:
        sgd_v = hist_sgd[paso]
        mom_v = hist_mom[paso]
        if sgd_v > 0 and mom_v > 0:
            ventaja = f'{sgd_v/mom_v:.3g} veces menor' if mom_v < sgd_v else 'igual'
        else:
            ventaja = 'convergido'
        print(f'{paso:>6}  {sgd_v:>12.6f}  {mom_v:>14.6f}  {ventaja}')

    print(f'\nPosicion final SGD:          {p_sgd.round(4)}')
    print(f'Posicion final SGD+Momentum: {p_mom.round(4)}')
    print(f'Posicion optima:             [1.0, 1.0]')


      > Salida del programa:
    === SGD vs SGD+MOMENTUM — Funcion de Rosenbrock ===
    Minimo en (1.0, 1.0), f=0.  Partimos de [-0.5, 0.8]

      Paso           SGD    SGD+Momentum  Pérdida del Momentum
    ------------------------------------------------
         0     32.500000       32.500000  igual
        50      2.895213        0.164685  17.6 veces menor
       200      2.268292        0.017729  128 veces menor
       500      0.878829        0.001051  836 veces menor
      1000      0.227352        0.000015  1.52e+04 veces menor
      2000      0.053682        0.000000  1.49e+07 veces menor
      3000      0.018243        0.000000  2.07e+10 veces menor

    Posicion final SGD:          [0.8651 0.7477]
    Posicion final SGD+Momentum: [1. 1.]
    Posicion optima:             [1.0, 1.0]
```

El resultado es espectacular: con 3000 pasos, el SGD puro todavía está a una distancia notable del mínimo (pérdida = 0.018), mientras que el Momentum ha convergido prácticamente al óptimo exacto (pérdida < 0.000001). La última columna divide las dos pérdidas: a partir de los 2000 pasos el Momentum ya está prácticamente en el mínimo y el cociente se dispara porque divide entre casi cero. Lo que importa es la posición final: el Momentum llega a (1, 1) y SGD se queda en (0.87, 0.75).

> ℹ️ ¿Por qué beta=0.9 y no 0.99 o 0.5? Con beta=0.9 se recuerdan efectivamente los últimos ~10 pasos (1/(1-0.9)=10). Con beta=0.99 se recuerdan ~100 pasos: muy suavizante pero lento para cambiar de dirección. Con beta=0.5 la memoria es corta y el efecto es pequeño. 0.9 es el valor estándar que funciona bien en la mayoría de problemas.

5.4 RMSprop — Un ritmo diferente para cada parámetro

El Momentum resuelve el problema del zigzagueo, pero no aborda otro problema igualmente importante: en una red neuronal, distintos parámetros tienen gradientes de magnitudes muy diferentes.

Los pesos de las primeras capas suelen tener gradientes pequeños (el efecto del gradiente se diluye a través de muchas capas). Los pesos de las últimas capas tienen gradientes mucho mayores. Si usamos una sola tasa de aprendizaje para todos, los parámetros con gradientes pequeños casi no se mueven, mientras que los de gradientes grandes pueden oscilar o divergir.

RMSprop (Root Mean Square Propagation, propuesto por Geoffrey Hinton en 2012) resuelve esto manteniendo una estimación de la magnitud reciente de los gradientes para cada parámetro, y dividiendo la actualización por esa estimación. Los parámetros con gradientes grandes reciben actualizaciones pequeñas; los de gradientes pequeños reciben actualizaciones grandes.

📄 **RMSprop — Regla de actualización**

```
# Inicializacion:
s = ceros (misma forma que los pesos)  # media cuadratica del gradiente

# En cada paso t:
grad = calcular_gradiente(batch, pesos)

s     = beta2 * s + (1 - beta2) * grad^2    # actualizar promedio cuadratico
pesos = pesos - lr * grad / (sqrt(s) + eps)  # actualizar normalizado

# Parametros tipicos:
#   beta2 = 0.99   (media de los ultimos ~100 gradientes al cuadrado)
#   eps   = 1e-8   (evitar division por cero cuando s es muy pequeno)
#   lr    = 0.001  (suele ser mas pequeno que en SGD)

# Intuicion:
#   grad^2 grande -> s grande -> actualizacion pequena (frena el param.)
#   grad^2 pequeno -> s pequeno -> actualizacion grande (acelera el param.)
```

5.5 Adam — Lo mejor de los dos mundos

Adam (Adaptive Moment Estimation, Kingma y Ba, 2014) combina las ideas de Momentum y RMSprop en un solo algoritmo. Mantiene dos estimaciones para cada parámetro: la media de los gradientes recientes (el 'primer momento', que es el Momentum) y la media de los cuadrados de los gradientes recientes (el 'segundo momento', que es RMSprop).

Adam añade además un mecanismo de corrección de sesgo que resuelve un problema sutil: al principio del entrenamiento, m y v se inicializan a cero. Las dos medias quedan sesgadas hacia cero en los primeros pasos, pero no por igual: v (con beta2 = 0.999) mucho más que m (con beta1 = 0.9). Como el paso es m / √v, sin corrección los primeros pasos no saldrían pequeños, sino demasiado grandes. La corrección de sesgo divide cada media entre (1 - beta^t) y lo compensa.

📄 **Adam — Regla de actualización completa**

```
# Inicializacion:
m = ceros    # primer momento (Momentum)
v = ceros    # segundo momento (RMSprop)
t = 0        # contador de pasos

# Hiperparametros estandar (raramente se cambian):
beta1 = 0.9      # decaimiento del primer momento
beta2 = 0.999    # decaimiento del segundo momento
eps   = 1e-8     # estabilidad numerica
lr    = 0.001    # este SI se ajusta

# En cada paso t:
t    += 1
grad  = calcular_gradiente(batch, pesos)

# Actualizar los momentos
m     = beta1 * m + (1 - beta1) * grad          # media del gradiente
v     = beta2 * v + (1 - beta2) * grad^2        # media del gradiente^2

# Correccion de sesgo (crucial en los primeros pasos)
m_hat = m / (1 - beta1^t)    # en t=1: m_hat = m/0.1 -> amplifica
v_hat = v / (1 - beta2^t)    # en t=1: v_hat = v/0.001 -> amplifica

# Actualizacion final
pesos = pesos - lr * m_hat / (sqrt(v_hat) + eps)
```

La corrección de sesgo es un detalle elegante que a menudo se omite en explicaciones simplificadas, pero que importa en la práctica. En el paso t=1, con beta1=0.9 y

beta2=0.999, m = 0.1*grad y v = 0.001*grad², así que m/√v ≈ 3.16 (con el signo del gradiente): sin corrección, el primer paso sería unas 3 veces mayor de lo debido. Corregir solo m (m_hat = m/0.1) lo empeoraría, hasta ≈ 31.6. Con las dos correcciones, m_hat = m/0.1 y v_hat = v/0.001, m_hat/√v_hat = 1: el primer paso mide lr.

> ℹ️ Adam es el optimizador por defecto en la mayoría de proyectos de Deep Learning por una buena razón: es robusto a la elección del learning rate (funciona bien en un rango amplio), converge rápidamente, y maneja bien los gradientes dispersos. Sus únicas debilidades son que puede generalizar peor que SGD+Momentum en algunos casos, y que interactúa mal con L2 regularization (que es lo que corrige AdamW).

5.6 AdamW — La corrección del weight decay

Cuando usamos Adam con regularización L2, ocurre algo sutil pero importante. (La regularización L2, que verás a fondo en 7.6, suma a la pérdida un castigo proporcional al cuadrado de los pesos para mantenerlos pequeños; el weight decay consigue lo mismo encogiendo directamente cada peso un poco en cada paso.) En SGD, añadir L2 al gradiente (grad += 2*lambda*W) y usar weight decay (W *= (1-lr*wd)) son matemáticamente equivalentes (tomando wd = 2·lambda). En Adam, NO lo son.

El problema es que Adam escala el gradiente por 1/sqrt(v). Cuando añadimos el término de L2 al gradiente, ese término también queda escalado por 1/sqrt(v). Esto hace que la fuerza efectiva de la regularización varíe entre parámetros de forma incontrolada: los parámetros con historial de gradientes grandes reciben menos regularización de la esperada.

AdamW (Loshchilov y Hutter, 2017) desacopla el weight decay del gradiente: primero aplica la regularización directamente sobre el peso, y luego aplica la actualización de Adam sobre el gradiente. De esta forma, la regularización tiene la misma fuerza para todos los parámetros, independientemente de sus historiales de gradiente.

📄 Adam vs AdamW — La diferencia clave

```
# ADAM con L2 (incorrecto para weight decay):
grad_con_L2 = grad + 2 * lambda * W      # L2 en el gradiente
m  = beta1 * m + (1-beta1) * grad_con_L2
v  = beta2 * v + (1-beta2) * grad_con_L2^2
W -= lr * m_hat / (sqrt(v_hat) + eps)    # L2 queda escalada por Adam

# ADAMW (correcto — weight decay desacoplado):
W *= (1 - lr * wd)                       # weight decay DIRECTO sobre W
m  = beta1 * m + (1-beta1) * grad        # gradiente SIN L2
v  = beta2 * v + (1-beta2) * grad^2
W -= lr * m_hat / (sqrt(v_hat) + eps)    # actualizacion de Adam pura

# En PyTorch: optim.AdamW(params, lr=3e-4, weight_decay=0.01)
# Es el optimizador estandar para entrenar Transformers y LLMs.
```

PROGRAMA O-2 — Implementación completa de todos los optimizadores

📄 programa_02_optimizadores.py (todos los optimizadores)

```
import numpy as np

# ============================================================
# FAMILIA DE OPTIMIZADORES — IMPLEMENTACION DESDE CERO
# Todos implementan la misma interfaz: paso(params, grads)
# ============================================================

class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr
    def paso(self, params, grads):
        for k in params:
            params[k] -= self.lr * grads[k]


class SGDMomentum:
    def __init__(self, lr=0.01, beta=0.9):
        self.lr   = lr
```

```
            self.beta = beta
            self.v    = {}
        def paso(self, params, grads):
            for k in params:
                if k not in self.v:
                    self.v[k] = np.zeros_like(params[k])
                self.v[k]  = self.beta * self.v[k] + grads[k]
                params[k] -= self.lr * self.v[k]


    class RMSprop:
        def __init__(self, lr=0.001, beta=0.99, eps=1e-8):
            self.lr   = lr
            self.beta = beta
            self.eps  = eps
            self.s    = {}
        def paso(self, params, grads):
            for k in params:
                if k not in self.s:
                    self.s[k] = np.zeros_like(params[k])
                g          = grads[k]
                self.s[k]  = self.beta * self.s[k] + (1-self.beta) * g**2
                params[k] -= self.lr * g / (np.sqrt(self.s[k]) + self.eps)


    class Adam:
        """
        El optimizador mas usado en Deep Learning.
        Combina Momentum (primer momento) + RMSprop (segundo momento)
        con correccion de sesgo para los primeros pasos.
        """
        def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
            self.lr    = lr
            self.beta1 = beta1
            self.beta2 = beta2
            self.eps   = eps
            self.m = {}    # primer momento (media del gradiente)
            self.v = {}    # segundo momento (media del gradiente^2)
            self.t = 0     # contador de pasos (para correccion de sesgo)

        def paso(self, params, grads):
            self.t += 1
            for k in params:
                if k not in self.m:
                    self.m[k] = np.zeros_like(params[k])
                    self.v[k] = np.zeros_like(params[k])
                g = grads[k]
                # Actualizar momentos
                self.m[k] = self.beta1 * self.m[k] + (1 - self.beta1) * g
                self.v[k] = self.beta2 * self.v[k] + (1 - self.beta2) * g**2
                # Correccion de sesgo
                m_hat = self.m[k] / (1 - self.beta1**self.t)
                v_hat = self.v[k] / (1 - self.beta2**self.t)
                # Actualizacion final
                params[k] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)


    class AdamW(Adam):
        """
        Adam con weight decay desacoplado.
        Estandar para entrenar Transformers y LLMs.
        """
        def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, wd=0.01):
            super().__init__(lr, beta1, beta2, eps)
            self.wd = wd

        def paso(self, params, grads):
            self.t += 1
            for k in params:
                if k not in self.m:
                    self.m[k] = np.zeros_like(params[k])
                    self.v[k] = np.zeros_like(params[k])
                # Weight decay desacoplado (ANTES del gradiente)
```

```
            params[k] *= (1 - self.lr * self.wd)
            g = grads[k]
            self.m[k] = self.beta1 * self.m[k] + (1 - self.beta1) * g
            self.v[k] = self.beta2 * self.v[k] + (1 - self.beta2) * g**2
            m_hat = self.m[k] / (1 - self.beta1**self.t)
            v_hat = self.v[k] / (1 - self.beta2**self.t)
            params[k] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)


print('Optimizadores implementados correctamente.')
print('Clases disponibles: SGD, SGDMomentum, RMSprop, Adam, AdamW')
print()
# Verificacion: todos tienen la misma interfaz
opts = [SGD(0.01), SGDMomentum(0.01), RMSprop(0.001), Adam(0.001), AdamW(0.001)]
params_test = {'W': np.array([1.0, -0.5, 2.0])}
grads_test  = {'W': np.array([0.1, -0.2, 0.3])}
for opt in opts:
    p = {k: v.copy() for k,v in params_test.items()}
    opt.paso(p, grads_test)
    print(f'{opt.__class__.__name__:>14}: W -> {p["W"].round(5)}')
```

> Salida del programa:

```
Optimizadores implementados correctamente.
Clases disponibles: SGD, SGDMomentum, RMSprop, Adam, AdamW

           SGD: W -> [ 0.999 -0.498  1.997]
   SGDMomentum: W -> [ 0.999 -0.498  1.997]
       RMSprop: W -> [ 0.99 -0.49  1.99]
          Adam: W -> [ 0.999 -0.499  1.999]
         AdamW: W -> [ 0.99899 -0.499    1.99898]
```

ℹ️ Fíjate en Adam: aunque los gradientes son [0.1, -0.2, 0.3] con magnitudes distintas, las tres actualizaciones son casi iguales (≈0.001). Adam normaliza el gradiente por su magnitud histórica, haciendo que todos los parámetros se actualicen con pasos similares independientemente de la magnitud de su gradiente.

ℹ️ El Programa O-3 usa cuatro términos que el Capítulo 7 explica con calma: una época es una pasada completa por los datos de entrenamiento; el conjunto de validación son ejemplos que el modelo no usa para aprender, y sirven para medir cómo generaliza (7.5); la precisión es el porcentaje de ejemplos bien clasificados; y las «dos lunas» son dos nubes de puntos con forma de media luna, encajadas una en la otra, que ninguna recta puede separar. El Programa O-3 entrena una red de dos capas, la que construiremos paso a paso en el Capítulo 6; de momento basta con verla como la caja que el optimizador ajusta. Y reutiliza las clases del Programa O-2 sin copiarlas: lee su archivo y ejecuta la parte que va antes de su primera instrucción print, con exec(fuente.split(...)[0]). Es un atajo para que cada programa sea un solo archivo que se lee de principio a fin; en un proyecto real ese código iría en un módulo y se usaría import, como haremos con el Programa 3. Los dos archivos tienen que estar en la misma carpeta.

PROGRAMA O-3 — Comparativa de todos los optimizadores en el MLP

📄 **programa_03_comparativa.py**

```
import numpy as np
fuente = open('programa_02_optimizadores.py', encoding='utf-8').read()
exec(fuente.split("print('Optimizadores")[0])   # las clases del Programa O-2

np.random.seed(42)

# ============================================================
# PROBLEMA: clasificacion de dos lunas (no lineal)
# Generamos dos semilunares entrelazadas — clasico para
# demostrar la diferencia entre optimizadores
# ============================================================
def dos_lunas(n=300, ruido=0.15):
    """Genera dos semilunares entrelazadas"""
    n_half = n // 2
    # Luna superior
```

```
    ang1 = np.linspace(0, np.pi, n_half)
    X1   = np.column_stack([np.cos(ang1), np.sin(ang1)])
    X1  += np.random.normal(0, ruido, X1.shape)
    # Luna inferior (desplazada)
    ang2 = np.linspace(0, np.pi, n_half)
    X2   = np.column_stack([1 - np.cos(ang2), 0.5 - np.sin(ang2)])
    X2  += np.random.normal(0, ruido, X2.shape)
    X = np.vstack([X1, X2])
    y = np.array([0]*n_half + [1]*n_half)
    idx = np.random.permutation(n)
    return X[idx], y[idx]

X, y = dos_lunas(300)
X_tr, y_tr   = X[:240], y[:240]
X_val, y_val = X[240:], y[240:]

# ============================================================
# RED NEURONAL COMUN PARA TODOS LOS OPTIMIZADORES
# ============================================================
def sigmoid(z): return 1/(1+np.exp(-np.clip(z,-500,500)))
def relu(z):    return np.maximum(0, z)
def d_relu(z):  return (z > 0).astype(float)

def inicializar(s=0):
    np.random.seed(s)
    return {
        'W1': np.random.randn(2, 16) * np.sqrt(2/2),
        'b1': np.zeros(16),
        'W2': np.random.randn(16, 1) * 0.1,
        'b2': np.zeros(1),
    }

def forward(X, p):
    Z1 = X @ p['W1'] + p['b1']
    A1 = relu(Z1)
    Z2 = A1 @ p['W2'] + p['b2']
    return sigmoid(Z2), Z1, A1

def backward(X, y, pred, Z1, A1, p):
    m    = len(X)
    dZ2  = (pred - y.reshape(-1,1)) / m
    grads = {
        'W2': A1.T @ dZ2,
        'b2': dZ2.sum(axis=0),
    }
    dA1  = dZ2 @ p['W2'].T
    dZ1  = dA1 * d_relu(Z1)
    grads['W1'] = X.T @ dZ1
    grads['b1'] = dZ1.sum(axis=0)
    return grads

def bce_loss(pred, y):
    p = np.clip(pred.ravel(), 1e-7, 1-1e-7)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

def accuracy(X, y, p):
    pred, _, _ = forward(X, p)
    return np.mean((pred.ravel()>0.5) == y)

# ============================================================
# COMPARATIVA: mismos datos, misma red, distintos optimizadores
# ============================================================
configs = [
    ('SGD              lr=0.1 ', SGD(lr=0.1)),
    ('SGD+Momentum     lr=0.1 ', SGDMomentum(lr=0.1, beta=0.9)),
    ('RMSprop          lr=0.01', RMSprop(lr=0.01)),
    ('Adam             lr=0.01', Adam(lr=0.01)),
    ('AdamW            lr=0.01', AdamW(lr=0.01, wd=0.01)),
]

EPOCAS   = 500
BATCH    = 32
```

```
print('=== COMPARATIVA DE OPTIMIZADORES ===')
print(f'Problema: clasificacion de dos lunas (no lineal)')
print(f'Red: 2 -> 16 (ReLU) -> 1 (Sigmoid)  |  {EPOCAS} epocas\n')
print(f"{'Optimizador':>30}  {'Loss ep.10':>12}  {'Loss ep.100':>12}  ",
      f"{'Loss ep.500':>12}  {'Acc val':>8}")
print('-' * 85)

for nombre, opt in configs:
    params = inicializar(s=42)
    hist   = []
    for ep in range(EPOCAS):
        # Mini-batch SGD
        idx = np.random.permutation(len(X_tr))
        for start in range(0, len(X_tr), BATCH):
            xb = X_tr[idx[start:start+BATCH]]
            yb = y_tr[idx[start:start+BATCH]]
            pred, Z1, A1 = forward(xb, params)
            grads = backward(xb, yb, pred, Z1, A1, params)
            opt.paso(params, grads)
        pred_tr, _, _ = forward(X_tr, params)
        hist.append(bce_loss(pred_tr, y_tr))

    acc = accuracy(X_val, y_val, params)
    l10  = hist[9]  if len(hist)>9  else hist[-1]
    l100 = hist[99] if len(hist)>99 else hist[-1]
    l500 = hist[-1]
    print(f'{nombre:>30}  {l10:>12.5f}  {l100:>12.5f}  {l500:>12.5f}  {acc:>8.1%}')
```

> Salida del programa:

```
=== COMPARATIVA DE OPTIMIZADORES ===
Problema: clasificacion de dos lunas (no lineal)
Red: 2 -> 16 (ReLU) -> 1 (Sigmoid)  |  500 epocas

                    Optimizador    Loss ep.10   Loss ep.100   Loss ep.500   Acc val
-------------------------------------------------------------------------------------
         SGD              lr=0.1       0.30191       0.21244       0.02772   100.0%
         SGD+Momentum     lr=0.1       0.20937       0.01290       0.00603   100.0%
         RMSprop          lr=0.01      0.22907       0.01335       0.00224   100.0%
         Adam             lr=0.01      0.25832       0.02132       0.00412   100.0%
         AdamW            lr=0.01      0.25861       0.02355       0.00567   100.0%
```

La tabla cuenta una historia clara, aunque en la precisión final no se note: el problema de las dos lunas es lo bastante sencillo como para que los 5 optimizadores terminen clasificando el 100% de los ejemplos de validación a 500 épocas. Donde sí se aprecia la diferencia es en la pérdida: el SGD puro llega a 500 épocas con una pérdida bastante más alta (0.028) que el resto, es decir, con predicciones menos seguras aunque igual de correctas. El Momentum ya mejora notablemente (0.006), y RMSprop, Adam y AdamW bajan todavía más: RMSprop llega a la pérdida más baja (0.002) y AdamW queda algo por encima de Adam (0.0057 frente a 0.0041). Es lo esperable: el weight decay frena a propósito el crecimiento de los pesos, y con él la seguridad de las predicciones, a cambio de regularizar; en un problema tan sencillo, esa regularización no se traduce en mejor precisión.

5.7 Learning Rate Scheduling — El ritmo cambia

Incluso con Adam o AdamW, la tasa de aprendizaje fija es subóptima. Al inicio del entrenamiento, queremos pasos grandes para explorar el espacio de parámetros rápidamente. Al final, queremos pasos pequeños para afinar los pesos con precisión alrededor del mínimo.

Los LLMs modernos usan casi universalmente una estrategia de dos fases: un calentamiento (warm-up) seguido de un decaimiento en coseno. Puede parecer contradictorio empezar con pasos pequeños si al principio queremos pasos grandes, pero tiene una razón: en los primeros pasos los pesos son aleatorios, los gradientes son grandes e inestables y las medias de Adam (m y v) se apoyan todavía en muy pocos datos, así que un paso grande podría estropear la red. El calentamiento sube la tasa poco a poco durante esos primeros pasos; después, el decaimiento en coseno la va bajando con suavidad, siguiendo media onda de coseno, hasta casi cero.

PROGRAMA O-4 — Learning Rate Scheduling

📄 **programa_04_lr_scheduling.py**

```
import numpy as np

# ============================================================
# ESTRATEGIAS DE LEARNING RATE SCHEDULING
# Cada funcion recibe el numero de paso actual y
# devuelve el lr a usar en ese paso.
# ============================================================

def lr_constante(t, lr_max, **kw):
    return lr_max

def lr_step_decay(t, lr_max, step=100, factor=0.5, **kw):
    """Reduce el lr a la mitad cada 'step' epocas"""
    return lr_max * (factor ** (t // step))

def lr_exponencial(t, lr_max, decay=0.995, **kw):
    """Reduce el lr exponencialmente en cada paso"""
    return lr_max * (decay ** t)

def lr_coseno(t, lr_max, t_total, lr_min=1e-6, **kw):
    """
    Decaimiento en coseno: el lr baja suavemente siguiendo
    la mitad de un periodo del coseno.
    """
    return lr_min + 0.5*(lr_max - lr_min)*(1 + np.cos(np.pi*t/t_total))

def lr_warmup_coseno(t, lr_max, t_total, t_warmup=100, lr_min=1e-6, **kw):
    """
    Estandar en LLMs: sube linealmente durante t_warmup pasos,
    luego baja en coseno hasta lr_min.
    Motivo del warmup: al inicio los gradientes son inestables.
    Comenzar con lr alto puede causar divergencia.
    """
    if t < t_warmup:
        return lr_max * t / t_warmup          # subida lineal
    t_adj   = t - t_warmup
    t_decay = t_total - t_warmup
    return lr_min + 0.5*(lr_max - lr_min)*(1 + np.cos(np.pi*t_adj/t_decay))


# ============================================================
# VISUALIZACION DE LAS CURVAS DE LR
# ============================================================
T      = 500   # total de pasos
LR_MAX = 0.01

schedules = {
    'Constante':      lambda t: lr_constante(t, LR_MAX),
    'Step decay':     lambda t: lr_step_decay(t, LR_MAX, step=100),
    'Exponencial':    lambda t: lr_exponencial(t, LR_MAX),
    'Coseno':         lambda t: lr_coseno(t, LR_MAX, T),
    'Warmup+Coseno':  lambda t: lr_warmup_coseno(t, LR_MAX, T, t_warmup=50),
}

pasos_ref = [0, 25, 50, 100, 200, 300, 400, 500]

print('=== LEARNING RATE SCHEDULING ===')
print(f'lr_max={LR_MAX}, T={T} pasos\n')
print(f"{'Paso':>6}  ", end='')
for nombre in schedules: print(f'{nombre:>16}', end='')
print()
print('-' * (6 + 16*len(schedules) + 2))

for t in pasos_ref:
    print(f'{t:>6}  ', end='')
    for fn in schedules.values():
        print(f'{fn(t):>16.6f}', end='')
    print()

# Visualizacion ASCII de la curva Warmup+Coseno
```

```
fn_wc = schedules['Warmup+Coseno']
curva = [fn_wc(t) for t in range(T+1)]
print('\nWarmup+Coseno (el estandar en LLMs):')
print(f'  0.010 |', end='')
for t in range(0, T+1, 10):
    col = int(curva[t] / LR_MAX * 30)
    print('#' if col > 28 else ('|' if t==50 else ' '), end='')
print('|')
print(f'  0.005 |', end='')
for t in range(0, T+1, 10):
    col = int(curva[t] / LR_MAX * 30)
    marker = '#' if 14 < col <= 28 else ('.' if col==15 else ' ')
    print(marker, end='')
print('|')
print(f'  0.001 |' + '.' * 5 + '...warmup..|.......coseno decay......|')
print(f'         paso 0         50        100                       500')

print()
print('Durante el warmup (pasos 0-50): el lr sube de 0 a lr_max.')
print('Durante el decay (pasos 50-500): baja suavemente en coseno.')
print('Al final, lr ~ lr_min (casi cero) para ajuste fino preciso.')
```

> Salida del programa:

```
=== LEARNING RATE SCHEDULING ===
lr_max=0.01, T=500 pasos

  Paso         Constante      Step decay     Exponencial          Coseno   Warmup+Coseno
------------------------------------------------------------------------------------------
     0          0.010000        0.010000        0.010000        0.010000        0.000000
    25          0.010000        0.010000        0.008822        0.009938        0.005000
    50          0.010000        0.010000        0.007783        0.009755        0.010000
   100          0.010000        0.005000        0.006058        0.009045        0.009698
   200          0.010000        0.002500        0.003670        0.006545        0.007500
   300          0.010000        0.001250        0.002223        0.003456        0.004132
   400          0.010000        0.000625        0.001347        0.000956        0.001171
   500          0.010000        0.000313        0.000816        0.000001        0.000001

Warmup+Coseno (el estandar en LLMs):
  0.010 |     ######                                |
  0.005 |   ##      #################               |
  0.001 |........warmup..|.......coseno decay......|
         paso 0         50        100                       500

Durante el warmup (pasos 0-50): el lr sube de 0 a lr_max.
Durante el decay (pasos 50-500): baja suavemente en coseno.
Al final, lr ~ lr_min (casi cero) para ajuste fino preciso.
```

5.8 Gradient Clipping — La red de seguridad

Cuando el gradiente se hace muy grande — lo que se llama explosión del gradiente — la actualización de los pesos puede ser tan grande que el modelo 'sale disparado' lejos del mínimo y nunca se recupera. Esto es especialmente frecuente en redes recurrentes y en las primeras épocas del entrenamiento de modelos grandes.

El gradient clipping es una técnica de emergencia muy simple: si la norma del vector de gradiente supera cierto umbral, se reescala el vector para que su norma sea exactamente ese umbral. La dirección del gradiente se preserva; solo se limita la magnitud.

📄 Gradient Clipping — implementación

```
def clip_gradientes(grads, max_norma=1.0):
    """
    Recorta los gradientes si su norma supera max_norma.
    Preserva la direccion del gradiente.
    """
    # Calcular la norma total de todos los gradientes
    norma_total = np.sqrt(sum(
        np.sum(g**2) for g in grads.values()
    ))

    if norma_total > max_norma:
        factor = max_norma / (norma_total + 1e-10)
        grads  = {k: v * factor for k, v in grads.items()}
```

```
    return grads, float(norma_total)


# Ejemplo de uso en el bucle de entrenamiento:
for ep in range(EPOCAS):
    pred, Z1, A1 = forward(X_batch, params)
    grads        = backward(X_batch, y_batch, pred, Z1, A1, params)

    # Clipping ANTES de pasar al optimizador
    grads, norma = clip_gradientes(grads, max_norma=1.0)

    if norma > 1.0:
        print(f'  ep {ep}: gradiente recortado (norma={norma:.2f} -> 1.0)')

    optimizador.paso(params, grads)


# En PyTorch:
# loss.backward()
# torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
# optimizer.step()

# Valores tipicos de max_norma: 1.0 (transformers) a 5.0 (RNNs)
```

> 💡 El Nano-LLM del Capítulo 13 y el Apéndice D usarán gradient clipping, y ahora sabes por qué: en redes profundas, sobre todo al inicio del entrenamiento y con learning rates altos, aparecen de vez en cuando gradientes muy grandes, y el recorte impide que un solo paso desbarate lo aprendido. En el Programa 8 lo verás en el bucle de entrenamiento (sección 13.3.4): allí el recorte actúa en casi todos los pasos.

5.9 Receta práctica — Cómo elegir el optimizador

Con todo lo anterior, aquí está la guía de decisión que usan los profesionales:

```
  📄 Guia de seleccion de optimizador
PROBLEMA GENERAL (clasificacion, regresion, MLP):
  -> Adam(lr=0.001) como punto de partida
  -> Si hay regularizacion: AdamW(lr=0.001, weight_decay=0.01)
  -> Ajustar solo lr si no converge bien

TRANSFORMERS Y LLMs:
  -> AdamW(lr=3e-4, beta1=0.9, beta2=0.95, eps=1e-8, wd=0.1)
  -> Warmup lineal (500-2000 pasos) + cosine decay
  -> Gradient clipping (max_norm=1.0)

VISION POR COMPUTADOR (CNNs, entrenamiento largo):
  -> SGD+Momentum(lr=0.1, beta=0.9) con step decay
  -> O Adam si el entrenamiento es corto

SEÑALES DE PROBLEMAS Y SOLUCIONES:
  Loss no baja         -> lr demasiado pequeno: multiplicar por 10
  Loss oscila mucho    -> lr demasiado grande: dividir por 3
  Loss explota (NaN)   -> añadir gradient clipping (max_norm=1.0)
  Train ok, val malo   -> añadir weight decay o dropout
  Convergencia lenta   -> cambiar a Adam si estabas con SGD
  Buen train, val ok   -> añadir warmup+coseno para afinar

SEÑALES DE UN BUEN ENTRENAMIENTO:
  - Loss baja monotonamente las primeras N epocas
  - La norma del gradiente es estable (no explota ni colapsa)
  - Train loss y val loss bajan juntas al principio
  - La diferencia train-val crece lentamente (ok) o no crece (mejor)
```

Los optimizadores son el motor invisible de todo el Deep Learning. Cuando un LLM con miles de millones de parámetros aprende de billones de tokens, lo hace con AdamW y

warmup+coseno — exactamente los algoritmos que acabas de implementar desde cero. La escala es diferente; el principio es idéntico.

> El Programa 8 (Capítulo 13) sigue esta receta: Adam con warm-up y decaimiento coseno y recorte del gradiente. Lo único que no incluye es el weight decay; el Ejercicio 13.7 y la sección 13.6.1 exploran qué ocurre al añadirlo.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
> - SGD básico zigzaguea en valles estrechos; Momentum lo corrige acumulando velocidad de gradientes pasados.
> - Adam combina Momentum y RMSprop con corrección de sesgo; AdamW desacopla correctamente el weight decay.
> - El learning rate scheduling (warmup + cosine decay) es el estándar para entrenar Transformers de forma estable.

✏️ EJERCICIOS — Capítulo 5 — Los Optimizadores

Los optimizadores se entienden de verdad al verlos trabajar. Resuelve estos ejercicios antes de continuar y compara tus resultados con los propuestos: las diferencias suelen ser muy instructivas.

Ejercicio 5.1 Implementar Nesterov Accelerated Gradient

Nesterov Momentum es una variante de Momentum que 'mira hacia adelante' antes de calcular el gradiente: en lugar de calcular el gradiente en la posición actual, lo calcula en la posición que tendríamos si aplicáramos solo el momentum acumulado. Implementa esta variante y compárala con Momentum estándar en la función de Rosenbrock del Programa O-1.

> 💡 💡 Pista: La fórmula es: v = beta*v + grad(pesos - lr*beta*v); pesos -= lr*v. La diferencia clave es que el gradiente se evalúa en 'pesos - lr*beta*v', no en 'pesos'.

✅ **Solución:**

📄 **ejercicio_5_1.py**

```
import numpy as np

def f(p):      return (1-p[0])**2 + 100*(p[1]-p[0]**2)**2
def grad_f(p):
    dx = -2*(1-p[0]) - 400*p[0]*(p[1]-p[0]**2)
    dy = 200*(p[1]-p[0]**2)
    return np.array([dx, dy])

P0, N, LR, BETA = np.array([-0.5, 0.8]), 2000, 0.001, 0.9

# --- Momentum estandar ---
p_mom = P0.copy(); v = np.zeros(2)
for _ in range(N):
    g = grad_f(p_mom)
    v = BETA*v + g
    p_mom -= LR*v

# --- Nesterov Momentum ---
p_nes = P0.copy(); v = np.zeros(2)
for _ in range(N):
    # Evaluar el gradiente en la posicion 'adelantada'
    p_adelantado = p_nes - LR * BETA * v
    g = grad_f(p_adelantado)
    v = BETA*v + g
    p_nes -= LR*v

print('=== MOMENTUM vs NESTEROV ===')
print(f'Momentum: f={f(p_mom):.6f}  pos={p_mom.round(4)}')
print(f'Nesterov: f={f(p_nes):.6f}  pos={p_nes.round(4)}')
print(f'Optimo:   f=0.0  pos=[1.0, 1.0]')
print()
print('Nesterov suele converger algo mas rapido porque "corrige"')
print('su trayectoria antes de comprometerse con el paso completo.')
```

Ejercicio 5.2 Visualizar la trayectoria de Adam paso a paso

Implementa Adam y registra la posición (x, y) en cada uno de los primeros 15 pasos al optimizar la función $f(x,y) = x^2 + 10y^2$. Muestra cómo Adam normaliza automáticamente el paso en cada dimensión, a pesar de que el gradiente en y es mucho más grande que en x.

> 💡 💡 Pista: El gradiente de f es [2x, 20y]. La dimensión y tiene un gradiente 10 veces mayor que x. Sin normalización (como en SGD), esto causaría un movimiento desproporcionado. Observa cómo Adam compensa esto con v_hat.

✅ **Solución:**

📄 ejercicio_5_2.py

```
import numpy as np

def f(p):    return p[0]**2 + 10*p[1]**2
def grad(p): return np.array([2*p[0], 20*p[1]])

p = np.array([5.0, 5.0])
m = np.zeros(2); v = np.zeros(2)
beta1, beta2, eps, lr = 0.9, 0.999, 1e-8, 0.3

print('=== TRAYECTORIA DE ADAM ===')
print('f(x,y) = x^2 + 10y^2   (gradiente en y es 10x mayor que en x)')
print()
print(f"{'Paso':>5}  {'x':>8}  {'y':>8}  {'grad_x':>8}  {'grad_y':>8}  {'paso_x':>8}
{'paso_y':>8}")
print('-' * 65)

for t in range(1, 16):
    g = grad(p)
    m = beta1*m + (1-beta1)*g
    v = beta2*v + (1-beta2)*g**2
    m_hat = m / (1 - beta1**t)
    v_hat = v / (1 - beta2**t)
    paso = lr * m_hat / (np.sqrt(v_hat) + eps)
    p_anterior = p.copy()
    p -= paso
    if t <= 10 or t == 15:
        print(f'{t:>5}  {p[0]:>8.4f}  {p[1]:>8.4f}  {g[0]:>8.3f}  {g[1]:>8.3f}
{paso[0]:>8.4f}  {paso[1]:>8.4f}')

print()
print('Observa: aunque grad_y es ~10x mayor que grad_x al principio,')
print('los PASOS en x e y son de magnitud similar.')
print('Adam normaliza por la magnitud historica de cada gradiente.')
```

Ejercicio 5.3 Comparar AdamW con distintos valores de weight decay

Entrena el mismo MLP del Programa O-3 (clasificación de dos lunas) con AdamW usando weight_decay = 0.0, 0.001, 0.01, 0.1 y 1.0. Registra la precisión en validación para cada valor y determina cuál ofrece el mejor equilibrio.

> 💡 💡 Pista: Un weight_decay = 0 es equivalente a Adam normal. Valores muy grandes (1.0) regularizan tan fuerte que el modelo no puede ajustarse a los datos (underfitting). El óptimo suele estar entre 0.001 y 0.1, dependiendo del problema.

✅ **Solución:**

📄 ejercicio_5_3.py

```
import numpy as np
fuente = open('programa_O3_comparativa.py', encoding='utf-8').read()
exec(fuente.split('# COMPARATIVA')[0])   # datos, red y optimizadores (O-2 y O-3)

np.random.seed(42)
X, y = dos_lunas(300)
X_tr, y_tr   = X[:240], y[:240]
X_val, y_val = X[240:], y[240:]

valores_wd = [0.0, 0.001, 0.01, 0.1, 1.0]

print('=== EFECTO DEL WEIGHT DECAY EN ADAMW ===\n')
print(f"{'weight_decay':>14}  {'Loss train':>12}  {'Acc val':>10}  Diagnostico")
print('-' * 55)

for wd in valores_wd:
    params = inicializar(s=42)
    opt = AdamW(lr=0.01, wd=wd)
    for ep in range(300):
        pred, Z1, A1 = forward(X_tr, params)
        grads = backward(X_tr, y_tr, pred, Z1, A1, params)
        opt.paso(params, grads)
```

```
    pred_final, _, _ = forward(X_tr, params)
    loss = bce_loss(pred_final, y_tr)
    acc  = accuracy(X_val, y_val, params)
    diag = 'underfitting' if wd >= 1.0 else ('optimo' if 0.001<=wd<=0.1 else 'sin
regularizar')
    print(f'{wd:>14.3f}  {loss:>12.5f}  {acc:>10.1%}  {diag}')
```

> ℹ️ weight_decay demasiado grande fuerza todos los pesos hacia cero, impidiendo que el modelo aprenda patrones útiles — el mismo fenómeno de underfitting que verás con L2 en el Capítulo 7.

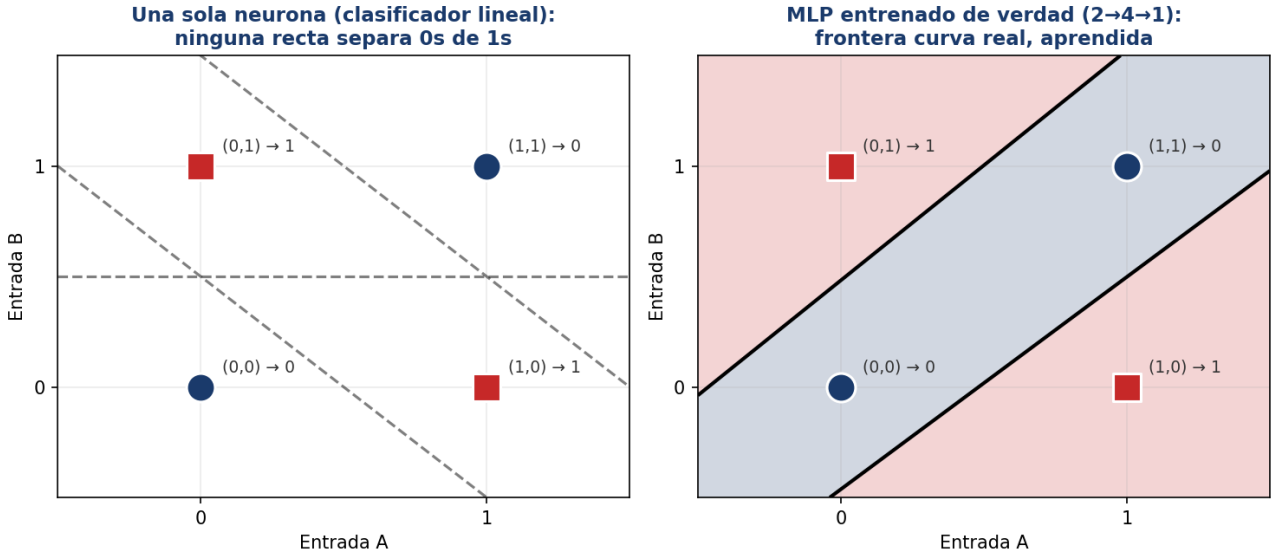
Capítulo 6: Red Neuronal Multicapa (MLP) (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar matemáticamente por qué una sola neurona no puede resolver XOR.
> ✓ Construir desde cero capas densas reutilizables con forward y backward.
> ✓ Implementar la clase RedNeuronal genérica y entrenarla con backpropagation.
> ✓ Resolver XOR con éxito y entender qué representación interna lo hace posible.

6.1 Por qué una sola neurona no es suficiente

En 1969, Marvin Minsky y Seymour Papert demostraron matemáticamente que el Perceptrón (una sola neurona) no puede resolver el problema XOR. La función XOR devuelve 1 cuando exactamente una de las dos entradas es 1:

```
📄 El problema XOR
# Tabla verdad del XOR (OR exclusivo):
#  Entrada A | Entrada B | Salida
#  ---------+-----------+-------
#     0     |     0     |   0
#     0     |     1     |   1    <- solo uno a 1
#     1     |     0     |   1    <- solo uno a 1
#     1     |     1     |   0

# ¿Por qué no puede resolverlo una sola neurona?
# Una neurona es un clasificador LINEAL:
# traza una sola línea recta en el espacio de entrada.
# XOR no es linealmente separable — no existe ninguna
# línea recta que separe los 0s de los 1s.
```



La solución llegó con las redes neuronales multicapa (MLP, Multi-Layer Perceptron). Al añadir una o más capas ocultas entre la entrada y la salida, la red puede aprender transformaciones no lineales del espacio de entrada que hacen el problema linealmente separable en capas superiores.

6.2 Arquitectura del MLP

Un MLP tiene tres tipos de capas:

* Capa de entrada: recibe los datos en bruto. No realiza ningún cálculo, solo pasa los datos a la siguiente capa.

* Capas ocultas: una o varias capas de neuronas con función de activación no lineal. Aquí ocurre la 'magia' de la representación.
* Capa de salida: produce la predicción final. El número de neuronas depende del problema.

Cada neurona de una capa está conectada con todas las neuronas de la capa siguiente (por eso se llaman capas 'densas' o 'totalmente conectadas'). Si la capa anterior tiene M neuronas y la siguiente tiene N, hay M×N pesos entre ellas.

> ℹ️ Una MLP con al menos una capa oculta con suficientes neuronas y una función de activación no lineal es un 'aproximador universal': puede aproximar cualquier función continua (en un intervalo acotado) con la precisión que se quiera. Esta propiedad, demostrada teóricamente, justifica el poder expresivo de las redes neuronales. Ojo: el teorema garantiza que esos pesos existen, pero no dice cuántas neuronas hacen falta ni que el entrenamiento vaya a encontrarlos.

6.3 Forward propagation en una red

El paso forward (hacia adelante) consiste en calcular la salida de la red dado un conjunto de entradas. La información fluye desde la capa de entrada hasta la de salida, capa por capa.

📄 Forward pass en una red de 2 capas (álgebra lineal)

```
# Red: 2 entradas -> 4 neuronas ocultas -> 1 salida
# Notación matricial: X tiene dimensión (batch, n_entradas)

# Capa oculta:
Z1 = X  @ W1 + b1   # (batch, 4)
A1 = ReLU(Z1)        # (batch, 4) — activación

# Capa de salida:
Z2 = A1 @ W2 + b2   # (batch, 1)
A2 = sigmoid(Z2)     # (batch, 1) — predicción final

# Dimensiones de los parámetros:
# W1: (2, 4)  b1: (4,)
# W2: (4, 1)  b2: (1,)

# Total de parámetros: 2*4 + 4 + 4*1 + 1 = 17
# Pequeño pero capaz de resolver XOR!
```

6.4 Backpropagation a través de capas

En una red multicapa, backpropagation propaga el error hacia atrás capa por capa, aplicando la regla de la cadena en cada paso. Para la última capa el cálculo es igual que el de la neurona individual. Para las capas anteriores, el gradiente que llega de la capa siguiente se multiplica por la derivada local de esa capa.

La belleza del algoritmo está en su eficiencia: calcula los gradientes de todos los parámetros en tiempo proporcional a una sola pasada hacia adelante. Sin backprop, entrenar redes profundas habría sido computacionalmente inviable.

Para la red de dos capas de 6.3, las fórmulas caben en unas líneas. Cada una es la regla de la cadena de 4.4 aplicada a una operación del forward, recorrido al revés:

📄 Backward de una red de 2 capas (álgebra lineal)

```
# Forward (6.3):  Z1 = X @ W1 + b1;  A1 = ReLU(Z1);  Z2 = A1 @ W2 + b2;  A2 =
sigmoid(Z2)
# Pérdida MSE sobre N ejemplos:  L = media de (A2 - Y)**2

dA2 = 2 * (A2 - Y) / N      # cuánto cambia L al mover cada predicción
dZ2 = dA2 * A2 * (1 - A2)   # atraviesa la sigmoide (su derivada, Capítulo 2)
dW2 = A1.T @ dZ2            # (4, 1): lo que entra en la capa por el error de su salida
db2 = dZ2.sum(axis=0)       # (1,)
dA1 = dZ2 @ W2.T            # (batch, 4): el error vuelve por los mismos pesos
dZ1 = dA1 * (Z1 > 0)        # atraviesa la ReLU: solo donde estaba activa
dW1 = X.T @ dZ1             # (2, 4)
```

```
    db1 = dZ1.sum(axis=0)       # (4,)

    # Regla para no perderse: el gradiente de cada matriz tiene su misma forma.
```

Son exactamente las operaciones de los métodos backward() del Programa 3: CapaDensa calcula dW, db y el gradiente que devuelve hacia atrás ($dX = dZ \cdot W^T$); ReLU y Sigmoid multiplican por su derivada. El 1/N de la media ya viene dentro de dPred, así que CapaDensa.backward() no vuelve a dividir entre el tamaño del lote: $dW = X^T \cdot dZ$, y db es la suma de dZ sobre los ejemplos.

6.5 Implementación: la clase MLP

PROGRAMA 3 — Red Neuronal Multicapa desde cero

📄 **programa_3_mlp.py (parte 1: capas y activaciones)**

```
import numpy as np

# ============================================================
# BLOQUES CONSTRUCTIVOS DEL MLP
# Cada bloque tiene métodos forward() y backward()
# ============================================================

class CapaDensa:
    """Capa totalmente conectada: salida = X @ W + b"""

    def __init__(self, n_ent, n_sal):
        # Inicialización He — buena para ReLU
        escala = np.sqrt(2.0 / n_ent)
        self.W = np.random.randn(n_ent, n_sal) * escala
        self.b = np.zeros(n_sal)
        # Cache: guardamos valores para backprop
        self._X = None

    def forward(self, X):
        self._X = X                      # guardamos para backprop
        return X @ self.W + self.b

    def backward(self, dZ, lr):
        dW  = self._X.T @ dZ             # gradiente de W (dZ ya trae el 1/N de la
pérdida media)
        db  = dZ.sum(axis=0)             # gradiente de b
        dX  = dZ @ self.W.T              # gradiente hacia atrás
        self.W -= lr * dW                # actualizar W
        self.b -= lr * db                # actualizar b
        return dX


class ReLU:
    """Activación ReLU con su derivada para backprop"""
    def __init__(self): self._Z = None
    def forward(self, Z):
        self._Z = Z
        return np.maximum(0, Z)
    def backward(self, dA, lr=None):
        return dA * (self._Z > 0)        # derivada es 0 ó 1


class Sigmoid:
    """Activación sigmoid con su derivada"""
    def __init__(self): self._A = None
    def forward(self, Z):
        self._A = 1.0 / (1.0 + np.exp(-np.clip(Z, -500, 500)))
        return self._A
    def backward(self, dA, lr=None):
        s = self._A
        return dA * s * (1.0 - s)
```

📄 **programa_3_mlp.py (parte 2: la clase RedNeuronal)**

```
class RedNeuronal:
    """
```

```
    MLP genérico: acepta cualquier secuencia de capas.
    Ejemplo de uso:
      red = RedNeuronal([
          CapaDensa(2, 4), ReLU(),
          CapaDensa(4, 1), Sigmoid()
      ])
    """

    def __init__(self, capas):
        self.capas = capas

    def predecir(self, X):
        """Paso forward completo"""
        salida = X
        for capa in self.capas:
            salida = capa.forward(salida)
        return salida

    def _backward(self, grad, lr):
        """Backpropagation: propaga el gradiente hacia atrás"""
        for capa in reversed(self.capas):
            grad = capa.backward(grad, lr)

    def entrenar(self, X, y, epocas=2000, lr=0.1, verbose=True):
        """Bucle de entrenamiento completo"""
        y = y.reshape(-1, 1)
        historico = []

        for epoca in range(epocas):
            # --- Forward ---
            pred  = self.predecir(X)
            # --- Pérdida MSE ---
            loss  = float(np.mean((pred - y) ** 2))
            historico.append(loss)
            # --- Gradiente de MSE respecto a la predicción ---
            dPred = 2.0 * (pred - y) / len(X)
            # --- Backward ---
            self._backward(dPred, lr)

            if verbose and (epoca % 500 == 0 or epoca == epocas - 1):
                print(f'  Época {epoca:5d}: pérdida = {loss:.6f}')

        return historico
```

PROGRAMA 4 — MLP resuelve XOR

📄 **programa_4_xor.py**

```
import numpy as np
from programa_3_mlp import CapaDensa, ReLU, Sigmoid, RedNeuronal   # Programa 3

# ============================================================
# RESOLVIENDO XOR CON UNA RED MULTICAPA
# ============================================================

np.random.seed(42)

# Datos del XOR
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0,  1,  1,  0],          dtype=float)

# Arquitectura: 2 -> 8 -> 1
# La capa oculta de 8 neuronas da más que suficiente capacidad
red = RedNeuronal([
    CapaDensa(2, 8),   # capa oculta: 2 entradas -> 8 neuronas
    ReLU(),
    CapaDensa(8, 1),   # capa de salida: 8 -> 1
    Sigmoid()
])

print('=== RESOLVIENDO XOR CON MLP ===')
print(f'Arquitectura: 2 entradas -> 8 ocultas (ReLU) -> 1 salida (Sigmoid)')
```

```
print(f'Parámetros: {2*8+8 + 8*1+1} pesos + sesgos\n')
print('Entrenamiento:')
historico = red.entrenar(X, y, epocas=5000, lr=0.1)

print('\n=== RESULTADOS FINALES ===')
pred = red.predecir(X).ravel()   # CORRECCION: aplanar (4,1)->(4,) para NumPy 2.x
print(f'{"Entradas":^15} {"Predicción":>12} {"Redondeado":>12} {"Correcto":>10}')
print('-' * 53)
for xi, yi, pi in zip(X, y, pred):
    entradas_str = f'{int(xi[0])} XOR {int(xi[1])}'
    correcto = '✓' if round(float(pi)) == int(yi) else '✗'
    print(f'{entradas_str:^15} {float(pi):12.4f} {round(float(pi)):12d}
{correcto:>10}')

print(f'\nPérdida final: {historico[-1]:.8f}')
```

> Salida del programa:

```
=== RESOLVIENDO XOR CON MLP ===
Arquitectura: 2 entradas -> 8 ocultas (ReLU) -> 1 salida (Sigmoid)
Parámetros: 33 pesos + sesgos

Entrenamiento:
  Época     0: pérdida = 0.329071
  Época   500: pérdida = 0.032847
  Época  1000: pérdida = 0.009912
  Época  1500: pérdida = 0.005003
  Época  2000: pérdida = 0.003151
  Época  2500: pérdida = 0.002231
  Época  3000: pérdida = 0.001696
  Época  3500: pérdida = 0.001352
  Época  4000: pérdida = 0.001115
  Época  4500: pérdida = 0.000943
  Época  4999: pérdida = 0.000814

=== RESULTADOS FINALES ===
    Entradas     Predicción   Redondeado    Correcto
-----------------------------------------------------
    0 XOR 0           0.0433            0          ✓
    0 XOR 1           0.9758            1          ✓
    1 XOR 0           0.9772            1          ✓
    1 XOR 1           0.0168            0          ✓

Pérdida final: 0.00081414
```

La red multicapa resuelve XOR perfectamente — algo que era matemáticamente imposible para una sola neurona. La capa oculta aprendió una representación intermedia del problema que lo hace linealmente separable para la última capa.

¿Qué representación ha aprendido? Se ve muy bien en una red hecha a mano, con solo dos neuronas ocultas y activación escalón (1 si z > 0; 0 si no). La primera, $h_1 = \text{escalón}(x_1 + x_2 - 0.5)$, es una puerta OR: se enciende si alguna entrada vale 1. La segunda, $h_2 = \text{escalón}(x_1 + x_2 - 1.5)$, es una AND: solo se enciende si valen 1 las dos. La salida, $\text{escalón}(h_1 - h_2 - 0.5)$, es «OR pero no AND», que es exactamente XOR:

📄 Ejemplo — XOR con pesos elegidos a mano

```
import numpy as np

escalon = lambda z: (z > 0).astype(int)        # 1 si z > 0; 0 si no

X  = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
W1 = np.array([[1, 1],                         # columna 0: h1 = OR
               [1, 1]])                        # columna 1: h2 = AND
b1 = np.array([-0.5, -1.5])
H  = escalon(X @ W1 + b1)                      # capa oculta, forma (4, 2)
y  = escalon(H @ np.array([1, -1]) - 0.5)      # salida: h1 y NO h2

print(' x1 x2  ->  h1 h2  ->  XOR')
for x, h, s in zip(X, H, y):
    print(f'  {x[0]}  {x[1]}  ->   {h[0]}  {h[1]}  ->   {s}')
```

> Salida del programa:

```
 x1 x2  ->  h1 h2  ->  XOR
  0  0  ->   0  0  ->   0
  0  1  ->   1  0  ->   1
  1  0  ->   1  0  ->   1
```

```
  1  1  ->  1  1  ->  0
```

Fíjate en la columna del medio. En el espacio de entrada, los puntos (0, 1) y (1, 0), que deben dar 1, están en una diagonal, y (0, 0) y (1, 1), que deben dar 0, en la otra: ninguna recta los separa. En el espacio de la capa oculta, (h_1, h_2), los cuatro puntos se convierten en tres, (0, 0), (1, 0) y (1, 1), y los dos que deben dar 1 caen en el mismo, (1, 0). Ahora una recta, $h_1 - h_2 = 0.5$, sí los separa. Eso es lo que significa que la capa oculta hace el problema linealmente separable. El Programa 4 aprende algo parecido por sí solo, con ReLU y sigmoide en lugar de escalones, y repartido entre sus 8 neuronas ocultas.

> 💡 Con solo 33 parámetros aprendibles (la red 2→8→1 del Programa 4: 2·8 + 8 + 8·1 + 1), nuestra MLP resuelve XOR. Un LLM comercial tiene miles de millones de parámetros, pero el principio es exactamente el mismo: capas de transformaciones lineales intercaladas con no linealidades, entrenadas con gradiente descendente.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - Una sola neurona es un clasificador lineal: no puede resolver problemas no linealmente separables como XOR.
> - Una capa oculta con suficientes neuronas y no-linealidad convierte al MLP en un aproximador universal.
> - Backpropagation se generaliza naturalmente a redes multicapa: el error fluye hacia atrás capa por capa.

✏️ EJERCICIOS — Capítulo 6 — Red Neuronal Multicapa

Antes de avanzar, pon a prueba lo aprendido sobre redes multicapa. Intenta cada ejercicio por tu cuenta y consulta la solución solo al final, para contrastarla con tu propio razonamiento.

Ejercicio 6.1 Resolver XNOR con el MLP

XNOR es el inverso de XOR: devuelve 1 cuando ambas entradas son iguales (ambas 0 o ambas 1). Tabla: (0, 0)->1, (0, 1)->0, (1, 0)->0, (1, 1)->1. Usa la clase RedNeuronal del Programa 3 para resolverlo. ¿Necesitas más o menos neuronas ocultas que para XOR?

> 💡 💡 Pista: XNOR tampoco es linealmente separable (igual que XOR), así que necesitas al menos una capa oculta. La misma arquitectura 2->8->1 funciona bien. Compara los pesos finales: ¿tienen alguna relación con los de XOR?

✅ Solución:

📄 ejercicio_6_1.py

```
import numpy as np
from programa_3_mlp import CapaDensa, ReLU, Sigmoid, RedNeuronal   # Programa 3

np.random.seed(0)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y_xor  = np.array([0, 1, 1, 0], dtype=float)  # XOR
y_xnor = np.array([1, 0, 0, 1], dtype=float)  # XNOR = NOT XOR

for nombre, y in [('XOR', y_xor), ('XNOR', y_xnor)]:
    np.random.seed(42)
    red = RedNeuronal([
        CapaDensa(2, 8), ReLU(),
        CapaDensa(8, 1), Sigmoid()
    ])
    hist = red.entrenar(X, y, epocas=5000, lr=0.1, verbose=False)
    pred = red.predecir(X).ravel()   # CORRECCION: aplanar para NumPy 2.x
    correcto = sum(round(float(p))==int(e) for p,e in zip(pred,y))
    print(f'{nombre}: pérdida={hist[-1]:.5f}  precision={correcto}/4  ', end='')
    preds_str = str([round(float(p)) for p in pred])
    print(f'preds={preds_str}')

print()
print('Observación: XNOR = NOT XOR.')
print('La misma arquitectura resuelve ambos porque ambos son')
print('no linealmente separables y tienen la misma complejidad.')
```

> ℹ️ Matemáticamente, XNOR(a,b) = NOT(XOR(a,b)). Si tuviéramos los pesos de XOR, podríamos obtener XNOR invirtiendo el signo de los pesos de la capa de salida y ajustando el sesgo. El MLP los aprende independientemente, pero la estructura subyacente es la misma.

Ejercicio 6.2 Contando parámetros

Escribe una función que reciba una lista de enteros representando la arquitectura de una red (por ejemplo [2, 8, 4, 1] significa 2 entradas, capa oculta de 8, capa oculta de 4, 1 salida) y devuelva el número total de parámetros (pesos + sesgos).

> 💡 💡 Pista: Entre cada par de capas adyacentes hay n_ent × n_sal pesos y n_sal sesgos. Itera por pares consecutivos de la lista.

✅ Solución:

📄 ejercicio_6_2.py

```
def contar_params(arquitectura):
    """Cuenta pesos + sesgos de una red densa."""
    total = 0
```

```
    for i in range(len(arquitectura) - 1):
        n_ent = arquitectura[i]
        n_sal = arquitectura[i+1]
        pesos  = n_ent * n_sal
        sesgos = n_sal
        total += pesos + sesgos
    return total

# Verificar con casos conocidos
casos = {
    'Una neurona (1 entrada)': [1, 1],
    'Neurona AND (2 entradas)': [2, 1],
    'MLP XOR (2-8-1)':          [2, 8, 1],
    'MLP mediano (10-64-32-1)':[10, 64, 32, 1],
    'FFN de un bloque de GPT-2 (768-3072-768)': [768, 3072, 768],
}

print(f"{'Arquitectura':>40}  {'Parámetros':>12}")
print('-' * 56)
for nombre, arq in casos.items():
    print(f'{nombre:>40}  {contar_params(arq):>12,}')

# Cálculo manual para verificar [2, 8, 1]:
# Capa 1: 2*8 + 8 = 16 + 8 = 24
# Capa 2: 8*1 + 1 =  8 + 1 =  9
# Total: 24 + 9 = 33, que coincide con el Programa 4 (8 ocultas): 2*8+8 + 8*1+1 = 33
```

Ejercicio 6.3 ¿Cuántas neuronas ocultas son suficientes para XOR?

Experimenta con distintos números de neuronas ocultas: 1, 2, 3, 4 y 8. Para cada configuración, entrena 10 veces con distintas semillas y calcula el porcentaje de veces que converge correctamente. ¿Cuál es el número mínimo de neuronas para resolver XOR de forma fiable?

> 💡 💡 Pista: Una sola neurona oculta nunca puede resolver XOR. Con 2 neuronas ocultas la solución existe, pero con ReLU la inicialización aleatoria decide a menudo si se encuentra: una neurona que empieza «apagada» para las cuatro entradas no recibe gradiente y no se recupera. Por eso conviene usar varias semillas.

✅ Solución:

📄 **ejercicio_6_3.py**

```
import numpy as np
from programa_3_mlp import CapaDensa, ReLU, Sigmoid, RedNeuronal   # Programa 3

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([0, 1, 1, 0], dtype=float)

print('¿Cuántas neuronas ocultas necesita XOR?\n')
print(f"{'N ocultas':>12}  {'Éxitos/10':>10}  {'Pérdida media':>15}")
print('-' * 42)

for n_ocultas in [1, 2, 3, 4, 8]:
    exitos = 0
    perdidas = []
    for semilla in range(10):
        np.random.seed(semilla)
        red = RedNeuronal([
            CapaDensa(2, n_ocultas), ReLU(),
            CapaDensa(n_ocultas, 1), Sigmoid()
        ])
        hist = red.entrenar(X, y, epocas=3000, lr=0.2, verbose=False)
        pred = red.predecir(X).ravel()   # CORRECCION: aplanar para NumPy 2.x
        ok = all(round(float(p))==int(e) for p,e in zip(pred,y))
        if ok: exitos += 1
        perdidas.append(hist[-1])
    print(f'{n_ocultas:>12}  {exitos:>10}/10  {np.mean(perdidas):>15.5f}')
```

Capítulo 7: Sobreajuste y Regularización (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Diagnosticar overfitting y underfitting leyendo curvas de aprendizaje.
> ✓ Dividir correctamente los datos en train/validación/test sin fuga de información.
> ✓ Implementar L2, Dropout y Early Stopping desde cero.
> ✓ Elegir la combinación de técnicas de regularización adecuada para cada escenario.

7.1 Aprender vs. Memorizar

Imagina que tienes que preparar un examen de matemáticas y decides estudiar memorizando las soluciones de los 30 ejercicios del libro. El día del examen, si aparece exactamente uno de esos 30 ejercicios, lo resuelves perfectamente. Si aparece uno similar pero ligeramente diferente, fallas. Has memorizado, pero no has aprendido.

Las redes neuronales cometen exactamente el mismo error cuando tienen demasiada capacidad en relación con los datos disponibles. En lugar de extraer el patrón general que explica los datos, aprenden de memoria cada ejemplo del conjunto de entrenamiento, incluyendo el ruido aleatorio que contienen. Esto se llama sobreajuste u overfitting, y es uno de los problemas más frecuentes y más importantes en el aprendizaje automático.

El objetivo de una red neuronal no es rendir bien en los datos de entrenamiento — eso es trivial con suficiente capacidad. El objetivo es generalizar: rendir bien en datos que nunca ha visto. Una red que memoriza tiene error de entrenamiento cercano a cero pero error muy alto en datos nuevos. Una red que aprende tiene errores parecidos en entrenamiento y en datos nuevos.

> ℹ️ Definición precisa: se dice que un modelo tiene overfitting cuando su error en el conjunto de entrenamiento es significativamente menor que su error en datos nuevos (conjunto de validación o test). La diferencia entre ambos errores es la medida del sobreajuste.

> 📌 **Avance**
> En el Capítulo 15 haremos este diagnóstico con nuestro propio Nano-LLM y descubriremos que memoriza: en frases que no ha visto, su pérdida es casi 20 veces la de entrenamiento.

7.2 El espectro completo: underfitting, buen ajuste y overfitting

El overfitting no es el único problema posible. Existe también el underfitting: cuando el modelo es demasiado simple para capturar el patrón real en los datos. Ambos extremos son indeseables, y encontrar el equilibrio correcto es el arte central del Machine Learning.

Visualicemos los tres estados con un ejemplo concreto: queremos que un modelo aprenda la función seno con ruido, usando solo 20 puntos de datos:

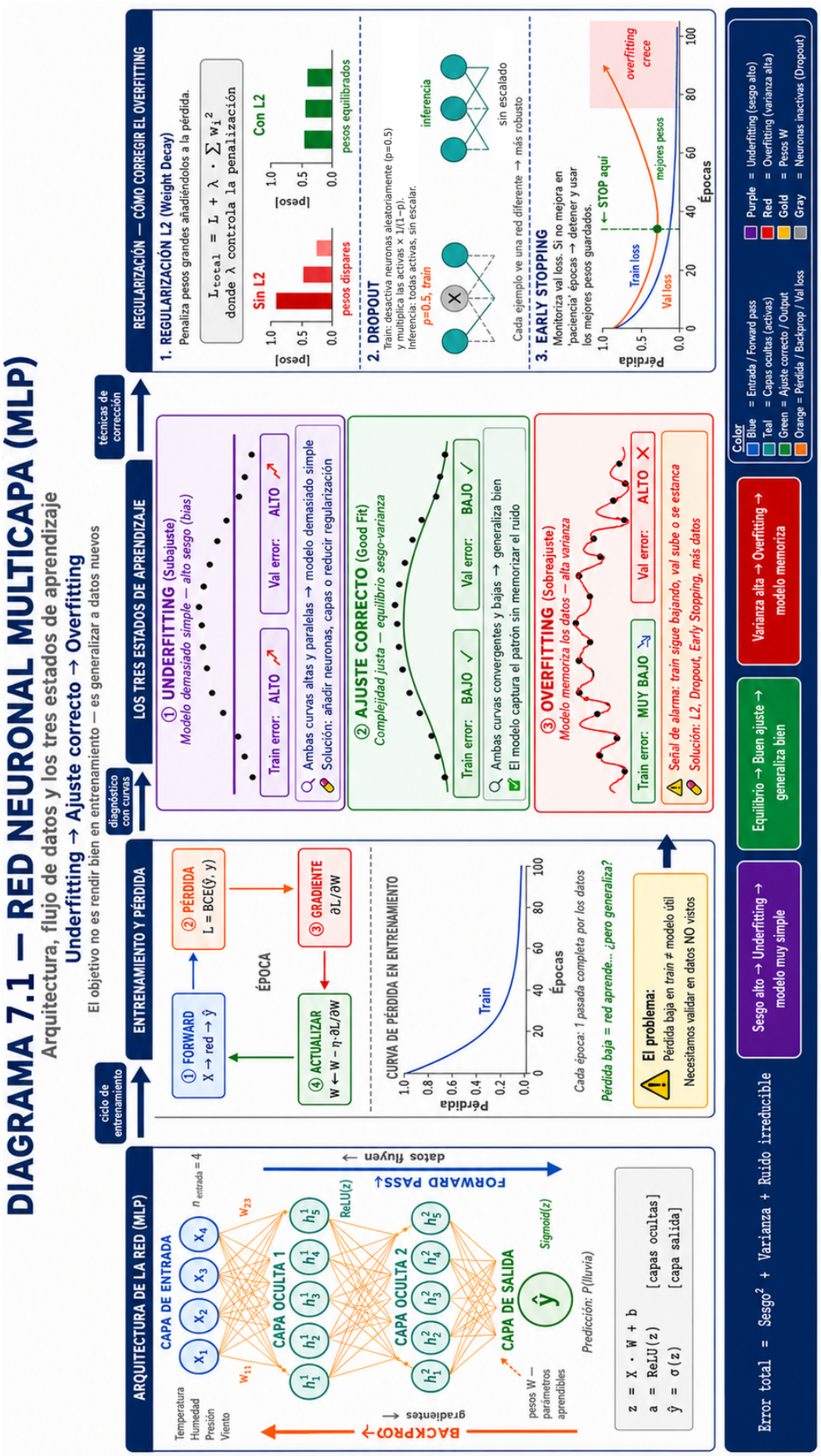


DIAGRAMA 7.1 — RED NEURONAL MULTICAPA (MLP)
Arquitectura, flujo de datos y los tres estados de aprendizaje
Underfitting → Ajuste correcto → Overfitting
El objetivo no es rendir bien en entrenamiento — es generalizar a datos nuevos
ARQUITECTURA DE LA RED (MLP)
CAPA DE ENTRADA
Temperatura
Humedad
Presión
Viento
CAPA OCULTA 1
CAPA OCULTA 2
CAPA DE SALIDA
ReLU(z)
Sigmoid(z)
FORWARD PASS →
← BACKPROP
datos fluyen
gradientes
pesos W — parámetros aprendibles
Predicción: P(lluvia)
z = X · W + b
a = ReLU(z) [capas ocultas]
ŷ = σ(z) [capa salida]
ciclo de entrenamiento
ENTRENAMIENTO Y PÉRDIDA
① FORWARD X → red → ŷ
② PÉRDIDA L = BCE(ŷ, y)
③ GRADIENTE ∂L/∂W
④ ACTUALIZAR W ← W − η·∂L/∂W
ÉPOCA
CURVA DE PÉRDIDA EN ENTRENAMIENTO
Train
Pérdida
Épocas
Cada época: 1 pasada completa por los datos
Pérdida baja = red aprende... ¿pero generaliza?
El problema:
Pérdida baja en train ≠ modelo útil
Necesitamos validar en datos NO vistos
diagnóstico con curvas
LOS TRES ESTADOS DE APRENDIZAJE
① UNDERFITTING (Subajuste)
Modelo demasiado simple — alto sesgo (bias)
Train error: ALTO
Val error: ALTO
Ambas curvas altas y paralelas → modelo demasiado simple
Solución: añadir neuronas, capas o reducir regularización
② AJUSTE CORRECTO (Good Fit)
Complejidad justa — equilibrio sesgo-varianza
Train error: BAJO
Val error: BAJO
Ambas curvas convergentes y bajas → generaliza bien
El modelo captura el patrón sin memorizar el ruido
③ OVERFITTING (Sobreajuste)
Modelo memoriza los datos — alta varianza
Train error: MUY BAJO
Val error: ALTO
Señal de alarma: train sigue bajando, val sube o se estanca
Solución: L2, Dropout, Early Stopping, más datos
técnicas de corrección
REGULARIZACIÓN — CÓMO CORREGIR EL OVERFITTING
1. REGULARIZACIÓN L2 (Weight Decay)
Penaliza pesos grandes añadiéndolos a la pérdida.
L_total = L + λ · Σ w_i²
donde λ controla la penalización
Sin L2
Con L2
peso
pesos dispares
pesos equilibrados
2. DROPOUT
Train: desactiva neuronas aleatoriamente (p=0.5)
y multiplica las activas × 1/(1−p).
Inferencia: todas activas, sin escalar.
p=0.5, train
inferencia
sin escalado
Cada ejemplo ve una red diferente → más robusto
3. EARLY STOPPING
Monitoriza val loss. Si no mejora en
'paciencia' épocas → detener y usar
los mejores pesos guardados.
← STOP aquí
Train loss
Val loss
mejores pesos
overfitting crece
Pérdida
Épocas
Error total = Sesgo² + Varianza + Ruido irreducible
Sesgo alto → Underfitting → modelo muy simple
Equilibrio → Buen ajuste → generaliza bien
Varianza alta → Overfitting → modelo memoriza
Color
Blue = Entrada / Forward pass
Teal = Capas ocultas (activas)
Green = Ajuste correcto / Output
Orange = Pérdida / Backprop / Val loss
Purple = Underfitting (sesgo alto)
Red = Overfitting (varianza alta)
Gold = Pesos W
Gray = Neuronas inactivas (Dropout)

7.3 Las curvas de aprendizaje — el diagnóstico esencial

La herramienta más importante para detectar overfitting es la curva de aprendizaje: un gráfico que muestra la evolución del error de entrenamiento y del error de validación a lo largo de las épocas.

Un modelo que generaliza bien muestra curvas de entrenamiento y validación que convergen hacia valores similares. Un modelo con overfitting muestra una curva de entrenamiento que sigue bajando mientras la curva de validación se estanca o sube. Ese momento de divergencia es exactamente cuando el modelo pasa de aprender a memorizar.

PROGRAMA R-1 — Demostración de Overfitting con curvas de aprendizaje

📄 **programa_R1_overfitting.py**

```
import numpy as np

np.random.seed(42)

# ============================================================
# DATASET: funcion seno con ruido gaussiano
# Usamos POCOS datos de entrenamiento a proposito,
# para que el overfitting emerja facilmente.
# ============================================================

def generar_datos(n, ruido=0.25, semilla=None):
    if semilla: np.random.seed(semilla)
    X = np.sort(np.random.uniform(0, 1, n))
    y = np.sin(2 * np.pi * X) + np.random.normal(0, ruido, n)
    return X.reshape(-1, 1), y

X_tr,  y_tr  = generar_datos(20, semilla=42)   # 20 datos de entrenamiento
X_val, y_val = generar_datos(200, semilla=99)  # 200 datos de validacion

# ============================================================
# MLP para regresion (salida lineal, sin sigmoid en la capa final)
# ============================================================
def relu(z):     return np.maximum(0, z)
def d_relu(z):   return (z > 0).astype(float)

class MLPRegresion:
    def __init__(self, n_oculta, semilla=0):
        np.random.seed(semilla)
        escala = np.sqrt(2.0 / 1)
        self.W1 = np.random.randn(1, n_oculta) * escala
        self.b1 = np.zeros(n_oculta)
        self.W2 = np.random.randn(n_oculta, 1) * 0.1
        self.b2 = np.zeros(1)
        self.m = {'W1': np.zeros_like(self.W1), 'b1': np.zeros_like(self.b1),
                  'W2': np.zeros_like(self.W2), 'b2': np.zeros_like(self.b2)}
        self.v = {k: np.zeros_like(val) for k, val in self.m.items()}
        self.t = 0   # contador de pasos, para la correccion de sesgo de Adam

    def forward(self, X):
        self.X  = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        return self.Z2

    def backward(self, dZ2, lr, beta1=0.9, beta2=0.999, eps=1e-8):
        dW2 = self.A1.T @ dZ2            # dZ2 ya trae el 1/N de la pérdida media
        db2 = dZ2.sum(axis=0)
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * d_relu(self.Z1)
        dW1 = self.X.T @ dZ1
        db1 = dZ1.sum(axis=0)
        self.t += 1
        for nombre, grad in [('W1',dW1),('b1',db1),('W2',dW2),('b2',db2)]:
            self.m[nombre] = beta1*self.m[nombre] + (1-beta1)*grad
            self.v[nombre] = beta2*self.v[nombre] + (1-beta2)*grad**2
            m_hat = self.m[nombre] / (1 - beta1**self.t)
```

```
            v_hat = self.v[nombre] / (1 - beta2**self.t)
            getattr(self, nombre)[...] -= lr * m_hat / (np.sqrt(v_hat) + eps)

    def mse(self, X, y):
        return float(np.mean((self.forward(X) - y.reshape(-1,1))**2))

    def entrenar_epoch(self, X, y, lr):
        pred = self.forward(X)
        loss = float(np.mean((pred - y.reshape(-1,1))**2))
        grad = 2 * (pred - y.reshape(-1,1)) / len(X)
        self.backward(grad, lr)
        return loss


# ============================================================
# ENTRENAR TRES REDES: pequena, mediana y grande
# ============================================================
EPOCAS = 20000   # aumentado de 5000: con Adam y mas tiempo, el sobreajuste emerge con
claridad
LR     = 0.01    # ajustado para Adam (antes 0.005, pensado para SGD)

modelos = [
    ('Modelo simple  (4 neuronas) ',  4),
    ('Modelo adecuado (16 neuronas)', 16),
    ('Modelo grande  (256 neuronas)', 256),
]

resultados = {}
for nombre, n_oc in modelos:
    red = MLPRegresion(n_oc, semilla=42)
    hist_tr, hist_val = [], []
    for ep in range(EPOCAS):
        lt = red.entrenar_epoch(X_tr, y_tr, LR)
        lv = red.mse(X_val, y_val)
        hist_tr.append(lt)
        hist_val.append(lv)
    resultados[nombre] = (hist_tr, hist_val)

# ============================================================
# MOSTRAR CURVAS DE APRENDIZAJE (formato tabla)
# ============================================================
def curva_ascii(hist_tr, hist_val, n_puntos=12):
    """Visualizacion ASCII de la curva de aprendizaje"""
    paso = len(hist_tr) // n_puntos
    print(f"  {'Epoca':>7}  {'Error train':>13}  {'Error val':>11}  Divergencia")
    print('  ' + '-'*55)
    for i in range(n_puntos):
        ep  = i * paso
        lt  = hist_tr[ep]
        lv  = hist_val[ep]
        div = lv / (lt + 1e-10)
        barra_tr = '*' * min(20, int(lt * 40))
        barra_vl = 'o' * min(20, int(lv * 40))
        if div > 2.0:
            señal = f'!!! OVERFITTING (val={div:.1f}x train)'
        elif lt > 0.15:
            señal = 'aun aprendiendo'
        else:
            señal = 'convergido'
        print(f'  {ep:>7}  {lt:>13.5f}  {lv:>11.5f}  {señal}')

print('='*60)
print('CURVAS DE APRENDIZAJE')
print('='*60)

for nombre, (hist_tr, hist_val) in resultados.items():
    print(f'\n{nombre}')
    curva_ascii(hist_tr, hist_val)
    mejor_val  = min(hist_val)
    epoca_mej  = hist_val.index(mejor_val)
    print(f'  -> Error train final:  {hist_tr[-1]:.5f}')
    print(f'  -> Error val final:    {hist_val[-1]:.5f}')
    print(f'  -> Mejor val (epoca {epoca_mej}): {mejor_val:.5f}')
```

```
    ratio = hist_val[-1] / (hist_tr[-1] + 1e-10)
    if ratio > 2.5:
        print(f'  -> DIAGNOSTICO: OVERFITTING grave (val es {ratio:.1f}x train)')
    elif hist_tr[-1] > 0.12:
        print(f'  -> DIAGNOSTICO: UNDERFITTING (error train alto)')
    else:
        print(f'  -> DIAGNOSTICO: Buen ajuste')

  > Salida del programa:
============================================================
CURVAS DE APRENDIZAJE
============================================================

Modelo simple  (4 neuronas)
    Epoca    Error train    Error val  Divergencia
  ------------------------------------------------------
       0        0.61752      0.67340  aun aprendiendo
    1666        0.18143      0.21486  aun aprendiendo
    3332        0.18143      0.21462  aun aprendiendo
    4998        0.18145      0.21472  aun aprendiendo
    6664        0.18142      0.21472  aun aprendiendo
    8330        0.18143      0.21462  aun aprendiendo
    9996        0.18142      0.21484  aun aprendiendo
   11662        0.18143      0.21453  aun aprendiendo
   13328        0.09008      0.10292  convergido
   14994        0.09008      0.10292  convergido
   16660        0.09009      0.10301  convergido
   18326        0.09008      0.10292  convergido
  -> Error train final:  0.09008
  -> Error val final:    0.10292
  -> Mejor val (epoca 13127): 0.10068
  -> DIAGNOSTICO: Buen ajuste

Modelo adecuado (16 neuronas)
    Epoca    Error train    Error val  Divergencia
  ------------------------------------------------------
       0        0.46126      0.46179  aun aprendiendo
    1666        0.14247      0.19840  convergido
    3332        0.14235      0.19800  convergido
    4998        0.14232      0.19816  convergido
    6664        0.14240      0.19828  convergido
    8330        0.14234      0.19826  convergido
    9996        0.14235      0.19808  convergido
   11662        0.14239      0.19827  convergido
   13328        0.14235      0.19839  convergido
   14994        0.14236      0.19855  convergido
   16660        0.14236      0.19770  convergido
   18326        0.14239      0.19809  convergido
  -> Error train final:  0.14240
  -> Error val final:    0.19800
  -> Mejor val (epoca 306): 0.19635
  -> DIAGNOSTICO: UNDERFITTING (error train alto)

Modelo grande  (256 neuronas)
    Epoca    Error train    Error val  Divergencia
  ------------------------------------------------------
       0        0.59756      0.80696  aun aprendiendo
    1666        0.14099      0.19911  convergido
    3332        0.08161      0.13848  convergido
    4998        0.04024      0.08005  convergido
    6664        0.03729      0.08267  !!! OVERFITTING (val=2.2x train)
    8330        0.03626      0.08351  !!! OVERFITTING (val=2.3x train)
    9996        0.03570      0.08377  !!! OVERFITTING (val=2.3x train)
   11662        0.03961      0.09508  !!! OVERFITTING (val=2.4x train)
   13328        0.03493      0.08453  !!! OVERFITTING (val=2.4x train)
   14994        0.03427      0.09049  !!! OVERFITTING (val=2.6x train)
   16660        0.03286      0.08725  !!! OVERFITTING (val=2.7x train)
   18326        0.04032      0.10471  !!! OVERFITTING (val=2.6x train)
  -> Error train final:  0.03226
  -> Error val final:    0.09107
  -> Mejor val (epoca 3655): 0.07874
  -> DIAGNOSTICO: OVERFITTING grave (val es 2.8x train)
```

Los números lo dicen todo: el modelo grande llega a un error de entrenamiento de 0.032, notablemente más bajo que su error de validación (0.091, casi 3 veces mayor). Ha empezado a memorizar peculiaridades de los 20 puntos de entrenamiento que no generalizan a datos nuevos. Fíjate también en que la mejor época de validación es la 3655: a partir de ahí, seguir entrenando solo empeora la generalización, aunque el error de entrenamiento siga bajando.

Los otros dos modelos merecen una mirada, porque no se comportan como dicen sus etiquetas. El de 16 neuronas, el «adecuado», pasa toda la ejecución en una meseta (error de entrenamiento 0.142), y el programa lo diagnostica como underfitting; el de 4 neuronas pasa por una meseta parecida (0.181) hasta cerca de la época 13 000, sale de ella y termina con el mejor resultado de los tres (0.090 en entrenamiento y 0.103 en validación). En esas mesetas la red solo ha aprendido la tendencia general de los puntos, no la forma del seno, y salir de ellas depende de la inicialización: el modelo grande también pasa por una (0.14) hasta cerca de la época 3000. Con 20 puntos y una sola semilla, qué tamaño «gana» es en parte cuestión de suerte. Para comparar arquitecturas en serio hay que repetir con varias semillas, una lección que el Capítulo 15 aplicará a fondo.

> 💡 Regla práctica: cuando el error de entrenamiento sigue bajando pero el error de validación se estanca o sube, el modelo está en overfitting. Ese es el momento de parar el entrenamiento o de añadir regularización.

7.4 El compromiso sesgo-varianza

El overfitting y el underfitting tienen una explicación matemática elegante llamada el compromiso sesgo-varianza (bias-variance tradeoff). Todo modelo comete errores de predicción que se pueden descomponer en tres términos:

📄 Descomposición del error de un modelo

```
Error total = Sesgo^2 + Varianza + Ruido irreducible

Donde:
  Sesgo^2       = error sistematico por suposiciones del modelo
                  Un modelo muy simple tiene sesgo alto:
                  siempre se equivoca en la misma direccion.

  Varianza      = sensibilidad a las fluctuaciones del dataset
                  Un modelo muy complejo tiene varianza alta:
                  cambia drasticamente si cambian unos pocos datos.

  Ruido irred.  = imprecision inherente de los propios datos.
                  Ningún modelo puede eliminar este termino.

  Underfitting  <-> sesgo alto,    varianza baja
  Overfitting   <-> sesgo bajo,    varianza alta
  Buen ajuste   <-> sesgo moderado, varianza moderada (equilibrio)
```

La intuición es la siguiente: un modelo muy simple (pocas neuronas, poco ajuste) comete siempre los mismos errores sistemáticos — tiene sesgo alto. Un modelo muy complejo (muchas neuronas, mucho ajuste) es tan sensible a los datos de entrenamiento que si cambiáramos diez ejemplos del dataset, el modelo cambiaría radicalmente — tiene varianza alta. El objetivo es encontrar la complejidad que minimiza la suma de ambos.

7.5 La división Train / Validación / Test

Antes de hablar de técnicas de regularización, necesitamos el instrumento que nos permite detectar el overfitting: la división correcta de los datos (Diagrama 7.2).

El error más común — y más costoso — en Machine Learning es evaluar el modelo sobre datos que influyeron en su diseño. Si usas el mismo conjunto para entrenar y para evaluar, siempre verás errores bajos, aunque el modelo no generalice nada.

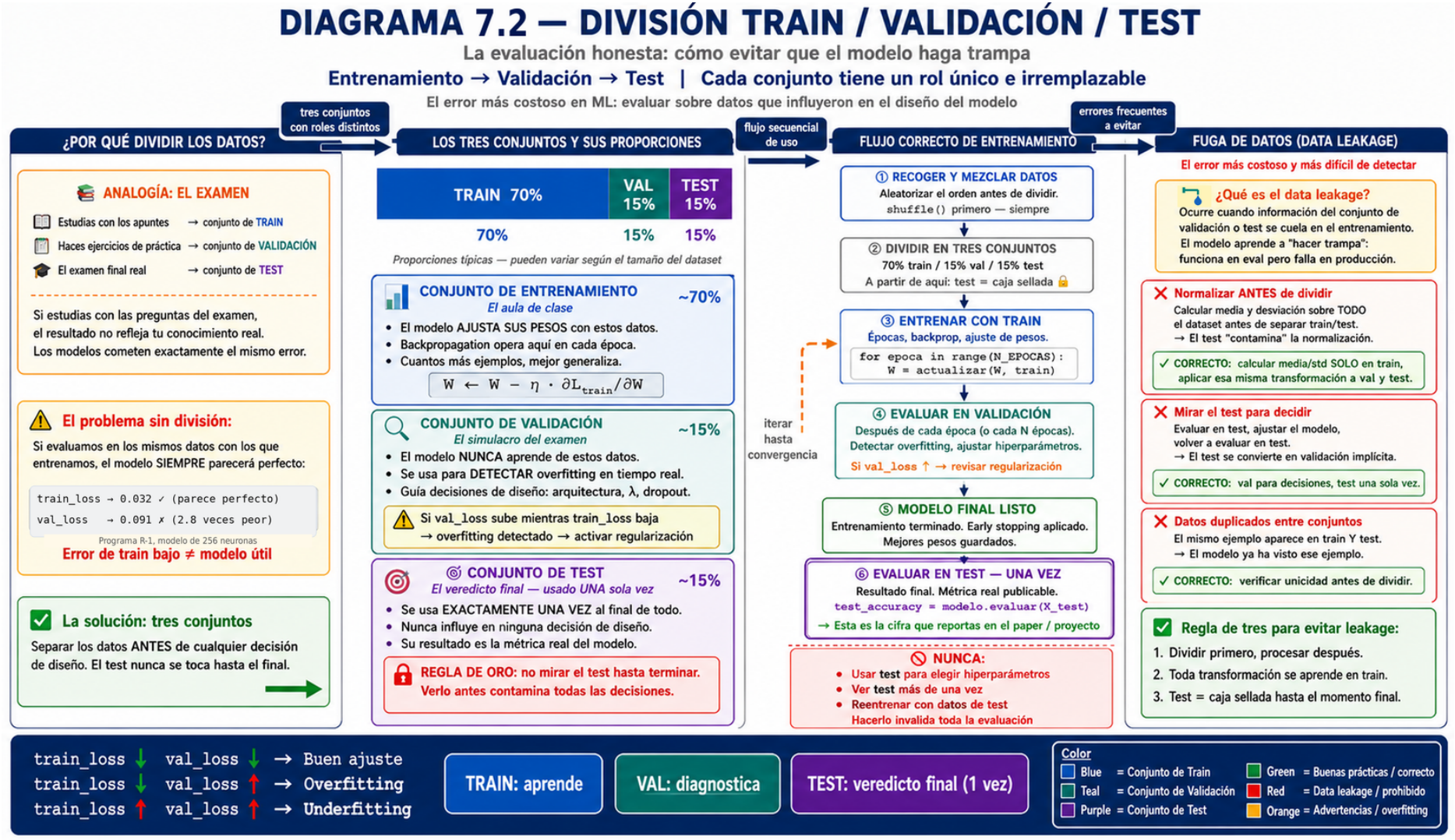


DIAGRAMA 7.2 — DIVISIÓN TRAIN / VALIDACIÓN / TEST
La evaluación honesta: cómo evitar que el modelo haga trampa
Entrenamiento → Validación → Test | Cada conjunto tiene un rol único e irremplazable
El error más costoso en ML: evaluar sobre datos que influyeron en el diseño del modelo
¿POR QUÉ DIVIDIR LOS DATOS?
tres conjuntos con roles distintos
LOS TRES CONJUNTOS Y SUS PROPORCIONES
flujo secuencial de uso
FLUJO CORRECTO DE ENTRENAMIENTO
errores frecuentes a evitar
FUGA DE DATOS (DATA LEAKAGE)
ANALOGÍA: EL EXAMEN
Estudias con los apuntes → conjunto de TRAIN
Haces ejercicios de práctica → conjunto de VALIDACIÓN
El examen final real → conjunto de TEST
Si estudias con las preguntas del examen, el resultado no refleja tu conocimiento real. Los modelos cometen exactamente el mismo error.
El problema sin división:
Si evaluamos en los mismos datos con los que entrenamos, el modelo SIEMPRE parecerá perfecto:
train_loss → 0.032 ✓ (parece perfecto)
val_loss → 0.091 ✗ (2.8 veces peor)
Programa R-1, modelo de 256 neuronas
Error de train bajo ≠ modelo útil
La solución: tres conjuntos
Separar los datos ANTES de cualquier decisión de diseño. El test nunca se toca hasta el final.
TRAIN 70%
VAL 15%
TEST 15%
70%
15%
15%
Proporciones típicas — pueden variar según el tamaño del dataset
CONJUNTO DE ENTRENAMIENTO
El aula de clase
~70%
El modelo AJUSTA SUS PESOS con estos datos.
Backpropagation opera aquí en cada época.
Cuantos más ejemplos, mejor generaliza.
W ← W − η · ∂L_train/∂W
CONJUNTO DE VALIDACIÓN
El simulacro del examen
~15%
El modelo NUNCA aprende de estos datos.
Se usa para DETECTAR overfitting en tiempo real.
Guía decisiones de diseño: arquitectura, λ, dropout.
Si val_loss sube mientras train_loss baja → overfitting detectado → activar regularización
CONJUNTO DE TEST
El veredicto final — usado UNA sola vez
~15%
Se usa EXACTAMENTE UNA VEZ al final de todo.
Nunca influye en ninguna decisión de diseño.
Su resultado es la métrica real del modelo.
REGLA DE ORO: no mirar el test hasta terminar. Verlo antes contamina todas las decisiones.
iterar hasta convergencia
① RECOGER Y MEZCLAR DATOS
Aleatorizar el orden antes de dividir.
shuffle() primero — siempre
② DIVIDIR EN TRES CONJUNTOS
70% train / 15% val / 15% test
A partir de aquí: test = caja sellada
③ ENTRENAR CON TRAIN
Épocas, backprop, ajuste de pesos.
for epoca in range(N_EPOCAS):
W = actualizar(W, train)
④ EVALUAR EN VALIDACIÓN
Después de cada época (o cada N épocas).
Detectar overfitting, ajustar hiperparámetros.
Si val_loss ↑ → revisar regularización
⑤ MODELO FINAL LISTO
Entrenamiento terminado. Early stopping aplicado.
Mejores pesos guardados.
⑥ EVALUAR EN TEST — UNA VEZ
Resultado final. Métrica real publicable.
test_accuracy = modelo.evaluar(X_test)
→ Esta es la cifra que reportas en el paper / proyecto
NUNCA:
Usar test para elegir hiperparámetros
Ver test más de una vez
Reentrenar con datos de test
Hacerlo invalida toda la evaluación
¿Qué es el data leakage?
Ocurre cuando información del conjunto de validación o test se cuela en el entrenamiento. El modelo aprende a "hacer trampa": funciona en eval pero falla en producción.
Normalizar ANTES de dividir
Calcular media y desviación sobre TODO el dataset antes de separar train/test.
→ El test "contamina" la normalización.
✓ CORRECTO: calcular media/std SOLO en train, aplicar esa misma transformación a val y test.
Mirar el test para decidir
Evaluar en test, ajustar el modelo, volver a evaluar en test.
→ El test se convierte en validación implícita.
✓ CORRECTO: val para decisiones, test una sola vez.
Datos duplicados entre conjuntos
El mismo ejemplo aparece en train Y test.
→ El modelo ya ha visto ese ejemplo.
✓ CORRECTO: verificar unicidad antes de dividir.
Regla de tres para evitar leakage:
1. Dividir primero, procesar después.
2. Toda transformación se aprende en train.
3. Test = caja sellada hasta el momento final.
train_loss ↓ val_loss ↓ → Buen ajuste
train_loss ↓ val_loss ↑ → Overfitting
train_loss ↑ val_loss ↑ → Underfitting
TRAIN: aprende
VAL: diagnostica
TEST: veredicto final (1 vez)
Color
Blue = Conjunto de Train
Teal = Conjunto de Validación
Purple = Conjunto de Test
Green = Buenas prácticas / correcto
Red = Data leakage / prohibido
Orange = Advertencias / overfitting

La función de cada conjunto es muy específica:

* Conjunto de entrenamiento: los datos con los que el modelo ajusta sus pesos mediante backpropagation. Cuantos más, mejor.
* Conjunto de validación: datos que el modelo nunca ve durante el entrenamiento. Se usa para detectar overfitting, comparar arquitecturas, ajustar hiperparámetros (learning rate, número de capas, etc.) y decidir cuándo parar de entrenar. Puede usarse muchas veces, aunque con un riesgo: si se elige entre muchas opciones mirando la validación, lo elegido acaba ajustado también a ella y su cifra es optimista. Por eso hace falta el test.
* Conjunto de test: el veredicto final. Se usa exactamente una vez, cuando el modelo está completamente terminado, para obtener una estimación fiable del rendimiento en producción. Usarlo más de una vez introduce un sesgo optimista.

> ℹ️ La fuga de datos (data leakage) ocurre cuando información del conjunto de validación o test se cuela en el entrenamiento. Ejemplos comunes: normalizar usando la media y desviación de todo el dataset (en lugar de solo el train), o seleccionar features basándose en correlaciones con el test. El modelo parece funcionar mejor de lo que realmente hace.

7.6 Regularización L2 — Weight Decay

La regularización L2 es la técnica más simple y más usada para prevenir el overfitting. La idea es añadir a la función de pérdida un término que penaliza los pesos grandes. El modelo se ve forzado a usar pesos pequeños, lo que produce predicciones más suaves y menos propensas al sobreajuste.

Matemáticamente, la función de pérdida pasa de ser simplemente el error de predicción a ser la suma del error más una penalización sobre los pesos:

📄 **La función de pérdida con regularización L2**

```
Sin regularizacion:
  L_total = L_datos
  L_datos  = MSE(predicciones, objetivos)

Con regularizacion L2:
  L_total = L_datos + lambda * sum(W^2)

  Donde lambda (hiperparametro) controla la fuerza:
    lambda = 0    -> sin regularizacion
    lambda = 0.01 -> regularizacion moderada
    lambda = 1.0  -> regularizacion muy fuerte

Efecto en el gradiente (actualizacion de pesos):
  Sin L2: W := W - lr * dL_datos/dW
  Con L2: W := W - lr * (dL_datos/dW + 2*lambda*W)
             = W * (1 - 2*lr*lambda) - lr * dL_datos/dW

  El termino (1 - 2*lr*lambda) < 1 reduce los pesos
  en cada actualizacion: de ahi el nombre 'weight decay'.
```

La regularización L2 tiene un efecto muy intuitivo: fuerza al modelo a distribuir la información entre muchos pesos en lugar de concentrarla en unos pocos. Las soluciones con pesos grandes se penalizan, incluso si producen error de entrenamiento menor. El resultado son modelos más simples y más generalizables.

PROGRAMA R-2 — Regularización L2 y comparativa de regularización

📄 **programa_R2_regularizacion.py**

```
import numpy as np

np.random.seed(42)

# Dataset: mismo que el Programa R-1
def generar(n, ruido=0.25, s=None):
    if s: np.random.seed(s)
    X = np.sort(np.random.uniform(0, 1, n)).reshape(-1,1)
```

```
    y = np.sin(2*np.pi*X.ravel()) + np.random.normal(0, ruido, n)
    return X, y

X_tr, y_tr   = generar(20, s=42)
X_val, y_val = generar(200, s=99)

def relu(z):   return np.maximum(0, z)
def d_relu(z): return (z > 0).astype(float)

# ============================================================
# MLP CON REGULARIZACION L2 OPCIONAL
# El unico cambio respecto al MLP basico: añadimos
# 2 * lambda * W al gradiente en cada actualizacion.
# ============================================================
class MLPConL2:
    def __init__(self, n_oc, lam=0.0):
        np.random.seed(42)
        self.W1  = np.random.randn(1, n_oc) * np.sqrt(2)
        self.b1  = np.zeros(n_oc)
        self.W2  = np.random.randn(n_oc, 1) * 0.1
        self.b2  = np.zeros(1)
        self.m = {'W1': np.zeros_like(self.W1), 'b1': np.zeros_like(self.b1),
                  'W2': np.zeros_like(self.W2), 'b2': np.zeros_like(self.b2)}
        self.v = {k: np.zeros_like(val) for k, val in self.m.items()}
        self.t = 0
        self.lam = lam          # fuerza de la regularizacion L2

    def forward(self, X):
        self.X  = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)
        return self.A1 @ self.W2 + self.b2

    def backward(self, dZ2, lr, beta1=0.9, beta2=0.999, eps=1e-8):
        dW2 = self.A1.T @ dZ2 + 2*self.lam*self.W2  # <-- L2 (dZ2 ya trae el 1/N del
MSE)
        db2 = dZ2.sum(axis=0)
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * d_relu(self.Z1)
        dW1 = self.X.T @ dZ1 + 2*self.lam*self.W1  # <-- L2
        db1 = dZ1.sum(axis=0)
        self.t += 1
        for nombre, grad in [('W1',dW1),('b1',db1),('W2',dW2),('b2',db2)]:
            self.m[nombre] = beta1*self.m[nombre] + (1-beta1)*grad
            self.v[nombre] = beta2*self.v[nombre] + (1-beta2)*grad**2
            m_hat = self.m[nombre] / (1 - beta1**self.t)
            v_hat = self.v[nombre] / (1 - beta2**self.t)
            getattr(self, nombre)[...] -= lr * m_hat / (np.sqrt(v_hat) + eps)

    def train(self, X, y, lr):
        pred  = self.forward(X)
        err   = pred - y.reshape(-1,1)
        loss  = float(np.mean(err**2))
        self.backward(2*err/len(X), lr)
        return loss

    def mse(self, X, y):
        return float(np.mean((self.forward(X)-y.reshape(-1,1))**2))

    def norma_pesos(self):
        return float(np.sum(self.W1**2) + np.sum(self.W2**2))


# Comparamos tres configuraciones del mismo modelo (256 neuronas)
configs = [
    ('Sin regularizacion  (lambda=0)',    0.0),
    ('L2 moderada         (lambda=0.001)',  0.001),
    ('L2 fuerte           (lambda=0.01)',   0.01),
]

print('=== EFECTO DE LA REGULARIZACION L2 ===')
print('Modelo: 256 neuronas ocultas (propenso a overfitting)\n')
```

```
print(f"{'Configuracion':>40}  {'Train':>8}  {'Val':>8}  {'|W|^2':>10}  Diagnostico")
print('-'*90)

for nombre, lam in configs:
    red = MLPConL2(256, lam=lam)
    for _ in range(20000):
        red.train(X_tr, y_tr, lr=0.01)
    err_tr  = red.mse(X_tr, y_tr)
    err_val = red.mse(X_val, y_val)
    norma   = red.norma_pesos()
    ratio   = err_val / (err_tr + 1e-10)
    if ratio > 2.5:
        diag = f'OVERFITTING (val={ratio:.1f}x train)'
    elif err_tr > 0.12:
        diag = 'underfitting'
    else:
        diag = 'buen ajuste'
    print(f'{nombre:>40}  {err_tr:>8.5f}  {err_val:>8.5f}  {norma:>10.1f}  {diag}')

# Analisis de los pesos
print('\nAnálisis de los pesos tras el entrenamiento:')
for nombre, lam in configs:
    red = MLPConL2(256, lam=lam)
    for _ in range(20000):
        red.train(X_tr, y_tr, lr=0.01)
    pesos = np.concatenate([red.W1.ravel(), red.W2.ravel()])
    print(f'  {nombre:>37}: max|w|={np.abs(pesos).max():.3f}  '
          f'mean|w|={np.abs(pesos).mean():.4f}')
```

> Salida del programa:

```
=== EFECTO DE LA REGULARIZACION L2 ===
Modelo: 256 neuronas ocultas (propenso a overfitting)

                           Configuracion     Train       Val       |W|^2  Diagnostico
------------------------------------------------------------------------------------------
          Sin regularizacion  (lambda=0)   0.03237   0.09107       608.5  OVERFITTING
(val=2.8x train)
      L2 moderada         (lambda=0.001)   0.04774   0.08097        32.5  buen ajuste
       L2 fuerte           (lambda=0.01)   0.17898   0.21058         4.2  underfitting

Análisis de los pesos tras el entrenamiento:
          Sin regularizacion  (lambda=0): max|w|=5.340  mean|w|=0.7169
      L2 moderada         (lambda=0.001): max|w|=2.564  mean|w|=0.0559
       L2 fuerte           (lambda=0.01): max|w|=1.020  mean|w|=0.0097
```

El efecto es contundente: sin regularización, la suma de los cuadrados de los pesos ($|W|^2$) llega a 608.5 y el modelo empieza a memorizar detalles del ruido (error de validación casi el triple que el de entrenamiento). Con lambda=0.001 cae a 32.5 y el modelo generaliza mejor: el error de validación baja de 0.091 a 0.081. Con lambda=0.01, la regularización ya es demasiado fuerte: el modelo no puede ajustarse lo suficiente y cae en underfitting, con $|W|^2$ de apenas 4.2. El valor adecuado de lambda depende también de cómo esté escalada la pérdida (aquí, la media del error cuadrático). Encontrar el lambda correcto es parte del proceso de ajuste de hiperparámetros — aquí la diferencia entre un buen resultado y uno malo está en un factor 10.

El programa usa la misma regla de diagnóstico que R-1: si el error de validación supera en más de 2.5 veces al de entrenamiento, hay sobreajuste; si el error de entrenamiento pasa de 0.12, la red no llega a ajustar los datos. Son umbrales prácticos para este problema, no leyes generales.

> ℹ️ L2 vs L1: la regularización L1 penaliza la suma de los valores absolutos de los pesos (sum|W|) en lugar de los cuadrados. Su efecto es distinto: tiende a producir pesos exactamente cero, haciendo el modelo disperso. Es útil cuando se sospecha que muchas features son irrelevantes. En redes profundas, L2 (o weight decay) es mucho más común que L1.

7.7 Dropout — Apagar neuronas para generalizar mejor

El Dropout fue propuesto por Geoffrey Hinton y su equipo en 2012 y rápidamente se convirtió en la técnica de regularización dominante en redes profundas. La idea es sorprendentemente simple: durante cada paso de entrenamiento, cada neurona de una capa se desactiva (su salida se pone a cero) al azar con una probabilidad p, la tasa de dropout: con p = 0.5, en promedio se apaga la mitad de las neuronas de esa capa.

Cada vez que la red procesa un ejemplo durante el entrenamiento, se sortea qué neuronas están activas y cuáles no. El resultado es que la red nunca puede apoyarse en ninguna neurona concreta: tiene que aprender representaciones robustas y redundantes, porque cualquier neurona puede estar ausente.

DIAGRAMA 7.3 — DROPOUT

Apagar neuronas aleatoriamente para generalizar mejor

Entrenamiento: neuronas desactivadas aleatoriamente | Inferencia: todas activas, sin escalar

Propuesto por Geoffrey Hinton *et al.* (2012) — la técnica de regularización más influyente del Deep Learning

¿POR QUÉ DROPOUT?

ANALOGÍA: EL EQUIPO DE FÚTBOL

Un equipo que siempre juega con los mismos titulares aprende a depender de sus estrellas. Si en los entrenos se rotan jugadores aleatoriamente, cada uno aprende a ser útil independientemente.

Las neuronas hacen exactamente lo mismo.

El problema que resuelve:

Sin Dropout, ciertas neuronas se especializan en memorizar patrones específicos del train.
Si una neurona falla → toda la red falla.

→ **Co-adaptación de neuronas**
→ **Overfitting estructural**

La solución Dropout:

En cada paso de entrenamiento:
Cada neurona se desactiva con probabilidad p.
La red aprende a funcionar sin ella.
Resultado: representaciones distribuidas y robustas → mejor generalización.

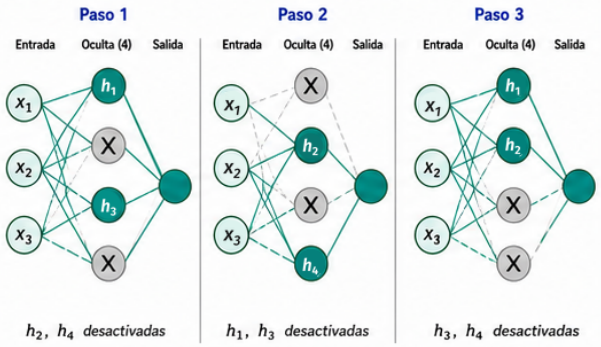
$p = 0.5$	→	50% neuronas desactivadas por ejemplo en cada paso
$p = 0.2$	→	20% desactivadas (*capas cercanas a salida*)
$p = 0.0$	→	sin Dropout (modelos pequeños)

p es un hiperparámetro a ajustar

cómo funciona el dropout →

DURANTE EL ENTRENAMIENTO ($p = 0.5$)

Tres pasos distintos — máscaras aleatorias diferentes



Por qué funciona — la idea del ensemble:

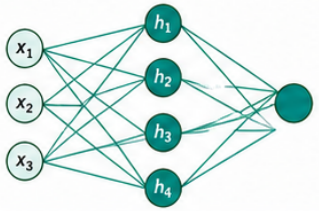
Con $p=0.5$ y N neuronas, el modelo entrena implícitamente 2^N sub-redes distintas.
En inferencia usa el promedio de todas ellas.
Es como un comité de expertos: cada uno entrenado en condiciones distintas.

$N=4$ neuronas → $2^4 = 16$ sub-redes distintas

de entreno a inferencia →

DURANTE LA INFERENCIA

TODAS las neuronas activas



Corrección de escala — CRÍTICO

Durante el entrenamiento, solo una fracción (1−p) de las neuronas estaba activa en promedio.
En inferencia TODAS están activas.
Sin corregir, la salida sería 1/(1−p) veces mayor.

SOLUCIÓN (dropout invertido): escalar AL ENTRENAR

$$h_{train} = h \cdot \text{máscara} / (1 - p)$$

$p = 0.5$: activas × 2 · inferencia: h sin tocar

Así lo hacen nn.Dropout (PyTorch) y el Programa R-3

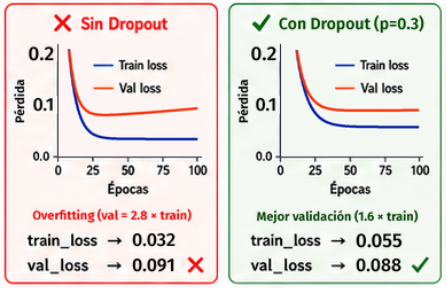
Comportamiento por fase:

	Train	Inferencia
Neuronas activas	$\sim(1-p) \times N$	N (todas)
Escalado	activas × 1/(1−p)	ninguno
Aleatoriedad	sí (máscara)	no (determinista)
Resultado	sub-red aleatoria	promedio implícito

resultados y mejores prácticas →

RESULTADOS Y CONFIGURACIÓN PRÁCTICA

Sin Dropout vs. Con Dropout:



¿Qué valor de p elegir?

$p = 0.5$	Capas ocultas profundas — el valor clásico. Propuesto por Hinton en el paper original.
$p = 0.2–0.3$	Capas cercanas a entrada/salida. Dropout agresivo cerca de la salida puede dificultar el aprendizaje.
$p = 0.0$	Sin Dropout — modelos pequeños o datasets grandes.

Implementación en PyTorch:

```
import torch
import torch.nn as nn
class MiRed(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(4, 8)
        self.drop = nn.Dropout(p=0.5)
        self.fc2 = nn.Linear(8, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.drop(x)   # solo en train
        return self.fc2(x)
```

nn.Dropout se desactiva automáticamente en inferencia al llamar model.eval()

train_loss ↓	val_loss ↓	→	Buen ajuste
train_loss ↓	val_loss ↑	→	Overfitting
train_loss ↑	val_loss ↑	→	Underfitting

Entrenamiento: Dropout ACTIVO

Inferencia: Dropout INACTIVO

Ensemble: 2^N sub-redes implícitas

Color

- Teal = Neurona activa / conexiones activas
- Gray = Neurona inactiva (dropout)
- Red = Overfitting / sin dropout
- Green = Buenas prácticas / generalización
- Orange = Probabilidad p / advertencias
- Purple = Ensemble / sub-redes
- Blue = Inferencia / sin escalar

Por qué funciona el Dropout puede explicarse de varias formas. La más intuitiva: al desactivar aleatoriamente neuronas en cada paso, estamos entrenando efectivamente 2^N redes distintas (donde N es el número de neuronas). Al hacer inferencia, usamos todas las neuronas a la vez, lo que equivale aproximadamente al promedio de todas esas redes. Los ensembles de modelos generalizan mucho mejor que los modelos individuales. Hay un detalle práctico: si al entrenar se apaga una fracción p de las neuronas, en inferencia, con todas encendidas, la capa siguiente recibiría una suma mayor que la que vio durante el entrenamiento. Para compensarlo, al entrenar se multiplican por $1/(1-p)$ las neuronas que siguen activas (con $p = 0.5$, por 2), y en inferencia no hay que tocar nada. Se llama dropout invertido (inverted dropout) y es lo que hace el Programa R-3.

PROGRAMA R-3 — Dropout implementado desde cero

📄 **programa_R3_dropout.py**

```
import numpy as np

np.random.seed(42)

# ============================================================
# DATASET: el mismo que el Programa R-1 (seno con ruido)
# ============================================================

def generar_datos(n, ruido=0.25, semilla=None):
    if semilla:
        np.random.seed(semilla)
    X = np.sort(np.random.uniform(0, 1, n))
    y = np.sin(2 * np.pi * X) + np.random.normal(0, ruido, n)
    return X.reshape(-1, 1), y


X_tr, y_tr = generar_datos(20, semilla=42)      # 20 datos de entrenamiento
X_val, y_val = generar_datos(200, semilla=99)   # 200 datos de validacion

# ============================================================
# MLP para regresion con Dropout (inverted dropout) integrado
# Reutiliza el mismo optimizador Adam que MLPRegresion (Programa R-1)
# para que el modelo "Sin Dropout" sea EXACTAMENTE el mismo modelo
# propenso a overfitting que ya conocemos.
# ============================================================

def relu(z): return np.maximum(0, z)
def d_relu(z): return (z > 0).astype(float)


class MLPConDropout:
    def __init__(self, n_oculta, p_dropout=0.5, semilla=0):
        np.random.seed(semilla)
        escala = np.sqrt(2.0 / 1)
        self.W1 = np.random.randn(1, n_oculta) * escala
        self.b1 = np.zeros(n_oculta)
        self.W2 = np.random.randn(n_oculta, 1) * 0.1
        self.b2 = np.zeros(1)
        self.p = p_dropout  # probabilidad de APAGAR una neurona

        self.m = {'W1': np.zeros_like(self.W1), 'b1': np.zeros_like(self.b1),
                  'W2': np.zeros_like(self.W2), 'b2': np.zeros_like(self.b2)}
        self.v = {k: np.zeros_like(val) for k, val in self.m.items()}
        self.t = 0  # contador de pasos, para la correccion de sesgo de Adam

    def forward(self, X, entrenando=True):
        # aplica dropout SOLO si entrenando=True
        self.X = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)

        if entrenando and self.p > 0:
            # Mascara binaria: True = neurona activa. Escalado 1/(1-p)
            # incluido en la propia mascara ("inverted dropout"): la
            # magnitud esperada de la activacion no cambia con o sin
            # dropout, asi que no hace falta tocar nada en inferencia.
            self.mask = (np.random.rand(*self.A1.shape) > self.p) / (1.0 - self.p)
```

```
            self.D1 = self.A1 * self.mask
        else:
            self.mask = None       # inferencia: todas las neuronas, sin mascara
            self.D1 = self.A1

        self.Z2 = self.D1 @ self.W2 + self.b2
        return self.Z2

    def backward(self, dZ2, lr, beta1=0.9, beta2=0.999, eps=1e-8):
        # el gradiente tambien pasa por la mascara: el gradiente de una
        # neurona apagada es 0 (se multiplica por la misma mascara del forward)
        dW2 = self.D1.T @ dZ2            # dZ2 ya trae el 1/N de la pérdida media
        db2 = dZ2.sum(axis=0)
        dD1 = dZ2 @ self.W2.T
        dA1 = dD1 * self.mask if self.mask is not None else dD1
        dZ1 = dA1 * d_relu(self.Z1)
        dW1 = self.X.T @ dZ1
        db1 = dZ1.sum(axis=0)

        self.t += 1
        for nombre, grad in [('W1', dW1), ('b1', db1), ('W2', dW2), ('b2', db2)]:
            self.m[nombre] = beta1 * self.m[nombre] + (1 - beta1) * grad
            self.v[nombre] = beta2 * self.v[nombre] + (1 - beta2) * grad ** 2
            m_hat = self.m[nombre] / (1 - beta1 ** self.t)
            v_hat = self.v[nombre] / (1 - beta2 ** self.t)
            getattr(self, nombre)[...] -= lr * m_hat / (np.sqrt(v_hat) + eps)

    def entrenar_epoch(self, X, y, lr):
        pred = self.forward(X, entrenando=True)          # dropout activado
        err = pred - y.reshape(-1, 1)
        loss = float(np.mean(err ** 2))
        self.backward(2 * err / len(X), lr)
        return loss

    def evaluar(self, X, y):
        pred = self.forward(X, entrenando=False)          # dropout desactivado
        return float(np.mean((pred - y.reshape(-1, 1)) ** 2))


# ============================================================
# COMPARATIVA: mismo modelo (256 neuronas, propenso a overfitting
# de R-1), una vez sin Dropout y otras con Dropout p=0.3 y p=0.5
# ============================================================

EPOCAS = 20000
LR = 0.01

configs = [
    ('Sin Dropout (p=0.00)', 0.00),
    ('Dropout moderado (p=0.30)', 0.30),
    ('Dropout estandar (p=0.50)', 0.50),
]

print('=== EFECTO DEL DROPOUT (red de 256 neuronas) ===\n')
print(f"{'Configuracion':>28} {'Train':>10} {'Val':>10} {'Ratio':>8}  Diagnostico")
print('-' * 78)

for nombre, p in configs:
    red = MLPConDropout(256, p_dropout=p, semilla=42)
    for _ in range(EPOCAS):
        red.entrenar_epoch(X_tr, y_tr, LR)
    err_tr = red.evaluar(X_tr, y_tr)
    err_val = red.evaluar(X_val, y_val)
    ratio = err_val / (err_tr + 1e-10)
    if ratio > 2.5:
        diag = f'OVERFITTING (val={ratio:.1f}x train)'
    elif err_tr > 0.12:
        diag = 'underfitting'
    else:
        diag = 'buen ajuste'
    print(f'{nombre:>28} {err_tr:>10.5f} {err_val:>10.5f} {ratio:>7.2f}x  {diag}')

# ============================================================
```

```
# Verificacion del escalado invertido: la media de la activacion
# debe ser aproximadamente la misma con y sin dropout.
# ============================================================

print('\n--- Verificacion del escalado de Dropout ---')
np.random.seed(0)
A_prueba = np.ones((1000, 8))  # activaciones simuladas, todas a 1

p = 0.5
mascara = (np.random.rand(*A_prueba.shape) > p) / (1.0 - p)
salida_entrenamiento = A_prueba * mascara
salida_inferencia = A_prueba  # sin mascara ni escalado

print(f'Media activacion (entrenamiento, con mascara):
{salida_entrenamiento.mean():.4f}')
print(f'Media activacion (inferencia, sin mascara):    {salida_inferencia.mean():.4f}')
print('Las medias son similares gracias al escalado 1/(1-p) ("inverted dropout").')
```

> Salida del programa:

```
=== EFECTO DEL DROPOUT (red de 256 neuronas) ===

               Configuracion      Train        Val    Ratio  Diagnostico
-------------------------------------------------------------------------------
        Sin Dropout (p=0.00)    0.03237    0.09107    2.81x  OVERFITTING (val=2.8x train)
   Dropout moderado (p=0.30)    0.05535    0.08764    1.58x  buen ajuste
   Dropout estandar (p=0.50)    0.08372    0.11701    1.40x  buen ajuste

--- Verificacion del escalado de Dropout ---
Media activacion (entrenamiento, con mascara): 0.9855
Media activacion (inferencia, sin mascara):    1.0000
Las medias son similares gracias al escalado 1/(1-p) ("inverted dropout").
```

El Dropout acerca mucho los errores de entrenamiento y validación del mismo modelo de 256 neuronas que en el Programa R-1 memorizaba (ratio val/train de 2.8x): con p=0.3 el ratio baja a 1.58x, y con p=0.5 a 1.40x. Pero lo que importa no es el ratio, sino el error de validación: con p=0.3 mejora un poco (de 0.091 a 0.088) y con p=0.5 empeora (0.117). Con p=0.5 el ratio baja porque sube el error de entrenamiento, no porque el modelo generalice mejor. Que el error de entrenamiento suba es normal: en cada paso la red solo usa una fracción de sus neuronas. Lo que hay que vigilar es la validación.

> ℹ️ En este dataset tan pequeño (20 puntos), Dropout mejora claramente el ratio val/train respecto al modelo sin regularizar, pero no llega a igualar a la L2 moderada (que en el apartado anterior conseguía un error de validación de 0.081, frente al 0.088 de Dropout p=0.3). Esto es coherente con la teoría: Dropout suele necesitar más datos para desplegar todo su potencial, ya que cada paso de entrenamiento solo ve una submuestra de la red.

7.8 Early Stopping — Parar en el momento justo

Hay una solución elegante al overfitting que no requiere ningún cambio en la arquitectura ni en la función de pérdida: simplemente dejar de entrenar en el momento en que el error de validación empieza a subir.

Esta técnica se llama early stopping (parada anticipada). Monitorizamos el error de validación en cada época y guardamos una copia de los pesos del modelo en el momento en que ese error es mínimo. Si el error de validación no mejora durante un número de épocas consecutivas —llamado paciencia— detenemos el entrenamiento y restauramos los pesos del mejor momento.

El early stopping actúa como regularización implícita: limita la cantidad de ajuste que puede hacer el modelo antes de que empiece a memorizar. Es especialmente poderoso combinado con un modelo suficientemente grande: en lugar de elegir la arquitectura perfecta a priori, usamos un modelo con capacidad de sobra y dejamos que el early stopping decida cuándo tiene suficiente.

PROGRAMA R-4 — Early Stopping con paciencia

📄 **programa_R4_early_stopping.py**

```
import numpy as np

np.random.seed(42)

# ============================================================
# CLASE EARLY STOPPING
# Monitoriza el error de validacion y detiene el entrenamiento
# cuando no mejora durante 'paciencia' epocas consecutivas.
# ============================================================
class EarlyStopping:
    def __init__(self, paciencia=50, delta=1e-4):
        """
        paciencia: epocas sin mejora antes de parar
        delta:     mejora minima para considerar que ha habido progreso
        """
        self.paciencia     = paciencia
        self.delta         = delta
        self.mejor_val     = float('inf')
        self.contador      = 0
        self.mejor_epoca   = 0
        self.mejores_pesos = None

    def registrar(self, val_loss, pesos, epoca):
        """
        Llamar en cada epoca con el error de validacion.
        Devuelve True si hay que parar, False si continuar.
        """
        if val_loss < self.mejor_val - self.delta:
            # Ha mejorado: guardar estos pesos
            self.mejor_val   = val_loss
            self.contador    = 0
            self.mejor_epoca = epoca
            # Copia profunda de los pesos
            self.mejores_pesos = {k: v.copy() for k, v in pesos.items()}
            return False  # seguir entrenando
        else:
            # No ha mejorado: incrementar contador
            self.contador += 1
            if self.contador >= self.paciencia:
                return True  # parar
            return False     # todavia tenemos paciencia


# MLP simple para la demo
def relu(z):   return np.maximum(0, z)
def d_relu(z): return (z > 0).astype(float)

class MLP:
    def __init__(self, n_oc, s=42):
        np.random.seed(s)
        self.W1=np.random.randn(1,n_oc)*np.sqrt(2); self.b1=np.zeros(n_oc)
        self.W2=np.random.randn(n_oc,1)*0.1;        self.b2=np.zeros(1)

self.m={'W1':np.zeros_like(self.W1),'b1':np.zeros_like(self.b1),'W2':np.zeros_like(self
.W2),'b2':np.zeros_like(self.b2)}
        self.v={k:np.zeros_like(x) for k,x in self.m.items()}; self.t=0
    def pesos(self): return {'W1':self.W1,'b1':self.b1,'W2':self.W2,'b2':self.b2}
    def cargar(self, d):
self.W1=d['W1'].copy();self.b1=d['b1'].copy();self.W2=d['W2'].copy();self.b2=d['b2'].co
py()
    def forward(self, X): self.X=X; self.Z1=X@self.W1+self.b1; self.A1=relu(self.Z1);
return self.A1@self.W2+self.b2
    def backward(self, g, lr, b1=0.9, b2=0.999, eps=1e-8):
        dW2=self.A1.T@g; db2=g.sum(0); dA1=g@self.W2.T          # g ya trae el 1/N del
MSE
        dZ1=dA1*d_relu(self.Z1); dW1=self.X.T@dZ1; db1=dZ1.sum(0)
        self.t+=1
        for n_,g_ in [('W1',dW1),('b1',db1),('W2',dW2),('b2',db2)]:
            self.m[n_]=b1*self.m[n_]+(1-b1)*g_; self.v[n_]=b2*self.v[n_]+(1-b2)*g_**2
            mh=self.m[n_]/(1-b1**self.t); vh=self.v[n_]/(1-b2**self.t)
```

```
            getattr(self,n_)[...] -= lr*mh/(np.sqrt(vh)+eps)
    def train_ep(self, X, y, lr):
        p=self.forward(X); e=p-y.reshape(-1,1); self.backward(2*e/len(X),lr); return
float(np.mean(e**2))
    def mse(self, X, y): return float(np.mean((self.forward(X)-y.reshape(-1,1))**2))


# Dataset
def gen(n, s): np.random.seed(s); X=np.sort(np.random.uniform(0,1,n)).reshape(-1,1);
return X, np.sin(2*np.pi*X.ravel())+np.random.normal(0,0.25,n)
X_tr, y_tr   = gen(20, 42)
X_val, y_val = gen(200, 99)

# ============================================================
# ENTRENAMIENTO CON EARLY STOPPING
# ============================================================
red = MLP(256)
es  = EarlyStopping(paciencia=1000, delta=1e-5)   # aumentado de 100: evita parar en
una meseta temprana

print('=== ENTRENAMIENTO CON EARLY STOPPING ===')
print(f'Modelo: 256 neuronas | Paciencia: 1000 epocas\n')
print(f"{'Epoca':>7}  {'Train':>10}  {'Val':>10}  Evento")
print('-'*52)

epocas_totales = 30_000   # aumentado de 10_000: con Adam hace falta mas margen
for epoca in range(epocas_totales):
    lt = red.train_ep(X_tr, y_tr, lr=0.01)   # ajustado para Adam (antes 0.005)
    lv = red.mse(X_val, y_val)

    parar = es.registrar(lv, red.pesos(), epoca)

    # Mostrar hitos importantes
    if epoca < 5 or epoca % 500 == 0 or parar:
        evento = ''
        if es.contador == 0: evento = '<-- nuevo mejor'
        if parar:            evento = '<-- PARADA ANTICIPADA'
        print(f'{epoca:>7}  {lt:>10.5f}  {lv:>10.5f}  {evento}')

    if parar:
        break

# Restaurar los mejores pesos
red.cargar(es.mejores_pesos)

print(f'\nResumen:')
print(f'  Epocas entrenadas:  {epoca + 1} de {epocas_totales} posibles')
print(f'  Mejor epoca:        {es.mejor_epoca}')
print(f'  Mejor val (guardado): {es.mejor_val:.5f}')
print(f'  Val con pesos finales s/ ES: podria ser mucho peor')
print(f'  Val restaurando mejores pesos: {red.mse(X_val, y_val):.5f}')
```

> Salida del programa:

```
=== ENTRENAMIENTO CON EARLY STOPPING ===
Modelo: 256 neuronas | Paciencia: 1000 epocas

  Epoca       Train        Val  Evento
----------------------------------------------------
      0     0.59756     0.80696  <-- nuevo mejor
      1     0.82116     0.54969  <-- nuevo mejor
      2     0.55503     0.47869  <-- nuevo mejor
      3     0.44115     0.62274
      4     0.53792     0.58042
    500     0.14159     0.19776
   1000     0.14117     0.19779
   1500     0.14087     0.19711
   2000     0.14000     0.19767
   2500     0.13957     0.19772
   3000     0.13998     0.19886
   3500     0.04529     0.08177
   4000     0.04129     0.07956
   4500     0.04685     0.08104
   4655     0.03972     0.08301  <-- PARADA ANTICIPADA
```

```
Resumen:
  Epocas entrenadas:  4656 de 30000 posibles
  Mejor epoca:        3655
  Mejor val (guardado): 0.07874
  Val con pesos finales s/ ES: podria ser mucho peor
  Val restaurando mejores pesos: 0.07874
```

El entrenamiento se detuvo en la época 4656, aunque se habría permitido hasta 30 000. Los mejores pesos correspondían a la época 3655, cuando el error de validación era 0.079 — justo el punto en el que el modelo, tras una larga meseta, había encontrado por fin un buen ajuste. A partir de ahí, 1000 épocas más sin mejora bastaron para que la parada anticipada actuara y recuperase esos pesos, en vez de dejar que el entrenamiento siguiera memorizando el ruido de los 20 puntos de entrenamiento.

> 💡 Early stopping tiene un efecto secundario valioso: también ahorra tiempo y energía computacional. En modelos grandes que tardan días en entrenarse, detectar el punto óptimo a tiempo puede representar un ahorro de horas o días de cómputo.

7.9 Combinando técnicas — La guía práctica

En la práctica, no se elige una única técnica de regularización: se combinan. Aquí tienes las recomendaciones que usan los profesionales:

```
📄 Guia de regularizacion segun el escenario
ESCENARIO 1: Pocos datos, modelo mediano
  -> L2 (probar lambda de 1e-4 a 1e-2 y elegir con validacion) + Early Stopping
  -> Dividir al menos 20% para validacion

ESCENARIO 2: Dataset grande, red profunda
  -> Dropout (p=0.3 a 0.5) en capas ocultas
  -> Early Stopping siempre
  -> L2 ligero (weight_decay=1e-4 en Adam)

ESCENARIO 3: LLMs y Transformers
  -> Dropout 0.1 en modelos pequenos y en ajuste fino (en el preentrenamiento de los
grandes LLM, a menudo 0)
  -> Weight decay (AdamW con wd=0.01 a 0.1)
  -> Early Stopping o LR scheduling (warmup + decay)
  -> Gradient clipping (clip_grad_norm <= 1.0)

SEÑALES DE DIAGNOSTICO:
  Train << Val          -> overfitting: mas regularizacion
  Train ~= Val (ambos altos) -> underfitting: mas capacidad
  Train ~= Val (ambos bajos) -> excelente generalizacion
  Val sube y baja        -> lr demasiado alta o lote demasiado pequeno

ORDEN DE PRIORIDAD:
  1. Siempre: train/val/test split correcto
  2. Siempre: Early Stopping (gratis y muy efectivo)
  3. Casi siempre: L2 / weight decay
  4. En redes profundas: Dropout
  5. Si tienes pocos datos: Data Augmentation
```

7.10 Data Augmentation — Más datos sin recoger más datos

Una causa raíz del overfitting es la escasez de datos. Una solución directa es conseguir más datos. Cuando eso no es posible, la data augmentation crea versiones modificadas de los datos existentes que son igualmente válidas para el aprendizaje.

En visión por computador, las transformaciones habituales son rotaciones, recortes, volteos horizontales, cambios de brillo o contraste y ruido gaussiano. En NLP, se usan técnicas como sustituir sinónimos, reorganizar oraciones o traducir y volver a traducir (back-translation). En todos los casos, la clave es que la modificación preserve el significado de la etiqueta.

Un caso límite especialmente útil para texto es el ruido de embeddings: durante el entrenamiento, añadir una pequeña perturbación gaussiana a los vectores de embedding. Esto hace que el modelo aprenda a ser robusto ante variaciones semánticas pequeñas, y actúa como regularización efectiva sin requerir más datos.

Con las técnicas de este capítulo (el Diagrama 7.4 las resume) — train/val/test split, L2, Dropout y Early Stopping — tienes el kit de herramientas completo para entrenar modelos que no solo funcionan bien en los datos que has visto, sino en los datos del mundo real. Son las mismas técnicas que usan los equipos que entrenan los LLMs más grandes del mundo, aplicadas a su escala.

DIAGRAMA 7.4 — OVERFITTING Y REGULARIZACIÓN

¿CÓMO EVITAMOS QUE LA RED MEMORICE EN LUGAR DE APRENDER?

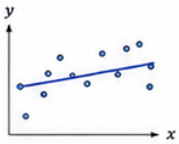
El objetivo es lograr que la red generalice bien a datos nuevos, no solo a los datos de entrenamiento.

1. EL PROBLEMA: OVERFITTING

Cuando la red es demasiado compleja o entrena por demasiado tiempo, puede memorizar el conjunto de entrenamiento, incluyendo el ruido, y fallar con datos nuevos.

SUBAJUSTE (Underfitting)

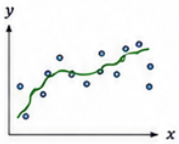
Modelo muy simple
No captura el patrón



Error alto en entrenamiento y en prueba

AJUSTE ADECUADO (Good fit)

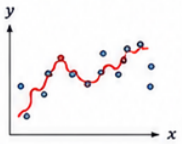
Modelo con la complejidad justa



Bajo error en entrenamiento y en prueba

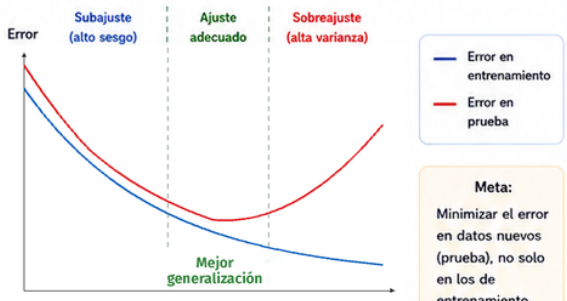
SOBREAJUSTE (Overfitting)

Modelo demasiado complejo
Memoriza el ruido



Error bajo en entrenamiento pero alto en prueba

2. ERROR EN ENTRENAMIENTO VS. PRUEBA

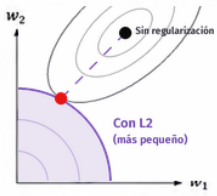


Meta:
Minimizar el error en datos nuevos (prueba), no solo en los de entrenamiento.

3. TÉCNICAS DE REGULARIZACIÓN MÁS UTILIZADAS

1. REGULARIZACIÓN L2 (Weight Decay)

Penaliza pesos grandes añadiendo el cuadrado de los pesos a la pérdida. Favorece modelos más simples.



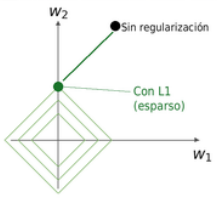
Nueva función de pérdida:

$$L_{total} = L + \lambda \sum_i w_i^2$$

λ : hiperparámetro que controla la fuerza de la regularización.

2. REGULARIZACIÓN L1 (Lasso)

Penaliza el valor absoluto de los pesos. Tiende a producir pesos exactamente iguales a cero (esparsidad).



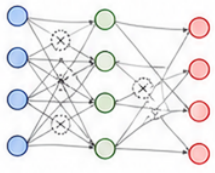
Nueva función de pérdida:

$$L_{total} = L + \lambda \sum_i |w_i|$$

Útil para selección de características y modelos interpretables.

3. DROPOUT

Durante el entrenamiento, desactiva aleatoriamente neuronas (y sus conexiones) para que la red no dependa de caminos específicos.



Reduce co-adaptación de neuronas y mejora la generalización.

4. EARLY STOPPING

Detener el entrenamiento cuando el error en el conjunto de validación deja de mejorar.

Evita que la red continúe ajustándose al ruido del entrenamiento.

5. DATA AUGMENTATION

Aumentar artificialmente el conjunto de entrenamiento aplicando transformaciones realistas a los datos.

La red ve más variedad y aprende patrones más robustos.

4. ¿CÓMO ELEGIR LA COMPLEJIDAD DEL MODELO?

Empieza simple.
Aumenta la complejidad solo si es necesario.

Usa validación cruzada para estimar el rendimiento real del modelo.

Observa la brecha entre entrenamiento y prueba:

Situación	Error entrenamiento	Error prueba	Diagnóstico	Qué hacer
	Alto	Alto	Subajuste	Aumentar complejidad del modelo
	Bajo	Bajo	Ajuste adecuado	Mantener
	Bajo	Alto	Sobreajuste	Aplicar regularización / Más datos / Menos épocas / Simplicidad

5. INTUICIÓN: SESGO–VARIANZA

El error total se puede descomponer en:

$$\text{Error total} = Sesgo^2 + Varianza + \text{Ruido irreducible (ruido de los datos)}$$

Sesgo (bias):
Error por suposiciones demasiado simples.
Alto sesgo → subajuste.

Varianza (variance):
Sensibilidad a pequeñas variaciones en los datos.
Alta varianza → sobreajuste.

Ruido irreducible:
Error debido al ruido inherente en los datos.
No se puede eliminar.

Modelo muy simple (alto sesgo) — Complejidad adecuada (mejor generalización) — Modelo muy complejo (alta varianza)

6. RECOMENDACIONES PRÁCTICAS

- **Más datos** — Mejoran la generalización de cualquier modelo.
- **Mantén el modelo lo más simple posible** que aún funcione bien.
- **Usa regularización** (L2, Dropout, etc.) desde el inicio.
- **Monitorea con un conjunto de validación** y aplica early stopping.
- **Visualiza curvas de aprendizaje** para entender el modelo.
- **Experimenta, mide y repite:** esa es la mejor estrategia.

MENSAJE CLAVE: El objetivo no es memorizar los datos de entrenamiento, sino aprender patrones que generalicen a datos nuevos y desconocidos.

EN UNA FRASE: La regularización es lo que convierte a una red neuronal de buena en excelente.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- Overfitting ocurre cuando el modelo memoriza en lugar de generalizar: error de train bajo, error de validación alto.
- L2, Dropout y Early Stopping son las tres técnicas de regularización más usadas, cada una con un mecanismo distinto.
- La división train/validación/test, sin fuga de información, es la base de cualquier evaluación rigurosa.

✏️ EJERCICIOS — Capítulo 7 — Sobreajuste y Regularización

Estos ejercicios te ayudarán a reconocer el sobreajuste cuando aparezca en tus propios modelos. Resuélvelos antes de seguir y compara tu solución con la propuesta para afinar tu criterio.

Ejercicio 7.1 Curva de complejidad vs. error

Entrena el MLP del Programa R-1 con 1, 2, 4, 8, 16, 32, 64, 128 y 256 neuronas ocultas. Para cada tamaño, registra el error de entrenamiento y de validación final. Dibuja (en texto) la 'curva en U' del error de validación característica del compromiso sesgo-varianza.

> 💡 💡 Pista: En teoría, el error de entrenamiento baja al añadir neuronas, y el de validación baja primero y luego vuelve a subir: el mínimo de esa curva en U marca dónde empieza el overfitting. Con 20 puntos y una sola semilla, la curva real tiene escalones (algunas redes se quedan en una meseta), así que no te extrañe que no sea monótona.

✅ **Solución:**

📄 ejercicio_7_1.py

```
import numpy as np
fuente = open('programa_R1_overfitting.py', encoding='utf-8').read()
exec(fuente.split('# ENTRENAR')[0])   # la clase MLPRegresion del Programa R-1

def gen(n, s): np.random.seed(s); X=np.sort(np.random.uniform(0,1,n)).reshape(-1,1);
return X, np.sin(2*np.pi*X.ravel())+np.random.normal(0,0.25,n)
X_tr, y_tr   = gen(20, 42)
X_val, y_val = gen(200, 99)

tamanos = [1, 2, 4, 8, 16, 32, 64, 128, 256]

print('=== CURVA DE COMPLEJIDAD (sesgo-varianza) ===\n')
print(f"{'Neuronas':>10}  {'Err. Train':>12}  {'Err. Val':>10}  Barra Val")
print('-' * 55)

resultados = []
for n_oc in tamanos:
    red = MLPRegresion(n_oc, semilla=42)
    for _ in range(20000):   # aumentado de 3000: con Adam hace falta mas tiempo
        red.entrenar_epoch(X_tr, y_tr, lr=0.01)
    err_tr  = red.mse(X_tr, y_tr)
    err_val = red.mse(X_val, y_val)
    resultados.append((n_oc, err_tr, err_val))
    barra = '#' * min(40, int(err_val * 80))
    print(f'{n_oc:>10}  {err_tr:>12.5f}  {err_val:>10.5f}  {barra}')

mejor = min(resultados, key=lambda r: r[2])
print(f'\nMejor configuracion: {mejor[0]} neuronas (err_val={mejor[2]:.5f})')
print('Antes de ese punto: underfitting. Despues: overfitting creciente.')
```

Ejercicio 7.2 El efecto del tamaño del dataset en el overfitting

Con una red fija de 256 neuronas (la más propensa a overfitting), entrena con conjuntos de entrenamiento de tamaño 10, 20, 50, 100 y 500 ejemplos. Muestra cómo el cociente error_val / error_train disminuye a medida que hay más datos, incluso sin cambiar la arquitectura.

> 💡 💡 Pista: Más datos dificultan la memorización: con 500 ejemplos, una red de 256 neuronas ya no tiene 'capacidad de sobra' para memorizar cada punto individualmente, así que se ve forzada a aprender el patrón general.

✅ **Solución:**

📄 **ejercicio_7_2.py**

```
import numpy as np
fuente = open('programa_R1_overfitting.py', encoding='utf-8').read()
exec(fuente.split('# ENTRENAR')[0])   # la clase MLPRegresion del Programa R-1

def gen(n, s): np.random.seed(s); X=np.sort(np.random.uniform(0,1,n)).reshape(-1,1);
return X, np.sin(2*np.pi*X.ravel())+np.random.normal(0,0.25,n)
X_val, y_val = gen(200, 99)

tamanos_dataset = [10, 20, 50, 100, 500]

print('=== EFECTO DEL TAMAÑO DEL DATASET (red fija: 256 neuronas) ===\n')
print(f"{'N ejemplos':>11}  {'Err Train':>10}  {'Err Val':>10}  {'Ratio
Val/Train':>16}")
print('-' * 55)

for n in tamanos_dataset:
    X_tr, y_tr = gen(n, 42)
    red = MLPRegresion(256, semilla=42)
    epocas = max(20000, 400000 // n)  # mas epocas si hay menos datos; suelo alto para
Adam
    for _ in range(epocas):
        red.entrenar_epoch(X_tr, y_tr, lr=0.01)
    err_tr  = red.mse(X_tr, y_tr)
    err_val = red.mse(X_val, y_val)
    ratio   = err_val / (err_tr + 1e-10)
    print(f'{n:>11}  {err_tr:>10.5f}  {err_val:>10.5f}  {ratio:>16.1f}x')

print()
print('Con pocos datos (n=10), la misma red memoriza perfectamente')
print('pero generaliza fatal. Con muchos datos (n=500), ya no puede')
print('memorizar cada punto y se ve forzada a generalizar mejor.')
```

ℹ️ Este ejercicio ilustra por qué los LLMs modernos, a pesar de tener miles de millones de parámetros, generalizan razonablemente bien: están entrenados con billones de tokens, una cantidad de datos que hace la memorización completa virtualmente imposible.

Ejercicio 7.3 Combinar Dropout y Early Stopping

Entrena la red de 256 neuronas usando Dropout (p=0.3) Y Early Stopping (paciencia=1000, como en el Programa R-4, restaurando al final los mejores pesos) simultáneamente. Compara el resultado con usar cada técnica por separado. ¿La combinación es mejor que cualquiera de las dos solas?

💡 💡 Pista: Dropout añade ruido al entrenamiento, lo que puede hacer que la curva de validación sea más ruidosa. Considera usar un delta de Early Stopping algo mayor (ej. 1e-4) para no parar prematuramente por ese ruido.

✅ **Solución:**

📄 **ejercicio_7_3.py**

```
import numpy as np
fuente = open('programa_R3_dropout.py', encoding='utf-8').read()
exec(fuente.split('# COMPARATIVA')[0])   # la clase MLPConDropout del Programa R-3

def gen(n, s): np.random.seed(s); X=np.sort(np.random.uniform(0,1,n)).reshape(-1,1);
return X, np.sin(2*np.pi*X.ravel())+np.random.normal(0,0.25,n)
X_tr, y_tr   = gen(20, 42)
X_val, y_val = gen(200, 99)

configs = [
    ('Solo Dropout (p=0.3)',        0.3,  None),
    ('Solo Early Stopping',         0.0,  1000),   # paciencia como en R-4: no parar en
una meseta
    ('Dropout + Early Stopping',    0.3,  1000),
]

print('=== COMBINANDO DROPOUT Y EARLY STOPPING ===\n')
```

```
print(f"{'Configuracion':>28}  {'Err Val Final':>14}  {'Mejor Err Val':>14}  {'Epoca
parada':>13}")
resumen = []
print('-' * 75)

for nombre, p_drop, paciencia in configs:
    red = MLPConDropout(256, p_dropout=p_drop)
    mejor_val = float('inf')
    contador  = 0
    epoca_final = 5000
    mejores = None
    for ep in range(5000):
        red.entrenar_epoch(X_tr, y_tr, lr=0.01)
        err_val = red.evaluar(X_val, y_val)
        if paciencia is not None:
            if err_val < mejor_val - 1e-4:
                mejor_val = err_val; contador = 0
                mejores = [x.copy() for x in (red.W1, red.b1, red.W2, red.b2)]   #
guardar los mejores pesos
            else:
                contador += 1
                if contador >= paciencia:
                    epoca_final = ep; break
        else:
            mejor_val = min(mejor_val, err_val)
    if mejores is not None:                        # early stopping: restaurar los
mejores pesos
        red.W1, red.b1, red.W2, red.b2 = mejores
    err_final = red.evaluar(X_val, y_val)
    resumen.append((err_final, nombre))
    print(f'{nombre:>28}  {err_final:>14.5f}  {mejor_val:>14.5f}  {epoca_final:>13}')

print()
print(f'Mejor en esta ejecucion: {min(resumen)[1]} (err_val={min(resumen)[0]:.5f}).')
print('Que la combinacion gane depende del problema y de la semilla: con 20 puntos')
print('hay que repetir con varias semillas antes de sacar conclusiones.')
```

PARTE II

Del texto al Transformer

Cómo se convierte el texto en números, el tokenizador BPE, las redes recurrentes y sus límites, el mecanismo de atención y el bloque Transformer. Todo lo de la Parte I sigue valiendo: las mismas neuronas, la misma retropropagación y los mismos optimizadores, ahora aplicados a secuencias.

Capítulo 8: Del Número a la Palabra — Texto como Vectores (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar por qué el texto debe convertirse en números antes de procesarlo.
> ✓ Diferenciar tokenización a nivel carácter, subpalabra y palabra.
> ✓ Implementar un tokenizador y una tabla de embeddings desde cero.
> ✓ Calcular y razonar sobre la similitud coseno entre vectores de embedding.

8.1 El problema fundamental del lenguaje

Nuestra red neuronal trabaja con números. El texto, por otro lado, son secuencias de caracteres y palabras. Para que una red pueda procesar lenguaje, necesitamos convertir el texto en números de una manera que preserve el significado.

Este proceso se llama codificación, y lo ideal es que cumpla dos requisitos: que palabras o caracteres similares en significado acaben teniendo representaciones matemáticamente parecidas (lo conseguirán los embeddings de 8.4, no la simple numeración), y que el sistema pueda manejar vocabularios de tamaño variable.

8.2 Tokenización: dividir el texto en piezas

El primer paso es dividir el texto en unidades mínimas llamadas tokens. Hay distintas estrategias:

* Nivel de carácter: cada carácter es un token ('h', 'o', 'l', 'a'). Vocabulario pequeño (~100 tokens), pero secuencias muy largas.
* Nivel de palabra: cada palabra es un token. Vocabulario enorme (>100 000 palabras) y no maneja palabras desconocidas.
* Nivel subpalabra (BPE, WordPiece): compromiso inteligente. Divide las palabras frecuentes en sus formas base. GPT-2 usa 50 257 tokens de este tipo; GPT-4, unos 100 000, y modelos más recientes, entre 128 000 y 256 000. Es el estándar en LLMs modernos.

Para nuestro Nano-LLM usaremos tokenización a nivel de carácter por su simplicidad. Cada carácter del texto se convierte en un índice entero.

8.3 One-hot encoding: la representación más simple

La codificación one-hot asigna a cada token un vector de ceros con un solo 1 en la posición correspondiente a ese token. Si nuestro vocabulario tiene V tokens, cada token se representa como un vector de tamaño V.

📄 One-hot encoding

```
# Vocabulario de 5 caracteres: a, b, c, d, e
# Índices:                     0  1  2  3  4

# 'a' -> [1, 0, 0, 0, 0]
# 'b' -> [0, 1, 0, 0, 0]
# 'c' -> [0, 0, 1, 0, 0]
# 'd' -> [0, 0, 0, 1, 0]
# 'e' -> [0, 0, 0, 0, 1]

# Problema: si el vocabulario tiene 50 000 tokens,
# cada vector tiene 50 000 dimensiones, casi todas a 0.
# Ineficiente y no captura similitud semántica.
```

8.4 Embeddings: representaciones densas y aprendidas

La solución es usar embeddings: vectores densos de dimensión mucho menor (típicamente 64 a 4096) que se aprenden durante el entrenamiento. Un embedding es simplemente una fila en una tabla de consulta (lookup table) que se indexa por el número del token. Tomar esa fila equivale a multiplicar el vector one-hot del token por la tabla: con 5 tokens y embeddings de dimensión 3, el one-hot [0, 0, 1, 0, 0], de forma (1 × 5), por la tabla, de forma (5 × 3), da exactamente su tercera fila, de forma (1 × 3), porque todos los demás productos valen cero. Por eso la tabla de embeddings es una capa lineal más y se aprende con gradientes como cualquier otro peso; buscar la fila es solo una forma mucho más rápida de hacer esa multiplicación.

La magia de los embeddings es que el modelo aprende a colocar tokens similares en regiones cercanas del espacio vectorial. En un modelo bien entrenado, el desplazamiento que lleva del vector de «hombre» al de «mujer» se parece mucho al que lleva de «rey» a «reina» (lo veremos en 8.5). Este fenómeno emerge naturalmente del entrenamiento, sin que nadie lo programe explícitamente.

¿Cómo se aprenden? Al principio la tabla se rellena con números aleatorios pequeños. Durante el entrenamiento, cada fila es un parámetro más: cuando su token aparece en un ejemplo, recibe un gradiente y se corrige un poco, como cualquier peso. Tokens que aparecen en contextos parecidos reciben correcciones parecidas, y por eso sus vectores acaban cerca.

PROGRAMA 5 — Tokenizador y tabla de embeddings

programa_5_embeddings.py

```
import numpy as np

# ========================================================
# TOKENIZADOR A NIVEL DE CARÁCTER + EMBEDDINGS
# ========================================================

class TokenizadorCaracter:
    """Convierte texto en secuencias de índices enteros"""

    def __init__(self, texto):
        # El vocabulario son los caracteres únicos del texto
        self.vocab = sorted(set(texto))
        self.V = len(self.vocab)
        self.c2i = {c: i for i, c in enumerate(self.vocab)}
        self.i2c = {i: c for i, c in enumerate(self.vocab)}
        print(f'Vocabulario: {self.V} caracteres únicos')

    def codificar(self, texto):
        """Texto -> lista de enteros. OJO: un carácter que no esté en el
        vocabulario se descarta sin avisar (el BPE del Capítulo 9 lo evita)."""
        return [self.c2i[c] for c in texto if c in self.c2i]

    def decodificar(self, indices):
        """Lista de enteros -> texto"""
        return ''.join(self.i2c[i] for i in indices)


class TablaEmbeddings:
    """
    Tabla de vectores densos para cada token.
    Cada token tiene asignado un vector de dimensión 'dim'.
    Los vectores se APRENDEN durante el entrenamiento.
    """

    def __init__(self, V, dim):
        # Inicialización con valores pequeños aleatorios
        self.E = np.random.randn(V, dim) * 0.01
        self.V   = V
        self.dim = dim

    def obtener(self, indices):
```

```
        """
        Lookup: dado un índice (o lista), devuelve su vector.
        Equivalente a: E[indices]
        """
        return self.E[indices]

    def similitud(self, i, j):
        """Coseno entre dos vectores de embedding"""
        u = self.E[i] / (np.linalg.norm(self.E[i]) + 1e-8)
        v = self.E[j] / (np.linalg.norm(self.E[j]) + 1e-8)
        return float(np.dot(u, v))


# -- Demostración ----------------------------------------
texto_ejemplo = 'hola mundo, aprendiendo inteligencia artificial'
tok = TokenizadorCaracter(texto_ejemplo)

# Codificación y decodificación
fragmento = 'hola'
codificado   = tok.codificar(fragmento)
decodificado = tok.decodificar(codificado)
print(f'\nOriginal:    "{fragmento}"')
print(f'Codificado:  {codificado}')
print(f'Decodificado:"{decodificado}"')

# Crear tabla de embeddings (dim=6 para visualización)
np.random.seed(7)
emb = TablaEmbeddings(V=tok.V, dim=6)

# Mostrar embeddings de algunos caracteres
print(f'\nEmbeddings iniciales (antes de entrenar):')
for char in ['h', 'o', 'l', 'a', ' ']:
    idx = tok.c2i[char]
    vec = emb.obtener(idx)
    print(f'  [{repr(char)}] índice={idx:2d}  {vec.round(3)}')

# Nota: después del entrenamiento, caracteres con
# contextos similares tendrán vectores más parecidos
print('\nNota: Los embeddings se aprenden durante el entrenamiento.')
print('Después, caracteres en contextos similares tienen')
print('vectores matemáticamente más cercanos.')
```

> Salida del programa:

```
Vocabulario: 18 caracteres únicos

Original:    "hola"
Codificado:  [8, 13, 10, 2]
Decodificado:"hola"

Embeddings iniciales (antes de entrenar):
  ['h'] índice= 8  [ 0.003 -0.005  0.019  0.002  0.001  0.003]
  ['o'] índice=13  [-0.002 -0.018  0.007  0.009  0.004 -0.009]
  ['l'] índice=10  [-0.004 -0.019 -0.001  0.017 -0.004 -0.009]
  ['a'] índice= 2  [ 0.005 -0.003 -0.002 -0.015  0.006  0.001]
  [' '] índice= 0  [ 0.017 -0.005  0.     0.004 -0.008  0.   ]

Nota: Los embeddings se aprenden durante el entrenamiento.
Después, caracteres en contextos similares tienen
vectores matemáticamente más cercanos.
```

> ℹ️ En GPT-3, cada token tiene un embedding de 12 288 dimensiones. En nuestro Nano-LLM usaremos embeddings de 48 dimensiones. La dimensionalidad controla cuánta información puede codificar cada token sobre su significado y contexto.

8.5 Medir el parecido: la similitud coseno

¿Cómo se mide que dos embeddings «se parecen»? Lo habitual es la similitud coseno: el producto escalar de los dos vectores dividido entre el producto de sus longitudes, cos(u, v) = (u · v) / (‖u‖ · ‖v‖). Vale 1 si apuntan en la misma dirección, 0 si son perpendiculares y −1

si apuntan en sentidos opuestos, y no depende de la longitud de los vectores, solo de su dirección. El Ejercicio 8.3 te pide implementarla.

Con embeddings bien entrenados, las palabras que aparecen en contextos parecidos («perro» y «gato») tienen similitudes altas, y las diferencias entre vectores capturan relaciones: en el ejemplo clásico de word2vec (Mikolov et al., 2013), el vector de «rey» menos el de «hombre» más el de «mujer» queda muy cerca del de «reina». La idea reaparece en dos sitios clave del libro: el producto Query · Key de la atención (Capítulo 11) es una similitud sin normalizar, y los sistemas RAG (Capítulo 20) buscan documentos por similitud coseno.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- El texto debe convertirse en números antes de procesarlo: tokenización seguida de embeddings.
- Los embeddings son vectores densos y aprendibles que capturan similitud semántica entre tokens.
- La similitud coseno mide el parecido direccional entre dos vectores, independientemente de su magnitud.

✏️ EJERCICIOS — Capítulo 8 — Texto como Vectores

Antes de pasar a la tokenización, comprueba que dominas la idea de representar palabras como vectores. Resuelve los ejercicios y contrasta tus respuestas con las soluciones propuestas.

Ejercicio 8.1 Tokenizar tu nombre

Crea un TokenizadorCaracter con tu propio nombre completo como texto base. Codifica y decodifica tu nombre, y verifica que el proceso es reversible. Luego muestra el índice asignado a cada carácter único de tu nombre.

> 💡 💡 Pista: El vocabulario estará formado únicamente por los caracteres que aparezcan en tu nombre. Si tu nombre tiene caracteres con tildes o espacios, esos también forman parte del vocabulario.

✅ **Solución:**

📄 ejercicio_8_1.py

```
import numpy as np

class TokenizadorCaracter:
    def __init__(self, texto):
        self.vocab = sorted(set(texto))
        self.V     = len(self.vocab)
        self.c2i   = {c:i for i,c in enumerate(self.vocab)}
        self.i2c   = dict(enumerate(self.vocab))
    def codificar(self, t):  return [self.c2i[c] for c in t if c in self.c2i]   #
descarta lo desconocido
    def decodificar(self, l): return ''.join(self.i2c[i] for i in l)

# Cambia por tu propio nombre
nombre = 'Alan Turing'

tok = TokenizadorCaracter(nombre)
print(f'Nombre: "{nombre}"')
print(f'Vocabulario ({tok.V} caracteres): {tok.vocab}')
print()

cod = tok.codificar(nombre)
dec = tok.decodificar(cod)
print(f'Codificado:   {cod}')
print(f'Decodificado: "{dec}"')
print(f'Reversible:   {nombre == dec}')
print()
print('Índice de cada carácter:')
for c, i in tok.c2i.items():
    print(f'  {repr(c):>6}  ->  índice {i}')
```

Ejercicio 8.2 Tabla de frecuencias de bigramas

Dado el texto 'la red neuronal aprende', calcula la frecuencia de cada par de caracteres consecutivos (bigrama). ¿Cuál es el bigrama más frecuente? Esta tabla de frecuencias es la base del modelo de lenguaje más simple posible.

> 💡 💡 Pista: Un bigrama es simplemente texto[i:i+2] para cada posición i. Puedes usar un diccionario de Python para contar. Un modelo de bigramas predice el siguiente carácter basándose únicamente en el carácter actual.

✅ **Solución:**

📄 ejercicio_8_2.py

```
from collections import Counter

texto = 'la red neuronal aprende'

# Contar todos los bigramas
```

```
bigramas = Counter(texto[i:i+2] for i in range(len(texto)-1))
total    = sum(bigramas.values())

print(f'Texto: "{texto}"')
print(f'Total de bigramas: {total}\n')
print('Top 10 bigramas:')
print(f"{'Bigrama':>10}  {'Frec.':>8}  {'Prob.':>8}  Barra")
print('-' * 45)

for bg, freq in bigramas.most_common(10):
    prob  = freq / total
    barra = '' * int(prob * 200)
    print(f'{repr(bg):>10}  {freq:>8}  {prob:>8.4f}  {barra}')

bg_max, f_max = bigramas.most_common(1)[0]
empates = [bg for bg, f in bigramas.items() if f == f_max]
print(f'\nBigrama mas frecuente: {bg_max!r} ({f_max} veces)'
      + (f'; empatan {len(empates)}: {empates}' if len(empates) > 1 else ''))
print()

# Probabilidad condicional: P(siguiente | actual)
print('Probabilidad de que despues de a venga p:')
p_ap = bigramas.get('ap', 0) / sum(v for k,v in bigramas.items() if k[0]=='a')
print(f'  P("p" | "a") = {p_ap:.4f}')
```

ℹ️ Un modelo de bigramas es tan simple como: P(siguiente|actual) = conteos[actual+siguiente] / conteos[actual]. Nuestro Nano-LLM aprende distribuciones condicionales mucho más ricas, condicionadas sobre los últimos 24 caracteres.

Ejercicio 8.3 Similitud coseno: ¿qué mide?

Implementa la similitud coseno entre dos vectores. Luego crea tres vectores de ejemplo: dos muy similares (ángulo pequeño) y uno perpendicular a ambos (ángulo 90°). Verifica que la similitud coseno captura estas relaciones.

💡 💡 Pista: La similitud coseno es el coseno del ángulo entre dos vectores: sim(u,v) = (u · v) / (||u|| · ||v||). Si los vectores apuntan en la misma dirección, sim = 1. Si son perpendiculares, sim = 0. Si apuntan en sentidos opuestos, sim = -1.

✅ **Solución:**

📄 **ejercicio_8_3.py**

```
import numpy as np

def sim_coseno(u, v):
    nu = np.linalg.norm(u) + 1e-10
    nv = np.linalg.norm(v) + 1e-10
    return float(np.dot(u, v) / (nu * nv))

# Vectores de demostración (en 3D para visualizar)
v_ref    = np.array([1.0, 0.0, 0.0])         # referencia
v_parec  = np.array([0.9, 0.4, 0.0])          # muy similar a v_ref
v_perp   = np.array([0.0, 0.0, 1.0])          # perpendicular a los dos
v_opuest = np.array([-1.0, 0.0, 0.0])         # opuesto
v_igual  = np.array([3.0, 0.0, 0.0])          # misma dirección, distinta magnitud

vectores = [
    (v_ref, 'referencia [1,0,0]'),
    (v_parec, 'parecido [0.9,0.4,0]'),
    (v_perp, 'perpendicular [0,0,1]'),
    (v_opuest, 'opuesto [-1,0,0]'),
    (v_igual, 'misma dirección [3,0,0]'),
]

print('Similitudes coseno respecto a v_ref = [1, 0, 0]\n')
print(f"{'Vector':>32}  {'Sim. coseno':>12}  Interpretación")
print('-' * 65)
for v, nombre in vectores:
```

```
        sim = sim_coseno(v_ref, v)
        if sim > 0.9:   interp = 'muy similares'
        elif sim > 0.5: interp = 'bastante similares'
        elif sim > 0.1: interp = 'algo similares'
        elif sim > -0.1: interp = 'perpendiculares'
        else:           interp = 'opuestos'
        print(f'{nombre:>32}  {sim:>12.4f}  {interp}')

    print(f'\nY entre los otros dos: sim(parecido, perpendicular) = {sim_coseno(v_parec,
    v_perp):.4f}')
    print('\nObserva: la similitud coseno NO depende de la magnitud,')
    print('solo de la dirección. [1,0,0] y [3,0,0] son identicos.')
```

Capítulo 9: Tokenización BPE — Del Texto al Token (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar por qué char-level y word-level tokenization no escalan bien.
> ✓ Ejecutar el algoritmo BPE a mano sobre un corpus pequeño.
> ✓ Implementar un entrenador y codificador BPE completos en Python.
> ✓ Razonar sobre la fertilidad del token y su impacto en distintos idiomas.

9.1 El problema que BPE resuelve

En el Capítulo 8 usamos tokenización a nivel de carácter: cada carácter del texto se convierte en un token. Es una solución sencilla y didáctica, pero los LLMs comerciales no la usan. ¿Por qué?

Considera la frase 'la inteligencia artificial aprende'. En tokenización por caracteres son 34 tokens (contando los espacios). El tokenizador BPE de GPT-2, el que usarás en el Capítulo 16, la convierte en 8. Esa diferencia, más de cuatro veces, no es trivial: el mecanismo de atención tiene coste $O(T^2)$, donde T es la longitud de la secuencia. Con secuencias 4 veces más largas, la atención cuesta 16 veces más. A lo largo de billones de ejemplos de entrenamiento, la diferencia es astronómica.

Pero tampoco podemos ir al otro extremo y usar palabras completas como tokens. Eso crea vocabularios de cientos de miles de entradas, no maneja palabras que el modelo nunca ha visto, y es especialmente problemático con idiomas de morfología rica (como el español, el alemán o el finlandés) donde una misma raíz puede generar decenas de formas distintas.

DIAGRAMA 9.1 — ESTRATEGIAS DE TOKENIZACIÓN

El espectro desde carácter hasta palabra — y por qué BPE es el punto justo

Carácter → Subpalabra (BPE) → Palabra | El puente real entre texto y LLM

BPE: Philip Gage (1994, compresión) · Rico Sennrich (2016, NLP)

EL ESPECTRO DE TOKENIZACIÓN

CARÁCTER — SUBPALABRA (BPE) — PALABRA

Nivel carácter

Frase: 'la IA'

l a ␣ I A

5 tokens

inteligencia →

i n t e l i g ...

- ✗ Tokens por frase: ~40
- ✗ Secuencias muy largas
- ✓ Vocab pequeño (~256)
- ✓ Sin OOV nunca

Subpalabra BPE ← ÓPTIMO

Frase: 'la IA'

la | I | A

3 tokens

inteligencia →

intel | igen | cia

3 tokens

- ✓ Vocab controlable (~50k)
- ✓ Secuencias razonables
- ✓ Sin OOV (descompone)
- ✓ Captura morfología

Nivel palabra

Frase: 'la IA'

la | IA

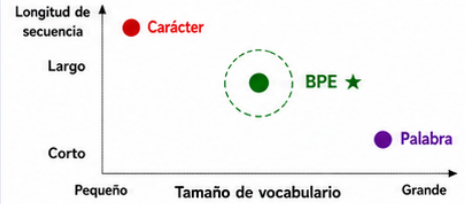
2 tokens

supercalifragilistico →

[OOV]

1 token (inútil)

- ✓ Tokens por frase: ~4
- ✗ Vocab enorme (>1M)
- ✗ OOV: palabras nuevas
- ✗ No escala



BPE: el punto de Goldilocks

cómo funciona BPE

EL ALGORITMO BPE — PASO A PASO

Ejemplo: corpus ['bajo':6, 'bajar':3, 'caja':4]

① Inicio: caracteres + marcador </w>

'bajo' (6): b a j o </w>

'bajar' (3): b a j a r </w>

'caja' (4): c a j a </w>

Vocab inicial: {b, a, j, o, r, c, </w>}

② Contar pares — fusionar el más frecuente

Par	Frecuencia
('a','j')	13 ← EL MÁS FRECUENTE
('b','a')	9
('j','a')	7
('j','o')	6

Fusionar: a + j → aj

'bajo': b aj o </w>

'bajar': b aj a r </w>

'caja': c aj a </w>

aj aparece ahora como un token único

③ Siguiente par más frecuente

('b','aj') → frecuencia 9 → fusionar: b + aj → baj

'bajo': baj o </w>

'bajar': baj a r </w>

El algoritmo descubre raices automáticamente

Resultado tras 5 fusiones:

'bajo' → [bajo</w>] ← 1 token

'bajar' → [baj][a][r][</w>] ← 4 tokens

'caja' → [caj][a][</w>] ← 3 tokens

BPE descubrió la raiz 'baj' sin conocimiento lingüístico

Sin saber nada de morfología, gramática ni idioma

quién lo usa hoy

VOCABULARIO, OOV Y FERTILIDAD

Tamaño del vocabulario (aprox.):

Carácter — ~256 tokens

BPE — ~50.000 tokens ★ GPT-4 usa 100.277

Palabra — >1.000.000 tokens inmanejable

El problema OOV (Out-Of-Vocabulary):

✗ **Nivel palabra:**

'supercalifragilistico' → [OOV]

'ChatGPT' (2022) → [OOV]

Palabras nuevas = invisibles

✓ **BPE:**

'supercalifragilistico' → ['super','cal','if','rag','il','istic','o']

Nunca hay OOV — siempre descompone

Fertilidad: ¿cuántos tokens por palabra?

Tokenizador de GPT-2 (entrenado casi todo en inglés)

Inglés: 'intelligence' → 1 token

Español: 'inteligencia' → 3 tokens

Alemán: 'Künstliche Intelligenz' → 9 tokens

Turco: 'evlerinizden' → 5 tokens

La misma frase: 12 tokens en inglés, 24 en español (Ej. 16.1)

⚠ **Implicación: el mismo texto ocupa más tokens fuera del inglés: más coste y menos contexto útil — consecuencia del corpus de entrenamiento.**

BPE EN EL ECOSISTEMA MODERNO

¿Quién usa qué tokenizador?

Modelo	Tokenizador	Vocab.
GPT-2	Byte-level BPE	50.257
GPT-4	Byte-level BPE (cl100k)	100.277
LLaMA 2	SentencePiece BPE	32.000
LLaMA 3	Byte-level BPE	128.256
BERT	WordPiece	30.522
T5	SentencePiece Unigram	32.100

Las tres familias de tokenización subpalabra:

BPE (Byte Pair Encoding)

Fusiona el par más **FRECUENTE** en cada paso.
Greedy, simple, muy efectivo.
Usado por: GPT-2, GPT-3, GPT-4, LLaMA

WordPiece

Fusiona el par que más aumenta la **PROBABILIDAD** del corpus (variante BERT).
Usado por: BERT, Google models

Unigram Language Model

Empieza con vocab grande y PODA tokens.
Criterio: mínimo impacto en la perplejidad.
Usado por: SentencePiece (T5, mT5)

Byte-level BPE — La variante de GPT

En lugar de caracteres Unicode,
empieza con los 256 bytes posibles.
Garantiza tokenización de CUALQUIER texto,
idioma o código — sin excepción.
Vocab inicial: bytes 0x00 → 0xFF

```
# GPT-4 tokeniza cualquier byte
# → nunca hay OOV absoluto
```

Carácter: vocab ~256 · secuencias largas · sin OOV

BPE: vocab ~50k · secuencias medias · sin OOV ★

Palabra: vocab >1M · secuencias cortas · OOV grave

Carácter: granular | BPE: equilibrio ★ | Palabra: semántico

Color
Red = Nivel carácter / problemas
Green = BPE / solución óptima
Purple = Nivel palabra / OOV
Teal = Pasos del algoritmo BPE
Blue = Tokens / vocabulario / inferencia
Orange = Fertilidad / advertencias
Gold = Métricas de vocabulario

9.2 BPE: un algoritmo de compresión convertido en tokenizador

Byte Pair Encoding fue propuesto originalmente en 1994 por Philip Gage como algoritmo de compresión de datos. La idea es elegante: encuentra el par de símbolos más frecuente en el texto y lo sustituye por un símbolo nuevo. Repite hasta alcanzar el nivel de compresión deseado.

En 2016, Rico Sennrich y su equipo lo adaptaron para NLP en el artículo 'Neural Machine Translation of Rare Words with Subword Units'. En lugar de comprimir bytes, se aplica sobre caracteres de texto. En lugar de comprimir hasta un único símbolo, se detiene cuando el vocabulario alcanza el tamaño deseado. El resultado es un sistema que aprende automáticamente qué fragmentos de palabras son útiles como unidades de tokenización.

> ℹ️ La belleza de BPE es que no necesita conocimiento lingüístico previo. No sabe nada sobre morfología, sílabas o raíces. Simplemente aprende estadística del corpus de entrenamiento. Si 'ción' aparece muy frecuentemente, aprenderá a tratarlo como una unidad. Si 'printf' aparece mucho en código, también.

9.3 El algoritmo paso a paso — con lápiz y papel

Veamos el algoritmo con un ejemplo mínimo. Supongamos que nuestro corpus es este conjunto de palabras con sus frecuencias:

📄 Corpus de ejemplo para seguir BPE manualmente

```
Palabras (frecuencia):
  'bajo'  x5      'baja'  x4      'bajar'  x3
  'bajada' x2

Paso 0 — Vocabulario inicial (caracteres + marcador fin de palabra):
  Cada palabra se representa como caracteres separados.
  El marcador </w> indica fin de palabra.

  'bajo'  -> [b][a][j][o][</w>]  x5
  'baja'  -> [b][a][j][a][</w>]  x4
  'bajar' -> [b][a][j][a][r][</w>]  x3
  ...

Vocabulario inicial: {b, a, j, o, r, d, </w>}  (7 simbolos)

Paso 1 — Contar todos los pares adyacentes:
  (b, a):  5+4+3+2 = 14  <-- el mas frecuente
  (a, j):  5+4+3+2 = 14  <-- empate
  (j, o):  5
  (j, a):  4+3+2 = 9
  ...

  -> Fusionamos (b, a) -> 'ba'   (empate con (a, j): gana el primero que se encontró)

Paso 2 — Despues de la primera fusion:
  'bajo'  -> [ba][j][o][</w>]  x5
  'baja'  -> [ba][j][a][</w>]  x4
  ...
  Vocabulario: {b, a, j, o, r, d, </w>, ba}  (8 simbolos)

Paso 3 — Contar pares de nuevo:
  (ba, j): 5+4+3+2 = 14  <-- ahora el mas frecuente
  -> Fusionamos (ba, j) -> 'baj'

...y asi sucesivamente hasta alcanzar el tamano de vocabulario deseado.
```

El resultado después de suficientes fusiones podría ser que 'bajo', 'baja', 'bajar' y 'bajada' se tokenicen como ['baj','o','</w>'], ['baj','a','</w>'], ['baj','ar','</w>'] y ['baj','ad','a','</w>'] respectivamente. La raíz 'baj' emerge automáticamente como unidad, sin que nadie le haya dicho al algoritmo nada sobre morfología española.

PROGRAMA BPE-1 — Entrenador BPE completo desde cero

📄 **programa_BPE1_entrenador.py**

```
from collections import Counter
import re

# ============================================================
# ENTRENADOR BPE (Byte Pair Encoding)
# Aprende las reglas de fusion optimas sobre un corpus de texto.
# ============================================================

def texto_a_vocab_inicial(texto):
    '''
    Convierte el texto en un vocabulario de frecuencias.
    Cada palabra se representa como una tupla de caracteres
    mas el marcador especial </w> (end of word).
    '''
    contador = Counter()
    for palabra in texto.lower().split():
        # Eliminar puntuacion basica
        # Simplificación didáctica: mantenemos solo letras minúsculas
        # Un tokenizador real trabaja sobre bytes UTF-8 completos.
        palabra = re.sub(r'[^a-záéíóúüñ]', '', palabra.lower())
        if palabra:
            contador[tuple(list(palabra) + ['</w>'])] += 1
    return dict(contador)


def contar_pares(vocab):
    '''Cuenta la frecuencia de todos los pares adyacentes.'''
    pares = Counter()
    for palabra, freq in vocab.items():
        for i in range(len(palabra) - 1):
            pares[(palabra[i], palabra[i+1])] += freq
    return pares


def fusionar_par(vocab, par):
    '''Fusiona el par dado en todas las palabras del vocabulario.'''
    a, b   = par
    nuevo  = {}
    for palabra, freq in vocab.items():
        nueva_pal = []
        i = 0
        while i < len(palabra):
            if i < len(palabra) - 1 and palabra[i] == a and palabra[i+1] == b:
                nueva_pal.append(a + b)   # fusion!
                i += 2
            else:
                nueva_pal.append(palabra[i])
                i += 1
        nuevo[tuple(nueva_pal)] = freq
    return nuevo


def entrenar_bpe(corpus, num_fusiones, verbose=True):
    '''
    Entrena el tokenizador BPE.
    Devuelve la lista de reglas de fusion aprendidas.
    '''
    vocab       = texto_a_vocab_inicial(corpus)
    vocab_chars = set(c for pal in vocab for c in pal)
    reglas      = []

    if verbose:
        print(f'Vocabulario inicial: {len(vocab_chars)} simbolos')
        print(f'Palabras unicas:     {len(vocab)}')
        print(f'Tokens iniciales en corpus: {sum(len(k)*v for k,v in vocab.items())}')
        print()
        print(f"{'Fusion':>7}  {'Par':>25}  {'Frec':>6}  {'Vocab size':>11}")
        print('-' * 55)
```

```
    for i in range(num_fusiones):
        pares = contar_pares(vocab)
        if not pares:
            break

        # Elegir el par mas frecuente
        mejor_par  = max(pares, key=pares.get)
        mejor_freq = pares[mejor_par]

        # Fusionar
        vocab  = fusionar_par(vocab, mejor_par)
        reglas.append(mejor_par)

        # El nuevo token es la concatenacion del par
        token_nuevo = mejor_par[0] + mejor_par[1]
        vocab_chars.add(token_nuevo)

        if verbose and (i < 10 or (i+1) % 10 == 0):
            print(f"  {i+1:>5}  {str(mejor_par):>25}  ",
                  f"{mejor_freq:>6}  {len(vocab_chars):>11}")

    if verbose:
        tokens_final = sum(len(k)*v for k,v in vocab.items())
        print(f'\nVocabulario final:   {len(vocab_chars)} simbolos')
        print(f'Tokens en corpus:    {tokens_final}')
        print(f'Reglas aprendidas:   {len(reglas)}')

    return reglas, vocab_chars


# ================================================================
# CORPUS DE ENTRENAMIENTO
# ================================================================
CORPUS = '''
la inteligencia artificial aprende de los datos de entrenamiento.
los modelos de lenguaje aprenden a predecir el siguiente token.
las redes neuronales aprenden representaciones de los datos.
el aprendizaje automatico permite que las maquinas aprendan.
la tokenizacion divide el texto en unidades llamadas tokens.
el vocabulario define el conjunto de tokens posibles.
los transformers usan atencion para procesar secuencias.
el entrenamiento ajusta los pesos de la red neuronal.
la inteligencia artificial transforma la tecnologia moderna.
aprender a programar inteligencia artificial es fascinante.
'''

print('=== ENTRENANDO TOKENIZADOR BPE ===')
print('Corpus de', len(CORPUS.split()), 'palabras\n')

reglas, vocab = entrenar_bpe(CORPUS, num_fusiones=30, verbose=True)

# Mostrar los tokens aprendidos mas interesantes
print('\nTokens aprendidos (subpalabras emergentes):')
tokens_subpal = [t for t in sorted(vocab) if len(t)>2 and '</w>' not in t]
print('  ' + '  '.join(sorted(tokens_subpal, key=len, reverse=True)[:20]))
```

> Salida del programa:

```
=== ENTRENANDO TOKENIZADOR BPE ===
Corpus de 82 palabras

Vocabulario inicial: 24 simbolos
Palabras unicas:     54
Tokens iniciales en corpus: 589

Fusion                     Par   Frec  Vocab size
-------------------------------------------------------
      1                  ('e', 'n')       27           25
      2              ('s', '</w>')       23           26
      3              ('a', '</w>')       15           27
      4                 ('d', 'e')       15           28
      5                 ('c', 'i')       12           29
      6                 ('t', 'o')       11           30
      7              ('l', '</w>')       10           31
```

```
       8                ('p', 'r')       10           32
       9                ('t', 'e')        8           33
      10            ('de', '</w>')        8           34
      20              ('en', 'ci')        5           44
      30               ('to', 'k')        4           54

Vocabulario final:   54 simbolos
Tokens en corpus:    350
Reglas aprendidas:   30

Tokens aprendidos (subpalabras emergentes):
  apren  enci  apr  tok
```

Observa cómo el algoritmo descubrió automáticamente raíces y palabras frecuentes del corpus. Tras unas pocas fusiones aparecen 'pr' (fusión 8), luego 'apr' y por fin 'apren', porque esa raíz está en 'aprende', 'aprenden', 'aprender' y 'aprendizaje'. El algoritmo no sabe nada de morfología española — simplemente está optimizando la compresión estadística del texto.

PROGRAMA BPE-2 — Codificador y decodificador BPE

📄 **programa_BPE2_codificador.py**

```
import re
fuente = open('programa_BPE1_entrenador.py', encoding='utf-8').read()
exec(fuente.split("print('=== ENTRENANDO")[0])   # entrenar_bpe() y CORPUS (BPE-1)

# ================================================================
# CODIFICADOR BPE
# Aplica las reglas de fusion aprendidas a texto nuevo.
# ================================================================

def codificar_bpe(texto, reglas):
    '''
    Convierte texto en una lista de tokens BPE.
    Aplica las reglas aprendidas en el mismo orden en que
    fueron aprendidas durante el entrenamiento.
    '''
    tokens_totales = []
    for palabra in texto.lower().split():
        palabra = re.sub(r'[^a-záéíóúüñ]', '', palabra)   # la misma normalización que
BPE-1
        if not palabra:
            continue
        # Empezar con caracteres individuales
        chars = list(palabra) + ['</w>']
        # Aplicar reglas en orden de aprendizaje
        for (a, b) in reglas:
            nueva = []
            i = 0
            while i < len(chars):
                if i < len(chars)-1 and chars[i]==a and chars[i+1]==b:
                    nueva.append(a + b)
                    i += 2
                else:
                    nueva.append(chars[i])
                    i += 1
            chars = nueva
        tokens_totales.extend(chars)
    return tokens_totales


def decodificar_bpe(tokens):
    '''Convierte tokens BPE de vuelta a texto (normalizado).
    Se unen los tokens tal cual y cada </w> marca el final de una palabra.
    Recupera las palabras normalizadas (minúsculas, sin cifras ni puntuación),
    no el texto original letra a letra.'''
    return ''.join(tokens).replace('</w>', ' ').strip()


def comparar_tokenizaciones(frase, reglas):
    '''Compara tokenizacion por caracter vs BPE.'''
    # Char-level
```

```
        tokens_char = list(frase.replace(' ', '_'))
        # BPE
        tokens_bpe  = codificar_bpe(frase, reglas)
        tokens_bpe_limpios = [t.replace('</w>', '+') for t in tokens_bpe]
        print(f'  Frase: "{frase}"')
        print(f'  Char: {len(tokens_char):3d} tokens  {tokens_char[:15]}...')
        print(f'  BPE:  {len(tokens_bpe):3d} tokens  {tokens_bpe_limpios}')
        ratio = len(tokens_char) / max(len(tokens_bpe), 1)
        print(f'  BPE es {ratio:.1f}x mas eficiente que char-level')
        print()


    # Entrenar con nuestro corpus
    reglas, vocab = entrenar_bpe(CORPUS, num_fusiones=30, verbose=False)

    print('=== TOKENIZACION BPE EN ACCION ===')
    print()

    frases = [
        'la inteligencia artificial aprende',
        'aprendizaje automatico',
        'redes neuronales profundas',
        'tokenizacion de texto',
    ]
    for frase in frases:
        comparar_tokenizaciones(frase, reglas)

    # Manejar palabras fuera del vocabulario
    print('--- Manejo de palabras desconocidas ---')
    frase_oov = 'supercalifragilistico'
    tokens_oov = codificar_bpe(frase_oov, reglas)
    print(f'  Palabra rara: "{frase_oov}"')
    print(f'  BPE la descompone: {[t.replace("</w>","+") for t in tokens_oov]}')
    print(f'  Sin OOV mientras sus letras esten en el corpus: cae hasta nivel caracter')

    # Ida y vuelta: codificar y decodificar
    print('\n--- Ida y vuelta (texto normalizado) ---')
    for f in ['hola mundo', 'El niño aprende, 2026.']:
        print(f'  {f!r} -> {decodificar_bpe(codificar_bpe(f, reglas))!r}')
    print('  Se recuperan las palabras normalizadas, no el texto literal:')
    print('  mayusculas, cifras y puntuacion se pierden (GPT-2 lo evita con BPE por
    bytes).')

    # Estadisticas de compresion
    print('\n--- Compresion del corpus completo ---')
    tokens_char = sum(len(w) for w in CORPUS.split())
    tokens_bpe  = len(codificar_bpe(CORPUS, reglas))
    print(f'  Chars en corpus:  {tokens_char}')
    print(f'  Tokens BPE:       {tokens_bpe}')
    print(f'  Ratio de compresion: {tokens_char/tokens_bpe:.2f}x')
```

> Salida del programa:

```
=== TOKENIZACION BPE EN ACCION ===

  Frase: "la inteligencia artificial aprende"
  Char:  34 tokens  ['l', 'a', '_', 'i', 'n', 't', 'e', 'l', 'i', 'g', 'e',
  BPE:   15 tokens  ['la+', 'in', 'te', 'l', 'ig', 'enci', 'a+', 'ar', 'ti',
  BPE es 2.3x mas eficiente que char-level

  Frase: "aprendizaje automatico"
  Char:  22 tokens  ['a', 'p', 'r', 'e', 'n', 'd', 'i', 'z', 'a', 'j', 'e',
  BPE:   16 tokens  ['apren', 'd', 'i', 'z', 'a', 'j', 'e+', 'a', 'u', 'to',
  BPE es 1.4x mas eficiente que char-level

  Frase: "redes neuronales profundas"
  Char:  26 tokens  ['r', 'e', 'd', 'e', 's', '_', 'n', 'e', 'u', 'r', 'o',
  BPE:   20 tokens  ['r', 'e', 'de', 's+', 'n', 'e', 'u', 'r', 'on', 'a', 'l',
  BPE es 1.3x mas eficiente que char-level

  Frase: "tokenizacion de texto"
  Char:  21 tokens  ['t', 'o', 'k', 'e', 'n', 'i', 'z', 'a', 'c', 'i', 'o',
  BPE:   12 tokens  ['tok', 'en', 'i', 'z', 'a', 'ci', 'on', '+', 'de+', 'te', 'x', 'to+']
  BPE es 1.8x mas eficiente que char-level

--- Manejo de palabras desconocidas ---
```

```
    Palabra rara: "supercalifragilistico"
    BPE la descompone: ['s', 'u', 'p', 'e', 'r', 'c', 'a', 'l', 'i', 'f', 'r', 'a', 'g',
    Sin OOV mientras sus letras esten en el corpus: cae hasta nivel caracter

  --- Ida y vuelta (texto normalizado) ---
    'hola mundo' -> 'hola mundo'
    'El niño aprende, 2026.' -> 'el niño aprende'
    Se recuperan las palabras normalizadas, no el texto literal:
    mayusculas, cifras y puntuacion se pierden (GPT-2 lo evita con BPE por bytes).

  --- Compresion del corpus completo ---
    Chars en corpus:  517
    Tokens BPE:       350
    Ratio de compresion: 1.48x
```

La demostración de la palabra desconocida 'supercalifragilistico' ilustra una propiedad fundamental de BPE: nunca puede fallar. Si un fragmento de la palabra fue aprendido en el vocabulario, se usa. Si no, cae hasta el nivel de caracteres individuales. No hay palabras fuera del vocabulario.

> 💡 Con el tokenizador de GPT-2, ' intelligence' es un único token, mientras que ' inteligencia' se parte en tres (' intel', 'igen', 'cia'), y 'supercalifragilisticexpialidocious', en 11. Las palabras frecuentes en el corpus de entrenamiento reciben tokens propios; las raras se fragmentan. Esto es exactamente lo que BPE optimiza.

9.4 Byte-level BPE — La variante de GPT-2 a GPT-4

GPT-2, GPT-3 y GPT-4 usan una variante llamada Byte-level BPE. En lugar de empezar con caracteres Unicode, empiezan con los 256 valores de byte posibles. Esto tiene una consecuencia radical: el tokenizador puede manejar CUALQUIER texto, incluyendo emojis, caracteres chinos, código de programación en cualquier lenguaje, y secuencias binarias, sin ninguna excepción.

```
📄 Byte-level BPE — Concepto fundamental
# BPE clasico: empieza con caracteres Unicode
# Vocabulario inicial: {'a','b','c',...,'z','A',...,'0',...} ~100 simbolos
# Problema: textos con caracteres raros o idiomas no romanos
#           pueden tener tokens OOV en el vocabulario inicial.

# Byte-level BPE: empieza con los 256 valores de byte (0-255)
# Vocabulario inicial: {0x00, 0x01, ..., 0xFF}  EXACTAMENTE 256 simbolos
# Ventaja: CUALQUIER texto es tokenizable, sin excepcion.

# GPT-2: 50 257 tokens  (256 bytes + 50 000 fusiones + token especial)
# LLaMA-3: 128 256 tokens (BPE al estilo de tiktoken)
# Claude: tamano no publicado

# Ejemplo de como un emoji se tokeniza en byte-level BPE:
# El emoji del robot (U+1F916) en UTF-8: F0 9F A4 96
# -> Se codifica como 4 bytes -> potencialmente 1 token si es frecuente
# -> O se queda en bytes si es raro

# El texto 'Hello' en bytes UTF-8: 48 65 6C 6C 6F
# BPE aprendera a fusionar esos bytes en el token 'Hello'
# porque 'Hello' aparece millones de veces en internet.
```

9.5 Curiosidades del tokenizador — Lo que revela sobre los LLMs

Estudiar cómo los LLMs tokenizan el texto revela aspectos sorprendentes sobre cómo ven el mundo. Algunos de los fenómenos más interesantes:

La fertilidad del token: el inglés es 'barato'

Los LLMs modernos fueron entrenados mayoritariamente en inglés. Como resultado, su vocabulario BPE tiene tokens muy eficientes para el inglés, pero menos para otros idiomas. Con el tokenizador de GPT-2, «The cat sat on the mat and looked at the window.» ocupa 12 tokens; su traducción al español, 20, y al turco, 24. En el Ejercicio 16.1 lo medirás en cinco

idiomas. Esto tiene implicaciones directas en el coste de la API (se cobra por token) y en la longitud de contexto efectiva.

📄 **Fertilidad del token con el tokenizador de GPT-2 (medida)**

```
# Tokenizador BPE de GPT-2 (el del Capitulo 16), con sus 50 000 fusiones:

# INGLES:  'The cat sat on the mat and looked at the window.'
# 12 tokens: The| cat| sat| on| the| mat| and| looked| at| the| window|.

# ESPAÑOL: 'El gato se sentó en la alfombra y miró por la ventana.'
# 20 tokens: El| g|ato| se| sent|ó| en| la| al|f|omb|ra| y| mir|ó| por| la| vent|ana|.

# TURCO:   'Kedi halının üzerine oturdu ve pencereden baktı.'
# 24 tokens: las letras ı y ü ocupan dos bytes y casi no se fusionaron.

# IMPLICACION PRACTICA:
# La misma frase cuesta 1.7 veces mas tokens en español y 2 veces mas
# en turco: mas coste en las API de pago y menos texto en el contexto.
```

Los números son sorprendentemente caros

Un número como 1.234.567 no se tokeniza como una unidad. El tokenizador de GPT-2 lo parte en seis piezas: ' 1', '.', '234', '.', '5' y '67'. Además, el mismo número puede partirse de forma distinta según lo que lo rodee. Por eso los LLMs tienen dificultades inherentes con la aritmética exacta: operan sobre fragmentos de números, no sobre los números en sí.

El código se tokeniza diferente al texto

Los nombres de variables cortos ('i', 'x', 'fn') suelen ser tokens individuales. Los nombres largos ('calculate_gradient_descent') se fragmentan. Los símbolos de puntuación del código ('{', '}', '(', ')', ';') son cada uno un token. Esto explica por qué los LLMs entrenados en código pueden generar código sintácticamente correcto: los tokens respetan la estructura del lenguaje de programación.

El misterio de los tokens fantasma

En 2023, dos investigadores descubrieron que el vocabulario que comparten GPT-2 y GPT-3 contenía tokens que casi nunca aparecían en los datos con los que se entrenaron los modelos (por ejemplo, nombres de usuario de un foro de Reddit). Sus embeddings apenas se habían entrenado, y cuando se pedía a GPT-3 que repitiera uno de ellos, respondía con palabras extrañas o se negaba. Este fenómeno, bautizado como 'SolidGoldMagikarp' por uno de los tokens problemáticos, ilustra cómo los detalles de la tokenización pueden tener efectos inesperados en el comportamiento del modelo.

9.6 Impacto en nuestro Nano-LLM

Nuestro Nano-LLM del Capítulo 13 usa tokenización a nivel de carácter. Si lo actualizáramos a BPE, ¿qué cambiaría?

📄 **Impacto de BPE en la arquitectura del Nano-LLM**

```
# NANO-LLM CON TOKENIZACION DE CARACTERES (actual):
# Vocabulario:    ~46 caracteres
# Secuencia:      'hola mundo' -> 10 tokens
# Embedding:      tabla de 46 x D
# Contexto:       CTX=24 caracteres ~ 4-5 palabras

# NANO-LLM CON BPE (hipotetico):
# Vocabulario:    500-2000 subpalabras (segun corpus)
# Secuencia:      'hola mundo' -> 2-3 tokens
# Embedding:      tabla de 1000 x D (mas parametros)
# Contexto:       CTX=24 tokens ~ 15-20 palabras

# VENTAJAS DEL BPE:
# + Secuencias mas cortas -> atencion mas eficiente
# + El mismo contexto CTX cubre mucho mas texto
# + El modelo aprende relaciones semanticas mas directamente
# + Menos pasos de generacion para el mismo texto
```

```
# DESVENTAJAS PARA USO DIDACTICO:
# - Vocabulario mas grande: tabla de embedding mayor (la misma que la de salida)
# - El entrenador BPE requiere mas texto para aprender bien
# - Mas complejo de implementar y depurar

# CONCLUSION:
# Para aprender los conceptos: char-level es perfecto.
# Para produccion: BPE con vocabulario de 32k-100k es el estandar.

# Los LLMs de produccion usan vocabularios de:
# GPT-2:    50 257 tokens
# LLaMA-2:  32 000 tokens  (SentencePiece BPE)
# LLaMA-3: 128 256 tokens
# Gemma:    256 000 tokens
# Claude:   no publicado
```

El Programa RZ-4 del Capítulo 15 usa precisamente este argumento: trabaja con tokens de palabra (un vocabulario fijo de 57 tokens: 40 conceptos, 8 verbos y 9 símbolos especiales) para que 10 frases quepan en 78 posiciones. A nivel de carácter ocuparían unas 400, y el entrenamiento en NumPy sería decenas de veces más lento: es la ventaja de las «secuencias más cortas» de BPE.

9.7 Alternativas a BPE

BPE no es el único algoritmo de tokenización subpalabra. Existen tres familias principales:

* BPE (Byte Pair Encoding): aprende reglas de fusión greedy basándose en frecuencia. Usado por GPT-2, GPT-3, GPT-4, LLaMA y RoBERTa. Es el más extendido.
* WordPiece: variante de BPE usada por BERT y los modelos de Google. En lugar de fusionar el par más frecuente, fusiona el par que más aumenta la verosimilitud del corpus si se añade al vocabulario. Produce tokenizaciones ligeramente diferentes a BPE.
* Unigram Language Model (SentencePiece): empieza con un vocabulario grande y lo poda iterativamente, eliminando tokens que afecten menos a la verosimilitud. Usado por T5, ALBERT y XLNet (la librería SentencePiece implementa también BPE, que es el que usan LLaMA y LLaMA-2). A diferencia de BPE y WordPiece, es probabilístico: el mismo texto puede tokenizarse de distintas formas, lo que actúa como regularización.

Todos comparten la misma filosofía: encontrar el equilibrio entre vocabularios manejables y secuencias cortas, usando estadística del corpus para decidir qué subpalabras merecen su propio token.

BPE es uno de esos casos en la historia de la tecnología donde un algoritmo de propósito general, diseñado para comprimir datos, encontró su vocación definitiva en un campo completamente diferente. Sin BPE o sus variantes, los LLMs modernos tal como los conocemos no existirían: o tendrían vocabularios inmanejables, o secuencias tan largas que la atención sería computacionalmente imposible. Es la pieza invisible que hace que todo funcione.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- BPE parte de caracteres y fusiona iterativamente los pares más frecuentes hasta alcanzar el vocabulario deseado.
- Es el algoritmo de tokenización usado por GPT-2, GPT-3, GPT-4 y LLaMA — el puente real entre texto y LLM.
- La fertilidad del token varía según el idioma: el inglés es más 'barato' que el español, alemán o turco.

✏️ EJERCICIOS — Capítulo 9 — Tokenización BPE

Toca practicar con BPE. Intenta estos ejercicios antes de continuar: comparar tu solución con la propuesta es la forma más rápida de ver qué partes del algoritmo aún no están claras.

Ejercicio 9.1 BPE a mano: dos iteraciones

Dado el mini-corpus {'bajo': 6, 'bajar': 3, 'caja': 4, 'cajón': 2}, ejecuta a mano (con lápiz y papel, luego verifica con código) las dos primeras fusiones del algoritmo BPE. Indica qué par se fusiona en cada paso y por qué.

> 💡 💡 Pista: Recuerda separar cada palabra en caracteres + marcador </w>, y contar TODOS los pares adyacentes ponderados por la frecuencia de la palabra. El par más frecuente gana la fusión.

✅ Solución:

📄 ejercicio_9_1.py

```
from collections import Counter

def texto_a_vocab(palabras_freq):
    return {tuple(list(p) + ['</w>']): f for p, f in palabras_freq.items()}

def contar_pares(vocab):
    pares = Counter()
    for palabra, freq in vocab.items():
        for i in range(len(palabra)-1):
            pares[(palabra[i], palabra[i+1])] += freq
    return pares

def fusionar(vocab, par):
    a, b = par; nuevo = {}
    for palabra, freq in vocab.items():
        nueva_pal, i = [], 0
        while i < len(palabra):
            if i<len(palabra)-1 and palabra[i]==a and palabra[i+1]==b:
                nueva_pal.append(a+b); i += 2
            else:
                nueva_pal.append(palabra[i]); i += 1
        nuevo[tuple(nueva_pal)] = freq
    return nuevo

corpus = {'bajo': 6, 'bajar': 3, 'caja': 4, 'cajon': 2}
vocab  = texto_a_vocab(corpus)

print('=== BPE A MANO: 2 ITERACIONES ===\n')
print('Vocabulario inicial:')
for p, f in vocab.items(): print(f'  {p} x{f}')

for it in range(1, 3):
    pares = contar_pares(vocab)
    mejor = max(pares, key=pares.get)
    print(f'\n--- Iteracion {it} ---')
    print(f'Pares mas frecuentes: {pares.most_common(3)}')
    print(f'Fusionando: {mejor} (frecuencia {pares[mejor]})')
    vocab = fusionar(vocab, mejor)
    print('Vocabulario tras fusion:')
    for p, f in vocab.items(): print(f'  {p} x{f}')
```

> ℹ️ El par ('a','j') debería ganar la primera fusión: aparece en 'bajo'(6) + 'bajar'(3) + 'caja'(4) + 'cajon'(2) = 15 veces, más que cualquier otro par.

Ejercicio 9.2 Calcular la tasa de compresión de BPE en función del número de fusiones

Entrena el tokenizador BPE del Programa BPE-1 con 5, 10, 20, 40 y 80 fusiones sobre el mismo corpus. Mide cuántos tokens resultan en cada caso y calcula la tasa de compresión respecto a char-level. ¿La compresión mejora indefinidamente o se satura?

> 💡 💡 Pista: Cada fusión adicional reduce el número de tokens, pero con rendimientos decrecientes: las primeras fusiones capturan los patrones más frecuentes (mayor impacto), las últimas capturan casos cada vez más raros.

✅ **Solución:**

📄 **ejercicio_9_2.py**

```
import numpy as np
fuente = open('programa_BPE2_codificador.py', encoding='utf-8').read()
exec(fuente.split('# Entrenar con nuestro corpus')[0])   # BPE-1 y BPE-2

CORPUS = '''
la inteligencia artificial aprende de los datos de entrenamiento.
los modelos de lenguaje aprenden a predecir el siguiente token.
las redes neuronales aprenden representaciones de los datos.
'''

n_fusiones_lista = [5, 10, 20, 40, 80]
n_chars = sum(len(w) for w in CORPUS.split())

print(f'Caracteres totales en el corpus: {n_chars}\n')
print(f"{'N fusiones':>11}  {'Tokens BPE':>11}  {'Compresion':>11}  {'Mejora vs
anterior':>19}")
print('-' * 60)

anterior = n_chars
for nf in n_fusiones_lista:
    reglas, vocab = entrenar_bpe(CORPUS, num_fusiones=nf, verbose=False)
    n_tokens = len(codificar_bpe(CORPUS, reglas))
    compresion = n_chars / n_tokens
    mejora = (anterior - n_tokens) / anterior * 100
    print(f'{nf:>11}  {n_tokens:>11}  {compresion:>10.2f}x  {mejora:>18.1f}%')
    anterior = n_tokens

print()
print('La compresion mejora con cada fusion, pero la GANANCIA marginal')
print('decrece: las primeras fusiones (patrones frecuentes) aportan')
print('mucho mas que las ultimas (casos cada vez mas raros).')
```

Ejercicio 9.3 Comparar la fertilidad del token entre dos idiomas simulados

Entrena dos tokenizadores BPE independientes: uno sobre un corpus 'simulando inglés' (palabras cortas, raíces simples) y otro sobre un corpus 'simulando alemán' (palabras largas compuestas). Compara cuántos tokens necesita cada uno para representar un texto de la misma longitud en caracteres.

> 💡 💡 Pista: Simula la diferencia concatenando palabras para crear 'compuestos' largos en el corpus alemán (ej. 'inteligenciaartificial' como una sola palabra), y mantén palabras cortas separadas en el corpus inglés.

✅ **Solución:**

📄 **ejercicio_9_3.py**

```
import numpy as np
fuente = open('programa_BPE2_codificador.py', encoding='utf-8').read()
exec(fuente.split('# Entrenar con nuestro corpus')[0])   # BPE-1 y BPE-2

# Simulamos las dos situaciones lingüísticas
CORPUS_TIPO_INGLES = '''
la red es buena la red aprende el dato es bueno
el dato fluye la red usa el dato la red es util
'''

CORPUS_TIPO_ALEMAN = '''
```

```
    lareduronal esbuenaparaaprender eldatoesimportanteparalared
    elflujodedatos esutilparalaredneuronalartificial
    '''

    for nombre, corpus in [('Tipo INGLES (palabras cortas)', CORPUS_TIPO_INGLES),
                           ('Tipo ALEMAN (compuestos largos)', CORPUS_TIPO_ALEMAN)]:
        reglas, vocab = entrenar_bpe(corpus, num_fusiones=20, verbose=False)
        n_chars  = len(corpus.replace(chr(10), '').replace(' ', ''))
        n_tokens = len(codificar_bpe(corpus, reglas))
        fertilidad = n_tokens / len(corpus.split())
        print(f'{nombre}:')
        print(f'  Caracteres: {n_chars}  Tokens: {n_tokens}  '
              f'Fertilidad: {fertilidad:.2f} tokens/palabra')
        print()

    print('El corpus con compuestos largos necesita MAS tokens por palabra')
    print('porque las palabras unicas son mas dificiles de comprimir:')
    print('cada compuesto aparece pocas veces, asi que BPE no puede')
    print('asignarle un token unico tan facilmente.')
    print()
    print('Es una ilustracion con dos corpus inventados, no una medida de')
    print('tokenizadores reales; pero apunta en la misma direccion que el fenomeno')
    print('real: idiomas de morfologia rica (aleman, finlandes, turco) gastan mas')
    print('tokens por palabra en tokenizadores entrenados sobre todo en ingles.')
```

ℹ️ Este es exactamente el fenómeno que documentamos en el capítulo: GPT-4 necesita más tokens por palabra en alemán que en inglés, lo que se traduce en mayor coste y menor contexto efectivo para hablantes de esos idiomas.

Capítulo 10: Redes Recurrentes — El Camino hacia el Transformer (***)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar la idea de recurrencia y el papel del estado oculto.
> ✓ Implementar un RNN básico y observar su estado oculto evolucionar.
> ✓ Demostrar numéricamente el problema del vanishing gradient.
> ✓ Justificar con argumentos técnicos por qué el Transformer reemplazó a las RNNs.

10.1 Un problema que el MLP no puede resolver

Todas las redes que hemos construido hasta ahora tienen una limitación fundamental: tratan cada entrada como un vector independiente, sin memoria de lo que vino antes. Para imágenes esto es aceptable — los píxeles de una imagen no tienen un orden temporal. Pero el lenguaje es radicalmente diferente.

En la frase 'el banco aprobó el préstamo', la palabra 'banco' tiene un significado. En 'me senté en el banco del parque', tiene otro. Para desambiguar, necesitamos recordar el contexto anterior. Y ese contexto puede estar a decenas o cientos de palabras de distancia del término que queremos entender.

Un MLP estándar, al que le damos una ventana fija de tokens, tiene exactamente la misma capacidad de procesar 'banco' en cualquier contexto — no puede conectar lo que ha visto antes con lo que está procesando ahora, más allá de la ventana de entrada. Necesitamos una arquitectura con memoria.

> ℹ️ Esta limitación del MLP para secuencias fue el problema central del procesamiento del lenguaje natural durante los años 80 y 90. Las Redes Neuronales Recurrentes (RNNs) fueron la respuesta dominante durante casi dos décadas, hasta que el Transformer las reemplazó en 2017.

10.2 La idea de la recurrencia

La solución conceptualmente más simple es añadir un estado oculto que persiste de un paso temporal al siguiente. En cada paso, la red recibe dos entradas: el token actual y el estado oculto del paso anterior. Produce dos salidas: una predicción (o representación) para el paso actual y un nuevo estado oculto que resume todo lo que ha procesado hasta ahora.

DIAGRAMA 10.1 — REDES RECURRENTES (RNN)

Memoria secuencial: cómo procesar el tiempo con redes neuronales

El estado oculto persiste · La recurrencia da memoria · El gradiente desaparece

Rumelhart, Hinton & Williams (1986) — precursor directo del Transformer

EL MLP NO TIENE MEMORIA

El problema del contexto:

'el **banco** aprobó el préstamo'
→ banco = INSTITUCIÓN FINANCIERA 🏦

'me senté en el **banco** del parque'
→ banco = ASIENTO 🪑

La misma palabra. Significado opuesto.
El contexto lo determina todo.

El MLP procesa cada token aislado:

x_t → MLP → y_t

El MLP solo ve x_t — el token actual.
No sabe nada de lo que vino antes.

→ Imposible resolver la ambigüedad de 'banco'
→ Imposible modelar dependencias temporales

Procesamiento independiente en el tiempo:

x_1 = "el" → MLP → y_1
x_2 = "banco" → MLP → y_2
x_3 = "del" → MLP → y_3

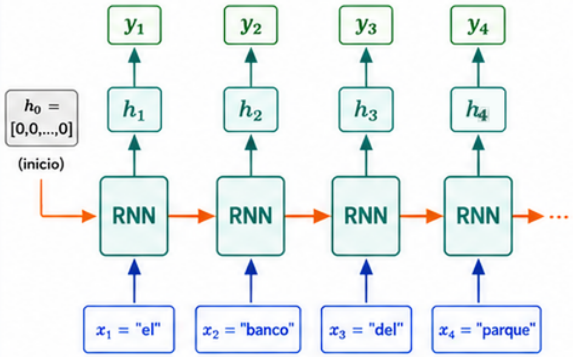
------ no hay conexión entre pasos

La solución: añadir un estado oculto h_t que viaje de un paso al siguiente.

la recurrencia da memoria →

LA RNN DESPLEGADA A TRAVÉS DEL TIEMPO

Frase: "el banco del parque"



x_t (entrada)

Estado oculto h_t y salida y_t

Ecuaciones de la RNN:

$$h_t = tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

$$y_t = W_y \cdot h_t + b_y$$

W_h: pesos recurrentes (hidden→hidden)
W_x: pesos de entrada (input→hidden)
W_y: pesos de salida (hidden→output)

Los mismos pesos W en cada paso → parameter sharing

h_t es la MEMORIA de la red.
Resume todo lo que ha visto hasta t.
Viaja de un paso al siguiente.
Da contexto.

Analogía:

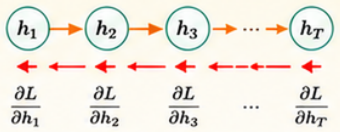
Como una persona que lee una frase:
Recuerda lo anterior para entender el siguiente token.
h_t = lo que recuerda hasta ahora.
y_t = lo que predice ahora.

el problema al entrenar →

EL PROBLEMA: GRADIENTE DESAPARECE

¿Qué pasa en el BPTT?

Para actualizar W_h, el gradiente debe viajar atrás a través de muchos pasos en el tiempo.



← gradiente retropropagado

$$\frac{\partial L}{\partial h_t} = \prod_{k=t+1}^{T} [W_h^T \, \mathrm{diag}(1 - h_k^2)] \frac{\partial L}{\partial h_T}$$

Si los autovalores de ese producto < 1
→ se encoge a 0; si son > 1 → explota (inestable)

Resultado típico: DESAPARECE a 0

Programa RNN-2: 1 → 0.13 → $6.8 \cdot 10^{-5}$ en 5 pasos

Consecuencias:

- La red olvida el contexto lejano.
- Solo aprende dependencias cortas.
- Bajo rendimiento en secuencias largas.
- Entrenamiento inestable o ineficiente.

Soluciones históricas:

- ✓ LSTM (1997) — puertas de memoria
- ✓ GRU (2014) — versión simplificada
- ✓ Inicialización ortogonal
- ✓ Clipping de gradientes
- ✓ Normalización (LayerNorm, etc.)

⚠ Límite fundamental de las RNN clásicas:
la memoria se degrada con la distancia temporal.

por qué el Transformer ganó →

RNN vs TRANSFORMER

¿Por qué el Transformer ganó?

Comparativa directa:

Característica	RNN	Transformer
Procesamiento	Secuencial ⚠	Paralelo ✓
Dep. largo alcance	Difícil (vanishing)	Directo (attention)
Velocidad train	Lento (paso a paso)	Rápido (GPU paralelo)
Contexto máximo	el gradiente muere en < 20 pasos	miles de tokens
Complejidad	O(T) pasos secuenciales	$O(T^2)$ en atención
Estado del arte	hasta ~2018	desde 2017 →

Por qué la RNN era un cuello de botella:

- Secuencial: no se puede paralelizar. GPUs modernas no aprovechadas.
- Memoria limitada: vanishing gradient. El pasado lejano es invisible.
- Contexto útil: pocas decenas de pasos. Documentos largos: imposibles.

La evolución histórica:

- **1986** RNN — Rumelhart, Hinton & Williams. Backpropagation through time (BPTT)
- **1997** LSTM — Hochreiter & Schmidhuber. Puertas de memoria: solución parcial
- **2014** GRU — Cho et al. Versión simplificada de LSTM
- **2017** TRANSFORMER — Vaswani et al. "Attention is All You Need". Las RNNs quedan obsoletas para NLP

Las RNNs siguen siendo útiles en audio, series temporales y recursos limitados.

$$h_t = tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$
$$y_t = W_y \cdot h_t + b_y$$

Mismos pesos W en cada paso → parameter sharing

h_t: memoria de la red

Recurrencia: tiempo → pasado

Transformer: lo supera en 2017

Color
Blue = Input x_t / tokens de entrada
Teal = Estado oculto h_t (memoria)
Green = Output y_t / salida
Orange = Flujo temporal / recurrencia
Red = Vanishing gradient / problemas
Gold = Pesos W (compartidos entre pasos)
Purple = Transformer / solución histórica

10.3 La matemática del RNN

Un RNN básico tiene dos ecuaciones. La primera calcula el nuevo estado oculto combinando la entrada actual y el estado anterior. La segunda calcula la salida (los logits para el siguiente token) a partir del estado oculto:

📄 **Ecuaciones del RNN básico**

```
# En cada paso temporal t:
# 1. Nuevo estado oculto (la 'memoria actualizada')
h_t = tanh( W_hh @ h_{t-1}  +  W_xh @ x_t  +  b_h )
      ^^^^   ^^^^^^^^^^^^         ^^^^^^^^^^^^
      activ. contribucion del    contribucion del
             estado anterior      token actual

# 2. Prediccion para el siguiente token
y_t = W_hy @ h_t + b_y

# Dimensiones tipicas:
# x_t:   (D,)     D = dimension del embedding
# h_t:   (H,)     H = dimension del estado oculto
# W_xh:  (H, D)   proyeccion entrada -> estado
# W_hh:  (H, H)   proyeccion estado anterior -> estado
# W_hy:  (V, H)   proyeccion estado -> vocabulario

# Los pesos W_hh, W_xh, W_hy se comparten en TODOS los pasos.
# Un RNN tiene muchos menos parametros que un Transformer,
# pero procesa los tokens secuencialmente.
```

PROGRAMA RNN-1 — Red Recurrente desde cero: paso a paso

📄 **programa_RNN1_rnn_basico.py**

```
import numpy as np

np.random.seed(42)

# ============================================================
# RNN BASICO PARA PREDICCION DE SECUENCIAS DE CARACTERES
# Mismo problema que el Nano-LLM del Cap 13, pero con RNN.
# ============================================================

# Texto de ejemplo
TEXTO = 'la inteligencia artificial aprende del texto de entrenamiento'
chars = sorted(set(TEXTO))
V     = len(chars)
c2i   = {c: i for i, c in enumerate(chars)}
i2c   = dict(enumerate(chars))
datos = [c2i[c] for c in TEXTO]

# Hiperparametros
D = 8    # dimension del embedding
H = 16   # dimension del estado oculto

# Parametros del RNN (escalas elegidas para que el estado oculto se vea)
W_xh = np.random.randn(H, D) * 0.25   # entrada -> estado
W_hh = np.random.randn(H, H) * 0.25   # estado -> estado
b_h  = np.zeros(H)                     # sesgo del estado
W_hy = np.random.randn(V, H) * 0.01   # estado -> salida
b_y  = np.zeros(V)                     # sesgo de la salida
E    = np.random.randn(V, D)          # tabla de embeddings

def softmax(x): e=np.exp(x-x.max()); return e/e.sum()

def rnn_forward(tokens):
    """
    Pasa una secuencia de tokens por el RNN paso a paso.
    Devuelve los logits y el historial de estados ocultos.
    """
    T       = len(tokens)
    h       = np.zeros(H)          # estado inicial: ceros
    estados = [h.copy()]           # guardar h0
    logits  = []
```

```
    for t in range(T):
        x_t = E[tokens[t]]          # embedding del token actual
        # Ecuacion del RNN: combinar estado anterior + entrada actual
        h   = np.tanh(W_hh @ h + W_xh @ x_t + b_h)
        y_t = W_hy @ h + b_y        # prediccion
        estados.append(h.copy())
        logits.append(y_t)

    return np.array(logits), estados

# ================================================================
# VISUALIZAR EL ESTADO OCULTO EVOLUCIONANDO
# ================================================================
secuencia  = datos[:12]            # primeros 12 caracteres
chars_seq  = TEXTO[:12]
logits_seq, estados = rnn_forward(secuencia)

print('=== RNN: ESTADO OCULTO A TRAVES DEL TIEMPO ===')
print(f'Secuencia: "{chars_seq}"')
print(f'Vocab: {V} chars | Embedding: {D}d | Estado oculto: {H}d')
print()
print('Cada columna es el estado oculto h_t (primeras 6 dims).')
print('Observa como cambia con cada nuevo caracter.')
print()
print(f"{'Paso':>5}  {'Char':>5}  {'h_t[:6] (estado oculto — la memoria)':>40}")
print('-' * 58)
for t, (c, h) in enumerate(zip('·'+chars_seq, estados)):   # paso 0: h0
    char_show = repr(c) if c != ' ' else "' '"
    h_str = '[' + ' '.join(f'{v:>5.2f}' for v in h[:6]) + ']'
    print(f'{t:>5}  {char_show:>5}  {h_str}')

# ================================================================
# ENTRENAMIENTO BASICO (10 pasos para demostrar el mecanismo)
# ================================================================
def calcular_perdida(tokens):
    logits, _ = rnn_forward(tokens[:-1])
    perdida   = 0.0
    for t in range(len(logits)):
        p = softmax(logits[t])
        perdida -= np.log(p[tokens[t+1]] + 1e-10)
    return perdida / len(logits)

print(f'\nPerdida inicial (antes de entrenar): {calcular_perdida(datos):.4f}')
print(f'Perdida aleatoria esperada: ln({V}) = {np.log(V):.4f}')

print()
print('=== DIFERENCIA CLAVE CON EL TRANSFORMER ===')
print(f'RNN: procesa {len(datos)} tokens de UNO EN UNO (secuencial).')
print(f'     Para procesar el token 50, necesita los pasos 1..49.')
print(f'     NO se puede paralelizar en GPUs.')
print(f'Transformer: procesa TODOS los tokens en PARALELO.')
print(f'     El entrenamiento es ordenes de magnitud mas rapido.')
```

> Salida del programa:

```
=== RNN: ESTADO OCULTO A TRAVES DEL TIEMPO ===
Secuencia: "la inteligen"
Vocab: 16 chars | Embedding: 8d | Estado oculto: 16d

Cada columna es el estado oculto h_t (primeras 6 dims).
Observa como cambia con cada nuevo caracter.

 Paso   Char      h_t[:6] (estado oculto — la memoria)
----------------------------------------------------------
    0    '·'  [ 0.00  0.00  0.00  0.00  0.00  0.00]
    1    'l'  [ 0.32 -0.67 -0.61  0.70 -0.96  0.06]
    2    'a'  [ 0.97 -0.83 -0.92 -0.31 -0.02  0.10]
    3    ' '  [-0.92  0.94 -0.25 -0.05  0.88  0.64]
    4    'i'  [-0.72  0.03  0.06 -0.35 -0.03  0.66]
    5    'n'  [ 0.72  0.47  0.61  0.22 -0.21 -0.22]
    6    't'  [ 0.23  0.10 -0.16  0.89 -0.82  0.30]
    7    'e'  [ 0.21 -0.96 -0.71  0.83 -0.79  0.50]
    8    'l'  [ 0.71 -0.77 -0.85  0.61 -0.95  0.02]
    9    'i'  [ 0.33 -0.22 -0.81 -0.39  0.47 -0.17]
```

```
    10    'g'  [ 0.93  0.54 -0.59  0.19 -0.93  0.23]
    11    'e'  [-0.12 -0.88 -0.79  0.85 -0.66  0.91]
    12    'n'  [ 0.33 -0.30 -0.82  0.38  0.56  0.04]

Perdida inicial (antes de entrenar): 2.7756
Perdida aleatoria esperada: ln(16) = 2.7726

=== DIFERENCIA CLAVE CON EL TRANSFORMER ===
RNN: procesa 61 tokens de UNO EN UNO (secuencial).
     Para procesar el token 50, necesita los pasos 1..49.
     NO se puede paralelizar en GPUs.
Transformer: procesa TODOS los tokens en PARALELO.
     El entrenamiento es ordenes de magnitud mas rapido.
```

Observa la columna del estado oculto: la fila 0 es el estado inicial, todo ceros, y cada carácter lo transforma. La «l» de los pasos 1 y 8 produce estados parecidos, pero no iguales: el estado depende de la letra y de todo lo que vino antes. Esa pequeña tabla de 16 números es todo lo que el RNN 'recuerda' del texto que ha procesado. Es una limitación fundamental: comprimir toda la historia de una secuencia en un vector fijo de dimensión H. Cuanto más larga la secuencia, más difícil es mantener información relevante del principio.

10.4 El problema del gradiente que desaparece

El problema más grave de las RNNs se descubrió al intentar entrenarlas: el gradiente que retropropaga hacia atrás a través del tiempo se desvanece exponencialmente.

El Programa RNN-2 lo simula de forma muy simplificada: parte de un gradiente de norma 1 en el último paso y lo multiplica hacia atrás, paso a paso, por W_hh^T y por la derivada de tanh (que fija en 0.5, un valor típico). Compara cuatro matrices W_hh, cada vez más grandes.

Durante el entrenamiento, para ajustar los pesos que conectan el token en la posición 1 con la pérdida calculada en la posición 100, el gradiente debe viajar hacia atrás a través de 100 aplicaciones de la ecuación del RNN. Cada aplicación multiplica el gradiente por la derivada de tanh (que está entre 0 y 1) y por la matriz W_hh. Si los valores propios de W_hh (los factores por los que la matriz estira o encoge los vectores en sus direcciones especiales) son menores que 1 en valor absoluto, el gradiente se multiplica por un número menor que 1 en cada paso — y después de 100 pasos, ha decrecido a un valor prácticamente nulo.

El efecto práctico es que a una RNN simple le cuesta mucho aprender dependencias de largo alcance. Lo que ocurrió hace 50 tokens apenas se refleja en el gradiente que actualiza los pesos. El modelo puede aprender a predecir el siguiente carácter basándose en los 5-10 anteriores, pero rara vez consigue conectar un pronombre con su antecedente a 30 palabras de distancia.

PROGRAMA RNN-2 — El gradiente que desaparece: demostración numérica

📄 **programa_RNN2_vanishing_gradient.py**

```
import numpy as np

np.random.seed(0)

# ============================================================
# DEMOSTRACION DEL VANISHING GRADIENT
# Simulamos como fluye el gradiente hacia atras en un RNN.
# En cada paso, el gradiente se multiplica por W_hh.T
# y por la derivada de tanh (entre 0 y 1).
# ============================================================

H    = 32   # dimension del estado oculto
T    = 60   # longitud de la secuencia

# Cuatro RNNs con distintas inicializaciones de W_hh
configs = {
    'W_hh pequena  (escala 0.05)':      np.random.randn(H,H) * 0.05,
    'W_hh moderada (escala 1/sqrt(H))': np.random.randn(H,H) * (1.0/np.sqrt(H)),
    'W_hh grande   (escala 0.3)':       np.random.randn(H,H) * 0.3,
    'W_hh muy grande (escala 0.5)':     np.random.randn(H,H) * 0.5,
```

```
}

def simular_gradiente(W_hh, T):
    """
    Simula el flujo del gradiente hacia atras en un RNN.
    En cada paso backward, el gradiente se multiplica por W_hh.T
    y por la derivada de tanh (simplificada a 0.5 de media).
    """
    grad   = np.ones(H) / np.sqrt(H)  # gradiente inicial normalizado
    normas = [float(np.linalg.norm(grad))]

    for t in range(T):
        # Derivada de tanh promediada: ~0.5 en un estado tipico
        d_tanh = 0.5 * np.ones(H)
        # Backprop un paso atras
        grad   = (W_hh.T @ grad) * d_tanh
        normas.append(float(np.linalg.norm(grad)))

    return normas

print('=== VANISHING / EXPLODING GRADIENT EN RNN ===')
print(f'Estado oculto H={H}, secuencia T={T}')
print()
print('Norma del gradiente a distintas distancias del final:\n')
print(f"{'Distancia':>10}", end='')
for nombre in configs: print(f'  {nombre.split("(")[0].strip():>17}', end='')
print()
print('-' * (10 + 19*len(configs)))

historicos = {}
for nombre, W in configs.items():
    historicos[nombre] = simular_gradiente(W, T)

pasos_mostrar = [0, 1, 2, 5, 10, 20, 30, 50, 60]
for dist in pasos_mostrar:
    print(f'{dist:>10}', end='')
    for nombre in configs:
        v = historicos[nombre][dist]
        if v < 1e-10:   fmt = '< 1e-10  (MUERTO)'
        elif v > 1e4:   fmt = f'{v:.1e}  (EXPLOTA)'
        else:           fmt = f'{v:.6f}'
        print(f'  {fmt:>17}', end='')
    print()

# Visualizacion ASCII del decaimiento
print('\nVisualizacion del gradiente (W_hh pequena — caso tipico):')
print('Norma del gradiente al retropropagar hacia atras:')
normas = historicos['W_hh pequena  (escala 0.05)']
vmax   = normas[0]
for dist in range(0, T+1, 5):
    v     = normas[dist]
    ratio = v / vmax if vmax > 0 else 0
    barra = '#' * int(ratio * 40)
    arrow = '<-- el gradiente es casi NULO' if v < 1e-4 else ''
    print(f'  Distancia {dist:>3}: {v:.2e}  {barra} {arrow}')

# Contraste: la derivada REAL de tanh, 1 - h^2, en una secuencia de verdad
# (entradas al azar ya proyectadas; mismo W_hh que arriba)
np.random.seed(1)
X_in = np.random.randn(T, H) * 0.5

def gradiente_real(W_hh):
    h, hs = np.zeros(H), []
    for t in range(T):                                  # forward real
        h = np.tanh(W_hh @ h + X_in[t]); hs.append(h)
    grad, normas, dts = np.ones(H) / np.sqrt(H), [1.0], []
    for t in reversed(range(T)):                        # backward real
        d_tanh = 1 - hs[t]**2
        dts.append(d_tanh.mean())
        grad = W_hh.T @ (grad * d_tanh)
        normas.append(float(np.linalg.norm(grad)))
    return normas, float(np.mean(dts))
```

```
print('\nContraste con la derivada real de tanh (1 - h^2):')
print(f"{'Configuracion':>18}  {'media 1-h^2':>11}  {'dist 10 (0.5)':>14}  {'dist 10
(real)':>14}  {'dist 30 (real)':>14}")
for nombre, W in configs.items():
    reales, media = gradiente_real(W)
    print(f"{nombre.split('(')[0].strip():>18}  {media:>11.3f}  {historicos[nombre]
[10]:>14.2e}  {reales[10]:>14.2e}  {reales[30]:>14.2e}")

print()
print('Conclusion:')
print('  Con W_hh pequena o moderada, el gradiente desaparece en pocos pasos,')
print('  tanto con la derivada fija (0.5) como con la real (1 - h^2):')
print('  los tokens lejanos apenas contribuyen al aprendizaje.')
print()
print('  Con escalas mayores, la derivada importa: con 0.5 fijo, el gradiente')
print('  decae (0.3) o EXPLOTA (0.5); con la real, que baja cuando tanh se')
print('  satura, se mantiene en el mismo orden de magnitud a distancia 30.')
print('  Una RNN simple vive en un equilibrio fragil entre ambos extremos.')
print()
print('  Esta es la razon fundamental por la que el Transformer')
print('  supero a los RNNs: la atencion conecta directamente')
print('  cualquier par de tokens sin importar la distancia.')
```

> Salida del programa:

```
=== VANISHING / EXPLODING GRADIENT EN RNN ===
Estado oculto H=32, secuencia T=60

Norma del gradiente a distintas distancias del final:

 Distancia       W_hh pequena      W_hh moderada        W_hh grande    W_hh muy grande
-------------------------------------------------------------------------------------
         0           1.000000           1.000000           1.000000           1.000000
         1           0.130100           0.454545           0.748099           1.706543
         2           0.017290           0.209258           0.566045           2.669450
         5           0.000068           0.018165           0.297183           8.238849
        10           0.000000           0.000482           0.053872          38.145981
        20  < 1e-10  (MUERTO)           0.000000           0.004103        1189.766160
        30  < 1e-10  (MUERTO)           0.000000           0.000358  6.1e+04  (EXPLOTA)
        50  < 1e-10  (MUERTO)  < 1e-10  (MUERTO)           0.000003  1.3e+08  (EXPLOTA)
        60  < 1e-10  (MUERTO)  < 1e-10  (MUERTO)           0.000000  1.1e+10  (EXPLOTA)

Visualizacion del gradiente (W_hh pequena — caso tipico):
Norma del gradiente al retropropagar hacia atras:
  Distancia   0: 1.00e+00  ########################################
  Distancia   5: 6.82e-05   <-- el gradiente es casi NULO
  Distancia  10: 4.94e-09   <-- el gradiente es casi NULO
  Distancia  15: 2.47e-13   <-- el gradiente es casi NULO
  Distancia  20: 1.91e-17   <-- el gradiente es casi NULO
  Distancia  25: 1.87e-21   <-- el gradiente es casi NULO
  Distancia  30: 1.06e-25   <-- el gradiente es casi NULO
  Distancia  35: 6.17e-30   <-- el gradiente es casi NULO
  Distancia  40: 6.74e-34   <-- el gradiente es casi NULO
  Distancia  45: 4.60e-38   <-- el gradiente es casi NULO
  Distancia  50: 2.22e-42   <-- el gradiente es casi NULO
  Distancia  55: 2.29e-46   <-- el gradiente es casi NULO
  Distancia  60: 1.88e-50   <-- el gradiente es casi NULO

Contraste con la derivada real de tanh (1 - h^2):
     Configuracion  media 1-h^2   dist 10 (0.5)  dist 10 (real)  dist 30 (real)
      W_hh pequena        0.818        4.94e-09        5.45e-07        1.85e-19
     W_hh moderada        0.727        4.82e-04        3.58e-02        4.23e-05
       W_hh grande        0.559        5.39e-02        8.38e-01        5.72e-01
   W_hh muy grande        0.311        3.81e+01        1.73e+01        7.73e+01

Conclusion:
  Con W_hh pequena o moderada, el gradiente desaparece en pocos pasos,
  tanto con la derivada fija (0.5) como con la real (1 - h^2):
  los tokens lejanos apenas contribuyen al aprendizaje.

  Con escalas mayores, la derivada importa: con 0.5 fijo, el gradiente
  decae (0.3) o EXPLOTA (0.5); con la real, que baja cuando tanh se
  satura, se mantiene en el mismo orden de magnitud a distancia 30.
  Una RNN simple vive en un equilibrio fragil entre ambos extremos.

  Esta es la razon fundamental por la que el Transformer
  supero a los RNNs: la atencion conecta directamente
  cualquier par de tokens sin importar la distancia.
```

> ℹ️ Con la escala pequeña, la norma del gradiente cae a $7 \cdot 10^{-5}$ en solo 5 pasos hacia atrás y por debajo de 10^{-10} antes de los 20. Esto significa que el error del token 100 prácticamente no modifica los pesos que procesaron el token 95 — y no modifica en absoluto los que procesaron el token 50. En la simulación, que fija la derivada de tanh en 0.5, la RNN vive en un presente muy corto. El contraste del final, con la derivada real $1 - h^2$ en una secuencia de verdad, da el mismo diagnóstico con las escalas pequeña y moderada; con escalas mayores, la saturación de tanh frena tanto la caída como la explosión, y el gradiente queda a merced de ese equilibrio. Fíjate también en las dos últimas columnas: la matriz «grande» amplifica (su mayor valor propio, en módulo, es 1.59), pero el gradiente decae igual, porque en cada paso se multiplica también por la derivada de tanh, que en la simulación vale 0.5 ($1.59 \times 0.5 \approx 0.8 < 1$). Solo con la escala 0.5 (valor propio 3.0; $3.0 \times 0.5 = 1.5 > 1$) el gradiente explota: a 30 pasos ya vale $6 \cdot 10^4$. Entre desvanecerse y explotar hay una franja estrecha, y por eso entrenar RNN largas era tan delicado.

10.5 LSTM — La solución elegante al gradiente

Hochreiter y Schmidhuber propusieron en 1997 las Long Short-Term Memory (LSTM), la arquitectura que dominó el procesamiento de secuencias durante veinte años antes de ser reemplazada por el Transformer. La idea clave: en lugar de un único estado oculto, añadir un estado de celda (cell state) que puede mantener información durante muchos pasos, controlado por tres puertas diferenciables. Cada puerta es una pequeña capa con sigmoide que da números entre 0 (cerrada) y 1 (abierta): la de olvido decide qué parte de la celda se borra; la de entrada, qué información nueva se escribe en ella; y la de salida, qué parte de la celda se muestra como estado oculto h_t en ese paso.

DIAGRAMA 10.2 — LSTM (Long Short-Term Memory)

La solución elegante al vanishing gradient

Tres puertas y un candidato · Estado de celda C_t · Memoria a largo y corto plazo

Hochreiter & Schmidhuber (1997) — dominó el NLP hasta 2017

¿POR QUÉ LSTM?

❌ La RNN clásica falla:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

Un solo estado h_t carga con TODO:
- la memoria a largo plazo
- el contexto inmediato
- la predicción actual

→ Vanishing gradient: gradiente → 0 en pocos pasos
→ El modelo olvida lo que ocurrió hace >20 tokens

💡 La idea clave de LSTM:

Separar la memoria en DOS canales:

C_t — Estado de celda
Memoria a LARGO PLAZO.
Fluye con mínima interferencia.
Es la autopista del gradiente.

h_t — Estado oculto
Memoria a CORTO PLAZO.
Lo que la celda decide mostrar.
Sale hacia la siguiente capa.

🧠 Analogía:

C_t: el "disco duro" de la celda.
Guarda información durante mucho tiempo.

h_t: lo que dices en voz alta.
La respuesta inmediata.

Las 3 puertas deciden:
¿qué borrar? ¿qué añadir? ¿qué emitir?

✅ LSTM resuelve vanishing gradient:

C_{t-1} solo se multiplica por f_t, sin pasar por pesos ni tanh: el gradiente decae menos.
→ Puede aprender dependencias muy largas.

las tres puertas

TRES PUERTAS Y UN CANDIDATO

Las puertas usan σ (sigmoid); el candidato g_t usa tanh

① PUERTA DE OLVIDO (Forget Gate)

¿Qué olvidar del pasado?

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

h_{t-1}, x_t → σ → $f_t \in (0,1)$

Si $f_t \approx 0$ → borra C_{t-1}
Si $f_t \approx 1$ → mantiene C_{t-1}

0 = olvidar todo · 1 = recordar todo

② PUERTA DE ENTRADA Y CANDIDATO

¿Qué información nueva añadir?

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \leftarrow \text{cuánto añadir}$$

$$g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \quad \leftarrow \text{qué añadir}$$

i_t controla la cantidad (0, 1)
g_t controla el contenido (-1, 1)

③ ESTADO DE CELDA (Cell State Update)

La autopista del gradiente

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t$$

Donde ⊙ = multiplicación elemento a elemento (Hadamard)

$f_t \odot C_{t-1}$ ← parte del pasado que se conserva
$i_t \odot g_t$ ← información nueva que se añade

C_{t-1} solo se multiplica por f_t → el gradiente decae menos

④ PUERTA DE SALIDA (Output Gate)

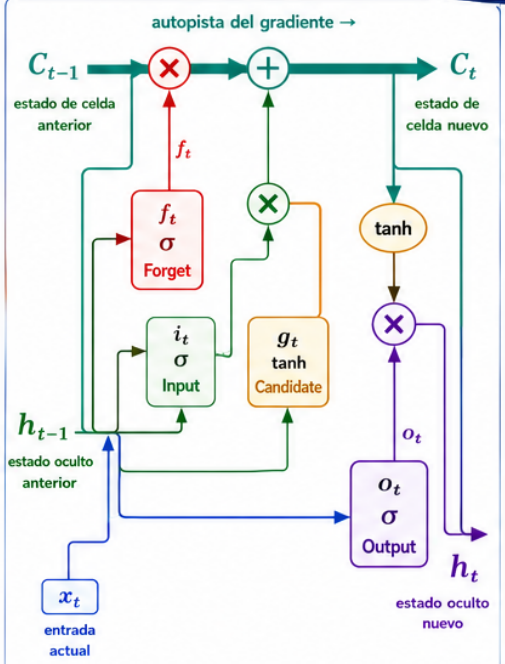
¿Qué emitir como h_t?

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

o_t decide qué parte de C_t exponer
$\tanh(C_t)$ normaliza el estado de celda
h_t es lo que sale hacia la siguiente capa

FLUJO COMPLETO — UNA CELDA LSTM



RNN clásica:

$h_t = \tanh(W \cdot h_{t-1} + W \cdot x_t + b)$

Un solo estado — todo mezclado

Gradiente: se atenúa en cada paso

LSTM:

$C_t = f_t \odot C_{t-1} + i_t \odot g_t$

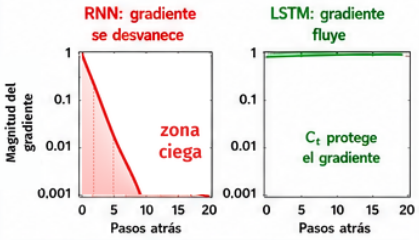
Dos estados — separación de roles

Gradiente: fluye por C_t (solo × f_t)

LSTM en el contexto actual

LSTM EN CONTEXTO

Solución al vanishing gradient:



Variantes de LSTM:

LSTM estándar	3 puertas y un candidato.
GRU (2014)	2 puertas, más simple y casi igual de buena.
Peephole LSTM	Las puertas también ven C_{t-1} directamente.

GRU ≈ LSTM en la práctica, con menos parámetros.

¿Cuándo usar LSTM hoy?

- ✓ Series temporales con dependencias largas
- ✓ Señales de audio y habla
- ✓ Dispositivos con recursos limitados
- ✓ Cuando el Transformer es excesivo
- ✗ NLP moderno de alto rendimiento → usa Transformer (más rápido, más potente)

El legado de LSTM:

1997–2017: LSTM dominó el procesamiento de secuencias en NLP, audio y video.

2017: el Transformer la supera en NLP.

Pero LSTM sigue siendo relevante en dominios donde la recurrencia tiene ventajas prácticas.

20 años de dominio — no es poco.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \leftarrow \text{olvido}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \leftarrow \text{entrada}$$
$$C_t = f_t \odot C_{t-1} + i_t \odot g_t \quad \leftarrow \text{celda}$$
$$h_t = o_t \odot \tanh(C_t) \quad \leftarrow \text{salida}$$

f_t: puerta de olvido

C_t: autopista del gradiente

h_t: salida hacia siguiente capa

Color

Blue	= Input x_t / tokens de entrada	Red	= Puerta de olvido (f_t)
Teal	= Estado de celda C_t (autopista)	Green	= Puerta de entrada (i_t, g_t)
Green	= Output y_t / salida	Purple	= Puerta de salida / h_t
Orange	= Activación sigmoid σ	Gold	= Activación tanh

La puerta de olvido es la innovación central: permite al modelo aprender a descartar información irrelevante del pasado. Cuando procesa el principio de una frase nueva, la puerta de olvido aprende a resetear el estado de celda, eliminando el contexto de la frase anterior. Cuando necesita mantener el sujeto del que se habla, la puerta de olvido aprende a no borrar esa información.

Las LSTMs solucionan parcialmente el vanishing gradient porque el estado de celda C_t fluye directamente de un paso al siguiente, solo multiplicado por la puerta de olvido (un valor entre 0 y 1). Los gradientes pueden viajar por esta 'autopista de gradiente' durante decenas de pasos sin decaer tan abruptamente.

> 💡 No implementamos LSTM desde cero en este capítulo porque, dado que el Transformer las superó en la práctica, sería tiempo mejor invertido en profundizar en la atención. Si quieres explorarlas, PyTorch incluye nn.LSTM con una sola línea. Lo que sí te llevamos es la intuición de las puertas, que reaparece en arquitecturas modernas.

10.6 El salto al Transformer — Por qué ganó

En 2017, Vaswani et al. propusieron reemplazar completamente la recurrencia por el mecanismo de atención. El resultado fue el Transformer que estudiamos en el Capítulo 12. Comparemos directamente ambas arquitecturas:

```
📄 RNN vs Transformer: comparativa técnica
CARACTERISTICA          RNN / LSTM              TRANSFORMER
-----------------------------------------------------------------
Procesamiento           Secuencial (t=1,2,...T) Paralelo (todos a la vez)
Dependencias largas     Dificiles (grad. muere) Directas (O(1) pasos)
Complejidad temporal    O(T * D^2)              O(T^2 * D + T * D^2)
Memoria al generar      O(1) (estado constante) O(T) (KV-cache)
Paralelizable           NO (barrera de datos)   SI (ideal para GPUs)
Rango de contexto       Ilimitado en teoria     La ventana de contexto
Entrenamiento           Lento (secuencial)      Rapido (paralelo)
Interpretabilidad       Caja negra              Atencion visible (pista)
Escalabilidad           Limitada                Excelente

Ventaja del Transformer: al procesar todos los tokens en paralelo,
se aprovecha al 100% la potencia de las GPUs modernas.
En el articulo original, el Transformer supero a los mejores
traductores con una fraccion de su coste de entrenamiento.

Por que los RNNs siguen existiendo:
- Inferencia con memoria constante (O(1) vs O(T))
- Streaming de datos sin limite de contexto
- Sistemas embebidos con memoria muy limitada
- Arquitecturas recurrentes modernas como Mamba (2023)
```

La victoria del Transformer no fue solo cualitativa (mejor manejo de dependencias largas) sino cuantitativa: el paralelismo permite entrenar modelos con billones de tokens en tiempo razonable. Un LSTM procesando un corpus de 1 billón de tokens tiene que hacer 1 billón de pasos secuenciales. El Transformer procesa lotes completos en paralelo, multiplicando la eficiencia por miles.

> ℹ️ La ironía histórica: las LSTMs dominaron durante veinte años parcialmente porque nadie había encontrado la forma de paralelizar secuencias. En cuanto Vaswani et al. mostraron que la atención podía hacer el mismo trabajo sin recurrencia, las LSTMs quedaron desplazadas en la gran mayoría de aplicaciones en pocos años.

10.7 La herencia de las RNNs

Aunque las RNNs ya no son el estado del arte en lenguaje, han dejado una herencia importante en la IA moderna:

* La intuición del estado oculto — el concepto de que la red mantiene una representación interna que resume el contexto pasado sigue presente en los LLMs,

aunque implementado de forma muy diferente (la atención consulta directamente todos los tokens anteriores y, al generar, sus claves y valores se guardan en el KV-cache).

* Las puertas de la LSTM — la idea de controlar con puertas qué información pasa y cuál se descarta reaparece en arquitecturas modernas como Mamba (2023), que intenta combinar la eficiencia de las RNNs con la expresividad de los Transformers.
* Backpropagation Through Time (BPTT) — el algoritmo de entrenamiento de RNNs es una generalización de backpropagation que inspiró técnicas para secuencias muy largas.
* La memoria constante — en dispositivos con poca memoria y en flujos en tiempo real (streaming), las RNN y sus herederas siguen siendo útiles, y las nuevas arquitecturas recurrentes (Mamba, RWKV) recuperan esa ventaja; en voz y música, en cambio, hoy dominan también los Transformers.

La historia de las RNNs es también una lección sobre el progreso en IA: una arquitectura puede dominar durante veinte años y luego ser reemplazada casi completamente en dos. No porque fuera errónea, sino porque aparece una solución que aprovecha mejor el hardware disponible. El Transformer se benefició de la disponibilidad de GPUs masivamente paralelas que simplemente no existían cuando se diseñaron las LSTMs.

Ahora que conoces las RNNs entiendes mejor por qué el Transformer fue revolucionario: no solo resolvió el problema del gradiente que desaparece, sino que transformó el entrenamiento de un proceso inherentemente secuencial en uno que explota el paralelismo masivo de las GPUs modernas. Ese salto tecnológico es el que hizo posible el preentrenamiento sobre billones de tokens y, con él, los LLMs que conocemos hoy.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- Un RNN mantiene un estado oculto que resume la historia de la secuencia procesada hasta el momento.
- El vanishing gradient hace que los RNNs no puedan aprender dependencias de largo alcance de forma fiable.
- El Transformer ganó por paralelismo: procesa todos los tokens a la vez, mientras el RNN los procesa uno a uno.

✏️ EJERCICIOS — Capítulo 10 — Redes Recurrentes

Las redes recurrentes tienen más matices de lo que parece. Resuelve estos ejercicios antes de seguir y, al terminar, compara tu trabajo con la solución para comprobar que no se te escapa nada.

Ejercicio 10.1 Medir el 'horizonte de memoria' efectivo de un RNN

Usando el Programa RNN-2, mide a qué distancia la norma del gradiente cae por debajo de 1e-6 (un umbral razonable de 'gradiente muerto') para tres tamaños de estado oculto: H=8, H=32 y H=128. ¿El tamaño del estado oculto afecta al horizonte de memoria efectivo?

> 💡 💡 Pista: Repite la simulación de gradiente del Programa RNN-2 con distintos valores de H mientras mantienes la misma escala de inicialización de W_hh. El horizonte de memoria depende más de los valores propios de W_hh que del tamaño H en sí.

✅ Solución:

📄 ejercicio_10_1.py

```
import numpy as np

def simular_gradiente(W_hh, T, H):
    grad = np.ones(H) / np.sqrt(H)
    normas = [float(np.linalg.norm(grad))]
    for t in range(T):
        d_tanh = 0.5 * np.ones(H)
        grad = (W_hh.T @ grad) * d_tanh
        normas.append(float(np.linalg.norm(grad)))
    return normas

np.random.seed(0)
T = 60
UMBRAL = 1e-6

print('=== HORIZONTE DE MEMORIA SEGUN TAMAÑO DEL ESTADO OCULTO ===\n')
print(f"{'H (estado oculto)':>18}  {'Horizonte (pasos)':>18}")
print('-' * 40)

for H in [8, 32, 128]:
    W_hh = np.random.randn(H, H) * 0.05   # misma escala para todos
    normas = simular_gradiente(W_hh, T, H)
    horizonte = next((i for i,v in enumerate(normas) if v < UMBRAL), T)
    print(f'{H:>18}  {horizonte:>18}')

print()
print('El horizonte de memoria es similar entre tamaños de H')
print('porque depende de la ESCALA de los pesos (eigenvalues de W_hh),')
print('no tanto del numero de neuronas del estado oculto.')
print('Esto es justamente lo que motivo el diseño de LSTM/GRU:')
print('controlar explicitamente esa escala con puertas aprendidas.')
```

Ejercicio 10.2 Implementar una puerta de olvido simplificada

Implementa una versión simplificada de la puerta de olvido de LSTM: en lugar de las 4 ecuaciones completas, añade solo una puerta f_t = sigmoid(W_f @ [h,x] + b_f) que module cuánto del estado anterior se conserva: h_t = f_t * h_{t-1} + (1-f_t) * tanh(...). Compara su comportamiento con el RNN básico del Programa RNN-1.

> 💡 💡 Pista: Esta es una versión simplificada inspirada en GRU (Gated Recurrent Unit), que usa menos puertas que LSTM pero mantiene la idea central: una puerta aprendida controla cuánta memoria se preserva.

✅ Solución:

📄 ejercicio_10_2.py

```
import numpy as np

np.random.seed(42)

def sigmoid(z): return 1/(1+np.exp(-np.clip(z,-500,500)))

class RNNConPuerta:
    '''RNN con una puerta de olvido simplificada (estilo GRU minimalista)'''
    def __init__(self, D, H):
        self.H = H
        self.W_xh = np.random.randn(H, D) * 0.1
        self.W_hh = np.random.randn(H, H) * 0.1
        self.b_h  = np.zeros(H)
        # Parametros de la puerta de olvido
        self.W_xf = np.random.randn(H, D) * 0.1
        self.W_hf = np.random.randn(H, H) * 0.1
        self.b_f  = np.zeros(H)

    def forward(self, tokens, embeddings):
        h = np.zeros(self.H)
        puertas_historial = []
        for tok in tokens:
            x = embeddings[tok]
            # Puerta de olvido: decide cuanto conservar del pasado
            f = sigmoid(self.W_xf @ x + self.W_hf @ h + self.b_f)
            # Candidato de nuevo estado (como el RNN normal)
            h_candidato = np.tanh(self.W_xh @ x + self.W_hh @ h + self.b_h)
            # Combinar: f decide cuanto pasado conservar
            h = f * h + (1 - f) * h_candidato
            puertas_historial.append(f.mean())
        return h, puertas_historial

# Comparacion con texto simple
V, D, H = 20, 8, 16
emb = np.random.randn(V, D) * 0.1
tokens = np.random.randint(0, V, 30)

red = RNNConPuerta(D, H)
h_final, puertas = red.forward(tokens, emb)

print('=== RNN CON PUERTA DE OLVIDO ===\n')
print(f'Valor medio de la puerta a lo largo de la secuencia:')
for i, p in enumerate(puertas[:10]):
    barra = '#' * int(p * 30)
    print(f'  paso {i:2d}: {p:.3f}  {barra}')
print()
print('f cercano a 1: el modelo conserva mucho el estado anterior')
print('f cercano a 0: el modelo prioriza la nueva informacion')
print('Esta puerta, ENTRENADA, aprenderia cuando "olvidar" el contexto.')
```

Ejercicio 10.3 Cuantificar la ventaja de paralelismo del Transformer

Calcula cuántas operaciones secuenciales (no paralelizables) requiere procesar una secuencia de T=1000 tokens con un RNN frente a un Transformer. Para el RNN, cada paso depende del anterior. Para el Transformer, calcula cuántas capas se necesitarían para tener el mismo 'camino más largo' de dependencia que el RNN.

> 💡 💡 Pista: En un RNN, el camino entre el token 1 y el token T tiene longitud T (secuencial). En un Transformer con self-attention, el camino entre cualquier par de tokens tiene longitud 1 (directo) sin importar T. La 'profundidad' de cómputo paralelizable es lo que marca la diferencia real.

✅ Solución:

📄 ejercicio_10_3.py

```
T = 1000   # longitud de la secuencia

print('=== PASOS SECUENCIALES: RNN vs TRANSFORMER ===\n')
```

```
print('RNN:')
print(f'  Pasos secuenciales necesarios: {T}')
print(f'  (cada paso DEBE esperar al anterior, sin excepcion)')
print(f'  Camino mas largo entre token 1 y token {T}: {T} pasos')
print()

print('Transformer (con self-attention):')
print(f'  Pasos secuenciales necesarios: 1 (todos los tokens en paralelo)')
print(f'  Camino mas largo entre cualquier par de tokens: 1 paso')
print(f'  (la atencion conecta directamente cualquier par)')
print()

# El tiempo real no se deduce de aqui: depende del hardware y del modelo
print('Lo que cambia es la profundidad del camino secuencial, no un tiempo fijo:')
print(f'la RNN encadena {T} pasos que no pueden solaparse; el Transformer')
print('procesa las T posiciones a la vez, a cambio de una atencion que cuesta')
print('O(T^2) en calculo y memoria. Cuanto se gana en tiempo hay que medirlo')
print('en cada maquina, pero el paralelismo es la razon fundamental por la que')
print('se puede entrenar con billones de tokens.')
```

Capítulo 11: El Mecanismo de Atención (***)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar la intuición de Query, Key y Value con la analogía de la biblioteca.
> ✓ Escribir la fórmula completa del self-attention y justificar cada término.
> ✓ Implementar self-attention con y sin máscara causal desde cero.
> ✓ Interpretar una matriz de pesos de atención y verificar sus propiedades.
> ✓ Calcular a mano, con números, un paso completo de atención.
> ✓ Leer los pesos de atención de un modelo ya entrenado.

11.1 El problema del contexto en las secuencias

Imagina que debes traducir la frase 'El banco está cerca del río'. Para traducir correctamente la palabra 'banco', necesitas saber si se refiere a una entidad financiera o a un banco de madera. El contexto —especialmente la palabra 'río'— resuelve la ambigüedad.

Las redes neuronales recurrentes (RNNs) que dominaban el procesamiento de lenguaje antes de 2017 tenían un problema fundamental: el contexto se propagaba secuencialmente y se iba 'diluyendo' con la distancia. Para textos largos, la red tendía a 'olvidar' información importante que había leído hace muchas posiciones.

La solución fue el mecanismo de atención: en lugar de pasar el contexto de izquierda a derecha secuencialmente, permitir que cada posición de la secuencia 'mire' directamente a todas las demás posiciones y decida cuáles son relevantes para su procesamiento.

11.2 La intuición de la atención: Query, Key, Value

El mecanismo de atención usa una analogía de sistema de búsqueda:

* Query (Q): la 'pregunta' que hace una posición. '¿Qué tipo de información necesito?'
* Key (K): la 'etiqueta' de cada posición. 'Esto es lo que yo tengo.'
* Value (V): la información real que contiene cada posición.

Para cada posición, se calcula su puntuación de compatibilidad con todas las demás posiciones comparando su Query con las Keys de todas. Esas puntuaciones se normalizan (softmax) para obtener pesos de atención que suman 1. Finalmente, la salida es la suma ponderada de los Values, usando esos pesos.

> ℹ️ Analogía concreta: imagina que buscas un libro en una biblioteca. Tu pregunta es el Query ('quiero un libro de ficción histórica'). El catálogo de la biblioteca son las Keys ('novela histórica', 'novela negra', 'poesía'). Los libros físicos son los Values. La atención es el proceso de encontrar cuánto se parece tu consulta a cada catálogo y recuperar los libros en proporción a esa similitud.

Un ejemplo a mano: tres palabras

Antes de ver la fórmula general, hagamos el cálculo completo con números pequeños, como se haría en una pizarra. La frase es «el gato duerme» y queremos saber qué información recoge «duerme». Usaremos vectores de solo 2 dimensiones y les daremos un significado a mano para que se entiendan: la primera dimensión vale 1 si la palabra es un sustantivo y la segunda, si es un determinante. En un modelo real nadie elige estos números: los aprende el entrenamiento, y sus dimensiones no tienen un significado tan limpio.

Palabra	Key (lo que ofrece)	Value (la información que aporta)
el	[0, 1] «soy un determinante»	[1, 0]
gato	[1, 0] «soy un sustantivo»	[0, 2]
duerme	[0, 0] «ni una cosa ni otra»	[1, 1]

La Query de «duerme» es [2, 0]: «busco un sustantivo», que es lo que necesita un verbo para saber quién duerme. El cálculo tiene tres pasos:

```
Paso 1. Puntuaciones: producto escalar de la Query con cada Key, dividido entre √2
        el     : (2·0 + 0·1) / 1,414 = 0,000
        gato   : (2·1 + 0·0) / 1,414 = 1,414
        duerme : (2·0 + 0·0) / 1,414 = 0,000

Paso 2. Softmax: e^0 = 1;  e^1,414 = 4,11;  suma = 1 + 4,11 + 1 = 6,11
        pesos = 1/6,11 ; 4,11/6,11 ; 1/6,11  =  0,164 ; 0,673 ; 0,164   (suman 1)

Paso 3. Suma ponderada de los Values
        0,164·[1, 0] + 0,673·[0, 2] + 0,164·[1, 1]  =  [0,327 ; 1,509]
```

El mismo cálculo en NumPy, que confirma los números:

📄 Ejemplo — atención a mano con tres palabras

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max())
    return e / e.sum()

tokens = ['el', 'gato', 'duerme']
# Dos dimensiones con significado: [¿es sustantivo?, ¿es determinante?]
K = np.array([[0.0, 1.0],    # el      -> "soy un determinante"
              [1.0, 0.0],    # gato    -> "soy un sustantivo"
              [0.0, 0.0]])   # duerme  -> "no soy ninguna de las dos"
V = np.array([[1.0, 0.0],    # información que aporta "el"
              [0.0, 2.0],    # información que aporta "gato"
              [1.0, 1.0]])   # información que aporta "duerme"
q = np.array([2.0, 0.0])     # Query de "duerme": "busco un sustantivo"

scores = K @ q / np.sqrt(2)  # producto escalar con cada Key, escalado por √d_k
pesos = softmax(scores)
salida = pesos @ V

np.set_printoptions(precision=3, suppress=True)
for t, s, w in zip(tokens, scores, pesos):
    print(f'{t:7s} score = {s:6.3f}   peso = {w:.3f}')
print('suma de pesos =', round(pesos.sum(), 6))
print('salida de "duerme" =', salida)
```

> Salida del programa:

```
el      score =  0.000   peso = 0.164
gato    score =  1.414   peso = 0.673
duerme  score =  0.000   peso = 0.164
suma de pesos = 1.0
salida de "duerme" = [0.327 1.509]
```

Fíjate en lo que ha pasado: «duerme» dedica el 67 % de su atención a «gato», porque es la Key que mejor encaja con lo que busca, y su nuevo vector, [0.327, 1.509], está dominado por la información de «gato» (el 2 de su segunda dimensión). Después de la atención, «duerme» ya «sabe» quién duerme. Eso es todo lo que hace la atención: decidir a quién escuchar y mezclar lo que dicen.

11.3 Matemática de la self-attention

El término 'self-attention' (auto-atención) indica que las Queries, Keys y Values se derivan todas de la misma secuencia de entrada. La secuencia se 'atiende a sí misma' (Diagrama 11.2).

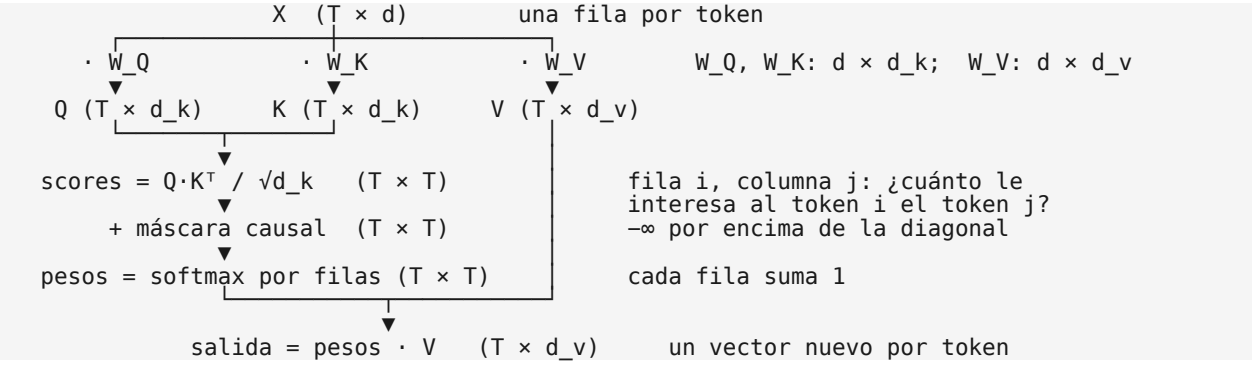
📄 Fórmula del self-attention (Vaswani et al., 2017)

```
Attention(Q, K, V) = softmax(Q @ K.T / sqrt(d_k)) @ V

# Donde:
#   Q = X @ W_Q   (proyección de entradas a Queries)
#   K = X @ W_K   (proyección de entradas a Keys)
#   V = X @ W_V   (proyección de entradas a Values)
```

```
#   d_k = dimensión de las Keys
#   sqrt(d_k): factor de escala para estabilizar gradientes

# Paso a paso:
# 1. scores = Q @ K.T / sqrt(d_k)  -> (T, T) — similaridades
# 2. pesos  = softmax(scores)       -> (T, T) — distribución
# 3. salida = pesos @ V             -> (T, d) — contexto
```

Las formas, paso a paso

Con T tokens de dimensión d, cada paso de la fórmula produce una matriz de forma conocida. Seguir las formas es la mejor manera de no perderse, y de encontrar errores cuando un programa falla:



Q y K deben tener la misma dimensión d_k para poder multiplicarse; V puede tener otra, d_v. En los programas de este libro, como en GPT-2, d_v = d_k. La matriz central, de T × T, es la que dibujan los mapas de atención: tiene una fila por cada token que pregunta (query) y una columna por cada token al que se puede mirar (key). Para «el gato duerme», con T = 3:

```
                  key:  el      gato    duerme
query: el            [ s₁₁     −∞      −∞   ]   «el» solo puede mirarse a sí mismo
query: gato          [ s₂₁     s₂₂     −∞   ]   «gato» mira «el» y a sí mismo
query: duerme        [ s₃₁     s₃₂     s₃₃  ]   «duerme» lo ve todo

s_ij = (Query del token i) · (Key del token j) / √d_k
−∞ = máscara causal: en un modelo que genera texto, el futuro aún no existe
```

DIAGRAMA 11.1 — MECANISMO DE SELF-ATTENTION (AUTOATENCIÓN)

Cómo cada token consulta a todos los demás para entender el contexto

Entrada → Proyecciones → Scores → Escalado → Máscara → Salida

Complejidad: $O(T^2 \cdot d)$ por cabeza — cuadrática en longitud de secuencia

PASO 1: ENTRADA Y PROYECCIONES

El | banco | río

W_Q (Query) | W_K (Key) | W_V (Value)

Q_1	Q_2	Q_3
K_1	K_2	K_3
V_1	V_2	V_3

Consulta: ¿qué busco? | Clave: qué ofrezco | Valor: información real

ANALOGÍA: LA BIBLIOTECA

- Query (Q) = Tu pregunta al bibliotecario
- Key (K) = La etiqueta en el lomo del libro
- Value (V) = El contenido real del libro

La atención mide qué libros (K) responden mejor a tu pregunta (Q), y devuelve su contenido (V).

$$Q = X \cdot W_q \qquad K = X \cdot W_k \qquad V = X \cdot W_v$$

Las 3 proyecciones vienen de la MISMA secuencia. Por eso se llama SELF-attention.

scores = Q @ K^T

PASO 2: PUNTUACIONES DE ATENCIÓN

Matriz de Puntuaciones Brutas ($T \times T$)

	K_1	K_2	K_3
Q_1	$Q_1 \cdot K_1$	$Q_1 \cdot K_2$	$Q_1 \cdot K_3$
Q_2	$Q_2 \cdot K_1$	$Q_2 \cdot K_2$	$Q_2 \cdot K_3$
Q_3	$Q_3 \cdot K_1$	$Q_3 \cdot K_2$	$Q_3 \cdot K_3$

Filas = Queries (Q)
Columnas = Keys (K)

```
scores = Q @ K^T
```

$$\div \sqrt{d_k}$$

Por qué es CRÍTICO el escalado:

- Sin escalar: varianza(QK^T) ≈ d_k → softmax saturada
- Con escalar: varianza ≈ 1 → gradientes estables
- Ejemplo: $d_k = 64$ → dividir por 8 ($\sqrt{64} = 8$)

```
scores = (Q @ K^T) / √d_k
```

softmax($\cdot / \sqrt{d_k}$ + mask)

PASO 3: SOFTMAX Y MÁSCARA

Atención Bidireccional (BERT / Encoder)

Todos los tokens ven a todos — uso en Encoder

	K_1	K_2	K_3	Σ
Q_1	0.24	0.38	0.38	= 1.0 ✓
Q_2	0.19	0.56	0.25	= 1.0 ✓
Q_3	0.21	0.35	0.44	= 1.0 ✓

Cada fila suma exactamente 1.0

MÁSCARA CAUSAL

```
mask = np.triu(-∞, k=1)  →  scores = scores + mask
```

Atención Causal (GPT / Decoder / LLMs generativos)

Solo el pasado: -∞ antes del softmax → 0 después

	K_1	K_2	K_3	Σ	
Q_1	1.00	0	0	= 1.0 ✓	Q_1 → ve solo K_1
Q_2	0.25	0.75	0	= 1.0 ✓	Q_2 → ve K_1, K_2
Q_3	0.21	0.35	0.44	= 1.0 ✓	Q_3 → ve K_1, K_2, K_3

Los LLMs generativos (GPT, LLaMA, Nano-LLM) **SIEMPRE** usan atención causal. Sin ella, el modelo vería tokens futuros → haría trampa.

```
# Python / NumPy (Capítulo 11)
mask   = np.triu(np.full((T,T), -1e9), k=1)
scores = scores + mask   # -∞ → 0 tras softmax
pesos  = softmax(scores, axis=-1)
```

pesos @ V

PASO 4: SALIDA Y MULTI-HEAD ATTENTION

Una cabeza — salida contextualizada

Pesos (causal) × V matrix →

Z_1: Contextual de 'El'
Z_2: Contextual de 'banco'
Z_3: Contextual de 'río'

```
salida = pesos @ V  # forma (T, d)
```

Cada Z_i contiene información de todos los tokens que puede ver según la máscara aplicada.

MULTI-HEAD ATTENTION

H cabezas aprenden en paralelo distintos tipos de relación, por ejemplo:

- Cabeza 1 — relaciones sintácticas
- Cabeza 2 — correferencias
- Cabeza 3 — relaciones semánticas
- Cabeza 4 — posición relativa

Concat → W_O → Z final

Proyección final

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\cdot W^O$$
$$\text{head}_i = \text{Attention}(Q\cdot W_i^Q,\ K\cdot W_i^K,\ V\cdot W_i^V)$$
$$= \text{softmax}(Q_i K_i^T/\sqrt{d_k} + M)\cdot V_i$$

```
d_head = d_model / n_heads
Ej: d=512, n_heads=8  →  d_head=64
```

$$\text{Attention}(Q, K, V) = \text{softmax}(\ QK^T / \sqrt{d_k} + \text{Máscara}\ ) \cdot V$$

- **Q** Consulta: lo que busca cada token
- **K** Clave: lo que ofrece cada token
- **V** Valor: información que se recupera
- $\sqrt{d_k}$ Escalado: estabiliza el gradiente
- Máscara Triangular superior → solo el pasado

Color

Teal	= Query (Q)	Green	= Output (Z)
Purple	= Key (K)	Orange	= Causal mask / warnings
Amber	= Value (V)	Red	= Blocked positions (-∞)

```
d_head = d_model / n_heads
Ej: d=512, n_heads=8  →  d_head=64
```

PROGRAMA 6 — Mecanismo de Self-Attention

📄 **programa_6_atencion.py**

```
import numpy as np

# ============================================================
# MECANISMO DE SELF-ATTENTION PASO A PASO
# ============================================================

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)  # estabilidad
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def self_attention(X, Wq, Wk, Wv, mascara_causal=False):
    """
    Calcula el self-attention de una secuencia.
    X:    (T, d) — T tokens, cada uno de dimensión d
    Wq,Wk,Wv: (d, d) — matrices de proyección aprendibles
    mascara_causal: True en el decoder (no ver el futuro)
    """
    T, d = X.shape
    d_k  = Wk.shape[1]

    # 1. Proyecciones
    Q = X @ Wq   # (T, d) — Queries
    K = X @ Wk   # (T, d) — Keys
    V = X @ Wv   # (T, d) — Values

    # 2. Puntuaciones de similitud (producto escalar escalado)
    scores = (Q @ K.T) / np.sqrt(d_k)   # (T, T)

    # 3. Máscara causal: el token en posición t no puede
    #    ver tokens en posiciones > t (el futuro)
    if mascara_causal:
        mascara = np.triu(np.full((T, T), -1e9), k=1)
        scores  = scores + mascara

    # 4. Softmax -> pesos de atención (cada fila suma 1)
    pesos = softmax(scores, axis=-1)     # (T, T)

    # 5. Suma ponderada de Values
    salida = pesos @ V                   # (T, d)

    return salida, pesos


# -- Demostración con una frase corta -------------------
np.random.seed(3)

# Simulamos 5 tokens con embeddings de dimensión 4
T, d = 5, 4
frase_tokens = ['la', 'red', 'aprende', 'del', 'texto']

# Embeddings (en la práctica vienen de la tabla de embeddings)
X = np.random.randn(T, d) * 0.5

# Matrices de proyección (en la práctica se aprenden)
Wq = np.random.randn(d, d) * 0.3
Wk = np.random.randn(d, d) * 0.3
Wv = np.random.randn(d, d) * 0.3

# Calculamos la atención con y sin máscara causal
sal_full, pesos_full   = self_attention(X, Wq, Wk, Wv, False)
sal_causal, pesos_caus = self_attention(X, Wq, Wk, Wv, True)

np.set_printoptions(precision=3, suppress=True, linewidth=80)

print('=== SELF-ATTENTION ===')
print(f'Secuencia: {frase_tokens}')
print(f'T={T} tokens, dimensión d={d}\n')
```

```
print('Pesos de atención BIDIRECCIONAL (fila=query, col=key):')
print('(cada fila indica cuánto atiende ese token a los demás)')
print(pesos_full.round(3))

print('\nPesos de atención CAUSAL (solo mira el pasado):')
print(pesos_caus.round(3))

print('\nInterpretación del token "aprende" (fila 2):')
for i, (tok, peso) in enumerate(zip(frase_tokens, pesos_caus[2])):
    barra = '' * int(peso * 40)
    print(f'  -> {tok:10s} {peso:.3f}  {barra}')
```

> Salida del programa:

```
=== SELF-ATTENTION ===
Secuencia: ['la', 'red', 'aprende', 'del', 'texto']
T=5 tokens, dimensión d=4

Pesos de atención BIDIRECCIONAL (fila=query, col=key):
(cada fila indica cuánto atiende ese token a los demás)
[[0.175 0.199 0.252 0.171 0.203]
 [0.194 0.206 0.2   0.179 0.22 ]
 [0.222 0.196 0.168 0.227 0.187]
 [0.194 0.19  0.227 0.213 0.176]
 [0.177 0.213 0.21  0.15  0.25 ]]

Pesos de atención CAUSAL (solo mira el pasado):
[[1.    0.    0.    0.    0.   ]
 [0.484 0.516 0.    0.    0.   ]
 [0.379 0.335 0.286 0.    0.   ]
 [0.236 0.231 0.275 0.258 0.   ]
 [0.177 0.213 0.21  0.15  0.25 ]]

Interpretación del token "aprende" (fila 2):
  -> la         0.379  
  -> red        0.335  
  -> aprende    0.286  
  -> del        0.000
  -> texto      0.000
```

En la atención causal (usada en los LLMs generativos), cada token solo puede atender a los tokens anteriores y a sí mismo. Esto garantiza que cuando generamos el token en la posición t, no estamos 'haciendo trampa' mirando el futuro que aún no existe.

11.4 La máscara causal, número a número

La máscara causal no borra nada: suma $-\infty$ (en la práctica, -10^9) a las puntuaciones del futuro antes del softmax. Como e^(−10⁹) es 0 a efectos prácticos, esas posiciones reciben peso cero, y el resto de la fila se reparte el 100 % entre lo que sí se puede ver. Veámoslo con la fila del segundo token de una secuencia de cuatro:

📄 Ejemplo — una fila de atención con y sin máscara

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max())
    return e / e.sum()

scores = np.array([1.2, 0.3, 2.0, -0.5])   # fila del token 2 (de 4): mira a los 4
mascara = np.array([0, 0, -1e9, -1e9])     # pero los tokens 3 y 4 son futuro
np.set_printoptions(precision=3, suppress=True)
print('sin máscara:', softmax(scores))
print('con máscara:', softmax(scores + mascara))
```

> Salida del programa:

```
sin máscara: [0.262 0.107 0.583 0.048]
con máscara: [0.711 0.289 0.    0.   ]
```

Sin máscara, el token 2 dedicaría el 58 % de su atención al token 3, que en un modelo generativo todavía no existe. Con la máscara, ese peso es 0 y los dos tokens visibles se

reparten el total, 0.711 y 0.289, manteniendo su proporción: e^1.2 / e^0.3 ≈ 2.46, igual que 0.711 / 0.289.

11.5 ¿Por qué dividir entre √d_k?

El producto escalar de dos vectores de d componentes es una suma de d términos. Si las componentes son aleatorias de media 0 y varianza 1, la varianza de la suma crece con d: con d = 64, las puntuaciones son típicamente 8 veces mayores que con d = 1. Y el softmax de números grandes es casi un «todo o nada»: toda la atención va a una sola posición, y los gradientes de las demás se anulan. Dividir entre √d_k devuelve la varianza a 1, sea cual sea la dimensión. Compruébalo con vectores al azar y 8 tokens:

📄 Ejemplo — la varianza crece con d; el escalado la corrige

```
import numpy as np

def softmax(x, axis=-1):
    e = np.exp(x - x.max(axis=axis, keepdims=True))
    return e / e.sum(axis=axis, keepdims=True)

rng = np.random.default_rng(0)
T = 8                                   # 8 tokens: 8 puntuaciones por fila
print('   d    varianza de q·k   peso máximo (sin escalar)   peso máximo (÷√d)')
for d in [4, 16, 64, 256, 1024]:
    q = rng.standard_normal((2000, d))    # 2000 queries al azar
    k = rng.standard_normal((2000, T, d)) # con 8 keys cada una
    s = np.einsum('nd,ntd->nt', q, k)     # 2000 filas de 8 puntuaciones
    print(f'{d:5d} {s.var():14.1f} {softmax(s).max(-1).mean():20.3f} '
          f'{softmax(s / np.sqrt(d)).max(-1).mean():22.3f}')
```

> Salida del programa:

```
    d    varianza de q·k   peso máximo (sin escalar)   peso máximo (÷√d)
    4            4.0                0.524                 0.342
   16           16.3                0.750                 0.358
   64           64.1                0.873                 0.356
  256          251.8                0.937                 0.359
 1024         1014.4                0.968                 0.361
```

La varianza de q·k es casi exactamente d. Sin escalar, el peso máximo medio sube del 52 % (d = 4) al 97 % (d = 1024): la atención se vuelve un interruptor. Con el escalado, se mantiene en torno al 35 % para todas las dimensiones. Los LLM reales usan d_k = 64 o 128 por cabeza: sin este factor no aprenderían bien. El Ejercicio 11.2 mide el mismo efecto con la entropía de los pesos.

11.6 El contexto resuelve la ambigüedad: banco y río

Volvamos a la frase del principio del capítulo. La palabra «banco» es ambigua, y la atención es el mecanismo que la desambigua: su vector de salida depende de qué otras palabras hay en la frase. Para verlo, usaremos vectores de 3 dimensiones con un significado elegido a mano, [naturaleza, dinero, palabra gramatical], y la Query de «banco» será «busco pistas de naturaleza o de dinero». Por simplicidad, los Values son iguales a las Keys.

📄 Ejemplo — la misma palabra en dos contextos

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max())
    return e / e.sum()

# Tres dimensiones con significado: [naturaleza, dinero, palabra gramatical]
claves = {'el': [0, 0, 1], 'banco': [0.5, 0.5, 0], 'está': [0, 0, 0.5], 'cerca': [0.2,
0, 0.2],
          'del': [0, 0, 1], 'río': [1, 0, 0], 'me': [0, 0, 1], 'dio': [0, 0, 0.5],
          'un': [0, 0, 1], 'préstamo': [0, 1, 0]}
valores = claves                                  # para simplificar, V = K
q_banco = np.array([2.0, 2.0, 0.0])               # "banco" busca pistas de naturaleza
o de dinero

for frase in ['el banco está cerca del río', 'el banco me dio un préstamo']:
```

```
    toks = frase.split()
    K = np.array([claves[t] for t in toks], dtype=float)
    V = np.array([valores[t] for t in toks], dtype=float)
    pesos = softmax(K @ q_banco / np.sqrt(3))
    salida = pesos @ V
    print(f'«{frase}»')
    for t, w in zip(toks, pesos):
        print(f'   {t:9s} {w:.3f}  ' + '█' * int(w * 40))
    print(f'   nuevo vector de "banco": naturaleza {salida[0]:.2f} | dinero
{salida[1]:.2f}\n')
```

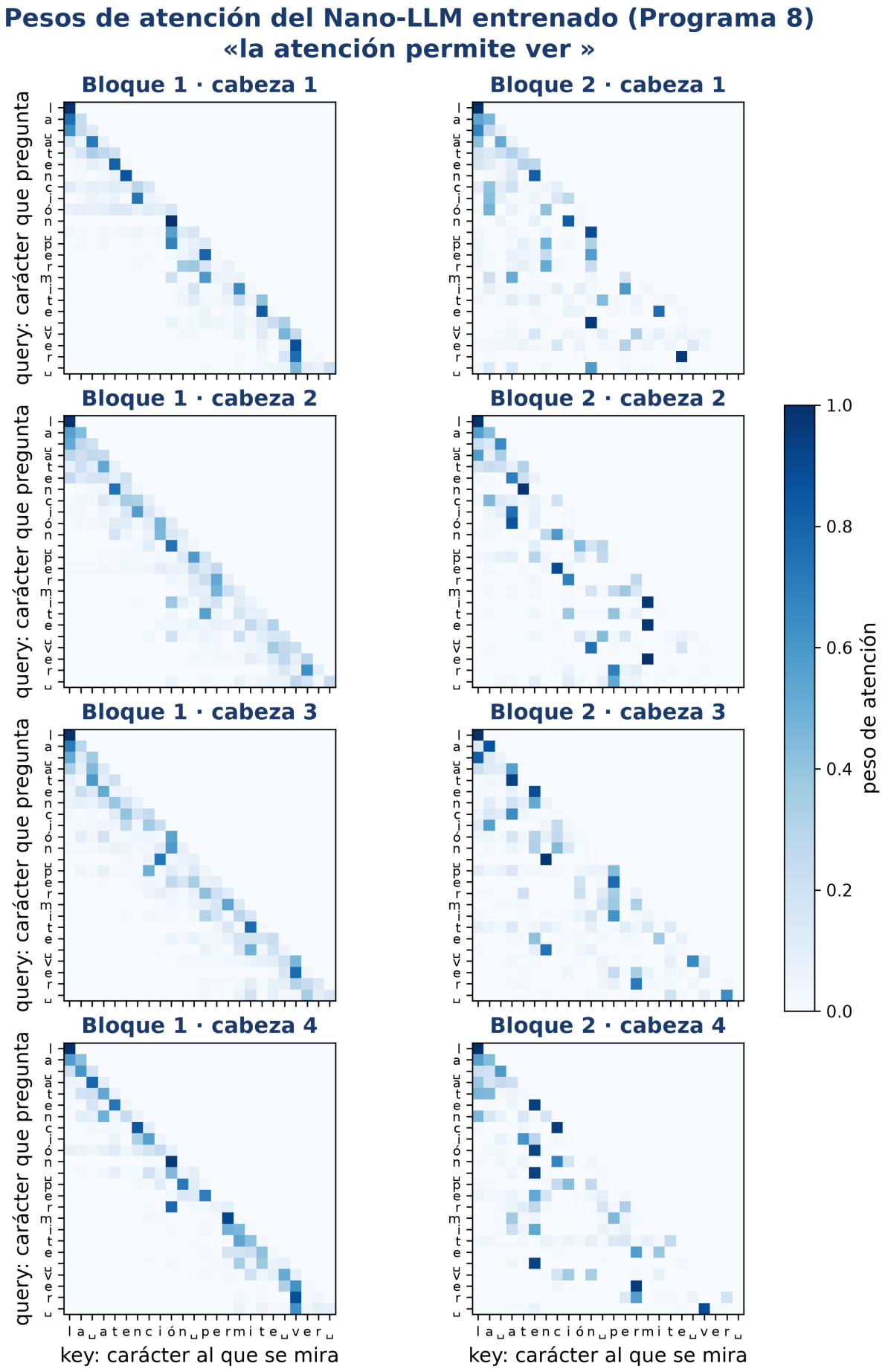
> Salida del programa:

```
«el banco está cerca del río»
   el        0.094  ███
   banco     0.299  ███████████
   está      0.094  ███
   cerca     0.119  ████
   del       0.094  ███
   río       0.299  ███████████
   nuevo vector de "banco": naturaleza 0.47 | dinero 0.15

«el banco me dio un préstamo»
   el        0.097  ███
   banco     0.307  ████████████
   me        0.097  ███
   dio       0.097  ███
   un        0.097  ███
   préstamo  0.307  ████████████
   nuevo vector de "banco": naturaleza 0.15 | dinero 0.46
```

«banco» empieza con el mismo vector en las dos frases, [0.5, 0.5, 0]: mitad naturaleza, mitad dinero. Tras la atención, en la primera frase sale con naturaleza 0.47 y dinero 0.15 (banco de río); en la segunda, al revés, naturaleza 0.15 y dinero 0.46 (entidad financiera). En los dos casos reparte casi un 30 % de su atención entre él mismo y la palabra que da la pista, y apenas atiende a «el», «del» o «un». Es exactamente lo que dibujan las dos gráficas de «banco» del Diagrama 11.2. En un modelo real, el entrenamiento descubre por sí mismo qué dimensiones usar y qué Queries formular para que esto ocurra.

11.7 Así atiende un modelo entrenado

Los ejemplos anteriores usaban números elegidos a mano. ¿Cómo son los pesos de atención de un modelo que los ha aprendido? La figura muestra los del Nano-LLM que construiremos en el Capítulo 13, ya entrenado, al leer la frase «la atención permite ver » (24 caracteres). Ese modelo tiene 2 bloques y, en cada uno, 4 cabezas de atención (12.2), así que hay ocho mapas. En cada uno, cada fila es un carácter que pregunta y cada columna, un carácter al que puede mirar; cuanto más oscuro, más peso. Todo lo que hay por encima de la diagonal es blanco: es la máscara causal.


Pesos de atención del Nano-LLM entrenado (Programa 8)
«la atención permite ver »
Bloque 1 · cabeza 1
Bloque 2 · cabeza 1
Bloque 1 · cabeza 2
Bloque 2 · cabeza 2
Bloque 1 · cabeza 3
Bloque 2 · cabeza 3
Bloque 1 · cabeza 4
Bloque 2 · cabeza 4
query: carácter que pregunta
key: carácter al que se mira
peso de atención
1.0
0.8
0.6
0.4
0.2
0.0

Los dos bloques han aprendido estrategias distintas sin que nadie se las dictara. En el bloque 1 las cuatro cabezas miran cerca: el peso se concentra en la diagonal y en las dos o tres posiciones anteriores. Las cabezas 1 y 4 son las más claras: el carácter más atendido es el inmediatamente anterior en 11 y 14 de las 24 filas, respectivamente. En el bloque 2 las cabezas saltan mucho más lejos, hasta 14 posiciones atrás, y cada una lo hace a su manera; por ejemplo, la «ó» de «atención» pone el 86 % de su atención en la «a» con la que empieza la palabra (cabeza 2) y el 92 % en la «e» (cabeza 4). Ninguna regla les ha dicho que se repartan el trabajo: cada cabeza tiene sus propias matrices, empieza con valores aleatorios distintos y el entrenamiento las lleva a especializarse. Leer estos mapas es una de las pocas ventanas que tenemos al interior de un transformer (Capítulo 19, 19.8). En el Capítulo 15 usaremos esta misma lectura para descubrir cómo suma un modelo.

> ℹ️ La figura se genera con el script figura_7_atencion_nano_llm.py, que se imprime a continuación y carga los pesos que guarda el Programa 8. Si todavía no has llegado al Capítulo 13, vuelve a esta página cuando lo termines y genera tú mismo el mapa de cualquier frase.

📄 figura_7_atencion_nano_llm.py

```
# Figura del Capítulo 11: pesos de atención del Nano-LLM ya entrenado (Programa 8)
# sobre una frase del corpus: 2 bloques × 4 cabezas.
# Requiere nano_llm_pesos.npz (lo guarda el Programa 8).
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec(fuente.split('# -- Learning rate')[0])          # datos, parámetros, forward
params.update(np.load('nano_llm_pesos.npz'))

frase = 'la atención permite ver todo'[:CTX]
tok = np.array([[C2I[c] for c in frase]])           # lote de una frase
T = tok.shape[1]
_, (_, _, cache) = forward(tok)
P = [cache[k]['pesos'][0] for k in range(N_BLQ)]      # cada uno: (H, T, T)

etiq = [c if c != ' ' else '␣' for c in frase]
fig, axes = plt.subplots(H, N_BLQ, figsize=(8, 11.5), dpi=300, layout='constrained')
# una columna por bloque
for k in range(N_BLQ):
    for c in range(H):
        ax = axes[c, k]
        im = ax.imshow(P[k][c], cmap='Blues', vmin=0, vmax=1)
        ax.set_xticks(range(T)); ax.set_xticklabels(etiq if c == H - 1 else [],
fontsize=8)
        ax.set_yticks(range(T)); ax.set_yticklabels(etiq, fontsize=8)
        ax.set_title(f'Bloque {k + 1} · cabeza {c + 1}', fontsize=13, color='#1A3A6B',
fontweight='bold')
        if c == H - 1:
            ax.set_xlabel('key: carácter al que se mira', fontsize=12)
        if k == 0:
            ax.set_ylabel('query: carácter que pregunta', fontsize=12)
fig.suptitle('Pesos de atención del Nano-LLM entrenado (Programa 8)\n«' + frase + '»',
             fontsize=15, color='#1A3A6B', fontweight='bold')
cb = fig.colorbar(im, ax=axes, shrink=0.5)
cb.set_label('peso de atención', fontsize=12); cb.ax.tick_params(labelsize=10)
plt.savefig('figura_7_atencion.png', bbox_inches='tight')

# Resumen numérico para el texto
np.set_printoptions(precision=2, suppress=True)
print('frase:', repr(frase), 'T =', T)
io = frase.index('ó')
for k in range(N_BLQ):
    for c in range(H):
        dist = [i - int(P[k][c][i].argmax()) for i in range(T)]
        print(f'Bloque {k + 1}, cabeza {c + 1}: distancias al más atendido', dist)
        print(f'   peso máximo medio {P[k][c].max(1).mean():.2f} | máximo en el
anterior: {dist.count(1)} de {T} '
              f'| en sí mismo: {dist.count(0)} | fila «ó»:', P[k][c][io, :io + 1])
```

> Salida del programa (además, guarda figura_7_atencion.png):

```
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas
frase: 'la atención permite ver ' T = 24
Bloque 1, cabeza 1: distancias al más atendido [0, 1, 2, 1, 2, 1, 1, 1, 2, 0, 1, 2, 3,
1, 3, 3, 1, 0, 1, 0, 1, 1, 2, 3]
   peso máximo medio 0.63 | máximo en el anterior: 11 de 24 | en sí mismo: 4 | fila
«ó»: [0.07 0.08 0.07 0.09 0.13 0.13 0.07 0.11 0.09 0.16]
Bloque 1, cabeza 2: distancias al más atendido [0, 1, 2, 3, 1, 5, 2, 2, 2, 1, 2, 2, 1,
0, 1, 2, 7, 5, 0, 4, 2, 0, 1, 3]
   peso máximo medio 0.47 | máximo en el anterior: 6 de 24 | en sí mismo: 4 | fila «ó»:
[0.01 0.02 0.03 0.04 0.11 0.11 0.02 0.1  0.47 0.11]
Bloque 1, cabeza 3: distancias al más atendido [0, 1, 2, 1, 2, 2, 2, 2, 1, 0, 1, 3, 5,
2, 2, 1, 4, 1, 0, 3, 0, 1, 2, 2]
   peso máximo medio 0.51 | máximo en el anterior: 7 de 24 | en sí mismo: 4 | fila «ó»:
[0.01 0.09 0.03 0.19 0.02 0.02 0.03 0.04 0.02 0.55]
Bloque 1, cabeza 4: distancias al más atendido [0, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 2, 2,
1, 5, 1, 2, 2, 1, 2, 1, 1, 2, 3]
   peso máximo medio 0.64 | máximo en el anterior: 14 de 24 | en sí mismo: 1 | fila
«ó»: [0.07 0.05 0.04 0.08 0.12 0.04 0.11 0.17 0.24 0.08]
Bloque 2, cabeza 1: distancias al más atendido [0, 1, 2, 1, 1, 1, 1, 6, 7, 8, 2, 1, 6,
3, 8, 12, 3, 6, 2, 9, 6, 8, 4, 13]
   peso máximo medio 0.59 | máximo en el anterior: 6 de 24 | en sí mismo: 1 | fila «ó»:
[0.02 0.48 0.   0.06 0.   0.02 0.4  0.   0.   0.01]
Bloque 2, cabeza 2: distancias al más atendido [0, 1, 0, 3, 0, 2, 2, 6, 5, 6, 3, 2, 7,
6, 6, 2, 1, 5, 3, 8, 10, 6, 10, 11]
   peso máximo medio 0.66 | máximo en el anterior: 2 de 24 | en sí mismo: 3 | fila «ó»:
[0.   0.02 0.02 0.86 0.   0.09 0.   0.   0.01 0.  ]
Bloque 2, cabeza 3: distancias al más atendido [0, 0, 2, 0, 1, 0, 1, 4, 7, 4, 3, 5, 0,
1, 2, 3, 4, 2, 13, 13, 1, 7, 8, 1]
   peso máximo medio 0.61 | máximo en el anterior: 5 de 24 | en sí mismo: 5 | fila «ó»:
[0.04 0.03 0.02 0.16 0.04 0.31 0.08 0.26 0.06 0.01]
Bloque 2, cabeza 4: distancias al más atendido [0, 1, 0, 3, 4, 0, 6, 0, 4, 4, 3, 6, 4,
8, 1, 3, 11, 0, 4, 14, 12, 7, 8, 3]
   peso máximo medio 0.64 | máximo en el anterior: 2 de 24 | en sí mismo: 5 | fila «ó»:
[0.   0.03 0.02 0.01 0.   0.92 0.02 0.   0.   0.  ]
```

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- El self-attention permite que cada token consulte directamente a todos los demás, sin importar la distancia.
- Query, Key y Value se calculan con proyecciones lineales aprendidas; softmax convierte similitudes en pesos.
- La máscara causal garantiza que un token solo pueda atender al pasado, no al futuro — esencial para generar texto.
- Un paso de atención son tres operaciones: puntuar (Query · Key / √d_k), normalizar (softmax) y mezclar (pesos · Value).
- Dividir entre √d_k mantiene la varianza de las puntuaciones en torno a 1 y evita que el softmax se convierta en un interruptor.
- La atención hace que el vector de una palabra dependa de su contexto: así se resuelven ambigüedades como «banco».

DIAGRAMA 11.2 — SELF-ATTENTION: REFERENCIA COMPLETA

Resumen visual del Capítulo 11 — puntuar, normalizar y mezclar

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k} + \text{Máscara}) \cdot V$$

Vaswani et al. (2017) — Attention Is All You Need

1. LA INTUICIÓN: Q, K, V

Query (Q)
= tu pregunta al bibliotecario
"¿qué busco?"

Key (K)
= la etiqueta del lomo del libro
"¿qué ofrezco?"

Value (V)
= el contenido del libro
"la información que aporto"

Las tres proyecciones:

$$Q = X \cdot W_Q$$
$$K = X \cdot W_K$$
$$V = X \cdot W_V$$

Las tres salen de la MISMA secuencia X:
por eso se llama SELF-attention.

2. UN EJEMPLO A MANO: «el gato duerme»

Palabra	Key	Value
el	[0, 1]	[1, 0]
gato	[1, 0]	[0, 2]
duerme	[0, 0]	[1, 1]

Query de «duerme» = [2, 0]
"busco un sustantivo"

1. **Puntuar:** $q \cdot k / \sqrt{2}$ →
   el 0 · gato 1.414 · duerme 0
2. **Softmax:** el 0.164 · gato 0.673 · duerme 0.164 (suman 1 ✓)
3. **Mezclar:** salida de «duerme» = [0.327, 1.509]

«duerme» dedica el 67 % de su atención a «gato».

3. LA MÁSCARA CAUSAL Y EL ESCALADO

Una fila con y sin máscara (token 2 de 4)

scores	1.2	0.3	2.0	−0.5
máscara	0	0	−∞	−∞
sin máscara → pesos	0.262	0.107	0.583	0.048
con máscara → pesos	0.711	0.289	0	0

−∞ antes del softmax → 0 después:
el futuro no recibe atención.

```
mask   = np.triu(np.full((T, T), -1e9), k=1)
pesos  = softmax(scores + mask, axis=-1)
```

¿Por qué dividir entre $\sqrt{d_k}$?

d	varianza de q·k	peso máx. sin escalar	peso máx. ÷√d
16	16.3	0.750	0.358
64	64.1	0.873	0.356
1024	1014.4	0.968	0.361

Sin escalar, la atención se vuelve un interruptor;
con $\sqrt{d_k}$ se mantiene en torno al **36 %**.

4. EL CONTEXTO DESAMBIGUA: «banco»

«el banco está cerca del río»

Nuevo vector de «banco»:
naturaleza 0.47 | dinero 0.15

«el banco me dio un préstamo»

Nuevo vector de «banco»:
naturaleza 0.15 | dinero 0.46

Mismo vector de entrada [0.5, 0.5, 0];
la atención lo inclina hacia el sentido que indica el contexto.

Varias cabezas (Multi-Head)

Cada cabeza tiene sus propias W_Q, W_K y W_V;
sus salidas se concatenan y W_O las mezcla.

```
MultiHead = Concat(head₁, …, head_H) · W_O
d_head = d_model / H
```

El Nano-LLM (Capítulo 13): 4 cabezas de 12 dimensiones
(d = 48). En el Capítulo 15, con 1 cabeza aprende a sumar
en 4 de 6 semillas; con 2, en 6 de 6.

PUNTUAR: $Q \cdot K^T / \sqrt{d_k}$ → **NORMALIZAR:** softmax (+ máscara causal) → **MEZCLAR:** pesos · V

Sistema de colores:
Q (Query) · K (Key) · V (Value) · Pesos de atención · Máscara causal / bloqueado · Comprobaciones

✏️ EJERCICIOS — Capítulo 11 — El Mecanismo de Atención

La atención es el corazón del Transformer, así que merece la pena practicarla a fondo. Trabaja estos ejercicios antes de continuar y contrasta después tu solución con la propuesta que se ofrece.

Ejercicio 11.1 Verificar que los pesos de atención suman 1

Implementa el mecanismo de self-attention del Programa 6 y añade una verificación explícita que compruebe, para cada fila de la matriz de pesos, que la suma es exactamente 1.0 (hasta cierta tolerancia numérica). Verifica también que no hay pesos negativos.

> 💡 💡 Pista: Softmax garantiza matemáticamente que la suma es 1 y todos los valores son positivos. Pero la aritmética de punto flotante puede introducir pequeños errores. Usa np.allclose() para verificar con tolerancia.

✅ Solución:

📄 ejercicio_11_1.py

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

np.random.seed(7)
T, d = 6, 8
X  = np.random.randn(T, d)
Wq = np.random.randn(d, d) * 0.2
Wk = np.random.randn(d, d) * 0.2

Q      = X @ Wq
K      = X @ Wk
scores = Q @ K.T / np.sqrt(d)
pesos  = softmax(scores)

print('=== VERIFICACIÓN DE PESOS DE ATENCIÓN ===\n')
print(f'Matriz de pesos ({T}x{T}):')
np.set_printoptions(precision=4, suppress=True)
print(pesos)

print('\nVerificaciones:')
sumas = pesos.sum(axis=1)
print(f'  Todas las filas suman 1: {np.allclose(sumas, 1.0)}')
print(f'  Todos los pesos >= 0:    {(pesos >= 0).all()}')
print(f'  Suma mínima de fila:  {sumas.min():.10f}')
print(f'  Suma máxima de fila:  {sumas.max():.10f}')
print(f'  Peso mínimo: {pesos.min():.2e}  (positivo por softmax)')
```

Ejercicio 11.2 ¿Qué ocurre sin el factor de escala sqrt(d_k)?

Compara el self-attention con y sin el factor de escala $1/\sqrt{d}$. Usa $d = 64$ (un tamaño realista). Calcula la varianza de los scores y la entropía de los pesos de atención en ambos casos. ¿Qué problema introduce no escalar?

> 💡 💡 Pista: Sin escalado, los productos $Q \cdot K^T$ tienen varianza $\approx d$ (en lugar de ≈ 1). Esto hace que softmax produzca distribuciones muy 'puntiagudas' (casi toda la probabilidad en un solo token), lo que equivale a gradientes muy pequeños.

✅ Solución:

📄 ejercicio_11_2.py

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x); return e / e.sum(axis=axis, keepdims=True)
```

```
def entropia(pesos):
    """Entropia media de las filas (bits). Mayor = más distribuida."""
    return -(pesos * np.log2(pesos + 1e-10)).sum(axis=1).mean()

np.random.seed(0)
T, d = 8, 64
X  = np.random.randn(T, d)
Wq = np.random.randn(d, d) * 0.02
Wk = np.random.randn(d, d) * 0.02
Q  = X @ Wq; K = X @ Wk
scores_raw    = Q @ K.T
scores_scaled = Q @ K.T / np.sqrt(d)
pesos_raw    = softmax(scores_raw)
pesos_scaled = softmax(scores_scaled)

print(f'Dimensión d={d}, T={T} tokens\n')
print(f"{'Métrica':>35}  {'Sin escala':>12}  {'Con escala':>12}")
print('-' * 62)
print(f"{'Varianza de scores':>35}  {scores_raw.var():>12.4f}
{scores_scaled.var():>12.4f}")
print(f"{'Max peso de atención':>35}  {pesos_raw.max():>12.4f}
{pesos_scaled.max():>12.4f}")
print(f"{'Entropía media (bits)':>35}  {entropia(pesos_raw):>12.4f}
{entropia(pesos_scaled):>12.4f}")
print(f"{'Máxima entropía posible':>35}  {np.log2(T):>12.4f}  {np.log2(T):>12.4f}")
print()
print('Sin escala: scores de alta varianza -> softmax muy puntuda')
print('-> gradientes casi nulos -> el modelo no aprende')
```

Ejercicio 11.3 Multi-Head Attention: varias perspectivas a la vez

Implementa Multi-Head Attention con 4 cabezas sobre una secuencia de T=6 tokens con d=16 dimensiones. Cada cabeza trabaja con dimensión d_head = d/n_heads = 4. Muestra que la salida tiene la misma forma que la entrada (T, d).

> 💡 💡 Pista: Divide d entre n_heads para obtener d_head. Cada cabeza tiene sus propias matrices Wq, Wk, Wv de tamaño (d, d_head). Las salidas de todas las cabezas se concatenan para recuperar la dimensión d; en un Transformer completo, una última matriz W_O (d × d) las mezcla.

✅ Solución:

📄 **ejercicio_11_3.py**

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)

np.random.seed(3)
T, d, n_heads = 6, 16, 4
d_head = d // n_heads   # 4
X = np.random.randn(T, d)

salidas_cabezas = []
for h in range(n_heads):
    Wq_h = np.random.randn(d, d_head) * 0.02
    Wk_h = np.random.randn(d, d_head) * 0.02
    Wv_h = np.random.randn(d, d_head) * 0.02
    Q = X @ Wq_h; K = X @ Wk_h; V = X @ Wv_h
    scores = Q @ K.T / np.sqrt(d_head)
    msk    = np.triu(np.full((T,T), -1e9), k=1)
    pesos  = softmax(scores + msk)
    sal_h  = pesos @ V              # (T, d_head)
    salidas_cabezas.append(sal_h)

# Concatenar cabezas: (T, n_heads*d_head) = (T, d)
concat = np.concatenate(salidas_cabezas, axis=-1)   # (6, 16)
# Proyección final
Wo     = np.random.randn(d, d) * 0.02
```

```
salida = concat @ Wo                                    # (6, 16)

print(f'Multi-Head Attention: {n_heads} cabezas x d_head={d_head}')
print(f'Entrada:  {X.shape} -> Salida: {salida.shape}')
print(f'(la forma se mantiene — condición del bloque transformer)')
params_atencion = n_heads * 3*d*d_head + d*d
print(f'Parámetros: {n_heads}x3x{d}x{d_head} + {d}x{d} = {params_atencion}')
```

En el Capítulo 15 verás que pasar de 1 a 2 cabezas puede decidir si un modelo pequeño aprende a sumar: con 8000 sumas de entrenamiento, aprenden 4 de cada 6 semillas con una cabeza y las 6 con dos (sección 15.3).

Ejercicio 11.4 Calcula a mano la atención de «gato»

Con las mismas Keys y Values de la sección 11.2 («el gato duerme»), calcula a mano los pesos y la salida de «gato» si su Query es [0, 2] («busco mi determinante»). Repite el cálculo con máscara causal. Después comprueba tus números con NumPy.

> 💡💡 Pista: Las puntuaciones son el producto escalar de [0, 2] con cada Key, dividido entre √2. Con máscara causal, «gato» (posición 2) no puede ver «duerme» (posición 3): su peso es 0 y los otros dos se reparten el total.

✅ **Solución:**

📄 **ejercicio_11_4.py**

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max())
    return e / e.sum()

tokens = ['el', 'gato', 'duerme']
K = np.array([[0.0, 1.0], [1.0, 0.0], [0.0, 0.0]])
V = np.array([[1.0, 0.0], [0.0, 2.0], [1.0, 1.0]])
q_gato = np.array([0.0, 2.0])          # "gato" busca su determinante

np.set_printoptions(precision=3, suppress=True)
scores = K @ q_gato / np.sqrt(2)
print('scores           :', scores)
print('pesos            :', softmax(scores))
print('salida           :', softmax(scores) @ V)
# Con máscara causal, "gato" (posición 2) no puede ver "duerme" (posición 3)
pesos_c = softmax(scores + np.array([0, 0, -1e9]))
print('pesos (causal)   :', pesos_c)
print('salida (causal)  :', pesos_c @ V)
```

> Salida del programa:

```
scores           : [1.414 0.    0.   ]
pesos            : [0.673 0.164 0.164]
salida           : [0.836 0.491]
pesos (causal)   : [0.804 0.196 0.   ]
salida (causal)  : [0.804 0.391]
```

A mano: las puntuaciones son 1.414 para «el» y 0 para las otras dos; el softmax da 4.11/6.11 = 0.673 para «el» y 0.164 para cada una de las demás. La salida es 0.673·[1, 0] + 0.164·[0, 2] + 0.164·[1, 1] = [0.836, 0.491]. Con máscara causal solo quedan «el» y «gato»: 4.11/5.11 = 0.804 y 1/5.11 = 0.196, y la salida es [0.804, 0.391]. Al ocultar «duerme», «gato» pierde parte de la primera dimensión que aportaba ese Value.

Capítulo 12: El Transformer — La Arquitectura que lo Cambió Todo (***)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar por qué el Transformer prescinde de la recurrencia.
> ✓ Describir el papel de la codificación posicional y el Multi-Head Attention.
> ✓ Justificar la necesidad de las conexiones residuales y LayerNorm.
> ✓ Implementar un bloque Transformer completo y verificar sus propiedades numéricas.

12.1 'Attention is All You Need' — 2017

En junio de 2017, ocho investigadores de Google publicaron un artículo con un título provocador: 'Attention is All You Need'. Proponían una nueva arquitectura llamada Transformer que prescindía completamente de las redes recurrentes (RNNs) y se basaba exclusivamente en mecanismos de atención.

La arquitectura fue revolucionaria por varios motivos. Primero, permitía procesar todos los tokens de una secuencia en paralelo (no uno a uno como las RNNs), lo que la hacía mucho más eficiente para entrenar en GPUs. Segundo, el mecanismo de atención capturaba dependencias de largo alcance directamente, sin el problema del olvido gradual de las RNNs. Tercero, era sorprendentemente escalable: al aumentar el tamaño, las capacidades del modelo resultaron mejorar de manera predecible (lo midieron después las leyes de escala de Kaplan et al., 2020).

Hoy, el Transformer es la base de prácticamente todos los modelos de lenguaje de alto rendimiento: GPT, Claude, Gemini, LLaMA, BERT y docenas más.

12.2 Componentes del Transformer

Codificación posicional (Positional Encoding)

El self-attention, tal como lo hemos visto, es 'ciego' al orden: si desordenamos la frase, cada token obtiene exactamente la misma salida que antes, solo que en otra posición. Para la atención, «el perro muerde al hombre» y «el hombre muerde al perro» serían la misma bolsa de palabras. Para que el modelo conozca el orden, añadimos a cada embedding una codificación posicional.

El Transformer original usa una codificación sinusoidal. Para la posición pos y cada pareja de dimensiones (2k, 2k + 1) del embedding, PE(pos, 2k) = sen(pos / 10000^(2k/d)) y PE(pos, 2k + 1) = cos(pos / 10000^(2k/d)), donde d es la dimensión del modelo. Cada pareja de dimensiones es una onda de distinta frecuencia: en las primeras, la onda es muy rápida y distingue posiciones vecinas; en las últimas, muy lenta, y distingue posiciones lejanas. Juntas dan a cada posición una huella distinta, como las agujas de un reloj (segundos, minutos y horas). El código siguiente calcula exactamente esa fórmula:

📄 **Positional Encoding sinusoidal**

```
def positional_encoding(T, d):
    """
    Genera una matriz (T, d) de codificaciones posicionales.
    Cada posición tiene un 'sello' vectorial único basado en
    funciones sinusoidales de distintas frecuencias.
    """
    PE = np.zeros((T, d))
    for pos in range(T):
        for i in range(0, d, 2):
            angulo = pos / (10000 ** (i / d))
            PE[pos, i]   = np.sin(angulo)   # dimensiones pares
            if i + 1 < d:
                PE[pos, i+1] = np.cos(angulo)  # dimensiones impares
    return PE
```

```
# La codificación posicional se suma al embedding:
# X_total = embedding(token) + positional_encoding(posicion)
```

Por qué los LLMs actuales ya no usan esta versión: RoPE

El positional encoding sinusoidal que acabas de implementar es el del Transformer original de 2017, pero no el que usa nuestro Nano-LLM: el del Capítulo 13, como GPT-2, aprende la posición con una tabla de parámetros (wpe). Y la mayoría de los LLM modernos no usan ninguno de los dos —LLaMA, Mistral, Qwen, GPT-NeoX y la mayoría de modelos open-source actuales usan RoPE (Rotary Position Embeddings, Su et al., 2021) en su lugar. Merece la pena entender por qué, aunque no lo implementemos: es un ejemplo perfecto de cómo un detalle aparentemente técnico resuelve un problema muy práctico.

El problema del encoding sinusoidal es que la posición se suma al embedding una sola vez, al principio, y a partir de ahí se mezcla con el contenido del token de forma indistinguible: tras varias capas de atención, ya no hay una separación limpia entre 'esto es contenido' y 'esto es posición'. Además, un modelo entrenado con secuencias de longitud 512 no generaliza bien a secuencias de longitud 4000: nunca ha visto esas posiciones durante el entrenamiento, y sus codificaciones sinusoidales para posiciones lejanas son, en la práctica, extrapolación no vista.

RoPE resuelve esto de una forma distinta: en vez de sumar un vector de posición al embedding, rota los vectores de consulta (Q) y clave (K) —los mismos que ya conoces de la sección 11.3— un ángulo que depende de la posición del token, antes de calcular el producto escalar de la atención. La idea clave es que, al calcular Q·K entre dos tokens, lo único que sobrevive a la rotación es la diferencia de ángulo entre ellos —es decir, la distancia relativa entre posiciones—, no la posición absoluta de cada uno. Esto tiene dos consecuencias prácticas: la atención se vuelve sensible a 'cómo de lejos está un token de otro' de forma mucho más directa que con la suma sinusoidal, y, como lo aprendido son relaciones de distancia relativa y no posiciones absolutas, con técnicas de reescalado de los ángulos se puede ampliar la ventana de contexto más allá de la usada en el entrenamiento.

> ℹ️ No es casualidad que RoPE y el resto de mejoras de los LLM recientes —RMSNorm (una LayerNorm que no resta la media), SwiGLU (una FFN con compuerta y activación SiLU) y GQA (cabezas que comparten K y V, sección 17.5) — aparezcan juntas en la mayoría de LLMs modernos: son piezas independientes, cada una sustituye a una pieza equivalente y más simple del Transformer original de 2017, sin cambiar la estructura general de bloques de atención + FFN que ya conoces. Entender el Transformer 'clásico' a fondo, como acabas de hacer, es precisamente lo que te permite entender estas variantes como mejoras incrementales en vez de como arquitecturas nuevas que aprender desde cero.

Multi-Head Attention

En lugar de calcular la atención una sola vez, el Transformer calcula varias atenciones en paralelo ('cabezas'), cada una con sus propias matrices W_Q, W_K, W_V. Esto permite que el modelo aprenda simultáneamente distintos tipos de relaciones entre tokens (por ejemplo, una cabeza puede especializarse en concordancia gramatical, otra en relaciones semánticas, otra en correferencias, etc.). Para que eso no multiplique el coste, cada cabeza trabaja en un espacio más pequeño: con n cabezas, sus Q, K y V tienen dimensión d_k = d / n. En el Nano-LLM (Capítulo 13), con d = 48 y 4 cabezas, cada una usa 12 dimensiones; así, las 4 cabezas juntas tienen los mismos parámetros y casi el mismo coste que una sola cabeza de dimensión 48. El Programa 7, a continuación, simplifica y usa una sola cabeza (de dimensión d); las cabezas separadas llegan con el Programa 8.

Las salidas de todas las cabezas se concatenan y se proyectan con otra matriz W_O para producir la salida final del bloque de atención.

Conexiones residuales y normalización

El Transformer usa dos técnicas de estabilización del entrenamiento que son cruciales para redes muy profundas:

* Conexiones residuales (skip connections): la salida de cada subcapa se suma a su entrada original: X_salida = X_entrada + subcapa(X_entrada). Esto permite que el gradiente fluya directamente hacia atrás sin atenuarse en capas profundas.
* Layer Normalization: normaliza las activaciones dentro de cada muestra (no entre muestras). Estabiliza los valores y acelera el entrenamiento.

Post-LN frente a Pre-LN. El Transformer original de 2017 normaliza *después* de cada suma residual, `X = LayerNorm(X + subcapa(X))` (Post-LN). GPT-2 y casi todos los LLM actuales normalizan *antes* de cada subcapa, `X = X + subcapa(LayerNorm(X))` (Pre-LN): la vía residual queda libre de normalizaciones, el gradiente la recorre sin obstáculos y el entrenamiento es más estable en redes profundas (Xiong et al., 2020). Nuestro Programa 7 y el Nano-LLM del Capítulo 13 usan Pre-LN, como GPT-2, y su LayerNorm tiene dos vectores de parámetros aprendibles: γ (escala) y β (desplazamiento), con los que el modelo puede reescalar la salida normalizada si le conviene.

Capa Feed-Forward

Después de la atención, cada posición pasa por una pequeña red densa de dos capas que se aplica de forma independiente e idéntica a cada posición:

📄 **Sublayer Feed-Forward del Transformer**

```
FFN(x) = ReLU(x @ W1 + b1) @ W2 + b2

# Dimensiones típicas:
# d_model = dimensión del modelo (ej: 512)
# d_ff    = dimensión interna (ej: 2048 — 4x más grande)

# W1: (d_model, d_ff)   W2: (d_ff, d_model)

# Esta subcapa procesa cada token de forma INDEPENDIENTE
# (a diferencia de la atención, que mezcla tokens).
# Se cree que guarda buena parte de lo 'memorizado'.
```

GPT-2, y con él nuestro Programa 7 y el Nano-LLM, usa en la FFN la activación **GELU** en lugar de ReLU: `gelu(x) = 0.5·x·(1 + tanh(√(2/π)·(x + 0.044715·x³)))`. Su forma se parece a la de ReLU (≈ x para valores positivos grandes, ≈ 0 para negativos grandes), pero es suave: no tiene esquina en 0 y deja pasar un poco de los valores ligeramente negativos.

12.3 El bloque Transformer completo

PROGRAMA 7 — Bloque Transformer

📄 **programa_7_transformer_bloque.py**

```
import numpy as np

# ============================================================
# UN BLOQUE TRANSFORMER COMPLETO (diseño de GPT-2: Pre-LN)
# (versión simplificada sin Multi-Head para claridad)
# ============================================================

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)


def layer_norm(x, g, b, eps=1e-5):
    """Normaliza cada fila y aplica escala g (γ) y desplazamiento b (β)"""
    mu  = x.mean(axis=-1, keepdims=True)
    var = x.var(axis=-1, keepdims=True)
    return (x - mu) / np.sqrt(var + eps) * g + b

def gelu(x):
    """La activación de GPT-2: una ReLU 'suave'"""
    return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715 * x**3)))
```

```
class BloqueTransformer:
    """
    Un bloque del Transformer decoder, como en GPT-2 (Pre-LN):
    1. LayerNorm -> self-attention causal -> suma residual
    2. LayerNorm -> Feed-Forward (GELU)   -> suma residual
    """

    def __init__(self, d, d_ff=None):
        d_ff = d_ff or d * 4
        # Dos LayerNorm con escala (γ) y desplazamiento (β)
        self.g1, self.b_ln1 = np.ones(d), np.zeros(d)
        self.g2, self.b_ln2 = np.ones(d), np.zeros(d)
        # Matrices de atención, cada una con su sesgo
        self.Wq = np.random.randn(d, d) * 0.02;  self.bq = np.zeros(d)
        self.Wk = np.random.randn(d, d) * 0.02;  self.bk = np.zeros(d)
        self.Wv = np.random.randn(d, d) * 0.02;  self.bv = np.zeros(d)
        self.Wo = np.random.randn(d, d) * 0.02;  self.bo = np.zeros(d)
        # Matrices FFN
        self.W1 = np.random.randn(d, d_ff) * 0.02
        self.b1 = np.zeros(d_ff)
        self.W2 = np.random.randn(d_ff, d) * 0.02
        self.b2 = np.zeros(d)

    def atencion_causal(self, X):
        T, d = X.shape
        Q = X @ self.Wq + self.bq
        K = X @ self.Wk + self.bk
        V = X @ self.Wv + self.bv
        scores = Q @ K.T / np.sqrt(d)
        # Máscara causal: -inf en el triángulo superior
        mascara = np.triu(np.full((T, T), -1e9), k=1)
        pesos   = softmax(scores + mascara)
        return (pesos @ V) @ self.Wo + self.bo

    def ffn(self, X):
        h = gelu(X @ self.W1 + self.b1)
        return h @ self.W2 + self.b2

    def forward(self, X):
        # Sub-bloque 1: norm -> self-attention -> residual
        X = X + self.atencion_causal(layer_norm(X, self.g1, self.b_ln1))
        # Sub-bloque 2: norm -> FFN -> residual
        X = X + self.ffn(layer_norm(X, self.g2, self.b_ln2))
        return X


# -- Demostración ----------------------------------------
np.random.seed(5)
T, d = 6, 16    # 6 tokens, dimensión 16

bloque = BloqueTransformer(d=16, d_ff=64)

# Entrada: 6 tokens aleatorios
X_entrada = np.random.randn(T, d)
X_salida  = bloque.forward(X_entrada)

print('=== BLOQUE TRANSFORMER ===')
print(f'Entrada: {T} tokens x {d} dimensiones')
print(f'Salida:  {X_salida.shape[0]} tokens x {X_salida.shape[1]} dimensiones')
print(f'El bloque mantiene las dimensiones (T={T}, d={d})')


# Contar parámetros: atención (4 matrices + 4 sesgos), FFN, 2 LayerNorm
n_params = (4*d*d + 4*d) + (2 * d*(d*4) + d*4 + d) + 2 * 2*d
print(f'Parámetros del bloque: {n_params}')


# Pre-LN: la salida es la entrada MÁS lo que aportan las subcapas
cambio = X_salida - X_entrada
print(f'\nLa vía residual conserva la entrada:')
print(f'  Std de la entrada:        {X_entrada.std():.4f}')
```

```
    print(f'  Std de lo que se suma:    {cambio.std():.4f}')
    print(f'  Std de la salida:         {X_salida.std():.4f}')
```

> Salida del programa:

```
=== BLOQUE TRANSFORMER ===
Entrada: 6 tokens x 16 dimensiones
Salida:  6 tokens x 16 dimensiones
El bloque mantiene las dimensiones (T=6, d=16)
Parámetros del bloque: 3280

La vía residual conserva la entrada:
  Std de la entrada:        1.1279
  Std de lo que se suma:    0.0061
  Std de la salida:         1.1264
```

El bloque conserva la forma (6 × 16) y tiene 3280 parámetros: los 3152 de las matrices y sesgos de la FFN y de la atención sin sesgo, más los 64 de los sesgos de Q, K, V y Wo y los 64 de γ y β de las dos LayerNorm. Fíjate en las tres últimas líneas: con los pesos recién iniciados (desviación 0.02), lo que las dos subcapas suman a la vía residual es diminuto (desviación 0.006) y la salida es casi idéntica a la entrada. Con Pre-LN, un bloque recién creado empieza siendo casi la identidad, y el entrenamiento le enseña qué correcciones sumar. Por eso la salida ya no tiene media 0 y desviación 1: la normalización está dentro de cada subcapa, no en la vía residual.

> ℹ️ Un LLM moderno apila decenas o cientos de estos bloques. GPT-3 tiene 96 bloques con d=12 288. Cada bloque refina progresivamente la representación de los tokens, añadiendo más contexto y razonamiento en cada nivel. Se suele decir que las capas bajas capturan sintaxis y las altas, semántica y razonamiento; es una descripción aproximada, observada en modelos grandes. En el Capítulo 15 (15.3) verás, leyendo su atención, qué hace realmente cada bloque de un modelo pequeño en una tarea concreta.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - El Transformer combina self-attention, Feed-Forward, conexiones residuales y LayerNorm en bloques apilables.
> - La codificación posicional aporta información de orden, que la atención por sí sola no captura.
> - Multi-Head Attention permite que el modelo aprenda simultáneamente distintos tipos de relación entre tokens.

DIAGRAMA 12.1 — EL BLOQUE TRANSFORMER

Resumen visual del Capítulo 12 — la arquitectura que lo cambió todo

LayerNorm → Atención causal → suma residual · LayerNorm → FFN (GELU) → suma residual · Apilado N veces

Diseño Pre-LN de GPT-2 — el mismo del Programa 7 y del Nano-LLM

1. ¿POR QUÉ ESTE DISEÑO?

La atención sola no basta

Mezcla información ENTRE tokens, pero no transforma cada token.

Multi-Head Attention

Cada token consulta a los anteriores (máscara causal). Varias cabezas, varios tipos de relación.

Feed-Forward (FFN)

Red de dos capas aplicada a cada posición por separado: transforma lo que la atención ha reunido.

Residual + LayerNorm

x + subcapa(LayerNorm(x)): la vía residual queda libre y el gradiente la recorre sin atenuarse.

2. UN BLOQUE (PRE-LN, COMO GPT-2)

1. **Entrada X** (T × d)
2. x = wte[tokens] + wpe[posiciones] (posición APRENDIDA)
3. **LayerNorm 1** (γ, β aprendibles)
4. **Atención causal multi-cabeza:** Q = x·W_Q, K = x·W_K, V = x·W_V; softmax(QKᵀ/√d_k + máscara)·V · W_O
5. x = x + MHA(LayerNorm(x))
6. **LayerNorm 2**
7. FFN(x) = GELU(x·W₁ + b₁)·W₂ + b₂ — W₁: d × 4d, W₂: 4d × d
8. x = x + FFN(LayerNorm(x))
9. **Salida del bloque** (T×d) — la misma forma que la entrada

× N bloques

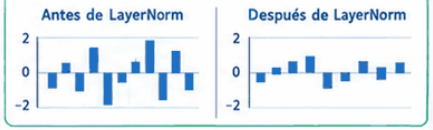
LayerNorm final → logits = x · wteᵀ (la misma tabla de embeddings, compartida) → softmax

3. LAS PIEZAS, DE CERCA

LayerNorm

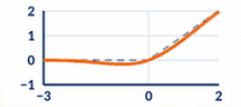
$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sqrt{\sigma^2 + \varepsilon} + \beta$$

(por fila: cada token por separado)



GELU

gelu(x) = 0.5·x·(1 + tanh(√(2/π)·(x + 0.044715·x³)))



suave, sin esquina en 0

Post-LN frente a Pre-LN

Transformer original (2017), Post-LN:

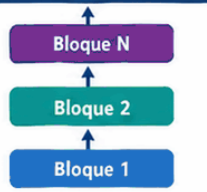
x = LayerNorm(x + subcapa(x))

GPT-2 y casi todos los LLM actuales, **Pre-LN:**

x = x + subcapa(LayerNorm(x)) ← el de este libro

Los LLM recientes cambian piezas, no la idea: RMSNorm, SwiGLU, RoPE, GQA.

4. APILADO Y ESCALA



Cada bloque refina la representación.

Arquitecturas reales

Modelo	Bloques	d	Parámetros
Programa 7	1	16	3280
Nano-LLM	2	48	60 000
GPT-2 small	12	768	124 M
GPT-3	96	12 288	175 000 M

Parámetros de un bloque ($d_{ff} = 4d$)

Atención: $4 \cdot d^2 + 4 \cdot d$ FFN: $8 \cdot d^2 + 5 \cdot d$

LayerNorm: $4 \cdot d$

≈ $12 \cdot d^2$ por bloque

Nano-LLM (d = 48): 28 272 por bloque

GPT-2 small (d = 768): 7 087 872 por bloque

GPT-2 small: 12 bloques ≈ 85 M + embeddings 39 M = 124 439 808

```
x = x + MHA(LayerNorm(x))   ← subcapa 1
x = x + FFN(LayerNorm(x))   ← subcapa 2
params ≈ 12 · N · d²        ← regla práctica (sin embeddings)
```

Atención: relaciones entre tokens → **FFN:** transformación por posición → **Residual + LayerNorm:** estabilidad y gradientes

Sistema de colores: Input / Embeddings · Multi-Head Attention · Feed-Forward (FFN) · LayerNorm · Residual / Conexión · Salida · Parámetros / Cifras

✏️ EJERCICIOS — Capítulo 12 — El Transformer

Antes de construir el Nano-LLM, asegúrate de que la arquitectura del Transformer ha quedado clara. Resuelve estos ejercicios y compara tus respuestas con las soluciones para detectar lagunas.

Ejercicio 12.1 Contar parámetros de un bloque Transformer

Escribe una función que, dado d (dimensión) y d_ff (dimensión interna FFN), calcule el número exacto de parámetros de un bloque Transformer completo: atención (4 matrices d×d, cada una con su sesgo), FFN (dos capas con sesgos) y dos LayerNorm (escala + desplazamiento). Calcula para d=16 (nuestro bloque) y d=768 (GPT-2 small).

> 💡 💡 Pista: Recuerda que LayerNorm también tiene parámetros aprendibles: un vector de escala gamma y uno de desplazamiento beta, ambos de tamaño d. Hay 2 LayerNorm por bloque. (Con d = 16 y d_ff = 64 sale 3280, lo que imprime el Programa 7; con d = 48 y d_ff = 192, los 28 272 de cada bloque del Nano-LLM. Con los tamaños de GPT-2 small, esta función y la tabla de 13.1 dan sus 124 439 808 parámetros.)

✅ Solución:

📄 ejercicio_12_1.py

```
def params_transformer(d, d_ff=None, n_heads=1):
    d_ff = d_ff or d * 4
    """Cuenta parámetros de un bloque transformer"""
    aten  = 4 * d * d + 4 * d      # Wq, Wk, Wv, Wo y sus sesgos
    ffn   = d*d_ff + d_ff + d_ff*d + d  # W1+b1 + W2+b2
    lnorm = 2 * 2 * d              # 2 LayerNorm x (gamma + beta)
    total = aten + ffn + lnorm
    return {'atención': aten, 'FFN': ffn, 'LayerNorm': lnorm, 'total': total}

configs = [
    ('Nuestro bloque (d=16)',     16,   64,  1),
    ('Nano-LLM completo (2 bl.)', 48,  192,  2),   # 28 272 por bloque, como el
Programa 8
    ('GPT-2 small (d=768, 12bl)',768, 3072, 12),
    ('GPT-3 (d=12288, 96bl)',  12288,49152, 96),
]

for nombre, d, d_ff, n_bl in configs:
    r = params_transformer(d, d_ff)
    total_modelo = r['total'] * n_bl
    print(f'{nombre}')
    print(f'  Atención: {r["atención"]:>10,}  FFN: {r["FFN"]:>10,}  LN:
{r["LayerNorm"]:>6,}')
    print(f'  Por bloque: {r["total"]:>10,}   x {n_bl} bloques = {total_modelo:>15,}
total')
    print()
```

Ejercicio 12.2 Layer Normalization desde cero

Implementa LayerNorm manualmente: normaliza cada fila de una matriz X para que tenga media = 0 y desviación estándar = 1. Verifica que funciona correctamente con una matriz que tiene media ≈ 5 y std ≈ 3.

> 💡 💡 Pista: LayerNorm normaliza a lo largo de la última dimensión (los features), no entre muestras. Para cada fila: x_norm = (x − media(x)) / √(varianza(x) + ε). Luego puede aplicarse escala y desplazamiento aprendibles.

✅ Solución:

📄 ejercicio_12_2.py

```
import numpy as np

def layer_norm(X, eps=1e-5):
    """Normaliza cada fila a media 0, std 1."""
    mu    = X.mean(axis=-1, keepdims=True)   # media por fila
```

```
    sigma = X.std(axis=-1,  keepdims=True)   # std por fila
    return (X - mu) / (sigma + eps)

np.random.seed(5)
T, d = 5, 8

# Crear datos con distribución no estándar
X = np.random.randn(T, d) * 3 + 5   # media≈5, std≈3

print('ANTES de LayerNorm:')
print(f'  Media global:    {X.mean():.4f}  (≈5)')
print(f'  Std global:      {X.std():.4f}   (≈3)')
print(f'  Media por fila:  {X.mean(axis=1).round(3)}')

X_norm = layer_norm(X)
print()
print('DESPUÉS de LayerNorm:')
print(f'  Media global:    {X_norm.mean():.8f}  (≈0)')
print(f'  Std global:      {X_norm.std():.8f}   (≈1)')
print(f'  Media por fila:  {X_norm.mean(axis=1).round(6)}')
print(f'  Std por fila:    {X_norm.std(axis=1).round(6)}')

print()
print('LayerNorm garantiza activaciones bien escaladas,')
print('lo cual es crucial para entrenar redes muy profundas.')
```

Ejercicio 12.3 Positional Encoding: ¿se parecen más los tokens cercanos?

Implementa el positional encoding sinusoidal y comprueba si posiciones cercanas tienen codificaciones más similares (mayor similitud coseno) que posiciones lejanas. Calcula la similitud entre la posición 0 y las posiciones 1, 2, 4, 8, 16 y 32, y comprueba si decrece siempre con la distancia.

> 💡 💡 Pista: Las codificaciones sinusoidales usan diferentes frecuencias para cada dimensión. Las posiciones cercanas tienen valores similares en las frecuencias bajas (que varían lentamente), mientras que difieren en las frecuencias altas.

✅ Solución:

📄 **ejercicio_12_3.py**

```
import numpy as np

def pos_encoding(T, d):
    PE = np.zeros((T, d))
    for pos in range(T):
        for i in range(0, d, 2):
            ang = pos / (10000 ** (i/d))
            PE[pos, i]   = np.sin(ang)
            if i+1 < d: PE[pos, i+1] = np.cos(ang)
    return PE

def sim_coseno(u, v):
    return np.dot(u,v) / (np.linalg.norm(u)*np.linalg.norm(v)+1e-10)

PE = pos_encoding(T=64, d=32)

print('Similitudes coseno entre posición 0 y otras posiciones\n')
print(f"{'Distancia':>10}  {'Pos A':>6}  {'Pos B':>6}  {'Similitud':>12}  Barra")
print('-' * 55)

for pos_b in [1, 2, 4, 8, 16, 32]:
    sim = sim_coseno(PE[0], PE[pos_b])
    barra = '' * int(max(0, sim) * 30)
    print(f'{pos_b:>10}  {0:>6}  {pos_b:>6}  {sim:>12.4f}  {barra}')

sims = [sim_coseno(PE[0], PE[k]) for k in range(1, 64)]
subidas = [k + 1 for k in range(len(sims) - 1) if sims[k + 1] > sims[k]]
print()
print(f'De la distancia 1 a la 4, la similitud cae de {sims[0]:.2f} a {sims[3]:.2f}.')
print(f'Pero no decrece siempre: entre las distancias 1 y 63 vuelve a subir')
```

```
    print(f'{len(subidas)} veces (por ejemplo, de la 16 a la 32: {sims[15]:.2f} ->
    {sims[31]:.2f}).')
    print('Las posiciones cercanas se parecen mas que la mayoria de las lejanas,')
    print('pero es una tendencia, no una ley: el modelo aprende a usarla.')
```

PARTE III

Construye tu GPT-2

El Nano-LLM: un GPT-2 en miniatura programado línea a línea, cómo genera texto, si memoriza o razona y, al final, el mismo código moviendo el GPT-2 real de OpenAI.

Capítulo 13: El Nano-LLM — Construyendo un Modelo de Lenguaje (***)

> **🎯 AL TERMINAR ESTE CAPÍTULO SABRÁS:**
>
> ✓ Ensamblar embeddings, codificación posicional y bloques Transformer en un modelo completo.
>
> ✓ Implementar el bucle de entrenamiento autorregresivo de un LLM con backpropagation exacto (regla de la cadena) y mini-lotes.
>
> ✓ Entrenar el Nano-LLM con Adam y un programa de tasa de aprendizaje (warm-up lineal + decaimiento coseno).
>
> ✓ Generar texto combinando temperatura y muestreo top-p (nucleus sampling) desde el propio Programa 8.
>
> ✓ Interpretar la pérdida en bits/carácter y reconocer los síntomas de sobreajuste en un modelo con más parámetros que datos.
>
> ✓ Implementar el backward completo de un transformer, derivada a derivada.
>
> ✓ Verificar un backward analítico con diferencias finitas (comprobación de gradientes).

13.1 Del bloque al modelo completo

Tenemos todos los ingredientes. Ahora los vamos a ensamblar en un modelo de lenguaje completo, aunque a escala nanoscópica. Nuestro Nano-LLM comparte la arquitectura fundamental de GPT, Claude y Gemini, solo que con decenas de miles de parámetros en lugar de miles de millones. En concreto, reproduce pieza a pieza la arquitectura de GPT-2 (OpenAI, 2019), el antecesor directo de los LLM actuales: solo cambian los tamaños, como verás en la tabla del final de esta sección.

La arquitectura del Nano-LLM es la siguiente:

- Tokenizador: convierte el texto en índices enteros (nivel carácter).
- Embedding de tokens: tabla de vectores aprendibles para cada token.
- Embedding posicional: añade información de posición a cada token.
- N bloques Transformer (usaremos 2): procesan la secuencia con atención causal de 4 cabezas y una FFN con GELU, normalizando antes de cada subcapa (Pre-LN), como GPT-2.
- LayerNorm final y capa de salida: proyecta al tamaño del vocabulario con la misma tabla del embedding de tokens (pesos compartidos) y aplica softmax para obtener probabilidades.

El modelo aprende a predecir el siguiente carácter dado un contexto de los CTX caracteres anteriores. Esa tarea, aparentemente simple, obliga al modelo a aprender gramática, estilo y patrones del texto de entrenamiento.

El programa que vamos a construir entrena este diseño con tres prácticas estándar en el entrenamiento de LLMs reales: **(1)** el gradiente se calcula de forma exacta mediante backpropagation; **(2)** cada actualización de pesos promedia un mini-lote de varias secuencias, no una sola ventana aleatoria; y **(3)** la generación de texto combina temperatura y muestreo top-p por defecto. El texto de entrenamiento consta de 93 líneas (5076 caracteres), con un vocabulario de 46 símbolos distintos.

El corpus más amplio cubre, a propósito, buena parte de los conceptos de este libro (neuronas, capas, retropropagación, atención, sobreajuste, generalización…), de modo que el propio texto de entrenamiento funciona como un resumen temático del curso.

La tabla siguiente recoge cada pieza del Nano-LLM con la forma de sus tensores y su número de parámetros. Los nombres son los del código de GPT-2: `wte` (*word token embedding*) y `wpe` (*word position embedding*). Merece la pena detenerse en ella: cada uno de los 60 000 parámetros del modelo sale de aquí.

Tensor	Forma	Parámetros
`wte` (embedding de tokens y matriz de salida)	46 × 48	2208
`wpe` (embedding posicional)	24 × 48	1152
Por bloque: `ln1` y `ln2` (γ y β de cada LayerNorm)	4 × 48	192
Por bloque: `Wq`, `Wk`, `Wv`, `Wo` y sus sesgos	4 × (48 × 48 + 48)	9408
Por bloque: `W1` + `b1`	48 × 192 + 192	9408
Por bloque: `W2` + `b2`	192 × 48 + 48	9264
Subtotal por bloque		**28 272** (× 2 = 56 544)
`lnf` (LayerNorm final: γ y β)	2 × 48	96
Total		**60 000**

Un mini-lote de 8 ventanas de 24 caracteres recorre el modelo así: los índices enteros, de forma (8 × 24), se convierten en embeddings (8 × 24 × 48); cada bloque Transformer recibe y devuelve un tensor (8 × 24 × 48); dentro de la atención, cada vector de 48 números se reparte entre las 4 cabezas, 12 para cada una (8 × 4 × 24 × 12); y la salida produce los logits (8 × 24 × 46): para cada ventana, una fila por posición y una columna por cada carácter del vocabulario. El softmax convierte cada fila en una distribución de probabilidad.

> ℹ️ **El Nano-LLM en cifras**
>
> El modelo tiene 13 capas densas (6 por bloque —`Wq`, `Wk`, `Wv`, `Wo`, `W1` y `W2`— más la de salida) y 910 neuronas: 432 por bloque y 46 en la salida. El forward de una ventana cuesta unos 2.98 millones de operaciones (MFLOPs) y el backward, aproximadamente el doble. Adam guarda dos valores extra por parámetro (m y v): 120 000 números más en memoria.

Tu Nano-LLM frente a GPT-2

Cada decisión de diseño de esta tabla es la de GPT-2: el embedding posicional se aprende (no es la codificación sinusoidal de la sección 12.2); la atención tiene varias cabezas, con sesgo en Q, K, V y Wo; la LayerNorm, con su escala γ y su desplazamiento β, va *antes* de cada subcapa (Pre-LN, que es más estable que el Post-LN del Transformer de 2017; sección 12.2); la FFN usa GELU, una versión suave de ReLU; hay una LayerNorm final; la salida reutiliza la tabla `wte` (pesos compartidos), y los pesos se inician con desviación 0.02, algo menor en las dos proyecciones que se suman a la vía residual. Lo único que cambia son los tamaños (la dimensión del modelo, que en los Capítulos 11 y 12 llamábamos d, se llama aquí D, como en el código; la de cada cabeza, DH, es la d_k):

	Tu Nano-LLM	GPT-2 small (2019)
Contexto (CTX)	24 caracteres	1024 tokens
Dimensión del modelo (D)	48	768
Cabezas de atención	4 (de 12 dimensiones)	12 (de 64 dimensiones)
Bloques	2	12
FFN (D_FF)	192	3072
Vocabulario	46 caracteres	50 257 tokens BPE (Capítulo 9)
Parámetros	60 000	124 439 808

	Tu Nano-LLM	GPT-2 small (2019)
Datos de entrenamiento	5 KB (93 frases)	40 GB de páginas web (WebText)

Si aplicas la cuenta de la primera tabla con los tamaños de GPT-2 small obtienes exactamente sus 124 439 808 parámetros. Dos diferencias más, ninguna de arquitectura: GPT-2 usa *dropout* durante el entrenamiento (aquí lo omitimos, y en el Capítulo 15 veremos lo que eso implica) y su tokenizador es BPE, no de caracteres.

> **ℹ️ Lo que ha cambiado desde GPT-2**
>
> Desde 2019 los LLM han cambiado poco en lo esencial y mucho en los detalles. Las piezas que sustituyen a las de GPT-2 en modelos como LLaMA, Mistral o Qwen son: **RMSNorm** en lugar de LayerNorm (normaliza sin restar la media); **RoPE** en lugar de `wpe` (la posición se codifica rotando Q y K; sección 12.2); **SwiGLU** en lugar de GELU (una FFN con «compuerta»); **GQA**, que comparte K y V entre grupos de cabezas para ahorrar memoria; y, en los más grandes, **mezcla de expertos (MoE)**, varias FFN de las que cada token usa solo unas pocas. Al generar, todos guardan K y V de los tokens anteriores (**KV-cache**, Capítulo 17), y los **modelos de razonamiento** se entrenan para escribir pasos intermedios antes de responder, la idea que probarás a pequeña escala en el Capítulo 15. Ninguna de estas piezas cambia lo que has aprendido aquí: forward, backward, atención causal, vía residual y Adam siguen siendo el esqueleto.

13.2 El corpus y el tokenizador

El texto de entrenamiento son 93 frases cortas, una por línea, con un total de 5076 caracteres. El tokenizador es el más simple posible: cada carácter distinto es un token. Ordenados, los 46 caracteres del vocabulario son el salto de línea, el espacio, nueve signos de puntuación y la cifra 2 (de «gpt-2»), que en el orden de los códigos de carácter queda entre ellos (! ( ) , - . 2 : ; ?), las 26 letras de la `a` a la `z` y ocho caracteres propios del español: `¡ ¿ á é í ñ ó ú`.

Cada carácter recibe como índice su posición en esa lista ordenada. Así, el texto «la red» se convierte en los índices `23 12 1 29 16 15`: la `l` es el carácter 23, la `a` el 12, el espacio el 1, y así sucesivamente. Puedes comprobarlo tú mismo con `[C2I[c] for c in 'la red']` tras ejecutar la primera parte del Programa 8.

> Si el modelo no hubiera aprendido nada y repartiera la probabilidad por igual entre los V = 46 caracteres, su pérdida de entropía cruzada sería exactamente ln(46) ≈ 3.83: es la referencia de «no haber aprendido nada». El primer valor que imprime el programa (3.8547) está muy cerca (un poco por encima, porque los pesos iniciales, aleatorios, no reparten la probabilidad exactamente por igual). Cualquier valor por debajo indica que el modelo ha aprendido algo; cuánto, y si es aprendizaje o memoria, lo discutiremos en 13.6 y en el Capítulo 15.

La pérdida que usa el Nano-LLM para aprender a predecir, como prácticamente todos los LLM, es la entropía cruzada, y su idea es sencilla. En cada posición, el modelo da una probabilidad a cada uno de los 46 caracteres; solo nos importa la que dio al carácter correcto, p. La pérdida de esa predicción es $-\ln(p)$: casi cero si el modelo acertó con seguridad, y grande si estaba seguro de otra cosa. La pérdida de un mini-lote es la media de esas cantidades sobre sus $B \cdot T = 8 \cdot 24 = 192$ predicciones.

📄 Ejemplo — la entropía cruzada con números

```
import numpy as np

# Pérdida de una predicción: -ln(probabilidad dada al carácter correcto)
for p in [0.9, 0.5, 0.1, 1 / 46]:
    print(f'p = {p:.3f}  ->  pérdida = {-np.log(p):.3f} nats'
```

```
            f' = {-np.log2(p):.2f} bits   perplejidad = {np.exp(-np.log(p)):.1f}')
  > Salida del programa:
p = 0.900  ->  pérdida = 0.105 nats = 0.15 bits   perplejidad = 1.1
p = 0.500  ->  pérdida = 0.693 nats = 1.00 bits   perplejidad = 2.0
p = 0.100  ->  pérdida = 2.303 nats = 3.32 bits   perplejidad = 10.0
p = 0.022  ->  pérdida = 3.829 nats = 5.52 bits   perplejidad = 46.0
```

Como usa el logaritmo natural, la pérdida se mide en nats; dividida entre ln 2 ≈ 0.693 se convierte en bits. Y su exponencial, e^pérdida, es la perplejidad (18.2): el número de candidatos entre los que el modelo duda en promedio, como si eligiera al azar entre ellos. La última fila es el modelo que no ha aprendido nada (p = 1/46): pérdida ln 46 ≈ 3.83 y perplejidad 46. Son las cifras que verás en este capítulo y en los Capítulos 15, 16 y 18.

13.3 El bucle de entrenamiento

El entrenamiento consiste en, para cada actualización de pesos (un paso de entrenamiento):

- Elegir BATCH=8 ventanas aleatorias del texto de entrenamiento, cada una de longitud CTX+1.
- En cada ventana, los primeros CTX caracteres son la entrada (X); los últimos CTX, el objetivo (y), desplazados un paso.
- Hacer un forward pass del mini-lote completo (un tensor de 8 × 24): la red predice la distribución del siguiente carácter para cada posición.
- Calcular la pérdida de entropía cruzada entre las predicciones y los objetivos reales.
- Backpropagation exacto: calcular, con la regla de la cadena, el gradiente de la pérdida respecto a todos los parámetros.
- Promediar el gradiente de las 8 secuencias del mini-lote, recortarlo si su norma supera CLIP = 1.0 y actualizar los pesos una sola vez con Adam.
- Repetir 12 000 pasos, con una tasa de aprendizaje que sube y baja según un programa de warm-up + coseno.

13.3.1 Mini-lotes: por qué promediar en vez de actualizar secuencia a secuencia

Actualizar los pesos tras cada secuencia individual es posible, pero resulta ruidoso: una ventana de texto poco representativa puede empujar los pesos en una dirección que no ayuda al conjunto. Promediar el gradiente de varias secuencias antes de actualizar —un mini-lote (mini-batch)— suaviza ese ruido: los errores particulares de cada secuencia tienden a cancelarse, y la dirección resultante representa mejor "qué conviene aprender en general". Es exactamente la técnica que usan PyTorch, TensorFlow y, a mayor escala, el entrenamiento de cualquier LLM comercial. El coste es computacional: procesar un mini-lote de 8 secuencias cuesta 8 veces más que procesar una sola, pero la actualización resultante es más fiable.

13.3.2 Pasos, no épocas

En el glosario, una **época** es una pasada completa por todos los datos de entrenamiento. El bucle del Programa 8 no funciona así: en cada iteración elige 8 ventanas al azar, calcula el gradiente medio y actualiza los pesos una vez. Cada iteración es un **paso** de optimización, y por eso la variable se llama `PASOS`.

¿Cuántas veces ve el modelo el corpus en 12 000 pasos? Cada paso procesa 8 ventanas de 24 caracteres, es decir, 192 predicciones. En total: 12 000 × 8 × 24 ≈ 2.3 millones de predicciones, unas 454 pasadas por los 5076 caracteres del corpus. Es mucha repetición para tan poco texto, y lo pagaremos en 13.6.

13.3.3 Warm-up y decaimiento coseno: programar la tasa de aprendizaje

Hasta ahora habíamos usado una tasa de aprendizaje (LR) constante durante todo el entrenamiento. En la práctica, los LLMs modernos —desde GPT-3 hasta LLaMA— no entrenan así: usan un programa (schedule) que varía el LR en el tiempo, típicamente en dos fases:

- Warm-up (calentamiento): durante los primeros pasos (aquí, 150), el LR sube linealmente desde 0 hasta su valor máximo. Al principio del entrenamiento, los estimadores de momento de Adam (las medias móviles m y v) todavía no son fiables porque solo han visto uno o dos gradientes; dar pasos grandes con esas estimaciones poco maduras puede desestabilizar el modelo desde el primer momento. Empezar con pasos pequeños y crecientes evita ese arranque brusco.
- Decaimiento coseno: superado el warm-up, el LR desciende siguiendo la mitad de una onda de coseno, desde el máximo hasta un valor mínimo (aquí, 1e-4), llegando muy suavemente casi a cero al final del entrenamiento. Esto permite dar pasos grandes mientras el modelo todavía tiene mucho que aprender, y pasos progresivamente más finos hacia el final, cuando conviene "afinar" los pesos sin desestabilizar lo ya aprendido.

En el programa, la función lr_warmup_coseno(t, lr_max, t_total) implementa ambas fases con una condición: si el paso t es anterior al warm-up, interpola linealmente; si no, aplica la fórmula del coseno sobre los pasos restantes.

Un detalle de implementación: durante el warm-up la tasa es `lr_max · (t + 1) / 150`. El `+ 1` hace que el paso 0 ya aprenda; con `t / 150`, el primer paso tendría una tasa de 0 y no cambiaría nada.

13.3.4 El recorte del gradiente

La última pieza del bucle es la red de seguridad de la sección 5.8: si la norma global del gradiente —la raíz de la suma de los cuadrados de todos sus elementos, en todas las matrices a la vez— supera `CLIP = 1.0`, el gradiente entero se multiplica por `CLIP / norma`. La dirección no cambia; solo se limita el tamaño del paso. En el programa son cuatro líneas, justo antes de la actualización de Adam.

¿Cuántas veces actúa? Registrando la norma en cada paso de la ejecución real, el resultado sorprende: supera 1.0 en 11 758 de los 12 000 pasos (el 98 %), con una mediana de 1.67 y un máximo de 5.65. En este modelo el recorte no es una red que se usa de vez en cuando: actúa casi siempre y fija el tamaño del gradiente que recibe Adam. El Ejercicio 13.7 compara el entrenamiento con y sin recorte.

13.4 El backward completo, derivada a derivada

Hay dos maneras de calcular el gradiente de la pérdida respecto a los parámetros de un modelo. La primera es numérica: a cada parámetro se le suma una pequeña perturbación ε y se mide cuánto cambia la pérdida, repitiendo esto para cada parámetro que se quiera estimar. Es un método intuitivo —de hecho, es la forma estándar de comprobar que una implementación de backpropagation es correcta (gradient checking)— pero es impracticable como método de entrenamiento habitual: para estimar el gradiente completo de un modelo con decenas de miles de parámetros haría falta rehacer el forward pass una vez por cada uno de ellos, en cada paso de entrenamiento.

Hagamos la cuenta para nuestro modelo: dos forwards por parámetro (uno con $+\varepsilon$ y otro con $-\varepsilon$) × 60 000 parámetros × 12 000 pasos son 1440 millones de forwards. A unos 3 MFLOPs cada uno, entrenar así llevaría días; con backpropagation, el entrenamiento completo dura unos dos minutos y medio.

La alternativa, y la que usa este programa, es el backpropagation exacto: partiendo de la pérdida, se recorre el grafo de cálculo del forward pass en sentido inverso, aplicando la regla de la cadena del cálculo diferencial paso a paso —salida → bloques transformer (FFN → LayerNorm → atención → LayerNorm) → embeddings— hasta obtener, en una única pasada, el gradiente exacto de la pérdida respecto a todos los parámetros del modelo (incluidas las matrices Wq, Wk, Wv y Wo de la atención, y W1, W2 de la FFN, en cada bloque). Es precisamente lo que hace loss.backward() en PyTorch, solo que aquí cada derivada está escrita explícitamente, matriz a matriz. El resultado es un entrenamiento tanto más rápido (una sola pasada hacia atrás, no una pasada hacia adelante por parámetro) como más completo: todos los parámetros del modelo reciben una señal de aprendizaje exacta en cada paso.

13.4.1 El recorrido hacia atrás

El backward recorre el forward en orden inverso. Una regla sencilla ayuda a no perderse: **gradiente de un tensor tiene siempre la misma forma que el tensor**. Si `Wq0` es 48 × 48, `grads['Wq0']` también; si una activación es 8 × 24 × 48, el gradiente que llega a ella también. Para un mini-lote de 8 ventanas de 24 caracteres, el camino es este:

```
pérdida (escalar)
  │  dlogits = (probs − one_hot) / (B·T)                (8 × 24 × 46)
  ▼
salida, logits = xf · wteᵀ:   dwte += xfᵀ·dlogits (46 × 48)   dxf (8 × 24 × 48)
LayerNorm final:   dlnf_g, dlnf_b (48,)   dX (8 × 24 × 48)
  ▼
bloque 2 → bloque 1, cada uno de atrás hacia delante:
  rama FFN: dW2 (192 × 48), GELU, dW1 (48 × 192), LayerNorm 2 ─┐
  vía residual (copia del gradiente) ──────────────────────────┤ suma
  rama atención: dWo, softmax (4 cabezas), dWq, dWk, dWv, LN 1 ─┐
  vía residual (copia del gradiente) ──────────────────────────┤ suma
  ▼
embeddings:  dwpe[:24] = Σ dX    dwte: np.add.at(…)  (se suma al de la salida)
```

13.4.2 Las cinco derivadas clave

Casi todo el backward son productos de matrices: si `Y = X · W + b`, entonces `dW = Xᵀ · dY`, `db` es la suma de `dY` y `dX = dY · Wᵀ`. Además de esa regla, hacen falta cinco derivadas:

- **Softmax + entropía cruzada.** Juntas tienen una derivada sorprendentemente simple: `dlogits = (probs − one_hot) / (B·T)`. A cada probabilidad se le resta 1 en la posición del carácter correcto, y se divide entre las B·T = 8 × 24 = 192 predicciones del mini-lote porque la pérdida es una media. Un matiz de exactitud: para no calcular nunca log(0), el código suma 10^{-10} dentro del logaritmo de la pérdida que imprime, y

(probs − one_hot) es la derivada de la pérdida sin ese término. Con probabilidades normales la diferencia es inapreciable; solo si la probabilidad del carácter correcto bajara a unos 10^{-10}, la pérdida impresa quedaría topada en ≈ 23. Las librerías lo resuelven calculando log-softmax de forma estable (log-sum-exp).

- **GELU.** `gelu(x) = 0.5·x·(1 + tanh(u))`, con `u = √(2/π)·(x + 0.044715·x³)`, es un producto de dos factores que dependen de x, así que su derivada tiene dos términos (regla del producto): `0.5·(1 + t) + 0.5·x·(1 − t²)·√(2/π)·(1 + 3·0.044715·x²)`, con `t = tanh(u)`. Olvidar el segundo es un error típico, y el Ejercicio 13.6 lo detecta.
- **Softmax de la atención.** Como cada fila de pesos suma 1, mover una puntuación afecta a toda la fila: `dscores = pesos ⊙ (dpesos − Σ pesos ⊙ dpesos)`, con la suma por filas. Después se divide entre √12 (la dimensión de cada cabeza), igual que en el forward. Las 4 cabezas se derivan a la vez, cada una con sus 12 columnas.
- **LayerNorm con γ y β.** Con `x̂ = (x − μ) / √(σ² + ε)` y `y = x̂ · γ + β`: `dβ` es la suma de `dy`, `dγ` es la suma de `dy · x̂`, y, llamando `dx̂ = dy · γ`, la derivada respecto a la entrada es `dx = (dx̂ − media(dx̂) − x̂ · media(dx̂ · x̂)) / √(σ² + ε)`. Los dos términos que se restan existen porque la media y la desviación típica dependen de toda la fila.
- **Pesos compartidos.** La tabla `wte` se usa dos veces: al principio, para buscar el embedding de cada carácter, y al final, para calcular los logits. Una variable usada en dos sitios recibe la **suma** de los gradientes de los dos usos: `grads['wte']` empieza con el de la salida y después se le añade el de la entrada.

13.4.3 Residuales y caracteres repetidos

Dos detalles completan el backward. El primero son las **sumas residuales**: si en el forward `X1 = Xin + atención(LN(Xin))`, el gradiente que llega a `X1` se bifurca y llega **entero** por las dos ramas: por el camino de la atención y por el atajo directo. Por eso el código termina cada subcapa con `dX = dX1 + dxin`: lo que llega por el atajo más lo que vuelve por `Wq`, `Wk`, `Wv` y la LayerNorm. Con Pre-LN, la vía residual no atraviesa ninguna LayerNorm dentro del bloque, así que el gradiente baja por ella sin obstáculos: es lo que evita que se desvanezca en redes profundas (Apéndice F.1).

El segundo son los **caracteres repetidos**. Si la letra «a» aparece cinco veces en la ventana, su fila de `wte` se usó cinco veces y debe recibir la suma de los cinco gradientes. Una asignación normal, `grads['wte'][tokens] += dX`, se quedaría solo con uno de ellos; `np.add.at(grads['wte'], tokens, dX)` acumula todos.

> **ℹ️ Verificar el backward**
>
> ¿Cómo sabemos que todas estas derivadas están bien? Comparándolas con el gradiente numérico en elementos elegidos al azar de cada tensor. Con los pesos del modelo entrenado, el error relativo máximo entre ambos es de $1.3 \cdot 10^{-8}$: coinciden hasta la precisión de la propia comparación. Si se olvida el segundo término de la derivada de GELU, el error sube a 0.79. El Ejercicio 13.6 te pide hacer esta comprobación tú mismo.

13.5 El Nano-LLM completo — Programa 8

Llegamos al programa completo. Lo presentamos de dos formas, que ejecutan exactamente el mismo código (misma arquitectura, mismos hiperparámetros, misma semilla) y producen la misma salida:

- Versión de referencia (13.5.1): el código con comentarios breves, dividido en cuatro partes con una explicación entre ellas. Las cuatro partes, una detrás de otra, forman el archivo `programa_8_nano_llm.py`.
- Versión comentada (13.5.2): el mismo código con una explicación exhaustiva línea a línea, pensada para una primera lectura detenida del forward y del backward. Son 1098 líneas, la mayoría comentarios, y se imprime completa en 13.5.2.

El programa tarda unos dos minutos y medio en un portátil. Al terminar guarda los pesos entrenados en `nano_llm_pesos.npz`, que reutilizarán el Capítulo 14, el Programa RZ-1 del Capítulo 15 y el Capítulo 18.

13.5.1 Versión de referencia, en cuatro partes

PROGRAMA 8 — Nano-LLM completo (versión de referencia)

📄 `programa_8_nano_llm.py`

📄 Parte 1 de 4 — datos, tokenizador, hiperparámetros y parámetros

```
# ============================================================
# PROGRAMA 8 — Nano-LLM: un GPT-2 en miniatura
# Modelo de lenguaje de nivel carácter con la arquitectura
# de GPT-2 (2019): embeddings de token y de posición
# aprendidos, 2 bloques Pre-LN (LayerNorm con γ y β antes de
# cada subcapa), atención causal de 4 cabezas, FFN con GELU,
# LayerNorm final y pesos compartidos entre la entrada y la
# salida. Solo cambian los tamaños (GPT-2 small: 12 bloques,
# 12 cabezas, D = 768, contexto 1024, 50 257 tokens BPE).
# NumPy puro: backpropagation analítico completo, mini-lotes,
# Adam con warmup + coseno y recorte del gradiente. Guarda
# los pesos en nano_llm_pesos.npz para los capítulos siguientes.
# ============================================================

import numpy as np

# -- Texto de entrenamiento (93 líneas, vocabulario de 46 caracteres) --
TEXTO = """
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas capas ocultas.
la atención permite ver todo el contexto a la vez.
los transformers son la base de los llm modernos.
entrenar un modelo requiere muchos datos y cálculo.
el futuro pertenece a quienes comprenden la ia.
un modelo pequeño puede aprender patrones simples.
la tokenización convierte texto en números enteros.
cada carácter del vocabulario tiene un índice único.
el embedding transforma índices en vectores densos.
la codificación posicional indica el orden del texto.
la función softmax convierte logits en probabilidades.
el optimizador adam ajusta los pesos, paso a paso.
una red profunda combina atención y capas ocultas.
generar texto significa predecir, uno a uno, cada token.
la temperatura controla cuánta variedad hay al generar.
un buen modelo equilibra memoria y generalización.
¿comprender la ia? requiere práctica, paciencia y datos.
también existe el software: gpt-2 fue un hito clave.
un modelo (bien entrenado) generaliza; uno malo, no.
¡el aprendizaje nunca termina! sigue explorando.
un vector es una lista ordenada de números reales.
una matriz agrupa vectores en filas y columnas.
el producto punto mide cuánto se parecen dos vectores.
la retropropagación calcula gradientes hacia atrás.
el descenso de gradiente ajusta los pesos poco a poco.
una época recorre todo el conjunto de entrenamiento.
el sobreajuste ocurre cuando el modelo memoriza demasiado.
```

```
la regularización ayuda a que el modelo generalice mejor.
un lote agrupa varios ejemplos para entrenar juntos.
la normalización estabiliza las activaciones internas.
una neurona combina entradas, pesos y un sesgo.
la función de pérdida mide el error del modelo.
el aprendizaje supervisado usa ejemplos etiquetados.
una capa oculta transforma la información paso a paso.
el descenso estocástico usa lotes pequeños y aleatorios.
la validación comprueba si el modelo generaliza bien.
un hiperparámetro se ajusta antes de entrenar el modelo.
la arquitectura define cómo se conectan las capas.
el vocabulario agrupa todos los símbolos posibles.
una secuencia es un orden concreto de tokens.
el contexto limita cuántos tokens ve el modelo a la vez.
la generalización permite responder a datos nuevos.
comprender estos conceptos abre la puerta a la ia moderna.
un dataset agrupa todos los ejemplos disponibles.
el conjunto de test evalúa el modelo al final, una sola vez.
la precisión mide cuántas predicciones son correctas.
el error mide cuánto se aleja la predicción del valor real.
una etiqueta indica la respuesta correcta de un ejemplo.
la inferencia usa el modelo ya entrenado, sin ajustar pesos.
el preentrenamiento aprende patrones generales del lenguaje.
el ajuste fino especializa un modelo en una tarea concreta.
un token puede ser una letra, una parte de palabra o una palabra.
la ventana de contexto limita cuánto texto ve el modelo.
un residual conecta la entrada directamente con la salida.
la normalización evita que las activaciones crezcan sin control.
el ruido en los datos dificulta encontrar el patrón real.
una red ancha tiene muchas neuronas en cada capa.
una red profunda tiene muchas capas apiladas.
el paralelismo permite entrenar varias partes a la vez.
la memoria limita cuántos datos caben en una gráfica.
un acelerador gráfico multiplica matrices muy rápido.
la escala mide cuántos parámetros tiene un modelo.
un modelo grande necesita muchos más datos para aprender bien.
la eficiencia importa tanto como la precisión final.
el costo de entrenar crece con el tamaño del modelo.
la interpretabilidad explica por qué el modelo decide algo.
un sesgo en los datos puede producir un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la privacidad protege los datos personales de los usuarios.
un modelo robusto resiste pequeños cambios en la entrada.
la incertidumbre mide cuánta confianza tiene el modelo.
un ejemplo difícil confunde incluso a un buen modelo.
la curva de aprendizaje muestra el progreso del entrenamiento.
un mínimo local complica encontrar la mejor solución.
el paisaje de pérdida describe cómo cambia el error.
una meseta ralentiza el progreso del entrenamiento.
el ruido ayuda a veces a escapar de mínimos malos.
la exploración prueba caminos nuevos durante el entrenamiento.
la explotación aprovecha lo que ya funciona bien.
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre categorías distintas.
la clasificación asigna una categoría a cada ejemplo.
la regresión predice un número continuo, no una categoría.
el aprendizaje por refuerzo premia las buenas decisiones.
un agente aprende actuando dentro de un entorno.
la recompensa guía el aprendizaje del agente.
la política define cómo actúa el agente en cada estado.
un entorno simulado permite entrenar sin riesgo real.
la simulación acelera enormemente el aprendizaje del agente.
comprender bien lo básico facilita entender lo avanzado.
"""

# -- Tokenizador ----------------------------------------

VOCAB = sorted(set(TEXTO))
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))
DATOS = np.array([C2I[c] for c in TEXTO], dtype=int)
# -- Hiperparámetros (entre paréntesis, los de GPT-2 small) --
CTX    = 24     # longitud del contexto (1024)
D      = 48     # dimensión del modelo (768)
H      = 4      # cabezas de atención (12)
DH     = D // H # dimensión de cada cabeza: 12 (64)
D_FF   = 192    # dimensión interna FFN, 4xD (3072)
N_BLQ  = 2      # bloques transformer (12)
LR     = 3e-3   # tasa de aprendizaje máxima
PASOS  = 12000  # pasos de entrenamiento (cada paso ve BATCH ventanas al azar;
                #  no son épocas: 12000 pasos ≈ 454 pasadas por el corpus)
BATCH  = 8      # secuencias por lote (promedia el gradiente, suaviza la curva)
CLIP   = 1.0    # recorte de la norma del gradiente (Capítulo 5, 5.8)

np.random.seed(42)
```

```
# -- Parámetros del modelo (inicialización de GPT-2) --------
def randn(shape, std=0.02):
    return np.random.randn(*shape) * std
# Las proyecciones que se suman a la vía residual (Wo y W2) se
# inician con una desviación menor, 0.02/sqrt(2·N_BLQ), como en GPT-2.
STD_RES = 0.02 / np.sqrt(2 * N_BLQ)

params = {
    'wte': randn((V, D)),     # embedding de tokens (y matriz de salida)
    'wpe': randn((CTX, D)),   # embedding de posiciones
    'lnf_g': np.ones(D), 'lnf_b': np.zeros(D),   # LayerNorm final
}
for k in range(N_BLQ):
    params[f'ln1_g{k}'] = np.ones(D);  params[f'ln1_b{k}'] = np.zeros(D)
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
    params[f'ln2_g{k}'] = np.ones(D);  params[f'ln2_b{k}'] = np.zeros(D)
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)

total = sum(p.size for p in params.values())
print(f'Vocabulario: {V} caracteres')
print(f'Total de parámetros: {total:,}')
print(f'Texto de entrenamiento: {len(DATOS)} tokens, '
      f'{TEXTO.strip().count(chr(10)) + 1} líneas')

# Máscara causal: se calcula una vez con el tamaño máximo (CTX, CTX)
# y se recorta a [:T, :T] cuando la secuencia es más corta.
MASCARA_CAUSAL = np.triu(np.full((CTX, CTX), -1e9), k=1)
```

La segunda parte es el corazón del programa: las utilidades (softmax, LayerNorm y GELU con sus derivadas exactas, y `separar()`/`unir()`, que reparten los 48 números de cada vector entre las 4 cabezas y los vuelven a juntar), `forward()`, que pasa un mini-lote entero por el modelo, y `forward_backward()`, que calcula la pérdida y recorre el modelo hacia atrás siguiendo el camino de la sección 13.4. La generación usa `forward()` con un lote de una sola secuencia.

📄 Parte 2 de 4 — forward y backward

```
# -- Utilidades ---------------------------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)



def layer_norm_fwd(x, g, b, eps=1e-5):
    """LayerNorm de GPT-2: normaliza cada fila y aplica escala g (γ) y desplazamiento b
(β)."""
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)
def layer_norm_bwd(dy, g, cache):
    """Derivada exacta de LayerNorm: devuelve dx, dg y db."""
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))      # todas las filas (lote y posiciones)
    dg = (dy * xhat).sum(ejes)
    db = dy.sum(ejes)
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, dg, db
C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    """GELU con la aproximación tanh que usa GPT-2."""
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))
def gelu_bwd(x):
    """Derivada de gelu(x)."""
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)
def separar(x):   # (B, T, D) -> (B, H, T, DH): una matriz por cabeza
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)
def unir(x):      # (B, H, T, DH) -> (B, T, D)
    B, _, T, _ = x.shape
```

```
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)
def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])

# -- Forward ---------------------------------------------------
def forward(tokens):
    """tokens (B, T) -> logits (B, T, V), y lo que el backward necesita."""
    B, T = tokens.shape
    X = params['wte'][tokens] + params['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        Xin = X
        # Subcapa 1: LayerNorm -> atención causal multi-cabeza -> suma residual
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        # Subcapa 2: LayerNorm -> FFN con GELU -> suma residual
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    logits = xf @ params['wte'].T          # pesos compartidos: la salida usa wte
    return logits, (xf, lnf, cache)

# -- Forward + Backward (un mini-lote entero) ----------------------
def forward_backward(tokens, targets):
    """
    Pasa el mini-lote por el modelo y calcula, con backpropagation
    exacto (regla de la cadena), el gradiente de la pérdida media
    respecto a TODOS los parámetros.
    """




    B, T = tokens.shape
    logits, (xf, lnf, cache) = forward(tokens)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -np.log(probs[bi, ti, targets] + 1e-10).mean()
    grads = {k: np.zeros_like(v) for k, v in params.items()}

    dlogits = probs.copy()

    dlogits[bi, ti, targets] -= 1.0          # softmax + entropía cruzada
    dlogits /= B * T
    grads['wte'] += tmm(dlogits, xf)         # wte como matriz de salida
    dxf = dlogits @ params['wte']
    dX, grads['lnf_g'], grads['lnf_b'] = layer_norm_bwd(dxf, params['lnf_g'], lnf)

    for k in reversed(range(N_BLQ)):









        c = cache[k]
        # Subcapa 2 hacia atrás. Residual: dX llega entero a X1 y a la FFN
        grads[f'W2{k}'] = tmm(c['h'], dX)
        grads[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        grads[f'W1{k}'] = tmm(c['m'], dh_pre)
        grads[f'b1{k}'] = dh_pre.sum((0, 1))
        dm = dh_pre @ params[f'W1{k}'].T
        dx1, grads[f'ln2_g{k}'], grads[f'ln2_b{k}'] = layer_norm_bwd(dm, params[f'ln2_g{k}'],
c['ln2'])
        dX1 = dX + dx1
        # Subcapa 1 hacia atrás. Residual: dX1 llega entero a Xin y a la atención
        grads[f'Wo{k}'] = tmm(c['att'], dX1)
        grads[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
```

```
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)   # derivada de softmax
        dQ = unir(dscores @ c['K'])
        dK = unir(dscores.transpose(0, 1, 3, 2) @ c['Q'])
        dV = unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            grads[f'W{n}{k}'] = tmm(c['a'], dn)
            grads[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, grads[f'ln1_g{k}'], grads[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    grads['wpe'][:T] = dX.sum(0)
    np.add.at(grads['wte'], tokens, dX)      # wte también como embedding de entrada

    return grads, loss
```

La tercera parte es el bucle de entrenamiento de la sección 13.3: la tasa de aprendizaje con warm-up y coseno, el mini-lote de 8 ventanas (un único tensor de 8 × 24), el recorte y la actualización de Adam (5.5). `forward_backward()` ya devuelve el gradiente promediado sobre el mini-lote; el recorte se aplica a ese gradiente, justo antes de Adam.

📄 Parte 3 de 4 — entrenamiento

```
# -- Learning rate: warmup lineal + decaimiento coseno --------
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    """
    Sube linealmente de 0 a lr_max durante t_warmup pasos (evita
    pasos erráticos al principio, cuando los gradientes son más
    inestables), luego decae en coseno hasta lr_min (afina los
    pesos al final). Es el esquema estándar al entrenar LLMs.
    """
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup     # +1: el paso 0 ya aprende
    t_adj = t - t_warmup
    t_decay = t_total - t_warmup
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * t_adj / t_decay))


# -- Entrenamiento con Adam (mini-lotes de BATCH secuencias) --
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
beta1, beta2, eps_adam, t_adam = 0.9, 0.999, 1e-8, 0
LR_MAX = LR

print('\nEntrenando Nano-LLM (backpropagation analítico, mini-lotes)...')
print(f'CTX={CTX} | D={D} | D_FF={D_FF} | Bloques={N_BLQ} | Batch={BATCH}\n')

for paso in range(PASOS):
    lr_actual = lr_warmup_coseno(paso, LR_MAX, PASOS)
    inicios = np.random.randint(0, len(DATOS) - CTX - 1, size=BATCH)
    xb = np.stack([DATOS[i:i + CTX] for i in inicios])          # (BATCH, CTX)
    yb = np.stack([DATOS[i + 1:i + CTX + 1] for i in inicios])  # desplazado 1
    grads_acc, loss = forward_backward(xb, yb)
    # Recorte del gradiente: si la norma global supera CLIP, se reescala
    norma = np.sqrt(sum((g ** 2).sum() for g in grads_acc.values()))
    if norma > CLIP:
        for k in grads_acc:
            grads_acc[k] *= CLIP / norma

    t_adam += 1
    for k in params:
        g = grads_acc[k]
        m_adam[k] = beta1 * m_adam[k] + (1 - beta1) * g
        v_adam[k] = beta2 * v_adam[k] + (1 - beta2) * g ** 2
        m_h = m_adam[k] / (1 - beta1 ** t_adam)
        v_h = v_adam[k] / (1 - beta2 ** t_adam)
        params[k] -= lr_actual * m_h / (np.sqrt(v_h) + eps_adam)

    if paso % 1000 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida = {loss:.4f}   lr = {lr_actual:.5f}   '
              f'(bits/char = {loss/np.log(2):.2f})')
```

La cuarta parte define las estrategias de muestreo —greedy, top-k y top-p, que estudiaremos a fondo en 14.2—, la función `generar()`, que combina temperatura y top-p, y por último guarda los pesos entrenados.

📄 Parte 4 de 4 — muestreo, generación y guardado

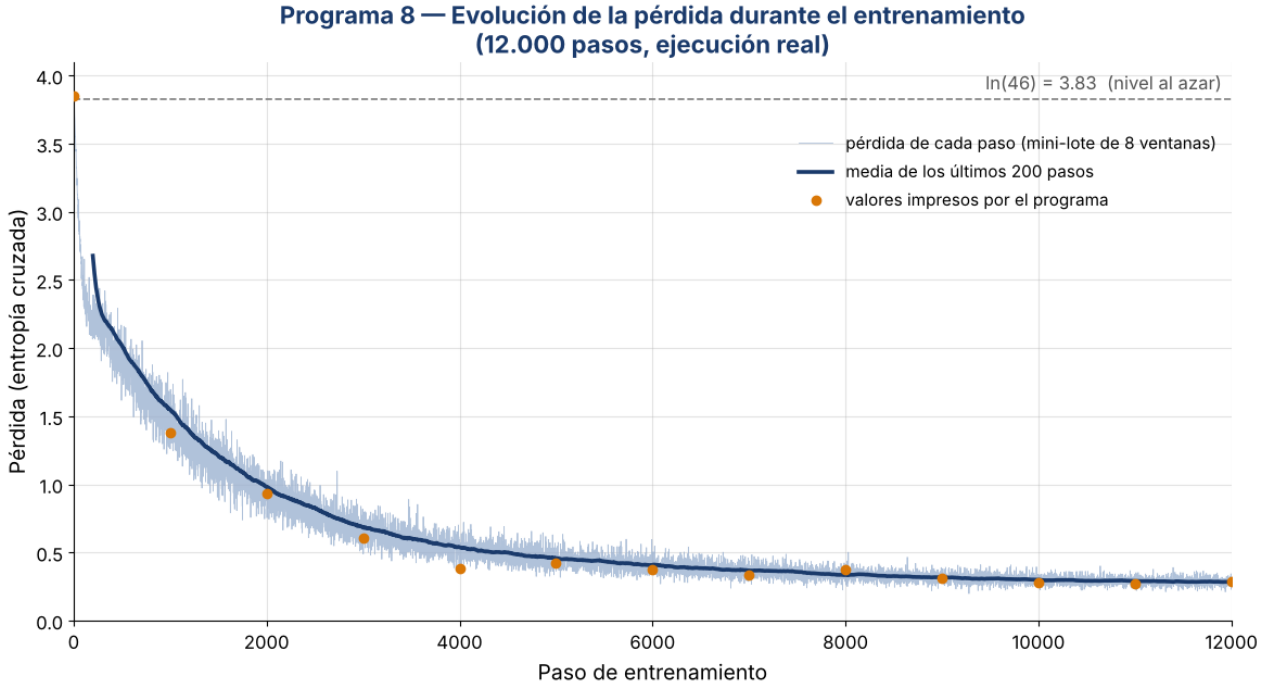
```
# -- Estrategias de muestreo (se estudian a fondo en el Capítulo 14) --
# generar() usa temperatura + top_p_filtrar; greedy y top_k_filtrar
# quedan definidas para experimentar con ellas.
def greedy(probs):
    """Siempre el token más probable. Puede quedar 'atrapado' en bucles."""
    return np.argmax(probs)

def top_k_filtrar(probs, k):
    """Conserva solo los k tokens más probables."""
    indices_ord = np.argsort(probs)[::-1]
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:k]] = probs[indices_ord[:k]]
    return nueva / nueva.sum()

def top_p_filtrar(probs, p):
    """Conserva el mínimo conjunto de tokens que suman probabilidad p (nucleus sampling)."""
    indices_ord = np.argsort(probs)[::-1]
    probs_ord   = probs[indices_ord]
    suma_acum   = np.cumsum(probs_ord)
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
    return nueva / nueva.sum()


# -- Generación de texto ----------------------------------
def generar(semilla, n_chars=120, temperatura=0.8, top_p=0.9):
    """
    Genera texto carácter a carácter.
    temperatura > 1: más aleatorio | temperatura < 1: más determinista
    top_p: nucleus sampling — conserva solo el conjunto mínimo de
    caracteres cuya probabilidad acumulada alcanza top_p, evitando
    elegir caracteres de probabilidad residual que producen "sopa
    de letras". Con top_p=1.0 equivale al muestreo puro.
    """
    tokens = [C2I.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla

    for _ in range(n_chars):
        ctx = tokens[-CTX:]
        logits = forward(np.array([ctx]))[0][0, -1]
        probs = softmax(logits / temperatura)

        if top_p < 1.0:
            probs = top_p_filtrar(probs, top_p)

        siguiente = int(np.random.choice(V, p=probs))
        tokens.append(siguiente)
        resultado += I2C[siguiente]

    return resultado


print('\n=== TEXTO GENERADO POR EL NANO-LLM ===')
print(generar('la inteligencia', n_chars=120))
print()
print(generar('los modelos de', n_chars=120))
# -- Guardar los pesos entrenados -----------------------------
# Los Capítulos 11, 14, 15 y 18 reutilizan este modelo ya entrenado:
#   params = dict(np.load('nano_llm_pesos.npz'))
np.savez('nano_llm_pesos.npz', **params)
print("\nPesos guardados en nano_llm_pesos.npz")
```

Al ejecutar el programa se obtiene lo siguiente (con la semilla fijada, el resultado es reproducible):

> Salida del programa (unos 2.5 minutos):

```
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas

Entrenando Nano-LLM (backpropagation analítico, mini-lotes)...
CTX=24 | D=48 | D_FF=192 | Bloques=2 | Batch=8
```

```
Paso     0: pérdida = 3.8547   lr = 0.00002   (bits/char = 5.56)
Paso  1000: pérdida = 1.3802   lr = 0.00296   (bits/char = 1.99)
Paso  2000: pérdida = 0.9349   lr = 0.00283   (bits/char = 1.35)
Paso  3000: pérdida = 0.6037   lr = 0.00261   (bits/char = 0.87)
Paso  4000: pérdida = 0.3814   lr = 0.00231   (bits/char = 0.55)
Paso  5000: pérdida = 0.4244   lr = 0.00196   (bits/char = 0.61)
Paso  6000: pérdida = 0.3764   lr = 0.00158   (bits/char = 0.54)
Paso  7000: pérdida = 0.3344   lr = 0.00120   (bits/char = 0.48)
Paso  8000: pérdida = 0.3753   lr = 0.00084   (bits/char = 0.54)
Paso  9000: pérdida = 0.3107   lr = 0.00053   (bits/char = 0.45)
Paso 10000: pérdida = 0.2764   lr = 0.00030   (bits/char = 0.40)
Paso 11000: pérdida = 0.2689   lr = 0.00015   (bits/char = 0.39)
Paso 11999: pérdida = 0.2866   lr = 0.00010   (bits/char = 0.41)

=== TEXTO GENERADO POR EL NANO-LLM ===
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas

los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas

Pesos guardados en nano_llm_pesos.npz
```



Fíjate en la curva: la pérdida de cada paso (en azul claro) es muy ruidosa, porque cada mini-lote toma 8 ventanas al azar y unas son más «difíciles» que otras. Por eso los valores impresos no bajan siempre: de 0.3814 en el paso 4000 sube a 0.4244 en el 5000, y de 0.3344 en el 7000 a 0.3753 en el 8000. Lo que importa es la tendencia de la media móvil (en azul oscuro), que desciende de 3.85 a unos 0.3. El último valor, 0.2866, es la pérdida de un solo mini-lote; medida sobre todo el texto de entrenamiento, la pérdida final es 0.284 (Ejercicio 13.3).

13.5.2 La versión comentada, completa

Esta es la versión del Programa 8 que explica, línea a línea, qué hace cada instrucción y por qué. Son 1098 líneas, la mayoría comentarios, pensadas para leerlas con calma junto a 13.5.1, sobre todo si es la primera vez que te enfrentas a un backpropagation escrito a mano. Ejecuta exactamente el mismo cálculo: se ha comprobado que imprime la misma salida que la versión de referencia y que guarda los mismos pesos, número a número.

PROGRAMA 8 — Nano-LLM, versión comentada

📄 programa_8_nano_llm_documentado.py

```
# ================================================================================
#  PROGRAMA 8 — NANO-LLM: UN GPT-2 EN MINIATURA (versión extensamente comentada)
#  ------------------------------------------------------------------------------
#  Modelo de lenguaje (LLM) de nivel CARÁCTER, implementado ÚNICAMENTE
#  con NumPy — sin PyTorch, sin TensorFlow, sin ninguna librería de
#  aprendizaje profundo. Todo lo que hace un framework "por debajo"
#  (propagar hacia adelante, calcular gradientes, actualizar pesos)
#  está escrito aquí a mano, línea por línea, para que se pueda ver
#  y entender exactamente qué ocurre en cada paso.
#
#  Hace lo mismo, y da exactamente la misma salida, que la versión
#  compacta del Programa 8 impresa en el Capítulo 13 (13.5.1).
#
#  ¿QUÉ ES UN "NANO-LLM"?
#  Es GPT-2 (OpenAI, 2019) en miniatura: la MISMA arquitectura, pieza
#  a pieza, con tamaños diminutos. Aprende a predecir el SIGUIENTE
#  CARÁCTER de un texto, dado un fragmento de contexto. Repitiendo ese
#  proceso carácter a carácter, "escribe" frases que se parecen a las
#  del texto con el que fue entrenado.
#
#                        este modelo     GPT-2 small
#      contexto (CTX)         24             1024
#      dimensión (D)          48              768
#      cabezas (H)             4               12
#      bloques (N_BLQ)         2               12
#      vocabulario (V)        46           50 257 (BPE)
#      parámetros         60 000      124 millones
#
#  ARQUITECTURA (de abajo hacia arriba):
#
#      texto de entrada (caracteres)
#              │
#              ▼
#        Tokenizador (carácter → número entero)
#              │
#              ▼
#        wte[token] + wpe[posición]          (número → vector de D = 48)
#              │
#              ▼
#       ┌──────────────────────────────────────┐   │   × N_BLQ  (aquí, 2 veces)
#       │ Bloque Transformer (Pre-LN)          │   │    4 cabezas, máscara causal
#       │  X = X + Atención(LayerNorm(X))      │   │    48 → 192 (GELU) → 48
#       │  X = X + FFN(LayerNorm(X))           │   │
#       └──────────────────────────────────────┘   │
#              │
#              ▼
#        LayerNorm final
#              │
#              ▼
#        logits = vector · wteᵀ   (la MISMA tabla de la entrada)
#              │
#              ▼
#        Carácter muestreado  →  se añade al texto  →  se repite
#
#  FLUJO GENERAL DEL PROGRAMA (en el orden en que aparece el código):
#    1. Se define el texto de entrenamiento (TEXTO).
#    2. Se construye el tokenizador (carácter <-> índice entero).
#    3. Se fijan los hiperparámetros (tamaños, tasa de aprendizaje...).
#    4. Se inicializan aleatoriamente todos los parámetros (pesos).
#    5. Se definen el forward (predicción) y el forward + backward
#       (pérdida y gradiente exacto de todos los pesos, con
#       backpropagation escrito a mano).
#    6. Se entrena con Adam durante PASOS pasos, con mini-lotes,
#       warmup + coseno y recorte del gradiente (gradient clipping).
#    7. Se genera texto nuevo usando el modelo ya entrenado.
#    8. Se guardan los pesos entrenados en nano_llm_pesos.npz.
#
```

```
#  Todas las matrices que el modelo "aprende" están en el diccionario
#  `params`. Entrenar consiste, únicamente, en ir ajustando poco a
#  poco los números de esas matrices para que el modelo se equivoque
#  cada vez menos al predecir el siguiente carácter.
# ================================================================================

import numpy as np   # única dependencia externa: cálculo numérico con arrays/matrices

# --------------------------------------------------------------------------------
# TEXTO DE ENTRENAMIENTO
# --------------------------------------------------------------------------------
# Es el "libro" del que el modelo aprenderá. Cuanto más texto y más
# variado, más patrones distintos puede aprender — pero también hace
# falta más capacidad de modelo (ver D, D_FF más abajo) para no perder
# calidad, un compromiso conocido como "leyes de escalado".
#
# 93 líneas en español, con tildes, la letra "ñ" y signos de puntuación
# variados (¿ ? ¡ ! , . ; : ( ) -), pensadas para que el vocabulario
# resultante sea exactamente de 46 caracteres distintos (ver más abajo).
# El contenido cubre, a propósito, muchos conceptos de IA/ML para que
# el propio texto de entrenamiento sirva como resumen del tema.
TEXTO = """
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas capas ocultas.
la atención permite ver todo el contexto a la vez.
los transformers son la base de los llm modernos.
entrenar un modelo requiere muchos datos y cálculo.
el futuro pertenece a quienes comprenden la ia.
un modelo pequeño puede aprender patrones simples.
la tokenización convierte texto en números enteros.
cada carácter del vocabulario tiene un índice único.
el embedding transforma índices en vectores densos.
la codificación posicional indica el orden del texto.
la función softmax convierte logits en probabilidades.
el optimizador adam ajusta los pesos, paso a paso.
una red profunda combina atención y capas ocultas.
generar texto significa predecir, uno a uno, cada token.
la temperatura controla cuánta variedad hay al generar.
un buen modelo equilibra memoria y generalización.
¿comprender la ia? requiere práctica, paciencia y datos.
también existe el software: gpt-2 fue un hito clave.
un modelo (bien entrenado) generaliza; uno malo, no.
¡el aprendizaje nunca termina! sigue explorando.
un vector es una lista ordenada de números reales.
una matriz agrupa vectores en filas y columnas.
el producto punto mide cuánto se parecen dos vectores.
la retropropagación calcula gradientes hacia atrás.
el descenso de gradiente ajusta los pesos poco a poco.
una época recorre todo el conjunto de entrenamiento.
el sobreajuste ocurre cuando el modelo memoriza demasiado.
la regularización ayuda a que el modelo generalice mejor.
un lote agrupa varios ejemplos para entrenar juntos.
la normalización estabiliza las activaciones internas.
una neurona combina entradas, pesos y un sesgo.
la función de pérdida mide el error del modelo.
el aprendizaje supervisado usa ejemplos etiquetados.
una capa oculta transforma la información paso a paso.
el descenso estocástico usa lotes pequeños y aleatorios.
la validación comprueba si el modelo generaliza bien.
un hiperparámetro se ajusta antes de entrenar el modelo.
la arquitectura define cómo se conectan las capas.
el vocabulario agrupa todos los símbolos posibles.
una secuencia es un orden concreto de tokens.
el contexto limita cuántos tokens ve el modelo a la vez.
la generalización permite responder a datos nuevos.
comprender estos conceptos abre la puerta a la ia moderna.
un dataset agrupa todos los ejemplos disponibles.
el conjunto de test evalúa el modelo al final, una sola vez.
la precisión mide cuántas predicciones son correctas.
el error mide cuánto se aleja la predicción del valor real.
una etiqueta indica la respuesta correcta de un ejemplo.
la inferencia usa el modelo ya entrenado, sin ajustar pesos.
el preentrenamiento aprende patrones generales del lenguaje.
el ajuste fino especializa un modelo en una tarea concreta.
un token puede ser una letra, una parte de palabra o una palabra.
la ventana de contexto limita cuánto texto ve el modelo.
un residual conecta la entrada directamente con la salida.
la normalización evita que las activaciones crezcan sin control.
el ruido en los datos dificulta encontrar el patrón real.
una red ancha tiene muchas neuronas en cada capa.
una red profunda tiene muchas capas apiladas.
el paralelismo permite entrenar varias partes a la vez.
la memoria limita cuántos datos caben en una gráfica.
un acelerador gráfico multiplica matrices muy rápido.
```

```
la escala mide cuántos parámetros tiene un modelo.
un modelo grande necesita muchos más datos para aprender bien.
la eficiencia importa tanto como la precisión final.
el costo de entrenar crece con el tamaño del modelo.
la interpretabilidad explica por qué el modelo decide algo.
un sesgo en los datos puede producir un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la privacidad protege los datos personales de los usuarios.
un modelo robusto resiste pequeños cambios en la entrada.
la incertidumbre mide cuánta confianza tiene el modelo.
un ejemplo difícil confunde incluso a un buen modelo.
la curva de aprendizaje muestra el progreso del entrenamiento.
un mínimo local complica encontrar la mejor solución.
el paisaje de pérdida describe cómo cambia el error.
una meseta ralentiza el progreso del entrenamiento.
el ruido ayuda a veces a escapar de mínimos malos.
la exploración prueba caminos nuevos durante el entrenamiento.
la explotación aprovecha lo que ya funciona bien.
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre categorías distintas.
la clasificación asigna una categoría a cada ejemplo.
la regresión predice un número continuo, no una categoría.
el aprendizaje por refuerzo premia las buenas decisiones.
un agente aprende actuando dentro de un entorno.
la recompensa guía el aprendizaje del agente.
la política define cómo actúa el agente en cada estado.
un entorno simulado permite entrenar sin riesgo real.
la simulación acelera enormemente el aprendizaje del agente.
comprender bien lo básico facilita entender lo avanzado.
"""

# ------------------------------------------------------------------------------
# TOKENIZADOR A NIVEL DE CARÁCTER
# ------------------------------------------------------------------------------
# Una red neuronal solo entiende números, no letras. El "tokenizador"
# es el traductor entre texto y números en ambas direcciones.
#
# Este es el tokenizador MÁS SIMPLE posible: cada carácter distinto
# del texto (una letra, un espacio, un signo de puntuación...) se
# convierte en UN token. Los LLMs de verdad (GPT, Claude...) usan
# tokenizadores más sofisticados a nivel de "subpalabra" (por ejemplo,
# BPE), donde un token puede ser una palabra entera o un trozo de ella,
# lo que hace las secuencias más cortas y eficientes. Aquí, por
# claridad didáctica, se usa el nivel de carácter: más lento de
# entrenar, pero mucho más fácil de entender y depurar.
VOCAB  = sorted(set(TEXTO))
# `set(TEXTO)` recorre el string TEXTO y se queda con cada carácter
# ÚNICO que aparece en él (sin repeticiones). `sorted(...)` los ordena
# (por su código Unicode) para que el orden sea siempre el mismo cada
# vez que se ejecuta el programa — importante para reproducibilidad.
# Resultado: una lista de 46 caracteres distintos, p. ej.
# ['\n', ' ', '!', '(', ')', ',', ..., 'z', '¡', '¿', 'á', 'é', ...].

V      = len(VOCAB)
# V = tamaño del vocabulario = número de caracteres distintos = 46.
# Es la "V" que aparecerá en muchas fórmulas y tamaños de matrices
# más abajo (p. ej. la tabla de embeddings tiene forma (V, D)).

C2I    = {c: i for i, c in enumerate(VOCAB)}
# "Character To Index": diccionario que traduce cada carácter a su
# número entero (su posición en VOCAB). Ejemplo: C2I['a'] podría ser 12.
# `enumerate(VOCAB)` genera pares (índice, carácter); el diccionario
# por comprensión los invierte a {carácter: índice}.

I2C    = dict(enumerate(VOCAB))
# "Index To Character": el diccionario inverso, traduce cada número
# de vuelta a su carácter original. Se usa al GENERAR texto, para
# convertir los índices que predice el modelo de nuevo en letras.

DATOS  = np.array([C2I[c] for c in TEXTO], dtype=int)
# Se recorre el texto completo, carácter a carácter, y se sustituye
# cada uno por su índice entero (usando C2I). El resultado es un
# array de NumPy de enteros: la "traducción numérica" de todo el
# texto de entrenamiento. Es sobre este array (DATOS) de donde se
# extraerán, más adelante, las ventanas de contexto para entrenar.

# ------------------------------------------------------------------------------
# HIPERPARÁMETROS
# ------------------------------------------------------------------------------
# Los "hiperparámetros" son las decisiones de diseño que el programador
# fija ANTES de entrenar (a diferencia de los "parámetros", que son los
# pesos que el modelo aprende solo). Cambiarlos cambia la forma y el
# comportamiento del modelo.
CTX    = 24
# Longitud del "contexto": cuántos caracteres previos puede ver el
# modelo como máximo para predecir el siguiente. Con CTX=24, el modelo
```

```
# solo "recuerda" los últimos 24 caracteres — más o menos 4-5 palabras
# cortas en español. Los LLMs reales usan contextos de miles o cientos
# de miles de tokens; aquí se mantiene pequeño para que el modelo siga
# siendo manejable y rápido de entrenar.

D      = 48
# Dimensión del "embedding": cuántos números se usan para representar
# internamente cada carácter (su "vector de significado"). Cuanto
# mayor es D, más matices puede capturar el modelo sobre cada
# carácter y su contexto, pero también hay más parámetros que
# aprender y más lento va el entrenamiento. GPT-2 small usa D = 768.

H      = 4
# Número de "cabezas" de atención. En vez de una única atención sobre
# los 48 números de cada vector, se hacen 4 atenciones en paralelo,
# cada una sobre 12 de ellos (DH = D / H = 12). Cada cabeza puede
# especializarse en una relación distinta: una puede mirar el carácter
# anterior; otra, el principio de la palabra. GPT-2 small usa 12
# cabezas de 64 dimensiones.

DH     = D // H

D_FF   = 192
# Dimensión interna de la red "Feed-Forward" (FFN) que hay dentro de
# cada bloque transformer (ver más abajo). Por convención, suele ser
# 4 veces D (48 × 4 = 192): esa capa intermedia más ancha le da al
# modelo más "espacio de trabajo" para procesar cada posición antes
# de comprimir el resultado de vuelta a dimensión D.

N_BLQ  = 2
# Número de "bloques transformer" apilados uno tras otro (ver el
# diagrama de la cabecera). Cada bloque añade una ronda más de
# "prestar atención al contexto + procesar la información". Los LLMs
# grandes apilan decenas o cientos de bloques; aquí, con solo 2, el
# modelo ya es capaz de aprender patrones de lenguaje simples.

LR     = 3e-3
# "Learning Rate" o tasa de aprendizaje: de qué tamaño son los pasos
# con los que se ajustan los pesos en cada actualización. Un valor
# demasiado alto hace que el entrenamiento sea inestable (los pesos
# "saltan" de un lado a otro sin asentarse); demasiado bajo hace que
# el aprendizaje sea desesperantemente lento. 3e-3 (0.003) es un
# valor típico para el optimizador Adam con modelos de este tamaño.
# Nótese que este valor es el MÁXIMO: durante el entrenamiento se
# modula con un "warmup + decaimiento coseno" (ver más abajo).

PASOS  = 12000
# Número de veces que se actualizan los pesos: PASOS de optimización.
# Cada paso procesa un mini-lote de BATCH ventanas elegidas al azar,
# no todo el texto, así que un paso NO es una "época" (una época es
# una pasada completa por todos los datos; ver el glosario). Cuenta:
# 12000 pasos × 8 ventanas × 24 caracteres ≈ 2.3 millones de
# predicciones ≈ 454 pasadas por los 5076 caracteres del corpus.

BATCH  = 8
# Cuántas secuencias de entrenamiento distintas se procesan juntas
# antes de actualizar los pesos una vez. Promediar el gradiente de
# varias secuencias (en vez de actualizar tras cada una) suaviza el
# entrenamiento: reduce el "ruido" de que una sola secuencia rara
# empuje los pesos en una dirección poco representativa.

CLIP   = 1.0
# Recorte de la norma del gradiente (Capítulo 5, 5.8): si la
# norma global del gradiente (la raíz de la suma de los cuadrados de
# TODOS sus elementos) supera CLIP, el gradiente entero se reescala
# para que su norma valga exactamente CLIP. La dirección no cambia;
# solo se limita el tamaño. Es una red de seguridad contra un paso
# desmesurado provocado por un mini-lote anómalo.

np.random.seed(42)
# Fija la "semilla" del generador de números aleatorios de NumPy.
# Con la misma semilla, todos los números "aleatorios" que se generen
# a partir de aquí (inicialización de pesos, elección de qué ventana
# de texto usar en cada paso, muestreo al generar texto...) serán
# EXACTAMENTE los mismos cada vez que se ejecute el programa. Esto
# hace que el experimento sea reproducible: cualquiera que ejecute
# este mismo código debería obtener los mismos resultados.

# ------------------------------------------------------------------------------
# INICIALIZACIÓN DE LOS PARÁMETROS (PESOS) DEL MODELO
# ------------------------------------------------------------------------------
# Un modelo "sin entrenar" no es más que un conjunto de matrices
# llenas de números aleatorios pequeños. El entrenamiento consiste,
# únicamente, en ir corrigiendo esos números poco a poco (con
# backpropagation + Adam) para que el modelo acierte cada vez más.
```

```
def randn(shape, std=0.02):
    """
    Genera una matriz de la forma (shape) dada, con valores aleatorios
    siguiendo una distribución normal (campana de Gauss) de media 0 y
    desviación típica `std` (0.02 por defecto, como en GPT-2): la
    mayoría de los valores quedan entre -0.06 y 0.06.

    ¿Por qué valores tan pequeños? Si los pesos iniciales fueran muy
    grandes, las primeras predicciones del modelo serían caóticas y
    los gradientes iniciales podrían "explotar" (números enormes que
    desestabilizan el entrenamiento desde el primer paso). Empezar
    "casi en cero" da un punto de partida neutro y estable.
    """
    return np.random.randn(*shape) * std

# Las dos proyecciones cuya salida se SUMA a la vía residual (Wo, al
# final de la atención, y W2, al final de la FFN) se inician con una
# desviación todavía menor: 0.02 / sqrt(2·N_BLQ). Es la receta de
# GPT-2: cada bloque suma dos contribuciones a la vía residual, y con
# N_BLQ bloques hay 2·N_BLQ sumas; dividir por su raíz cuadrada evita
# que, al apilar muchos bloques, la vía residual crezca sin control
# antes de empezar a entrenar.
STD_RES = 0.02 / np.sqrt(2 * N_BLQ)

# `params` es un diccionario que contiene TODAS las matrices que el
# modelo puede aprender. Cada entrada es una pieza del "cerebro" del
# modelo. Los nombres son los mismos que usa GPT-2 (wte, wpe, ln_f...).
params = {
    # Tabla de embeddings de TOKEN ("word token embedding"): una fila
    # por cada carácter del vocabulario (V filas), cada una con D
    # números. Es la "traducción" de cada carácter a un vector que la
    # red puede procesar. Forma: (V, D) = (46, 48).
    # En GPT-2 esta MISMA matriz se usa también a la salida, para
    # convertir el vector final en una puntuación por carácter: se
    # dice que entrada y salida "comparten pesos" (weight tying). Por
    # eso no hay una matriz de salida aparte.
    'wte':  randn((V, D)),

    # Tabla de embeddings POSICIONALES ("word position embedding"):
    # una fila por cada posición de la ventana de contexto (CTX
    # filas). Le dice al modelo "en qué lugar de la secuencia" está
    # cada carácter, algo que la atención, por sí sola, no distingue.
    # Es APRENDIDA, como en GPT-2 (el Transformer original de 2017 la
    # calculaba con senos y cosenos; ver Capítulo 12). Forma: (CTX, D).
    'wpe':  randn((CTX, D)),

    # LayerNorm FINAL: se aplica una vez, después del último bloque y
    # antes de la salida. g (gamma, escala) empieza en 1 y b (beta,
    # desplazamiento) en 0: al principio la LayerNorm solo normaliza.
    'lnf_g': np.ones(D), 'lnf_b': np.zeros(D),
}

# A continuación se crean los parámetros de cada BLOQUE TRANSFORMER
# (hay N_BLQ = 2 bloques, numerados 0 y 1). Cada bloque tiene su
# propio juego de matrices, completamente independiente.
for k in range(N_BLQ):
    # --- LayerNorm previa a la atención (ln1) ---
    params[f'ln1_g{k}'] = np.ones(D);  params[f'ln1_b{k}'] = np.zeros(D)

    # --- Matrices de ATENCIÓN (self-attention) ---
    # Wq, Wk, Wv proyectan la entrada en tres "vistas" del mismo
    # vector, cada una con un papel distinto:
    #   Q (Query)  = "¿qué información estoy buscando?"
    #   K (Key)    = "¿qué información ofrezco yo a los demás?"
    #   V (Value)  = "el contenido real que se transmite si me
    #                 prestan atención"
    # Cada una es (D, D) y lleva su sesgo de tamaño D, como en GPT-2.
    # Las H = 4 cabezas NO tienen matrices propias: cada cabeza usa
    # una "rebanada" de DH = 12 columnas de estas matrices (ver
    # separar(), más abajo).
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
    # Wo junta lo que han recogido las 4 cabezas y lo proyecta de
    # vuelta a dimensión D. Forma: (D, D), con sesgo.
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)

    # --- LayerNorm previa a la FFN (ln2) ---
    params[f'ln2_g{k}'] = np.ones(D);  params[f'ln2_b{k}'] = np.zeros(D)

    # --- Matrices de la FEED-FORWARD NETWORK (FFN) ---
    # Dos capas lineales con una activación GELU en medio. Procesa
    # CADA posición por separado (la atención es la que mezcla
    # información entre posiciones).
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)   # expande D ->
D_FF
```

```
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)  # comprime
D_FF -> D

# Comprobación rápida en pantalla: tamaño del vocabulario, número
# total de parámetros (60 000 justos con estos tamaños) y cuánto texto
# se va a usar para entrenar.
total = sum(p.size for p in params.values())
print(f'Vocabulario: {V} caracteres')
print(f'Total de parámetros: {total:,}')
print(f'Texto de entrenamiento: {len(DATOS)} tokens, '
      f'{TEXTO.strip().count(chr(10)) + 1} líneas')

# ------------------------------------------------------------------------
# MÁSCARA CAUSAL (para que el modelo no "haga trampa" mirando el futuro)
# ------------------------------------------------------------------------
# Cuando el modelo predice el carácter en la posición t, NO debe
# poder "espiar" los caracteres en las posiciones t+1, t+2... (eso
# sería aprender a copiar la respuesta en vez de predecirla). La
# máscara causal se lo impide: es una matriz (CTX, CTX) de ceros,
# excepto por encima de la diagonal principal, donde vale -1e9.
#
# La máscara se SUMA a las puntuaciones de atención antes del
# softmax: sumar -1e9 hace que esa posición reciba un peso de
# atención prácticamente 0, es decir, "apaga" el futuro.
#
# Al entrenar, la longitud de la secuencia (T) es siempre CTX; al
# generar puede ser menor (al principio, con semillas cortas). Por eso
# la máscara se calcula UNA SOLA VEZ aquí, con el tamaño máximo, y se
# usa su esquina superior izquierda [:T, :T].
MASCARA_CAUSAL = np.triu(np.full((CTX, CTX), -1e9), k=1)


# ------------------------------------------------------------------------
# UTILIDADES MATEMÁTICAS
# ------------------------------------------------------------------------
def softmax(x, axis=-1):
    """
    Convierte un vector de números cualesquiera en una distribución
    de probabilidad: valores entre 0 y 1 que suman exactamente 1 a lo
    largo del eje `axis`.

    Fórmula: softmax(x)_i = exp(x_i) / sum_j(exp(x_j))

    TRUCO DE ESTABILIDAD NUMÉRICA: antes de exponenciar se resta el
    máximo. El resultado matemático no cambia, pero se evita calcular
    exponenciales enormes que desbordarían el float (inf o nan).
    """
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)


def layer_norm_fwd(x, g, b, eps=1e-5):
    """
    LayerNorm ("normalización de capa"), exactamente como en GPT-2:
    para cada fila (cada posición) de x, resta su media y divide por
    su desviación típica (con eps dentro de la raíz), y después
    REESCALA con g (gamma) y DESPLAZA con b (beta):

        xhat = (x − media) / sqrt(varianza + eps)
        y    = xhat · g + b

    ¿Para qué sirve? A medida que la información atraviesa bloques,
    sus valores pueden crecer o desplazarse sin control. LayerNorm
    devuelve cada fila a una escala estándar antes de que entre en la
    atención o en la FFN. Con g y b el modelo puede, si le conviene,
    aprender otra escala y otro centro distintos de (1, 0).

    Devuelve y y, en un "cache", lo que el backward necesitará:
    xhat (la fila normalizada) e inv = 1 / sqrt(varianza + eps).
    """
    mu = x.mean(-1, keepdims=True)                       # media de cada fila
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)  # 1 / desviación típica
    xhat = (x - mu) * inv                                # fila normalizada
    return xhat * g + b, (xhat, inv)


def layer_norm_bwd(dy, g, cache):
    """
    Derivada (backward) EXACTA de la LayerNorm de arriba. Recibe
    `dy`, el gradiente de la pérdida respecto a la SALIDA y, y
    devuelve tres gradientes: respecto a la entrada x, a g y a b.

      · db = suma de dy sobre todas las filas   (y = ... + b)
      · dg = suma de dy·xhat sobre todas las filas   (y = xhat·g + ...)
      · dx: primero se deshace la escala, dxhat = dy·g, y después la
        normalización, con la fórmula exacta
```

```
        dx = inv · ( dxhat − media(dxhat) − xhat · media(dxhat·xhat) )

    Intuición de los tres términos dentro del paréntesis:
      · dxhat              -> el gradiente "tal cual", si la
                              normalización no acoplara las componentes.
      · − media(dxhat)     -> corrige que restar la media acopla TODAS
                              las componentes de la fila entre sí.
      · − xhat · media(...) -> corrige que dividir por la desviación
                              típica también depende de toda la fila.

    Es la misma matemática que usa la LayerNorm de PyTorch por
    debajo; aquí está escrita a la vista. El Ejercicio 13.6 la
    comprueba numéricamente.
    """
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))      # todas las filas (lote y posiciones)
    dg = (dy * xhat).sum(ejes)
    db = dy.sum(ejes)
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, dg, db


C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    """
    GELU ("Gaussian Error Linear Unit"), la activación de GPT-2, con
    la misma aproximación por tanh que usa su código original:

        gelu(x) = 0.5 · x · (1 + tanh( sqrt(2/π) · (x + 0.044715·x³) ))

    Se parece a ReLU (≈ x para x grande y positivo, ≈ 0 para x muy
    negativo), pero es SUAVE: no tiene esquina en 0 y deja pasar un
    poco de los valores ligeramente negativos. x³ se escribe x*x*x
    porque, en NumPy, la potencia x**3 es unas quince veces más lenta.
    """
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))


def gelu_bwd(x):
    """
    Derivada de gelu(x), por la regla del producto y la de la cadena.
    Con t = tanh(u) y u = sqrt(2/π)·(x + 0.044715·x³):

        gelu'(x) = 0.5·(1 + t) + 0.5·x·(1 − t²)·sqrt(2/π)·(1 + 3·0.044715·x²)

    (la derivada de tanh(u) es 1 − tanh²(u)). Sustituye a la derivada
    de ReLU (1 si x > 0, 0 si no) de la versión anterior del modelo.
    """
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)


def separar(x):   # (B, T, D) -> (B, H, T, DH): una matriz por cabeza
    """
    Reparte las D = 48 columnas entre las H = 4 cabezas: la cabeza 0
    se queda con las columnas 0-11, la 1 con 12-23, etc. El resultado
    tiene un eje nuevo, H, para que las cuatro cabezas calculen su
    atención a la vez, con las mismas multiplicaciones de matrices.
    """
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)


def unir(x):      # (B, H, T, DH) -> (B, T, D)
    """La operación inversa: vuelve a poner las 4 cabezas una al lado de otra."""
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)


def tmm(a, b):
    """
    Gradiente de una capa lineal y = a @ W: dW = aᵀ · dy, SUMADO
    sobre todas las filas del lote y todas las posiciones. Para
    hacerlo con una sola multiplicación se aplanan los ejes del lote
    y de las posiciones: (B, T, n) -> (B·T, n).
    """
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])


# ============================================================================
# FORWARD: de los tokens a los logits
# ============================================================================
# Todo el programa trabaja con MINI-LOTES: en vez de una secuencia de
```

```
# T caracteres, se procesan B secuencias a la vez, apiladas en un
# tensor de forma (B, T). Las multiplicaciones de matrices de NumPy
# aceptan ejes extra delante, así que el código es el mismo que para
# una secuencia, y mucho más rápido que un bucle de Python.
def forward(tokens):
    """
    tokens : array de enteros de forma (B, T).
    Devuelve los logits (B, T, V) — una puntuación por carácter
    posible, para cada posición de cada secuencia — y un "cache" con
    los valores intermedios que el backward necesitará.
    """
    B, T = tokens.shape

    # --- Embeddings: qué carácter es + en qué posición está ---
    X = params['wte'][tokens] + params['wpe'][:T]       # (B, T, D)

    cache = []
    for k in range(N_BLQ):
        Xin = X   # entrada del bloque k (vía residual)

        # --- SUB-CAPA 1: LayerNorm -> atención causal (4 cabezas) -> suma residual ---
        # PRE-LN, como GPT-2: se normaliza ANTES de la subcapa, y la
        # salida de la subcapa se SUMA a Xin sin normalizarla. La vía
        # residual (Xin, X1, X...) nunca pasa por una LayerNorm dentro
        # del bloque: es un "carril" por el que la información, y en el
        # backward el gradiente, circula sin obstáculos.
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])

        # Q, K, V para las 4 cabezas a la vez: (B, H, T, DH)
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')

        # Puntuaciones Q·Kᵀ de cada cabeza, divididas por sqrt(DH)
        # (12, la dimensión de CADA cabeza: es la que fija la escala de
        # los productos), más la máscara causal; softmax por filas.
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])

        # Suma ponderada de los Values; las 4 cabezas se vuelven a unir
        att = unir(pesos @ Vm)                           # (B, T, D)

        # Proyección de salida y suma a la vía residual
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']

        # --- SUB-CAPA 2: LayerNorm -> FFN con GELU -> suma residual ---
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']  # (B, T, D_FF)
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']

        # Se guarda todo lo que el backward necesitará de este bloque
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))

    # --- LayerNorm final y salida ---
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    # PESOS COMPARTIDOS: la salida usa la MISMA matriz wte de la
    # entrada, traspuesta. El logit de un carácter es el producto
    # escalar del vector final con el embedding de ese carácter:
    # "¿a qué carácter se parece más lo que viene ahora?".
    logits = xf @ params['wte'].T                        # (B, T, V)
    return logits, (xf, lnf, cache)


# ============================================================================
# FORWARD + BACKWARD: la pérdida y su gradiente exacto
# ============================================================================
# Esta función hace DOS cosas:
#
#   1. FORWARD: llama a forward() y, comparando las predicciones con
#      los caracteres reales (targets), calcula la PÉRDIDA (loss), un
#      único número que mide cuánto se equivoca el modelo.
#
#   2. BACKWARD (backpropagation): recorre el camino del forward AL
#      REVÉS aplicando la regla de la cadena, y obtiene el GRADIENTE
#      de la pérdida respecto a CADA parámetro: "en qué dirección y
#      cuánto debería cambiar ese peso para reducir un poco la
#      pérdida". Es lo que hace `loss.backward()` en PyTorch, escrito
#      aquí a la vista, matriz a matriz.
#
# Dos reglas se repiten una y otra vez:
#   · Capa lineal y = a @ W + b:  dW = aᵀ·dy (tmm),  db = suma de dy,
#     y hacia atrás da = dy @ Wᵀ.
#   · Una SUMA en el forward (la vía residual) COPIA el gradiente a
#     las dos ramas; una variable usada en DOS sitios (como wte, en la
#     entrada y en la salida) SUMA los gradientes de ambos usos.
def forward_backward(tokens, targets):
    """
```

```
    tokens, targets : arrays de enteros (B, T); targets es la misma
                      ventana de texto desplazada un carácter.
    Devuelve (grads, loss): un diccionario con el gradiente de cada
    matriz de `params` y la pérdida media del mini-lote.
    """
    B, T = tokens.shape
    logits, (xf, lnf, cache) = forward(tokens)
    probs = softmax(logits)

    # --- PÉRDIDA: entropía cruzada media ---
    # probs[bi, ti, targets] es la probabilidad que el modelo dio al
    # carácter CORRECTO en cada posición de cada secuencia.
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -np.log(probs[bi, ti, targets] + 1e-10).mean()

    grads = {k: np.zeros_like(v) for k, v in params.items()}

    # --- Gradiente respecto a los logits ---
    # Para softmax + entropía cruzada la derivada es sencillísima:
    # (probabilidades − 1 en el carácter correcto), dividido entre el
    # número de predicciones promediadas (B·T).
    dlogits = probs.copy()
    dlogits[bi, ti, targets] -= 1.0
    dlogits /= B * T

    # --- Salida: logits = xf @ wteᵀ ---
    grads['wte'] += tmm(dlogits, xf)         # 1er uso de wte (como matriz de salida)
    dxf = dlogits @ params['wte']
    dX, grads['lnf_g'], grads['lnf_b'] = layer_norm_bwd(dxf, params['lnf_g'], lnf)

    # --- Bloques transformer, del último al primero ---
    for k in reversed(range(N_BLQ)):
        c = cache[k]

        # -- Subcapa 2 hacia atrás: X = X1 + h @ W2 + b2 --
        # La suma residual copia dX a las dos ramas: a X1 (atajo) y a
        # la FFN.
        grads[f'W2{k}'] = tmm(c['h'], dX)
        grads[f'b2{k}'] = dX.sum((0, 1))
        # Deshacer la GELU: se multiplica por su derivada
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        grads[f'W1{k}'] = tmm(c['m'], dh_pre)
        grads[f'b1{k}'] = dh_pre.sum((0, 1))
        dm = dh_pre @ params[f'W1{k}'].T
        # Deshacer la LayerNorm ln2 (devuelve también dg y db)
        dx1, grads[f'ln2_g{k}'], grads[f'ln2_b{k}'] = layer_norm_bwd(dm, params[f'ln2_g{k}'],
c['ln2'])
        dX1 = dX + dx1                       # atajo residual + rama FFN

        # -- Subcapa 1 hacia atrás: X1 = Xin + att @ Wo + bo --
        grads[f'Wo{k}'] = tmm(c['att'], dX1)
        grads[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)   # de vuelta a 4 cabezas

        # Deshacer la suma ponderada: att = pesos @ Vm
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt

        # Deshacer el softmax de la atención, fila a fila:
        #     dscores_i = p_i · (dp_i − Σ_j dp_j·p_j)
        # y, en la misma línea, la división por sqrt(DH).
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)

        # Deshacer Q @ Kᵀ, y volver a unir las cabezas
        dQ = unir(dscores @ c['K'])
        dK = unir(dscores.transpose(0, 1, 3, 2) @ c['Q'])
        dV = unir(dVm)

        # Deshacer las tres proyecciones Q, K, V. Las tres parten de la
        # MISMA entrada a, así que sus contribuciones a da se SUMAN.
        # (Curiosidad: el gradiente de bk sale siempre 0 exacto. Sumar
        # el mismo vector a todas las claves suma la misma constante a
        # toda una fila de puntuaciones, y el softmax no cambia si a
        # toda la fila se le suma lo mismo. GPT-2 tiene bk de todos
        # modos; aquí se mantiene para que el modelo sea idéntico.)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            grads[f'W{n}{k}'] = tmm(c['a'], dn)
            grads[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        # Deshacer la LayerNorm ln1
        dxin, grads[f'ln1_g{k}'], grads[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin                      # atajo residual + rama de atención
```

```
    # --- Embeddings: X = wte[tokens] + wpe[:T] ---
    # Hacia wpe: cada posición recibe la suma de su gradiente en las
    # B secuencias del lote.
    grads['wpe'][:T] = dX.sum(0)
    # Hacia wte (2º uso, como tabla de entrada): un mismo carácter
    # aparece muchas veces en el lote, y cada aparición aporta su
    # gradiente a la MISMA fila. np.add.at los SUMA todos (una
    # asignación normal se quedaría solo con el último). Y se suman
    # también al gradiente de wte como matriz de salida, calculado
    # arriba: una matriz usada en dos sitios recibe los dos.
    np.add.at(grads['wte'], tokens, dX)
    return grads, loss


# ============================================================================
# LEARNING RATE: WARMUP LINEAL + DECAIMIENTO EN COSENO
# ============================================================================
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    """
    Calcula qué tasa de aprendizaje usar en el paso `t` del
    entrenamiento (de un total de `t_total` pasos). En vez de usar
    siempre el mismo LR fijo, se sigue una curva en dos tramos —el
    esquema estándar para entrenar modelos tipo Transformer:

      TRAMO 1 — "warmup" (calentamiento), los primeros `t_warmup`
      pasos: el LR sube LINEALMENTE desde 0 hasta lr_max. Al
      principio del entrenamiento los pesos son aleatorios y los
      gradientes pueden ser grandes y poco fiables; empezar con un
      LR muy bajo evita que esos primeros pasos, inestables, den
      saltos demasiado bruscos que desestabilicen el entrenamiento.

      TRAMO 2 — decaimiento en COSENO, el resto de pasos: el LR baja
      suavemente desde lr_max hasta lr_min, siguiendo la forma de
      media onda de coseno (empieza bajando despacio, acelera en el
      medio, y vuelve a suavizarse al final). Esto permite dar pasos
      grandes mientras el modelo aún tiene mucho que aprender, y
      pasos cada vez más pequeños y precisos a medida que se acerca
      a una buena solución — como "afinar" el resultado al final.

    Parámetros
    ----------
    t         : paso actual del bucle de entrenamiento (0, 1, 2...).
    lr_max    : tasa de aprendizaje máxima, alcanzada al final del warmup.
    t_total   : número total de pasos de entrenamiento (PASOS).
    t_warmup  : cuántos pasos dura la subida lineal inicial.
    lr_min    : tasa de aprendizaje mínima, al final del decaimiento.
    """
    if t < t_warmup:
        # Subida lineal. El +1 hace que el paso 0 ya aprenda (con
        # t / t_warmup, el primer paso tendría lr = 0 y no cambiaría
        # nada) y que en el paso t_warmup − 1 se alcance lr_max.
        return lr_max * (t + 1) / t_warmup
    # A partir de aquí, decaimiento en coseno desde lr_max hasta lr_min.
    t_adj = t - t_warmup           # pasos transcurridos DESDE que acabó el warmup
    t_decay = t_total - t_warmup   # duración total de la fase de decaimiento
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * t_adj / t_decay))


# ============================================================================
# BUCLE DE ENTRENAMIENTO: OPTIMIZADOR ADAM CON MINI-LOTES
# ============================================================================
# Adam ("Adaptive Moment Estimation") es el optimizador más usado en
# la práctica para entrenar redes neuronales (los LLMs reales usan
# su variante AdamW, 5.6). En vez de simplemente restar
# `LR * gradiente` a cada peso (como el descenso de gradiente más
# básico), Adam mantiene, PARA CADA PARÁMETRO POR SEPARADO, dos
# "memorias" que se actualizan en cada paso:
#   m (primer momento)  -> una media móvil del gradiente reciente
#                          (funciona como un "impulso": suaviza la
#                          dirección de la actualización).
#   v (segundo momento) -> una media móvil del CUADRADO del
#                          gradiente reciente (mide cuánto "varía"
#                          el gradiente de ese parámetro).
# Dividir por sqrt(v) hace que los parámetros con gradientes
# históricamente grandes y erráticos reciban pasos más pequeños
# (para no desestabilizarlos), y los que tienen gradientes pequeños
# y estables reciban pasos relativamente mayores — cada parámetro
# "aprende a su propio ritmo".

# m_adam, v_adam: un diccionario con la misma forma que `params`,
# inicializado todo a cero — son las "memorias" de Adam para cada
# matriz de parámetros, que se irán actualizando en cada paso.
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}

# Hiperparámetros estándar de Adam (los valores por defecto que
```

```
# recomienda el artículo original y que casi nunca hace falta tocar):
#   beta1 = 0.9   -> cuánto "recuerda" la media móvil del gradiente (m)
#   beta2 = 0.999 -> cuánto "recuerda" la media móvil del cuadrado (v)
#   eps_adam      -> número minúsculo para evitar dividir entre cero
#   t_adam        -> contador de pasos, usado para la "corrección de
#                    sesgo" (bias correction) de m y v — sin ella,
#                    m y v estarían artificialmente cerca de cero en
#                    los primeros pasos, porque empiezan en cero y
#                    aún no han "visto" suficientes gradientes.
beta1, beta2, eps_adam, t_adam = 0.9, 0.999, 1e-8, 0
LR_MAX = LR   # la tasa de aprendizaje máxima que usará el planificador coseno

print('\nEntrenando Nano-LLM (backpropagation analítico, mini-lotes)...')
print(f'CTX={CTX} | D={D} | D_FF={D_FF} | Bloques={N_BLQ} | Batch={BATCH}\n')

# --- BUCLE PRINCIPAL DE ENTRENAMIENTO ---
# Se repite PASOS veces. Cada iteración es un PASO de optimización:
# procesa solo BATCH secuencias elegidas al azar (no todo el texto)
# y actualiza los pesos una vez.
for paso in range(PASOS):

    # 1) Calcular la tasa de aprendizaje de este paso, según el
    #    esquema de warmup + coseno definido arriba.
    lr_actual = lr_warmup_coseno(paso, LR_MAX, PASOS)

    # 2) Elegir un MINI-LOTE de BATCH ventanas al azar. Para cada una
    #    se toma una posición de inicio dentro de DATOS (el texto ya
    #    convertido a números) y se extrae una ventana de CTX
    #    caracteres como entrada; la misma ventana desplazada un
    #    carácter a la derecha es el objetivo: "dado este trozo de
    #    texto, en cada posición viene después este carácter".
    inicios = np.random.randint(0, len(DATOS) - CTX - 1, size=BATCH)
    xb = np.stack([DATOS[i:i + CTX] for i in inicios])          # (BATCH, CTX)
    yb = np.stack([DATOS[i + 1:i + CTX + 1] for i in inicios])  # desplazado 1

    # 3) y 4) Forward + backward del mini-lote ENTERO de una vez. La
    #    pérdida y el gradiente ya salen PROMEDIADOS sobre las BATCH
    #    ventanas (dlogits se divide entre BATCH·CTX). Promediar da una
    #    estimación más estable de "hacia dónde" mover los pesos que
    #    actualizar tras cada ventana suelta.
    grads_acc, loss = forward_backward(xb, yb)

    # 5) RECORTE DEL GRADIENTE: se calcula la norma global del
    #    gradiente promedio (todas las matrices a la vez) y, si supera
    #    CLIP, se reescala todo el gradiente por el factor CLIP / norma.
    norma = np.sqrt(sum((g ** 2).sum() for g in grads_acc.values()))
    if norma > CLIP:
        for k in grads_acc:
            grads_acc[k] *= CLIP / norma

    # 6) ACTUALIZACIÓN DE ADAM: se aplica, a cada matriz de parámetros
    #    por separado, la fórmula completa de Adam.
    t_adam += 1
    for k in params:
        g = grads_acc[k]
        # Actualizar las medias móviles del gradiente (m) y de su
        # cuadrado (v) para este parámetro:
        m_adam[k] = beta1 * m_adam[k] + (1 - beta1) * g
        v_adam[k] = beta2 * v_adam[k] + (1 - beta2) * g ** 2
        # Corrección de sesgo: en los primeros pasos (t_adam pequeño),
        # m_adam y v_adam están artificialmente cerca de 0 porque
        # empezaron en 0; dividir entre (1 - beta^t) lo compensa.
        m_h = m_adam[k] / (1 - beta1 ** t_adam)
        v_h = v_adam[k] / (1 - beta2 ** t_adam)
        # Actualización final del peso: se mueve en la dirección
        # contraria al gradiente (para REDUCIR la pérdida), con un
        # tamaño de paso que combina la tasa de aprendizaje actual
        # (lr_actual) y la adaptación por parámetro que aporta Adam.
        params[k] -= lr_actual * m_h / (np.sqrt(v_h) + eps_adam)

    # 7) Cada 1000 pasos (y en el último), imprimir el progreso:
    #    la pérdida actual (y
    #    su equivalente en "bits por carácter", una forma alternativa
    #    e intuitiva de leer la misma pérdida: cuántos bits de
    #    información hacen falta, en promedio, para "corregir" al
    #    modelo en cada carácter — cuanto más bajo, mejor predice).
    if paso % 1000 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida = {loss:.4f}   lr = {lr_actual:.5f}   '
              f'(bits/char = {loss/np.log(2):.2f})')


# ============================================================================
# ESTRATEGIAS DE MUESTREO: ¿CÓMO ELEGIR EL SIGUIENTE CARÁCTER?
# ============================================================================
# El modelo, para cada posición, produce una distribución de
# probabilidad sobre los 46 caracteres posibles (gracias a softmax).
```

```
# Pero de esa distribución hay que ELEGIR uno solo para continuar el
# texto. Existen varias estrategias, cada una con sus ventajas:

def greedy(probs):
    """
    Estrategia "voraz": elegir SIEMPRE el carácter de mayor
    probabilidad, sin ninguna aleatoriedad. `np.argmax` devuelve el
    índice del valor más alto del array.

    Ventaja: el resultado es siempre el mismo (determinista) dado el
    mismo modelo y el mismo texto de partida.
    Inconveniente: el modelo puede quedar "atrapado" repitiendo el
    mismo patrón una y otra vez, porque si un carácter es el más
    probable en un contexto, seguirá siéndolo al volver a aparecer
    ese mismo contexto — produce bucles como "el modelo el modelo
    el modelo...".

    (Esta función se deja como referencia / para experimentar; la
    generación por defecto de este programa usa temperatura + top-p,
    ver `generar()` más abajo.)
    """
    return np.argmax(probs)


def top_k_filtrar(probs, k):
    """
    "Top-k sampling": se queda solo con los `k` caracteres de mayor
    probabilidad, pone a CERO la probabilidad de todos los demás, y
    renormaliza (para que las probabilidades restantes vuelvan a
    sumar 1). Después, se muestrea al azar entre esos k candidatos
    según sus probabilidades relativas.

    Con k=1 equivale a `greedy` (solo puede elegir el más probable).
    Con k grande, se acerca al muestreo "puro" (casi sin filtrar).

    Limitación: k es un número FIJO, sin importar si la distribución
    de probabilidad está muy concentrada (pocos candidatos buenos) o
    muy repartida (muchos candidatos razonables) — de ahí que top-p
    (más abajo) suela funcionar mejor en la práctica.
    """
    indices_ord = np.argsort(probs)[::-1]      # índices ordenados de mayor a menor prob.
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:k]] = probs[indices_ord[:k]]   # conservar solo los k mejores
    return nueva / nueva.sum()                  # renormalizar para que sumen 1


def top_p_filtrar(probs, p):
    """
    "Top-p sampling" o "nucleus sampling": en vez de fijar CUÁNTOS
    caracteres conservar (como top-k), se fija CUÁNTA probabilidad
    acumulada se quiere cubrir. Se ordenan los caracteres de mayor a
    menor probabilidad y se van sumando uno a uno hasta alcanzar
    (como mínimo) `p` de probabilidad acumulada — esos son los
    únicos candidatos que se conservan; el resto se descarta.

    Ventaja sobre top-k: el número de candidatos se ADAPTA a la
    forma de la distribución en cada paso. Si el modelo está muy
    "seguro" (una probabilidad domina claramente), bastarán 1 o 2
    caracteres para cubrir el p%. Si el modelo está más "indeciso"
    (probabilidades repartidas), se conservarán más candidatos.
    Esto es justo lo que se usa en `generar()` para evitar elegir
    caracteres de probabilidad residual ("ruido") que producirían
    texto sin sentido.
    """
    indices_ord = np.argsort(probs)[::-1]        # de mayor a menor probabilidad
    probs_ord   = probs[indices_ord]
    suma_acum   = np.cumsum(probs_ord)           # suma acumulada: 0.4, 0.7, 0.85, ...
    # Buscar cuántos candidatos hacen falta para llegar (o superar) a p:
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
    return nueva / nueva.sum()                   # renormalizar


# ============================================================================
# GENERACIÓN DE TEXTO: usar el modelo ya entrenado para "escribir"
# ============================================================================
def generar(semilla, n_chars=120, temperatura=0.8, top_p=0.9):
    """
    Genera texto nuevo, carácter a carácter, de forma AUTORREGRESIVA:
    en cada paso se predice UN carácter, se añade al final del texto
    generado hasta ahora, y ese texto (ya un carácter más largo) se
    usa como entrada para predecir el SIGUIENTE carácter — y así
    sucesivamente hasta completar `n_chars` caracteres nuevos.

    Parámetros
```

```
    ----------
    semilla     : texto inicial (string) que "arranca" la generación.
    n_chars     : cuántos caracteres nuevos generar.
    temperatura : controla la aleatoriedad de la elección.
                  - temperatura = 1.0 → probabilidades tal cual las
                    predice el modelo, sin modificar.
                  - temperatura < 1.0 → "afila" la distribución
                    (favorece aún más a los caracteres ya probables):
                    salida más determinista y repetitiva.
                  - temperatura > 1.0 → "aplana" la distribución
                    (da más oportunidad a caracteres poco probables):
                    salida más aleatoria y creativa, con riesgo de
                    volverse incoherente si se sube demasiado.
    top_p       : parámetro del nucleus sampling (ver
                  `top_p_filtrar` arriba). Con top_p=1.0 se desactiva
                  el filtrado y se muestrea de la distribución
                  completa (tras aplicar la temperatura).
    """
    # Convertir la semilla de texto a una lista de índices enteros,
    # usando como máximo los últimos CTX caracteres (el modelo no
    # puede usar un contexto más largo que su ventana CTX).
    # `C2I.get(c, 0)`: si algún carácter de la semilla no estuviera
    # en el vocabulario, se usa el índice 0 como valor por defecto
    # en vez de fallar con un error.
    tokens = [C2I.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla   # aquí se irá acumulando el texto completo generado

    for _ in range(n_chars):
        # Tomar los últimos CTX tokens como contexto actual.
        ctx = tokens[-CTX:]
        # NO se rellena si el contexto aún es más corto que CTX (al
        # principio de la generación, con semillas cortas): forward()
        # acepta cualquier longitud real gracias a la máscara
        # recortada [:T, :T]. Rellenar con un token artificial
        # desplazaría la posición relativa de los caracteres reales
        # respecto a las ventanas vistas en el entrenamiento.

        # Forward del modelo sobre el contexto actual. forward()
        # trabaja con lotes, así que el contexto se mete en un lote de
        # UNA secuencia: np.array([ctx]), de forma (1, T). Devuelve los
        # logits (1, T, V) y el cache (que aquí no se usa); solo
        # interesa la ÚLTIMA posición, la predicción de "qué viene
        # después del último carácter del contexto": [0][0, -1].
        logits = forward(np.array([ctx]))[0][0, -1]

        # Aplicar la temperatura ANTES del softmax: dividir los
        # logits por `temperatura` cambia cuánto se "exageran" las
        # diferencias entre ellos una vez calculado el softmax.
        probs  = softmax(logits / temperatura)

        # Aplicar nucleus sampling (top-p), salvo que top_p=1.0 (en
        # cuyo caso no se filtra nada y se usa la distribución
        # completa tal cual).
        if top_p < 1.0:
            probs = top_p_filtrar(probs, top_p)

        # Elegir el siguiente carácter MUESTREANDO al azar según las
        # probabilidades finales (no simplemente el más probable):
        # así, ejecuciones distintas pueden generar textos distintos,
        # aunque partan de la misma semilla.
        siguiente = int(np.random.choice(V, p=probs))

        # Añadir el nuevo token a la lista de contexto (para que
        # influya en la predicción del SIGUIENTE carácter) y su
        # versión en texto al resultado final.
        tokens.append(siguiente)
        resultado += I2C[siguiente]

    return resultado


# ------------------------------------------------------------------------------
# DEMOSTRACIÓN FINAL: generar un par de ejemplos de texto con el modelo
# ya entrenado, a partir de dos semillas distintas.
# ------------------------------------------------------------------------------
print('\n=== TEXTO GENERADO POR EL NANO-LLM ===')
print(generar('la inteligencia', n_chars=120))
print()
print(generar('los modelos de', n_chars=120))

# ------------------------------------------------------------------------------
# GUARDAR LOS PESOS ENTRENADOS
# ------------------------------------------------------------------------------
# np.savez guarda todas las matrices de `params` en un único archivo.
# Los Capítulos 11, 14, 15 (Programa RZ-1) y 18 lo cargan con
#   params = dict(np.load('nano_llm_pesos.npz'))
```

```
# para usar el modelo ya entrenado sin repetir el entrenamiento.
np.savez('nano_llm_pesos.npz', **params)
print("\nPesos guardados en nano_llm_pesos.npz")
```

> ℹ️ **Misma arquitectura, misma salida**
>
> La versión comentada ejecuta exactamente el mismo cálculo que la de 13.5.1 (misma arquitectura, mismos hiperparámetros, misma semilla). Lo hemos comprobado ejecutando las dos: producen, carácter a carácter, la misma salida.

13.6 ¿Qué aprendió el Nano-LLM?

Con 60 000 parámetros y entrenado sobre 93 líneas de texto (5076 caracteres), nuestro Nano-LLM muestra, a primera vista, comportamientos muy interesantes:

- Aprendió a generar texto en español gramaticalmente correcto, con concordancia de género, número y conjugación; aunque, como verás enseguida, sobre todo porque reproduce el texto con el que se entrenó.
- Aprendió la estructura de las frases: sujeto, verbo, complemento, y el punto final que cierra cada oración.
- La pérdida bajó de 3.85 (≈ ln 46, la de no haber aprendido nada) a 0.29 en el último mini-lote, es decir, 0.41 bits por carácter; medida sobre todo el texto de entrenamiento es 0.28. Atención: es la pérdida en el mismo texto que el modelo ha recorrido unas 454 veces.
- Con más texto de entrenamiento, más parámetros y más pasos, el modelo generaría texto de mayor calidad. GPT-2 (2019) tiene 124 millones de parámetros en su versión pequeña y 1500 millones en la mayor (GPT-2 XL), entrenadas sobre unos 40 GB de texto de Internet; GPT-3 tiene 175 000 millones. El principio es idéntico al nuestro; la escala es radicalmente diferente.

Un aviso importante: sobreajuste

Una pérdida de 0.28 (la media sobre todo el corpus) es, en términos absolutos, muy baja: la perplejidad equivalente (ver Ejercicio 13.3) es 1.33, es decir, el modelo apenas duda entre 1 y 2 candidatos por carácter en promedio. Esto no es casualidad ni una hazaña: es la consecuencia directa de tener 60 000 parámetros para memorizar apenas 5076 caracteres de texto —casi 12 parámetros "disponibles" por cada carácter de entrenamiento—. En ese régimen, el modelo tiene capacidad de sobra para memorizar el corpus casi literalmente, en vez de aprender un patrón general del idioma que se generalice a frases nuevas.

Se aprecia claramente en los dos ejemplos generados: los dos recitan, letra por letra y en el mismo orden, el principio del corpus. El primero reproduce las dos primeras frases y empieza la tercera («las redes neuronales tienen capas»); el segundo, a partir de «los modelos de», sigue con la segunda frase, la tercera y el principio de la cuarta («el aprendizaje profundo usa muchas»). El modelo no está «inventando» gramática nueva: está recitando, palabra por palabra, el propio corpus. Con un dataset de este tamaño y un modelo de esta capacidad, esa es exactamente la conducta esperable, y es una buena primera lección práctica sobre el sobreajuste del Capítulo 7, que en el Capítulo 15 mediremos con rigor.

Conviene decirlo sin rodeos, porque es fácil pasarlo por alto: lo que has construido en este capítulo es una prueba de concepto de la arquitectura Transformer —demuestra que el mecanismo de atención, el backpropagation exacto y el bucle de entrenamiento funcionan tal como se han descrito—, no una demostración de la capacidad de generalización de un LLM real. Con 93 líneas recorridas unas 454 veces cada una, el modelo memoriza; con billones de tokens distintos, un LLM real generaliza. Es la misma arquitectura y el mismo principio matemático, pero la diferencia de escala no es un matiz menor: es la diferencia entre memorizar un texto y modelar un idioma. Si terminas este capítulo pensando que has entendido por qué GPT-4 "sabe cosas", vuelve a leer este párrafo: lo que has visto es cómo memoriza un modelo pequeño un texto pequeño, que es el primer paso necesario, pero no el mismo fenómeno.

La pregunta que probablemente te estás haciendo ahora mismo es la buena: ¿qué tendría que cambiar, aunque sea en teoría, para que este mismo Nano-LLM se convirtiera en un GPT-4? La respuesta corta es tranquilizadora: ningún ingrediente mágico oculto. La arquitectura es la de GPT-2 (13.1), que acabas de programar; los modelos actuales solo le

han cambiado algunas piezas (cuadro «Lo que ha cambiado desde GPT-2»). Lo que cambia es la escala en varios ejes a la vez: la ventana de contexto (de CTX=24 caracteres a cientos de miles de tokens, o incluso un millón), la profundidad (de 2 bloques a decenas), la tokenización (de carácter a subpalabra) y, sobre todo, datos y parámetros creciendo juntos varios órdenes de magnitud. El Capítulo 17 cuantifica exactamente esa distancia y explica por qué, al crecer la escala, parecen aparecer de golpe capacidades nuevas, y cuánto de ese salto depende de cómo se mida. Lo que ya sabes hacer —forward, backward, atención, mini-lotes— es exactamente lo que hace un LLM real; lo que falta es, literalmente, más. Y lo comprobarás en el Capítulo 16: este mismo código, con los pesos reales de GPT-2, escribe inglés fluido.

> La pérdida en bits/carácter es un buen indicador de aprendizaje, pero no distingue por sí sola entre «aprender el idioma» y «memorizar el texto». Para esa distinción hace falta evaluar sobre texto que el modelo no haya visto durante el entrenamiento: un conjunto de validación (7.5). Es exactamente lo que haremos en el Capítulo 15 con el Programa RZ-1: apartaremos 10 de las 93 frases, y el resultado será revelador.

Queda, por tanto, una pregunta incómoda, y es la que abre el Capítulo 15: ¿el Nano-LLM ha entendido algo, o simplemente lo recuerda? Y, si solo recuerda, ¿podría esta misma arquitectura, con otros datos, aprender una regla de verdad?

13.6.1 Comprobación empírica — ¿arregla el ajuste de hiperparámetros el sobreajuste?

El aviso anterior se puede comprobar, no solo enunciar. Hemos ejecutado el Programa 8 seis veces, cambiando en cada una un único aspecto del entrenamiento, y hemos medido cada modelo con las mismas pruebas de copia literal que usa el Programa RZ-1 del Capítulo 15: 40 textos generados de 150 caracteres (10 por cada semilla: «la inteligencia», «un modelo », «el gato » y «mañana la »), con temperatura 0.8 y top-p 0.9. La columna «copia» es el porcentaje de caracteres generados que forman parte de algún fragmento de 20 o más caracteres que aparece tal cual en el corpus; «líneas idénticas», cuántas líneas completas generadas son una frase del corpus; «inventadas», cuántas palabras distintas no existen en el corpus. La pérdida está medida sobre todo el texto de entrenamiento.

Los dos tipos de weight decay se programan cambiando la actualización de Adam (`wd` es la intensidad):

```
# Weight decay estilo AdamW: la penalización se multiplica por lr_actual
params[k] -= lr_actual * (m_h / (np.sqrt(v_h) + eps_adam) + wd * params[k])

# Decay constante: la penalización NO depende de lr_actual
params[k] -= lr_actual * m_h / (np.sqrt(v_h) + eps_adam)
params[k] -= wd * params[k]
```

Configuración	Pérdida	Copia	Líneas idénticas	Inventadas	Texto generado
Original (12 000 pasos, sin regularizar)	0.284	94 %	69 de 73	15	Correcto, casi todo copiado
Entrenamiento corto (3000 pasos)	0.622	14 %	0 de 79	263	Roto: «limiza», «prámictes»
Weight decay 0.01 estilo AdamW	0.283	96 %	74 de 76	13	Igual que el original
Weight decay 1.0 estilo AdamW	1.919	0 %	0 de 37	654	No se reconoce como español
Decay constante 0.00005	0.319	83 %	33 de 58	61	Correcto casi siempre; empalma más
Decay constante 0.0005	2.544	0 %	0 de 15	721	Destruido: la pérdida sube al final

El resultado con weight decay = 0.01 merece explicación, porque a primera vista debería haber ayudado y no lo hizo. La explicación más probable (no la hemos aislado con un experimento) es un detalle técnico fácil de pasar por alto: al implementarse como weight decay desacoplado (estilo AdamW), la penalización se multiplica por el learning rate del paso actual (lr_actual). Como este programa usa decaimiento coseno, en la fase final del entrenamiento —justo donde ocurre la memorización— el lr ya ha caído a ~0.0001, así que la penalización efectiva es lr × weight_decay = 0.000001: prácticamente cero cuando más falta hace.

ℹ️ Ninguna variante resuelve el problema. Acortar el entrenamiento o regularizar con fuerza reduce la copia literal, pero a costa del idioma: el modelo deja de saber escribir (263, 654 y 721 palabras inventadas). El weight decay suave al estilo de AdamW no cambia nada: el 96 % del texto generado sigue dentro de fragmentos literales del corpus. Con este corpus, el techo está en la cantidad de texto de entrenamiento, no en el algoritmo de optimización, el mismo principio que, a mayor escala, explica por qué los LLM necesitan a la vez muchos parámetros y muchísimo texto (Capítulo 17). Hacer early stopping de verdad exige un conjunto de validación: lo haremos en el Ejercicio 15.1.

La configuración de decay constante ataca la causa diagnosticada arriba: en vez de una penalización que se multiplica por el LR del paso actual (y por tanto se desvanece justo cuando el LR cae a casi 0), aplica en cada paso una penalización de magnitud constante. Es la única que cambia algo sin romper nada: la pérdida apenas sube (0.319 frente a 0.284), la copia literal baja del 94 al 83 % y las líneas idénticas a una frase del corpus, de 69 de 73 a 33 de 58. El modelo pasa de recitar frases completas a empalmar trozos y, a

veces, a inventar palabras (61 distintas, frente a 15). Pero no es generalización: el 83 % del texto sigue dentro de fragmentos literales de 20 caracteres o más. Y el ajuste es delicado: con un decay constante 10 veces mayor (0.0005), al final del entrenamiento, cuando el paso de Adam ya es minúsculo, la penalización domina, borra lo aprendido más deprisa de lo que el modelo puede reaprenderlo y la pérdida vuelve a subir (de 1.03 en el paso 9000 a 2.56 al final).

DIAGRAMA 13.1 — EL NANO-LLM DE EXTREMO A EXTREMO

Resumen visual del Capítulo 13 — el Programa 8 de principio a fin

Texto → Tokenizar → Embeber → 2 Bloques Transformer → Softmax → Siguiente carácter

La arquitectura de GPT-2, a escala nano: 60 000 parámetros

ARQUITECTURA COMPLETA

GPT-2 pieza a pieza, con tamaños diminutos

1. Texto de entrenamiento
93 frases · 5076 caracteres

↓

2. Tokenizador (nivel carácter) — (24,)
cada carácter → un número entero
'la red' → [23, 12, 1, 29, 16, 15]
V = 46 caracteres

↓

3. Embedding de tokens + embedding posicional — (24 × 48)
x = wte[tokens] + wpe[0:24]
wte: 46 × 48 wpe: 24 × 48

↓

4. 2 Bloques Transformer (Pre-LN, como GPT-2) — (24 × 48)
x = x + Atención(LN(x)) ← causal, 4 cabezas
x = x + FFN(LN(x)) ← 48 → 192 → 48, GELU
× 2 bloques
máscara causal cada posición solo ve el pasado

↓

5. LayerNorm final + capa de salida — (24 × 46)
logits = LN(x) · wte^T
pesos compartidos: la misma tabla wte de la entrada

↓

6. Softmax → probabilidades
P(siguiente carácter | contexto)
46 probabilidades que suman 1

↓

7. Siguiente carácter
muestreo con temperatura 0.8 y top-p 0.9

PARÁMETROS DEL NANO-LLM

V = 46	CTX = 24
D = 48	D_FF = 192
cabezas = 4	bloques = 2

Total: 60 000 parámetros

wte 2208 · wpe 1152 · 2 bloques × 28 272 · LN final 96

cómo se entrena

ENTRENAMIENTO AUTORREGRESIVO

El modelo aprende a predecir el siguiente carácter

① Preparar un mini-lote:
8 ventanas al azar del corpus, cada una de 25 caracteres (CTX + 1)

los modelos de lenguaje p

X = entrada = ventana[0:24]
'los modelos de lenguaje '

y = objetivo = ventana[1:25]
'os modelos de lenguaje p'

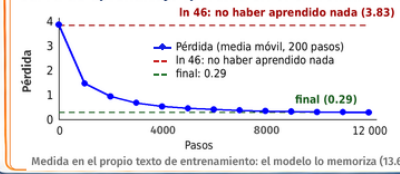
en cada posición t, X[0..t] debe predecir y[t]
objetivo desplazado 1 posición → 8 × 24 = 192 predicciones por paso

② Forward pass:
X (8 × 24) → Nano-LLM → logits (8 × 24 × 46)
logits[t] → softmax → P sobre los 46 caracteres

③ Pérdida — entropía cruzada:
$L = -(1/192) \cdot \Sigma \ln P(y_t \mid x_1 \ldots x_t)$
Paso 0: L = 3.85 = ln 46 = 3.83 (azar)
Paso 11 999: L = 0.29 (0.41 bits/carácter)
bits/carácter = L / ln 2
→ la pérdida baja: el modelo aprende el texto

④ Backprop + recorte + Adam:
grads = backward(L) ← regla de la cadena, exacta
si |grads| > 1: recortar ← CLIP = 1.0
params = adam(params, grads, lr)
lr: warm-up 150 pasos → coseno de 0.003 a 0.0001
12 000 pasos · unos 2.5 min en un portátil

Curva de aprendizaje (pérdida en el entrenamiento)



Medida en el propio texto de entrenamiento: el modelo lo memoriza (13.6)

cómo genera texto

GENERACIÓN DE TEXTO

Autorregresiva: un carácter cada vez

El bucle de generación:

① Contexto actual: 'un modelo ' [32, 25, 1, 24, 26, 15, 16, 23, 26, 1]
② Forward pass: contexto → Nano-LLM → logits; nos quedamos con la última posición
③ Elegir el siguiente carácter: P('g')=0.31 P('e')=0.20 P('r')=0.12 P('i')=0.11 (con temperatura 0.8)
④ Añadirlo al contexto: 'un modelo g' — si pasa de 24 caracteres, se usan los 24 últimos
repetir para cada carácter nuevo

Temperatura y top-p:
$P_i = \exp(\text{logit}_i / \tau) / \Sigma_j \exp(\text{logit}_j / \tau)$

Probabilidad de 'g' tras 'un modelo ' (valor real):

τ = 0.5	P('g') = 0.45 — más concentrada, más predecible
τ = 0.8	P('g') = 0.31 — la del Programa 8 ★
τ = 2.0	P('g') = 0.17 — casi plana, errática

τ → 0: siempre el más probable · τ → ∞: todos igual de probables

top-p = 0.9: se sortea solo entre los 6 más probables (los que juntos suman al menos el 90 %)

Texto generado por el Nano-LLM (salida real):
Semilla: 'la inteligencia'
→ 'la inteligencia artificial aprende de los datos.'
→ 'los modelos de lenguaje predicen el siguiente token.'

⚠ Son frases del corpus, en el mismo orden: el modelo recita, no inventa.

Con 60 000 parámetros y 5076 caracteres, memoriza (13.6 y Capítulo 15).

conexión con GPT-2

TU NANO-LLM FRENTE A GPT-2

La misma arquitectura, otra escala:

Concepto	Tu Nano-LLM	GPT-2 small (2019)
Parámetros	60 000	124 439 808
Bloques	2	12
Cabezas	4 (de 12 dim.)	12 (de 64 dim.)
Dimensión D	48	768
FFN (D_FF)	192	3072
Contexto	24 caracteres	1024 tokens
Vocabulario	46 caracteres	50 257 tokens BPE
Datos	5 KB (93 frases)	40 GB (WebText)
Pesos en disco	0.49 MB	548 MB

≈ 2000 veces más parámetros

Piezas idénticas en los dos:

- ✓ Embeddings de tokens y de posición aprendidos (wte, wpe)
- ✓ Bloques Pre-LN con atención causal multi-cabeza
- ✓ FFN con GELU
- ✓ LayerNorm final
- ✓ Salida con pesos compartidos (wte)
- ✓ Entrenamiento: predecir el siguiente token
- ✓ Generación: un token cada vez, con temperatura y top-p

Solo cambian dos cosas fuera de la arquitectura: GPT-2 usa dropout y tokenizador BPE.

💡 Lo que demuestra el Programa 8:

La arquitectura funciona: atención, backward exacto y bucle de entrenamiento.

En el Capítulo 16, el mismo forward() carga los pesos reales de GPT-2 small.

Lo que aún no demuestra: generalizar. Para eso hacen falta muchos más datos.

Hacia los modelos actuales: Diagrama 17.1 (Capítulo 17).

```
x = wte[tokens] + wpe[0:24]      ← entrada
x = Bloque(x)  × 2   (Pre-LN)    ← proceso
P = softmax(LN(x) · wte^T)       ← salida
```

Tokenizar → Embeber → Transformar
Entrenar: predecir el siguiente carácter
Generar: un carácter cada vez

Azul = Entrada / tokenizador / embeddings
Verde azulado = Bloques Transformer
Verde = Salida / texto generado
Naranja = Pérdida / entrenamiento
Morado = Generación / muestreo
Dorado = Parámetros / formas

Texto de entrenamiento (93 líneas, 5076 caracteres) → Tokenizador (carácter → entero) → Mini-lote de 8 ventanas aleatorias de longitud CTX=24 (X = entrada, y = objetivo desplazado 1 posición).
1. Embedding de token (V×D) + Embedding posicional (CTX×D), con D = 48. Forma: (24,) → (24 × 48); por ejemplo, «la red» → 23 12 1 29 16 15.

2. Bloque Transformer × 2: LayerNorm → Self-Attention causal (4 cabezas) → Residual; LayerNorm → FFN (D→192→D, GELU) → Residual (Pre-LN, como GPT-2). Forma: (24 × 48) → (24 × 48).

3. LayerNorm final + capa de salida (D→V, con la matriz del embedding de tokens) + softmax → distribución de probabilidad sobre 46 caracteres. Forma: (24 × 48) → (24 × 46).

4. Pérdida (entropía cruzada) → Backpropagation exacto (regla de la cadena) sobre los 60 000 parámetros → recorte del gradiente (CLIP = 1.0).

5. Adam + LR con warm-up (150 pasos) y decaimiento coseno → actualización de pesos (12 000 pasos).

6. Generación: forward → temperatura (τ=0.8) → top-p nucleus sampling (p=0.9) → muestreo → siguiente carácter.

Este esquema incorpora el mini-lote, el backpropagation exacto y el programa de tasa de aprendizaje como etapas explícitas del pipeline, y refleja que la generación combina temperatura y top-p (un adelanto de las estrategias de muestreo que se estudian en profundidad en el Capítulo 14).

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- El Nano-LLM ensambla embeddings, codificación posicional y bloques Transformer en un modelo generativo completo.
- El entrenamiento autorregresivo consiste en predecir el siguiente token dado todo el contexto anterior.
- El backpropagation exacto (regla de la cadena) calcula, en una sola pasada, el gradiente exacto de todos los parámetros del modelo.
- Los mini-lotes promedian el gradiente de varias secuencias por paso, suavizando el ruido del entrenamiento.
- Un programa de tasa de aprendizaje (warm-up + decaimiento coseno) estabiliza el arranque y afina el final del entrenamiento.
- El Nano-LLM es GPT-2 en miniatura: las mismas piezas (embeddings aprendidos, Pre-LN, 4 cabezas, GELU, LayerNorm final y pesos compartidos), con tamaños diminutos.
- Cada gradiente del backward escrito a mano se puede verificar con diferencias finitas; con los pesos entrenados, el error relativo máximo es de $1.3 \cdot 10^{-8}$.
- El recorte del gradiente (CLIP = 1.0) actúa en el 98 % de los pasos de este modelo: fija el tamaño del gradiente que recibe Adam.
- Con 60 000 parámetros, el modelo ya genera texto gramaticalmente correcto — el mismo principio que los LLMs comerciales — aunque a este tamaño y con este corpus, gran parte de lo "aprendido" es memorización, no generalización.

✏️ EJERCICIOS — Capítulo 13 — El Nano-LLM

Has construido tu primer modelo de lenguaje; ahora toca experimentar con él. Intenta estos ejercicios antes de seguir y compara tus resultados con los propuestos para entender cada cambio.

Ejercicio 13.1 Cambiar el texto de entrenamiento

Modifica el Programa 8 para entrenar sobre un texto diferente: usa una lista de 8 a 10 frases cortas sobre programación en Python (bucles, funciones, listas, etc.). Observa cómo cambia el vocabulario, la pérdida inicial esperada y el texto generado. Con un corpus tan pequeño (8-10 frases) frente a las 93 líneas del Programa 8, es razonable reducir PASOS (por ejemplo, a 3000) para no sobreajustar aún más de lo inevitable.

> 💡 **Pista**
>
> La pérdida inicial depende del vocabulario: ln(V). Si el nuevo texto tiene V=31 caracteres únicos, la pérdida inicial debe ser ≈ ln(31) ≈ 3.43. El modelo generará texto temático sobre Python.
> Con D = 48 y 3000 pasos, el entrenamiento dura alrededor de medio minuto.

✅ Solución:

📄 ejercicio_13_1.py

```
# ============================================================
# EJERCICIO 13.1 — Entrenar el Nano-LLM con otro texto
# Reutiliza el Programa 8 tal cual: solo sustituye TEXTO,
# reduce PASOS y guarda los pesos con otro nombre.
# ============================================================
import numpy as np


NUEVO_TEXTO = """
python es el lenguaje de la inteligencia artificial.
las listas se crean con corchetes: [1, 2, 3].
los diccionarios usan llaves y pares clave valor.
las funciones se definen con la palabra reservada def.
los bucles for recorren secuencias de elementos.
las clases permiten crear objetos con metodos y atributos.
importa numpy para operaciones numericas eficientes.
las redes neuronales aprenden de los datos de entrenamiento.
"""



# Análisis del nuevo vocabulario (antes de entrenar)
vocab = sorted(set(NUEVO_TEXTO))
V = len(vocab)
print(f'Texto: {len(NUEVO_TEXTO)} caracteres | vocabulario: {V} caracteres únicos')
print(f'Pérdida inicial esperada: ln({V}) = {np.log(V):.4f}\n')

# Programa 8 con tres cambios: TEXTO, PASOS y nombre del archivo de pesos
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
viejo = fuente.split('TEXTO = """')[1].split('"""')[0]
fuente = fuente.replace(viejo, NUEVO_TEXTO)
fuente = fuente.replace('PASOS  = 12000', 'PASOS  = 3000')
fuente = fuente.replace('nano_llm_pesos.npz', 'pesos_ejercicio_13_1.npz')
exec(fuente)
```

> Salida del programa:

```
Texto: 427 caracteres | vocabulario: 31 caracteres únicos
Pérdida inicial esperada: ln(31) = 3.4340

Vocabulario: 31 caracteres
Total de parámetros: 59,280
Texto de entrenamiento: 427 tokens, 8 líneas

Entrenando Nano-LLM (backpropagation analítico, mini-lotes)...
```

```
CTX=24 | D=48 | D_FF=192 | Bloques=2 | Batch=8

Paso     0: pérdida = 3.4565   lr = 0.00002   (bits/char = 4.99)
Paso  1000: pérdida = 0.1853   lr = 0.00241   (bits/char = 0.27)
Paso  2000: pérdida = 0.1503   lr = 0.00090   (bits/char = 0.22)
Paso  2999: pérdida = 0.0953   lr = 0.00010   (bits/char = 0.14)

=== TEXTO GENERADO POR EL NANO-LLM ===
la inteligencia artificial.
las listas se crean con corchetes: [1, 2, 3].
los diccionarios usan llaves y pares clave valor.
las funcion

los modelos dene entos.
las clases permiten crear objetos con metodos y atributos.
importa numpy para operaciones numericas eficientes

Pesos guardados en pesos_ejercicio_13_1.npz
```

La pérdida inicial ln(V) es la referencia de «no haber aprendido nada»: si el modelo asigna probabilidad uniforme 1/V a todos los tokens, la entropía cruzada es exactamente ln(V). Cualquier valor por debajo indica que el modelo ha aprendido algo. Fíjate también en el final: con solo 427 caracteres de texto, 3000 pasos bastan para que la pérdida baje a 0.10 y el modelo recite las frases casi de memoria.

Ejercicio 13.2 Explorar el efecto de la temperatura

Con el Nano-LLM ya entrenado (los pesos que guardó el Programa 8 en nano_llm_pesos.npz), genera texto con la misma semilla pero cinco temperaturas diferentes (0.2, 0.5, 0.8, 1.2 y 2.0), dejando top_p=1.0 para aislar solo el efecto de la temperatura. Describe cómo cambia el texto en cada caso: ¿cuándo es más coherente? ¿cuándo más creativo? ¿cuándo se vuelve ininteligible?

💡 Pista

T < 0.5 → texto repetitivo y conservador (casi siempre elige el token más probable, reforzando la memorización descrita en 13.6). T ≈ 0.8 → buen balance. T > 1.5 → el modelo empieza a elegir tokens improbables y el texto pierde coherencia.

✅ Solución:

📄 ejercicio_13_2.py

```
# ============================================================
# EJERCICIO 13.2 — El efecto de la temperatura, con el modelo real
# Carga los pesos que guardó el Programa 8 y genera desde la
# misma semilla con cinco temperaturas (top_p = 1.0 para aislar
# el efecto de la temperatura).
# ============================================================
import numpy as np



fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec(fuente.split('# -- Learning rate')[0])              # datos, parámetros, forward
exec('def greedy' + fuente.split('def greedy')[1]
                    .split("print('\\n=== TEXTO")[0])   # generar() y filtros
params.update(np.load('nano_llm_pesos.npz'))             # el modelo ya entrenado

# Cuánto se concentra la probabilidad del siguiente carácter
contexto = [C2I[c] for c in 'un modelo ']
logits = forward(np.array([contexto]))[0][0, -1]   # lote de 1; última posición
print('\nProbabilidad del carácter más probable tras «un modelo »:')
for tau in [0.2, 0.5, 0.8, 1.2, 2.0]:
    p = softmax(logits / tau)
    print(f'  τ = {tau:.1f}: {p.max():.3f}  («{I2C[int(p.argmax())]}»)')
```

```
print('\nTexto generado (semilla «un modelo», top_p = 1.0):')
for tau in [0.2, 0.5, 0.8, 1.2, 2.0]:
    np.random.seed(0)
    texto = generar('un modelo', n_chars=90, temperatura=tau, top_p=1.0)
    print(f'\n--- τ = {tau} ---')
    print(texto)
```

> Salida del programa:

```
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas

Probabilidad del carácter más probable tras «un modelo »:
  τ = 0.2: 0.810  («g»)
  τ = 0.5: 0.445  («g»)
  τ = 0.8: 0.311  («g»)
  τ = 1.2: 0.234  («g»)
  τ = 2.0: 0.167  («g»)

Texto generado (semilla «un modelo», top_p = 1.0):

--- τ = 0.2 ---
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre c

--- τ = 0.5 ---
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre c

--- τ = 0.8 ---
un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la precisión permite

--- τ = 1.2 ---
un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la precisión permite

--- τ = 2.0 ---
un modelo memoriza dembed.
unconteniperapo a ortepar todo.
una camina la entoriónal.
ediccial de lo
```

La semilla «un modelo» está elegida a propósito: en el corpus le siguen seis continuaciones distintas («pequeño», «(bien entrenado)», «grande», «robusto», «generativo», «discriminativo»), así que el modelo no tiene una única respuesta memorizada. Con $\tau = 0.2$ el carácter más probable, la «g», concentra el 81 % de la probabilidad; con $\tau = 2.0$, solo el 17 %. Con 0.2 y 0.5 el texto es idéntico: el modelo elige siempre lo más probable y recita dos frases seguidas del corpus. Con 0.8 y 1.2 toma otro camino y empalma: «un modelo injusto.» es el final de «un sesgo en los datos puede producir un modelo injusto». Con 2.0 el texto se deshace («dembed», «unconteniperapo»). Prueba también con «la inteligencia»: ahí el modelo está tan seguro (probabilidad 0.853 para la «a» incluso con $\tau = 2.0$) que hasta $\tau = 1.2$ genera exactamente el mismo texto.

Ejercicio 13.3 Calcular la Perplexity del modelo

La perplexity (PPL) mide qué tan "sorprendido" está el modelo ante el texto de evaluación: PPL = exp(pérdida_media_por_token). Una PPL igual al tamaño del vocabulario (V) significa comportamiento aleatorio; cuanto más baja, mejor. Calcula la perplexity real del Nano-LLM entrenado sobre todo su texto de entrenamiento y explica qué significa para un modelo con 46 caracteres de vocabulario.

> 💡 **Pista**
>
> Adivinar al azar entre los V caracteres da PPL = V; cuanto más baja, menos duda el modelo en promedio. Una PPL de 1.33, muy cercana a 1, indica que el modelo apenas duda entre 1 y 2 candidatos por carácter

— coherente con el diagnóstico de sobreajuste de la sección 13.6: no es que el modelo "entienda" el español, es que ha memorizado buena parte de las 93 líneas de entrenamiento.

✅ Solución:

📄 ejercicio_13_3.py

```
# ============================================================
# EJERCICIO 13.3 — La perplejidad del Nano-LLM
# PPL = exp(pérdida media por carácter). Se calcula con el
# modelo entrenado sobre TODO el texto de entrenamiento.
# ============================================================
import numpy as np



fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec(fuente.split('# -- Learning rate')[0])
params.update(np.load('nano_llm_pesos.npz'))

# Pérdida media en ventanas consecutivas de CTX caracteres
perdidas = []
for i in range(0, len(DATOS) - CTX - 1, CTX):
    x, y = DATOS[i:i + CTX], DATOS[i + 1:i + CTX + 1]
    p = softmax(forward(np.array([x]))[0][0])        # lote de 1 ventana
    perdidas.extend(-np.log(p[np.arange(CTX), y] + 1e-10))
L = np.mean(perdidas)
print(f'\nPérdida media en el texto de entrenamiento: {L:.4f}')
print(f'Perplejidad: e^{L:.4f} = {np.exp(L):.2f}\n')

print(f"{'Pérdida':>8}  {'PPL':>6}  Interpretación")
print('-' * 58)
for perd in [np.log(V), 3.0, 2.0, 1.5, 1.0, 0.5, L]:
    ppl = np.exp(perd)
    if np.isclose(perd, np.log(V)): interp = 'Referencia: la uniforme (al azar)'
    elif ppl > V:      interp = f'Peor que el azar ({ppl - V:+.1f} sobre V)'
    elif ppl > 15:     interp = 'Algo de estructura local'
    elif ppl > 4:      interp = 'Patrones de carácter bien aprendidos'
    elif ppl > 1.8:    interp = 'Buen modelo: captura el estilo'
    else:              interp = 'Excelente... o sobreajuste (ver Capítulo 15)'
    print(f'{perd:8.4f}  {ppl:6.2f}  {interp}')
```

> Salida del programa:

```
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas

Pérdida media en el texto de entrenamiento: 0.2835
Perplejidad: e^0.2835 = 1.33

 Pérdida     PPL  Interpretación
----------------------------------------------------------
  3.8286   46.00  Referencia: la uniforme (al azar)
  3.0000   20.09  Algo de estructura local
  2.0000    7.39  Patrones de carácter bien aprendidos
  1.5000    4.48  Patrones de carácter bien aprendidos
  1.0000    2.72  Buen modelo: captura el estilo
  0.5000    1.65  Excelente... o sobreajuste (ver Capítulo 15)
  0.2835    1.33  Excelente... o sobreajuste (ver Capítulo 15)
```

La pérdida ln(V) es la referencia de «no haber aprendido nada»: si el modelo asigna probabilidad uniforme 1/V a todos los tokens, la entropía cruzada es exactamente ln(V) y la PPL vale V. Nuestro valor real (PPL = 1.33) cae en la última fila de la tabla: «Excelente… o sobreajuste». Un valor tan bajo en relación con el tamaño del corpus es indicio de memorización, no de un modelo mejor. En el Capítulo 15 (Programa RZ-1) veremos que en frases que el modelo no ha visto la PPL sube a unos 65: peor que adivinar al azar (46).

Ejercicio 13.4 CTX y D: cómo escalan los parámetros

El Programa 8 usa CTX=24 y D=48. Duplica D a 96 (deja CTX y el resto de hiperparámetros igual) y reentrena. Compara el número total de parámetros y la pérdida final con la versión original. ¿Por qué duplicar D hace crecer los parámetros mucho más que duplicar CTX?

> 💡 Pista: D interviene en casi todos los componentes (embedding de token V×D, embedding posicional CTX×D, y las matrices internas de los bloques Transformer, que escalan aproximadamente con D^2 y D×D_FF), mientras que duplicar solo CTX afectaría únicamente al embedding posicional (CTX×D). Por eso D domina el crecimiento de parámetros.

✅ Solución:

📄 ejercicio_13_4.py — el Programa 8 completo con D = 96 y PASOS = 4000 (los cambios llevan el comentario «antes:»)

```
# ============================================================
# EJERCICIO 13.4 — El Programa 8 con D = 96 (el doble) y PASOS = 4000
# ============================================================
# PROGRAMA 8 — Nano-LLM: un GPT-2 en miniatura
# Modelo de lenguaje de nivel carácter con la arquitectura
# de GPT-2 (2019): embeddings de token y de posición
# aprendidos, 2 bloques Pre-LN (LayerNorm con γ y β antes de
# cada subcapa), atención causal de 4 cabezas, FFN con GELU,
# LayerNorm final y pesos compartidos entre la entrada y la
# salida. Solo cambian los tamaños (GPT-2 small: 12 bloques,
# 12 cabezas, D = 768, contexto 1024, 50 257 tokens BPE).
# NumPy puro: backpropagation analítico completo, mini-lotes,
# Adam con warmup + coseno y recorte del gradiente. Guarda
# los pesos en nano_llm_pesos.npz para los capítulos siguientes.
# ============================================================

import numpy as np

# -- Texto de entrenamiento (93 líneas, vocabulario de 46 caracteres) --
TEXTO = """
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas capas ocultas.
la atención permite ver todo el contexto a la vez.
los transformers son la base de los llm modernos.
entrenar un modelo requiere muchos datos y cálculo.
el futuro pertenece a quienes comprenden la ia.
un modelo pequeño puede aprender patrones simples.
la tokenización convierte texto en números enteros.
cada carácter del vocabulario tiene un índice único.
el embedding transforma índices en vectores densos.
la codificación posicional indica el orden del texto.
la función softmax convierte logits en probabilidades.
el optimizador adam ajusta los pesos, paso a paso.
una red profunda combina atención y capas ocultas.
generar texto significa predecir, uno a uno, cada token.
la temperatura controla cuánta variedad hay al generar.
un buen modelo equilibra memoria y generalización.
¿comprender la ia? requiere práctica, paciencia y datos.
también existe el software: gpt-2 fue un hito clave.
un modelo (bien entrenado) generaliza; uno malo, no.
¡el aprendizaje nunca termina! sigue explorando.
un vector es una lista ordenada de números reales.
una matriz agrupa vectores en filas y columnas.
el producto punto mide cuánto se parecen dos vectores.
la retropropagación calcula gradientes hacia atrás.
el descenso de gradiente ajusta los pesos poco a poco.
una época recorre todo el conjunto de entrenamiento.
el sobreajuste ocurre cuando el modelo memoriza demasiado.
la regularización ayuda a que el modelo generalice mejor.
un lote agrupa varios ejemplos para entrenar juntos.
la normalización estabiliza las activaciones internas.
una neurona combina entradas, pesos y un sesgo.
la función de pérdida mide el error del modelo.
el aprendizaje supervisado usa ejemplos etiquetados.
una capa oculta transforma la información paso a paso.
el descenso estocástico usa lotes pequeños y aleatorios.
la validación comprueba si el modelo generaliza bien.
un hiperparámetro se ajusta antes de entrenar el modelo.
la arquitectura define cómo se conectan las capas.
el vocabulario agrupa todos los símbolos posibles.
una secuencia es un orden concreto de tokens.
el contexto limita cuántos tokens ve el modelo a la vez.
la generalización permite responder a datos nuevos.
comprender estos conceptos abre la puerta a la ia moderna.
```

```
un dataset agrupa todos los ejemplos disponibles.
el conjunto de test evalúa el modelo al final, una sola vez.
la precisión mide cuántas predicciones son correctas.
el error mide cuánto se aleja la predicción del valor real.
una etiqueta indica la respuesta correcta de un ejemplo.
la inferencia usa el modelo ya entrenado, sin ajustar pesos.
el preentrenamiento aprende patrones generales del lenguaje.
el ajuste fino especializa un modelo en una tarea concreta.
un token puede ser una letra, una parte de palabra o una palabra.
la ventana de contexto limita cuánto texto ve el modelo.
un residual conecta la entrada directamente con la salida.
la normalización evita que las activaciones crezcan sin control.
el ruido en los datos dificulta encontrar el patrón real.
una red ancha tiene muchas neuronas en cada capa.
una red profunda tiene muchas capas apiladas.
el paralelismo permite entrenar varias partes a la vez.
la memoria limita cuántos datos caben en una gráfica.
un acelerador gráfico multiplica matrices muy rápido.
la escala mide cuántos parámetros tiene un modelo.
un modelo grande necesita muchos más datos para aprender bien.
la eficiencia importa tanto como la precisión final.
el costo de entrenar crece con el tamaño del modelo.
la interpretabilidad explica por qué el modelo decide algo.
un sesgo en los datos puede producir un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la privacidad protege los datos personales de los usuarios.
un modelo robusto resiste pequeños cambios en la entrada.
la incertidumbre mide cuánta confianza tiene el modelo.
un ejemplo difícil confunde incluso a un buen modelo.
la curva de aprendizaje muestra el progreso del entrenamiento.
un mínimo local complica encontrar la mejor solución.
el paisaje de pérdida describe cómo cambia el error.
una meseta ralentiza el progreso del entrenamiento.
el ruido ayuda a veces a escapar de mínimos malos.
la exploración prueba caminos nuevos durante el entrenamiento.
la explotación aprovecha lo que ya funciona bien.
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre categorías distintas.
la clasificación asigna una categoría a cada ejemplo.
la regresión predice un número continuo, no una categoría.
el aprendizaje por refuerzo premia las buenas decisiones.
un agente aprende actuando dentro de un entorno.
la recompensa guía el aprendizaje del agente.
la política define cómo actúa el agente en cada estado.
un entorno simulado permite entrenar sin riesgo real.
la simulación acelera enormemente el aprendizaje del agente.
comprender bien lo básico facilita entender lo avanzado.
"""

# -- Tokenizador ----------------------------------------

VOCAB = sorted(set(TEXTO))
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))
DATOS = np.array([C2I[c] for c in TEXTO], dtype=int)
# -- Hiperparámetros (entre paréntesis, los de GPT-2 small) --
CTX    = 24     # longitud del contexto (1024)
D      = 96     # antes: 48
H      = 4      # cabezas de atención (12)
DH     = D // H # dimensión de cada cabeza: 12 (64)
D_FF   = 192    # dimensión interna FFN, 4xD (3072)
N_BLQ  = 2      # bloques transformer (12)
LR     = 3e-3   # tasa de aprendizaje máxima
PASOS  = 4000   # antes: 12000 (para comparar más rápido)
BATCH  = 8      # secuencias por lote (promedia el gradiente, suaviza la curva)
CLIP   = 1.0    # recorte de la norma del gradiente (Capítulo 5, 5.8)

np.random.seed(42)

# -- Parámetros del modelo (inicialización de GPT-2) --------
def randn(shape, std=0.02):
    return np.random.randn(*shape) * std
# Las proyecciones que se suman a la vía residual (Wo y W2) se
# inician con una desviación menor, 0.02/sqrt(2·N_BLQ), como en GPT-2.
STD_RES = 0.02 / np.sqrt(2 * N_BLQ)

params = {
    'wte': randn((V, D)),     # embedding de tokens (y matriz de salida)
    'wpe': randn((CTX, D)),   # embedding de posiciones
    'lnf_g': np.ones(D), 'lnf_b': np.zeros(D),   # LayerNorm final
}
for k in range(N_BLQ):
    params[f'ln1_g{k}'] = np.ones(D);  params[f'ln1_b{k}'] = np.zeros(D)
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
```

```
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
    params[f'ln2_g{k}'] = np.ones(D);  params[f'ln2_b{k}'] = np.zeros(D)
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)

total = sum(p.size for p in params.values())
print(f'Vocabulario: {V} caracteres')
print(f'Total de parámetros: {total:,}')
print(f'Texto de entrenamiento: {len(DATOS)} tokens, '
      f'{TEXTO.strip().count(chr(10)) + 1} líneas')

# Máscara causal: se calcula una vez con el tamaño máximo (CTX, CTX)
# y se recorta a [:T, :T] cuando la secuencia es más corta.
MASCARA_CAUSAL = np.triu(np.full((CTX, CTX), -1e9), k=1)
# -- Utilidades ---------------------------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)



def layer_norm_fwd(x, g, b, eps=1e-5):
    """LayerNorm de GPT-2: normaliza cada fila y aplica escala g (γ) y desplazamiento b
(β)."""
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)
def layer_norm_bwd(dy, g, cache):
    """Derivada exacta de LayerNorm: devuelve dx, dg y db."""
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))      # todas las filas (lote y posiciones)
    dg = (dy * xhat).sum(ejes)
    db = dy.sum(ejes)
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, dg, db
C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    """GELU con la aproximación tanh que usa GPT-2."""
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))
def gelu_bwd(x):
    """Derivada de gelu(x)."""
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)
def separar(x):   # (B, T, D) -> (B, H, T, DH): una matriz por cabeza
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)
def unir(x):      # (B, H, T, DH) -> (B, T, D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)
def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])

# -- Forward ------------------------------------------------
def forward(tokens):
    """tokens (B, T) -> logits (B, T, V), y lo que el backward necesita."""
    B, T = tokens.shape
    X = params['wte'][tokens] + params['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        Xin = X
        # Subcapa 1: LayerNorm -> atención causal multi-cabeza -> suma residual
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        # Subcapa 2: LayerNorm -> FFN con GELU -> suma residual
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    logits = xf @ params['wte'].T          # pesos compartidos: la salida usa wte
    return logits, (xf, lnf, cache)

# -- Forward + Backward (un mini-lote entero) ----------------
def forward_backward(tokens, targets):
    """
    Pasa el mini-lote por el modelo y calcula, con backpropagation
```

```
    exacto (regla de la cadena), el gradiente de la pérdida media
    respecto a TODOS los parámetros.
    """




    B, T = tokens.shape
    logits, (xf, lnf, cache) = forward(tokens)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -np.log(probs[bi, ti, targets] + 1e-10).mean()
    grads = {k: np.zeros_like(v) for k, v in params.items()}

    dlogits = probs.copy()

    dlogits[bi, ti, targets] -= 1.0          # softmax + entropía cruzada
    dlogits /= B * T
    grads['wte'] += tmm(dlogits, xf)         # wte como matriz de salida
    dxf = dlogits @ params['wte']
    dX, grads['lnf_g'], grads['lnf_b'] = layer_norm_bwd(dxf, params['lnf_g'], lnf)

    for k in reversed(range(N_BLQ)):









        c = cache[k]
        # Subcapa 2 hacia atrás. Residual: dX llega entero a X1 y a la FFN
        grads[f'W2{k}'] = tmm(c['h'], dX)
        grads[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        grads[f'W1{k}'] = tmm(c['m'], dh_pre)
        grads[f'b1{k}'] = dh_pre.sum((0, 1))
        dm = dh_pre @ params[f'W1{k}'].T
        dx1, grads[f'ln2_g{k}'], grads[f'ln2_b{k}'] = layer_norm_bwd(dm, params[f'ln2_g{k}'],
c['ln2'])
        dX1 = dX + dx1
        # Subcapa 1 hacia atrás. Residual: dX1 llega entero a Xin y a la atención
        grads[f'Wo{k}'] = tmm(c['att'], dX1)
        grads[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)   # derivada de softmax
        dQ = unir(dscores @ c['K'])
        dK = unir(dscores.transpose(0, 1, 3, 2) @ c['Q'])
        dV = unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            grads[f'W{n}{k}'] = tmm(c['a'], dn)
            grads[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, grads[f'ln1_g{k}'], grads[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    grads['wpe'][:T] = dX.sum(0)
    np.add.at(grads['wte'], tokens, dX)      # wte también como embedding de entrada

    return grads, loss
# -- Learning rate: warmup lineal + decaimiento coseno --------
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    """
    Sube linealmente de 0 a lr_max durante t_warmup pasos (evita
    pasos erráticos al principio, cuando los gradientes son más
    inestables), luego decae en coseno hasta lr_min (afina los
    pesos al final). Es el esquema estándar al entrenar LLMs.
    """
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup     # +1: el paso 0 ya aprende
    t_adj = t - t_warmup
    t_decay = t_total - t_warmup
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * t_adj / t_decay))


# -- Entrenamiento con Adam (mini-lotes de BATCH secuencias) --
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
```

```
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
beta1, beta2, eps_adam, t_adam = 0.9, 0.999, 1e-8, 0
LR_MAX = LR

print('\nEntrenando Nano-LLM (backpropagation analítico, mini-lotes)...')
print(f'CTX={CTX} | D={D} | D_FF={D_FF} | Bloques={N_BLQ} | Batch={BATCH}\n')

for paso in range(PASOS):
    lr_actual = lr_warmup_coseno(paso, LR_MAX, PASOS)
    inicios = np.random.randint(0, len(DATOS) - CTX - 1, size=BATCH)
    xb = np.stack([DATOS[i:i + CTX] for i in inicios])          # (BATCH, CTX)
    yb = np.stack([DATOS[i + 1:i + CTX + 1] for i in inicios])  # desplazado 1
    grads_acc, loss = forward_backward(xb, yb)
    # Recorte del gradiente: si la norma global supera CLIP, se reescala
    norma = np.sqrt(sum((g ** 2).sum() for g in grads_acc.values()))
    if norma > CLIP:
        for k in grads_acc:
            grads_acc[k] *= CLIP / norma

    t_adam += 1
    for k in params:
        g = grads_acc[k]
        m_adam[k] = beta1 * m_adam[k] + (1 - beta1) * g
        v_adam[k] = beta2 * v_adam[k] + (1 - beta2) * g ** 2
        m_h = m_adam[k] / (1 - beta1 ** t_adam)
        v_h = v_adam[k] / (1 - beta2 ** t_adam)
        params[k] -= lr_actual * m_h / (np.sqrt(v_h) + eps_adam)

    if paso % 1000 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida = {loss:.4f}   lr = {lr_actual:.5f}   '
              f'(bits/char = {loss/np.log(2):.2f})')
# -- Estrategias de muestreo (se estudian a fondo en el Capítulo 14) --
# generar() usa temperatura + top_p_filtrar; greedy y top_k_filtrar
# quedan definidas para experimentar con ellas.
def greedy(probs):
    """Siempre el token más probable. Puede quedar 'atrapado' en bucles."""
    return np.argmax(probs)

def top_k_filtrar(probs, k):
    """Conserva solo los k tokens más probables."""
    indices_ord = np.argsort(probs)[::-1]
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:k]] = probs[indices_ord[:k]]
    return nueva / nueva.sum()

def top_p_filtrar(probs, p):
    """Conserva el mínimo conjunto de tokens que suman probabilidad p (nucleus sampling)."""
    indices_ord = np.argsort(probs)[::-1]
    probs_ord   = probs[indices_ord]
    suma_acum   = np.cumsum(probs_ord)
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
    return nueva / nueva.sum()


# -- Generación de texto ------------------------------------
def generar(semilla, n_chars=120, temperatura=0.8, top_p=0.9):
    """
    Genera texto carácter a carácter.
    temperatura > 1: más aleatorio | temperatura < 1: más determinista
    top_p: nucleus sampling — conserva solo el conjunto mínimo de
    caracteres cuya probabilidad acumulada alcanza top_p, evitando
    elegir caracteres de probabilidad residual que producen "sopa
    de letras". Con top_p=1.0 equivale al muestreo puro.
    """
    tokens = [C2I.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla

    for _ in range(n_chars):
        ctx = tokens[-CTX:]
        logits = forward(np.array([ctx]))[0][0, -1]
        probs = softmax(logits / temperatura)

        if top_p < 1.0:
            probs = top_p_filtrar(probs, top_p)

        siguiente = int(np.random.choice(V, p=probs))
        tokens.append(siguiente)
        resultado += I2C[siguiente]

    return resultado


print('\n=== TEXTO GENERADO POR EL NANO-LLM ===')
print(generar('la inteligencia', n_chars=120))
```

```
print()
print(generar('los modelos de', n_chars=120))
# -- Guardar los pesos entrenados ----------------------------
# Los Capítulos 11, 14, 15 y 18 reutilizan este modelo ya entrenado:
#   params = dict(np.load('nano_llm_pesos.npz'))
np.savez('pesos_ejercicio_13_4.npz', **params)
print("\nPesos guardados en pesos_ejercicio_13_4.npz")
```

> Salida del programa (D = 48, original, con PASOS = 4000):

```
Total de parámetros: 60,000
Paso     0: pérdida = 3.8547   lr = 0.00002   (bits/char = 5.56)
Paso  1000: pérdida = 1.3384   lr = 0.00266   (bits/char = 1.93)
Paso  2000: pérdida = 0.8760   lr = 0.00164   (bits/char = 1.26)
Paso  3000: pérdida = 0.4716   lr = 0.00056   (bits/char = 0.68)
Paso  3999: pérdida = 0.4683   lr = 0.00010   (bits/char = 0.68)
```

> Salida del programa (D = 96, el doble):

```
Total de parámetros: 156,480
Paso     0: pérdida = 3.9088   lr = 0.00002   (bits/char = 5.64)
Paso  1000: pérdida = 1.1512   lr = 0.00266   (bits/char = 1.66)
Paso  2000: pérdida = 0.6265   lr = 0.00164   (bits/char = 0.90)
Paso  3000: pérdida = 0.4638   lr = 0.00056   (bits/char = 0.67)
Paso  3999: pérdida = 0.3027   lr = 0.00010   (bits/char = 0.44)
```

Duplicar D casi triplica los parámetros (60 000 → 156 480, ×2.6) porque D aparece elevado al cuadrado en las matrices de atención y multiplicado por D_FF en la FFN — mientras que duplicar solo CTX habría añadido apenas CTX×D parámetros más (una cantidad pequeña frente al total). Con más capacidad, la pérdida también baja más rápido para el mismo número de pasos (0.30 frente a 0.47 tras 4000 pasos, aunque cada valor es el de un solo mini-lote) — coherente con la discusión de la sección 13.6 sobre sobreajuste: más parámetros para el mismo corpus memorizan todavía más rápido.

Ejercicio 13.5 ¿Hace falta de verdad el warm-up + coseno?

La sección 13.3.3 explica el warm-up y el decaimiento coseno de la tasa de aprendizaje. Compruébalo: modifica lr_warmup_coseno para que devuelva siempre LR_MAX constante (sin warm-up ni decaimiento) y reentrena con el mismo corpus y número de pasos (4000). Compara la curva de pérdida, sobre todo al principio y al final del entrenamiento.

> 💡 Pista: fíjate especialmente en los últimos pasos. Un LR alto y constante impide que el modelo termine de "afinar" cuando ya está cerca de una buena solución — vigila si la pérdida desciende de forma suave o si oscila sin asentarse.

✅ Solución:

📄 ejercicio_13_5.py — el Programa 8 completo con LR constante y PASOS = 4000 (cambios: «antes:» y «LR constante»)

```
# ============================================================
# EJERCICIO 13.5 — El Programa 8 con LR constante y PASOS = 4000
# ============================================================
# PROGRAMA 8 — Nano-LLM: un GPT-2 en miniatura
# Modelo de lenguaje de nivel carácter con la arquitectura
# de GPT-2 (2019): embeddings de token y de posición
# aprendidos, 2 bloques Pre-LN (LayerNorm con γ y β antes de
# cada subcapa), atención causal de 4 cabezas, FFN con GELU,
# LayerNorm final y pesos compartidos entre la entrada y la
# salida. Solo cambian los tamaños (GPT-2 small: 12 bloques,
# 12 cabezas, D = 768, contexto 1024, 50 257 tokens BPE).
# NumPy puro: backpropagation analítico completo, mini-lotes,
# Adam con warmup + coseno y recorte del gradiente. Guarda
# los pesos en nano_llm_pesos.npz para los capítulos siguientes.
# ============================================================

import numpy as np

# -- Texto de entrenamiento (93 líneas, vocabulario de 46 caracteres) --
TEXTO = """
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas capas ocultas.
la atención permite ver todo el contexto a la vez.
los transformers son la base de los llm modernos.
entrenar un modelo requiere muchos datos y cálculo.
el futuro pertenece a quienes comprenden la ia.
un modelo pequeño puede aprender patrones simples.
```

```
la tokenización convierte texto en números enteros.
cada carácter del vocabulario tiene un índice único.
el embedding transforma índices en vectores densos.
la codificación posicional indica el orden del texto.
la función softmax convierte logits en probabilidades.
el optimizador adam ajusta los pesos, paso a paso.
una red profunda combina atención y capas ocultas.
generar texto significa predecir, uno a uno, cada token.
la temperatura controla cuánta variedad hay al generar.
un buen modelo equilibra memoria y generalización.
¿comprender la ia? requiere práctica, paciencia y datos.
también existe el software: gpt-2 fue un hito clave.
un modelo (bien entrenado) generaliza; uno malo, no.
¡el aprendizaje nunca termina! sigue explorando.
un vector es una lista ordenada de números reales.
una matriz agrupa vectores en filas y columnas.
el producto punto mide cuánto se parecen dos vectores.
la retropropagación calcula gradientes hacia atrás.
el descenso de gradiente ajusta los pesos poco a poco.
una época recorre todo el conjunto de entrenamiento.
el sobreajuste ocurre cuando el modelo memoriza demasiado.
la regularización ayuda a que el modelo generalice mejor.
un lote agrupa varios ejemplos para entrenar juntos.
la normalización estabiliza las activaciones internas.
una neurona combina entradas, pesos y un sesgo.
la función de pérdida mide el error del modelo.
el aprendizaje supervisado usa ejemplos etiquetados.
una capa oculta transforma la información paso a paso.
el descenso estocástico usa lotes pequeños y aleatorios.
la validación comprueba si el modelo generaliza bien.
un hiperparámetro se ajusta antes de entrenar el modelo.
la arquitectura define cómo se conectan las capas.
el vocabulario agrupa todos los símbolos posibles.
una secuencia es un orden concreto de tokens.
el contexto limita cuántos tokens ve el modelo a la vez.
la generalización permite responder a datos nuevos.
comprender estos conceptos abre la puerta a la ia moderna.
un dataset agrupa todos los ejemplos disponibles.
el conjunto de test evalúa el modelo al final, una sola vez.
la precisión mide cuántas predicciones son correctas.
el error mide cuánto se aleja la predicción del valor real.
una etiqueta indica la respuesta correcta de un ejemplo.
la inferencia usa el modelo ya entrenado, sin ajustar pesos.
el preentrenamiento aprende patrones generales del lenguaje.
el ajuste fino especializa un modelo en una tarea concreta.
un token puede ser una letra, una parte de palabra o una palabra.
la ventana de contexto limita cuánto texto ve el modelo.
un residual conecta la entrada directamente con la salida.
la normalización evita que las activaciones crezcan sin control.
el ruido en los datos dificulta encontrar el patrón real.
una red ancha tiene muchas neuronas en cada capa.
una red profunda tiene muchas capas apiladas.
el paralelismo permite entrenar varias partes a la vez.
la memoria limita cuántos datos caben en una gráfica.
un acelerador gráfico multiplica matrices muy rápido.
la escala mide cuántos parámetros tiene un modelo.
un modelo grande necesita muchos más datos para aprender bien.
la eficiencia importa tanto como la precisión final.
el costo de entrenar crece con el tamaño del modelo.
la interpretabilidad explica por qué el modelo decide algo.
un sesgo en los datos puede producir un modelo injusto.
la ética en la ia estudia el impacto social de los modelos.
la privacidad protege los datos personales de los usuarios.
un modelo robusto resiste pequeños cambios en la entrada.
la incertidumbre mide cuánta confianza tiene el modelo.
un ejemplo difícil confunde incluso a un buen modelo.
la curva de aprendizaje muestra el progreso del entrenamiento.
un mínimo local complica encontrar la mejor solución.
el paisaje de pérdida describe cómo cambia el error.
una meseta ralentiza el progreso del entrenamiento.
el ruido ayuda a veces a escapar de mínimos malos.
la exploración prueba caminos nuevos durante el entrenamiento.
la explotación aprovecha lo que ya funciona bien.
un modelo generativo produce contenido nuevo y original.
un modelo discriminativo distingue entre categorías distintas.
la clasificación asigna una categoría a cada ejemplo.
la regresión predice un número continuo, no una categoría.
el aprendizaje por refuerzo premia las buenas decisiones.
un agente aprende actuando dentro de un entorno.
la recompensa guía el aprendizaje del agente.
la política define cómo actúa el agente en cada estado.
un entorno simulado permite entrenar sin riesgo real.
la simulación acelera enormemente el aprendizaje del agente.
comprender bien lo básico facilita entender lo avanzado.
"""
```

```
# -- Tokenizador ----------------------------------------

VOCAB = sorted(set(TEXTO))
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))
DATOS = np.array([C2I[c] for c in TEXTO], dtype=int)
# -- Hiperparámetros (entre paréntesis, los de GPT-2 small) --
CTX    = 24     # longitud del contexto (1024)
D      = 48     # dimensión del modelo (768)
H      = 4      # cabezas de atención (12)
DH     = D // H # dimensión de cada cabeza: 12 (64)
D_FF   = 192    # dimensión interna FFN, 4xD (3072)
N_BLQ  = 2      # bloques transformer (12)
LR     = 3e-3   # tasa de aprendizaje máxima
PASOS  = 4000   # antes: 12000 (para comparar más rápido)
BATCH  = 8      # secuencias por lote (promedia el gradiente, suaviza la curva)
CLIP   = 1.0    # recorte de la norma del gradiente (Capítulo 5, 5.8)

np.random.seed(42)

# -- Parámetros del modelo (inicialización de GPT-2) --------
def randn(shape, std=0.02):
    return np.random.randn(*shape) * std
# Las proyecciones que se suman a la vía residual (Wo y W2) se
# inician con una desviación menor, 0.02/sqrt(2·N_BLQ), como en GPT-2.
STD_RES = 0.02 / np.sqrt(2 * N_BLQ)

params = {
    'wte': randn((V, D)),     # embedding de tokens (y matriz de salida)
    'wpe': randn((CTX, D)),   # embedding de posiciones
    'lnf_g': np.ones(D), 'lnf_b': np.zeros(D),   # LayerNorm final
}
for k in range(N_BLQ):
    params[f'ln1_g{k}'] = np.ones(D);  params[f'ln1_b{k}'] = np.zeros(D)
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
    params[f'ln2_g{k}'] = np.ones(D);  params[f'ln2_b{k}'] = np.zeros(D)
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)

total = sum(p.size for p in params.values())
print(f'Vocabulario: {V} caracteres')
print(f'Total de parámetros: {total:,}')
print(f'Texto de entrenamiento: {len(DATOS)} tokens, '
      f'{TEXTO.strip().count(chr(10)) + 1} líneas')

# Máscara causal: se calcula una vez con el tamaño máximo (CTX, CTX)
# y se recorta a [:T, :T] cuando la secuencia es más corta.
MASCARA_CAUSAL = np.triu(np.full((CTX, CTX), -1e9), k=1)
# -- Utilidades -----------------------------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)



def layer_norm_fwd(x, g, b, eps=1e-5):
    """LayerNorm de GPT-2: normaliza cada fila y aplica escala g (γ) y desplazamiento b
(β)."""
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)
def layer_norm_bwd(dy, g, cache):
    """Derivada exacta de LayerNorm: devuelve dx, dg y db."""
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))      # todas las filas (lote y posiciones)
    dg = (dy * xhat).sum(ejes)
    db = dy.sum(ejes)
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, dg, db
C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    """GELU con la aproximación tanh que usa GPT-2."""
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))
def gelu_bwd(x):
    """Derivada de gelu(x)."""
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)
def separar(x):   # (B, T, D) -> (B, H, T, DH): una matriz por cabeza
    B, T, _ = x.shape
```

```
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)
def unir(x):      # (B, H, T, DH) -> (B, T, D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)
def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])

# -- Forward ---------------------------------------------------
def forward(tokens):
    """tokens (B, T) -> logits (B, T, V), y lo que el backward necesita."""
    B, T = tokens.shape
    X = params['wte'][tokens] + params['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        Xin = X
        # Subcapa 1: LayerNorm -> atención causal multi-cabeza -> suma residual
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        # Subcapa 2: LayerNorm -> FFN con GELU -> suma residual
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    logits = xf @ params['wte'].T          # pesos compartidos: la salida usa wte
    return logits, (xf, lnf, cache)

# -- Forward + Backward (un mini-lote entero) ----------------------
def forward_backward(tokens, targets):
    """
    Pasa el mini-lote por el modelo y calcula, con backpropagation
    exacto (regla de la cadena), el gradiente de la pérdida media
    respecto a TODOS los parámetros.
    """




    B, T = tokens.shape
    logits, (xf, lnf, cache) = forward(tokens)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -np.log(probs[bi, ti, targets] + 1e-10).mean()
    grads = {k: np.zeros_like(v) for k, v in params.items()}

    dlogits = probs.copy()

    dlogits[bi, ti, targets] -= 1.0          # softmax + entropía cruzada
    dlogits /= B * T
    grads['wte'] += tmm(dlogits, xf)         # wte como matriz de salida
    dxf = dlogits @ params['wte']
    dX, grads['lnf_g'], grads['lnf_b'] = layer_norm_bwd(dxf, params['lnf_g'], lnf)

    for k in reversed(range(N_BLQ)):









        c = cache[k]
        # Subcapa 2 hacia atrás. Residual: dX llega entero a X1 y a la FFN
        grads[f'W2{k}'] = tmm(c['h'], dX)
        grads[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        grads[f'W1{k}'] = tmm(c['m'], dh_pre)
        grads[f'b1{k}'] = dh_pre.sum((0, 1))
        dm = dh_pre @ params[f'W1{k}'].T
        dx1, grads[f'ln2_g{k}'], grads[f'ln2_b{k}'] = layer_norm_bwd(dm, params[f'ln2_g{k}'],
c['ln2'])
        dX1 = dX + dx1
        # Subcapa 1 hacia atrás. Residual: dX1 llega entero a Xin y a la atención
        grads[f'Wo{k}'] = tmm(c['att'], dX1)
        grads[f'bo{k}'] = dX1.sum((0, 1))
```

```
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)   # derivada de softmax
        dQ = unir(dscores @ c['K'])
        dK = unir(dscores.transpose(0, 1, 3, 2) @ c['Q'])
        dV = unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            grads[f'W{n}{k}'] = tmm(c['a'], dn)
            grads[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, grads[f'ln1_g{k}'], grads[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    grads['wpe'][:T] = dX.sum(0)
    np.add.at(grads['wte'], tokens, dX)      # wte también como embedding de entrada

    return grads, loss
# -- Learning rate: warmup lineal + decaimiento coseno --------
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    """
    Sube linealmente de 0 a lr_max durante t_warmup pasos (evita
    pasos erráticos al principio, cuando los gradientes son más
    inestables), luego decae en coseno hasta lr_min (afina los
    pesos al final). Es el esquema estándar al entrenar LLMs.
    """
    return lr_max                               # LR constante: sin warm-up ni coseno


# -- Entrenamiento con Adam (mini-lotes de BATCH secuencias) --
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
beta1, beta2, eps_adam, t_adam = 0.9, 0.999, 1e-8, 0
LR_MAX = LR

print('\nEntrenando Nano-LLM (backpropagation analítico, mini-lotes)...')
print(f'CTX={CTX} | D={D} | D_FF={D_FF} | Bloques={N_BLQ} | Batch={BATCH}\n')

for paso in range(PASOS):
    lr_actual = lr_warmup_coseno(paso, LR_MAX, PASOS)
    inicios = np.random.randint(0, len(DATOS) - CTX - 1, size=BATCH)
    xb = np.stack([DATOS[i:i + CTX] for i in inicios])          # (BATCH, CTX)
    yb = np.stack([DATOS[i + 1:i + CTX + 1] for i in inicios])  # desplazado 1
    grads_acc, loss = forward_backward(xb, yb)
    # Recorte del gradiente: si la norma global supera CLIP, se reescala
    norma = np.sqrt(sum((g ** 2).sum() for g in grads_acc.values()))
    if norma > CLIP:
        for k in grads_acc:
            grads_acc[k] *= CLIP / norma

    t_adam += 1
    for k in params:
        g = grads_acc[k]
        m_adam[k] = beta1 * m_adam[k] + (1 - beta1) * g
        v_adam[k] = beta2 * v_adam[k] + (1 - beta2) * g ** 2
        m_h = m_adam[k] / (1 - beta1 ** t_adam)
        v_h = v_adam[k] / (1 - beta2 ** t_adam)
        params[k] -= lr_actual * m_h / (np.sqrt(v_h) + eps_adam)

    if paso % 1000 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida = {loss:.4f}   lr = {lr_actual:.5f}   '
              f'(bits/char = {loss/np.log(2):.2f})')
# -- Estrategias de muestreo (se estudian a fondo en el Capítulo 14) --
# generar() usa temperatura + top_p_filtrar; greedy y top_k_filtrar
# quedan definidas para experimentar con ellas.
def greedy(probs):
    """Siempre el token más probable. Puede quedar 'atrapado' en bucles."""
    return np.argmax(probs)

def top_k_filtrar(probs, k):
    """Conserva solo los k tokens más probables."""
    indices_ord = np.argsort(probs)[::-1]
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:k]] = probs[indices_ord[:k]]
    return nueva / nueva.sum()

def top_p_filtrar(probs, p):
    """Conserva el mínimo conjunto de tokens que suman probabilidad p (nucleus sampling)."""
    indices_ord = np.argsort(probs)[::-1]
    probs_ord   = probs[indices_ord]
    suma_acum   = np.cumsum(probs_ord)
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
```

```
    return nueva / nueva.sum()


# -- Generación de texto ----------------------------------
def generar(semilla, n_chars=120, temperatura=0.8, top_p=0.9):
    """
    Genera texto carácter a carácter.
    temperatura > 1: más aleatorio | temperatura < 1: más determinista
    top_p: nucleus sampling — conserva solo el conjunto mínimo de
    caracteres cuya probabilidad acumulada alcanza top_p, evitando
    elegir caracteres de probabilidad residual que producen "sopa
    de letras". Con top_p=1.0 equivale al muestreo puro.
    """
    tokens = [C2I.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla

    for _ in range(n_chars):
        ctx = tokens[-CTX:]
        logits = forward(np.array([ctx]))[0][0, -1]
        probs = softmax(logits / temperatura)

        if top_p < 1.0:
            probs = top_p_filtrar(probs, top_p)

        siguiente = int(np.random.choice(V, p=probs))
        tokens.append(siguiente)
        resultado += I2C[siguiente]

    return resultado


print('\n=== TEXTO GENERADO POR EL NANO-LLM ===')
print(generar('la inteligencia', n_chars=120))
print()
print(generar('los modelos de', n_chars=120))
# -- Guardar los pesos entrenados -------------------------------
# Los Capítulos 11, 14, 15 y 18 reutilizan este modelo ya entrenado:
#   params = dict(np.load('nano_llm_pesos.npz'))
np.savez('pesos_ejercicio_13_5.npz', **params)
print("\nPesos guardados en pesos_ejercicio_13_5.npz")
```

> Salida del programa (original, warm-up + coseno, PASOS = 4000):

```
Paso     0: pérdida = 3.8547   lr = 0.00002   (bits/char = 5.56)
Paso  1000: pérdida = 1.3384   lr = 0.00266   (bits/char = 1.93)
Paso  2000: pérdida = 0.8760   lr = 0.00164   (bits/char = 1.26)
Paso  3000: pérdida = 0.4716   lr = 0.00056   (bits/char = 0.68)
Paso  3999: pérdida = 0.4683   lr = 0.00010   (bits/char = 0.68)
```

> Salida del programa (LR constante = 0.003):

```
Paso     0: pérdida = 3.8547   lr = 0.00300   (bits/char = 5.56)
Paso  1000: pérdida = 1.3674   lr = 0.00300   (bits/char = 1.97)
Paso  2000: pérdida = 0.8820   lr = 0.00300   (bits/char = 1.27)
Paso  3000: pérdida = 0.5741   lr = 0.00300   (bits/char = 0.83)
Paso  3999: pérdida = 0.6111   lr = 0.00300   (bits/char = 0.88)
```

Con LR constante la pérdida final es claramente peor (0.61 frente a 0.47). Al principio, en cambio, van casi a la par (1.37 frente a 1.34 en el paso 1000): con Adam y el recorte del gradiente, este modelo pequeño apenas necesita el warm-up para arrancar. La diferencia la marca el decaimiento: sin él, el tamaño de paso sigue siendo grande cuando el modelo ya está cerca de un buen mínimo, y los pasos grandes lo hacen «rebotar» alrededor de la solución en vez de asentarse en ella, que es exactamente lo que predice la sección 13.3.3. Con solo cinco valores impresos no se ve el rebote paso a paso; si quieres verlo, registra la pérdida de cada paso, como en la curva de la sección 13.5.1.

Ejercicio 13.6 Verifica tu backward

El backward del Programa 8 tiene más de cien líneas de derivadas escritas a mano. Un signo cambiado o un factor olvidado no rompen el programa: el modelo sigue entrenando, solo que peor, y nadie avisa. Escribe una **comprobación de gradientes**: para elementos elegidos al azar de cada tensor, compara el gradiente analítico de `forward_backward()` con el numérico, (L(w + ε) − L(w − ε)) / 2ε. Después sustituye `gelu_bwd` por una versión que olvida la regla del producto (solo el término `0.5·(1 + t)`) y repite la comprobación.

> 💡 **Pista**
>
> Usa ε = 10^{-5} y la diferencia centrada, mucho más precisa que (L(w + ε) − L(w)) / ε. En cada uno de los 36 tensores se comprueban 6 elementos elegidos al azar (216 derivadas en total). Mide el error de cada tensor con ‖num − ana‖ / (‖num‖ + ‖ana‖). Con los pesos de la inicialización, los gradientes de Wq y Wk son diminutos y la comparación es poco fiable por el redondeo: carga los pesos entrenados (nano_llm_pesos.npz). Un error por debajo de 10^{-7} indica que el backward es correcto.

✅ Solución:

📄 ejercicio_13_6.py

```
# ============================================================
# EJERCICIO 13.6 — Verifica tu backward con diferencias finitas
# Compara el gradiente analítico de forward_backward() con el
# gradiente numérico (L(w+ε) − L(w−ε)) / 2ε en elementos al azar
# de TODOS los tensores del Programa 8.
# ============================================================
import numpy as np

# Cargamos del Programa 8 solo la parte anterior al entrenamiento
# (datos, parámetros, utilidades, forward y forward_backward) y
# después los pesos que guardó al terminar (nano_llm_pesos.npz).
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec(fuente.split('# -- Learning rate')[0])
# Con los pesos ya entrenados los gradientes son mayores que en la
# inicialización y la comparación numérica es más fiable.
params.update(np.load('nano_llm_pesos.npz'))

rng = np.random.RandomState(0)
inicios = rng.randint(0, len(DATOS) - CTX - 1, size=2)      # un lote de 2 ventanas
x = np.stack([DATOS[i:i + CTX] for i in inicios])
y = np.stack([DATOS[i + 1:i + CTX + 1] for i in inicios])


def comprobar(n_muestras=6, eps=1e-5):
    """Error relativo ||num − ana|| / (||num|| + ||ana||) por tensor."""
    grads, _ = forward_backward(x, y)
    errores = {}
    for nombre, P in params.items():
        num, ana = [], []
        for _ in range(n_muestras):
            j = tuple(rng.randint(s) for s in P.shape)
            orig = P[j]
            P[j] = orig + eps; l_mas = forward_backward(x, y)[1]
            P[j] = orig - eps; l_menos = forward_backward(x, y)[1]
            P[j] = orig
            num.append((l_mas - l_menos) / (2 * eps))
            ana.append(grads[nombre][j])
        num, ana = np.array(num), np.array(ana)
        if max(abs(num).max(), abs(ana).max()) < 1e-8:
            errores[nombre] = None          # los dos gradientes son ≈ 0
        else:
            errores[nombre] = np.linalg.norm(num - ana) / (np.linalg.norm(num) +
np.linalg.norm(ana))
    return errores


def maximo(errores):
    return max(e for e in errores.values() if e is not None)


errores = comprobar()
print('\nTensor      error relativo')
for nombre, e in errores.items():
    print(f'{nombre:10s}  ' + ('gradiente ≈ 0 (numérico y analítico)' if e is None else
f'{e:.1e}'))
print(f'\nError relativo máximo: {maximo(errores):.1e}')

# ¿Y si olvidamos la regla del producto en la derivada de GELU?
def gelu_bwd(x):                                # versión incompleta
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t)                        # falta el segundo término

rng = np.random.RandomState(0)
errores_mal = comprobar()
print('\nSin el segundo término de la derivada de GELU:')
for nombre in ['W10', 'b10', 'W20', 'Wq0', 'wte']:
    print(f'{nombre:10s}  {errores_mal[nombre]:.1e}')
print(f'Error relativo máximo: {maximo(errores_mal):.1e}')
```

```
> Salida del programa:
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas

Tensor      error relativo
wte         1.4e-09
wpe         6.0e-10
lnf_g       2.1e-09
lnf_b       1.4e-09
ln1_g0      1.9e-09
ln1_b0      3.5e-09
Wq0         2.8e-09
bq0         2.0e-09
Wk0         1.8e-09
bk0         gradiente ≈ 0 (numérico y analítico)
Wv0         1.3e-09
bv0         7.6e-09
Wo0         3.3e-09
bo0         6.1e-09
ln2_g0      4.5e-09
ln2_b0      3.3e-09
W10         5.5e-09
b10         4.2e-09
W20         4.5e-09
b20         3.1e-09
ln1_g1      3.6e-09
ln1_b1      6.8e-09
Wq1         1.7e-09
bq1         5.2e-09
Wk1         4.3e-09
bk1         gradiente ≈ 0 (numérico y analítico)
Wv1         4.1e-09
bv1         4.1e-09
Wo1         2.7e-09
bo1         3.5e-09
ln2_g1      3.3e-09
ln2_b1      2.4e-09
W11         1.3e-09
b11         1.3e-08
W21         3.5e-09
b21         2.8e-09

Error relativo máximo: 1.3e-08

Sin el segundo término de la derivada de GELU:
W10         7.9e-01
b10         1.8e-01
W20         1.1e-01
Wq0         1.3e-01
wte         3.9e-02
Error relativo máximo: 7.9e-01
```

Todos los tensores coinciden con errores de entre 10^{-9} y 10^{-8}: el backward es correcto en las 216 derivadas comprobadas, repartidas por los 36 tensores y no solo en la salida. Es una prueba por muestreo: no recorre las 60 000, pero un error en una fórmula afecta a casi todos los elementos de su tensor, y por eso una muestra basta para detectarlo. Hay dos tensores especiales, `bk0` y `bk1`, los sesgos de las claves: su gradiente es exactamente 0, tanto el analítico como el numérico. No es un error. Sumar el mismo vector `bk` a todas las claves suma la misma cantidad, Q·bk, a todas las puntuaciones de una fila, y el softmax no cambia si a toda una fila se le suma lo mismo: `bk` no puede influir en la pérdida. GPT-2 lo tiene de todos modos, y nosotros también, para que la arquitectura sea idéntica. En cambio, sin el segundo término de la derivada de GELU el error salta a 0.79 en `W10`, y se propaga hacia atrás a casi todos los tensores (0.13 en `Wq0`, 0.04 en `wte`): la comprobación lo detecta sin ambigüedad.

Ejercicio 13.7 El recorte del gradiente, con y sin

La sección 13.3.4 afirma que, en el Programa 8, el recorte actúa en el 98 % de los pasos. Compruébalo registrando la norma del gradiente en cada paso. Después desactiva el

recorte y compara la pérdida y las normas. Como ampliación, la sección 13.6.1 muestra qué ocurre al añadir weight decay al estilo de AdamW (5.6).

> 💡 **Pista**
>
> Basta con añadir una línea antes de `if norma > CLIP:` que guarde la norma en una lista. Para desactivar el recorte, `CLIP = float('inf')`: la condición no se cumple nunca.

✅ Solución:

📄 ejercicio_13_7.py

```
# ============================================================
# EJERCICIO 13.7 — ¿Qué hace de verdad el recorte del gradiente?
# Ejecuta el Programa 8 registrando la norma del gradiente en
# cada paso, con recorte (CLIP = 1.0) o sin él.
# Uso:  python ejercicio_13_7.py            -> con recorte
#       python ejercicio_13_7.py sin_recorte -> sin recorte
# ============================================================
import sys
import numpy as np

SIN_RECORTE = len(sys.argv) > 1 and sys.argv[1] == 'sin_recorte'
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
fuente = fuente.replace("    if norma > CLIP:", "    NORMAS.append(norma)\n    if norma >
CLIP:")
if SIN_RECORTE:
    fuente = fuente.replace("CLIP   = 1.0 ", "CLIP   = float('inf')")
fuente = fuente.replace('nano_llm_pesos.npz', 'pesos_ejercicio_13_7.npz')
NORMAS = []
exec(fuente)
NORMAS = np.array(NORMAS)
print(f'\n{"SIN" if SIN_RECORTE else "CON"} recorte — norma del gradiente antes de recortar:')
print(f'  mediana {np.median(NORMAS):.2f} | máxima {NORMAS.max():.2f} | '
      f'pasos con norma > 1: {(NORMAS > 1).sum()} de {len(NORMAS)}')
for a, b in [(0, 1000), (1000, 6000), (6000, 12000)]:
    print(f'  pasos {a:5d}-{b - 1:5d}: mediana {np.median(NORMAS[a:b]):.2f}')
```

> Salida del programa (con recorte; extracto):

```
Paso     0: pérdida = 3.8547   lr = 0.00002   (bits/char = 5.56)
Paso  1000: pérdida = 1.3802   lr = 0.00296   (bits/char = 1.99)
Paso  2000: pérdida = 0.9349   lr = 0.00283   (bits/char = 1.35)
Paso  3000: pérdida = 0.6037   lr = 0.00261   (bits/char = 0.87)
Paso  4000: pérdida = 0.3814   lr = 0.00231   (bits/char = 0.55)
Paso  5000: pérdida = 0.4244   lr = 0.00196   (bits/char = 0.61)
Paso  6000: pérdida = 0.3764   lr = 0.00158   (bits/char = 0.54)
Paso  7000: pérdida = 0.3344   lr = 0.00120   (bits/char = 0.48)
Paso  8000: pérdida = 0.3753   lr = 0.00084   (bits/char = 0.54)
Paso  9000: pérdida = 0.3107   lr = 0.00053   (bits/char = 0.45)
Paso 10000: pérdida = 0.2764   lr = 0.00030   (bits/char = 0.40)
Paso 11000: pérdida = 0.2689   lr = 0.00015   (bits/char = 0.39)
Paso 11999: pérdida = 0.2866   lr = 0.00010   (bits/char = 0.41)
CON recorte — norma del gradiente antes de recortar:
  mediana 1.67 | máxima 5.65 | pasos con norma > 1: 11758 de 12000
  pasos     0-  999: mediana 1.52
  pasos  1000- 5999: mediana 2.26
  pasos  6000-11999: mediana 1.34
```

> Salida del programa (sin recorte; extracto):

```
Paso     0: pérdida = 3.8547   lr = 0.00002   (bits/char = 5.56)
Paso  1000: pérdida = 1.3656   lr = 0.00296   (bits/char = 1.97)
Paso  2000: pérdida = 0.9621   lr = 0.00283   (bits/char = 1.39)
Paso  3000: pérdida = 0.5490   lr = 0.00261   (bits/char = 0.79)
Paso  4000: pérdida = 0.4463   lr = 0.00231   (bits/char = 0.64)
Paso  5000: pérdida = 0.4254   lr = 0.00196   (bits/char = 0.61)
Paso  6000: pérdida = 0.4157   lr = 0.00158   (bits/char = 0.60)
Paso  7000: pérdida = 0.2983   lr = 0.00120   (bits/char = 0.43)
Paso  8000: pérdida = 0.3607   lr = 0.00084   (bits/char = 0.52)
Paso  9000: pérdida = 0.3326   lr = 0.00053   (bits/char = 0.48)
Paso 10000: pérdida = 0.3031   lr = 0.00030   (bits/char = 0.44)
Paso 11000: pérdida = 0.3041   lr = 0.00015   (bits/char = 0.44)
Paso 11999: pérdida = 0.2654   lr = 0.00010   (bits/char = 0.38)
SIN recorte — norma del gradiente antes de recortar:
  mediana 1.57 | máxima 5.69 | pasos con norma > 1: 11716 de 12000
  pasos     0-  999: mediana 1.54
  pasos  1000- 5999: mediana 2.11
  pasos  6000-11999: mediana 1.31
```

Con y sin recorte, el entrenamiento es prácticamente el mismo: la pérdida del último mini-lote es 0.2866 con recorte y 0.2654 sin él (es la de un solo mini-lote, así que esa diferencia es ruido), y las normas del gradiente tienen la misma forma (mediana 1.67 y 1.57; máximo 5.65 y 5.69: no hay explosiones). ¿Cómo puede ser, si el recorte actúa en el 98 % de los pasos? Por Adam: cada actualización divide la media móvil del gradiente (m) por la raíz de la media móvil de su cuadrado (√v). Si el gradiente se multiplica por un factor parecido en todos los pasos, m y √v se multiplican por el mismo factor y el paso apenas cambia. El recorte importa cuando aparece un gradiente anómalo, mucho mayor que los habituales: entonces limita su efecto antes de que contamine m y v. En este modelo pequeño y estable eso no ocurre, pero en redes profundas o con tasas de aprendizaje altas sí (5.8). Es, literalmente, una red de seguridad: en una ejecución normal no se nota; está ahí para el día en que algo va mal.

Capítulo 14: Estrategias de Muestreo — Top-k y Top-p (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Comparar Greedy, temperatura, Top-k y Top-p sobre la misma distribución.
> ✓ Explicar por qué Top-p es adaptativo y Top-k no lo es.
> ✓ Implementar un generador configurable con repetition penalty.
> ✓ Elegir la configuración de muestreo adecuada según el tipo de tarea.

14.1 El momento de elegir el siguiente token

Nuestro Nano-LLM, al generar texto, hace siempre lo mismo en cada paso: calcular un vector de logits de tamaño V (uno por cada token del vocabulario) y convertirlo en una distribución de probabilidad con softmax. La pregunta es: ¿cómo elegimos el siguiente token a partir de esa distribución?

Esa decisión, que parece técnica y menor, tiene un efecto enorme en la calidad y el carácter del texto generado. Es la diferencia entre un modelo que siempre repite las mismas frases y uno que sorprende con variedad manteniendo coherencia. Los LLMs modernos emplean estrategias específicas, y entenderlas te permite controlar con precisión el comportamiento del modelo.

Para visualizar los efectos, trabajaremos con una distribución de ejemplo de 12 tokens que representa un caso típico: unos pocos tokens muy probables y muchos con probabilidades bajas.

14.2 Las cuatro estrategias fundamentales

Estrategia 1: Greedy Decoding (decodificación codiciosa)

La más simple: siempre elegir el token con mayor probabilidad. Es completamente determinista — dado el mismo contexto, producirá siempre el mismo texto. Su ventaja es la coherencia; su desventaja, la monotonía.

📄 Greedy decoding

```
def greedy(probs):
    return np.argmax(probs)   # siempre el maximo

# Problema: el modelo queda 'atrapado' en bucles.
# Ejemplo de texto greedy:
# 'el modelo aprende de los datos el modelo aprende
#  de los datos el modelo aprende de los datos...'
# Una vez que el modelo entra en un patron frecuente,
# el siguiente token mas probable refuerza ese patron.
```

Estrategia 2: Muestreo con temperatura

Que ya conocemos del Capítulo 13: dividir los logits por τ antes del softmax. Temperatura baja → distribución más concentrada (más determinista). Temperatura alta → distribución más plana (más aleatoria). El problema que no resuelve: aunque la distribución esté bien calibrada, tokens con probabilidad 0.001 siguen pudiendo ser elegidos, lo que puede producir palabras completamente fuera de contexto.

Estrategia 3: Top-k

Top-k resuelve el problema de los tokens improbables con una idea simple: antes de muestrear, nos quedamos solo con los k tokens más probables y asignamos probabilidad cero al resto. Luego renormalizamos para que sumen 1 y muestreamos de ese conjunto reducido.

📄 **Top-k: la idea central**

```
def top_k_filtrar(probs, k):
    """Conserva solo los k tokens mas probables."""
    # Ordenar por probabilidad descendente
    indices_ord = np.argsort(probs)[::-1]
    # Los k primeros se quedan; el resto a cero
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:k]] = probs[indices_ord[:k]]
    # Renormalizar para que sumen 1
    return nueva / nueva.sum()

# k=1:   equivalente a greedy (solo el mejor token)
# k=50:  los 50 mejores -> buena variedad sin tokens raros
# k=V:   todos los tokens -> muestreo completo

# Problema de Top-k: k es fijo independientemente
# de la forma de la distribucion.
# Con distribucion muy concentrada: k=50 incluye
# tokens con prob 0.001 que son 'basura'.
# Con distribucion muy plana: k=50 excluye tokens
# razonables con prob 0.01.
```

Estrategia 4: Top-p (nucleus sampling)

Top-p, propuesto por Holtzman et al. en 2020 en el artículo 'The Curious Case of Neural Text Degeneration', resuelve el problema de Top-k de forma elegante. En lugar de fijar un número de tokens, fijamos una probabilidad acumulada p. Se selecciona el conjunto mínimo de tokens cuyas probabilidades suman al menos p, y se muestrea de ese 'núcleo' (nucleus).

La clave es que este conjunto se adapta automáticamente a la forma de la distribución. Cuando la distribución es muy concentrada (hay un token claramente más probable), el núcleo contiene pocos tokens. Cuando es más plana (muchas opciones razonables), el núcleo es más grande. En ambos casos, los tokens muy improbables quedan siempre excluidos.

📄 **Top-p (nucleus sampling): la idea central**

```
def top_p_filtrar(probs, p):
    """Conserva el minimo conjunto de tokens que suman probabilidad p."""
    # Ordenar por probabilidad descendente
    indices_ord = np.argsort(probs)[::-1]
    probs_ord   = probs[indices_ord]
    # Suma acumulada
    suma_acum   = np.cumsum(probs_ord)
    # Encontrar cuantos tokens necesitamos para llegar a p
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    # Conservar solo esos tokens
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
    return nueva / nueva.sum()

# p=0.9:  conservar tokens hasta cubrir el 90% de la prob.
#         Con dist. concentrada: puede bastar 2-3 tokens.
#         Con dist. plana: puede necesitar 30-40 tokens.
# p=1.0:  equivalente al muestreo completo.
# p=0.0:  equivalente a greedy (solo el primer token).
```

PROGRAMA S-1 — Visualización de todas las estrategias de muestreo

📄 **programa_S1_visualizacion.py**

```
import numpy as np

# ============================================================
# FUNCIONES DE MUESTREO
# ============================================================

def softmax(x, tau=1.0):
    z = (x - x.max()) / tau
    e = np.exp(z)
    return e / e.sum()

def top_k_filtrar(probs, k):
```

```
    idx  = np.argsort(probs)[::-1]
    mask = np.zeros_like(probs)
    mask[idx[:k]] = probs[idx[:k]]
    return mask / mask.sum()

def top_p_filtrar(probs, p):
    idx       = np.argsort(probs)[::-1]
    acum      = np.cumsum(probs[idx])
    n         = int(np.searchsorted(acum, p)) + 1
    mask      = np.zeros_like(probs)
    mask[idx[:n]] = probs[idx[:n]]
    return mask / mask.sum()

def muestrear(probs, n=10000):
    """Muestrea n tokens y devuelve las frecuencias observadas."""
    muestras = np.random.choice(len(probs), size=n, p=probs)
    freqs    = np.bincount(muestras, minlength=len(probs)) / n
    return freqs

def visualizar_dist(probs, nombre, tokens):
    """Muestra la distribucion como diagrama de barras ASCII."""
    print(f'  {nombre}:')
    activos = sum(p > 0.001 for p in probs)
    print(f'  Tokens activos (prob > 0.1%): {activos}/{len(probs)}')
    for i, (tok, p) in enumerate(zip(tokens, probs)):
        if p < 0.001: continue
        barra  = int(p * 50)
        bloque = '#' * barra
        print(f'    {tok:>12}: {p:>6.3f}  {bloque}')
    print()


np.random.seed(42)

# ================================================================
# DISTRIBUCION DE EJEMPLO: vocabulario de 12 tokens
# Simula la salida de un LLM en un contexto concreto
# ================================================================
tokens = ['el', 'la', 'un', 'los', 'de',
          'que', 'en', 'y', 'a', 'por', 'extraño', 'xyz']

# Logits tipicos: unos pocos tokens muy probables, muchos improbables
logits = np.array([3.8, 3.2, 2.1, 1.8, 1.5,
                   1.0, 0.5, 0.2, -0.3, -0.8, -2.0, -4.0])

print('='*60)
print('VISUALIZACION DE ESTRATEGIAS DE MUESTREO')
print('='*60)
print()

# --- DISTRIBUCION BASE (temperatura 1.0) ---
probs_base = softmax(logits, tau=1.0)
visualizar_dist(probs_base, 'BASE  (tau=1.0, sin filtros)', tokens)

# --- TEMPERATURA BAJA ---
probs_fria = softmax(logits, tau=0.5)
visualizar_dist(probs_fria, 'TEMPERATURA BAJA  (tau=0.5)', tokens)

# --- TOP-K ---
probs_k3 = top_k_filtrar(probs_base, k=3)
visualizar_dist(probs_k3, 'TOP-K=3  (solo los 3 mejores)', tokens)

probs_k6 = top_k_filtrar(probs_base, k=6)
visualizar_dist(probs_k6, 'TOP-K=6  (los 6 mejores)', tokens)

# --- TOP-P ---
probs_p80 = top_p_filtrar(probs_base, p=0.80)
n_tokens_80 = sum(probs_p80 > 0)
visualizar_dist(probs_p80, f'TOP-P=0.80  ({n_tokens_80} tokens cubren 80%)', tokens)

probs_p95 = top_p_filtrar(probs_base, p=0.95)
n_tokens_95 = sum(probs_p95 > 0)
visualizar_dist(probs_p95, f'TOP-P=0.95  ({n_tokens_95} tokens cubren 95%)', tokens)
```

```
# --- COMPARATIVA: tokens incluidos por cada estrategia ---
print('COMPARATIVA: Tokens incluidos por cada estrategia')
print(f"{'Token':>12}  {'Base':>8}  {'tau=0.5':>8}  ",
      f"{'Top-k=3':>8}  {'Top-k=6':>8}  {'Top-p.80':>9}  {'Top-p.95':>9}")
print('-' * 75)
for i, tok in enumerate(tokens):
    p0 = probs_base[i]
    p1 = probs_fria[i]
    p2 = probs_k3[i]
    p3 = probs_k6[i]
    p4 = probs_p80[i]
    p5 = probs_p95[i]
    def fmt(p): return f'{p:.3f}' if p > 0.001 else '  ---'
    print(f'{tok:>12}  {fmt(p0):>8}  {fmt(p1):>8}  ',
          f'{fmt(p2):>8}  {fmt(p3):>8}  {fmt(p4):>9}  {fmt(p5):>9}')

print()
print('Observacion clave:')
print('  Top-k=3: exactamente 3 tokens siempre, sin importar la distribucion.')
print('  Top-p=0.80: el numero de tokens se adapta a la forma de la dist.')
print('  Con dist. concentrada: Top-p usara MENOS tokens que Top-k=6.')
print('  Con dist. plana: Top-p usara MAS tokens que Top-k=6.')
```

> Salida del programa:

```
============================================================
VISUALIZACION DE ESTRATEGIAS DE MUESTREO
============================================================

  BASE  (tau=1.0, sin filtros):
  Tokens activos (prob > 0.1%): 11/12
             el:  0.471  ######################
             la:  0.259  ############
             un:  0.086  ####
            los:  0.064  ###
             de:  0.047  ##
            que:  0.029  #
             en:  0.017
              y:  0.013
              a:  0.008
            por:  0.005
         extraño:  0.001

  TEMPERATURA BAJA  (tau=0.5):
  Tokens activos (prob > 0.1%): 6/12
             el:  0.730  ####################################
             la:  0.220  ##########
             un:  0.024  #
            los:  0.013
             de:  0.007
            que:  0.003

  TOP-K=3  (solo los 3 mejores):
  Tokens activos (prob > 0.1%): 3/12
             el:  0.578  ############################
             la:  0.317  ###############
             un:  0.106  #####

  TOP-K=6  (los 6 mejores):
  Tokens activos (prob > 0.1%): 6/12
             el:  0.493  ########################
             la:  0.271  #############
             un:  0.090  ####
            los:  0.067  ###
             de:  0.049  ##
            que:  0.030  #

  TOP-P=0.80  (3 tokens cubren 80%):
  Tokens activos (prob > 0.1%): 3/12
             el:  0.578  ############################
             la:  0.317  ###############
             un:  0.106  #####

  TOP-P=0.95  (6 tokens cubren 95%):
  Tokens activos (prob > 0.1%): 6/12
             el:  0.493  ########################
             la:  0.271  #############
             un:  0.090  ####
```

```
              los:  0.067  ###
               de:  0.049  ##
              que:  0.030  #

COMPARATIVA: Tokens incluidos por cada estrategia
       Token      Base   tau=0.5    Top-k=3   Top-k=6   Top-p.80   Top-p.95
------------------------------------------------------------------------------
          el     0.471     0.730      0.578     0.493      0.578      0.493
          la     0.259     0.220      0.317     0.271      0.317      0.271
          un     0.086     0.024      0.106     0.090      0.106      0.090
         los     0.064     0.013        ---     0.067        ---      0.067
          de     0.047     0.007        ---     0.049        ---      0.049
         que     0.029     0.003        ---     0.030        ---      0.030
          en     0.017       ---        ---       ---        ---        ---
           y     0.013       ---        ---       ---        ---        ---
           a     0.008       ---        ---       ---        ---        ---
         por     0.005       ---        ---       ---        ---        ---
     extraño     0.001       ---        ---       ---        ---        ---
         xyz       ---       ---        ---       ---        ---        ---

Observacion clave:
  Top-k=3: exactamente 3 tokens siempre, sin importar la distribucion.
  Top-p=0.80: el numero de tokens se adapta a la forma de la dist.
  Con dist. concentrada: Top-p usara MENOS tokens que Top-k=6.
  Con dist. plana: Top-p usara MAS tokens que Top-k=6.
```

Fíjate en el resultado central: con Top-p=0.80, solo 3 tokens son necesarios para cubrir el 80% de la probabilidad, porque 'el' tiene ya un 47.1% de probabilidad. Top-k=6, en cambio, incluiría siempre 6 tokens independientemente de esta concentración. Top-p es adaptativo; Top-k no lo es.

> ℹ️ En la práctica, los dos tokens 'extraño' y 'xyz' tienen probabilidad tan baja que ninguna estrategia razonable los incluiría. Pero sin filtros, con temperatura alta, podrían salir. Top-k y Top-p los descartan antes de muestrear: con un filtro adecuado, los tokens con probabilidad residual ya no pueden salir.

14.3 La Repetition Penalty — evitar los bucles

Un problema frecuente en la generación de texto es que el modelo entra en bucles: repite las mismas palabras o frases una y otra vez. Ocurre porque copiar lo que ya está en el contexto es una de las regularidades más frecuentes del texto, y el modelo lo ha aprendido (en el Capítulo 16 verás las cabezas de atención que lo hacen): cuando una frase aparece, el modelo predice que se repetirá, y con greedy cada repetición refuerza la siguiente.

La repetition penalty es una técnica complementaria a Top-k y Top-p: antes de aplicar softmax, se reducen los logits de los tokens ya generados con un factor de penalización mayor que 1: los positivos se dividen entre él y los negativos se multiplican por él. Esto reduce su probabilidad relativa, incentivando la variedad. (Dividir un logit negativo lo acercaría a cero y, al revés de lo que se busca, subiría su probabilidad: $-2 / 1.2 = -1.67$.) El Programa S-2 lo hace así y solo mira los últimos 20 tokens generados.

📄 Repetition Penalty — implementación

```
def aplicar_repetition_penalty(logits, tokens_generados, penalizacion=1.3):
    """
    Reduce la probabilidad de tokens que ya aparecieron.
    penalizacion > 1: cuanto mayor, mas fuerte la penalizacion.
    Valor tipico: 1.1 a 1.5
    """
    logits = logits.copy()
    for token in set(tokens_generados):
        if logits[token] > 0:
            logits[token] /= penalizacion   # reducir si positivo
        else:
            logits[token] *= penalizacion   # reducir si negativo
    return logits

# Ejemplo:
# Si 'el' ya aparecio 3 veces y tiene logit=3.8:
# Con penalizacion=1.3: nuevo logit = 3.8/1.3 = 2.92
# La probabilidad de 'el' baja, incentivando otros tokens.
```

```
# Advertencia: penalizacion muy alta produce textos incoherentes.
# El modelo puede evitar palabras necesarias gramaticalmente.
# 1.1 a 1.3 son valores razonables para la mayoria de casos.
```

PROGRAMA S-2 — Generación con todas las estrategias

📄 **programa_S2_generacion_completa.py**

```
import numpy as np
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec(fuente.split('# -- Learning rate')[0])     # tokenizador y forward() del Programa
8
params = dict(np.load('nano_llm_pesos.npz'))   # los pesos que guardó el Programa 8

# ================================================================
# GENERADOR UNIFICADO CON TODAS LAS ESTRATEGIAS
# Este codigo amplia la funcion generar() del Nano-LLM
# con soporte para todas las estrategias de muestreo.
# ================================================================

def softmax(x, axis=-1): e=np.exp(x-x.max(axis, keepdims=True)); return e/e.sum(axis,
keepdims=True)
def top_k_filtrar(p, k): idx=np.argsort(p)[::-1]; m=np.zeros_like(p);
m[idx[:k]]=p[idx[:k]]; return m/m.sum()
def top_p_filtrar(p, pp): idx=np.argsort(p)[::-1]; ac=np.cumsum(p[idx]);
n=int(np.searchsorted(ac,pp))+1; m=np.zeros_like(p); m[idx[:n]]=p[idx[:n]]; return
m/m.sum()

def generar_avanzado(
    modelo_forward,  # funcion forward del Nano-LLM
    c2i, i2c,        # diccionarios del tokenizador
    semilla,         # texto inicial
    n       = 80,    # tokens a generar
    CTX     = 24,    # longitud de contexto
    # Hiperparametros de muestreo:
    tau     = 1.0,   # temperatura
    k       = 0,     # top-k (0 = desactivado)
    p       = 0.0,   # top-p (0 = desactivado)
    rep_pen = 1.0,   # repetition penalty (1.0 = sin penalizacion)
):
    """
    Generacion flexible con todas las estrategias combinables.
    El orden de aplicacion es: temperatura -> top-k -> top-p -> muestreo
    """
    V       = len(c2i)
    tokens  = [c2i.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla
    generados = list(tokens)   # para repetition penalty

    for _ in range(n):
        ctx    = tokens[-CTX:]   # SIN relleno: se usa la longitud real

        # 1. Obtener logits del modelo
        logits = modelo_forward(ctx)[-1]

        # 2. Aplicar repetition penalty
        if rep_pen != 1.0:
            for t in set(generados[-20:]):   # solo los ultimos 20
                if logits[t] > 0: logits[t] /= rep_pen
                else:             logits[t] *= rep_pen

        # 3. Aplicar temperatura
        probs = softmax(logits / max(tau, 1e-5))

        # 4. Aplicar top-k (si esta activado)
        if k > 0:
            probs = top_k_filtrar(probs, k)

        # 5. Aplicar top-p (si esta activado)
        if 0 < p < 1.0:
            probs = top_p_filtrar(probs, p)
```

```
            # 6. Muestrear
            siguiente = int(np.random.choice(V, p=probs))
            tokens.append(siguiente)
            generados.append(siguiente)
            resultado += i2c[siguiente]

        return resultado


    # ============================================================
    # COMPARATIVA: misma semilla, distintas estrategias
    # (Nano-LLM del Cap 13 ya entrenado: params = dict(np.load('nano_llm_pesos.npz')))
    # ============================================================

    estrategias = [
        {'nombre': 'Greedy (tau=0.01)',
         'tau': 0.01, 'k': 0, 'p': 0.0, 'rep_pen': 1.0},

        {'nombre': 'Solo temperatura (tau=0.8)',
         'tau': 0.8,  'k': 0, 'p': 0.0, 'rep_pen': 1.0},

        {'nombre': 'Top-k=5, tau=0.8',
         'tau': 0.8,  'k': 5, 'p': 0.0, 'rep_pen': 1.0},

        {'nombre': 'Top-p=0.90, tau=0.8',
         'tau': 0.8,  'k': 0, 'p': 0.9, 'rep_pen': 1.0},

        {'nombre': 'Top-p=0.90 + rep_pen=1.2',
         'tau': 0.8,  'k': 0, 'p': 0.9, 'rep_pen': 1.2},
    ]

    SEMILLA = 'la inteligencia'
    print(f'=== COMPARATIVA DE ESTRATEGIAS ===')
    print(f'Semilla: "{SEMILLA}"  |  70 tokens a generar')
    print()

    # El forward del Programa 8 trabaja con lotes: le pasamos un lote de una
    # sola secuencia y nos quedamos con sus logits, de forma (T, V)
    def logits_nano(ctx):
        return forward(np.array([ctx]))[0][0]

    np.random.seed(7)
    for conf in estrategias:
        np.random.seed(7)   # misma semilla aleatoria para comparar
        nombre = conf.pop('nombre')
        texto = generar_avanzado(logits_nano, C2I, I2C, SEMILLA, n=70, CTX=CTX, **conf)
        # (C2I, I2C y forward son los del Nano-LLM ya entrenado, Programa 8)
        print(f'  [{nombre}]')
        print(f'  -> {texto}')
        print()

    # ============================================================
    # ANALISIS: efecto de tau y top-p sobre la diversidad
    # ============================================================
    print('=== EFECTO EN LA DIVERSIDAD DEL TEXTO ===')
    print()
    print('Tau muy bajo (< 0.3):')
    print('  El modelo siempre elige el token mas probable.')
    print('  Texto coherente pero repetitivo. Mismo texto siempre.')
    print()
    print('Tau 0.7-0.9 + Top-p=0.90 (configuracion estandar):')
    print('  Balance entre coherencia y variedad.')
    print('  Cada ejecucion produce texto distinto pero sensato.')
    print('  Es la configuracion por defecto de la mayoria de LLMs.')
    print()
    print('Tau muy alto (> 1.5):')
    print('  El texto se vuelve incoherente. Palabras aleatorias.')
    print('  Util para explorar espacios creativos extremos.')
    print()
    print('Top-p=0.90 + rep_pen=1.2:')
    print('  Evita la repeticion. Necesario en textos largos.')
    print('  Riesgo: penalizacion muy alta puede evitar palabras necesarias.')
```

```
> Salida del programa:
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Texto de entrenamiento: 5076 tokens, 93 líneas
=== COMPARATIVA DE ESTRATEGIAS ===
Semilla: "la inteligencia"  |  70 tokens a generar

  [Greedy (tau=0.01)]
  -> la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el

  [Solo temperatura (tau=0.8)]
  -> la inteligencia artificial aprende de los datos.
los modelos de lenguades.
el aprendi

  [Top-k=5, tau=0.8]
  -> la inteligencia artificial aprende de los datos.
los modelos de lenguades.
el aprendi

  [Top-p=0.90, tau=0.8]
  -> la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el

  [Top-p=0.90 + rep_pen=1.2]
  -> la inteligencia artificial aprende de los datos.
los modelos de lenguaje práctica, un

=== EFECTO EN LA DIVERSIDAD DEL TEXTO ===

Tau muy bajo (< 0.3):
  El modelo siempre elige el token mas probable.
  Texto coherente pero repetitivo. Mismo texto siempre.

Tau 0.7-0.9 + Top-p=0.90 (configuracion estandar):
  Balance entre coherencia y variedad.
  Cada ejecucion produce texto distinto pero sensato.
  Es la configuracion por defecto de la mayoria de LLMs.

Tau muy alto (> 1.5):
  El texto se vuelve incoherente. Palabras aleatorias.
  Util para explorar espacios creativos extremos.

Top-p=0.90 + rep_pen=1.2:
  Evita la repeticion. Necesario en textos largos.
  Riesgo: penalizacion muy alta puede evitar palabras necesarias.
```

Con nuestro Nano-LLM, que ha memorizado su corpus (lo comprobaremos en el Capítulo 15), el greedy no entra en bucle: recita la primera frase del corpus y sigue con la segunda. La temperatura 0.8 introduce un desvío («lenguades»), y Top-k = 5 produce exactamente el mismo texto: el modelo está tan seguro que sus cinco candidatos más probables ya contienen casi toda la probabilidad. Top-p = 0.90, en cambio, recorta los candidatos residuales que causaron el desvío y vuelve al texto del corpus. Y rep_pen = 1.2 penaliza los caracteres ya usados y saca al modelo de la frase memorizada («predicen» se convierte en «práctica»). Los bucles del greedy aparecen cuando el modelo no tiene una continuación memorizada a la que agarrarse: en los LLM grandes son el fenómeno más visible, y el que justifica por sí solo la repetition penalty.

14.4 Guía práctica — ¿Qué parámetros usar?

Esta tabla recoge las configuraciones más habituales según el tipo de tarea:

```
  📄 Configuraciones recomendadas por tipo de tarea
COMPLETAR CODIGO:
  tau=0.2, top-p=0.95, rep_pen=1.0
  -> Codigo correcto requiere mucho determinismo.
  -> Top-p evita sintaxis invalida de tokens raros.

RESPONDER PREGUNTAS FACTUALES:
  tau=0.3, top-k=20, rep_pen=1.0
  -> Queremos la respuesta mas probable, no creatividad.

ESCRITURA CREATIVA:
  tau=0.9, top-p=0.92, rep_pen=1.1
  -> Alta temperatura para variedad.
  -> Rep_pen suave para evitar repeticion sin perder coherencia.
```

```
CHAT Y ASISTENTE GENERAL (el mas comun):
  tau=0.7, top-p=0.90, rep_pen=1.05
  -> Balance coherencia/variedad.
  -> Es un punto de partida habitual en asistentes.

GENERACION DETERMINISTA (pruebas/evaluacion):
  tau=0.0  o bien  greedy (argmax)
  -> Siempre el mismo output dado el mismo input.
  -> Indispensable para reproducibilidad en benchmarks.

REGLAS GENERALES:
  1. Nunca usar temperatura sola si tau > 0.8.
     Siempre combinar con top-p=0.9 o top-k=50.
  2. Top-p suele funcionar mejor que top-k en la practica.
  3. Rep_pen: empezar en 1.05-1.1 y subir solo si hay bucles.
  4. En evaluacion: fijar siempre la semilla aleatoria.
  5. tau y top-p son los dos hiperparametros que mas importan.
```

En tareas con una única respuesta correcta, como las sumas y deducciones de los Programas RZ-2 a RZ-4 (Capítulo 15), se usa greedy, sin muestreo: no queremos variedad, queremos la respuesta.

14.5 Beam Search — La estrategia que los LLMs abandonaron

Antes de Top-k y Top-p, el estándar en generación de texto era el Beam Search: en lugar de generar un solo token a la vez, se mantienen las B mejores secuencias parciales (los 'rayos') y se expanden en paralelo. Al final se elige la secuencia con mayor probabilidad acumulada.

Beam Search fue muy popular en traducción automática y sigue usándose en tareas en las que importa encontrar la secuencia más probable, como la traducción o el reconocimiento de voz. Pero para la generación de texto abierto tiene un problema crítico: produce texto genérico, seguro y aburrido. Las secuencias más probables no son las más interesantes ni las más humanas. En 2020, el artículo que propuso Top-p ('The Curious Case of Neural Text Degeneration') demostró sistemáticamente esta limitación y estableció el nucleus sampling como estándar.

📄 Por qué Beam Search falla en generacion abierta

```
# Beam Search maximiza la probabilidad CONJUNTA de toda la secuencia.
# El problema: texto humano no es 'el mas probable'.

# Ejemplo: beam search con B=4 generando el comienzo de un articulo:
# Rayo 1: 'The the the the the...'          <- muy alta prob. x token
# Rayo 2: 'In this paper we present...'     <- coherente pero generico
# Rayo 3: 'We propose a novel approach...'  <- formulaico
# Rayo 4: 'This work introduces a method...' <- idem

# El texto humano REAL tiene sorpresas, giros, metaforas:
# 'Imagine que tienes que preparar un examen...' <- prob. individual baja
# pero totalmente coherente y mas interesante.

# Conclusion: maximizar la probabilidad != generar el mejor texto.
# Top-p introduce la variabilidad necesaria para textos naturales.
```

Las estrategias de muestreo son uno de los pocos lugares donde la ingeniería de los LLMs toca directamente a la experiencia del usuario. Cuando Claude o ChatGPT producen textos que se sienten naturales y variados, es en parte porque Top-p y la temperatura están cuidadosamente calibrados. Con lo que has aprendido en este capítulo, puedes ahora ajustar esos parámetros con conocimiento de causa — y entender exactamente qué está haciendo el modelo en cada token que genera.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- Greedy decoding es determinista pero propenso a bucles; la temperatura sola no filtra tokens improbables.
- Top-k conserva un número fijo de tokens; Top-p (nucleus) se adapta a la forma real de la distribución.
- La repetition penalty evita bucles repetitivos penalizando tokens ya generados recientemente.

✏️ EJERCICIOS — Capítulo 14 — Estrategias de Muestreo

Muestrear bien es tan importante como entrenar bien. Resuelve estos ejercicios antes de continuar y compara tus salidas con las propuestas: verás cómo cada estrategia cambia el carácter del texto.

Ejercicio 14.1 Implementar Min-P sampling

Min-P es una estrategia de muestreo más reciente (2024) que conserva todos los tokens cuya probabilidad es al menos p_min veces la probabilidad del token más probable. Implementa min_p_filtrar(probs, p_min) y compárala con Top-p sobre la misma distribución del Programa S-1.

> 💡 💡 Pista: Min-P es conceptualmente simple: umbral = p_min * max(probs). Conserva solo los tokens con probabilidad >= umbral. A diferencia de Top-p, no necesita ordenar ni acumular — es más directo de calcular.

✅ Solución:

📄 **ejercicio_14_1.py**

```
import numpy as np

def softmax(x): e=np.exp(x-x.max()); return e/e.sum()

def min_p_filtrar(probs, p_min):
    '''Conserva tokens con prob >= p_min * prob_maxima'''
    umbral = p_min * probs.max()
    mask = np.where(probs >= umbral, probs, 0)
    return mask / mask.sum()

def top_p_filtrar(probs, p):
    idx = np.argsort(probs)[::-1]
    acum = np.cumsum(probs[idx])
    n = int(np.searchsorted(acum, p)) + 1
    mask = np.zeros_like(probs)
    mask[idx[:n]] = probs[idx[:n]]
    return mask / mask.sum()

tokens = ['el', 'la', 'un', 'los', 'de', 'que', 'en', 'y', 'a', 'por']
logits = np.array([3.8, 3.2, 2.1, 1.8, 1.5, 1.0, 0.5, 0.2, -0.3, -0.8])
probs  = softmax(logits)

print('=== MIN-P vs TOP-P ===\n')
for p_min in [0.05, 0.1, 0.3]:
    filtrado = min_p_filtrar(probs, p_min)
    n_activos = (filtrado > 0).sum()
    print(f'Min-P={p_min}: {n_activos} tokens activos')
    for tok, p in zip(tokens, filtrado):
        if p > 0: print(f'    {tok}: {p:.3f}')
    print()

for p in [0.8, 0.9]:
    filtrado = top_p_filtrar(probs, p)
    n_activos = (filtrado > 0).sum()
    print(f'Top-P={p}: {n_activos} tokens activos')
```

> ℹ️ Min-P es popular en la comunidad open-source porque es computacionalmente más barato que Top-p (no requiere ordenar) y se comporta de forma similar, adaptándose a la concentración de la distribución.

Ejercicio 14.2 Medir la diversidad léxica con distintas temperaturas

Genera 20 textos distintos (usando distintas semillas aleatorias) con el Nano-LLM para temperaturas 0.3, 0.7, 1.0 y 1.3. Mide la diversidad léxica como el número de palabras

únicas dividido por el número total de palabras generadas (type-token ratio). ¿Cómo cambia con la temperatura?

> 💡 💡 Pista: Un type-token ratio cercano a 1 indica alta diversidad (pocas repeticiones). Un ratio bajo indica que el modelo repite las mismas palabras. Espera ver el ratio aumentar con la temperatura, hasta cierto punto donde el texto se vuelve incoherente.

✅ **Solución:**

📄 **ejercicio_14_2.py**

```
import numpy as np

def type_token_ratio(texto):
    palabras = texto.lower().split()
    if not palabras: return 0
    return len(set(palabras)) / len(palabras)

# Simulacion: con el Nano-LLM real, cargar params = dict(np.load('nano_llm_pesos.npz'))
def simular_generacion(tau, semilla, vocab_efectivo=15):
    '''Simula texto generado con cierta temperatura'''
    np.random.seed(semilla)
    palabras_base = ['la','red','aprende','de','los','datos','el',
                     'modelo','predice','token','siguiente','con',
                     'atencion','capas','neuronas']
    n_palabras = 30
    # A mayor tau, mas aleatoriedad en la eleccion (simulado)
    if tau < 0.5:
        # Baja temperatura: repite mucho un subconjunto pequeño
        pool = palabras_base[:4]
    elif tau < 1.0:
        pool = palabras_base[:10]
    else:
        pool = palabras_base
    return ' '.join(np.random.choice(pool, n_palabras))

print('=== DIVERSIDAD LEXICA SEGUN TEMPERATURA ===\n')
print(f"{'Temperatura':>12}  {'TTR medio':>10}  {'TTR std':>10}")
print('-' * 38)

for tau in [0.3, 0.7, 1.0, 1.3]:
    ratios = []
    for semilla in range(20):
        texto = simular_generacion(tau, semilla)
        ratios.append(type_token_ratio(texto))
    print(f'{tau:>12.1f}  {np.mean(ratios):>10.3f}  {np.std(ratios):>10.3f}')

print()
print('TTR mayor = mas variedad lexica = menos repeticion')
print('Temperaturas bajas producen TTR bajo (texto repetitivo)')
print('Temperaturas muy altas pueden producir TTR alto pero incoherente')
```

Ejercicio 14.3 Diseñar una estrategia de muestreo adaptativa

Diseña una función de muestreo que use una temperatura baja (0.3) para los primeros 10 tokens generados (para anclar el inicio del texto de forma coherente) y luego cambie a temperatura más alta (0.9) con Top-p=0.9 para el resto (para añadir variedad). Justifica por qué esta estrategia podría ser útil.

> 💡 💡 Pista: La idea es que los primeros tokens fijan el tema y la estructura de la frase, y los siguientes admiten más libertad. Adaptar el muestreo según la posición es una idea razonable que puedes medir tú mismo; no hay una receta universal.

✅ **Solución:**

📄 **ejercicio_14_3.py**

```
import numpy as np

def softmax(x): e=np.exp(x-x.max()); return e/e.sum()
def top_p_filtrar(p, pp):
    idx=np.argsort(p)[::-1]; ac=np.cumsum(p[idx])
    n=int(np.searchsorted(ac,pp))+1
    m=np.zeros_like(p); m[idx[:n]]=p[idx[:n]]; return m/m.sum()

def generar_adaptativo(modelo_forward, c2i, i2c, semilla, n=60,
                       CTX=24, tau_inicial=0.3, tau_resto=0.9,
                       n_tokens_iniciales=10, top_p_resto=0.9):
    '''
    Estrategia adaptativa: temperatura baja al principio (coherencia),
    temperatura mas alta + top-p despues (variedad).
    '''
    V = len(c2i)
    tokens = [c2i.get(c, 0) for c in semilla[-CTX:]]
    resultado = semilla

    for i in range(n):
        ctx = tokens[-CTX:]   # SIN relleno: se usa la longitud real
        logits = modelo_forward(ctx)[-1]

        # Estrategia adaptativa segun la posicion
        if i < n_tokens_iniciales:
            tau = tau_inicial
            probs = softmax(logits / tau)
        else:
            tau = tau_resto
            probs = softmax(logits / tau)
            probs = top_p_filtrar(probs, top_p_resto)

        siguiente = int(np.random.choice(V, p=probs))
        tokens.append(siguiente)
        resultado += i2c[siguiente]

    return resultado

print('=== ESTRATEGIA DE MUESTREO ADAPTATIVA ===\n')
print('Primeros 10 tokens:  tau=0.3 (determinista, ancla el inicio)')
print('Tokens siguientes:   tau=0.9 + top-p=0.9 (variedad creativa)')
print()
print('Justificacion:')
print('  - El inicio de una respuesta suele tener pocas opciones')
print('    razonables (saludo, estructura gramatical inicial).')
print('  - Una vez establecido el contexto, hay mas margen creativo')
print('    sin perder coherencia.')
print('  - Esta tecnica reduce la varianza de calidad entre generaciones')
print('    manteniendo diversidad donde realmente aporta valor.')
```

ℹ️ Algunos sistemas de producción usan variantes de esta idea: temperatura dinámica según la posición, el tipo de tarea, o incluso la confianza del modelo en cada paso.

Capítulo 15: ¿Memoriza o razona? — Del loro estadístico al razonador (***)

> **🎯 AL TERMINAR ESTE CAPÍTULO SABRÁS:**
>
> ✓ Diagnosticar si un modelo de lenguaje memoriza o generaliza: validación, línea base de n-gramas y detección de copia literal.
>
> ✓ Explicar por qué la cantidad y la forma de los datos deciden si una red memoriza o extrae una regla.
>
> ✓ Entender por qué escribir los pasos intermedios (cadena de razonamiento) permite a un modelo de solo 2 bloques resolver problemas largos.
>
> ✓ Medir la generalización a problemas más largos que los del entrenamiento y reconocer sus límites.

15.1 La pregunta incómoda

Al terminar el Capítulo 13 el Nano-LLM escribía frases impecables: «los modelos de lenguaje predicen el siguiente token», «las redes neuronales tienen capas de neuronas». Pero también vimos que esas frases están, letra por letra, en su texto de entrenamiento. Así que la pregunta es inevitable: ¿qué ha aprendido realmente?

Piensa en dos alumnos que preparan un examen de matemáticas con la misma colección de problemas resueltos. El primero se aprende las soluciones de memoria; el segundo entiende cómo se resuelven. Con los problemas de la colección, los dos sacan un 10. La diferencia solo aparece cuando el profesor pone un problema nuevo: entonces el primero no sabe por dónde empezar. Es la distinción entre aprender y memorizar de la sección 7.1, y para detectarla hace falta exactamente lo mismo que hace el profesor: examinar con casos que el alumno no ha visto.

En este capítulo usaremos la palabra **razonar** con un sentido muy concreto y comprobable: un modelo razona sobre una tarea cuando resuelve correctamente casos que no pudo ver durante el entrenamiento aplicando un procedimiento, y lo verificamos con datos apartados de antemano. No hablamos de consciencia ni de razonamiento humano general: cada modelo de este capítulo solo sabe hacer la tarea para la que se entrenó, y fuera de ella no sabe hacer nada.

El camino tiene cuatro etapas, cada una con su programa:

- **Diagnóstico (Programa RZ-1).** ¿Memoriza el Nano-LLM del Capítulo 13? Tres pruebas lo deciden.
- **Una regla (Programa RZ-2).** La misma red, con otros datos, aprende a sumar números que nunca ha visto.
- **Razonamiento en cadena (Programa RZ-3).** El modelo deduce encadenando hechos, siempre que escriba los pasos.
- **Razonamiento con frases (Programa RZ-4).** Lo mismo con frases al estilo del corpus del Programa 8.

> **ℹ️ Cuatro programas, un mismo modelo**
>
> Los cuatro programas usan el modelo del Programa 8, el GPT-2 en miniatura: embeddings aprendidos, 2 bloques Pre-LN con γ y β, atención causal multi-cabeza, FFN con GELU, LayerNorm final, salida con pesos compartidos, backward exacto y Adam con warm-up, coseno y recorte del gradiente. RZ-1 lo usa tal cual. RZ-2 a RZ-4 cambian lo que corresponde a cada tarea: los datos, dos cabezas en lugar de cuatro, una pérdida que solo puntúa lo que escribe el modelo y, en RZ-3 y RZ-4, una ayuda para leer los hechos (emb_prev). RZ-3 necesita además embeddings iniciales más grandes (15.4).

> Para que los experimentos tarden minutos y no horas, los cuatro procesan el mini-lote como un único tensor (B, T, D), igual que el Programa 8. Tiempos aproximados en un portátil: RZ-1, dos minutos (tras haber ejecutado el Programa 8); RZ-2, tres minutos; RZ-3, ocho minutos; RZ-4, unos treinta y cinco.

15.2 Diagnóstico: ¿memoriza el Nano-LLM? (Programa RZ-1)

El Programa RZ-1 somete al Nano-LLM a tres pruebas:

- **Prueba 1 — frases nunca vistas.** Aparta 10 de las 93 frases (una de cada nueve) como conjunto de validación (7.5), entrena el mismo modelo con las otras 83 y mide la pérdida en las dos partes a lo largo del entrenamiento.
- **Prueba 2 — una línea base sin red neuronal.** Un contador de n-gramas predice cada carácter contando qué siguió a los n − 1 caracteres anteriores en el texto de entrenamiento. Si el transformer no lo supera en frases nuevas, no ha aprendido nada que un contador no sepa.
- **Prueba 3 — copia literal.** Con el modelo del Programa 8 (cargado de `nano_llm_pesos.npz`) genera 10 textos por semilla y mide qué parte del texto generado es una copia literal de algún trozo del corpus de 20 caracteres o más, cuántas líneas generadas son idénticas a una frase del corpus y cuántas palabras inventa. Dos de las semillas, «el gato » y «mañana la », no aparecen en el corpus.

El modelo, los hiperparámetros y el optimizador son exactamente los del Programa 8; sus funciones se copian con un único cambio: los parámetros llegan como argumento, `p`, porque la Prueba 3 usa a la vez el modelo nuevo y el del Programa 8. Se ha comprobado que, con los mismos pesos y el mismo lote, la pérdida y el gradiente coinciden con los del Programa 8 número a número: la diferencia es exactamente 0.

PROGRAMA RZ-1 — ¿El Nano-LLM memoriza o generaliza?

📄 `programa_RZ1_memoriza.py`

📄 Parte 1 de 4 — datos, partición y modelo

```
# ============================================================
# PROGRAMA RZ-1 — ¿El Nano-LLM memoriza o generaliza?
# Capítulo 15 — Diagnóstico riguroso del Programa 8
#
# Tres pruebas sobre el mismo corpus de 93 frases:
#   1. Apartamos 10 frases que el modelo NUNCA verá y medimos
#      la pérdida en ellas a lo largo del entrenamiento.
#   2. Comparamos con un contador de n-gramas (sin red
#      neuronal): ¿supera el transformer a contar letras?
#   3. Con el modelo del Programa 8 ya entrenado, medimos qué
#      parte del texto que genera es copia literal del corpus.
#
# Requisitos: programa_8_nano_llm.py en la misma carpeta (se
# lee su TEXTO) y haberlo ejecutado antes (nano_llm_pesos.npz).
# El modelo es el MISMO del Programa 8 (GPT-2 en miniatura):
# sus funciones se copian aquí con un único cambio, que los
# parámetros se pasan como argumento (p), porque la prueba 3
# usa a la vez el modelo nuevo y el del Programa 8.
# ============================================================
import re
from collections import defaultdict
import numpy as np

# -- Corpus: el TEXTO del Programa 8 -------------------------
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
TEXTO = fuente.split('TEXTO = """')[1].split('"""')[0]
VOCAB = sorted(set(TEXTO))
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))
FRASES = TEXTO.strip().split('\n')
```

```
# 1 de cada 9 frases va a VALIDACIÓN (Capítulo 7, 7.5)
VAL_IDX = list(range(4, len(FRASES), 9))
VAL = [FRASES[i] for i in VAL_IDX]
TRAIN = [f for i, f in enumerate(FRASES) if i not in VAL_IDX]
texto_train = '\n' + '\n'.join(TRAIN) + '\n'
DATOS = np.array([C2I[c] for c in texto_train])
print(f'Frases: {len(FRASES)} | entrenamiento: {len(TRAIN)} | validación: {len(VAL)}')
print('Ejemplos de frases de validación (el modelo no las verá):')
for f in VAL[:3]:
    print('   ', f)

# -- Hiperparámetros: idénticos al Programa 8 ----------------
CTX, D, H, D_FF, N_BLQ = 24, 48, 4, 192, 2
DH = D // H
LR, PASOS, BATCH, CLIP = 3e-3, 12000, 8, 1.0
STD_RES = 0.02 / np.sqrt(2 * N_BLQ)
np.random.seed(42)


def randn(shape, std=0.02):
    return np.random.randn(*shape) * std


def nuevos_params():
    """Los mismos tensores del Programa 8, con su misma inicialización."""
    p = {'wte': randn((V, D)), 'wpe': randn((CTX, D)),
         'lnf_g': np.ones(D), 'lnf_b': np.zeros(D)}
    for k in range(N_BLQ):
        p[f'ln1_g{k}'] = np.ones(D); p[f'ln1_b{k}'] = np.zeros(D)
        for n in 'qkv':
            p[f'W{n}{k}'] = randn((D, D)); p[f'b{n}{k}'] = np.zeros(D)
        p[f'Wo{k}'] = randn((D, D), STD_RES); p[f'bo{k}'] = np.zeros(D)
        p[f'ln2_g{k}'] = np.ones(D); p[f'ln2_b{k}'] = np.zeros(D)
        p[f'W1{k}'] = randn((D, D_FF)); p[f'b1{k}'] = np.zeros(D_FF)
        p[f'W2{k}'] = randn((D_FF, D), STD_RES); p[f'b2{k}'] = np.zeros(D)
    return p


MASCARA = np.triu(np.full((CTX, CTX), -1e9), k=1)
```

La segunda parte son las funciones del Programa 8, las mismas líneas con los parámetros como argumento `p`. La función `ventanas()` prepara la evaluación: cada frase se evalúa con su propio contexto, y el vector de pesos `w` marca qué posiciones cuentan para la pérdida.

📄 Parte 2 de 4 — forward, backward y evaluación por frases

```
# -- Las funciones del Programa 8, con los parámetros p como argumento --
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)


def ln_fwd(x, g, b, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)


def ln_bwd(dy, g, cache):
    xhat, inv = cache
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, (dy * xhat).sum((0, 1)), dy.sum((0, 1))


C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))


def gelu_bwd(x):
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)


def separar(x):   # (B, T, D) -> (B, H, T, DH)
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)
```

```
def unir(x):      # (B, H, T, DH) -> (B, T, D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)


def tmm(a, b):
    """Σ sobre lote y posiciones de aᵀ·b (gradiente de una capa lineal)."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])


# -- Forward por lotes: tok (B, T) -> logits (B, T, V) -------
def forward(p, tok):
    B, T = tok.shape
    X = p['wte'][tok] + p['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        a, c1 = ln_fwd(X, p[f'ln1_g{k}'], p[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ p[f'W{n}{k}'] + p[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA[:T, :T])
        att = unir(pesos @ Vm)
        X1 = X + att @ p[f'Wo{k}'] + p[f'bo{k}']
        m_, c2 = ln_fwd(X1, p[f'ln2_g{k}'], p[f'ln2_b{k}'])
        hp = m_ @ p[f'W1{k}'] + p[f'b1{k}']
        h = gelu(hp)
        X = X1 + h @ p[f'W2{k}'] + p[f'b2{k}']
        cache.append((a, c1, Q, K, Vm, pesos, att, m_, c2, hp, h))
    xf, cf = ln_fwd(X, p['lnf_g'], p['lnf_b'])
    return xf @ p['wte'].T, (xf, cf, cache)


def perdida(p, tok, tgt, w):
    probs = softmax(forward(p, tok)[0])
    B, T = tok.shape
    lp = np.log(probs[np.arange(B)[:, None], np.arange(T)[None, :], tgt] + 1e-10)
    return -(lp * w).sum() / w.sum()


# -- Backward (las mismas derivadas del Programa 8) -----------
def forward_backward(p, tok, tgt):
    B, T = tok.shape
    logits, (xf, cf, cache) = forward(p, tok)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -np.log(probs[bi, ti, tgt] + 1e-10).mean()
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dl = probs.copy(); dl[bi, ti, tgt] -= 1.0; dl /= B * T
    g['wte'] += tmm(dl, xf)
    dX, g['lnf_g'], g['lnf_b'] = ln_bwd(dl @ p['wte'], p['lnf_g'], cf)
    for k in reversed(range(N_BLQ)):
        a, c1, Q, K, Vm, pesos, att, m_, c2, hp, h = cache[k]
        g[f'W2{k}'] = tmm(h, dX); g[f'b2{k}'] = dX.sum((0, 1))
        dhp = (dX @ p[f'W2{k}'].T) * gelu_bwd(hp)
        g[f'W1{k}'] = tmm(m_, dhp); g[f'b1{k}'] = dhp.sum((0, 1))
        dx1, g[f'ln2_g{k}'], g[f'ln2_b{k}'] = ln_bwd(dhp @ p[f'W1{k}'].T, p[f'ln2_g{k}'], c2)
        dX1 = dX + dx1
        g[f'Wo{k}'] = tmm(att, dX1); g[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ p[f'Wo{k}'].T)
        dpesos = datt @ Vm.transpose(0, 1, 3, 2)
        dVm = pesos.transpose(0, 1, 3, 2) @ datt
        dsc = pesos * (dpesos - (dpesos * pesos).sum(-1, keepdims=True)) / np.sqrt(DH)
        da = np.zeros_like(a)
        for n, dn in zip('qkv', (unir(dsc @ K), unir(dsc.transpose(0, 1, 3, 2) @ Q),
unir(dVm))):
            g[f'W{n}{k}'] = tmm(a, dn); g[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ p[f'W{n}{k}'].T
        dxin, g[f'ln1_g{k}'], g[f'ln1_b{k}'] = ln_bwd(da, p[f'ln1_g{k}'], c1)
        dX = dX1 + dxin
    g['wpe'][:T] = dX.sum(0)
    np.add.at(g['wte'], tok, dX)
    return g, loss


def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * (t - t_warmup) / (t_total -
t_warmup)))


# -- Ventanas de evaluación: cada frase con su propio contexto --
def ventanas(frases):
    xs, ys, ws = [], [], []
    for f in frases:
        ids = [C2I[c] for c in '\n' + f + '\n']
        for s in range(0, len(ids) - 1, CTX):
            n = min(CTX, len(ids) - 1 - s)
```

```
            st = max(0, s + n - CTX)             # máximo contexto posible
            x, y = ids[st:st + CTX], ids[st + 1:st + CTX + 1]
            w = [0] * (s - st) + [1] * (len(y) - (s - st))
            pad = CTX - len(y)
            xs.append(x[:len(y)] + [0] * pad); ys.append(y + [0] * pad); ws.append(w + [0] *
pad)
    return np.array(xs), np.array(ys), np.array(ws, float)


XT, YT, WT = ventanas(TRAIN)
XV, YV, WV = ventanas(VAL)
```

La Prueba 1 es un entrenamiento normal que, cada 250 pasos, mide la pérdida en las frases de validación y recuerda el mejor momento: ahí es donde se habría detenido el early stopping de la sección 7.8.

📄 Parte 3 de 4 — Prueba 1: entrenamiento con validación

```
# ============================================================
# PRUEBA 1 — Curva de pérdida en frases vistas y no vistas
# ============================================================
print('\n=== PRUEBA 1: pérdida en entrenamiento y en validación ===')
print(f'Referencia: adivinar al azar = ln({V}) = {np.log(V):.3f}')
p = nuevos_params()
m = {k: np.zeros_like(v) for k, v in p.items()}
v = {k: np.zeros_like(x) for k, x in p.items()}
rng = np.random.RandomState(7)
mejor = (9.9, 0)
for paso in range(PASOS):
    ii = rng.randint(0, len(DATOS) - CTX - 1, BATCH)
    x = np.stack([DATOS[i:i + CTX] for i in ii]); y = np.stack([DATOS[i + 1:i + CTX + 1] for i
in ii])
    g, _ = forward_backward(p, x, y)
    norma = np.sqrt(sum((gg ** 2).sum() for gg in g.values()))
    if norma > CLIP:
        g = {k: gg * CLIP / norma for k, gg in g.items()}
    t, lr = paso + 1, lr_warmup_coseno(paso, LR, PASOS)
    for k in p:
        m[k] = 0.9 * m[k] + 0.1 * g[k]
        v[k] = 0.999 * v[k] + 0.001 * g[k] ** 2
        p[k] -= lr * (m[k] / (1 - 0.9 ** t)) / (np.sqrt(v[k] / (1 - 0.999 ** t)) + 1e-8)
    if paso % 250 == 0 or paso == PASOS - 1:
        lv = perdida(p, XV, YV, WV)
        if lv < mejor[0]:
            mejor = (lv, paso)
        if paso % 1000 == 0 or paso == PASOS - 1:
            lt = perdida(p, XT, YT, WT)
            print(f'Paso {paso:5d}: entrenamiento {lt:.3f} | validación {lv:.3f}')
print(f'Mejor validación: {mejor[0]:.3f} en el paso {mejor[1]} '
      f'(ahí habría parado el Early Stopping del Capítulo 7)')
```

La Prueba 2 construye contadores de n-gramas con suavizado de Witten-Bell (cuando un contexto largo no se ha visto, la probabilidad se apoya en el contexto más corto), y la Prueba 3 genera texto y busca copias literales.

📄 Parte 4 de 4 — Pruebas 2 y 3: n-gramas y copia literal

```
# ============================================================
# PRUEBA 2 — ¿Supera el transformer a un contador de n-gramas?
# ============================================================
def ngramas(n):
    """Modelo de n-gramas de caracteres con suavizado Witten-Bell."""
    cnt = [defaultdict(lambda: defaultdict(int)) for _ in range(n)]
    for i in range(len(texto_train)):
        for k in range(min(n, i + 1)):
            cnt[k][texto_train[i - k:i]][texto_train[i]] += 1

    def prob(ctx, c):
        q = 1.0 / V
        for k in range(min(n, len(ctx) + 1)):
            d = cnt[k].get(ctx[len(ctx) - k:] if k else '')
            if d:
                N, Tt = sum(d.values()), len(d)
                q = (d.get(c, 0) + Tt * q) / (N + Tt)
        return q
    return prob


def entropia_cruzada(prob, frases):
    tot = n = 0
```

```
    for f in frases:
        s = '\n' + f + '\n'
        for j in range(1, len(s)):
            tot -= np.log(prob(s[max(0, j - CTX):j], s[j])); n += 1
    return tot / n


print('\n=== PRUEBA 2: contadores de n-gramas (sin red neuronal) ===')
print('contexto   entrenamiento   validación')
for n in range(1, 7):
    pr = ngramas(n)
    print(f'{n - 1:5d} car. {entropia_cruzada(pr, TRAIN):12.3f} {entropia_cruzada(pr,
VAL):12.3f}')

# ============================================================
# PRUEBA 3 — ¿Cuánto del texto generado es copia literal?
# ============================================================
print('\n=== PRUEBA 3: texto generado por el Programa 8 (todo el corpus) ===')
p8 = dict(np.load('nano_llm_pesos.npz'))
PALABRAS = set(re.findall(r'[a-záéíóúñ]+', TEXTO))


def top_p(pr, tp=0.9):
    o = np.argsort(pr)[::-1]
    n = int(np.searchsorted(np.cumsum(pr[o]), tp)) + 1
    q = np.zeros_like(pr); q[o[:n]] = pr[o[:n]]
    return q / q.sum()


def generar_lote(semilla, n_chars, k, rng, tau=0.8):
    toks = np.array([[C2I[c] for c in semilla]] * k)
    for _ in range(n_chars):
        lg = forward(p8, toks[:, -CTX:])[0][:, -1]
        sig = [rng.choice(V, p=top_p(softmax(lg[i] / tau))) for i in range(k)]
        toks = np.concatenate([toks, np.array(sig)[:, None]], 1)
    return [''.join(I2C[t] for t in fila) for fila in toks]


def fraccion_copiada(s, L=20):
    """Fracción de caracteres dentro de algún trozo de >= L caracteres
    que aparece tal cual en el corpus."""
    cub = np.zeros(len(s), bool)
    for i in range(len(s) - L + 1):
        if s[i:i + L] in TEXTO:
            cub[i:i + L] = True
    return cub.mean()


rng = np.random.RandomState(2026)
print('semilla             copia(>=20 car.)  líneas idénticas  palabras inventadas')
for semilla in ['la inteligencia', 'un modelo ', 'el gato ', 'mañana la ']:
    textos = generar_lote(semilla, 150, 10, rng)
    copia = np.mean([fraccion_copiada(t) for t in textos])
    lineas = [l for t in textos for l in t[len(semilla):].split('\n')[1:-1]]
    identicas = sum(l in FRASES for l in lineas)
    palabras = [w for t in textos for w in re.findall(r'[a-záéíóúñ]+', t[len(semilla):])[:-1]]
    nuevas = sorted({w for w in palabras if w not in PALABRAS})
    print(f'{semilla!r:20s} {copia:10.0%} {identicas:10d}/{len(lineas):<3d} '
          f'{len(nuevas):8d} {", ".join(nuevas[:3])}')
print('\nEjemplo con una semilla que no está en el corpus:')
print(textos[0])
```

> Salida del programa (unos dos minutos):

```
Frases: 93 | entrenamiento: 83 | validación: 10
Ejemplos de frases de validación (el modelo no las verá):
    la atención permite ver todo el contexto a la vez.
    la función softmax convierte logits en probabilidades.
    ¡el aprendizaje nunca termina! sigue explorando.

=== PRUEBA 1: pérdida en entrenamiento y en validación ===
Referencia: adivinar al azar = ln(46) = 3.829
Paso     0: entrenamiento 3.827 | validación 3.825
Paso  1000: entrenamiento 1.378 | validación 1.934
Paso  2000: entrenamiento 0.823 | validación 2.188
Paso  3000: entrenamiento 0.519 | validación 2.613
Paso  4000: entrenamiento 0.403 | validación 3.091
Paso  5000: entrenamiento 0.368 | validación 3.497
Paso  6000: entrenamiento 0.319 | validación 3.534
Paso  7000: entrenamiento 0.282 | validación 3.708
Paso  8000: entrenamiento 0.251 | validación 3.967
Paso  9000: entrenamiento 0.236 | validación 4.017
Paso 10000: entrenamiento 0.228 | validación 4.050
Paso 11000: entrenamiento 0.216 | validación 4.118
Paso 11999: entrenamiento 0.212 | validación 4.175
Mejor validación: 1.923 en el paso 750 (ahí habría parado el Early Stopping del Capítulo 7)
```

```
=== PRUEBA 2: contadores de n-gramas (sin red neuronal) ===
contexto   entrenamiento   validación
    0 car.        2.945        2.964
    1 car.        2.128        2.263
    2 car.        1.540        1.874
    3 car.        0.946        1.691
    4 car.        0.572        1.702
    5 car.        0.376        1.762

=== PRUEBA 3: texto generado por el Programa 8 (todo el corpus) ===
semilla             copia(>=20 car.)  líneas idénticas  palabras inventadas
'la inteligencia'          100%         20/20          0
'un modelo '               100%         16/16          0
'el gato '                  81%         15/17         15 ativo, ente, entenece
'mañana la '                97%         18/20          0

Ejemplo con una semilla que no está en el corpus:
mañana la vez.
los transformers son la base de los llm modernos.
entrenar un modelo requiere muchos datos y cálculo.
el futuro pertenece a quienes comprenden la
```

Qué dicen las tres pruebas

Prueba 1. Al principio las dos pérdidas bajan juntas, pero muy pronto se separan: la mejor validación, 1.923, se alcanza en el paso 750 de 12 000, y a partir de ahí la pérdida de entrenamiento sigue bajando hasta 0.212 mientras la de validación sube hasta 4.175, **por encima** de la de adivinar al azar (ln 46 = 3.829). Traducido a perplejidad (18.2): e^0.212 ≈ 1.24 en el texto de entrenamiento frente a e^4.175 ≈ 65 en frases nuevas, unas 50 veces más. En frases que no ha visto, el modelo lo hace peor que si repartiera la probabilidad por igual entre los 46 caracteres: está tan seguro de las continuaciones que memorizó que, cuando la frase es otra, se equivoca con mucha confianza. Es, de libro, el «CASO 3 — memorización» del Programa EVAL-1 (18.2).

Prueba 2. Un simple contador que mira los 3 caracteres anteriores consigue 1.691 en las frases de validación: mejor que el mejor momento del transformer (1.923) y muchísimo mejor que el transformer al final del entrenamiento (4.175). Con más contexto el contador empeora en validación (1.702 con 4, 1.762 con 5): también él empieza a memorizar.

Prueba 3. Entre el 81 y el 100 % del texto generado está dentro de fragmentos copiados literalmente del corpus, y casi todas las líneas completas que genera (69 de 73) son frases del corpus. Con las semillas que no están en él, el modelo salta enseguida a una frase conocida: en el ejemplo, «mañana la» continúa con «vez.», el final de «la atención permite ver todo el contexto a la vez.», y a partir de ahí recita, en orden, las tres frases que la siguen en el corpus. Las únicas palabras inventadas («ativo», «ente», «entenece») aparecen con «el gato », justo cuando no hay ninguna frase que copiar.

El veredicto es claro: el Nano-LLM del Capítulo 13 **memoriza**. Y no es un defecto del programa, sino una consecuencia de los datos: con 5 KB de texto no hay ninguna regla general del español que extraer; lo eficiente es recordar. Los LLM reales se enfrentan al mismo riesgo a otra escala, y por eso se vigila la contaminación de los benchmarks (18.7): si las preguntas del examen estaban en el texto de entrenamiento, una buena nota no demuestra nada.

> El Programa RZ-1 aplica tres pasos del protocolo de evaluación de 18.9: dividir los datos antes de empezar (paso 1), vigilar la pérdida de validación durante el entrenamiento (paso 2) y evaluar cualitativamente el texto generado con varias semillas (paso 4). Añade una cuarta comprobación que el protocolo no incluye: compararse con una línea base sencilla. Los Programas RZ-2 a RZ-4 aplican el paso 5: un examen diseñado para la tarea.

15.3 La misma red aprende una regla: sumar (Programa RZ-2)

Si el problema son los datos, cambiemos los datos. El Programa RZ-2 entrena el mismo transformer con sumas de dos cifras escritas como texto, por ejemplo `47+85=231`. Sí, 231: el resultado se escribe **al revés**, empezando por las unidades, igual que cuando sumamos a mano. Así, cada cifra del resultado depende de las cifras de los sumandos y de lo que el propio modelo ya ha escrito (la llevada), nunca de algo que todavía no ha escrito.

Hay 10 000 sumas posibles (de 00+00 a 99+99). Antes de entrenar se apartan 2000 que el modelo no verá nunca, y se entrena con las otras 8000. Si después acierta las 2000 apartadas, no le basta con memorizar. Con un matiz: en 1563 de ellas (el 78 %) la suma conmutada (85+47 si la apartada es 47+85) sí está en el entrenamiento, y podría acertarlas aplicando solo la conmutatividad; pero acierta también las otras 437, que no tienen pareja: eso muestra que generaliza a combinaciones que nunca ha visto (qué mecanismo interno lo consigue es otra pregunta, que la exactitud sola no responde). La pérdida solo puntúa las tres cifras del resultado (el resto del texto, «47+85=», es el enunciado), y la atención usa dos cabezas (el Programa 8 usa cuatro).

PROGRAMA RZ-2 — El Nano-LLM aprende una regla: sumar

📄 `programa_RZ2_suma.py`

📄 Parte 1 de 3 — datos y parámetros

```
# ============================================================
# PROGRAMA RZ-2 — El Nano-LLM aprende una regla: sumar
# Capítulo 15 — De memorizar a generalizar
#
# El modelo del Programa 8 (GPT-2 en miniatura, NumPy puro)
# entrenado con sumas de dos cifras.
# Se examina con 2 000 sumas que NUNCA ha visto: si acierta,
# no le basta con memorizar: tiene que aplicar la regla.
#
# Cambios respecto al Programa 8:
#   1. Datos: sumas «47+85=» en lugar de frases. De las 10 000
#      sumas posibles, 2 000 se apartan para el examen.
#   2. Resultado escrito al revés (unidades primero), como se
#      suma a mano: cada cifra depende solo de lo ya escrito.
#   3. Atención de 2 cabezas (el Programa 8 usa 4).
#   4. Lotes vectorizados (B, T, D) en lugar de un bucle.
#   5. La pérdida solo cuenta las 3 cifras del resultado.
#   6. Todo lo demás es el Programa 8: bloques Pre-LN con γ y β,
#      GELU, LayerNorm final, pesos compartidos, inicialización de
#      GPT-2 y Adam con warmup + coseno y recorte del gradiente.
#
# Uso:  python programa_RZ2_suma.py [semilla] [n_entreno] [cabezas] [excluir]
#   n_entreno  sumas de entrenamiento (por defecto 8000)
#   cabezas    cabezas de atención (por defecto 2)
#   excluir    p. ej. 75: no se entrena con ninguna suma cuyas
#              unidades sean 7 y 5 (ni 5 y 7); el examen son esas
#              200 sumas. Comprueba si el modelo deduce un hecho
#              que nunca vio.
# Tiempo aproximado: unos 3 minutos.
# ============================================================
import sys
import numpy as np

# -- Datos: sumas de dos cifras ----------------------------
VOCAB = list('0123456789+=\n')
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))


def formatear(a, b):
    """47+85=132  ->  '47+85=231\\n'  (resultado al revés, 3 cifras)"""
    return f'{a:02d}+{b:02d}=' + str(a + b).zfill(3)[::-1] + '\n'


SEMILLA = int(sys.argv[1]) if len(sys.argv) > 1 else 42
N_ENTRENO = int(sys.argv[2]) if len(sys.argv) > 2 else 8000
H = int(sys.argv[3]) if len(sys.argv) > 3 else 2          # cabezas de atención
EXCLUIR = sys.argv[4] if len(sys.argv) > 4 else None
```

```
pares = [(a, b) for a in range(100) for b in range(100)]      # 10 000 sumas
np.random.RandomState(0).shuffle(pares)                       # partición fija
TEST, TRAIN = pares[:2000], pares[2000:2000 + N_ENTRENO]      # 2 000 nunca vistas
if EXCLUIR:
    u1, u2 = int(EXCLUIR[0]), int(EXCLUIR[1])
    prohibida = lambda a, b: sorted((a % 10, b % 10)) == sorted((u1, u2))
    TEST = [q for q in pares if prohibida(*q)]
    TRAIN = [q for q in pares if not prohibida(*q)][:N_ENTRENO]


def a_tensor(ps):
    S = np.array([[C2I[c] for c in formatear(a, b)] for a, b in ps])
    return S[:, :-1], S[:, 1:]                                # entrada, objetivo


X_TR, Y_TR = a_tensor(TRAIN)
X_TE, Y_TE = a_tensor(TEST)
PESO = np.zeros(9); PESO[5:8] = 1      # solo se puntúan las 3 cifras del resultado

# -- Hiperparámetros ----------------------------------------
CTX, D, D_FF, N_BLQ = 9, 48, 192, 2
DH = D // H                            # 24 dimensiones por cabeza (con H = 2)
LR, PASOS, BATCH, CLIP = 3e-3, 6000, 64, 1.0
np.random.seed(SEMILLA)

STD_RES = 0.02 / np.sqrt(2 * N_BLQ)    # Wo y W2, como en el Programa 8


def randn(shape, std=0.02):
    return np.random.randn(*shape) * std


params = {'wte': randn((V, D)), 'wpe': randn((CTX, D))}
params['lnf_g'] = np.ones(D); params['lnf_b'] = np.zeros(D)
for k in range(N_BLQ):
    params[f'ln1_g{k}'] = np.ones(D); params[f'ln1_b{k}'] = np.zeros(D)
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
    params[f'ln2_g{k}'] = np.ones(D); params[f'ln2_b{k}'] = np.zeros(D)
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)
print(f'Parámetros: {sum(p.size for p in params.values()):,}')
print(f'Semilla {SEMILLA} | cabezas {H} | sumas de entrenamiento: {len(TRAIN)} | '
      f'sumas de examen (no vistas): {len(TEST)}' + (f' | excluidas unidades {EXCLUIR[0]} y
{EXCLUIR[1]}' if EXCLUIR else ''))
MASCARA = np.triu(np.full((CTX, CTX), -1e9), k=1)
```

La segunda parte es el modelo del Programa 8 con dos cabezas: las mismas utilidades, el mismo forward y el mismo backward; separar() reparte las 48 dimensiones en 2 cabezas de 24 y unir() las vuelve a juntar. El backward tiene un detalle nuevo: la pérdida se multiplica por el peso w, que vale 1 solo en las cifras del resultado.

📄 Parte 2 de 3 — forward y backward con dos cabezas

```
# -- Utilidades (las del Programa 8) ------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)


def layer_norm_fwd(x, g, b, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)


def layer_norm_bwd(dy, g, cache):
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, (dy * xhat).sum(ejes), dy.sum(ejes)


C_GELU = np.sqrt(2 / np.pi)


def gelu(x):
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))
```

```
def gelu_bwd(x):
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)


def separar(x):        # (B,T,D) -> (B,H,T,DH)
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)


def unir(x):           # (B,H,T,DH) -> (B,T,D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)


def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])


# -- Forward (el del Programa 8) ----------------------------
def forward(tok):
    B, T = tok.shape
    X = params['wte'][tok] + params['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        Xin = X
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    return xf @ params['wte'].T, (xf, lnf), cache


# -- Forward + Backward (el del Programa 8, con la pérdida ponderada por w) --
def forward_backward(tok, tgt, w):
    B, T = tok.shape
    logits, (xf, lnf), cache = forward(tok)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -(np.log(probs[bi, ti, tgt] + 1e-10) * w).sum() / w.sum()
    g = {k: np.zeros_like(v) for k, v in params.items()}
    dlog = probs.copy()
    dlog[bi, ti, tgt] -= 1.0
    dlog *= (w / w.sum())[..., None]          # solo cuentan las posiciones con w = 1
    g['wte'] += tmm(dlog, xf)                 # wte como matriz de salida
    dX, g['lnf_g'], g['lnf_b'] = layer_norm_bwd(dlog @ params['wte'], params['lnf_g'], lnf)
    for k in reversed(range(N_BLQ)):
        c = cache[k]
        g[f'W2{k}'] = tmm(c['h'], dX)
        g[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        g[f'W1{k}'] = tmm(c['m'], dh_pre)
        g[f'b1{k}'] = dh_pre.sum((0, 1))
        dx1, g[f'ln2_g{k}'], g[f'ln2_b{k}'] = layer_norm_bwd(dh_pre @ params[f'W1{k}'].T,
params[f'ln2_g{k}'], c['ln2'])
        dX1 = dX + dx1
        g[f'Wo{k}'] = tmm(c['att'], dX1)
        g[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)
        dQ, dK, dV = unir(dscores @ c['K']), unir(dscores.transpose(0, 1, 3, 2) @ c['Q']),
unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            g[f'W{n}{k}'] = tmm(c['a'], dn)
            g[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, g[f'ln1_g{k}'], g[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    g['wpe'][:T] = dX.sum(0)
    np.add.at(g['wte'], tok, dX)              # wte también como embedding de entrada
```

```
    return g, loss
```

La tercera parte examina con decodificación greedy (14.2): para una pregunta de respuesta única no se muestrea, se elige siempre la cifra más probable. Después entrena y, al final, imprime a qué posiciones atiende cada cabeza.

📄 Parte 3 de 3 — examen, entrenamiento y análisis de la atención

```
# -- Examen: decodificación greedy de las 3 cifras ---------
def resolver(X):
    seq = X[:, :6].copy()                                   # 'aa+bb='
    for _ in range(3):
        logits, _, _ = forward(seq)
        seq = np.concatenate([seq, logits[:, -1].argmax(-1)[:, None]], 1)
    return seq[:, 6:9]


def exactitud(X, Y):
    return (resolver(X) == Y[:, 5:8]).all(1).mean()


def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup                  # +1: el paso 0 ya aprende
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * (t - t_warmup) / (t_total -
t_warmup)))


# -- Entrenamiento (Adam + recorte de gradiente) -----------
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
print('\nEntrenando...  (exactitud = suma entera correcta)')
for paso in range(PASOS):
    lr = lr_warmup_coseno(paso, LR, PASOS)
    idx = np.random.randint(0, len(X_TR), BATCH)
    g, loss = forward_backward(X_TR[idx], Y_TR[idx], np.tile(PESO, (BATCH, 1)))
    norma = np.sqrt(sum((x ** 2).sum() for x in g.values()))
    if norma > CLIP:
        g = {k: x * CLIP / norma for k, x in g.items()}
    t = paso + 1
    for k in params:
        m_adam[k] = 0.9 * m_adam[k] + 0.1 * g[k]
        v_adam[k] = 0.999 * v_adam[k] + 0.001 * g[k] ** 2
        params[k] -= lr * (m_adam[k] / (1 - 0.9 ** t)) / (np.sqrt(v_adam[k] / (1 - 0.999 **
t)) + 1e-8)
    if paso % 500 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida {loss:.4f} | exactitud entrenamiento '
              f'{exactitud(X_TR[:2000], Y_TR[:2000]):6.1%} | sumas NUNCA vistas
{exactitud(X_TE, Y_TE):6.1%}')

# -- Resultado ---------------------------------------------
print('\n=== EXAMEN CON SUMAS QUE EL MODELO NUNCA VIO ===')
res = resolver(X_TE[:10])
for (a, b), r in zip(TEST[:10], res):
    pred = int(''.join(I2C[i] for i in r)[::-1])
    print(f'{a:2d} + {b:2d} = {pred:3d}   {"✓" if pred == a + b else "✗ (correcto: " + str(a +
b) + ")"}')
print(f'\nAciertos en las {len(TEST)} sumas no vistas: {exactitud(X_TE, Y_TE):.1%}')

# -- ¿Cómo lo hace? A qué cifras mira cada paso ------------
_, _, cache = forward(X_TE[:, :9])
nombres = ['a(dec)', 'a(uni)', '+', 'b(dec)', 'b(uni)', '=', 'r(uni)', 'r(dec)', 'r(cen)']
print(f'\n=== ¿A QUÉ CIFRAS ATIENDE CADA CABEZA? (media sobre {len(TEST)} sumas) ===')
for k in range(N_BLQ):
    for q, que in [(5, 'unidades'), (6, 'decenas '), (7, 'centenas')]:
        txt = []
        for c in range(H):
            fila = cache[k]['pesos'][:, c, q].mean(0)
            top = np.argsort(fila)[::-1][:2]
            txt.append(f'cabeza {c}: ' + ' + '.join(f'{nombres[j]} {fila[j]:.2f}' for j in
top))
        print(f'Bloque {k}, al escribir las {que} -> ' + ' | '.join(txt))
```

> Salida del programa (unos tres minutos):

```
Parámetros: 57,696
Semilla 42 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000

Entrenando...  (exactitud = suma entera correcta)
Paso     0: pérdida 2.5840 | exactitud entrenamiento   0.0% | sumas NUNCA vistas   0.0%
Paso   500: pérdida 0.6589 | exactitud entrenamiento  35.9% | sumas NUNCA vistas  36.4%
Paso  1000: pérdida 0.0605 | exactitud entrenamiento  94.4% | sumas NUNCA vistas  94.0%
Paso  1500: pérdida 0.0434 | exactitud entrenamiento  86.3% | sumas NUNCA vistas  87.1%
```

```
Paso  2000: pérdida 0.0096 | exactitud entrenamiento  98.8% | sumas NUNCA vistas  98.5%
Paso  2500: pérdida 0.0099 | exactitud entrenamiento 100.0% | sumas NUNCA vistas  99.9%
Paso  3000: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  3500: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  4000: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  4500: pérdida 0.0000 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  5000: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  5500: pérdida 0.0000 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  5999: pérdida 0.0000 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%

=== EXAMEN CON SUMAS QUE EL MODELO NUNCA VIO ===
93 + 94 = 187   ✓
 8 + 98 = 106   ✓
23 + 98 = 121   ✓
59 +  6 =  65   ✓
23 + 43 =  66   ✓
82 + 25 = 107   ✓
55 +  6 =  61   ✓
64 + 51 = 115   ✓
26 + 70 =  96   ✓
34 + 97 = 131   ✓

Aciertos en las 2000 sumas no vistas: 100.0%

=== ¿A QUÉ CIFRAS ATIENDE CADA CABEZA? (media sobre 2000 sumas) ===
Bloque 0, al escribir las unidades -> cabeza 0: b(uni) 0.50 + a(uni) 0.48 | cabeza 1: b(uni)
0.56 + a(uni) 0.43
Bloque 0, al escribir las decenas  -> cabeza 0: b(dec) 0.39 + a(dec) 0.39 | cabeza 1: b(dec)
0.36 + a(dec) 0.33
Bloque 0, al escribir las centenas -> cabeza 0: a(dec) 0.34 + b(dec) 0.34 | cabeza 1: + 0.93 +
a(dec) 0.04
Bloque 1, al escribir las unidades -> cabeza 0: a(uni) 0.50 + = 0.49 | cabeza 1: a(uni) 0.67 +
= 0.25
Bloque 1, al escribir las decenas  -> cabeza 0: b(dec) 0.75 + + 0.15 | cabeza 1: b(uni) 0.42 +
b(dec) 0.28
Bloque 1, al escribir las centenas -> cabeza 0: r(dec) 0.19 + b(dec) 0.16 | cabeza 1: a(dec)
0.29 + = 0.21
```

Las dos columnas de exactitud suben juntas: en el paso 500, un 35.9 % en las sumas de entrenamiento y un 36.4 % en las nunca vistas; en el 1000, 94.4 % y 94.0 %; desde el 3000, el 100 % en ambas. Esa es la firma de una regla: lo que el modelo aprende en unas sumas le sirve igual en las demás. Una memoria, en cambio, acertaría las vistas y fallaría las nuevas.

Cuántos datos hacen falta

¿Por qué aquí aparece una regla y con las frases no? Repitamos el experimento con menos sumas de entrenamiento (el número es el segundo argumento: `python programa_RZ2_suma.py 42 250`):

Sumas de entrenamiento	250	500	1000	2000	8000
Exactitud en entrenamiento	100 %	100 %	100 %	100 %	100 %
Exactitud en sumas no vistas	**58.8 %**	99.4 %	100 %	100 %	100 %

Con 250 sumas, la misma red, con los mismos hiperparámetros y el mismo 100 % en lo que ha visto, acierta solo un 58.8 % de las nuevas: ha extraído una parte de la regla y ha memorizado el resto. Con 500 sumas ya acierta el 99.4 %, y con 1000, el 100 %: memorizar se ha vuelto más difícil que aprender a sumar. Mismo modelo, misma exactitud en entrenamiento, comportamiento distinto: lo decide la relación entre los datos y la capacidad del modelo, exactamente como en el Nano-LLM del Capítulo 13.

Una cabeza o dos, y las mesetas

¿Importa el número de cabezas? Con 8000 sumas y seis semillas aleatorias distintas (el tercer argumento es el número de cabezas):

	42	1	2	3	4	5	Aprenden
1 cabeza	98.0 %	100 %	100 %	100 %	8.0 %	8.8 %	**4 de 6**

	42	1	2	3	4	5	Aprenden
2 cabezas	100 %	100 %	100 %	100 %	100 %	100 %	**6 de 6**

Cuando un modelo falla, falla también en las sumas de entrenamiento (≈ 10 %): no está memorizando, está atascado. Es una meseta (plateau): un tramo del entrenamiento en el que la pérdida apenas baja y del que algunas inicializaciones no consiguen salir en los 6000 pasos. Con dos cabezas aprenden las seis semillas; con una, dos de las seis se quedan atascadas. Pero dos cabezas no lo garantizan: en el experimento de la sección siguiente, con otros datos, dos semillas con dos cabezas se quedan también en la meseta. Por eso el Programa RZ-4 incluirá un reinicio automático.

¿Cómo suma? Leer la atención

La última parte de la salida es una pequeña ventana al interior del modelo (19.8). Para cada cifra del resultado, muestra a qué posiciones atiende cada cabeza, en media sobre las 2000 sumas del examen:

- Al escribir las unidades, las dos cabezas del bloque 0 miran las unidades de los dos sumandos, b(uni) y a(uni), casi a partes iguales. Es lo que haríamos a mano.
- Al escribir las decenas, el bloque 0 reparte la atención entre las decenas de los dos sumandos, a(dec) y b(dec). En el bloque 1, una cabeza mira b(dec) (0.75) y la otra, sobre todo, las unidades del segundo sumando, b(uni) (0.42).
- Al escribir las centenas, el bloque 0 vuelve a mirar las decenas de los dos sumandos; una de sus cabezas mira casi solo el signo + (0.93), que no aporta ninguna cifra: en esa posición no parece hacer nada útil.

¿Por qué mirar b(uni) para las decenas? Porque la llevada está en dos cifras: la de las unidades del resultado, que el modelo acaba de escribir, y la de las unidades del segundo sumando. Hay llevada exactamente cuando r(uni) es menor que b(uni). En 47 + 85, r(uni) = 2 es menor que b(uni) = 5: hay llevada. En 41 + 23, r(uni) = 4 no es menor que 3: no la hay. La cifra r(uni) no hace falta buscarla: es el carácter de la posición desde la que se escribe la decena, así que ya está en su vía residual; basta con traer b(uni), y eso hace la segunda cabeza del bloque 1. Conviene leerlo como una interpretación plausible de los pesos de atención, no como una demostración: los pesos dicen dónde mira el modelo, no qué calcula con lo que ve. Comprobarlo de verdad (por ejemplo, alterando b(uni) y viendo si cambia la decena) es una buena ampliación del Ejercicio 15.2.

Un límite, y una sorpresa

¿Puede el modelo deducir un hecho que nunca vio? El cuarto argumento del programa excluye del entrenamiento todas las sumas cuyas unidades sean 7 y 5, en cualquier orden (`python programa_RZ2_suma.py <semilla> 8000 2 75`). El examen son precisamente esas 200 sumas: el modelo nunca ha visto que 7 + 5 = 12 en las unidades.

Semilla	Entrenamiento	200 sumas «7 y 5»	Comportamiento
1	100 %	**100 %**	Deduce 7 + 5 = 12 sin haberlo visto nunca
4	100 %	**100 %**	Ídem
5	100 %	**100 %**	Ídem
7	100 %	0 %	Siempre una unidad menos (95 + 47 = 141)
42	10.3 %	10.0 %	No salió de la meseta (no cuenta)
2	9.6 %	0 %	Ídem

De los cuatro modelos que aprendieron a sumar, tres dedujeron por completo un hecho que nunca vieron, y el otro no: la semilla 7 acierta todas las sumas de entrenamiento y falla las 200 del examen, siempre por una unidad. Las semillas 42 y 2 no salieron de la meseta. La red puede construir una representación interna de las cifras que le permite completar combinaciones ausentes, pero que lo consiga depende de la semilla aleatoria. Es un ejemplo excelente de por qué hay que repetir los experimentos con varias semillas antes de sacar conclusiones: con dos semillas cualesquiera se puede llegar a la conclusión contraria.

> ℹ️ **Para saber más**
>
> Escribir el resultado al revés es una idea estudiada en investigación: Lee et al. (2023), «Teaching Arithmetic to Small Transformers», muestran que el formato de los datos (invertir el resultado, o escribir pasos intermedios) cambia drásticamente cuántos ejemplos necesita un transformer pequeño para aprender a sumar. En el Ejercicio 15.2 comprobarás qué ocurre con nuestro modelo si el resultado se escribe en orden natural.

15.4 Pensar en voz alta: razonar paso a paso (Programa RZ-3)

Sumar es una regla fija. Razonar suele ser otra cosa: combinar hechos que cambian de un problema a otro. El Programa RZ-3 plantea problemas de deducción con letras. Cada problema da 12 hechos del tipo `qm`, que significa «q es mayor que m», y una pregunta, «¿q > x?». Para responder hay que encadenar hechos: si q > m, m > c y c > x, entonces q > x. Los hechos forman tres cadenas separadas; la pregunta une el principio de la primera con el final de la primera (respuesta SÍ) o con el final de otra cadena (respuesta NO). Aquí NO significa «no se deduce de estos hechos»: las cadenas no están conectadas. En un orden real, q podría ser mayor que x por razones que el problema no da.

Lo importante es que **las letras se sortean en cada problema**: la «q» de un problema no tiene nada que ver con la «q» del siguiente. No hay ningún hecho que memorizar. La única forma de acertar es leer los hechos dados y encadenarlos.

El programa permite tres experimentos, y la comparación entre ellos es el corazón de esta sección:

- **Paso a paso.** El modelo escribe la cadena antes de responder: `q m c x S`. Cada letra que escribe es un paso del razonamiento.
- **Directo.** El modelo debe responder S o N sin escribir nada más.
- **Sin `emb_prev`.** Paso a paso, pero sin una ayuda que explicaremos enseguida.

Además, el modelo se entrena **solo** con deducciones de 1 a 4 pasos y se examina con 1 a 10 pasos: de 5 a 10, nunca ha visto nada parecido.

PROGRAMA RZ-3 — El Nano-LLM razona paso a paso

📄 `programa_RZ3_deduce_letras.py`

📄 Parte 1 de 3 — la tarea y los parámetros

```
# ============================================================
# PROGRAMA RZ-3 — El Nano-LLM razona paso a paso
# Capítulo 15 — Deducir con hechos que nunca ha visto
#
# El modelo del Programa 8 (GPT-2 en miniatura, NumPy puro),
# como en RZ-2, aprende a DEDUCIR a partir de hechos que cambian
# en cada problema.
#
#   Hechos:   q>m, m>c, c>x, ...   («q es mayor que m»)
#   Pregunta: ¿q>x?
#   Razonamiento escrito por el modelo:  q > m > c > x  ⇒ SÍ
#
```

```
# Cambios respecto al Programa RZ-2:
#   1. Tarea de deducción: 12 hechos con letras al azar; la
#      respuesta exige encadenar 1, 2, 3… hechos (transitividad).
#      Como las letras cambian en cada problema, no hay nada que
#      memorizar: solo sirve razonar sobre los hechos dados.
#   2. Cadena de razonamiento: el modelo escribe cada paso antes
#      de responder (modo 'paso_a_paso'). En modo 'directo' debe
#      responder sí/no sin escribir pasos.
#   3. Embedding del carácter anterior (emb_prev): cada posición
#      sabe qué letra la precede, así un hecho «cx» se puede
#      buscar por su primera letra con una sola atención.
#   4. Se entrena SOLO con deducciones de 1 a 4 pasos y se
#      examina con 1 a 10 pasos: de 5 a 10 nunca los ha visto.
#   5. Los embeddings (wte, wpe, emb_prev) empiezan con desviación
#      EMB_STD = 0.5 en lugar del 0.02 del Programa 8: con 0.02 el
#      modelo no sale de la meseta en esta tarea (sección 15.4).
#
# Uso:  python programa_RZ3_deduce_letras.py [modo] [emb_prev]
#   modo      paso_a_paso (por defecto) o directo
#   emb_prev  si (por defecto) o no
# Los tres experimentos del capítulo:
#   python programa_RZ3_deduce_letras.py                  -> aprende
#   python programa_RZ3_deduce_letras.py directo          -> no aprende
#   python programa_RZ3_deduce_letras.py paso_a_paso no   -> no aprende
# Tiempo aproximado: unos 8 minutos en un portátil.
# ============================================================
import sys
import numpy as np

MODO = sys.argv[1] if len(sys.argv) > 1 else 'paso_a_paso'     # o 'directo'
USAR_EMB_PREV = (sys.argv[2] if len(sys.argv) > 2 else 'si') == 'si'
N_HECHOS, PASOS_MAX_ENTRENO = 12, 4

# -- Vocabulario ---------------------------------------------
LETRAS = 'abcdefghijklmnopqrstuvwxyz'
VOCAB = ['.'] + list(LETRAS) + [',', '|', '?', ':', 'S', 'N', '\n']
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))


def problema(L, rng, relleno=0):
    """Tres cadenas disjuntas de hechos; la primera tiene L pasos.
    Pregunta: ¿inicio de la cadena 1 > t?  t es el final de la
    cadena 1 (respuesta S) o el final de otra cadena (N)."""
    r1 = rng.randint(1, N_HECHOS - L); r2 = N_HECHOS - L - r1
    n = list(rng.permutation(list(LETRAS))[:N_HECHOS + 3])
    c1, c2, c3 = n[:L + 1], n[L + 1:L + 2 + r1], n[L + 2 + r1:]
    hechos = [a + b for c in (c1, c2, c3) for a, b in zip(c[:-1], c[1:])]
    rng.shuffle(hechos)
    si = rng.rand() < 0.5
    t = c1[-1] if si else (c2[-1] if rng.rand() < 0.5 else c3[-1])
    prompt = '.' * relleno + ','.join(hechos) + '|' + c1[0] + '?' + t + ':'
    razonamiento = ''.join(c1) if MODO == 'paso_a_paso' else ''
    return prompt, razonamiento + ('S' if si else 'N')


# -- Hiperparámetros -----------------------------------------
CTX, D, D_FF, N_BLQ, H = 56, 48, 192, 2, 2
DH = D // H
LR, PASOS, BATCH, CLIP, RELLENO_MAX = 3e-3, 2000, 64, 1.0, 8
EMB_STD = 0.5          # desviación inicial de los embeddings (Programa 8: 0.02)
np.random.seed(42)


STD_RES = 0.02 / np.sqrt(2 * N_BLQ)


def randn(shape, std=0.02):
    return np.random.randn(*shape) * std


params = {'wte': randn((V, D), EMB_STD), 'wpe': randn((CTX, D), EMB_STD),
          'emb_prev': randn((V, D), EMB_STD)}
params['lnf_g'] = np.ones(D); params['lnf_b'] = np.zeros(D)
for k in range(N_BLQ):
    params[f'ln1_g{k}'] = np.ones(D); params[f'ln1_b{k}'] = np.zeros(D)
    for n in 'qkv':
        params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
    params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
    params[f'ln2_g{k}'] = np.ones(D); params[f'ln2_b{k}'] = np.zeros(D)
    params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
    params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)
if not USAR_EMB_PREV:
    params['emb_prev'][:] = 0.0          # existe pero no se usa ni se entrena
n_par = sum(p.size for k, p in params.items() if USAR_EMB_PREV or k != 'emb_prev')
```

```
print(f'Modo: {MODO} | emb_prev: {"sí" if USAR_EMB_PREV else "no"} | parámetros: {n_par:,}')
MASCARA = np.triu(np.full((CTX, CTX), -1e9), k=1)
```

La segunda parte es, de nuevo, el modelo de RZ-2 con dos cabezas, con una sola novedad: las líneas de emb_prev. En el forward, cada posición recibe además el embedding del carácter que tiene delante; en el backward, np.add.at le devuelve su gradiente. Hay un cambio más, en la primera parte: los embeddings (wte, wpe y emb_prev) empiezan con desviación EMB_STD = 0.5, no con el 0.02 del Programa 8. Con 0.02 el modelo no salía de la meseta en esta tarea: escribía siempre la misma letra, también con 4000 pasos, con otra semilla y con tasas de aprendizaje de 0.001 y 0.0003. Con 0.5 aprende en unos 750 pasos. La inicialización también es un hiperparámetro, y a veces el que decide. Por eso la pérdida empieza tan alta (14.2): con embeddings mayores, los logits de partida, calculados con la misma tabla wte, son grandes, y el modelo empieza muy seguro de respuestas al azar.

📄 Parte 2 de 3 — utilidades, forward y backward (con emb_prev)

```
# -- Utilidades (las del Programa 8) -------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)


def layer_norm_fwd(x, g, b, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)


def layer_norm_bwd(dy, g, cache):
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, (dy * xhat).sum(ejes), dy.sum(ejes)


C_GELU = np.sqrt(2 / np.pi)


def gelu(x):
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))


def gelu_bwd(x):
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)


def separar(x):        # (B,T,D) -> (B,H,T,DH)
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)


def unir(x):           # (B,H,T,DH) -> (B,T,D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)


def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])


# -- Forward (el del Programa 8) -----------------------------
def forward(tok):
    B, T = tok.shape
    X = params['wte'][tok] + params['wpe'][:T]
    if USAR_EMB_PREV:
        X[:, 1:T] += params['emb_prev'][tok[:, :-1]]      # quién va delante de mí
    cache = []
    for k in range(N_BLQ):
        Xin = X
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA[:T, :T])
        att = unir(pesos @ Vm)
```

```
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    return xf @ params['wte'].T, (xf, lnf), cache


# -- Forward + Backward (el del Programa 8, con la pérdida ponderada por w) --
def forward_backward(tok, tgt, w):
    B, T = tok.shape
    logits, (xf, lnf), cache = forward(tok)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -(np.log(probs[bi, ti, tgt] + 1e-10) * w).sum() / w.sum()
    g = {k: np.zeros_like(v) for k, v in params.items()}
    dlog = probs.copy()
    dlog[bi, ti, tgt] -= 1.0
    dlog *= (w / w.sum())[..., None]          # solo cuentan las posiciones con w = 1
    g['wte'] += tmm(dlog, xf)                 # wte como matriz de salida
    dX, g['lnf_g'], g['lnf_b'] = layer_norm_bwd(dlog @ params['wte'], params['lnf_g'], lnf)
    for k in reversed(range(N_BLQ)):
        c = cache[k]
        g[f'W2{k}'] = tmm(c['h'], dX)
        g[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        g[f'W1{k}'] = tmm(c['m'], dh_pre)
        g[f'b1{k}'] = dh_pre.sum((0, 1))
        dx1, g[f'ln2_g{k}'], g[f'ln2_b{k}'] = layer_norm_bwd(dh_pre @ params[f'W1{k}'].T,
params[f'ln2_g{k}'], c['ln2'])
        dX1 = dX + dx1
        g[f'Wo{k}'] = tmm(c['att'], dX1)
        g[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)
        dQ, dK, dV = unir(dscores @ c['K']), unir(dscores.transpose(0, 1, 3, 2) @ c['Q']),
unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
            g[f'W{n}{k}'] = tmm(c['a'], dn)
            g[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, g[f'ln1_g{k}'], g[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    g['wpe'][:T] = dX.sum(0)
    np.add.at(g['wte'], tok, dX)              # wte también como embedding de entrada
    if USAR_EMB_PREV:
        np.add.at(g['emb_prev'], tok[:, :-1], dX[:, 1:])
    return g, loss
```

La tercera parte genera los mini-lotes (con un relleno de puntos al azar al principio, para que las cadenas cortas ocupen también las posiciones que usarán las largas), resuelve con greedy y examina con 200 problemas nuevos por cada número de pasos.

📄 Parte 3 de 3 — entrenamiento, examen y ejemplos

```
# -- Lotes de entrenamiento (deducciones de 1 a 4 pasos) ---
def lote(rng):
    X = np.zeros((BATCH, CTX - 1), int); Y = np.zeros_like(X); W = np.zeros(X.shape)
    for i in range(BATCH):
        p, r = problema(rng.randint(1, PASOS_MAX_ENTRENO + 1), rng, rng.randint(0, RELLENO_MAX
+ 1))
        s = [C2I[c] for c in p + r + '\n']
        s += [0] * (CTX - len(s))
        X[i], Y[i] = s[:-1], s[1:]
        W[i, len(p) - 1:len(p) + len(r)] = 1            # solo lo que escribe el modelo
    return X, Y, W


# -- Generación greedy hasta S o N -------------------------
def resolver(prompts, max_chars=12):
    seqs = [[C2I[c] for c in p] for p in prompts]
    salida = ['' for _ in prompts]
    for _ in range(max_chars):
        T = max(len(s) for s in seqs)
        X = np.array([s + [0] * (T - len(s)) for s in seqs])
```

```
        logits, _, _ = forward(X)
        for i, s in enumerate(seqs):
            if salida[i][-1:] in ('S', 'N'):
                continue
            nx = int(logits[i, len(s) - 1].argmax())
            s.append(nx); salida[i] += I2C[nx]
    return salida


def examen(L, n=200):
    rng = np.random.RandomState(1000 + L)
    probs = [problema(L, rng) for _ in range(n)]
    sal = resolver([p for p, _ in probs])
    resp = np.mean([s[-1:] == r[-1] for s, (_, r) in zip(sal, probs)])
    todo = np.mean([s == r for s, (_, r) in zip(sal, probs)])
    return resp, todo


# -- Entrenamiento ------------------------------------------
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * (t - t_warmup) / (t_total -
t_warmup)))


m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
rng = np.random.RandomState(7)
print(f'\nEntrenando con deducciones de 1 a {PASOS_MAX_ENTRENO} pasos...')
for paso in range(PASOS):
    X, Y, W = lote(rng)
    g, loss = forward_backward(X, Y, W)
    norma = np.sqrt(sum((x ** 2).sum() for x in g.values()))
    if norma > CLIP:
        g = {k: x * CLIP / norma for k, x in g.items()}
    t, lr = paso + 1, lr_warmup_coseno(paso, LR, PASOS)
    for k in params:
        m_adam[k] = 0.9 * m_adam[k] + 0.1 * g[k]
        v_adam[k] = 0.999 * v_adam[k] + 0.001 * g[k] ** 2
        params[k] -= lr * (m_adam[k] / (1 - 0.9 ** t)) / (np.sqrt(v_adam[k] / (1 - 0.999 **
t)) + 1e-8)
    if paso % 250 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida {loss:.4f} | aciertos con 3 pasos {examen(3, 50)
[0]:5.0%}'
              f' | con 7 pasos (nunca vistos) {examen(7, 50)[0]:5.0%}')

# -- Examen final -------------------------------------------
print('\n=== EXAMEN: 200 problemas nuevos por cada número de pasos ===')
print('pasos  respuesta  razonamiento completo correcto')
for L in range(1, N_HECHOS - 1):
    r, t = examen(L)
    nota = '' if L <= PASOS_MAX_ENTRENO else '  ← nunca visto al entrenar'
    print(f'{L:5d}  {r:9.1%}  {t:9.1%}{nota}')

# -- Tres ejemplos legibles ---------------------------------
print('\n=== EJEMPLOS ===')
rng = np.random.RandomState(2026)
for L in (2, 6, 9):
    p, r = problema(L, rng)
    sal = resolver([p])[0]
    hechos = p.split('|')[0].split(',')
    s, t = p.split('|')[1][0], p.split('|')[1][2]
    print('Hechos:', ', '.join(f'{h[0]}>{h[1]}' for h in hechos))
    print(f'¿{s}>{t}?  Modelo: {" > ".join(sal[:-1]) + "  ⇒  " if sal[:-1] else ""}'
          f'{"SÍ" if sal[-1:] == "S" else "NO"}   (correcto: {"SÍ" if r[-1] == "S" else
"NO"})\n')
```

> Salida del programa (modo paso a paso; unos ocho minutos):

```
Modo: paso_a_paso | emb_prev: sí | parámetros: 62,592

Entrenando con deducciones de 1 a 4 pasos...
Paso     0: pérdida 14.1593 | aciertos con 3 pasos    0% | con 7 pasos (nunca vistos)    0%
Paso   250: pérdida 2.3249 | aciertos con 3 pasos   44% | con 7 pasos (nunca vistos)   46%
Paso   500: pérdida 0.4703 | aciertos con 3 pasos   66% | con 7 pasos (nunca vistos)   60%
Paso   750: pérdida 0.0013 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  1000: pérdida 0.0031 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  1250: pérdida 0.0008 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  1500: pérdida 0.0003 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  1750: pérdida 0.0006 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  1999: pérdida 0.0003 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%

=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1     100.0%     100.0%
```

```
    2     100.0%     100.0%
    3     100.0%     100.0%
    4     100.0%     100.0%
    5     100.0%     100.0%  ← nunca visto al entrenar
    6     100.0%     100.0%  ← nunca visto al entrenar
    7     100.0%     100.0%  ← nunca visto al entrenar
    8     100.0%     100.0%  ← nunca visto al entrenar
    9      99.5%      99.0%  ← nunca visto al entrenar
   10     100.0%      99.5%  ← nunca visto al entrenar

=== EJEMPLOS ===
Hechos: d>q, v>r, b>e, q>i, j>a, s>o, p>b, e>s, z>p, o>h, h>j, r>k
¿v>k?   Modelo: v > r > k  ⇒  SÍ   (correcto: SÍ)

Hechos: g>s, o>u, z>m, s>x, q>t, l>k, m>l, u>z, x>h, n>o, y>g, h>i
¿n>k?   Modelo: n > o > u > z > m > l > k  ⇒  SÍ   (correcto: SÍ)

Hechos: k>x, p>n, h>f, e>g, w>h, r>v, s>i, v>y, g>k, x>m, i>e, m>r
¿s>n?   Modelo: s > i > e > g > k > x > m > r > v > y  ⇒  NO   (correcto: NO)
```

Y estas son las últimas líneas de los otros dos experimentos:

```
> python programa_RZ3_deduce_letras.py directo
Modo: directo | emb_prev: sí | parámetros: 62,592

...
=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1      47.5%      47.5%
    2      50.0%      50.0%
    3      53.0%      53.0%
    4      54.5%      54.5%
    5      49.0%      49.0%  ← nunca visto al entrenar
    6      46.5%      46.5%  ← nunca visto al entrenar
    7      50.0%      50.0%  ← nunca visto al entrenar
    8      51.0%      51.0%  ← nunca visto al entrenar
    9      47.5%      47.5%  ← nunca visto al entrenar
   10      51.5%      51.5%  ← nunca visto al entrenar


> python programa_RZ3_deduce_letras.py paso_a_paso no
Modo: paso_a_paso | emb_prev: no | parámetros: 60,960

...
=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1      55.5%       4.0%
    2      42.0%       0.0%
    3      51.5%       0.0%
    4      49.5%       0.0%
    5      48.5%       0.0%  ← nunca visto al entrenar
    6      52.0%       0.0%  ← nunca visto al entrenar
    7      47.0%       0.0%  ← nunca visto al entrenar
    8      49.0%       0.0%  ← nunca visto al entrenar
    9      49.5%       0.0%  ← nunca visto al entrenar
   10      57.0%       0.0%  ← nunca visto al entrenar
```

Por qué escribir los pasos lo cambia todo

Los resultados no dejan lugar a dudas. Escribiendo los pasos, el modelo aprende en unos 750 pasos de entrenamiento y acierta prácticamente todo incluso con 10 pasos de deducción (el 100 % de las respuestas; el 99.5 % con 9), aunque solo se entrenó con 1 a 4. En modo directo se queda clavado en el azar desde el principio. Su pérdida final lo delata: vale lo mismo que ln(2)/2. El modelo predice siempre bien el salto de línea final (pérdida 0) y lanza una moneda para el S/N (pérdida ln 2); la media de las dos posiciones es ln(2)/2. La propia pérdida dice que el modelo está echando a suertes la respuesta.

¿Por qué? Fíjate en lo que hace cada paso de la cadena. Para escribir la letra que sigue a «o» hay que buscar, entre los 12 hechos, el que empieza por «o» y copiar su segunda letra. Es **una sola búsqueda**, justo lo que sabe hacer una capa de atención: la query de la posición actual («busco la letra o») encuentra la key del hecho que empieza por «o». Responder de golpe a una pregunta de 7 pasos exigiría encadenar 7 búsquedas dentro de la

red, una detrás de otra, y nuestro modelo solo tiene 2 bloques. Escribiendo, en cambio, cada búsqueda queda fijada en el texto y la siguiente vuelta del bucle autorregresivo parte de ella. **La longitud del texto sustituye a la profundidad de la red.**

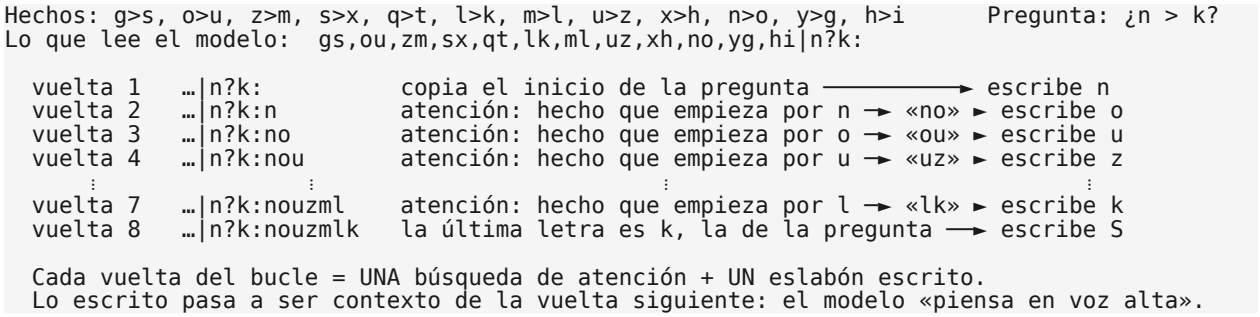


Diagrama 15.1 — Pensar en voz alta: cada vuelta del bucle autorregresivo hace una búsqueda y escribe un eslabón.

El tercer experimento explica la ayuda `emb_prev`. Para buscar «el hecho que empieza por o», la posición de la segunda letra de cada hecho tiene que «saber» qué letra va delante de ella. Un transformer profundo aprende a hacerlo por sí solo con un circuito de dos capas de atención llamado **cabeza de inducción** (induction head): una cabeza copia en cada posición la identidad del token anterior, y otra la usa para buscar. Olsson et al. (2022) mostraron que estos circuitos aparecen en los LLM y son la base del aprendizaje en contexto. Nuestro modelo de 2 bloques no lo consiguió en ninguna de las configuraciones probadas (sin emb_prev seguía en el azar tras 4000 pasos, también con 10 hechos en lugar de 12 o con una tasa de aprendizaje de 0.001), así que `emb_prev` se lo da hecho: suma a cada posición el embedding del carácter anterior. Con esa pieza, aprende en unos 750 pasos.

Por último, la **generalización por longitud**: entrenado con cadenas de 1 a 4 pasos, el modelo resuelve cadenas de hasta 10, que nunca ha visto. No ha memorizado «cadenas de 3 pasos»; ha aprendido el procedimiento de dar un paso, y el procedimiento se repite tantas veces como haga falta.

> ℹ️ **Cadenas de razonamiento**
>
> Es la misma idea que se descubrió en los LLM reales: Nye et al. (2021) los entrenaron para escribir sus cálculos intermedios en un «borrador» (scratchpad), y Wei et al. (2022) mostraron que basta con pedir a un LLM grande que razone paso a paso para que resuelva problemas de varios pasos que respondiendo de golpe falla. Lo llamaron cadena de razonamiento (chain of thought).

15.5 Razonar con frases del corpus (Programa RZ-4)

El último paso es acercar la tarea al lenguaje del Programa 8. En el Programa RZ-4 los hechos son frases: «la regularización dificulta el sobreajuste», «el sobreajuste limita la generalización». Los 40 conceptos y los 8 verbos están tomados del corpus del Capítulo 13, y se sortean en cada problema.

Hay tres novedades. La primera, los **tokens de palabra**: cada concepto («la regularización») y cada verbo es un token, como hace el tokenizador de un LLM real (8.2, 9.6). A nivel de carácter, 10 frases ocuparían unas 400 posiciones y el entrenamiento en NumPy sería decenas de veces más lento; con tokens de palabra caben en 78.

La segunda, **el razonamiento tiene signo**. Cuatro verbos favorecen («ayuda a», «estabiliza», «acelera», «mejora») y cuatro perjudican («dificulta», «limita», «complica», «ralentiza»). En cada paso el modelo debe acumular el efecto: perjudicar algo que perjudica equivale a favorecer. Si la regularización dificulta el sobreajuste y el sobreajuste

limita la generalización, la regularización favorece la generalización. Es una regla del mundo formal que define el generador (favorecer vale +1, perjudicar −1, y los signos se multiplican): en el mundo real, dos relaciones así no bastan para concluir la tercera, y aquí los sustantivos son etiquetas intercambiables.

La tercera, **tres respuestas**: SÍ, NO o NO SE PUEDE DEDUCIR, cuando la pregunta une conceptos de cadenas distintas.

Dos lecciones de diseño (o cómo fallar dos veces)

El programa que vas a leer no salió a la primera, y las dos correcciones enseñan tanto como el resultado final:

- **Escribir el verbo de cada paso.** En la primera versión el modelo escribía solo la cadena de conceptos y el efecto acumulado. Seguía bien la cadena, pero acertaba el SÍ/NO alrededor del 50 %: para calcular el signo tenía que buscar el verbo y combinarlo en un único paso. Escribiendo primero el verbo, cada paso vuelve a ser una sola operación: la misma lección de 15.4.
- **Formular la pregunta con forma de hecho.** Con la pregunta en un formato propio, el modelo aprendía con unas semillas y se atascaba con otras (en una prueba, las semillas 1, 101 y 201). La solución fue escribir «¿A verbo B?» internamente como «¿ A ? ¿ B verbo ?»: así, «B verbo ?» tiene la forma de un hecho, y comprobar al final si la cadena ha llegado a B es exactamente la misma búsqueda que dar un paso más. Desde entonces aprenden las seis semillas probadas.

PROGRAMA RZ-4 — El Nano-LLM razona con frases del corpus

📄 programa_RZ4_deduce_frases.py

📄 Parte 1 de 3 — vocabulario de palabras y la tarea

```
# ============================================================
# PROGRAMA RZ-4 — El Nano-LLM razona con frases del corpus
# Capítulo 15 — Deducción con frases al estilo del Programa 8
# El modelo del Programa 8 (GPT-2 en miniatura, NumPy puro),
# como en RZ-2 y RZ-3, deduce conclusiones a partir de frases
# al estilo del corpus de IA del Programa 8:
#
#   Hechos:   la regularización dificulta el sobreajuste.
#             el sobreajuste limita la generalización.
#             ... (más frases, algunas sin relación)
#   Pregunta: ¿la regularización ayuda a la generalización?
#   Razonamiento escrito por el modelo (cita cada hecho que usa):
#     la regularización dificulta el sobreajuste   [efecto acumulado: perjudica]
#     el sobreajuste limita la generalización      [efecto acumulado: favorece]
#     la pregunta dice «ayuda a» (favorece): coincide  ⇒  SÍ
#
# Cambios respecto al Programa RZ-3:
#   1. Frases en vez de letras: 40 conceptos y 8 verbos tomados
#      del corpus del Programa 8 (datos, sobreajuste, atención,
#      «dificulta», «limita», «acelera», «estabiliza»…).
#   2. Tokens de palabra en vez de caracteres: cada concepto
#      («la regularización») y cada verbo es un token, como hacen
#      los LLM reales con su tokenizador. A nivel de carácter, 10
#      frases ocuparían ~400 posiciones y el entrenamiento en NumPy
#      sería decenas de veces más lento.
#   3. Razonamiento con signo: unos verbos favorecen y otros
#      perjudican, así que en cada paso el modelo debe acumular el
#      efecto (perjudicar algo que perjudica = favorecer). En cada
#      paso escribe el verbo del hecho que usa, luego el efecto
#      acumulado y luego el concepto siguiente. Escribir el verbo
#      fue necesario: sin él, el modelo encontraba bien la cadena
#      de conceptos pero acertaba el efecto solo por azar, porque
#      tenía que buscar el verbo y combinarlo en un único paso.
#   4. Tres respuestas: SÍ, NO o NO SE PUEDE DEDUCIR (cuando la
#      pregunta une conceptos de cadenas distintas). La pregunta se
#      escribe internamente con forma de hecho («B verbo ?»), así que
#      comprobar si se ha llegado a B es la misma operación que dar
#      un paso más. Fue la clave: con la pregunta en otro formato, el
#      modelo seguía bien la cadena pero, al final, aprendía a
```

```
#      comprobar si había llegado a B solo con algunas semillas.
#   5. emb_prev2 además de emb_prev: cada posición sabe también
#      qué token va dos lugares antes. Así, en «A verbo B», la
#      posición de B «sabe» a la vez su sujeto (A) y su verbo, y
#      el modelo puede buscar el hecho por su sujeto con una sola
#      atención.
#   6. Los conceptos, los verbos y el orden de las frases se
#      sortean en cada problema: no hay nada que memorizar.
#   7. Se entrena SOLO con deducciones de 1 a 4 pasos y se
#      examina con 1 a 8: de 5 a 8 nunca los ha visto.
#   8. Reinicio de seguridad: si en el paso 3000 el modelo no acierta
#      el 80 % de las deducciones de 3 pasos (se quedó atascado en una
#      meseta), el programa vuelve a empezar con otra semilla.
# Uso: python programa_RZ4_deduce_frases.py [semilla]
# Tiempo aproximado: unos 35 minutos en un portátil.
# ============================================================
import sys
import numpy as np

SEMILLA = int(sys.argv[1]) if len(sys.argv) > 1 else 42
N_HECHOS, PASOS_MAX_ENTRENO = 10, 4

# -- Vocabulario de palabras (todas aparecen en el corpus del Programa 8) --
CONCEPTOS = ['los datos', 'el modelo', 'el entrenamiento', 'el sobreajuste', 'la
regularización',
             'la generalización', 'la atención', 'el embedding', 'la tokenización', 'la
pérdida',
             'el gradiente', 'la normalización', 'el ruido', 'la memoria', 'la precisión',
             'el error', 'la escala', 'la eficiencia', 'el costo', 'la incertidumbre',
             'la validación', 'el contexto', 'el vocabulario', 'la temperatura', 'la
variedad',
             'el sesgo', 'la privacidad', 'la interpretabilidad', 'el paralelismo', 'la
recompensa',
             'la exploración', 'la explotación', 'el aprendizaje', 'la inferencia',
             'el preentrenamiento', 'el ajuste fino', 'la simulación', 'la confianza',
             'la arquitectura', 'el optimizador']
FAVORECE = ['ayuda a', 'estabiliza', 'acelera', 'mejora']          # efecto +
PERJUDICA = ['dificulta', 'limita', 'complica', 'ralentiza']       # efecto −
VERBOS = FAVORECE + PERJUDICA
ESPECIALES = ['<pad>', '.', '¿', '?', '[+]', '[−]', 'SÍ', 'NO', 'NO SE PUEDE DEDUCIR']
VOCAB = ESPECIALES + CONCEPTOS + VERBOS
V = len(VOCAB)
W2I = {w: i for i, w in enumerate(VOCAB)}
RESPUESTAS = {W2I['SÍ'], W2I['NO'], W2I['NO SE PUEDE DEDUCIR']}


def signo(verbo):
    return +1 if verbo in FAVORECE else -1


def problema(L, rng, relleno=0):
    """Tres cadenas de hechos «A verbo B.»; la primera tiene L pasos.
    Pregunta: ¿inicio de la cadena 1 <verbo> t?  Si t es el final de la
    cadena 1, la respuesta depende del efecto acumulado; si t es el final
    de otra cadena, no se puede deducir."""
    r1 = rng.randint(1, N_HECHOS - L); r2 = N_HECHOS - L - r1
    c = [CONCEPTOS[i] for i in rng.permutation(len(CONCEPTOS))[:N_HECHOS + 3]]
    c1, c2, c3 = c[:L + 1], c[L + 1:L + 2 + r1], c[L + 2 + r1:]
    verbo = {}
    hechos = []
    for cad in (c1, c2, c3):
        for a, b in zip(cad[:-1], cad[1:]):
            v = VERBOS[rng.randint(len(VERBOS))]
            verbo[a] = v
            hechos.append([a, v, b, '.'])
    rng.shuffle(hechos)
    conectado = rng.rand() < 0.5
    t = c1[-1] if conectado else (c2[-1] if rng.rand() < 0.5 else c3[-1])
    vq = VERBOS[rng.randint(len(VERBOS))]
    # La pregunta «¿A verbo B?» se escribe como «¿ A ? ¿ B verbo ?»:
    #  - «B verbo ?» tiene la misma forma que un hecho («B verbo …»). Al llegar al
    #    final de la cadena, la MISMA búsqueda que el modelo hace en cada paso
    #    («¿qué verbo sigue a este concepto?») encuentra el verbo de la pregunta
    #    solo si ha llegado a B. Si no lo encuentra, la cadena se ha cortado.
    #  - Los «?» y «¿» de en medio evitan que la pregunta parezca un hecho cuyo
    #    sujeto sea A, que confundiría el primer paso.
    prompt = ['<pad>'] * relleno + [w for h in hechos for w in h] + ['¿', c1[0], '?', '¿', t,
vq, '?']
    # razonamiento: concepto, y por cada paso: verbo, efecto acumulado, concepto
    razon, s = [c1[0]], +1
    for a, b in zip(c1[:-1], c1[1:]):
        s *= signo(verbo[a])
        razon += [verbo[a], '[+]' if s > 0 else '[−]', b]
    if not conectado:
        # la cadena termina sin llegar a B: no hay más hechos que seguir
        return prompt, razon + ['NO SE PUEDE DEDUCIR']
```

```
    # último «paso» con el verbo de la pregunta: si el efecto total coincide con
    # lo que afirma la pregunta, el efecto combinado es + y la respuesta es SÍ
    s2 = s * signo(vq)
    return prompt, razon + [vq, '[+]' if s2 > 0 else '[−]', '?', 'SÍ' if s2 > 0 else 'NO']
```

La segunda parte define los hiperparámetros, el reinicio de seguridad y los parámetros; el transformer es el de RZ-2 con dos novedades. Además de `emb_prev`, aparece `emb_prev2`: cada posición recibe también el embedding del token que va dos lugares antes. Así, en «A verbo B», la posición de B «sabe» a la vez su sujeto (A) y su verbo, y el modelo puede buscar el hecho por su sujeto con una sola atención.

📄 Parte 2 de 3 — hiperparámetros, reinicio, forward y backward (con emb_prev y emb_prev2)

```
# -- Hiperparámetros ----------------------------------------
CTX, D, D_FF, N_BLQ, H = 78, 48, 192, 2, 2
DH = D // H
LR, PASOS, BATCH, CLIP, RELLENO_MAX = 3e-3, 6000, 64, 1.0, 14
# RELLENO_MAX: al entrenar se añade un relleno al azar al principio, para que las
# cadenas cortas ocupen también las posiciones finales que usarán las largas.
# Reinicio: con tan pocos parámetros, algunas inicializaciones se quedan atascadas
# en una meseta. Si en el paso COMPROBACION el modelo no acierta al menos el 80 %
# de las deducciones de 3 pasos, se reinicia con otra semilla (máx. INTENTOS).
COMPROBACION, UMBRAL, INTENTOS = 3000, 0.80, 4
if len(sys.argv) > 2:
    PASOS = int(sys.argv[2])


STD_RES = 0.02 / np.sqrt(2 * N_BLQ)


def randn(shape, std=0.02):
    return np.random.randn(*shape) * std


params = {}


def inicializar(semilla):
    np.random.seed(semilla)
    params.clear()
    params.update({'wte': randn((V, D)), 'wpe': randn((CTX, D)),
                   'emb_prev': randn((V, D)), 'emb_prev2': randn((V, D))})
    params['lnf_g'] = np.ones(D); params['lnf_b'] = np.zeros(D)
    for k in range(N_BLQ):
        params[f'ln1_g{k}'] = np.ones(D); params[f'ln1_b{k}'] = np.zeros(D)
        for n in 'qkv':
            params[f'W{n}{k}'] = randn((D, D)); params[f'b{n}{k}'] = np.zeros(D)
        params[f'Wo{k}'] = randn((D, D), STD_RES); params[f'bo{k}'] = np.zeros(D)
        params[f'ln2_g{k}'] = np.ones(D); params[f'ln2_b{k}'] = np.zeros(D)
        params[f'W1{k}'] = randn((D, D_FF)); params[f'b1{k}'] = np.zeros(D_FF)
        params[f'W2{k}'] = randn((D_FF, D), STD_RES); params[f'b2{k}'] = np.zeros(D)


inicializar(SEMILLA)
print(f'Semilla {SEMILLA} | vocabulario: {V} tokens | parámetros: {sum(p.size for p in
params.values()):,}')
MASCARA = np.triu(np.full((CTX, CTX), -1e9), k=1)


# -- Utilidades (las del Programa 8) ------------------------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)


def layer_norm_fwd(x, g, b, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)


def layer_norm_bwd(dy, g, cache):
    xhat, inv = cache
    ejes = tuple(range(dy.ndim - 1))
    dxhat = dy * g
    dx = inv * (dxhat - dxhat.mean(-1, keepdims=True)
                - xhat * (dxhat * xhat).mean(-1, keepdims=True))
    return dx, (dy * xhat).sum(ejes), dy.sum(ejes)
```

```
C_GELU = np.sqrt(2 / np.pi)


def gelu(x):
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))


def gelu_bwd(x):
    t = np.tanh(C_GELU * (x + 0.044715 * x * x * x))
    return 0.5 * (1 + t) + 0.5 * x * (1 - t * t) * C_GELU * (1 + 3 * 0.044715 * x * x)


def separar(x):        # (B,T,D) -> (B,H,T,DH)
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)


def unir(x):           # (B,H,T,DH) -> (B,T,D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)


def tmm(a, b):
    """Suma sobre lote y posiciones de aᵀ·b: el gradiente de una capa lineal."""
    return a.reshape(-1, a.shape[-1]).T @ b.reshape(-1, b.shape[-1])


# -- Forward (el del Programa 8) ----------------------------
def forward(tok):
    B, T = tok.shape
    X = params['wte'][tok] + params['wpe'][:T]
    X[:, 1:] += params['emb_prev'][tok[:, :-1]]          # token anterior
    X[:, 2:] += params['emb_prev2'][tok[:, :-2]]         # token de dos posiciones antes
    cache = []
    for k in range(N_BLQ):
        Xin = X
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(a=a, ln1=ln1, Q=Q, K=K, Vm=Vm, pesos=pesos,
                          att=att, m=m, ln2=ln2, h_pre=h_pre, h=h))
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    return xf @ params['wte'].T, (xf, lnf), cache


# -- Forward + Backward (el del Programa 8, con la pérdida ponderada por w) --
def forward_backward(tok, tgt, w):
    B, T = tok.shape
    logits, (xf, lnf), cache = forward(tok)
    probs = softmax(logits)
    bi, ti = np.arange(B)[:, None], np.arange(T)[None, :]
    loss = -(np.log(probs[bi, ti, tgt] + 1e-10) * w).sum() / w.sum()
    g = {k: np.zeros_like(v) for k, v in params.items()}
    dlog = probs.copy()
    dlog[bi, ti, tgt] -= 1.0
    dlog *= (w / w.sum())[..., None]          # solo cuentan las posiciones con w = 1
    g['wte'] += tmm(dlog, xf)                 # wte como matriz de salida
    dX, g['lnf_g'], g['lnf_b'] = layer_norm_bwd(dlog @ params['wte'], params['lnf_g'], lnf)
    for k in reversed(range(N_BLQ)):
        c = cache[k]
        g[f'W2{k}'] = tmm(c['h'], dX)
        g[f'b2{k}'] = dX.sum((0, 1))
        dh_pre = (dX @ params[f'W2{k}'].T) * gelu_bwd(c['h_pre'])
        g[f'W1{k}'] = tmm(c['m'], dh_pre)
        g[f'b1{k}'] = dh_pre.sum((0, 1))
        dx1, g[f'ln2_g{k}'], g[f'ln2_b{k}'] = layer_norm_bwd(dh_pre @ params[f'W1{k}'].T,
params[f'ln2_g{k}'], c['ln2'])
        dX1 = dX + dx1
        g[f'Wo{k}'] = tmm(c['att'], dX1)
        g[f'bo{k}'] = dX1.sum((0, 1))
        datt = separar(dX1 @ params[f'Wo{k}'].T)
        dpesos = datt @ c['Vm'].transpose(0, 1, 3, 2)
        dVm = c['pesos'].transpose(0, 1, 3, 2) @ datt
        s = (dpesos * c['pesos']).sum(-1, keepdims=True)
        dscores = c['pesos'] * (dpesos - s) / np.sqrt(DH)
        dQ, dK, dV = unir(dscores @ c['K']), unir(dscores.transpose(0, 1, 3, 2) @ c['Q']),
unir(dVm)
        da = np.zeros_like(c['a'])
        for n, dn in zip('qkv', (dQ, dK, dV)):
```

```
            g[f'W{n}{k}'] = tmm(c['a'], dn)
            g[f'b{n}{k}'] = dn.sum((0, 1))
            da += dn @ params[f'W{n}{k}'].T
        dxin, g[f'ln1_g{k}'], g[f'ln1_b{k}'] = layer_norm_bwd(da, params[f'ln1_g{k}'],
c['ln1'])
        dX = dX1 + dxin
    g['wpe'][:T] = dX.sum(0)
    np.add.at(g['wte'], tok, dX)              # wte también como embedding de entrada
    np.add.at(g['emb_prev'], tok[:, :-1], dX[:, 1:])
    np.add.at(g['emb_prev2'], tok[:, :-2], dX[:, 2:])
    return g, loss
```

La tercera parte entrena (con el reinicio automático), examina con 200 problemas nuevos por cada longitud de 1 a 8 pasos y traduce tres razonamientos del modelo a frases legibles.

📄 Parte 3 de 3 — entrenamiento, examen y ejemplos

```
# -- Lotes de entrenamiento (deducciones de 1 a 4 pasos) ---
def lote(rng):
    X = np.zeros((BATCH, CTX - 1), int); Y = np.zeros_like(X); W = np.zeros(X.shape)
    for i in range(BATCH):
        p, r = problema(rng.randint(1, PASOS_MAX_ENTRENO + 1), rng, rng.randint(0, RELLENO_MAX
+ 1))
        s = [W2I[w] for w in p + r]
        s += [0] * (CTX - len(s))
        X[i], Y[i] = s[:-1], s[1:]
        W[i, len(p) - 1:len(p) + len(r) - 1] = 1        # solo lo que escribe el modelo
    return X, Y, W


# -- Generación greedy hasta la respuesta -----------------
def resolver(prompts, max_tokens=3 * N_HECHOS + 4):
    seqs = [[W2I[w] for w in p] for p in prompts]
    salida = [[] for _ in prompts]
    for _ in range(max_tokens):
        activos = [i for i in range(len(seqs)) if len(seqs[i]) < CTX
                   and not (salida[i] and W2I[salida[i][-1]] in RESPUESTAS)]
        if not activos:
            break
        T = max(len(seqs[i]) for i in activos)
        X = np.array([seqs[i] + [0] * (T - len(seqs[i])) for i in activos])
        logits, _, _ = forward(X)
        for j, i in enumerate(activos):
            nx = int(logits[j, len(seqs[i]) - 1].argmax())
            seqs[i].append(nx); salida[i].append(VOCAB[nx])
    return salida


def examen(L, n=200):
    rng = np.random.RandomState(1000 + L)
    probs = [problema(L, rng) for _ in range(n)]
    sal = resolver([p for p, _ in probs])
    resp = np.mean([s[-1:] == r[-1:] for s, (_, r) in zip(sal, probs)])
    todo = np.mean([s == r for s, (_, r) in zip(sal, probs)])
    return resp, todo


# -- Entrenamiento ----------------------------------------
def lr_warmup_coseno(t, lr_max, t_total, t_warmup=150, lr_min=1e-4):
    if t < t_warmup:
        return lr_max * (t + 1) / t_warmup
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * (t - t_warmup) / (t_total -
t_warmup)))


def entrenar(semilla):
    """Entrena desde cero; devuelve False si en la comprobación sigue atascado."""
    inicializar(semilla)
    m_adam = {k: np.zeros_like(v) for k, v in params.items()}
    v_adam = {k: np.zeros_like(v) for k, v in params.items()}
    rng = np.random.RandomState(semilla + 7)
    print(f'\nSemilla {semilla}: entrenando con deducciones de 1 a {PASOS_MAX_ENTRENO}
pasos...', flush=True)
    for paso in range(PASOS):
        X, Y, W = lote(rng)
        g, loss = forward_backward(X, Y, W)
        norma = np.sqrt(sum((x ** 2).sum() for x in g.values()))
        if norma > CLIP:
            g = {k: x * CLIP / norma for k, x in g.items()}
        t, lr = paso + 1, lr_warmup_coseno(paso, LR, PASOS)
        for k in params:
            m_adam[k] = 0.9 * m_adam[k] + 0.1 * g[k]
            v_adam[k] = 0.999 * v_adam[k] + 0.001 * g[k] ** 2
```

```
            params[k] -= lr * (m_adam[k] / (1 - 0.9 ** t)) / (np.sqrt(v_adam[k] / (1 - 0.999
** t)) + 1e-8)
        if paso % 250 == 0 or paso == PASOS - 1:
            print(f'Paso {paso:5d}: pérdida {loss:.4f} | aciertos con 3 pasos {examen(3, 50)
[0]:5.0%}'
                  f' | con 7 pasos (nunca vistos) {examen(7, 50)[0]:5.0%}', flush=True)
        if paso == COMPROBACION and examen(3, 100)[0] < UMBRAL:
            print(f'  → atascado en una meseta en el paso {paso}: se reinicia con otra
semilla', flush=True)
            return False
    return True


for intento in range(INTENTOS):
    if entrenar(SEMILLA + 100 * intento):
        break
else:
    print(f'\nAVISO: ninguno de los {INTENTOS} intentos salió de la meseta; '
          'el examen siguiente corresponde a un modelo que no ha aprendido.')

# -- Examen final ------------------------------------------
print('\n=== EXAMEN: 200 problemas nuevos por cada número de pasos ===')
print('pasos  respuesta  razonamiento completo correcto')
for L in range(1, N_HECHOS - 1):
    r, t = examen(L)
    nota = '' if L <= PASOS_MAX_ENTRENO else '  ← nunca visto al entrenar'
    print(f'{L:5d}  {r:9.1%}  {t:9.1%}{nota}')


# -- Ejemplos legibles -------------------------------------
def frase(a, v, b):
    txt = f'{a} {v} {b}'
    return txt.replace(' a el ', ' al ')


def legible(razon, objetivo):
    out, actual = '', razon[0]
    for i in range(1, len(razon) - 1, 3):
        v, sg, b = (razon[i:i + 3] + ['?', '?', '?'])[:3]
        if b == '?':                                  # paso con el verbo de la pregunta
            tipo = 'favorece' if v in FAVORECE else 'perjudica'
            out += (f'\n      la pregunta dice «{v}» ({tipo}): '
                    f'{"coincide" if sg == "[+]" else "no coincide"} con el efecto acumulado')
            break
        efecto = 'favorece' if sg == '[+]' else 'perjudica' if sg == '[−]' else sg
        out += f'\n      {frase(actual, v, b):48s} [efecto acumulado: {efecto}]'
        actual = b
    else:
        out += f'\n      la cadena termina en «{actual}», no en «{objetivo}»'
    return out


print('\n=== EJEMPLOS ===')
rng = np.random.RandomState(2026)
for L in (2, 5, 7):
    p, r = problema(L, rng)
    sal = resolver([p])[0]
    toks = [w for w in p if w != '<pad>']
    q = toks.index('¿')
    print('Hechos:')
    for i in range(0, q, 4):
        print('   ', frase(*toks[i:i + 3]) + '.')
    print(f'Pregunta: ¿{frase(toks[q + 1], toks[q + 5], toks[q + 4])}?')
    print('Razonamiento del modelo:', legible(sal, toks[q + 4]))
    print(f'      ⇒  {sal[-1]}   (correcto: {r[-1]})\n')
```

> Salida del programa (semilla 42, por defecto; unos treinta y cinco minutos):

```
Semilla 42 | vocabulario: 57 tokens | parámetros: 68,592

Semilla 42: entrenando con deducciones de 1 a 4 pasos...
Paso     0: pérdida 4.0679 | aciertos con 3 pasos    0% | con 7 pasos (nunca vistos)    0%
Paso   250: pérdida 1.9364 | aciertos con 3 pasos   18% | con 7 pasos (nunca vistos)   20%
Paso   500: pérdida 1.8536 | aciertos con 3 pasos   16% | con 7 pasos (nunca vistos)   26%
Paso   750: pérdida 1.6847 | aciertos con 3 pasos   32% | con 7 pasos (nunca vistos)   26%
Paso  1000: pérdida 0.2564 | aciertos con 3 pasos   84% | con 7 pasos (nunca vistos)   52%
Paso  1250: pérdida 0.0787 | aciertos con 3 pasos   80% | con 7 pasos (nunca vistos)   86%
Paso  1500: pérdida 0.0383 | aciertos con 3 pasos   86% | con 7 pasos (nunca vistos)   84%
Paso  1750: pérdida 0.0282 | aciertos con 3 pasos   98% | con 7 pasos (nunca vistos)   88%
Paso  2000: pérdida 0.0243 | aciertos con 3 pasos   92% | con 7 pasos (nunca vistos)   94%
Paso  2250: pérdida 0.0023 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)   96%
Paso  2500: pérdida 0.0018 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)   98%
Paso  2750: pérdida 0.0011 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3000: pérdida 0.0007 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3250: pérdida 0.0008 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3500: pérdida 0.0003 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
```

```
Paso  3750: pérdida 0.0004 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  4000: pérdida 0.0002 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  4250: pérdida 0.0002 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  4500: pérdida 0.0002 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  4750: pérdida 0.0002 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  5000: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  5250: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  5500: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  5750: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  5999: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%

=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1     100.0%     100.0%
    2     100.0%     100.0%
    3     100.0%     100.0%
    4     100.0%     100.0%
    5     100.0%     100.0%  ← nunca visto al entrenar
    6     100.0%     100.0%  ← nunca visto al entrenar
    7     100.0%     100.0%  ← nunca visto al entrenar
    8     100.0%     100.0%  ← nunca visto al entrenar

=== EJEMPLOS ===
Hechos:
    el gradiente mejora la incertidumbre.
    la normalización estabiliza el costo.
    la explotación ayuda a la normalización.
    la inferencia estabiliza la explotación.
    la eficiencia estabiliza la precisión.
    la exploración mejora el contexto.
    el contexto dificulta la tokenización.
    la precisión limita el ajuste fino.
    el costo mejora la pérdida.
    la pérdida acelera el gradiente.
Pregunta: ¿la exploración limita la tokenización?
Razonamiento del modelo:
      la exploración mejora el contexto               [efecto acumulado: favorece]
      el contexto dificulta la tokenización           [efecto acumulado: perjudica]
      la pregunta dice «limita» (perjudica): coincide con el efecto acumulado
      ⇒  SÍ   (correcto: SÍ)

Hechos:
    la temperatura ralentiza la recompensa.
    el gradiente limita la regularización.
    el costo mejora la privacidad.
    la tokenización mejora el sesgo.
    el contexto complica la temperatura.
    la atención estabiliza el gradiente.
    la regularización mejora el costo.
    la privacidad ayuda a la memoria.
    la recompensa dificulta la pérdida.
    la inferencia estabiliza el contexto.
Pregunta: ¿la atención mejora el sesgo?
Razonamiento del modelo:
      la atención estabiliza el gradiente             [efecto acumulado: favorece]
      el gradiente limita la regularización           [efecto acumulado: perjudica]
      la regularización mejora el costo               [efecto acumulado: perjudica]
      el costo mejora la privacidad                   [efecto acumulado: perjudica]
      la privacidad ayuda a la memoria                [efecto acumulado: perjudica]
      la cadena termina en «la memoria», no en «el sesgo»
      ⇒  NO SE PUEDE DEDUCIR   (correcto: NO SE PUEDE DEDUCIR)

Hechos:
    el entrenamiento estabiliza el error.
    la arquitectura dificulta la memoria.
    la exploración dificulta la recompensa.
    la precisión estabiliza el preentrenamiento.
    la normalización ayuda al ruido.
    el preentrenamiento complica el entrenamiento.
    el sobreajuste ayuda a la arquitectura.
    el ruido mejora el vocabulario.
    el error dificulta la simulación.
    la simulación limita la normalización.
Pregunta: ¿la precisión complica la recompensa?
Razonamiento del modelo:
      la precisión estabiliza el preentrenamiento     [efecto acumulado: favorece]
      el preentrenamiento complica el entrenamiento   [efecto acumulado: perjudica]
      el entrenamiento estabiliza el error            [efecto acumulado: perjudica]
      el error dificulta la simulación                [efecto acumulado: favorece]
      la simulación limita la normalización           [efecto acumulado: perjudica]
      la normalización ayuda al ruido                 [efecto acumulado: perjudica]
      el ruido mejora el vocabulario                  [efecto acumulado: perjudica]
      la cadena termina en «el vocabulario», no en «la recompensa»
      ⇒  NO SE PUEDE DEDUCIR   (correcto: NO SE PUEDE DEDUCIR)
```

El primer ejemplo muestra el razonamiento con signo en acción: «la exploración mejora el contexto» (favorece) y «el contexto dificulta la tokenización» (perjudica). Favorecer algo que perjudica es perjudicar, así que la exploración perjudica a la tokenización; la pregunta dice «limita», que también perjudica, y el modelo responde SÍ. El segundo es el caso de las cadenas cortadas: el modelo sigue la cadena desde «la atención» hasta «la memoria», no encuentra ningún hecho que empiece por «la memoria» y concluye que no se puede deducir.

Observa la meseta del principio: durante unos 750 pasos el modelo acierta entre un 16 y un 32 % de las deducciones de 3 pasos, y de repente, entre los pasos 750 y 1000, salta al 84 %. Algunas inicializaciones no salen nunca de una meseta así (lo vimos en RZ-2). Por eso el programa incluye un reinicio de seguridad: si en el paso 3000 no acierta al menos el 80 % de las deducciones de 3 pasos, vuelve a empezar con otra semilla, hasta cuatro veces. Ninguna de las seis ejecuciones de la tabla siguiente lo necesitó.

Semilla	1–4 pasos	5	6	7	8
42 (por defecto)	100 %	100 %	100 %	100 %	**100 %**
1	100 %	100 %	100 %	100 %	100 %
2	100 %	100 %	100 %	100 %	100 %
7	99.5–100 %	99.0 %	99.0 %	98.5 %	97.5 %
123	100 %	100 %	100 %	100 %	100 %
2026	100 %	100 %	100 %	100 %	100 %

Las seis semillas resuelven las longitudes de entrenamiento y generalizan hasta 8 pasos. Cinco aciertan el 100 % en todas las longitudes; la semilla 7 es la más débil, con un 97.5 % en las deducciones de 8 pasos. Analizar en qué se equivoca es el Ejercicio 15.5.

15.6 Lo que hemos demostrado y lo que no

Conviene terminar con el mismo rigor con el que empezamos. Estos experimentos **sí** demuestran que:

- Memorizar o generalizar no es una propiedad de la arquitectura: el mismo modelo, el del Programa 8, memoriza 93 frases y extrae la regla de la suma. Lo deciden los datos (cuántos hay y qué estructura tienen) y el formato de la tarea.
- Escribir los pasos intermedios permitió a nuestros modelos poco profundos (con las ayudas de entrada descritas) resolver problemas que, en nuestras pruebas, no resolvían de golpe, y resolverlos más largos que todos los que vieron al entrenar.
- Las conclusiones dependen de la semilla: hay que repetir los experimentos antes de creérselos.

Y **no** demuestran que nuestros modelos «razonen» en sentido general. Cada uno sabe hacer exactamente una tarea, con un vocabulario fijo y un contexto de entre 9 y 78 posiciones. RZ-3 no sabe sumar y RZ-2 no sabe deducir; ninguno sabría qué hacer con una pregunta formulada de otra manera.

Los LLM actuales aplican la misma idea a una escala enormemente mayor. Además de pedirles que razonen paso a paso, desde 2024 existen modelos entrenados específicamente para escribir un razonamiento largo antes de responder, como OpenAI o1 (presentado en septiembre de 2024) o DeepSeek-R1 (enero de 2025). En el fondo hacen lo mismo que nuestro modelo de 62 592 parámetros con sus letras: convertir un problema que exigiría muchas operaciones en cadena dentro de la red en una secuencia de pasos sencillos, escritos uno detrás de otro. Antes de recorrer, en el Capítulo 17, el camino que separa

nuestros modelos de esos gigantes, el Capítulo 16 da el primer paso de ese camino: cargar en tu propio código un modelo de verdad.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

• Para saber si un modelo memoriza hay que examinarlo con datos apartados, compararlo con una línea base sencilla y buscar copias literales: el Nano-LLM del Capítulo 13 memoriza (validación 4.18 frente a 0.21 en entrenamiento).

• La misma red extrae una regla cuando los datos lo permiten: con 8000 sumas generaliza al 100 %; con 250, acierta solo el 58.8 % de las sumas nuevas.

• Escribir los pasos intermedios (cadena de razonamiento) convierte un problema de muchas búsquedas en muchas búsquedas de una en una: la longitud del texto sustituye a la profundidad de la red.

• Los modelos entrenados con 1 a 4 pasos resuelven hasta 8 o 10, pero solo en su tarea, y los resultados dependen de la semilla.

✏️ EJERCICIOS — Capítulo 15 — ¿Memoriza o razona?

Resuelve los ejercicios siguientes antes de continuar. Todas las salidas que se muestran proceden de ejecuciones reales de las soluciones.

Ejercicio 15.1 Early stopping de verdad

La Prueba 1 del Programa RZ-1 encuentra el mejor momento según la validación (paso 750), pero sigue entrenando hasta el final. Modifícalo para que guarde una copia de los pesos cada vez que la validación mejore, y compara el modelo guardado con el final: pérdidas y texto generado.

> 💡 **Pista**
>
> Basta con copiar el diccionario de pesos (`{k: x.copy() for k, x in p.items()}`) en el mismo `if` que actualiza `mejor`. Para generar con cualquiera de los dos modelos, reutiliza las funciones de la Prueba 3, que usan la variable `p8`.

✅ Solución:

📄 ejercicio_15_1.py

```
# ============================================================
# EJERCICIO 15.1 — Early Stopping de verdad en el Programa RZ-1
# Guarda los pesos del momento con MEJOR pérdida de validación
# y compara ese modelo con el del final del entrenamiento.
# ============================================================
import numpy as np

codigo = open("programa_RZ1_memoriza.py", encoding="utf-8").read()
# 1) Cada vez que mejora la validación, se copian los pesos
codigo = codigo.replace(
    "            mejor = (lv, paso)",
    "            mejor = (lv, paso)\n"
    "            mejores_pesos = {k: x.copy() for k, x in p.items()}")
# 2) Se ejecuta solo la Prueba 1 (entrenamiento con validación)
exec(codigo.split('# PRUEBA 2')[0])
# 3) Funciones de generación y de copia literal de la Prueba 3
exec(codigo.split("p8 = dict(np.load('nano_llm_pesos.npz'))")[1]
           .split('rng = np.random.RandomState(2026)')[0])

print('\n=== MODELO FINAL frente a MODELO CON EARLY STOPPING ===')
for nombre, pesos in [('final (paso 11999)', p), (f'mejor validación (paso {mejor[1]})',
mejores_pesos)]:
    p8 = pesos                                      # generar_lote() usa p8
    rng = np.random.RandomState(2026)
    textos = generar_lote('mañana la ', 150, 10, rng)
    copia = np.mean([fraccion_copiada(t) for t in textos])
    print(f'\n{nombre}: entrenamiento {perdida(pesos, XT, YT, WT):.3f} | '
          f'validación {perdida(pesos, XV, YV, WV):.3f} | copia literal {copia:.0%}')
    print(textos[0])
```

> Salida del programa (extracto final):

```
...
=== MODELO FINAL frente a MODELO CON EARLY STOPPING ===

final (paso 11999): entrenamiento 0.212 | validación 4.175 | copia literal 90%
mañana la entrada directamente con la salida.
la normalización evita que las activaciones crezcan sin control.
una red ancha tiene muchas neuronas en cada capa.

mejor validación (paso 750): entrenamiento 1.587 | validación 1.923 | copia literal 0%
mañana la de trenales.
la compreficelizadale alentro muchactas.
la finación espación conternad de el modelo.
la dentexto perdenso diconcicaliza prefibila aciste
```

El early stopping hace exactamente lo que promete: el modelo guardado en el paso 750 tiene la mejor pérdida en frases nuevas (1.923 frente a 4.175) y no copia nada del corpus (0 % frente a 90 %). Pero su texto revela el precio: «trenales», «compreficelizadale»,

«espación». En el paso 750 el modelo todavía no ha aprendido a escribir, y cuando aprende lo hace memorizando. Con 83 frases no existe un punto intermedio en el que escriba bien sin copiar: el early stopping elige el modelo que mejor generaliza, pero no puede crear una generalización que los datos no permiten. Es el mismo diagnóstico de la sección 13.6.1, ahora con una validación de verdad.

Ejercicio 15.2 Sumar sin invertir el resultado

Cambia el formato de RZ-2 para que el resultado se escriba en orden natural (47+85=132). Antes de ejecutarlo, predice qué pasará: ¿aprenderá igual, peor o nada? Después compruébalo con varias semillas.

> 💡 **Pista**
>
> Solo cambian dos líneas: la que forma el texto (`formatear`) y la que lee el resultado en el examen. El análisis de la atención supone el orden invertido; quítalo.

✅ Solución:

📄 ejercicio_15_2.py

```
# ============================================================
# EJERCICIO 15.2 — RZ-2 con el resultado en orden natural
# 47+85=132 en lugar de 47+85=231: la primera cifra que
# escribe el modelo es ahora la de las centenas.
# Uso: python ejercicio_15_2.py [semilla]
# ============================================================
fuente = open('programa_RZ2_suma.py', encoding='utf-8').read()
cambios = [
    ("str(a + b).zfill(3)[::-1] + '\\n'", "str(a + b).zfill(3) + '\\n'"),     # sin invertir
    ("pred = int(''.join(I2C[i] for i in r)[::-1])", "pred = int(''.join(I2C[i] for i in
r))"),
]
for viejo, nuevo in cambios:
    assert viejo in fuente, viejo
    fuente = fuente.replace(viejo, nuevo)
fuente = fuente.split('# -- ¿Cómo lo hace?')[0]   # el análisis de atención supone el orden
invertido
exec(fuente)
```

> Salida del programa (semilla 42; extracto):

```
Parámetros: 57,696
Semilla 42 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000

Entrenando...  (exactitud = suma entera correcta)
Paso     0: pérdida 2.6059 | exactitud entrenamiento   0.0% | sumas NUNCA vistas   0.0%
Paso   500: pérdida 0.1657 | exactitud entrenamiento  90.7% | sumas NUNCA vistas  88.8%
Paso  1000: pérdida 0.0415 | exactitud entrenamiento  98.2% | sumas NUNCA vistas  97.8%
Paso  1500: pérdida 0.0236 | exactitud entrenamiento  98.4% | sumas NUNCA vistas  98.7%
Paso  2000: pérdida 0.0010 | exactitud entrenamiento  99.4% | sumas NUNCA vistas  99.7%
Paso  2500: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  3000: pérdida 0.0001 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
Paso  3500: pérdida 0.0000 | exactitud entrenamiento 100.0% | sumas NUNCA vistas 100.0%
...
Aciertos en las 2000 sumas no vistas: 100.0%
```

> python ejercicio_15_2.py 1 … 5 (primera y última línea de cada ejecución):

```
Semilla 1 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000
Aciertos en las 2000 sumas no vistas: 100.0%
Semilla 2 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000
Aciertos en las 2000 sumas no vistas: 100.0%
Semilla 3 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000
Aciertos en las 2000 sumas no vistas: 100.0%
Semilla 4 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000
Aciertos en las 2000 sumas no vistas: 100.0%
Semilla 5 | cabezas 2 | sumas de entrenamiento: 8000 | sumas de examen (no vistas): 2000
Aciertos en las 2000 sumas no vistas: 100.0%
```

Si predijiste que aprendería peor, el experimento te lleva la contraria, y es una buena lección. Con el resultado en orden natural, el modelo aprende las 2000 sumas no vistas al

100 % con las seis semillas probadas (42, 1, 2, 3, 4 y 5, igual que con el orden invertido) y, con la semilla 42, más deprisa: un 88.8 % en el paso 500, frente al 36.4 % del orden invertido. ¿Por qué? Con números de dos cifras, la cifra de las centenas solo depende de cuatro cifras, todas visibles en el enunciado; dos bloques de atención pueden mirarlas a la vez sin necesidad de haber escrito antes las unidades. La ventaja del orden invertido que describen Lee et al. (2023) aparece con números largos, en los que la llevada se propaga a través de muchas cifras. Moraleja: una idea razonable sobre el formato de los datos es una hipótesis, no un resultado, hasta que la compruebas.

Ejercicio 15.3 Aprender a restar

Adapta RZ-2 para que aprenda restas a − b con a ≥ b (las dos de dos cifras), con el resultado de dos cifras escrito al revés. Hay 5050 restas posibles; aparta 1000 para el examen.

> **💡 Pista**
>
> Cambia el vocabulario (`-` en lugar de `+`), `formatear`, la lista de pares, el tamaño de la partición, el vector `PESO` (ahora son 2 cifras), `CTX` (la secuencia es una posición más corta) y el examen (2 cifras). Como ampliación, prueba con sumas de 3 cifras: necesitarás `CTX = 12` y 4 cifras de resultado.

✅ Solución:

📄 ejercicio_15_3.py

```
# ============================================================
# EJERCICIO 15.3 — RZ-2 aprende a RESTAR
# a − b con a ≥ b (dos cifras), resultado de 2 cifras escrito
# al revés: 85−47=38 se escribe «85-47=83». Hay 5 050 restas
# posibles; 1 000 se apartan para el examen.
# Uso: python ejercicio_15_3.py [semilla]
# ============================================================
fuente = open('programa_RZ2_suma.py', encoding='utf-8').read()
cambios = [
    ("VOCAB = list('0123456789+=\\n')", "VOCAB = list('0123456789-=\\n')"),
    ("return f'{a:02d}+{b:02d}=' + str(a + b).zfill(3)[::-1] + '\\n'",
     "return f'{a:02d}-{b:02d}=' + str(a - b).zfill(2)[::-1] + '\\n'"),
    ("pares = [(a, b) for a in range(100) for b in range(100)]",
     "pares = [(a, b) for a in range(100) for b in range(a + 1)]"),
    ("TEST, TRAIN = pares[:2000], pares[2000:2000 + N_ENTRENO]",
     "TEST, TRAIN = pares[:1000], pares[1000:1000 + N_ENTRENO]"),
    ("PESO = np.zeros(9); PESO[5:8] = 1", "PESO = np.zeros(8); PESO[5:7] = 1"),
    ("CTX, D, D_FF, N_BLQ = 9, 48, 192, 2", "CTX, D, D_FF, N_BLQ = 8, 48, 192, 2"),
    ("    for _ in range(3):", "    for _ in range(2):"),
    ("    return seq[:, 6:9]", "    return seq[:, 6:8]"),
    ("return (resolver(X) == Y[:, 5:8]).all(1).mean()", "return (resolver(X) == Y[:,
5:7]).all(1).mean()"),
    ("print(f'{a:2d} + {b:2d} = {pred:3d}   {\"✓\" if pred == a + b else \"✗ (correcto: \" +
str(a + b) + \")\"}')",
     "print(f'{a:2d} - {b:2d} = {pred:3d}   {\"✓\" if pred == a - b else \"✗ (correcto: \" +
str(a - b) + \")\"}')"),
    ("=== EXAMEN CON SUMAS QUE EL MODELO NUNCA VIO ===", "=== EXAMEN CON RESTAS QUE EL MODELO
NUNCA VIO ==="),
    ("print(f'\\nAciertos en las {len(TEST)} sumas no vistas", "print(f'\\nAciertos en las
{len(TEST)} restas no vistas"),
    ("sumas de entrenamiento: {len(TRAIN)}", "restas de entrenamiento: {len(TRAIN)}"),
    ("sumas de examen (no vistas)", "restas de examen (no vistas)"),
    ("(exactitud = suma entera correcta)", "(exactitud = resta entera correcta)"),
    ("| sumas NUNCA vistas", "| restas NUNCA vistas"),
]
for viejo, nuevo in cambios:
    assert viejo in fuente, viejo
    fuente = fuente.replace(viejo, nuevo)
fuente = fuente.split('# -- ¿Cómo lo hace?')[0]
exec(fuente)
```

> Salida del programa:

```
Parámetros: 57,648
Semilla 42 | cabezas 2 | restas de entrenamiento: 4050 | restas de examen (no vistas): 1000

Entrenando...  (exactitud = resta entera correcta)
Paso     0: pérdida 2.6066 | exactitud entrenamiento   0.0% | restas NUNCA vistas   0.0%
Paso   500: pérdida 0.1492 | exactitud entrenamiento  91.8% | restas NUNCA vistas  91.7%
Paso  1000: pérdida 0.0270 | exactitud entrenamiento 100.0% | restas NUNCA vistas  99.9%
```

```
Paso  1500: pérdida 0.0002 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  2000: pérdida 0.0001 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  2500: pérdida 0.0001 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  3000: pérdida 0.0001 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  3500: pérdida 0.0001 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  4000: pérdida 0.0000 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  4500: pérdida 0.0000 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  5000: pérdida 0.0000 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  5500: pérdida 0.0000 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%
Paso  5999: pérdida 0.0000 | exactitud entrenamiento 100.0% | restas NUNCA vistas 100.0%

=== EXAMEN CON RESTAS QUE EL MODELO NUNCA VIO ===
46 - 35 =  11   ✓
31 -  6 =  25   ✓
94 - 52 =  42   ✓
30 - 22 =   8   ✓
94 - 88 =   6   ✓
33 - 14 =  19   ✓
96 - 86 =  10   ✓
33 -  1 =  32   ✓
23 - 22 =   1   ✓
98 - 38 =  60   ✓

Aciertos en las 1000 restas no vistas: 100.0%
```

El modelo aprende a restar exactamente igual que a sumar: el 100 % de las 1000 restas que nunca vio, desde el paso 1500 del entrenamiento (un 99.9 % ya en el 1000). En la resta, el papel de la llevada lo hace el «préstamo» (en 85 − 47, a 5 no se le puede restar 7 y hay que pedir prestada una decena), y escribir primero las unidades vuelve a dejar ese dato escrito antes de necesitarlo. Fíjate en que hay menos datos que en la suma (4050 restas de entrenamiento frente a 8000 sumas) y aun así basta para extraer la regla.

Ejercicio 15.4 ¿Hasta dónde llega la cadena?

En RZ-3 el examen llega hasta 10 pasos porque cada problema tiene 12 hechos. Aumenta `N_HECHOS` a 16 para poder examinar hasta 14 pasos, sin cambiar el entrenamiento (de 1 a 4 pasos). ¿Sigue generalizando?

> 💡 **Pista**
>
> Más hechos significa un prompt más largo: hay que ampliar `CTX` (y con él la tabla wpe), el relleno aleatorio, para que las posiciones del final también se vean al entrenar, y la longitud máxima que escribe `resolver`.

✅ Solución:

📄 ejercicio_15_4.py

```
# ============================================================
# EJERCICIO 15.4 — RZ-3 con cadenas más largas
# 16 hechos por problema (antes 12). El examen llega hasta
# 14 pasos; se sigue entrenando SOLO con 1 a 4 pasos.
# Más hechos = prompt más largo: hay que ampliar CTX (y con
# él wpe) y el relleno, para que las posiciones finales
# también se vean al entrenar.
# Uso: python ejercicio_15_4.py [pasos]   (por defecto 2000)
# ============================================================
import sys
PASOS_EJ = int(sys.argv[1]) if len(sys.argv) > 1 else 2000
sys.argv = sys.argv[:1]          # RZ-3 lee sus propios argumentos
fuente = open('programa_RZ3_deduce_letras.py', encoding='utf-8').read()
cambios = [
    ("N_HECHOS, PASOS_MAX_ENTRENO = 12, 4", "N_HECHOS, PASOS_MAX_ENTRENO = 16, 4"),
    ("CTX, D, D_FF, N_BLQ, H = 56, 48, 192, 2, 2", "CTX, D, D_FF, N_BLQ, H = 72, 48, 192, 2,
2"),
    ("LR, PASOS, BATCH, CLIP, RELLENO_MAX = 3e-3, 2000, 64, 1.0, 8",
     f"LR, PASOS, BATCH, CLIP, RELLENO_MAX = 3e-3, {PASOS_EJ}, 64, 1.0, 12"),
    ("def resolver(prompts, max_chars=12):", "def resolver(prompts, max_chars=16):"),
    ("for L in (2, 6, 9):", "for L in (3, 9, 13):"),
]
for viejo, nuevo in cambios:
    assert viejo in fuente, viejo
    fuente = fuente.replace(viejo, nuevo)
exec(fuente)
```

> Salida del programa (2000 pasos; extracto):

```
Modo: paso_a_paso | emb_prev: sí | parámetros: 63,360
...
Paso  1250: pérdida 2.3451 | aciertos con 3 pasos   50% | con 7 pasos (nunca vistos)   44%
Paso  1500: pérdida 2.2621 | aciertos con 3 pasos   50% | con 7 pasos (nunca vistos)   34%
Paso  1750: pérdida 2.3061 | aciertos con 3 pasos   50% | con 7 pasos (nunca vistos)   32%
Paso  1999: pérdida 2.1989 | aciertos con 3 pasos   48% | con 7 pasos (nunca vistos)   42%

=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1      50.0%       0.5%
    2      48.0%       0.0%
    3      46.5%       0.0%
    4      49.0%       0.0%
    5      48.0%       0.0%  ← nunca visto al entrenar
    6      43.5%       0.0%  ← nunca visto al entrenar
    7      50.5%       0.0%  ← nunca visto al entrenar
    8      57.5%       0.0%  ← nunca visto al entrenar
    9      46.0%       0.0%  ← nunca visto al entrenar
   10      49.5%       0.0%  ← nunca visto al entrenar
   11      46.5%       0.0%  ← nunca visto al entrenar
   12      51.5%       0.0%  ← nunca visto al entrenar
   13      50.0%       0.0%  ← nunca visto al entrenar
   14      50.5%       0.0%  ← nunca visto al entrenar

=== EJEMPLOS ===
Hechos: k>d, e>s, j>a, q>i, s>o, p>b, v>r, c>m, w>c, b>e, l>w, a>l, i>z, o>h, h>j, r>k
¿v>d?   Modelo: l > v  ⇒  NO   (correcto: SÍ)

Hechos: p>s, e>x, y>k, n>a, q>e, h>m, x>p, o>q, k>f, l>i, u>o, f>n, b>v, i>y, z>b, v>u
¿z>a?   Modelo: t > v  ⇒  NO   (correcto: NO)
```

Con 16 hechos y los 2000 pasos de RZ-3, el modelo no sale de la meseta: con el entrenamiento terminado, la respuesta sigue en el azar con cualquier longitud y el razonamiento completo no sale nunca. ¿Es un límite del modelo o falta de entrenamiento? Repitámoslo con el doble de pasos:

> python ejercicio_15_4.py 4000 (extracto):

```
...
Paso  3250: pérdida 0.0002 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3500: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3750: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%
Paso  3999: pérdida 0.0001 | aciertos con 3 pasos  100% | con 7 pasos (nunca vistos)  100%

=== EXAMEN: 200 problemas nuevos por cada número de pasos ===
pasos  respuesta  razonamiento completo correcto
    1     100.0%     100.0%
    2     100.0%     100.0%
    3     100.0%     100.0%
    4     100.0%     100.0%
    5     100.0%     100.0%  ← nunca visto al entrenar
    6     100.0%      99.5%  ← nunca visto al entrenar
    7     100.0%     100.0%  ← nunca visto al entrenar
    8     100.0%     100.0%  ← nunca visto al entrenar
    9     100.0%     100.0%  ← nunca visto al entrenar
   10     100.0%     100.0%  ← nunca visto al entrenar
   11     100.0%     100.0%  ← nunca visto al entrenar
   12      99.5%      99.0%  ← nunca visto al entrenar
   13     100.0%      99.5%  ← nunca visto al entrenar
   14     100.0%     100.0%  ← nunca visto al entrenar
```

Con 4000 pasos el resultado cambia por completo: el modelo sale de la meseta hacia el paso 1000 y resuelve prácticamente todas las deducciones hasta 14 pasos (el peor caso, 12 pasos, con un 99.5 % de respuestas y un 99.0 % de razonamientos completos), aunque solo se entrenó con 1 a 4. Ojo: cambiar PASOS no solo da más tiempo, también hace más lento el decaimiento coseno, así que la tasa de aprendizaje se mantiene alta durante más pasos. Conclusión: alargar la cadena sigue funcionando, pero exige más contexto y un entrenamiento suficiente; con un presupuesto justo, el resultado depende de si el modelo escapa a tiempo de la meseta.

Ejercicio 15.5 Análisis de errores de RZ-4

Con la semilla 7, RZ-4 acierta el 97.5 % de las deducciones de 8 pasos: falla 5 de 200. Clasifica sus errores según el primer token del razonamiento que difiere del correcto: ¿se equivoca al buscar el concepto siguiente, al copiar el verbo, al acumular el signo o en la respuesta final? ¿En qué paso de la cadena aparece el primer error?

> 💡 **Pista**
>
> Compara, token a token, la salida de `resolver` con el razonamiento correcto que devuelve `problema`. Como el razonamiento alterna concepto, verbo y signo, la posición del primer error dice de qué tipo es y en qué paso ocurre. Usa los mismos 200 problemas del examen (`RandomState(1000 + 8)`).

✅ Solución:

📄 ejercicio_15_5.py

```
# ============================================================
# EJERCICIO 15.5 — ¿En qué se equivoca RZ-4 con 8 pasos?
# Entrena el Programa RZ-4 con la semilla 7 (la única de las
# seis de 15.5 que falla con 8 pasos) y clasifica cada error
# según el PRIMER token del razonamiento que difiere del correcto.
# Tiempo: el del Programa RZ-4 (~35 min) + 1 min de análisis.
# ============================================================
import sys
import numpy as np
from collections import Counter

sys.argv = [sys.argv[0], '7']          # RZ-4 lee la semilla de sus argumentos
exec(open('programa_RZ4_deduce_frases.py', encoding='utf-8').read())


def tipo_error(sal, correcto):
    for i, (a, b) in enumerate(zip(sal, correcto)):
        if a != b:
            break
    else:
        return 'razonamiento demasiado largo o corto'
    if i == len(correcto) - 1:
        return 'cadena correcta, respuesta final mal'
    if b in CONCEPTOS:
        return 'se equivoca de concepto (búsqueda del hecho)'
    if b in VERBOS:
        return 'se equivoca de verbo'
    if b in ('[+]', '[-]'):
        return 'se equivoca de signo (efecto acumulado)'
    return f'otro ({b!r} → {a!r})'


print('\n=== ANÁLISIS DE ERRORES CON 8 PASOS (200 problemas) ===')
rng = np.random.RandomState(1000 + 8)                  # los mismos del examen
probs = [problema(8, rng) for _ in range(200)]
sal = resolver([pr for pr, _ in probs])
errores = Counter()
posiciones = Counter()
ejemplo = None
for s, (pr, r) in zip(sal, probs):
    if s == r:
        continue
    t = tipo_error(s, r)
    errores[t] += 1
    i = next((k for k, (a, b) in enumerate(zip(s, r)) if a != b), len(r))
    posiciones[(i - 1) // 3 + 1] += 1                  # en qué paso de la cadena
    if ejemplo is None and 'concepto' in t:
        ejemplo = (pr, s, r)
print(f'Problemas con algún error: {sum(errores.values())} de 200')
for t, n in errores.most_common():
    print(f'  {n:3d}  {t}')
print('Paso de la cadena en el que aparece el primer error:',
      ', '.join(f'paso {k}: {v}' for k, v in sorted(posiciones.items())))
if ejemplo:
    pr, s, r = ejemplo
    i = next(k for k, (a, b) in enumerate(zip(s, r)) if a != b)
    print('\nEjemplo (primer error marcado con >>):')
    print('  correcto:', ' '.join(r[:i]), '>>', r[i], '...')
    print('  modelo:  ', ' '.join(s[:i]), '>>', s[i], '...')
```

> Salida del programa (extracto final):

```
...
=== ANÁLISIS DE ERRORES CON 8 PASOS (200 problemas) ===
Problemas con algún error: 5 de 200
```

```
    5  se equivoca de concepto (búsqueda del hecho)
Paso de la cadena en el que aparece el primer error: paso 7: 2, paso 8: 3

Ejemplo (primer error marcado con >>):
  correcto: la temperatura dificulta [−] la inferencia dificulta [+] la exploración complica
[−] los datos acelera [−] la normalización ralentiza [+] la precisión ralentiza [−] el
vocabulario dificulta [+] el preentrenamiento ralentiza [−] >> la memoria ...
  modelo:   la temperatura dificulta [−] la inferencia dificulta [+] la exploración complica
[−] los datos acelera [−] la normalización ralentiza [+] la precisión ralentiza [−] el
vocabulario dificulta [+] el preentrenamiento ralentiza [−] >> ? ...
```

Los cinco errores son del mismo tipo, y no son de verbo ni de signo: el modelo da por terminada la cadena antes de tiempo. En lugar del concepto siguiente escribe «?», el cierre que corresponde a la pregunta, como si el último concepto que ha escrito fuera ya el de la pregunta. Además, aparecen todos al final de cadenas largas, en los pasos 7 (dos) y 8 (tres), más largos que todos los que vio al entrenar. Es decir: lo que falla es reconocer cuándo se ha llegado al final, la búsqueda que enlaza con la pregunta. Es una pista concreta para mejorarlo (más relleno al entrenar, para que esas posiciones lejanas se practiquen, o más pasos, como en el Ejercicio 15.4), pero eso ya es una hipótesis que tendrías que comprobar.

Capítulo 16: Tu código mueve GPT-2 (***)

> **🎯 AL TERMINAR ESTE CAPÍTULO SABRÁS:**
>
> ✓ Cargar en el forward() del Programa 8 los 124 millones de pesos reales de GPT-2 small, leyendo a mano su archivo.
>
> ✓ Usar el tokenizador BPE real y medir por qué GPT-2 «habla» sobre todo inglés.
>
> ✓ Mirar dentro de un modelo real: la lente de logits, las cabezas de inducción y la cabeza «token anterior».
>
> ✓ Implementar y medir el KV-cache, la optimización que hace rápidos a los LLM.
>
> ✓ Ajustar GPT-2 con el backward del Programa 8 y responder a la pregunta del Capítulo 15: ¿qué le faltaba al Nano-LLM?

16.1 El mismo código, 124 millones de parámetros

En el Capítulo 13 construiste un GPT-2 en miniatura y en el Capítulo 15 descubriste su límite: con 5 KB de texto, memoriza. Este capítulo da el paso que lo cambia todo. En 2019 OpenAI publicó los pesos de GPT-2 small, el modelo que tu Nano-LLM imita, entrenado con unos 40 GB de texto de internet (Radford et al., 2019). Como tu Programa 8 tiene exactamente su arquitectura, puedes cargar esos pesos en **tu** función `forward()` y ver qué pasa.

No hace falta PyTorch, ni la librería transformers, ni una GPU: solo NumPy y los cuatro archivos del modelo. Lo que cambia respecto al Nano-LLM son los tamaños:

	Nano-LLM (Programa 8)	GPT-2 small (Programa 8 bis)
Parámetros	60 000	124 439 808
Bloques × cabezas	2 × 4	12 × 12
Dimensión D (por cabeza)	48 (12)	768 (64)
Capa oculta de la FFN	192	3072
Contexto	24 caracteres	1024 tokens
Vocabulario	46 caracteres	50 257 tokens BPE
Texto de entrenamiento	93 frases, unos 5 KB	unos 40 GB (WebText)
Archivo de pesos	0.49 MB (float64)	548 MB (float32)

Dentro de un archivo .safetensors

Los pesos se distribuyen en el formato **safetensors**, muy sencillo: 8 bytes con la longitud N de una cabecera; N bytes de cabecera en JSON, que dice para cada tensor su tipo, su forma y dónde están sus números; y, a continuación, todos los números seguidos. La función `leer_safetensors()` del programa lo lee en 15 líneas con `np.frombuffer`. El archivo de GPT-2 tiene 160 tensores: 148 de pesos y 12 que no son pesos sino la máscara causal de cada bloque (`h.k.attn.bias`), que el programa no necesita porque ya tiene la suya.

OpenAI usa otros nombres que el Programa 8, pero las piezas son las mismas. Esta es la correspondencia (k es el número de bloque, de 0 a 11):

> **📥 Antes de empezar: los pesos de GPT-2**
>
> Descarga los cuatro archivos de la página huggingface.co/openai-community/gpt2, pestaña «Files and versions»: `model.safetensors` (548 MB), `vocab.json`, `merges.txt` y `config.json`. Guárdalos en una carpeta llamada `gpt2`, junto a los programas de este capítulo. OpenAI publicó estos pesos con licencia

> MIT. No hace falta registrarse ni instalar nada más que NumPy (y matplotlib para la figura de 16.6).

Nombre en el archivo de GPT-2	Forma	Nombre en el Programa 8
wte.weight	50 257 × 768	wte (también la capa de salida)
wpe.weight	1024 × 768	wpe
h.k.ln_1.weight / .bias	768	ln1_gk / ln1_bk
h.k.attn.c_attn.weight	768 × 2304	Wqk, Wkk, Wvk: se corta en tres de 768 × 768
h.k.attn.c_attn.bias	2304	bqk, bkk, bvk
h.k.attn.c_proj.weight / .bias	768 × 768	Wok / bok
h.k.ln_2.weight / .bias	768	ln2_gk / ln2_bk
h.k.mlp.c_fc.weight / .bias	768 × 3072	W1k / b1k
h.k.mlp.c_proj.weight / .bias	3072 × 768	W2k / b2k
ln_f.weight / .bias	768	lnf_g / lnf_b
h.k.attn.bias	1 × 1 × 1024 × 1024	(no se usa: es la máscara causal)

Hay una suerte histórica: GPT-2 guarda sus matrices en la misma orientación que el Programa 8, para multiplicar `x @ W`, así que no hay que trasponer nada. Y comparte los pesos de entrada y de salida exactamente como el Nano-LLM (13.1): la última línea de `forward()`, `logits = xf @ params['wte'].T`, sirve tal cual.

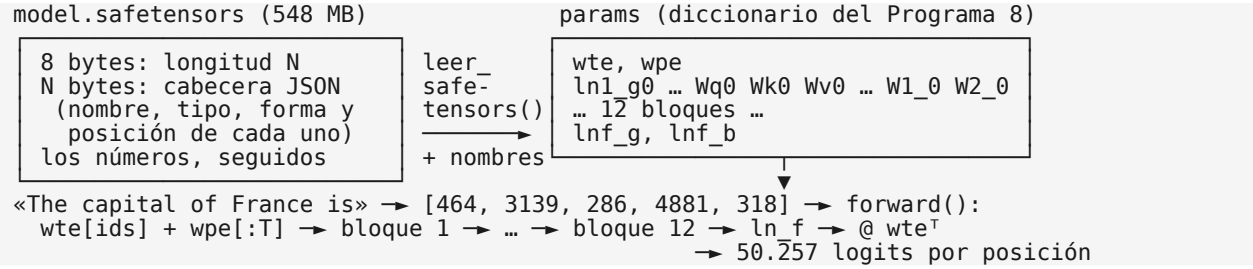


Diagrama 16.1 — Del archivo de OpenAI a tu forward(): los mismos nombres y las mismas operaciones del Programa 8.

El Programa 8 bis se imprime en cinco partes. Lee con atención la tercera: son las funciones del Programa 8.

PROGRAMA 8 bis — GPT-2 de verdad, con el forward del Programa 8

📄 `programa_8bis_gpt2_real.py`

📄 Parte 1 de 5 — leer el archivo safetensors

```
# ============================================================
# PROGRAMA 8 bis — GPT-2 de verdad, con el forward del Programa 8
# Carga los pesos REALES de GPT-2 small (124 millones de
# parámetros, OpenAI 2019) y los pasa por EXACTAMENTE la misma
# función forward() del Programa 8. Solo cambian los tamaños y
# el tokenizador (BPE en lugar de caracteres).
#
# Requisitos: NumPy y una carpeta gpt2/ junto a este archivo con
# los cuatro archivos de huggingface.co/openai-community/gpt2:
#   model.safetensors  (pesos, ~548 MB)
#   vocab.json, merges.txt  (tokenizador BPE)
#   config.json  (tamaños del modelo)
# No necesita PyTorch ni la librería transformers.
# ============================================================
import json
import re
import sys
import time
import numpy as np
```

```
sys.stdout.reconfigure(encoding='utf-8')   # para que Windows muestre «Ġ», «␣», ñ...

CARPETA = 'gpt2'

# -- 1. Leer los pesos: el formato safetensors, a mano ------------
# Un archivo .safetensors es muy simple:
#   8 bytes  -> N, la longitud de la cabecera (entero sin signo)
#   N bytes  -> cabecera en JSON: para cada tensor, su tipo, su
#               forma y en qué bytes del resto del archivo está
#   resto    -> los números de todos los tensores, uno detrás de otro
def leer_safetensors(ruta):
    with open(ruta, 'rb') as f:
        n = int.from_bytes(f.read(8), 'little')
        cabecera = json.loads(f.read(n))
        datos = f.read()
    tipos = {'F32': np.float32, 'F16': np.float16}
    tensores = {}
    for nombre, info in cabecera.items():
        if nombre == '__metadata__':
            continue
        ini, fin = info['data_offsets']
        t = np.frombuffer(datos[ini:fin], dtype=tipos[info['dtype']])
        tensores[nombre.removeprefix('transformer.')] = t.reshape(info['shape'])
    return tensores


t0 = time.time()
cfg = json.load(open(f'{CARPETA}/config.json'))
gpt2 = leer_safetensors(f'{CARPETA}/model.safetensors')
print(f'Pesos leídos en {time.time() - t0:.1f} s: {len(gpt2)} tensores')
```

La segunda parte fija los tamaños leyéndolos de `config.json` y construye el diccionario `params` con los nombres del Programa 8, siguiendo la tabla anterior. La última línea crea la máscara causal de 1024 × 1024.

📄 Parte 2 de 5 — tamaños y nombres del Programa 8

```
# -- 2. Los tamaños de GPT-2 small (en vez de los del Programa 8) --
N_BLQ = cfg['n_layer']          # 12 bloques     (Programa 8: 2)
H     = cfg['n_head']           # 12 cabezas     (Programa 8: 4)
D     = cfg['n_embd']           # 768            (Programa 8: 48)
CTX   = cfg['n_positions']      # 1024 tokens    (Programa 8: 24)
DH    = D // H                  # 64 por cabeza  (Programa 8: 12)
EPS   = cfg.get('layer_norm_epsilon', 1e-5)

# -- 3. Los pesos con los nombres del Programa 8 --------------------
# GPT-2 guarda Q, K y V juntas en una matriz c_attn de 768 × 2304;
# la cortamos en tres de 768 × 768. Sus matrices ya están en la
# orientación del Programa 8 (x @ W), así que no hay que trasponer.
params = {'wte': gpt2['wte.weight'], 'wpe': gpt2['wpe.weight'],
          'lnf_g': gpt2['ln_f.weight'], 'lnf_b': gpt2['ln_f.bias']}
for k in range(N_BLQ):
    b = f'h.{k}.'
    params[f'ln1_g{k}'], params[f'ln1_b{k}'] = gpt2[b + 'ln_1.weight'], gpt2[b + 'ln_1.bias']
    W, bias = gpt2[b + 'attn.c_attn.weight'], gpt2[b + 'attn.c_attn.bias']
    for i, n in enumerate('qkv'):
        params[f'W{n}{k}'] = W[:, i * D:(i + 1) * D]
        params[f'b{n}{k}'] = bias[i * D:(i + 1) * D]
    params[f'Wo{k}'], params[f'bo{k}'] = gpt2[b + 'attn.c_proj.weight'], gpt2[b +
'attn.c_proj.bias']
    params[f'ln2_g{k}'], params[f'ln2_b{k}'] = gpt2[b + 'ln_2.weight'], gpt2[b + 'ln_2.bias']
    params[f'W1{k}'], params[f'b1{k}'] = gpt2[b + 'mlp.c_fc.weight'], gpt2[b +
'mlp.c_fc.bias']
    params[f'W2{k}'], params[f'b2{k}'] = gpt2[b + 'mlp.c_proj.weight'], gpt2[b +
'mlp.c_proj.bias']
V = params['wte'].shape[0]
print(f'Vocabulario: {V} tokens | Bloques: {N_BLQ} | Cabezas: {H} | D: {D} | Contexto: {CTX}')
print(f'Total de parámetros: {sum(p.size for p in params.values()):,}')
MASCARA_CAUSAL = np.triu(np.full((CTX, CTX), -1e9, dtype=np.float32), k=1)
```

La tercera parte es el corazón del capítulo. `softmax`, `layer_norm_fwd`, `gelu`, `separar`, `unir` y `forward` son **las mismas instrucciones del Programa 8 (13.5.1)**, línea a línea. Solo hay tres diferencias, ninguna en las operaciones: el valor de `eps` se lee de la configuración (es el mismo, 10^{-5}), se han quitado los comentarios y `cache` guarda solo la salida de cada bloque y sus pesos de atención, que usaremos en 16.5 y 16.6 (lo demás solo lo necesita el backward, y con este tamaño ocuparía mucha memoria).

📄 Parte 3 de 5 — las funciones del Programa 8

```
# -- 4. Las funciones del Programa 8, sin cambiar una línea ---------
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return np.exp(x) / np.exp(x).sum(axis=axis, keepdims=True)
def layer_norm_fwd(x, g, b, eps=EPS):
    mu = x.mean(-1, keepdims=True)
    inv = 1.0 / np.sqrt(x.var(-1, keepdims=True) + eps)
    xhat = (x - mu) * inv
    return xhat * g + b, (xhat, inv)
C_GELU = np.sqrt(2 / np.pi)
def gelu(x):
    return 0.5 * x * (1 + np.tanh(C_GELU * (x + 0.044715 * x * x * x)))
def separar(x):   # (B, T, D) -> (B, H, T, DH)
    B, T, _ = x.shape
    return x.reshape(B, T, H, DH).transpose(0, 2, 1, 3)
def unir(x):      # (B, H, T, DH) -> (B, T, D)
    B, _, T, _ = x.shape
    return x.transpose(0, 2, 1, 3).reshape(B, T, D)

def forward(tokens):
    """tokens (B, T) -> logits (B, T, V). El mismo código del Programa 8."""
    B, T = tokens.shape
    X = params['wte'][tokens] + params['wpe'][:T]
    cache = []
    for k in range(N_BLQ):
        Xin = X
        a, ln1 = layer_norm_fwd(Xin, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        Q, K, Vm = (separar(a @ params[f'W{n}{k}'] + params[f'b{n}{k}']) for n in 'qkv')
        pesos = softmax(Q @ K.transpose(0, 1, 3, 2) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])
        att = unir(pesos @ Vm)
        X1 = Xin + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, ln2 = layer_norm_fwd(X1, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        h_pre = m @ params[f'W1{k}'] + params[f'b1{k}']
        h = gelu(h_pre)
        X = X1 + h @ params[f'W2{k}'] + params[f'b2{k}']
        cache.append(dict(X=X, pesos=pesos))   # salida y atención de cada bloque (16.5, 16.6)
    xf, lnf = layer_norm_fwd(X, params['lnf_g'], params['lnf_b'])
    logits = xf @ params['wte'].T          # pesos compartidos, como en el Programa 8
    return logits, (xf, lnf, cache)

def top_p_filtrar(probs, p):
    indices_ord = np.argsort(probs)[::-1]
    suma_acum = np.cumsum(probs[indices_ord])
    n_tokens = int(np.searchsorted(suma_acum, p)) + 1
    nueva = np.zeros_like(probs)
    nueva[indices_ord[:n_tokens]] = probs[indices_ord[:n_tokens]]
    return nueva / nueva.sum()
```

La cuarta parte es el tokenizador BPE de GPT-2 (Capítulo 9), con sus 50 000 fusiones, y generar(), la misma función del Programa 8 adaptada a tokens. La tabla de bytes a caracteres, bytes_a_unicode(), y la expresión regular del troceo reproducen las del tokenizador que OpenAI publicó con GPT-2 (src/encoder.py del repositorio openai/gpt-2, © 2019 OpenAI, licencia MIT): tienen que ser idénticas para que cada token reciba su número de vocab.json.

📄 Parte 4 de 5 — el tokenizador BPE y generar()

```
# -- 5. El tokenizador BPE de GPT-2 (Capítulo 9) ----------------
# GPT-2 trabaja con BYTES: cada uno de los 256 valores posibles de
# un byte se representa con un carácter imprimible (el espacio, por
# ejemplo, se convierte en «Ġ»). Sobre esos caracteres se aplican,
# por orden, las 50 000 fusiones aprendidas que hay en merges.txt.
def bytes_a_unicode():
    bs = (list(range(ord('!'), ord('~') + 1)) + list(range(ord('¡'), ord('¬') + 1))
          + list(range(ord('®'), ord('ÿ') + 1)))
    cs = bs[:]
    n = 0
    for b in range(256):
        if b not in bs:
            bs.append(b)
            cs.append(256 + n)
            n += 1
    return dict(zip(bs, map(chr, cs)))

B2U = bytes_a_unicode()
U2B = {u: b for b, u in B2U.items()}
VOCAB = json.load(open(f'{CARPETA}/vocab.json', encoding='utf-8'))
ID2TOK = {i: t for t, i in VOCAB.items()}
lineas = open(f'{CARPETA}/merges.txt', encoding='utf-8').read().split('\n')
```

```
RANGO = {tuple(l.split()): i for i, l in enumerate(l for l in lineas[1:] if l.strip())}
# Cómo trocea GPT-2 el texto antes de aplicar BPE (palabras con su
# espacio delante, números, signos, espacios). Versión con el módulo
# re estándar de Python: [^\W\d_] son las letras.
TROCEO = re.compile(r"""'s|'t|'re|'ve|'m|'ll|'d| ?[^\W\d_]+| ?\d+| ?(?:[^\s\w]|_)+|\s+(?!\S)|\
s+""")

def bpe(palabra):
    """Aplica las fusiones de merges.txt, siempre la de menor rango primero."""
    partes = list(palabra)
    while len(partes) > 1:
        pares = [(RANGO.get((a, b), 1e18), i) for i, (a, b) in enumerate(zip(partes,
partes[1:]))]
        rango, i = min(pares)
        if rango == 1e18:
            break
        partes = partes[:i] + [partes[i] + partes[i + 1]] + partes[i + 2:]
    return partes

def codificar(texto):
    ids = []
    for trozo in TROCEO.findall(texto):
        palabra = ''.join(B2U[b] for b in trozo.encode('utf-8'))
        ids += [VOCAB[p] for p in bpe(palabra)]
    return ids

def decodificar(ids):
    texto = ''.join(ID2TOK[i] for i in ids)
    return bytes(U2B[c] for c in texto).decode('utf-8', errors='replace')

def generar(semilla, n_tokens=40, temperatura=0.8, top_p=0.9):
    """Como generar() del Programa 8, pero con tokens BPE.
    temperatura=0 -> greedy (siempre el más probable)."""
    ids = codificar(semilla)
    for _ in range(n_tokens):
        logits = forward(np.array([ids[-CTX:]]))[0][0, -1].astype(np.float64)
        if temperatura == 0:
            siguiente = int(np.argmax(logits))
        else:
            probs = softmax(logits / temperatura)
            if top_p < 1.0:
                probs = top_p_filtrar(probs, top_p)
            siguiente = int(np.random.choice(V, p=probs))
        ids.append(siguiente)
    return decodificar(ids)
```

La quinta parte son los experimentos que comentaremos en las tres secciones siguientes.

📄 Parte 5 de 5 — experimentos

```
# ============================================================
# EXPERIMENTOS
# ============================================================
print('\n=== 1. EL TOKENIZADOR BPE (Ġ = espacio delante de la palabra) ===')
for frase in ['Hello world', 'The capital of France is',
              'la inteligencia artificial aprende de los datos.']:
    ids = codificar(frase)
    print(f'{frase!r}: {len(frase)} caracteres -> {len(ids)} tokens')
    print('   ', [ID2TOK[i] for i in ids])
    print('   ', ids)

print('\n=== 2. ¿QUÉ TOKEN VIENE DESPUÉS? (las 5 opciones más probables) ===')
for frase in ['The capital of France is', 'Paris is the capital of',
              'The cat sat on the', 'One, two, three, four,']:
    t0 = time.time()
    logits = forward(np.array([codificar(frase)]))[0][0, -1].astype(np.float64)
    probs = softmax(logits)
    print(f'{frase!r}   ({time.time() - t0:.2f} s)')
    for i in np.argsort(probs)[::-1][:5]:
        print(f'    {decodificar([i]).replace(" ", "␣").replace(chr(10), "↵"):>12s}
{probs[i]:.3f}')

print('\n=== 3. PÉRDIDA Y PERPLEJIDAD EN UN TEXTO QUE NO ESTÁ EN SU CORPUS ===')
texto = ('Neural networks learn by adjusting their weights. During training, the network makes
a '
         'prediction, measures its error with a loss function, and uses backpropagation to
compute '
         'how each weight should change to reduce that error.')
ids = codificar(texto)
logits = forward(np.array([ids]))[0][0].astype(np.float64)
probs = softmax(logits[:-1])
perdida = -np.mean(np.log(probs[np.arange(len(ids) - 1), ids[1:]]))
print(f'{len(ids)} tokens | pérdida media = {perdida:.3f} | perplejidad =
{np.exp(perdida):.1f}')
```

```
print(f'Referencia «al azar»: ln({V}) = {np.log(V):.2f}, perplejidad {V}')

print('\n=== 4. GENERACIÓN GREEDY (comprobación) ===')
t0 = time.time()
print(generar('I enjoy walking with my cute dog', n_tokens=30, temperatura=0))
print(f'({time.time() - t0:.1f} s)')

print('\n=== 5. GENERACIÓN CON TEMPERATURA 0.8 Y TOP-P 0.9 ===')
np.random.seed(42)
for semilla in ['The meaning of life is', 'In a shocking finding, scientists discovered',
                'La inteligencia artificial es']:
    print(f'\n--- {semilla!r} ---')
    print(generar(semilla, n_tokens=40))
```

> Salida del programa (inicio):

```
Pesos leídos en 5.2 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
```

La cuenta sale exacta: **124 439 808 parámetros**, la cifra oficial de GPT-2 small, y la misma que da la función del Ejercicio 12.1 con sus tamaños. La salida completa se comenta por partes en las tres secciones siguientes. El programa entero tardó 1 minuto y 43 segundos en el ordenador de pruebas (2 núcleos; con un solo hilo de cálculo, 1 minuto y 59 segundos).

> **✅ Comprobado tres veces**
>
> Antes de confiar en lo que imprime, el programa se comprobó contra la implementación oficial de Hugging Face (librería transformers, con PyTorch). Los logits de nuestro `forward()` coinciden con los de `GPT2LMHeadModel` con una diferencia máxima de $8 \cdot 10^{-5}$ en float32 y de $1.6 \cdot 10^{-12}$ en float64; el tokenizador da los mismos tokens que `GPT2TokenizerFast` en 50 textos de prueba (inglés, español, números, código, emojis); y la continuación greedy de «I enjoy walking with my cute dog» es, palabra por palabra, la que publica Hugging Face en su guía de generación de texto. Los dos programas de comprobación están en el repositorio del libro, en la carpeta `verificaciones`.

16.2 El tokenizador de verdad

La primera parte de la salida aplica el tokenizador BPE del Capítulo 9, ahora con las 50 000 fusiones que OpenAI aprendió de su texto. «Ġ» es la forma en que GPT-2 escribe el espacio que va delante de una palabra:

> Salida del Programa 8 bis (parte 1):

```
=== 1. EL TOKENIZADOR BPE (Ġ = espacio delante de la palabra) ===
'Hello world': 11 caracteres -> 2 tokens
    ['Hello', 'Ġworld']
    [15496, 995]
'The capital of France is': 24 caracteres -> 5 tokens
    ['The', 'Ġcapital', 'Ġof', 'ĠFrance', 'Ġis']
    [464, 3139, 286, 4881, 318]
'la inteligencia artificial aprende de los datos.': 48 caracteres -> 13 tokens
    ['la', 'Ġintel', 'igen', 'cia', 'Ġartificial', 'Ġap', 'rend', 'e', 'Ġde', 'Ġlos', 'Ġdat',
'os', '.']
    [5031, 33649, 9324, 33743, 11666, 2471, 10920, 68, 390, 22346, 4818, 418, 13]
```

«Hello world» son dos tokens, [15496, 995]: los mismos números que da el tokenizador oficial. En inglés, casi cada palabra es un token. En español no: «inteligencia» se parte en tres piezas («Ġintel», «igen», «cia»), «aprende» en otras tres y «datos» en dos. La frase española gasta 13 tokens para 48 caracteres; la inglesa, 5 para 24.

No es un capricho: BPE fusiona los pares más frecuentes de su texto de entrenamiento, y el de GPT-2 era casi todo inglés. Las palabras inglesas frecuentes se convirtieron en un token; las españolas se quedaron a medio fusionar. En el Ejercicio 16.1 medirás la diferencia en cinco idiomas: 1.2 tokens por palabra en inglés, 2 en español y 8 en griego. Y eso tiene

consecuencias prácticas: más tokens significa menos texto en el mismo contexto, más cálculo y, en las API de pago, más coste.

16.3 Primeras predicciones

La segunda parte de la salida es lo que ocurre en la última posición de `forward()`: las cinco continuaciones más probables, después de la softmax, para cuatro frases:

```
> Salida del Programa 8 bis (parte 2):
=== 2. ¿QUÉ TOKEN VIENE DESPUÉS? (las 5 opciones más probables) ===
'The capital of France is'   (0.40 s)
            ␣the  0.085
            ␣now  0.048
              ␣a  0.046
          ␣France  0.032
           ␣Paris  0.032
'Paris is the capital of'   (0.41 s)
            ␣the  0.257
              ␣a  0.070
          ␣France  0.041
          ␣Europe  0.024
         ␣Germany  0.020
'The cat sat on the'   (0.40 s)
           ␣floor  0.076
             ␣bed  0.065
           ␣couch  0.054
          ␣ground  0.052
            ␣edge  0.048
'One, two, three, four,'   (0.95 s)
            ␣five  0.742
             ␣ten  0.026
             ␣six  0.019
             ␣and  0.018
            ␣four  0.014
```

Mira la primera. Tras «The capital of France is», la opción favorita no es « Paris» sino « the» (0.085), seguida de « now» y « a». « Paris» aparece en quinto lugar, con un 3.2 %. No es un error del programa: es GPT-2 small. Un modelo de 124 millones de parámetros sabe que tras «is» suele venir un artículo, y reparte la probabilidad entre muchas continuaciones gramaticales («the largest city…», «now…», «a city…»). En 16.5 miraremos dentro del modelo y veremos que « Paris» sí llega a estar en segundo lugar… y que los últimos bloques lo hacen bajar.

Las otras tres frases salen mejor. Tras «The cat sat on the», las cinco opciones son superficies donde puede sentarse un gato. Y tras «One, two, three, four,», « five» acapara el 74 %: el patrón de contar está muy presente en su texto de entrenamiento.

Perplejidad y bits por carácter

La tercera parte de la salida mide la pérdida en un párrafo inglés escrito para este libro, que por tanto no estaba en el texto de entrenamiento de GPT-2:

```
> Salida del Programa 8 bis (parte 3):
=== 3. PÉRDIDA Y PERPLEJIDAD EN UN TEXTO QUE NO ESTÁ EN SU CORPUS ===
44 tokens | pérdida media = 3.534 | perplejidad = 34.3
Referencia «al azar»: ln(50257) = 10.82, perplejidad 50257
```

Una perplejidad de 34.3 significa que, de media, GPT-2 duda como si eligiera entre 34 tokens igual de probables, cuando podría dudar entre 50 257. ¿Es mejor o peor que el Nano-LLM, cuya perplejidad en frases nuevas era de unos 65 (15.2)? No se puede saber así: el Nano-LLM predice **caracteres** y GPT-2 predice **tokens**, que en este párrafo miden de media 5.2 caracteres. Adivinar un token es mucho más difícil que adivinar un carácter.

La unidad común es la pérdida **por carácter**: se suma la pérdida de todos los tokens predichos (en nats) y se divide entre el número de caracteres que ocupan. Dividiendo

además entre ln 2 se pasa de nats a **bits**: cuántas preguntas de sí o no harían falta, de media, para adivinar cada carácter. Este fragmento hace la cuenta:

📄 **fragmento_9q3_bits.py**

```
# Fragmento 16.3 — Pasar de nats por token a bits por carácter
import numpy as np
fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])

texto = ('Neural networks learn by adjusting their weights. During training, the network makes
a '
         'prediction, measures its error with a loss function, and uses backpropagation to
compute '
         'how each weight should change to reduce that error.')
ids = codificar(texto)
lp = np.log(softmax(forward(np.array([ids]))[0][0, :-1].astype(np.float64)))
nats = -lp[np.arange(len(ids) - 1), ids[1:]].sum()       # suma de las 43 pérdidas
chars = len(texto) - len(decodificar(ids[:1]))           # caracteres predichos
print(f'{len(ids) - 1} tokens predichos, {chars} caracteres')
print(f'{nats / (len(ids) - 1):.3f} nats/token  =  {nats / chars:.3f} nats/carácter  =  '
      f'{nats / chars / np.log(2):.2f} bits/carácter')
print(f'Un token ocupa de media {chars / (len(ids) - 1):.1f} caracteres')
```

> Salida:

```
Pesos leídos en 1.0 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
43 tokens predichos, 225 caracteres
3.534 nats/token  =  0.675 nats/carácter  =  0.97 bits/carácter
Un token ocupa de media 5.2 caracteres
```

GPT-2 necesita **0.97 bits por carácter** en un párrafo inglés que no ha visto nunca: menos de una pregunta de sí o no por carácter. Con el mismo método, el Programa G-4 (16.8) mide 2.82 bits por carácter en las 10 frases españolas que el Programa RZ-1 apartó para validar. GPT-2 se defiende bastante peor en español que en inglés (aunque no es una comparación limpia: cambian el texto, su longitud y el contexto disponible, y los bits por carácter solo igualan la unidad de medida), y aun así prácticamente iguala, sin haber visto ni una frase del corpus del Capítulo 13, al Nano-LLM en su mejor momento: 1.923 nats por carácter, que son 2.77 bits. Al final de su entrenamiento, el Nano-LLM estaba en 6.02.

16.4 Generar texto: los bucles del greedy, por fin visibles

En el Capítulo 14 viste que la decodificación greedy, que elige siempre el token más probable, tiende a caer en bucles. Con el Nano-LLM no se notaba, porque había memorizado su corpus y simplemente lo recitaba. GPT-2 no tiene un corpus que recitar, y el bucle aparece en cuanto se le deja:

> Salida del Programa 8 bis (parte 4):

```
=== 4. GENERACIÓN GREEDY (comprobación) ===
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with my dog.
I'm not sure if I'll ever be able to walk
(19.4 s)
```

«I'm not sure if I'll ever be able to walk with my dog.» y vuelta a empezar. Es la continuación que publica Hugging Face para este mismo modelo, y el ejemplo clásico de por qué nadie genera texto con greedy puro. La quinta parte de la salida usa el muestreo del Programa 8, con temperatura 0.8 y top-p 0.9:

> Salida del Programa 8 bis (parte 5):

```
=== 5. GENERACIÓN CON TEMPERATURA 0.8 Y TOP-P 0.9 ===

--- 'The meaning of life is' ---
The meaning of life is and always will be a question of importance to us, both those who are
here and those who are not. I am not willing to give up the hope of a lifetime that will be
the exact opposite

--- 'In a shocking finding, scientists discovered' ---
In a shocking finding, scientists discovered that the U.S. has the highest rate of autism
prevalence among children younger than 5.
```

```
The findings are in the latest medical journal Neurology, a peer-reviewed journal.


--- 'La inteligencia artificial es' ---
La inteligencia artificial español, de 20 minografico, el vento desempos de cœuriales del
consejo

cœuriales del consejo
```

En inglés, el texto es fluido y variado: una reflexión sobre el sentido de la vida, una noticia inventada con apariencia de verdad. Fíjate en la segunda: cita una revista médica real, Neurology, y un dato sobre el autismo que el modelo no ha comprobado en ningún sitio. Es una **alucinación** en estado puro: GPT-2 escribe lo que es probable, no lo que es cierto. Y en español, con un tokenizador y un texto de entrenamiento casi solo en inglés, produce una mezcla de palabras inventadas, francés e italiano.

La otra herramienta del Capítulo 14 contra los bucles era la penalización por repetición. Este fragmento la añade al greedy con la regla del Programa S-2, copiada tal cual (los logits de los últimos 20 tokens se dividen entre 1.3 si son positivos, y se multiplican si son negativos):

📄 fragmento_9q4_repeticion.py

```
# Fragmento 16.4 — Greedy con y sin penalización por repetición
# La penalización es la del Programa S-2 (14.3), copiada tal cual.
import numpy as np
fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])


def greedy(semilla, n=40, rep_pen=1.0):
    tokens = codificar(semilla)
    generados = list(tokens)
    for _ in range(n):
        logits = forward(np.array([tokens]))[0][0, -1].astype(np.float64)
        if rep_pen != 1.0:                                  # como en el Programa S-2
            for t in set(generados[-20:]):
                if logits[t] > 0: logits[t] /= rep_pen
                else:             logits[t] *= rep_pen
        siguiente = int(np.argmax(logits))
        tokens.append(siguiente)
        generados.append(siguiente)
    return decodificar(tokens)


for pen in [1.0, 1.3]:
    print(f'--- repetition penalty = {pen} ---')
    print(greedy('I enjoy walking with my cute dog', rep_pen=pen).replace('\n', ' / '))
```

> Salida:

```
Pesos leídos en 1.0 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
--- repetition penalty = 1.0 ---
I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with my dog.
I'm not sure if I'll ever be able to walk with my dog. /  / I'm not sure
--- repetition penalty = 1.3 ---
I enjoy walking with my cute dog, but I'm not sure if it's because of the dogs or just a lack.
/ I've been to many dog shows and I'm always impressed by the dogs that come out of my house
```

Sin penalización, el bucle aparece igual que en la salida del programa y no se rompe: la misma frase una y otra vez. Con una penalización de 1.3, el greedy escapa del bucle en cuanto los tokens repetidos pierden fuerza, y el texto avanza: habla de los perros, de exposiciones caninas... aunque termine con un disparate («the dogs that come out of my house»). La penalización evita repetir, no garantiza tener sentido. Por eso los sistemas reales combinan varias estrategias del Capítulo 14, como hace `generar_avanzado()` del Programa S-2.

16.5 Mirar dentro (I): la lente de logits

En GPT-2, como en el Nano-LLM, la vía residual (12.2) tiene siempre la misma forma: 768 números por token. Cada bloque lee esos números y les **suma** algo. Al final, la LayerNorm final y el producto por wte^T los convierten en 50 257 logits.

En 2020, un investigador que firma como nostalgebraist tuvo una idea muy simple: aplicar esa misma conversión final a la vía residual **después de cada bloque**, como si el modelo terminara ahí. La llamó **lente de logits** (logit lens). Muestra cómo se va formando la predicción capa a capa, y con lo que ya tienes cabe en 15 líneas.

PROGRAMA G-1 — La lente de logits: ver pensar a GPT-2

📄 programa_G1_lente_logits.py

```
# ============================================================
# PROGRAMA G-1 — La lente de logits: ver pensar a GPT-2
# Capítulo 16, sección 16.5
#
# La vía residual de GPT-2 tiene siempre 768 números por token.
# Al final, la LayerNorm final y wteᵀ la convierten en una
# predicción. ¿Y si aplicamos esa misma conversión a la salida
# de CADA bloque, no solo del último? Veremos cómo se va formando
# la predicción capa a capa (nostalgebraist, 2020).
# Requisitos: programa_8bis_gpt2_real.py y la carpeta gpt2/.
# ============================================================
import numpy as np

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])     # pesos, tokenizador y forward()


def lente(frase, objetivo):
    """Predicción tras cada bloque para el token que sigue a `frase`."""
    ids = codificar(frase)
    obj = codificar(objetivo)[0]
    _, (_, _, cache) = forward(np.array([ids]))
    # La entrada del bloque 1 (embeddings) y la salida de cada bloque
    salidas = [params['wte'][ids] + params['wpe'][:len(ids)]] + [c['X'][0] for c in cache]
    print(f'\n«{frase}» → ¿«{objetivo}»?')
    print('  capa          más probable        prob.   P(objetivo)  puesto')
    for k, X in enumerate(salidas):
        xf, _ = layer_norm_fwd(X[-1], params['lnf_g'], params['lnf_b'])   # última posición
        probs = softmax((xf @ params['wte'].T).astype(np.float64))
        mejor = int(np.argmax(probs))
        puesto = int((probs > probs[obj]).sum()) + 1
        capa = 'embeddings' if k == 0 else f'bloque {k:2d}'
        print(f'  {capa:12s}  {decodificar([mejor])!r:18s}  {probs[mejor]:.3f}   '
              f'{probs[obj]:.4f}     {puesto:>6d}')


lente('The capital of France is', ' Paris')
lente('The Eiffel Tower is located in the city of', ' Paris')
lente('One, two, three, four,', ' five')
lente('I enjoy walking with my cute', ' dog')
```

> Salida del programa (unos 25 segundos):

```
Pesos leídos en 1.3 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808

«The capital of France is» → ¿« Paris»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' destro'           0.533   0.0000      21416
  bloque  1     ' not'              0.332   0.0000      16524
  bloque  2     ' now'              0.271   0.0000      18483
  bloque  3     ' now'              0.366   0.0000      13624
  bloque  4     ' now'              0.515   0.0000       8649
  bloque  5     ' now'              0.627   0.0000       3106
  bloque  6     ' now'              0.552   0.0000       3040
  bloque  7     ' now'              0.572   0.0001        200
  bloque  8     ' now'              0.689   0.0000        275
  bloque  9     ' now'              0.423   0.0006         52
  bloque 10     ' France'           0.621   0.1819          2
  bloque 11     ' France'           0.242   0.1391          2
  bloque 12     ' the'              0.085   0.0322          5
```

```
«The Eiffel Tower is located in the city of» → ¿« Paris»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' destro'           0.670   0.0000      22947
  bloque  1     ' the'              0.707   0.0000       6060
  bloque  2     ' the'              0.815   0.0000       5618
  bloque  3     ' the'              0.713   0.0000       2104
  bloque  4     ' the'              0.660   0.0000        838
  bloque  5     ' the'              0.318   0.0001        449
  bloque  6     ' England'          0.055   0.0004        255
  bloque  7     ' East'             0.064   0.0009        145
  bloque  8     ' Ing'              0.230   0.0013         96
  bloque  9     ' Rome'             0.168   0.0150         13
  bloque 10     ' London'           0.248   0.1935          2
  bloque 11     ' Paris'            0.165   0.1649          1
  bloque 12     ' Paris'            0.064   0.0638          1

«One, two, three, four,» → ¿« five»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' mathemat'         0.763   0.0000      14028
  bloque  1     ' which'            0.148   0.0002        563
  bloque  2     ' which'            0.122   0.0002        397
  bloque  3     ' etc'              0.265   0.0003        162
  bloque  4     ' etc'              0.925   0.0000        178
  bloque  5     ' etc'              0.806   0.0000        260
  bloque  6     ' etc'              0.992   0.0000        315
  bloque  7     ' etc'              0.987   0.0000         79
  bloque  8     ' etc'              0.993   0.0000         21
  bloque  9     ' etc'              0.994   0.0001         11
  bloque 10     ' five'             0.976   0.9762          1
  bloque 11     ' five'             0.970   0.9695          1
  bloque 12     ' five'             0.742   0.7417          1

«I enjoy walking with my cute» → ¿« dog»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' cute'             1.000   0.0000       2811
  bloque  1     ' cute'             0.722   0.0002        131
  bloque  2     ' cute'             0.365   0.0001        161
  bloque  3     ' little'           0.483   0.0002        171
  bloque  4     ' little'           0.585   0.0002        157
  bloque  5     ' little'           0.595   0.0008         38
  bloque  6     ' little'           0.590   0.0013         28
  bloque  7     ' girl'             0.399   0.0023         28
  bloque  8     ' girl'             0.329   0.0019         26
  bloque  9     ' friends'          0.609   0.0018         23
  bloque 10     ' little'           0.380   0.0021         22
  bloque 11     ' little'           0.307   0.0139          9
  bloque 12     ' little'           0.424   0.0620          2
```

Qué se ve a través de la lente

- **«The capital of France is».** Durante nueve bloques el modelo apuesta por « now» («…is now…»). En el bloque 10 cambia de golpe: « France» con 0.62 y « Paris» segundo, con 0.18, unas seis veces más que al final. Los dos últimos bloques reparten la probabilidad y dejan a « Paris» en quinto lugar. A media altura, la lente ya encuentra « Paris» entre los candidatos, y las capas finales lo relegan. Es una lectura de las representaciones intermedias, no una prueba de que el modelo «sepa» la respuesta y la descarte: para afirmarlo habría que intervenir, por ejemplo anulando esas capas y midiendo el efecto.
- **«The Eiffel Tower is located in the city of».** Aquí se ve una búsqueda: « the» hasta el bloque 5, luego países y regiones (« England», « East»), luego ciudades (« Rome» en el bloque 9, « London» en el 10) y, en el 11, « Paris», que ya no suelta. Es tentador leerlo como «primero el tipo de respuesta, después el lugar exacto», pero la lente solo muestra qué token sería el más probable si el modelo terminara ahí; no dice qué calcula cada bloque.
- **«One, two, three, four,».** Del bloque 4 al 9 el modelo está seguro, al 99 %, de « etc»: la enumeración sigue. En el bloque 10 aparece « five» con un 97.6 %, y ya no cambia.

- **«I enjoy walking with my cute».** « dog» nunca llega a ser el primero: termina segundo, con un 6 %, por detrás de « little». Una frase con muchas continuaciones razonables se resuelve tarde y sin seguridad.

Un patrón se repite en las cuatro frases: la respuesta aparece en los últimos bloques, sobre todo en el 10, y no antes. Los primeros bloques de un transformer no «adivinan» nada aún; preparan información. La fila de los embeddings, en cambio, no dice nada útil: con «cute» la lente predice la propia palabra de entrada (su vector se parece sobre todo a sí mismo), y con las demás, tokens sin relación. La lente funciona porque cada bloque suma una corrección pequeña a la misma vía residual, que conserva su significado de bloque en bloque.

> ℹ️ **Para saber más**
>
> nostalgebraist (2020), «interpreting GPT: the logit lens», publicado en LessWrong, presentó la técnica precisamente con GPT-2. Más tarde se refinó entrenando una pequeña capa por bloque para traducir mejor la vía residual (la «tuned lens», Belrose et al., 2023). En el Ejercicio 16.2 la aplicarás a tus propias frases.

16.6 Mirar dentro (II): cabezas de inducción en un modelo real

En el Capítulo 15, el Programa RZ-3 solo aprendió a encadenar hechos cuando le dimos hecho, con `emb_prev`, lo que un transformer profundo aprende por sí solo: un circuito de dos cabezas llamado **cabeza de inducción**. Una cabeza copia en cada posición qué token tiene delante; otra usa esa información para buscar «dónde apareció antes el token actual» y copiar el que vino después. ¿Lo tiene GPT-2?

El experimento es tan simple como ingenioso. Se eligen 50 tokens al azar, que no forman ninguna frase, y se repiten dos veces. La primera copia es imposible de predecir. La segunda es trivial… si el modelo se da cuenta de que la secuencia se está repitiendo. Después se mide, para cada una de las 144 cabezas, cuánta atención presta cada token de la segunda copia al token que **siguió** a su aparición anterior. Si una cabeza hace eso, es una cabeza de inducción.

PROGRAMA G-2 — Cabezas de inducción en GPT-2 real

📄 `programa_G2_induccion.py`

```
# ============================================================
# PROGRAMA G-2 — Cabezas de inducción en GPT-2 real
# Capítulo 16, sección 16.6
#
# Una secuencia de 50 tokens elegidos al azar no tiene ningún
# sentido: GPT-2 no puede predecirla... la primera vez. Si la
# repetimos, ¿aprovecha lo que acaba de ver? Medimos la pérdida
# en las dos copias y buscamos, entre sus 144 cabezas (12 bloques
# × 12), las que miran «al token que vino después de la vez
# anterior»: las cabezas de inducción (Olsson et al., 2022).
# Requisitos: programa_8bis_gpt2_real.py, la carpeta gpt2/ y
# matplotlib (para la figura).
# ============================================================
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])     # pesos, tokenizador y forward()

N = 50
rng = np.random.RandomState(0)
R = list(rng.randint(1000, 30000, size=N))  # 50 tokens al azar
ids = [50256] + R + R                       # <|endoftext|>, copia 1, copia 2
print('Secuencia (principio):', decodificar(R[:12]).replace('\n', ' '), '...')

logits, (_, _, cache) = forward(np.array([ids]))
```

```
logp = np.log(softmax(logits[0, :-1].astype(np.float64)))
perdida = -logp[np.arange(len(ids) - 1), ids[1:]]      # pérdida de cada predicción
acierto = logp.argmax(-1) == np.array(ids[1:])
# La predicción en la posición i es del token i+1. Copia 1: tokens 2..50;
# copia 2: tokens 52..100 (el primero de cada copia no se puede deducir).
c1, c2 = slice(1, N), slice(N + 1, 2 * N)
print(f'\nPérdida media en la 1.ª copia: {perdida[c1].mean():.2f}   aciertos:
{acierto[c1].mean():.0%}')
print(f'Pérdida media en la 2.ª copia: {perdida[c2].mean():.2f}   aciertos:
{acierto[c2].mean():.0%}')
print(f'Referencia «al azar»: ln({V}) = {np.log(V):.2f}')

# Puntuaciones por cabeza. En la 2.ª copia, la posición i contiene el mismo
# token que la i − N; el que vino después de aquel está en la i − N + 1.
pos = np.arange(N + 2, 2 * N + 1)
induccion = np.array([[c['pesos'][0, h, pos, pos - N + 1].mean() for h in range(H)] for c in
cache])
anterior = np.array([[c['pesos'][0, h, np.arange(1, 2 * N + 1), np.arange(0, 2 * N)].mean()
                      for h in range(H)] for c in cache])
print('\nCabezas de inducción (atención media al token que siguió a la vez anterior):')
for k, h in sorted(np.ndindex(N_BLQ, H), key=lambda kh: -induccion[kh])[:5]:
    print(f'   bloque {k + 1:2d}, cabeza {h + 1:2d}: {induccion[k, h]:.2f}')
print('Cabezas «token anterior» (atención media al token inmediatamente anterior):')
for k, h in sorted(np.ndindex(N_BLQ, H), key=lambda kh: -anterior[kh])[:3]:
    print(f'   bloque {k + 1:2d}, cabeza {h + 1:2d}: {anterior[k, h]:.2f}')
print(f'Media de las {N_BLQ * H} cabezas: inducción {induccion.mean():.2f}, '
      f'token anterior {anterior.mean():.2f}')

# Figura: pérdida por posición y mapa de la mejor cabeza de inducción
kb, hb = np.unravel_index(induccion.argmax(), induccion.shape)
# Letra grande y negra: la figura se imprime a 17 cm de ancho
plt.rcParams.update({'font.size': 17, 'axes.titlesize': 18, 'text.color': 'black',
'axes.labelcolor': 'black',
                     'xtick.color': 'black', 'ytick.color': 'black', 'axes.edgecolor':
'black'})
fig, (a1, a2) = plt.subplots(1, 2, figsize=(13, 5.8), dpi=150, layout='constrained')
a1.plot(np.arange(1, 2 * N + 1), perdida, color='#1A3A6B', lw=1.5)
a1.axvline(N + 0.5, color='#D97706', ls='--', lw=1.5)
a1.set_ylim(-0.5, perdida.max() * 1.15)
a1.text(N * 0.5, perdida.max() * 1.07, '1.ª copia', ha='center')
a1.text(N * 1.5, perdida.max() * 1.07, '2.ª copia', ha='center')
a1.set_xlabel('posición del token predicho'); a1.set_ylabel('pérdida (nats)')
a1.set_title('GPT-2 ante 50 tokens al azar, repetidos', fontweight='bold')
im = a2.imshow(cache[kb]['pesos'][0, hb], cmap='Blues', vmin=0, vmax=1)
a2.set_title(f'Atención del bloque {kb + 1}, cabeza {hb + 1}', fontweight='bold')
a2.set_xlabel('key: posición a la que se mira'); a2.set_ylabel('query: posición que pregunta')
fig.colorbar(im, ax=a2, shrink=0.8, label='peso de atención')
plt.savefig('figura_9q_induccion.png')
print(f'\nFigura guardada en figura_9q_induccion.png (bloque {kb + 1}, cabeza {hb + 1})')
```

> Salida del programa (unos segundos):

```
Pesos leídos en 1.1 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
Secuencia (principio): innppers cul Quad zones 512 corporations invari lonely
realization~~~~~~~~ Whatever ...

Pérdida media en la 1.ª copia: 12.87   aciertos: 0%
Pérdida media en la 2.ª copia: 0.26   aciertos: 98%
Referencia «al azar»: ln(50257) = 10.82

Cabezas de inducción (atención media al token que siguió a la vez anterior):
   bloque  6, cabeza  6: 0.95
   bloque  7, cabeza 10: 0.94
   bloque  8, cabeza 11: 0.93
   bloque  6, cabeza  2: 0.90
   bloque  8, cabeza  3: 0.85
Cabezas «token anterior» (atención media al token inmediatamente anterior):
   bloque  5, cabeza 12: 0.99
   bloque  4, cabeza  8: 0.52
   bloque  7, cabeza  9: 0.50
Media de las 144 cabezas: inducción 0.09, token anterior 0.10

Figura guardada en figura_9q_induccion.png (bloque 6, cabeza 6)
```

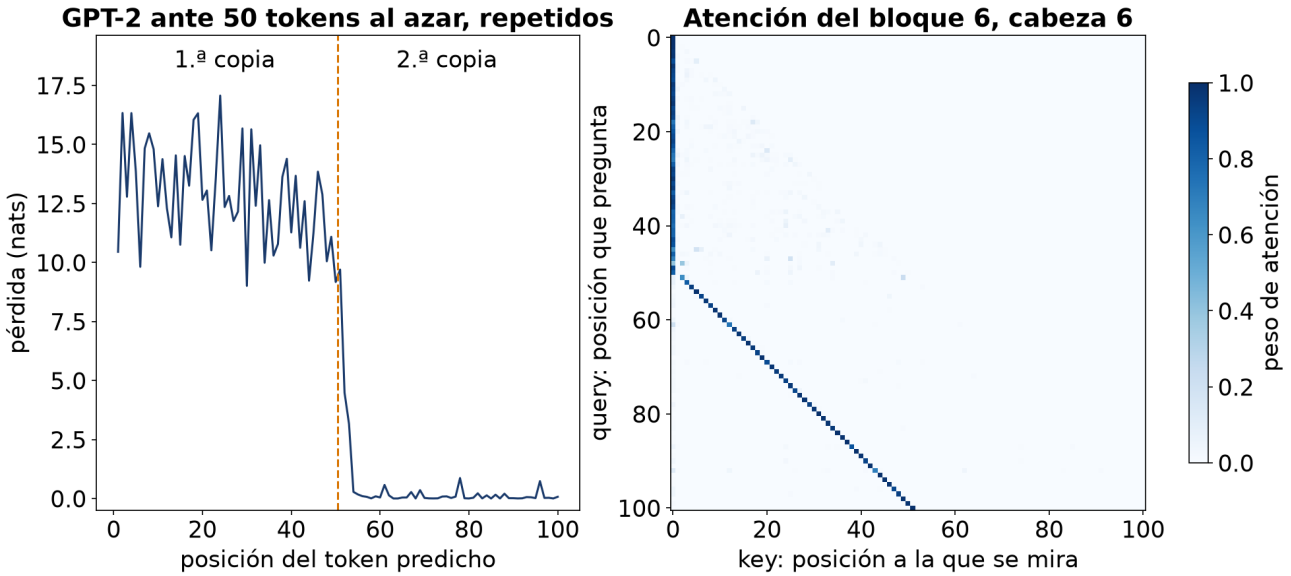


Diagrama 16.2 — Izquierda: pérdida de GPT-2 en cada posición de la secuencia repetida. Derecha: atención del bloque 6, cabeza 6; cada fila es una posición que pregunta y cada columna, una a la que puede mirar.

Qué dicen las cifras y la figura

En la primera copia, la pérdida media es de 12.87 nats, **peor** que repartir la probabilidad por igual entre los 50 257 tokens (10.82): GPT-2 espera inglés, y una secuencia al azar le parece muy improbable. En la segunda copia baja a 0.26 y acierta el 98 % de los tokens. En la figura se ve el desplome justo después de la línea naranja: nadie le ha enseñado a hacer esto con secuencias al azar, pero en cuanto el patrón se repite, lo aprovecha. Es el **aprendizaje en contexto** más elemental: el modelo aprende algo de su propio prompt, sin cambiar un solo peso.

El mapa de la derecha muestra la cabeza que mejor encaja con ese patrón (comprobar que es la causa exigiría anularla y medir el efecto). En la segunda copia (filas 51 a 100), cada posición mira, casi con todo su peso, a la posición que está 49 lugares antes: la que siguió a la aparición anterior del mismo token. Es la diagonal desplazada. En la primera copia, donde no hay nada que copiar, la cabeza deja su atención en la primera columna, el token de inicio: es su forma de «no mirar a ningún sitio».

Cinco cabezas pasan de 0.85, en los bloques 6, 7 y 8; la media de las 144 es 0.09. Y la cabeza «token anterior» que el circuito necesita está justo antes: el bloque 5, cabeza 12, mira al token inmediatamente anterior con un peso medio de 0.99. Es exactamente la pieza que `emb_prev` le daba hecha al Programa RZ-3, y GPT-2 la aprendió solo, en el sitio que el circuito exige: antes que las cabezas de inducción que la usan.

> **ℹ️ Para saber más**
>
> En los artículos de interpretabilidad, bloques y cabezas se numeran desde 0. Allí, las cabezas de este programa son la 5.5, la 6.9 y la 7.10 (inducción) y la 4.11 (token anterior): son las mismas que se han descrito en GPT-2 small. Las cabezas de inducción se estudiaron primero en modelos pequeños de solo atención (Elhage et al., 2021) y después en modelos de lenguaje completos, donde Olsson et al. (2022) las relacionaron con el aprendizaje en contexto.

16.7 Más rápido: el KV-cache, implementado y medido

La salida de 16.4 delata un problema: generar 30 tokens con greedy tardó 19.4 segundos. La razón está en `generar()`: para cada token nuevo llama a `forward()` con la secuencia **entera**, y recalcula las claves (K) y los valores (V) de todos los tokens anteriores, que no han cambiado. Con la máscara causal, la K y la V de un token dependen solo de él y de los anteriores: se pueden calcular una vez y guardar. Es el **KV-cache** que describe el Capítulo 17, y que usan todos los LLM.

El Programa G-3 lo implementa en dos funciones. `prefill()` procesa la semilla entera de una vez, como `forward()`, y guarda sus K y V; `forward_un_token()` procesa solo el token nuevo: calcula su q, su k y su v, guarda k y v, y compara q con todas las K guardadas. No necesita máscara, porque en el cache solo existe el pasado. El cache es exacto porque GPT-2 usa posiciones absolutas y la semilla más los tokens generados caben en sus 1024 posiciones; si se superaran, habría que recortar y el cache dejaría de servir tal cual.

PROGRAMA G-3 — El KV-cache, implementado y medido

📄 programa_G3_kvcache.py

📄 Parte 1 de 3 — el cache, un token y el prefill

```
# ============================================================
# PROGRAMA G-3 — El KV-cache, implementado y medido
# Capítulo 16, sección 16.7
#
# generar() del Programa 8 bis recalcula, para cada token nuevo,
# el forward de TODA la secuencia. Pero con la máscara causal las
# claves (K) y los valores (V) de los tokens anteriores no cambian
# nunca: se pueden guardar y reutilizar. Así, cada paso solo
# procesa UN token. Es el «KV-cache» que usan todos los LLM.
# Requisitos: programa_8bis_gpt2_real.py y la carpeta gpt2/.
# ============================================================
import time
import numpy as np

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])     # pesos, tokenizador, forward() y generar()


def nuevo_cache():
    """Espacio para las K y V de hasta CTX tokens en cada bloque: (H, CTX, DH)."""
    return [{'K': np.zeros((H, CTX, DH), np.float32), 'V': np.zeros((H, CTX, DH), np.float32)}
            for _ in range(N_BLQ)]


def forward_un_token(token, t, kv):
    """Logits del token que sigue al token `token`, que está en la posición t.
    Usa las K y V guardadas de las posiciones 0..t-1 y guarda las suyas."""
    x = params['wte'][token] + params['wpe'][t]                     # (D,)
    for k in range(N_BLQ):
        a, _ = layer_norm_fwd(x, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
        q, kk, v = ((a @ params[f'W{n}{k}'] + params[f'b{n}{k}']).reshape(H, DH) for n in
'qkv')
        kv[k]['K'][:, t], kv[k]['V'][:, t] = kk, v                  # se guardan para siempre
        K, Vm = kv[k]['K'][:, :t + 1], kv[k]['V'][:, :t + 1]        # (H, t+1, DH)
        pesos = softmax(np.einsum('hd,htd->ht', q, K) / np.sqrt(DH))  # (H, t+1): sin máscara:
        att = np.einsum('ht,htd->hd', pesos, Vm).reshape(D)         # solo existe el pasado
        x = x + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, _ = layer_norm_fwd(x, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        x = x + gelu(m @ params[f'W1{k}'] + params[f'b1{k}']) @ params[f'W2{k}'] +
params[f'b2{k}']
    xf, _ = layer_norm_fwd(x, params['lnf_g'], params['lnf_b'])
    return xf @ params['wte'].T                                      # (V,)


def prefill(ids, kv):
    """Procesa la semilla ENTERA de una vez (como forward) y guarda sus K y V.
    Los LLM reales lo hacen así: «prefill» de la pregunta y después «decode»,
    token a token, de la respuesta."""
    T = len(ids)
    X = params['wte'][ids] + params['wpe'][:T]                       # (T, D)
    for k in range(N_BLQ):
        a, _ = layer_norm_fwd(X, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
```

```
        Q, K, Vm = ((a @ params[f'W{n}{k}'] + params[f'b{n}{k}']).reshape(T, H,
DH).transpose(1, 0, 2)
                    for n in 'qkv')                                    # (H, T, DH)
        kv[k]['K'][:, :T], kv[k]['V'][:, :T] = K, Vm
        pesos = softmax(Q @ K.transpose(0, 2, 1) / np.sqrt(DH) + MASCARA_CAUSAL[:T, :T])
        att = (pesos @ Vm).transpose(1, 0, 2).reshape(T, D)
        X = X + att @ params[f'Wo{k}'] + params[f'bo{k}']
        m, _ = layer_norm_fwd(X, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
        X = X + gelu(m @ params[f'W1{k}'] + params[f'b1{k}']) @ params[f'W2{k}'] +
params[f'b2{k}']
    xf, _ = layer_norm_fwd(X[-1], params['lnf_g'], params['lnf_b'])
    return xf @ params['wte'].T                                         # logits del último
```

Las dos funciones de generación hacen lo mismo, greedy, de dos formas: un forward completo por token o un prefill y después un token cada vez.

📄 Parte 2 de 3 — generar con y sin cache

```
def generar_sin_cache(ids, n_tokens):
    """Greedy como generar() del Programa 8 bis: un forward completo por token."""
    ids = list(ids)
    for _ in range(n_tokens):
        ids.append(int(np.argmax(forward(np.array([ids[-CTX:]]))[0][0, -1])))
    return ids


def generar_con_cache(ids, n_tokens):
    """Greedy con KV-cache: prefill de la semilla y después un token cada vez."""
    if len(ids) + n_tokens > CTX:                  # el cache tiene CTX posiciones
        raise ValueError(f'semilla + tokens nuevos = {len(ids) + n_tokens} > {CTX}')
    ids, kv = list(ids), nuevo_cache()
    logits = prefill(ids, kv)
    for k in range(n_tokens):
        ids.append(int(np.argmax(logits)))
        if k < n_tokens - 1:                       # el último token no necesita otro paso
            logits = forward_un_token(ids[-1], len(ids) - 1, kv)
    return ids
```

La tercera parte comprueba primero que el cache da los mismos logits que el forward completo, posición a posición, y después mide el tiempo con semillas de 50, 200 y 800 tokens.

📄 Parte 3 de 3 — comprobación y medida

```
# 1. ¿Da exactamente lo mismo que el forward completo?
ids = codificar('The quick brown fox jumps over the lazy dog because it')
kv, dif = nuevo_cache(), 0.0
completo = forward(np.array([ids]))[0][0]
for t, tok in enumerate(ids):
    dif = max(dif, np.abs(forward_un_token(tok, t, kv) - completo[t]).max())
dif = max(dif, np.abs(prefill(ids, nuevo_cache()) - completo[-1]).max())
print(f'Diferencia máxima entre los logits con y sin cache: {dif:.1e} '
      f'(los logits valen hasta {np.abs(completo).max():.0f})')

# 2. ¿Cuánto ahorra? Depende de la longitud del contexto.
texto = open('programa_8_nano_llm.py', encoding='utf-8').read().split('TEXTO = """')
[1].split('"""')[0]
largo = codificar(texto * 3)                    # un texto largo del que cortar contextos
N_GEN = 10
print(f'\nTiempo por token generado (greedy, {N_GEN} tokens), según la longitud del
contexto:')
print(' contexto   sin cache   con cache   ahorro   mismos tokens')
for L in (50, 200, 800):
    semilla = largo[:L]
    t0 = time.time(); a = generar_sin_cache(semilla, N_GEN); t_sin = (time.time() - t0) /
N_GEN
    t0 = time.time(); b = generar_con_cache(semilla, N_GEN); t_con = (time.time() - t0) /
N_GEN
    print(f'{L:7d}   {t_sin:7.2f} s   {t_con:7.2f} s   {t_sin / t_con:5.1f}x   '
          f'{"sí" if a == b else "NO"}')
print('(el tiempo «con cache» incluye el prefill de la semilla, repartido entre los tokens
generados)')
```

> Salida del programa (casi dos minutos):

```
Pesos leídos en 1.1 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
Diferencia máxima entre los logits con y sin cache: 6.7e-06 (los logits valen hasta 164)
```

```
Tiempo por token generado (greedy, 10 tokens), según la longitud del contexto:
 contexto   sin cache   con cache   ahorro   mismos tokens
     50      0.97 s      0.62 s     1.6x   sí
    200      1.39 s      0.62 s     2.2x   sí
    800      5.27 s      0.94 s     5.6x   sí
(el tiempo «con cache» incluye el prefill de la semilla, repartido entre los tokens generados)
```

Primero, la corrección: los logits coinciden con una diferencia máxima de $7 \cdot 10^{-6}$ cuando valen hasta 164, el redondeo de float32, y los tokens generados son idénticos en los tres casos.

Después, el ahorro, que **crece con el contexto**: 1.6 veces con 50 tokens de semilla, 2.2 con 200 y 5.6 con 800. Sin cache, cada token nuevo vuelve a procesar toda la secuencia, así que su coste crece con su longitud: de 0.97 a 5.27 segundos. Con cache, cada token procesa solo uno, y su coste apenas cambia (0.62 s; con 800 tokens, 0.94 s, porque incluye el prefill y la atención sobre 800 claves). ¿Y por qué un solo token tarda 0.62 segundos? Leer 124 millones de pesos de la memoria no debería costar tanto. En el Ejercicio 16.5 encontrarás al culpable, un detalle de los tipos de NumPy que también está en el Programa 8, y verás que, corrigiéndolo, el mismo token con cache tarda unos 22 milisegundos. Es una lección que los ingenieros de LLM conocen bien: al generar, el coste ya no está en hacer cuentas, sino en mover datos.

16.8 La respuesta al Capítulo 15: ¿qué le faltaba al Nano-LLM?

Volvamos a la pregunta que dejó abierta el Capítulo 15. El Nano-LLM, entrenado con 83 frases, se aprendió esas frases y fracasó con las 10 apartadas: en ellas, su pérdida acabó en 4.175 nats por carácter, peor que el azar. Su mejor momento, en el paso 750, fue 1.923. La hipótesis natural es que le faltaba **preentrenamiento**: haber leído antes muchísimo texto, para que 83 frases no fueran todo lo que sabe del idioma. Ahora podemos contrastarla, no suponerla.

El Programa G-4 toma GPT-2, que ya leyó 40 GB de texto, y lo **ajusta** (fine-tuning) con las mismas 83 frases, separadas exactamente igual que en el Programa RZ-1. Y lo hace con tu backward: ejecuta las funciones del Programa 8, `forward_backward()` incluida, sin cambiar una línea. Funcionan con cualquier tamaño, porque leen H, DH, D y N_BLQ de las variables globales, que ahora son las de GPT-2. El entrenamiento es el del Programa 8: Adam, warm-up y coseno, y recorte del gradiente; con una tasa de aprendizaje mucho menor (10^{-4}), porque no se trata de aprender desde cero sino de corregir un poco un modelo que ya sabe mucho.

Para comparar con el Nano-LLM, la pérdida se mide en bits por carácter (16.3) sobre los mismos caracteres: cada frase se predice a partir del salto de línea que la precede, incluido el salto final.

> **✅ Tu backward, comprobado con GPT-2**
>
> Antes de entrenar, el backward del Programa 8 se comprobó con los pesos de GPT-2 contra el de PyTorch (autograd de la librería transformers). En float64, el error relativo de los gradientes está entre $4 \cdot 10^{-15}$ y $3 \cdot 10^{-13}$ en los tensores comprobados (wte, wpe, Wq0, Wv5, W1 del bloque 11, ln1_g0 y lnf_b): el mismo cálculo, con la precisión de la máquina. Ni una derivada del Capítulo 13 ha tenido que cambiar para un modelo 2000 veces mayor.

PROGRAMA G-4 — Ajuste fino de GPT-2 con TU backward

📄 programa_G4_ajuste_fino.py

📄 Parte 1 de 3 — GPT-2 y el backward del Programa 8

```
# ============================================================
# PROGRAMA G-4 — Ajuste fino de GPT-2 con TU backward
# Capítulo 16, sección 16.8
#
# En el Capítulo 15, el Nano-LLM entrenado con 83 frases no supo nada
# de las 10 frases apartadas: memorizaba. ¿Qué le faltaba? Aquí
# tomamos GPT-2, que ya «leyó» 40 GB de texto (preentrenamiento),
# y lo ajustamos con las MISMAS 83 frases usando la función
# forward_backward() del Programa 8, sin cambiar una línea. Luego
# medimos las 10 frases apartadas y comparamos.
# Requisitos: programa_8bis_gpt2_real.py, programa_8_nano_llm.py,
# la carpeta gpt2/ y unos 3 GB de memoria libre.
# ============================================================
import sys
import time
import numpy as np

PASOS = int(sys.argv[1]) if len(sys.argv) > 1 else 200

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])     # pesos de GPT-2, tokenizador y generar()

# -- El backward del Programa 8, tal cual -------------------------
# Del Programa 8 tomamos sus funciones (desde «Utilidades» hasta
# antes del bucle de entrenamiento). Funcionan con cualquier tamaño:
# usan las variables globales H, DH, D, N_BLQ, MASCARA_CAUSAL y
# params, que ahora son las de GPT-2.
p8 = open('programa_8_nano_llm.py', encoding='utf-8').read()
exec('# -- Utilidades' + p8.split('# -- Utilidades')[1].split('# -- Learning rate')[0])
# Los pesos leídos del archivo son de solo lectura y algunas matrices
# son trozos de otras (Q, K, V): para entrenar hacen falta copias propias.
params = {k: np.array(v, dtype=np.float32) for k, v in params.items()}
del gpt2          # los pesos originales ya no hacen falta: se libera memoria
```

La segunda parte prepara los datos de RZ-1 (las 83 frases de entrenamiento, que en BPE son 1618 tokens) y la medida en bits por carácter, y la aplica antes de ajustar.

📄 Parte 2 de 3 — los datos de RZ-1 y la medida

```
# -- Los mismos datos que el Programa RZ-1 ------------------------
TEXTO = p8.split('TEXTO = """')[1].split('"""')[0]
FRASES = TEXTO.strip().split('\n')
VAL_IDX = list(range(4, len(FRASES), 9))
VAL = [FRASES[i] for i in VAL_IDX]
TRAIN = [f for i, f in enumerate(FRASES) if i not in VAL_IDX]
DATOS = np.array(codificar('\n' + '\n'.join(TRAIN) + '\n'))
print(f'Entrenamiento: {len(TRAIN)} frases = {len(DATOS)} tokens BPE | validación: {len(VAL)}
frases')


def bits_por_caracter(frases):
    """Pérdida en bits por carácter, comparable con la del Nano-LLM: se
    predice cada frase y su salto de línea final a partir del salto inicial."""
    nats, chars = 0.0, 0
    for f in frases:
        ids = [50256] + codificar('\n' + f + '\n')
        lp = np.log(softmax(forward(np.array([ids]))[0][0, :-1].astype(np.float64)))
        nats -= lp[np.arange(1, len(ids) - 1), ids[2:]].sum()   # tras el '\n' inicial
        chars += len(f) + 1
    return nats / chars / np.log(2)


print(f'\nAntes del ajuste:   entrenamiento {bits_por_caracter(TRAIN):.2f} | '
      f'validación {bits_por_caracter(VAL):.2f} bits/carácter')
np.random.seed(0)
print('Muestra antes:', generar('la atención permite', n_tokens=25).replace('\n', ' / '))
```

La tercera parte es el bucle de entrenamiento del Programa 8 con mini-lotes de 4 trozos de 64 tokens, y al final vuelve a medir y genera texto con tres semillas.

📄 Parte 3 de 3 — el ajuste fino

```
# -- Ajuste fino: Adam, warm-up + coseno y recorte, como el Programa 8
BATCH, T, LR, CLIP = 4, 64, 1e-4, 1.0
m_adam = {k: np.zeros_like(v) for k, v in params.items()}
v_adam = {k: np.zeros_like(v) for k, v in params.items()}
np.random.seed(42)
t0 = time.time()
for paso in range(PASOS):
    lr = LR * min(1.0, (paso + 1) / 20) * 0.5 * (1 + np.cos(np.pi * paso / PASOS))
    inicios = np.random.randint(0, len(DATOS) - T - 1, size=BATCH)
    xb = np.stack([DATOS[i:i + T] for i in inicios])
    yb = np.stack([DATOS[i + 1:i + T + 1] for i in inicios])
    grads, loss = forward_backward(xb, yb)
    norma = np.sqrt(sum(float(np.dot(g.ravel(), g.ravel())) for g in grads.values()))
    if norma > CLIP:
        for k in grads:
            grads[k] *= CLIP / norma
    for k in params:
        m_adam[k] = 0.9 * m_adam[k] + 0.1 * grads[k]
        v_adam[k] = 0.999 * v_adam[k] + 0.001 * grads[k] ** 2
        params[k] -= lr * (m_adam[k] / (1 - 0.9 ** (paso + 1))) / (
            np.sqrt(v_adam[k] / (1 - 0.999 ** (paso + 1))) + 1e-8)
    if (paso + 1) % 25 == 0 or paso == 0:
        print(f'Paso {paso + 1:4d}: pérdida del lote = {loss:.3f} nats/token   norma =
{norma:5.2f}   '
              f'validación = {bits_por_caracter(VAL):.2f} bits/carácter   ({time.time() -
t0:.0f} s)',
              flush=True)

print(f'\nDespués del ajuste: entrenamiento {bits_por_caracter(TRAIN):.2f} | '
      f'validación {bits_por_caracter(VAL):.2f} bits/carácter')
print('Nano-LLM (Programa RZ-1), validación: mejor momento 1.923 nats = '
      f'{1.923 / np.log(2):.2f} bits; al final 4.175 nats = {4.175 / np.log(2):.2f} bits')
np.random.seed(0)
for semilla in ['la atención permite', 'la función softmax', 'un modelo']:
    print(f'\n--- {semilla!r} ---')
    print(generar(semilla, n_tokens=30))
```

> Salida del programa (200 pasos; unos 30 minutos):

```
Pesos leídos en 1.6 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
Entrenamiento: 83 frases = 1618 tokens BPE | validación: 10 frases

Antes del ajuste:   entrenamiento 2.85 | validación 2.82 bits/carácter
Muestra antes: la atención permite de la broix de la confluence des champs de la mesa, a los
informaciones en vechas
Paso    1: pérdida del lote = 4.676 nats/token   norma = 18.34   validación = 2.81
bits/carácter   (20 s)
Paso   25: pérdida del lote = 2.667 nats/token   norma = 17.52   validación = 2.05
bits/carácter   (217 s)
Paso   50: pérdida del lote = 0.840 nats/token   norma = 11.39   validación = 2.39
bits/carácter   (414 s)
Paso   75: pérdida del lote = 0.572 nats/token   norma =  8.08   validación = 2.68
bits/carácter   (618 s)
Paso  100: pérdida del lote = 0.157 nats/token   norma =  3.02   validación = 2.80
bits/carácter   (815 s)
Paso  125: pérdida del lote = 0.216 nats/token   norma =  4.10   validación = 2.82
bits/carácter   (1016 s)
Paso  150: pérdida del lote = 0.133 nats/token   norma =  3.10   validación = 2.85
bits/carácter   (1215 s)
Paso  175: pérdida del lote = 0.151 nats/token   norma =  2.95   validación = 2.91
bits/carácter   (1415 s)
Paso  200: pérdida del lote = 0.144 nats/token   norma =  3.09   validación = 2.92
bits/carácter   (1609 s)

Después del ajuste: entrenamiento 0.24 | validación 2.92 bits/carácter
Nano-LLM (Programa RZ-1), validación: mejor momento 1.923 nats = 2.77 bits; al final 4.175
nats = 6.02 bits

--- 'la atención permite' ---
la atención permite responder a datos nuevos.
comprender estos conceptos abre la puerta a la ia moderna

--- 'la función softmax' ---
la función softmaxima cuántos datos caben en una gráfica.
un acelerador gráfico multiplica matrices

--- 'un modelo' ---
un modelo profundo usa muchas capas o una gráfica.
el aprendizaje por refuerzo premia las
```

Qué ha pasado

Antes de ajustar, GPT-2 ya está en 2.82 bits por carácter en las frases de validación, como el Nano-LLM en su mejor momento (2.77), y su español es una mezcla de idiomas («de la broix de la confluence des champs»).

Tras 25 pasos, la validación baja a **2.05**: un 26 % menos que el mejor momento del Nano-LLM. Es la respuesta a la pregunta del Capítulo 15: con lo que aprendió leyendo 40 GB de inglés, GPT-2 aprovecha 83 frases para predecir mejor frases **que no ha visto**, algo que el Nano-LLM no consiguió en ningún momento. El resultado apoya la hipótesis del preentrenamiento, aunque no la aísla del todo: GPT-2 se diferencia del Nano-LLM también en tamaño, tokenizador y contexto.

Pero el experimento no termina ahí. Desde el paso 25 la validación sube (2.39, 2.68, 2.80…) mientras la pérdida del lote cae hasta 0.14 nats por token y la de entrenamiento queda en 0.24 bits por carácter. En 200 pasos de 4 × 64 tokens el modelo ve 51 200 tokens, unas 32 pasadas por las 1618 del texto de entrenamiento: GPT-2 también **memoriza**. Y los textos generados lo demuestran: «la generalización permite responder a datos nuevos.», «comprender estos conceptos abre la puerta a la ia moderna», «el aprendizaje por refuerzo premia las…» son frases, o trozos de frases, del texto de entrenamiento. Es la misma conducta del Nano-LLM en el Capítulo 15.

La diferencia está en cómo termina. El Nano-LLM acabó en 6.02 bits por carácter, peor que el azar; GPT-2, en 2.92, prácticamente donde empezó. En este experimento, la pérdida de validación final quedó cerca de la inicial: en ese conjunto, lo que sabía actuó como un suelo. Saber si conserva también otras capacidades exigiría evaluarlas. No es una garantía general: con más pasos o un lr mayor, un modelo preentrenado también puede olvidar mucho. Así que la respuesta completa tiene dos partes. Al Nano-LLM le faltaba **preentrenamiento**, y con él el mismo ajuste generaliza. Pero el preentrenamiento no vacuna contra el sobreajuste: con tan pocos datos, sigue haciendo falta **parar a tiempo** (early stopping, 7.8), vigilando la validación como hizo el Programa RZ-1.

¿Y si se entrena solo 25 pasos? Ejecutando `python programa_G4_ajuste_fino.py 25` el decaimiento coseno termina mucho antes, y el resultado es de **2.25 bits por carácter en validación y 1.96 en entrenamiento**: el modelo generaliza (las dos cifras están cerca), pero aún no escribe bien («la atención permite en español.»). Su validación es peor que la del paso 25 de la ejecución larga (2.05), porque allí la tasa de aprendizaje seguía alta. Encontrar el punto justo entre aprender y memorizar, con 83 frases, es delicado: medir la validación más a menudo es la forma de encontrarlo.

> ⏱️ **Tiempo y memoria**
>
> El ajuste de 200 pasos tardó 29 minutos y 48 segundos en el ordenador de pruebas (2 núcleos), unos 8 segundos por paso más las medidas de validación, y llegó a ocupar 4.6 GB de memoria: los pesos, sus gradientes y los dos momentos de Adam son cuatro copias de 124 millones de números, más las activaciones que guarda el backward. Con menos memoria, se pueden congelar wte y wpe (no actualizarlas) o reducir BATCH a 2. Si no quieres esperar media hora, la versión de 25 pasos tarda unos 6 minutos y medio.

> 🏁 **Lo que acabas de hacer**
>
> • Has leído a mano el archivo de pesos de GPT-2 y lo has hecho funcionar con el forward que escribiste en el Capítulo 13: 124 millones de parámetros, ni una operación nueva.
>
> • Has visto cómo un tokenizador entrenado en inglés trocea el español, y por qué eso importa.
>
> • Has mirado dentro de un LLM real: la predicción formándose bloque a bloque y el circuito de inducción, con la cabeza «token anterior» que en el Capítulo 15 tuviste que dar hecha.

- Has implementado el KV-cache y has medido por qué su ahorro crece con el contexto.
- Has ajustado GPT-2 con tu propio backward y has contrastado la hipótesis del preentrenamiento con datos.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- El forward() del Programa 8 ejecuta, sin cambiar sus operaciones, los 124 439 808 parámetros reales de GPT-2 small; basta leer su archivo safetensors y cambiar los nombres de los tensores.
- Nuestro código coincide con la implementación oficial: logits con diferencias menores que 10^{-4} en float32, tokenizador idéntico y la misma continuación greedy publicada por Hugging Face.
- El tokenizador de GPT-2 gasta 1.2 tokens por palabra en inglés, 2 en español y 8 en griego: un LLM «habla» mejor los idiomas de su texto de entrenamiento.
- Para comparar modelos con tokens distintos se usan los bits por carácter: GPT-2 está en 0.97 en inglés y en 2.82 en las frases de validación en español.
- La lente de logits muestra que la predicción se forma en los últimos bloques; el experimento de los tokens repetidos revela cabezas de inducción en los bloques 6 a 8 y una cabeza «token anterior» en el 5.
- El KV-cache da los mismos logits y su ahorro crece con el contexto (5.6 veces con 800 tokens). Al generar, el coste está en mover los pesos, no en calcular.
- Ajustado con las 83 frases del Capítulo 15 y nuestro backward, GPT-2 baja a 2.05 bits por carácter en frases nuevas, frente a 2.77 del Nano-LLM en su mejor momento. Si se sigue entrenando, también memoriza, pero no se hunde: el preentrenamiento es lo que faltaba, y parar a tiempo sigue siendo necesario.

✏️ EJERCICIOS — Capítulo 16 — Tu código mueve GPT-2

Resuelve los ejercicios siguientes antes de continuar. Todos necesitan la carpeta gpt2/ junto a los programas, y todas las salidas que se muestran proceden de ejecuciones reales de las soluciones.

Ejercicio 16.1 Tokens por palabra en cinco idiomas

Escribe la misma frase en inglés, español, alemán, griego y chino y cuenta cuántos tokens BPE gasta GPT-2 en cada una. Calcula los tokens por palabra (en chino, por carácter) y muestra las piezas del inglés y del español.

> 💡 **Pista**
>
> Para usar solo el tokenizador no hace falta leer los 548 MB de pesos: ejecuta con `exec` la parte del Programa 8 bis que va de `def bytes_a_unicode` a `def generar`.

✅ Solución:

📄 ejercicio_16_1.py

```
# EJERCICIO 16.1 — ¿Cuántos tokens gasta GPT-2 en cada idioma?
# Solo hace falta el tokenizador: se carga la parte del Programa 8 bis
# que va desde el tokenizador hasta los experimentos (sin leer los pesos).
import json, re
import numpy as np

CARPETA = 'gpt2'
fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec('def bytes_a_unicode' + fuente.split('def bytes_a_unicode')[1].split('def generar')[0])

frases = {'inglés':  'Artificial intelligence learns from data and predicts the next word.',
          'español': 'La inteligencia artificial aprende de los datos y predice la siguiente
palabra.',
          'alemán':  'Künstliche Intelligenz lernt aus Daten und sagt das nächste Wort
voraus.',
          'griego':  'Η τεχνητή νοημοσύνη μαθαίνει από δεδομένα και προβλέπει την επόμενη
λέξη.',
          'chino':   '人工智能从数据中学习并预测下一个词。'}
print(f"{'idioma':8s} {'caract.':>8s} {'palabras':>9s} {'tokens':>7s}
{'tokens/palabra':>15s}")
for idioma, f in frases.items():
    ids = codificar(f)
    # en chino no hay espacios: contamos cada carácter (sin el punto) como una palabra
    palabras = len(f.split()) if idioma != 'chino' else len(f) - 1
    print(f'{idioma:8s} {len(f):8d} {palabras:9d} {len(ids):7d} {len(ids) / palabras:15.1f}')
for idioma in ['inglés', 'español']:
    print(f'\nPiezas del {idioma}:', [ID2TOK[i].replace("Ġ", "␣") for i in
codificar(frases[idioma])])
```

> Salida del programa:

```
idioma    caract.  palabras  tokens  tokens/palabra
inglés         68        10      12             1.2
español        79        12      24             2.0
alemán         72        11      31             2.8
griego         73        11      88             8.0
chino          18        17      35             2.1

Piezas del inglés: ['Art', 'ificial', '␣intelligence', '␣learns', '␣from', '␣data', '␣and',
'␣predicts', '␣the', '␣next', '␣word', '.']

Piezas del español: ['La', '␣intel', 'igen', 'cia', '␣artificial', '␣ap', 'rend', 'e', '␣de',
'␣los', '␣dat', 'os', '␣y', '␣pred', 'ice', '␣la', '␣sig', 'u', 'ient', 'e', '␣pal', 'ab',
'ra', '.']
```

En inglés, la frase gasta 12 tokens para 10 palabras; en español, 24 para 12; en alemán, 31 para 11; en griego, 88 para 11, porque cada letra griega ocupa dos bytes y pocas combinaciones llegaron a fusionarse; en chino, unos 2 tokens por carácter. Las piezas lo explican: «intelligence» es un solo token, mientras que «inteligencia» se queda en tres y

«siguiente» en cuatro. La misma idea cuesta el doble de tokens en español que en inglés, y por tanto el doble de cálculo y la mitad de contexto.

Ejercicio 16.2 La lente de logits con tus frases

Aplica la lente del Programa G-1 a dos frases tuyas cuya continuación sea clara, por ejemplo «The opposite of hot is» → « cold». ¿En qué bloque aparece la respuesta? ¿Se repite el patrón de 16.5?

> 💡 **Pista**
>
> Ejecuta el Programa G-1 sin sus cuatro llamadas finales (`fuente.split("\nlente('")[0]`) y llama a `lente()` con tus frases. El objetivo debe empezar por espacio, porque así lo tokeniza GPT-2.

✅ Solución:

📄 ejercicio_16_2.py

```
# EJERCICIO 16.2 — La lente de logits con frases propias
# Reutiliza la función lente() del Programa G-1.
import numpy as np

fuente = open('programa_G1_lente_logits.py', encoding='utf-8').read()
exec(fuente.split("\nlente('")[0])          # todo menos las llamadas finales

lente('The opposite of hot is', ' cold')
lente('Monday, Tuesday,', ' Wednesday')
```

> Salida del programa:

```
Pesos leídos en 1.1 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808

«The opposite of hot is» → ¿« cold»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' destro'           0.533   0.0000       8950
  bloque  1     ' not'              0.289   0.0000       3647
  bloque  2     ' not'              0.211   0.0000       3931
  bloque  3     ' now'              0.275   0.0000       4137
  bloque  4     ' now'              0.447   0.0000       4714
  bloque  5     ' now'              0.518   0.0000       2673
  bloque  6     ' now'              0.314   0.0000       3524
  bloque  7     ' not'              0.230   0.0000       1347
  bloque  8     ' also'             0.170   0.0000        508
  bloque  9     ' cold'             0.171   0.1711          1
  bloque 10     ' cooler'           0.396   0.3931          2
  bloque 11     ' cold'             0.643   0.6435          1
  bloque 12     ' cold'             0.163   0.1627          1

«Monday, Tuesday,» → ¿« Wednesday»?
  capa          más probable        prob.   P(objetivo)  puesto
  embeddings    ' mathemat'         0.702   0.0000      19296
  bloque  1     ' which'            0.140   0.0000       5544
  bloque  2     ' which'            0.107   0.0000       3999
  bloque  3     ' release'          0.064   0.0000       1966
  bloque  4     ' release'          0.154   0.0001       1161
  bloque  5     ' July'             0.093   0.0003        271
  bloque  6     ' July'             0.195   0.0011         57
  bloque  7     ' July'             0.427   0.0016         21
  bloque  8     ' July'             0.215   0.0401          7
  bloque  9     ' Tuesday'          0.715   0.1561          2
  bloque 10     ' Wednesday'        0.921   0.9212          1
  bloque 11     ' Wednesday'        0.775   0.7751          1
  bloque 12     ' Wednesday'        0.467   0.4667          1
```

Las dos frases repiten el patrón de 16.5: nada útil hasta el bloque 8 y la respuesta en los bloques 9 y 10. En la primera, « cold» aparece en el bloque 9, el 10 duda con « cooler» y el 11 la fija con un 64 %; los días de la semana pasan por « July» y por « Tuesday», el día que acaba de leer, antes de que en el bloque 10 aparezca « Wednesday» con un 92 %. De nuevo, el último bloque baja la confianza (al 16 % y al 47 %) sin cambiar la respuesta.

Ejercicio 16.3 La cabeza «token anterior» en un texto normal

El Programa G-2 encontró, con tokens al azar, una cabeza que mira casi siempre al token anterior. Compruébalo con un texto inglés normal: para cada una de las 144 cabezas, ¿en qué porcentaje de posiciones su máximo de atención es el token anterior? Muestra a quién mira cada token en el bloque 5, cabeza 12.

> 💡 **Pista**
>
> `cache[k]["pesos"][0, h, t]` son los pesos de la cabeza h del bloque k para la posición t; su `argmax()` es la posición a la que más mira.

✅ Solución:

📄 ejercicio_16_3.py

```
# EJERCICIO 16.3 — La cabeza «token anterior» en un texto normal
# El Programa G-2 encontró, con tokens al azar, una cabeza que mira casi
# siempre al token anterior. ¿Hace lo mismo con un texto de verdad?
import numpy as np

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])

texto = ('The neural network reads the sentence one token at a time. Each layer adds a little
'
         'more information to the vectors, and the attention heads decide where to look.')
ids = codificar(texto)
_, (_, _, cache) = forward(np.array([ids]))
T = len(ids)
i = np.arange(1, T)
print(f'{T} tokens. Para cada cabeza: % de posiciones cuyo máximo de atención es el token
anterior')
res = np.array([[np.mean(c['pesos'][0, h, i].argmax(-1) == i - 1) for h in range(H)] for c in
cache])
for k, h in sorted(np.ndindex(N_BLQ, H), key=lambda kh: -res[kh])[:3]:
    print(f'   bloque {k + 1:2d}, cabeza {h + 1:2d}: {res[k, h]:.0%}')
print(f'   media de las {N_BLQ * H} cabezas: {res.mean():.0%}')
k, h = 4, 11                                    # bloque 5, cabeza 12 (numeradas desde 1)
print(f'\nBloque 5, cabeza 12, en las 8 primeras posiciones (a quién mira cada token):')
for t in range(1, 9):
    j = int(cache[k]['pesos'][0, h, t].argmax())
    peso = cache[k]['pesos'][0, h, t, j]
    print(f'   {decodificar([ids[t]])!r:12s} → {decodificar([ids[j]])!r:12s} ({peso:.2f})')
```

> Salida del programa:

```
Pesos leídos en 0.9 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
32 tokens. Para cada cabeza: % de posiciones cuyo máximo de atención es el token anterior
   bloque  5, cabeza 12: 100%
   bloque  3, cabeza  3: 74%
   bloque  2, cabeza  1: 65%
   media de las 144 cabezas: 11%

Bloque 5, cabeza 12, en las 8 primeras posiciones (a quién mira cada token):
   ' neural'    → 'The'         (1.00)
   ' network'   → ' neural'     (1.00)
   ' reads'     → ' network'    (1.00)
   ' the'       → ' reads'      (1.00)
   ' sentence'  → ' the'        (1.00)
   ' one'       → ' sentence'   (0.98)
   ' token'     → ' one'        (1.00)
   ' at'        → ' token'      (0.96)
```

En un texto normal, la cabeza 12 del bloque 5 mira al token anterior en el 100 % de las posiciones, casi siempre con todo su peso (entre 0.96 y 1.00). Es una pieza fija del modelo, no un truco que aparezca solo con tokens al azar: hace siempre lo mismo que `emb_prev` en el Programa RZ-3, copiar en cada posición la identidad del token anterior, para que las cabezas de inducción de los bloques siguientes puedan buscar «qué vino después». Otras dos cabezas lo hacen en parte (74 % y 65 %), y la media de las 144 es del 11 %.

Ejercicio 16.4 Temperatura frente a perplejidad

Genera 30 tokens a partir de «The meaning of life is» con temperaturas 0.3, 0.7, 1.0 y 1.5 (sin top-p) y mide, con el propio GPT-2, la perplejidad de cada texto generado. ¿Qué relación encuentras?

> 💡 **Pista**
>
> Calcula la pérdida solo en los tokens generados, no en los de la semilla, con la softmax a temperatura 1.

✅ Solución:

📄 ejercicio_16_4.py

```
# EJERCICIO 16.4 — Temperatura frente a perplejidad del texto generado
# Se genera con varias temperaturas y se mide, con el propio GPT-2 a
# temperatura 1, cuánto le «sorprende» cada texto generado.
import numpy as np

fuente = open('programa_8bis_gpt2_real.py', encoding='utf-8').read()
exec(fuente.split('# EXPERIMENTOS')[0])

semilla = 'The meaning of life is'
n_sem = len(codificar(semilla))
print(f"{'τ':>4s}  {'PPL':>7s}  texto generado")
for tau in [0.3, 0.7, 1.0, 1.5]:
    np.random.seed(0)
    texto = generar(semilla, n_tokens=30, temperatura=tau, top_p=1.0)
    ids = codificar(texto)
    lp = np.log(softmax(forward(np.array([ids]))[0][0, :-1].astype(np.float64)))
    nll = -lp[np.arange(n_sem - 1, len(ids) - 1), ids[n_sem:]]       # solo lo generado
    print(f'{tau:4.1f}  {np.exp(nll.mean()):7.1f}  {texto[len(semilla):].strip()[:90]!r}')
```

> Salida del programa:

```
Pesos leídos en 0.9 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
   τ      PPL  texto generado
 0.3      7.5  'not to be understood as a single individual or a single life. Life is a
process of living '
 0.7     22.2  'not based on an idealized version of reality. To put it bluntly, the "norm" is
merely an a'
 1.0    143.8  'against him; as he looks outwards towards the elevated sky. Having this with a
glance over'
 1.5   4251.2  'updated symbolically because God campaigns around glorifies things referred to
here conseq'
```

La perplejidad crece muy deprisa con la temperatura: 7.5 con $\tau = 0.3$; 22.2 con 0.7; 144 con 1.0 y más de 4000 con 1.5. Con temperatura baja, el modelo escoge siempre continuaciones que él mismo considera muy probables, y el texto es predecible (y tiende a repetirse). Con temperatura alta, elige muchos tokens poco probables y el texto se convierte en una sucesión de palabras sin sentido. Las temperaturas habituales, entre 0.7 y 1, quedan entre ambos extremos: es el equilibrio entre coherencia y variedad del Capítulo 14.

Ejercicio 16.5 ¿En qué se va el tiempo con el KV-cache?

Con el KV-cache, cada token nuevo atraviesa los 12 bloques y la proyección final sobre los 50 257 tokens. Mide por separado cuánto tarda cada parte y relaciónalo con el número de pesos que hay que leer en cada una.

> 💡 **Pista**
>
> Reutiliza las funciones del Programa G-3 y repite cada medida 20 veces. Los pesos de un bloque son $4 \cdot D^2$ de la atención y $2 \cdot D \cdot 4D$ de la FFN; los de la salida, $D \cdot V$.

✅ Solución:

📄 ejercicio_16_5.py

```
# EJERCICIO 16.5 — ¿En qué se va el tiempo de un token con KV-cache?
# Con el cache, cada token nuevo atraviesa los 12 bloques y la proyección
# final sobre los 50 257 tokens. Medimos cada parte por separado.
import time
import numpy as np

fuente = open('programa_G3_kvcache.py', encoding='utf-8').read()
exec(fuente.split('# 1. ¿Da exactamente lo mismo')[0])     # funciones del Programa G-3

ids = codificar('The quick brown fox jumps over the lazy dog because it')
kv = nuevo_cache()
prefill(ids, kv)
x = params['wte'][ids[-1]] + params['wpe'][len(ids) - 1]


def medir(escala, rep=20):
    """Un token con el cache lleno: tiempo de los 12 bloques y de la salida."""
    t_bloques = t_salida = 0.0
    for _ in range(rep):
        t0 = time.time()
        y = x
        for k in range(N_BLQ):              # como forward_un_token, sin guardar nada
            a, _ = layer_norm_fwd(y, params[f'ln1_g{k}'], params[f'ln1_b{k}'])
            q, kk, v = (a @ params[f'W{n}{k}'] + params[f'b{n}{k}']).reshape(H, DH) for n in
'qkv')
            K, Vm = kv[k]['K'][:, :len(ids)], kv[k]['V'][:, :len(ids)]
            pesos = softmax(np.einsum('hd,htd->ht', q, K) / escala)
            att = np.einsum('ht,htd->hd', pesos, Vm).reshape(D)
            y = y + att @ params[f'Wo{k}'] + params[f'bo{k}']
            m, _ = layer_norm_fwd(y, params[f'ln2_g{k}'], params[f'ln2_b{k}'])
            y = y + gelu(m @ params[f'W1{k}'] + params[f'b1{k}']) @ params[f'W2{k}'] +
params[f'b2{k}']
        t1 = time.time()
        xf, _ = layer_norm_fwd(y, params['lnf_g'], params['lnf_b'])
        logits = xf @ params['wte'].T        # la proyección sobre el vocabulario
        t2 = time.time()
        t_bloques += t1 - t0
        t_salida += t2 - t1
    print(f'   pesos de atención en {pesos.dtype}, vía residual en {y.dtype}')
    print(f'   12 bloques:    {t_bloques / rep * 1000:6.1f} ms por token')
    print(f'   salida (wteᵀ): {t_salida / rep * 1000:6.1f} ms por token')
    return logits


n_bloques = N_BLQ * (4 * D * D + 2 * D * 4 * D)
print(f'Pesos de los 12 bloques: {n_bloques / 1e6:.1f} millones; '
      f'de la salida: {D * V / 1e6:.1f} millones')
print('\nCon np.sqrt(DH), como en el Programa 8:')
l64 = medir(np.sqrt(DH))
# np.sqrt() devuelve un número float64 de NumPy, y NumPy convierte a float64 todo
# lo que se opera con él. Pasa con np.sqrt(DH) y con C_GELU = np.sqrt(2 / np.pi).
print('\nCon esas dos constantes en float32:')
C_GELU = np.float32(C_GELU)                  # gelu() la lee de aquí
l32 = medir(np.float32(np.sqrt(DH)))
print(f'\nDiferencia máxima entre los logits: {np.abs(l64 - l32).max():.1e}; '
      f'mismo token elegido: {"sí" if l64.argmax() == l32.argmax() else "no"}')
```

> Salida del programa:

```
Pesos leídos en 1.0 s: 160 tensores
Vocabulario: 50257 tokens | Bloques: 12 | Cabezas: 12 | D: 768 | Contexto: 1024
Total de parámetros: 124,439,808
Pesos de los 12 bloques: 84.9 millones; de la salida: 38.6 millones

Con np.sqrt(DH), como en el Programa 8:
   pesos de atención en float64, vía residual en float64
   12 bloques:      98.8 ms por token
   salida (wteᵀ):  466.1 ms por token

Con esas dos constantes en float32:
   pesos de atención en float32, vía residual en float32
   12 bloques:      16.3 ms por token
   salida (wteᵀ):    6.1 ms por token

Diferencia máxima entre los logits: 6.2e-05; mismo token elegido: sí
```

La primera medida parece absurda: la proyección final tiene menos de la mitad de pesos que los 12 bloques y tarda casi cinco veces más. La pista está en la línea que imprime los tipos: la vía residual está en **float64**, aunque los pesos son float32. El culpable es `np.sqrt()`: devuelve un número float64 de NumPy y, desde NumPy 2, todo lo que se opera

con él pasa a float64. Ocurre con `np.sqrt(DH)` en la atención y con `C_GELU` en la GELU, y desde ese momento NumPy tiene que convertir a float64 cada matriz de pesos que multiplica, en cada token: la de la salida, 38.6 millones de números, la que más. Con las dos constantes en float32, el token pasa de 565 a 22 milisegundos, unas 25 veces menos, y los logits cambian en menos de 10^{-4}, el redondeo de float32; el token elegido es el mismo. Ahora sí, los bloques tardan más que la salida, en proporción a sus pesos. El Programa 8 tiene las mismas dos líneas, pero con 60 000 parámetros la diferencia no se nota; las hemos dejado tal cual para que el Programa 8 bis siga siendo el Programa 8 línea a línea.

PARTE IV

Del laboratorio al mundo real

La escala de los grandes modelos y cómo un modelo base se convierte en asistente, cómo se evalúan, la ética y los sesgos, y las herramientas para seguir por tu cuenta.

Capítulo 17: Del Nano-LLM a los Gigantes (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar las capacidades emergentes y por qué aparecen a cierta escala.
> ✓ Describir la arquitectura Decoder-Only y la generación autorregresiva.
> ✓ Distinguir un modelo base de un asistente y las fases que los separan: SFT y RLHF o DPO.
> ✓ Cuantificar la brecha de datos, parámetros y cómputo entre el Nano-LLM y GPT-3.

17.1 La escala importa — y cómo

Entre nuestro Nano-LLM de 60 000 parámetros y los mayores modelos actuales hay unos siete órdenes de magnitud en número de parámetros (el tamaño exacto de GPT-4 no es público), y todavía más en cantidad de datos de entrenamiento y potencia de cómputo. Pero la brecha no es solo cuantitativa: a cierta escala, los modelos exhiben capacidades emergentes que nadie programó explícitamente.

Uno de los hallazgos más comentados de la investigación en LLMs (Wei et al., 2022) es que algunas capacidades parecen aparecer de golpe cuando el modelo supera cierto tamaño: por debajo, el modelo falla casi siempre; por encima, acierta a menudo. Estas capacidades emergentes incluyen razonamiento aritmético, traducción entre idiomas nunca vistos en ese par específico, resolución de analogías y capacidad de seguir instrucciones complejas.

> ℹ️ **¿Emergencia o medida?**
>
> Dos matices a las «capacidades emergentes»: (a) en el Capítulo 15, un modelo de solo 58 000 parámetros aprende a sumar números que nunca ha visto si los datos tienen el formato adecuado, así que no todo depende del tamaño; (b) parte de la aparente «emergencia» depende de cómo se mide: con métricas de todo o nada (acierto exacto) una mejora gradual puede parecer un salto brusco, y con métricas continuas el salto se suaviza (Schaeffer et al., 2023).

17.2 GPT y la arquitectura Decoder-Only

La arquitectura Transformer original de 2017 tenía dos partes: un encoder (que lee el texto de entrada) y un decoder (que genera el texto de salida). GPT (Generative Pre-trained Transformer) simplificó esto usando solo la parte decoder, entrenando el modelo con un solo objetivo: predecir el siguiente token.

Esta arquitectura, llamada 'causal' o 'autorregresiva', es la que usan GPT, Claude, Gemini y Llama. El modelo genera texto de izquierda a derecha, un token a la vez, usando toda la secuencia anterior como contexto.

📄 **La cadena de generación autorregresiva**

```
# Dado el prompt: 'Las redes neuronales'
# El modelo predice:

# Paso 1: 'Las redes neuronales' -> P(siguiente) -> 'aprenden'
# Paso 2: 'Las redes neuronales aprenden' -> P(siguiente) -> 'de'
# Paso 3: 'Las redes neuronales aprenden de' -> P(siguiente) -> 'los'
# Paso 4: 'Las redes neuronales aprenden de los' -> ... -> 'datos'
# ... y así sucesivamente

# Cada nuevo token se añade al contexto para el siguiente paso.
# El límite es la ventana de contexto (context window):
#   GPT-2 (2019):          1024 tokens
#   GPT-3 (2020):          2048 tokens
#   Grandes LLM (2026):  ~1 000 000 tokens
```

17.3 De modelo base a asistente: la pieza que falta

Antes de comparar tamaños, merece la pena decir con claridad qué es exactamente el Nano-LLM que construiste en el Capítulo 13. No es "un ChatGPT en miniatura": es un modelo base (base model), y esa distinción no es cosmética. Un modelo base es el resultado de una sola fase, el preentrenamiento (la fase 1): predecir el siguiente token sobre un corpus, sin más objetivo que ese. Es exactamente lo que hiciste, a escala diminuta; en un LLM comercial, ese corpus son billones de tokens de páginas web, libros, artículos científicos y código, y el entrenamiento dura semanas o meses en miles de GPU y cuesta millones de dólares. Y es también exactamente lo que son GPT-2, GPT-3 o Llama-3-8B en su versión "base" —los mismos pesos que luego se convierten en ChatGPT o en un asistente de Meta pasan, antes, por dos fases más que este libro no ha cubierto.

Si alguna vez has mirado Hugging Face, seguramente has visto esta distinción sin prestarle atención: modelos con nombres como Llama-3-8B y Llama-3-8B-Instruct, o Mistral-7B y Mistral-7B-Instruct. El primero de cada par es un modelo base: predice texto plausible, pero si le escribes "¿Cuál es la capital de Francia?" puede responder igual de bien con la pregunta siguiente de un examen que con "París" —porque nunca ha visto ejemplos de que se espera una respuesta, solo ha visto texto de Internet, y en Internet las preguntas van seguidas de más preguntas casi tan a menudo como de respuestas—. El segundo, el "Instruct", es el mismo modelo después de las dos fases que faltan.

Fase 2: Supervised Fine-Tuning (SFT) — enseñar el formato

El SFT (ya definido en el glosario del Apéndice B) ajusta el modelo base sobre un conjunto, mucho más pequeño que el corpus de preentrenamiento, de pares (instrucción, respuesta) escritos por humanos: miles o decenas de miles de ejemplos del tipo "Explica qué es la fotosíntesis" -> "La fotosíntesis es el proceso...". El objetivo de entrenamiento sigue siendo el mismo de siempre —predecir el siguiente token, entropía cruzada, backpropagation—, exactamente el mecanismo que ya conoces del Capítulo 13. Lo único que cambia es el dato: en vez de texto libre de Internet, ejemplos que demuestran el formato "pregunta, luego respuesta útil". Tras el SFT, el modelo ya responde a instrucciones de forma razonable. InstructGPT (OpenAI, 2022) fue el primer modelo conocido entrenado con esta receta: primero esta fase y después la siguiente.

La letra pequeña del SFT: tokens especiales y chat template

El párrafo anterior simplifica algo importante: decir que el SFT entrena sobre "pares (instrucción, respuesta)" suena a que basta con concatenar el texto de la pregunta y el texto de la respuesta, tal cual, y entrenar igual que en el Capítulo 13. En la práctica falta una pieza sin la cual el modelo no tiene forma de saber dónde termina lo que ha escrito el usuario y dónde debe empezar a generar él: necesita marcadores explícitos de estructura, los llamados tokens especiales.

Los LLMs de chat reservan secuencias de texto que nunca aparecen en el uso normal del idioma —o que se fuerzan a no aparecer— y las añaden al vocabulario como tokens propios, con su propio embedding aprendible, igual que cualquier otro token. Cada familia de modelos usa su propio formato, llamado chat template: OpenAI y muchos modelos open-source usan una variante de ChatML, LLaMA-3 usa la suya propia con `<|start_header_id|>` y `<|eot_id|>`. La idea de fondo es la misma en todos:

📄 Chat template (formato ChatML, simplificado)

```
<|im_start|>system
Eres un asistente util y conciso.<|im_end|>
<|im_start|>user
Explica que es la fotosintesis<|im_end|>
<|im_start|>assistant
La fotosintesis es el proceso mediante el cual las plantas...<|im_end|>
```

Todo eso —incluidos `<|im_start|>`, `system`, `user`, `assistant` y `<|im_end|>` como unidades especiales— se tokeniza y se entrena exactamente como el resto: es la secuencia completa la que se le da al modelo durante el SFT, prediciendo el siguiente token en cada posición igual que en el preentrenamiento. La diferencia importante está en la función de pérdida: durante el SFT, la pérdida se enmascara (loss masking) sobre los tokens del

system y del user —el modelo los ve como contexto, pero no se le penaliza por "predecirlos" mejor o peor, porque no los está generando él—. Solo se calcula pérdida sobre los tokens que corresponden a la respuesta del assistant, que es lo único que queremos que el modelo aprenda a producir. En inferencia, el servidor añade automáticamente `<|im_start|>assistant` al final del prompt del usuario y deja que el modelo genere hasta que emita `<|im_end|>`: ese token es, literalmente, la señal de «he terminado mi turno».

> ℹ️ Si quisieras aplicar SFT al propio Nano-LLM del Capítulo 13, este es el obstáculo real con el que te encontrarías primero: su tokenizador es a nivel de carácter, sobre un vocabulario cerrado de 46 símbolos ya usados por el idioma. Añadir tokens especiales exigiría ampliar C2I/I2C con nuevas entradas reservadas (por ejemplo, marcar el turno del asistente con un carácter de control que nunca aparezca en el corpus) y modificar el bucle de entrenamiento del Programa 8 para enmascarar la pérdida en la parte de la instrucción. Nada de esto cambia la arquitectura Transformer en sí — es, de nuevo, el mismo mecanismo aplicado con más cuidado al dato.

Fase 3: Alinear con preferencias humanas — RLHF y DPO

El SFT enseña el formato, pero no enseña qué respuesta es mejor entre varias igualmente correctas en el formato. Para eso existe una tercera fase, conocida por su nombre clásico, RLHF (Reinforcement Learning from Human Feedback): evaluadores humanos comparan pares de respuestas del modelo y marcan cuál prefieren; con esas comparaciones se entrena un modelo de recompensa (reward model), y ese modelo de recompensa guía, mediante aprendizaje por refuerzo (el algoritmo PPO, concretamente), los ajustes finales de los pesos del LLM.

RLHF con PPO funciona, pero es notoriamente complejo de implementar bien: hace falta entrenar y mantener tres modelos a la vez (la política, el modelo de recompensa y una copia de referencia), y el bucle de refuerzo es inestable y sensible a hiperparámetros. En 2023, Rafailov et al. propusieron DPO (Direct Preference Optimization), una alternativa que consigue el mismo efecto —ajustar el modelo hacia las respuestas que los humanos prefieren— sin entrenar ningún modelo de recompensa ni ejecutar ningún bucle de RL: reformula el problema como una única función de pérdida, calculada directamente sobre pares (respuesta preferida, respuesta rechazada), que se optimiza con el mismo descenso de gradiente de siempre. Por su simplicidad, DPO (y variantes posteriores como ORPO o KTO) ha desplazado en gran parte a RLHF-con-PPO como método por defecto para esta fase en proyectos open-source.

```
  📄 De preentrenamiento a asistente — el mapa completo
FASE 1: Preentrenamiento          <- AQUI TERMINA ESTE LIBRO
  Objetivo: predecir el siguiente token sobre texto libre (Internet, libros...)
  Datos: billones de tokens, sin etiquetar
  Resultado: un MODELO BASE (predice texto plausible, no sigue instrucciones)

         |  (esto es lo que le falta al Nano-LLM)
         v

FASE 2: Supervised Fine-Tuning (SFT)
  Objetivo: el mismo de siempre (predecir el siguiente token), sobre otros datos
  Datos: miles de pares (instruccion, respuesta) escritos por humanos
  Resultado: el modelo ya sigue instrucciones

         v

FASE 3: Alineacion con preferencias humanas (RLHF o DPO)
  Objetivo: preferir la MEJOR respuesta entre varias ya correctas en formato
  Datos: miles de pares (respuesta preferida, respuesta rechazada)
  Metodo clasico: modelo de recompensa + PPO (RLHF, complejo)
  Metodo actual:  una sola funcion de perdida sobre preferencias (DPO, mas simple)
  Resultado: un ASISTENTE tipo ChatGPT / Claude / Gemini
```

Ninguna de las dos fases que faltan requiere una arquitectura distinta a la que ya construiste: siguen siendo el mismo Transformer, entrenado con el mismo backpropagation, ajustando los mismos pesos. Lo que cambia es exclusivamente el dato de

entrenamiento y, en la fase 3, la función de pérdida. Si quieres ver esto en código real en vez de en un mapa, la librería TRL de Hugging Face implementa SFT, RLHF y DPO sobre modelos como GPT-2 pequeño con relativamente poco código, y es un salto natural desde este libro (es el Nivel 2 de la hoja de ruta de la sección 20.7).

> ℹ️ Terminar este libro habiendo entendido a fondo la Fase 1 no es "la mitad del trabajo": es la parte que casi nadie entiende bien, ni siquiera muchos usuarios avanzados de IA. Las Fases 2 y 3 son, en comparación, mucho más simples de aprender una vez dominada la primera —son el mismo mecanismo, aplicado a otro dato. Sabes más de lo que crees sobre cómo se construye un ChatGPT real.

> ℹ️ **Razonar paso a paso a escala**
>
> Desde 2022 sabemos que pedir a un LLM que escriba su razonamiento paso a paso antes de responder (cadena de razonamiento, Wei et al., 2022) mejora mucho sus resultados en problemas de varios pasos. Desde 2024 existen además modelos entrenados específicamente para razonar por escrito antes de responder, como OpenAI o1 (septiembre de 2024) o DeepSeek-R1 (enero de 2025). Es la misma idea que viste funcionar en el Capítulo 15 con un modelo de 2 bloques: la longitud del texto sustituye a la profundidad de la red.

17.4 Lo que nos separa del nano al gigante

Para ir de nuestro Nano-LLM a un modelo comercial harían falta, aproximadamente:

* Datos: de ~5000 caracteres a entre uno y más de treinta billones de tokens (decenas de terabytes de texto).
* Parámetros: de 60 000 a 7000 millones (LLaMA-7B), 175 000 millones (GPT-3) o 671 000 millones (DeepSeek-V3, un modelo abierto de mezcla de expertos).
* Cómputo: de minutos en CPU a meses en miles o decenas de miles de GPUs (A100, H100 y sucesoras).
* Ingeniería: sistemas distribuidos, mixed-precision training, gradient checkpointing, frameworks especializados.
* Seguridad: filtrado de datos tóxicos, ajuste fino con RLHF, red teaming.

> 💡 La buena noticia: los modelos open-source como Llama (Meta), Mistral, Gemma, Qwen o DeepSeek hacen posible estudiar, ejecutar y ajustar modelos de miles de millones de parámetros en hardware doméstico. El conocimiento que has adquirido en este libro es exactamente lo que necesitas para entender y trabajar con ellos.

17.5 KV-Cache — por qué la generación real es rápida

El bucle de generación del Nano-LLM (Capítulo 13) tiene un defecto que pasa desapercibido a pequeña escala: en cada carácter nuevo, vuelve a llamar a forward() sobre toda la ventana de contexto desde cero, recalculando absolutamente todo —incluidas las claves y valores de atención de los tokens que ya se habían procesado en el paso anterior—. Con CTX=24 y un modelo de 60 000 parámetros, ese desperdicio es invisible: recalcular 24 posiciones tarda microsegundos. Pero el mismo patrón, a la escala de un LLM real con miles de tokens de contexto, es la diferencia entre una respuesta en segundos y una respuesta en minutos.

Sin optimizar, generar T tokens de forma autorregresiva cuesta más de lo que parece a primera vista. En el paso t, recalcular el forward completo implica repetir la atención sobre los t tokens ya generados, con coste $O(t^2)$ contando solo la atención (las capas densas añaden un coste proporcional a t). Sumando ese coste para t=1 hasta T, el total es $O(T^3)$ —no $O(T^2)$—: bastante peor que el coste de un único forward pass sobre toda la secuencia final, que calcularás en el Ejercicio 17.3 como $O(T^2)$.

El KV-cache resuelve exactamente ese desperdicio. La idea es simple: las claves (K) y valores (V) de atención de un token, una vez calculados, no cambian nunca —dependen solo de ese token y de los anteriores, gracias a la máscara causal—. Así que, en lugar de recalcularlos en cada paso, se guardan en memoria la primera vez y se reutilizan en todos los pasos siguientes. En cada nuevo token, el modelo solo necesita calcular su propia clave, su propio valor, y la atención de ese único token nuevo contra todo el historial ya cacheado.

📄 Generación sin KV-cache vs. con KV-cache (pseudocódigo)

```
# SIN KV-CACHE: recalcula todo en cada paso
for _ in range(n_tokens_generar):
    logits = forward(secuencia_completa)   # coste O(t^2) en el paso t
    siguiente = muestrear(logits[-1])
    secuencia_completa.append(siguiente)
# Coste total: O(T^3)

# CON KV-CACHE: reutiliza K y V ya calculados
cache_K, cache_V = [], []                  # uno por capa de atencion
for _ in range(n_tokens_generar):
    k_nuevo, v_nuevo = calcular_kv(token_actual)     # solo el token nuevo
    cache_K.append(k_nuevo); cache_V.append(v_nuevo)
    logits = forward_con_cache(token_actual, cache_K, cache_V)  # coste O(t)
    siguiente = muestrear(logits)
    token_actual = siguiente
# Coste total: O(T^2) -- el mismo que un unico forward sobre toda la secuencia
```

El precio a pagar es memoria: el cache crece linealmente con la longitud de la secuencia (hay que guardar K y V de cada token, en cada capa y cada cabeza de atención), y con contextos de cientos de miles de tokens y muchas peticiones simultáneas, esa memoria se convierte en el recurso más escaso del servidor —más incluso que el cómputo—. Ahí es donde entra vLLM: su técnica de PagedAttention (Apéndice F.4.3) gestiona ese KV-cache igual que un sistema operativo gestiona la memoria virtual, en páginas, para no desperdiciar memoria reservando de más ni fragmentarla entre peticiones de distinta longitud. Es la razón por la que servir un LLM en producción es, en gran medida, un problema de gestión de memoria, no solo de cómputo.

PagedAttention gestiona mejor la memoria que ya existe, pero hay una pregunta previa igual de importante: ¿se puede reducir directamente cuánta memoria ocupa el KV-cache, en vez de solo administrarla mejor? La respuesta que han adoptado la mayoría de LLMs recientes —incluido Mistral, ya mencionado en el glosario— es Grouped-Query Attention (GQA). En el Multi-Head Attention del Capítulo 12, cada cabeza tiene su propio juego de matrices Q, K y V; eso significa que el KV-cache necesita guardar un K y un V distintos por cada cabeza, en cada capa. GQA cambia esa proporción: mantiene muchas cabezas de consulta (Q) —para no perder la capacidad de atender a distintos tipos de relaciones—, pero agrupa varias de ellas para que compartan un único juego de claves y valores. Con, por ejemplo, 32 cabezas de Q agrupadas de 4 en 4 sobre 8 grupos de K/V, el KV-cache se reduce a una cuarta parte de su tamaño original, con una pérdida de calidad mínima frente a tener K/V independientes por cabeza. El caso extremo, con un único grupo de K/V compartido por todas las cabezas de Q, se llama Multi-Query Attention (MQA); GQA es, en la práctica, el punto intermedio que casi todos los modelos recientes prefieren, porque ahorra casi toda la memoria de MQA sin pagar tanto en calidad.

> ℹ️ El Nano-LLM del Capítulo 13 no implementa KV-cache a propósito: con CTX=24 fijo, el coste por paso ya es constante y el ahorro sería insignificante frente a la claridad del código. Y no serviría tal cual: el Nano-LLM recorta la ventana a los últimos 24 caracteres y reinicia las posiciones, así que al desplazarse la ventana cambian la posición y el contexto de cada carácter, y las K y V guardadas dejarían de valer. Las cuentas $O(T^3)$ y $O(T^2)$ de arriba son para una secuencia que crece sin recortes; con una ventana fija de C tokens, cada paso sin cache cuesta $O(C^2)$ y el total crece solo linealmente con T. Es una simplificación pedagógica razonable, pero conviene saber que se ha dejado fuera precisamente la optimización que hace posible que un LLM real responda en tiempo real. En el Capítulo 16 (16.7) la implementas para GPT-2 y la mides: con una semilla de 800

tokens, genera 5.6 veces más rápido, y el ahorro crece con la longitud del contexto.

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- Las capacidades emergentes parecen aparecer de golpe al superar ciertos umbrales de escala, aunque parte de ese salto depende de cómo se mida.
- GPT y los LLMs modernos usan la arquitectura Decoder-Only con generación autorregresiva token a token.
- Preentrenamiento, ajuste fino supervisado (SFT) y alineación con preferencias (RLHF o DPO) son la receta habitual (no la única) para convertir un modelo base en un asistente útil; las tres usan el mismo Transformer y el mismo backpropagation.
- **El KV-cache evita recalcular claves y valores en cada paso de la generación: cambia cómputo por memoria, y esa memoria es el recurso escaso al servir un LLM.**

DIAGRAMA 17.1 — DE NANO A GIGANTE

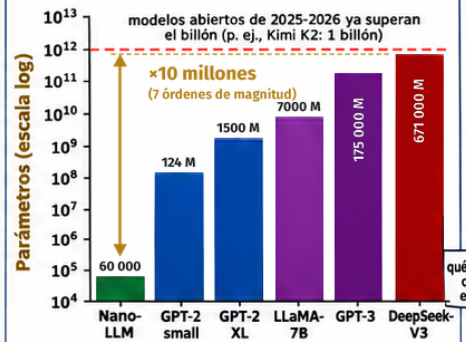
Resumen visual del Capítulo 17 — la escala de los LLM

Parámetros · Datos · Cómputo · Capacidades emergentes · Preentrenamiento → SFT → RLHF/DPO

Del Nano-LLM de 60 000 parámetros a modelos abiertos de más de un billón: la misma arquitectura de base

LA BRECHA DE ESCALA

Del Nano-LLM a los mayores modelos actuales: unos 7 órdenes de magnitud en parámetros



qué cambia con la escala

Lo que separa al Nano-LLM de un modelo grande:

📊 Datos	🧠 Parámetros	⚡ Cómputo
5 KB (93 frases) → 40 GB (GPT-2) → 14.8 billones de tokens (DeepSeek-V3)	60 000 → 175 000 M (GPT-3) → 671 000 M (DeepSeek-V3)	2.5 min en un portátil → 2.79 millones de horas de GPU H800 (DeepSeek-V3)
≈ $\times 10^{10}$	≈ $\times 10^{7}$	≈ 10^{12} FLOPs → ≈ $3 \cdot 10^{24}$ FLOPs

Cómputo estimado con la regla 6 × parámetros activos × tokens.

CAPACIDADES EMERGENTES

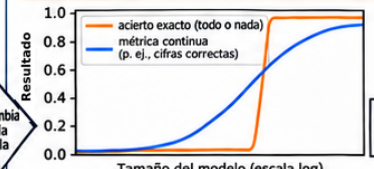
Parecen aparecer de golpe al superar cierto tamaño

¿Qué son las capacidades emergentes?

Habilidades que un modelo pequeño casi nunca muestra y que un modelo mayor muestra a menudo (Wei et al., 2022).

Ejemplos: aritmética de varios pasos, traducción entre pares de idiomas no entrenados, analogías, seguir instrucciones complejas.

Nadie las programa explícitamente: ***salen del mismo entrenamiento, a mayor escala.***



Esquema ilustrativo: la misma mejora parece un salto o una rampa según cómo se mida (Schaeffer et al., 2023).

¿Emergencia o medida?

① **No todo depende del tamaño:** en el Capítulo 15, un modelo de solo 58 000 parámetros aprende a sumar y acierta el 100 % de 2000 sumas que nunca vio. La clave fueron los datos.

② **Parte del «salto» depende de la métrica:** con métricas de todo o nada, una mejora gradual parece brusca.

Razonar paso a paso

Cadena de razonamiento (Wei et al., 2022): escribir los pasos mejora el resultado.
Modelos de razonamiento: OpenAI o1 (2024), DeepSeek-R1 (2025).
La misma idea que pruebas en el Capítulo 15 con un modelo de 2 bloques.

cómo se entrenan

LAS TRES FASES DE ENTRENAMIENTO

De modelo base a asistente: el mismo Transformer, otro dato

① PREENTRENAMIENTO

El modelo aprende a predecir texto

Datos: páginas web, libros, artículos, código (sin etiquetar)
Tarea: predecir el siguiente token
Escala: de 1 a más de 30 billones de tokens
Resultado: un **MODELO BASE:** predice texto plausible, no sigue instrucciones

```
GPT-3: 300 000 M tokens ·
DeepSeek-V3: 14.8 billones
```

★ AQUÍ TERMINA ESTE LIBRO: tu Nano-LLM es un modelo base

la fase más cara: meses en miles de GPU

sabe mucho, pero no responde como un asistente →

② AJUSTE FINO SUPERVISADO (SFT)

El modelo aprende el formato pregunta → respuesta

Datos: miles o decenas de miles de pares (instrucción, respuesta) escritos por humanos
Tarea: la misma de siempre (siguiente token), solo sobre la respuesta (*loss masking*)

```
<|im_start|>user … <|im_end|>  <|im_start|>assistant …
```

Resultado: sigue instrucciones

mucho más barato

***responde, pero no sabe cuál es la* MEJOR respuesta →**

③ ALINEACIÓN CON PREFERENCIAS: RLHF o DPO

El modelo aprende qué respuestas prefieren los humanos

Datos: miles de pares (respuesta preferida, respuesta rechazada)

RLHF (clásico)	DPO (Rafailov et al., 2023)
modelo de recompensa + PPO InstructGPT, 2022 complejo: 3 modelos a la vez	una sola función de pérdida más simple; muy usado en modelos abiertos

Resultado: un ASISTENTE tipo ChatGPT, Claude o Gemini

✅ **Preentrenamiento** → conocimiento del lenguaje y del mundo
SFT → formato de asistente
RLHF / DPO → la respuesta preferida

Las tres fases: mismo Transformer, mismo backpropagation. Cambian el dato y, en la 3.ª, la pérdida.

arquitectura y acceso

DECODER-ONLY Y MODELOS ABIERTOS

Arquitectura decoder-only (la de GPT):

- Solo decoder, sin encoder.
- La máscara causal hace que cada token solo vea el pasado.
- La usan GPT, Claude, Gemini, Llama… y tu Nano-LLM.

```
Contexto:  Nano-LLM 24 car. · GPT-2 1024 · GPT-3 2048 ·
           grandes LLM (2026) ~1 000 000 tokens
```

La misma base, detalles nuevos:

RMSNorm (en vez de LayerNorm) · RoPE (en vez de wpe)
SwiGLU (en vez de GELU) · GQA (comparte K y V)
MoE: cada token usa solo algunas FFN · KV-cache al generar

Modelos abiertos: se pueden descargar, estudiar y ajustar

Llama (Meta)	Llama 3.1: 8000 M, 70 000 M, 405 000 M
Qwen (Alibaba)	Qwen3: hasta 235 000 M (MoE)
DeepSeek	V3 / R1: 671 000 M (37 000 M activos)
Mistral	Mistral 7B y sucesores
Gemma (Google)	Gemma 3: de 1000 M a 27 000 M

Cifras de los informes técnicos de cada modelo.

```
memoria = parámetros × bytes por parámetro
(1 GB = 1024³ bytes)
```

Nano-LLM · float32	**0.24 MB**	cualquier CPU (cabe en su caché)
GPT-2 small · float32	**0.46 GB**	cualquier portátil
LLaMA-7B · float16	**13.0 GB**	una GPU de 24 GB o más
LLaMA-7B · int4	**3.3 GB**	un portátil
GPT-3 · float16	**326 GB**	varias GPU de centro de datos

Solo los pesos; al generar se suma la memoria del KV-cache.

Preentrenamiento → modelo base
SFT → sigue instrucciones
RLHF / DPO → elige la respuesta preferida

Escala: $\times 10^{7}$ en parámetros, $\times 10^{10}$ en datos

Emergencia: saltos que dependen también de la medida

Alineación: RLHF o DPO

Verde = Nano-LLM / modelos pequeños
Azul = GPT-2
Morado = Modelos grandes (GPT-3, LLaMA)
Rojo = Mayores modelos abiertos (MoE)
Verde azulado = Preentrenamiento
Naranja = Ajuste fino / emergencia
Dorado = Parámetros / datos / cómputo

✏️ EJERCICIOS — Capítulo 17 — Del Nano al Gigante

Estos ejercicios te invitan a hacer números con modelos reales. Resuélvelos antes de seguir y contrasta tus cálculos con la solución propuesta para hacerte una idea clara de su verdadera escala.

Ejercicio 17.1 Memoria necesaria para distintos modelos

Escribe una función que calcule la memoria mínima (en GB) para almacenar los pesos de un modelo dado su número de parámetros y la precisión numérica (float32 = 4 bytes, float16 = 2 bytes, int8 = 1 byte, int4 = 0.5 bytes). Calcula para LLaMA-7B y GPT-3.

> 💡 💡 Pista: 1 GB = 1024^3 bytes. Un modelo de 7000 millones de parámetros en float16 necesita 7e9 × 2 bytes = $14 \cdot 10^9$ bytes ≈ 13.0 GB (con 1 GB = 1024^3 bytes): cabe de sobra en una sola GPU A100, que tiene 40 u 80 GB. Con int4 (cuantización), $3.5 \cdot 10^9$ bytes ≈ 3.3 GB.

✅ Solución:

📄 **ejercicio_17_1.py**

```
def memoria_gb(n_params, precision='float16'):
    bytes_por = {'float32':4, 'float16':2, 'int8':1, 'int4':0.5}
    return n_params * bytes_por[precision] / (1024**3)

modelos = [
    ('Nano-LLM (nuestro)',     60_000),
    ('GPT-2 small',       124_000_000),
    ('GPT-2 XL',        1_558_000_000),
    ('LLaMA-7B',        7_000_000_000),
    ('LLaMA-13B',      13_000_000_000),
    ('GPT-3 175B',    175_000_000_000),
]

print(f"{'Modelo':>25}  {'FP32 (GiB)':>10}  {'FP16 (GiB)':>10}  {'INT4 (GiB)':>10}")
print('-' * 60)
for nombre, n in modelos:
    g32 = memoria_gb(n, 'float32')
    g16 = memoria_gb(n, 'float16')
    g4  = memoria_gb(n, 'int4')
    print(f'{nombre:>25}  {g32:>10.3f}  {g16:>10.3f}  {g4:>10.3f}')

print()
print('Una GPU A100: 80 GB  |  RTX 4090 (gaming): 24 GB')
print('INT4 permite ejecutar LLaMA-7B en una GPU de 4 GB!')
```

> ℹ️ En entrenamiento, la memoria real es 3-8x mayor (gradientes, optimizador, activaciones intermedias). En inferencia, solo se necesita cargar los pesos, de ahí la importancia de la cuantización.

Ejercicio 17.2 ¿Cuántos datos necesitamos?

La 'Chinchilla Scaling Law' (Hoffmann et al., 2022) encontró que, para un presupuesto de cómputo de entrenamiento dado, el reparto óptimo usa ≈ 20 tokens por parámetro (en las condiciones de su estudio; no es un mínimo de calidad). Calcula cuántos tokens y cuántos libros equivalentes se necesitan para entrenar modelos de 7B, 70B y 700B parámetros.

> 💡 💡 Pista: Un libro tiene aproximadamente 100 000 palabras ≈ 130 000 tokens (con tokenización BPE). Chinchilla mostró que muchos modelos anteriores estaban infraentrenados para su cómputo; hoy se usan muchos más de 20 tokens por parámetro cuando interesa un modelo más pequeño y barato de usar.

✅ Solución:

📄 **ejercicio_17_2.py**

```
def tokens_optimos(n_params):
    """Tokens óptimos según Chinchilla: 20 x n_params"""
    return 20 * n_params
```

```
TOKENS_POR_LIBRO = 130_000    # tokens BPE aprox.
TOKENS_POR_WEB   = 500        # tokens por página web típica

modelos = [
    ('LLaMA-7B',     7_000_000_000),
    ('LLaMA-13B',   13_000_000_000),
    ('LLaMA-70B',   70_000_000_000),
    ('Hipotético-700B', 700_000_000_000),
]

print('Tokens óptimos según Chinchilla (Hoffmann et al., 2022)\n')
print(f"{'Modelo':>20}  {'Tokens óptimos':>16}  {'Libros equiv.':>15}")
print('-' * 58)
for nombre, n in modelos:
    t = tokens_optimos(n)
    libros = t / TOKENS_POR_LIBRO
    print(f'{nombre:>20}  {t:>16,.0f}  {libros:>15,.0f}')

print()
print('Comparación: GPT-3 fue entrenado con ~300B tokens')
print('pero tiene 175B params -> solo 1.7x (infraentrenado)')
print('LLaMA-2-7B fue entrenado con 2T tokens -> 285x (sobreentrenado')
print('para tamaño, pero mejor en inference!)')
```

Ejercicio 17.3 El costo cuadrático de la atención

El self-attention tiene complejidad $O(T^2)$ en memoria y cómputo. Calcula y compara el costo de la atención para contextos de 512, 2048, 8192, 32 768 y 131 072 tokens (d=768, 12 cabezas). ¿Cuánto aumenta el costo al doblar el contexto?

> 💡 💡 Pista: La matriz de atención tiene T×T elementos. Para T=131 072 y 12 cabezas en float16, eso son $131\,072^2 \times 12 \times 2$ bytes $\approx 412 \cdot 10^9$ bytes (412 GB, o 384 GiB si, como el programa, se divide entre 1024^3) solo para la atención. Por eso los LLMs modernos usan 'Flash Attention' para calcularla sin materializarla completa.

✅ Solución:

📄 **ejercicio_17_3.py**

```
def costo_atencion(T, d, n_heads, bytes_por=2):
    """Memoria en GiB (1024³ bytes) y GFLOPs de la atención"""
    mem_gb = T * T * n_heads * bytes_por / (1024**3)
    gflops = 4 * T * T * d / 1e9
    return mem_gb, gflops

d, n_heads = 768, 12  # GPT-2 / BERT configuración

print(f'd={d}, n_heads={n_heads} (estilo GPT-2 small)\n')
print(f"{'Contexto T':>12}  {'Mem. aten. (GiB)':>17}  {'GFLOPs':>10}  Factor vs ant.")
print('-' * 60)

T_vals = [512, 2048, 8192, 32768, 131072]
prev_mem = None
for T in T_vals:
    mem, gflops = costo_atencion(T, d, n_heads)
    factor = f'{mem/prev_mem:.1f}x' if prev_mem else '—'
    print(f'{T:>12,}  {mem:>17.3f}  {gflops:>10.2f}  {factor:>13}')
    prev_mem = mem

print()
print('Al doblar T, el costo se CUADRUPLICA (x4).')
print('Por eso ampliar la ventana de contexto es tan caro.')
print('Flash Attention: evita materializar la matriz TxT completa,')
print('calculándola por bloques. Mismo resultado, mucha menos memoria.')
```

Capítulo 18: Evaluación de Modelos de Lenguaje (**)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Calcular perplejidad correctamente sobre datos de test separados.
> ✓ Diagnosticar overfitting en un LLM a partir del ratio PPL_test/PPL_train.
> ✓ Implementar BLEU desde cero y construir un mini-benchmark de evaluación.
> ✓ Identificar las trampas de la evaluación: contaminación, Goodhart, mala calibración.

18.1 El problema de medir la inteligencia

En el Capítulo 13 entrenamos nuestro Nano-LLM y vimos cómo la pérdida desciende de 3.85 a 0.29 en el texto de entrenamiento. Pero en el Capítulo 15 vimos que en frases que no ha visto el resultado es muy distinto: su pérdida sube hasta 4.18, peor que adivinar al azar (15.2). Una pérdida baja es una señal positiva, pero ¿significa que el modelo es bueno? ¿Bueno para qué? ¿Comparado con qué? La evaluación rigurosa de un modelo de lenguaje es más sutil y más difícil de lo que parece.

Un modelo puede tener una pérdida de entrenamiento muy baja y aun así ser inútil (si ha hecho overfitting al conjunto de entrenamiento), inexacto (si genera hechos incorrectos con total confianza), sesgado (si reproduce discriminaciones del corpus de entrenamiento), o simplemente aburrido (si siempre genera el texto más probable y nunca sorprende).

La evaluación de LLMs es un campo activo de investigación con sus propias trampas, controversias y evolución constante. En este capítulo aprenderás las métricas más importantes, cómo implementarlas y —quizás lo más valioso— cómo no dejarte engañar por ellas.

> ℹ️ La Ley de Goodhart, que el economista Charles Goodhart planteó en 1975 a propósito de la política monetaria, se cita casi siempre con las palabras de la antropóloga Marilyn Strathern (1997): 'Cuando una medida se convierte en un objetivo, deja de ser una buena medida.' En IA: tan pronto como se publica un benchmark, los equipos empiezan a optimizar sus modelos para ese benchmark específico, y la métrica deja de medir lo que pretendía medir.

18.2 Evaluación intrínseca — Perplejidad

La perplejidad (Perplexity, PPL) es la métrica más usada para evaluar modelos de lenguaje de forma intrínseca — es decir, sin necesidad de etiquetar datos ni definir una tarea específica. Mide cuánto 'le sorprende' al modelo el texto que está evaluando.

Matemáticamente, la perplejidad es la exponencial de la pérdida media por token:

📄 Perplejidad — definición y significado

```
PPL = exp( media de -log P(token_t | contexto) )
    = exp( pérdida_media_por_token )

Interpretación intuitiva:
  PPL = V  -> Es lo que da repartir la probabilidad por igual
              entre los V tokens, es decir, adivinar al azar.
              Con PPL = V, el modelo no mejora al azar.

  PPL = 10 -> El modelo actua como si eligiera entre 10 tokens posibles
              en cada posicion. Ha aprendido estructura significativa.

  PPL = 2  -> El modelo actua como si eligiera entre 2 opciones en
              cada paso. Predice muy bien el texto.

  PPL = 1  -> El modelo predice cada token con certeza absoluta.
              Imposible en la practica (o implica memorización).

Valores publicados en ingles (solo comparables con el mismo tokenizador):
  GPT-2 (124M): PPL ~ 37 en WikiText-103
```

```
  GPT-2 (1.5B): PPL ~ 18 en WikiText-103
  GPT-3 (175B): PPL ~ 20 en Penn Treebank
  Nuestro Nano-LLM: PPL ~ 1.3 en su corpus de entrenamiento y ~65 en frases nuevas
(15.2)

ADVERTENCIA CRUCIAL:
  La PPL DEBE calcularse sobre un conjunto de TEST separado,
  nunca sobre los datos de entrenamiento.
  PPL baja en entrenamiento y alta en test: señal de OVERFITTING.
```

PROGRAMA EVAL-1 — Perplejidad train vs. test: diagnóstico de overfitting

📄 **programa_EVAL1_perplexity.py**

```
import numpy as np

# ================================================================
# CALCULO CORRECTO DE PERPLEXITY
# El modelo NUNCA debe ver los datos de test durante el entreno.
# ================================================================

def softmax(x): e=np.exp(x-x.max()); return e/e.sum()

def calcular_perplexity(modelo_forward, tokens, CTX, nombre=''):
    """
    Calcula la perplejidad de un modelo sobre una secuencia.
    modelo_forward: funcion que recibe CTX tokens y devuelve logits
    tokens:         lista de indices enteros del texto
    CTX:            longitud del contexto del modelo
    """
    perdidas = []
    n_tokens = 0

    for i in range(CTX, len(tokens)):
        ctx    = tokens[i-CTX:i]      # contexto
        target = tokens[i]             # token a predecir
        logits = modelo_forward(ctx)   # logits del modelo
        # Tomar los logits del ULTIMO paso (prediccion del siguiente)
        if logits.ndim == 2: logits = logits[-1]
        probs  = softmax(logits)
        # -log P(token correcto | contexto)
        loss   = -np.log(probs[target] + 1e-10)
        perdidas.append(float(loss))
        n_tokens += 1

    perdida_media = np.mean(perdidas)
    ppl           = np.exp(perdida_media)
    bits_por_tok  = perdida_media / np.log(2)   # nats -> bits (aqui cada token es un
caracter)

    print(f'  {nombre or "Modelo"}')
    print(f'    Tokens evaluados:  {n_tokens}')
    print(f'    Perdida media:     {perdida_media:.4f}')
    print(f'    Perplejidad:       {ppl:.2f}')
    print(f'    Bits por token:    {bits_por_tok:.3f}')
    return ppl, perdida_media


# Simulacion: entrenamiento correcto con split train/test
# (El Programa RZ-1 del Cap 15 lo hace con el modelo real del Cap 13)
TEXTO_COMPLETO = '''
la inteligencia artificial aprende de los datos de entrenamiento.
los modelos de lenguaje predicen el siguiente token del texto.
las redes neuronales tienen capas de neuronas interconectadas.
el aprendizaje profundo usa muchas capas ocultas y atencion.
los transformers procesan secuencias en paralelo con atencion.
entrenar un modelo requiere datos etiquetados y potencia de calculo.
la generacion de texto usa estrategias de muestreo como top-p.
evaluar un modelo requiere datos de test que no vio en entrenamiento.
'''

vocab  = sorted(set(TEXTO_COMPLETO))
V      = len(vocab)
```

```
c2i    = {c:i for i,c in enumerate(vocab)}
i2c    = dict(enumerate(vocab))
tokens = [c2i[c] for c in TEXTO_COMPLETO]

# Division: 80% train, 20% test
corte      = int(0.8 * len(tokens))
tok_train  = tokens[:corte]
tok_test   = tokens[corte:]

print(f'Corpus total: {len(tokens)} tokens (vocab={V})')
print(f'Entrenamiento: {len(tok_train)} tokens ({len(tok_train)/len(tokens):.0%})')
print(f'Test:          {len(tok_test)}  tokens ({len(tok_test)/len(tokens):.0%})')
print(f'Perplejidad aleatoria esperada: {V:.0f} (= tamano vocabulario)')
print()

# Modelo ficticio para demo: distribuciones variadas
np.random.seed(42)

def modelo_aleatorio(ctx):
    """Distribucion uniforme: la referencia del azar (PPL = V)"""
    return np.zeros(V)    # logits uniformes -> prob 1/V cada token

def modelo_bigrama(ctx, tokens_ref=tok_train):
    """Bigrama: solo recuerda el ultimo token"""
    ultimo = ctx[-1]
    logits = np.zeros(V)
    # Buscar cuales tokens siguen a 'ultimo' en los datos de entreno
    for i in range(len(tokens_ref)-1):
        if tokens_ref[i] == ultimo:
            logits[tokens_ref[i+1]] += 1
    if logits.sum() == 0: logits[:] = 1  # fallback uniforme
    return np.log(logits / logits.sum() + 1e-10)  # log-probs

def modelo_uniforme_top3(ctx):
    """Simula un modelo que conoce los 3 tokens mas probables"""
    # El siguiente token es el ultimo caracter del contexto + algo
    # (simulacion simplificada para la demo)
    logits = np.random.randn(V) * 0.1
    # Dar algo de peso al token correcto con probabilidad variable
    logits[np.random.randint(V)] += 2.0
    return logits

CTX = 8
print('=== PERPLEJIDAD EN DISTINTOS CONJUNTOS ===')
print()

print('Modelo aleatorio (baseline):')
calcular_perplexity(modelo_aleatorio, tok_test, CTX, 'Test')
print()

print('Modelo bigrama (aprende pares de caracteres):')
calcular_perplexity(modelo_bigrama, tok_train, CTX, 'Train')
calcular_perplexity(modelo_bigrama, tok_test, CTX,  'Test ')
print()

# Demostrar el diagnostico de overfitting
print('=== DIAGNOSTICO DE OVERFITTING VIA PERPLEJIDAD ===')
print()
print('CASO 1 — Buen modelo (generaliza):')
print('  PPL train = 4.2,  PPL test = 4.8  -> diferencia pequeña: OK')
print()
print('CASO 2 — Overfitting leve:')
print('  PPL train = 2.1,  PPL test = 8.3  -> test 4x peor: señal de alerta')
print()
print('CASO 3 — Overfitting grave (memorizacion):')
print('  PPL train = 1.0,  PPL test = 45.2 -> test 45x peor: el modelo')
print('  memorizo el corpus de entrenamiento pero no generaliza.')
print()
print('Una pista, no una regla: un cociente PPL_test / PPL_train alto avisa de')
print('sobreajuste, pero no hay umbrales universales (un modelo que no aprende')
print('nada tiene cociente 1). Compara siempre con una linea base.')
```

> Salida del programa:

```
Corpus total: 519 tokens (vocab=28)
Entrenamiento: 415 tokens (80%)
Test:          104  tokens (20%)
Perplejidad aleatoria esperada: 28 (= tamano vocabulario)

=== PERPLEJIDAD EN DISTINTOS CONJUNTOS ===

Modelo aleatorio (baseline):
  Test
    Tokens evaluados:  96
    Perdida media:     3.3322
    Perplejidad:       28.00
    Bits por token:    4.807

Modelo bigrama (aprende pares de caracteres):
  Train
    Tokens evaluados:  407
    Perdida media:     1.7835
    Perplejidad:       5.95
    Bits por token:    2.573
  Test
    Tokens evaluados:  96
    Perdida media:     4.5798
    Perplejidad:       97.50
    Bits por token:    6.607

=== DIAGNOSTICO DE OVERFITTING VIA PERPLEJIDAD ===

CASO 1 — Buen modelo (generaliza):
  PPL train = 4.2,  PPL test = 4.8  -> diferencia pequeña: OK

CASO 2 — Overfitting leve:
  PPL train = 2.1,  PPL test = 8.3  -> test 4x peor: señal de alerta

CASO 3 — Overfitting grave (memorizacion):
  PPL train = 1.0,  PPL test = 45.2 -> test 45x peor: el modelo
  memorizo el corpus de entrenamiento pero no generaliza.

Una pista, no una regla: un cociente PPL_test / PPL_train alto avisa de
sobreajuste, pero no hay umbrales universales (un modelo que no aprende
nada tiene cociente 1). Compara siempre con una linea base.
```

18.3 Evaluación extrínseca — Tareas downstream

La perplejidad mide qué tan bien el modelo predice el texto. Pero lo que nos importa en la práctica es si el modelo puede hacer cosas útiles: responder preguntas, traducir, resumir, escribir código. Para esto necesitamos evaluación extrínseca: medir el rendimiento en tareas concretas con respuestas correctas e incorrectas.

Las tareas downstream más comunes para evaluar LLMs son:

* Preguntas de opción múltiple: el modelo lee una pregunta y elige entre varias respuestas. Métricas: precisión. Ejemplos: MMLU (57 materias académicas), HellaSwag (completar frases), ARC (preguntas de ciencias).
* Generación de código: el modelo debe escribir código que pase tests automáticos. Métricas: pass@k (porcentaje de problemas resueltos con k intentos). Ejemplo: HumanEval (164 problemas de Python).
* Razonamiento matemático: el modelo debe resolver problemas con varios pasos. Métricas: precisión en la respuesta final. Ejemplo: GSM8K (matemáticas de primaria/secundaria).
* Factualidad: el modelo debe responder preguntas con hechos verificables. Métricas: coincidencia exacta, F1 sobre tokens. Ejemplos: TriviaQA, TruthfulQA.
* Resumen: el modelo genera un resumen y se compara con referencias humanas. Métricas: ROUGE, BERTScore. Ejemplo: CNN/DailyMail.

18.4 BLEU — Midiendo la calidad del texto generado

BLEU (Bilingual Evaluation Understudy) fue propuesto en 2002 para evaluar traducciones automáticas y sigue siendo una de las métricas más usadas. Compara el texto generado con

una o varias traducciones de referencia contando cuántos n-gramas (secuencias de n palabras consecutivas) del candidato aparecen en las referencias.

📄 **BLEU — La idea central**

```
Texto candidato (generado): 'el gato se sentó en la alfombra'
Texto referencia (correcto): 'el gato estaba sentado en la alfombra'

Unigramas (n=1) del candidato que aparecen en la referencia:
  'el' -> SI   'gato' -> SI   'se' -> NO   'sentó' -> NO
  'en' -> SI   'la' -> SI    'alfombra' -> SI
  -> 5/7 = 0.714

Bigramas (n=2) del candidato que aparecen en la referencia:
  'el gato' -> SI   'gato se' -> NO   'se sentó' -> NO
  'sentó en' -> NO   'en la' -> SI   'la alfombra' -> SI
  -> 3/6 = 0.500

BLEU-2 = geometric_mean(0.714, 0.500) * brevity_penalty  (BP = 1:
       ≈ 0.598 × 1         candidato y referencia tienen 7 palabras)

Brevity penalty: penaliza candidatos mas cortos que la referencia.
  Si |candidato| < |referencia|: BP = exp(1 - |ref|/|cand|) < 1
  Si |candidato| >= |referencia|: BP = 1
```

PROGRAMA EVAL-2 — BLEU desde cero y framework de evaluación

📄 **programa_EVAL2_bleu_benchmark.py**

```
import numpy as np
import math
from collections import Counter

# ============================================================
# PARTE 1: BLEU DESDE CERO
# ============================================================

def ngramas(tokens, n):
    """Extrae todos los n-gramas de una lista de tokens."""
    return [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

def bleu(candidato, referencia, n_max=4):
    """
    Calcula el BLEU score entre un texto candidato y una referencia.
    candidato, referencia: listas de palabras (tokens)
    n_max:                 maximo orden de n-gramas (tipicamente 4)
    """
    cand = candidato.lower().split()
    ref  = referencia.lower().split()

    precisiones = []
    for n in range(1, n_max + 1):
        cand_ng = Counter(ngramas(cand, n))
        ref_ng  = Counter(ngramas(ref,  n))

        # Coincidencias con clipping (no contar mas que en la referencia)
        coincidencias = sum(
            min(cnt, ref_ng[ng]) for ng, cnt in cand_ng.items()
        )
        total = max(len(cand) - n + 1, 0)

        if total == 0:
            precisiones.append(0.0)
        else:
            precisiones.append(coincidencias / total)

    # Media geometrica de las precisiones
    if any(p == 0 for p in precisiones):
        bleu_score = 0.0
    else:
        log_avg    = sum(math.log(p) for p in precisiones) / n_max
        bleu_score = math.exp(log_avg)
```

```
    # Penalizacion por brevedad
    bp = math.exp(min(0, 1 - len(ref)/max(len(cand), 1)))

    return bp * bleu_score, precisiones, bp


# --- Ejemplos de calculo BLEU ---
print('=== BLEU SCORE: EJEMPLOS ===')
print()

casos = [
    ('Traduccion perfecta',
     'el modelo aprende de los datos',
     'el modelo aprende de los datos'),
    ('Traduccion buena',
     'el modelo aprende usando los datos',
     'el modelo aprende de los datos'),
    ('Traduccion parcial',
     'el sistema procesa informacion',
     'el modelo aprende de los datos'),
    ('Texto sin relacion',
     'la tarde cae sobre el monte',
     'el modelo aprende de los datos'),
]

for nombre, cand, ref in casos:
    score, precs, bp = bleu(cand, ref, n_max=2)
    print(f'  {nombre}:')
    print(f'    Candidato: "{cand}"')
    print(f'    Referencia: "{ref}"')
    print(f'    BLEU-2: {score:.4f}  (BLEU-1: {precs[0]:.3f}, BLEU-2: {precs[1]:.3f},
BP: {bp:.3f})')
    print()

# ================================================================
# PARTE 2: FRAMEWORK DE EVALUACION CON OPCION MULTIPLE
# ================================================================

print('=== FRAMEWORK DE EVALUACION: OPCION MULTIPLE ===')
print()

# Mini-benchmark sobre IA (preguntas de ejemplo)
BENCHMARK = [
    {
        'pregunta': 'Que funcion de activacion se usa mas en capas ocultas modernas?',
        'opciones': ['a) Sigmoid', 'b) Tanh', 'c) ReLU', 'd) Softmax'],
        'correcta': 'c',
        'explicacion': 'ReLU es la mas usada por eficiencia y por evitar el vanishing
gradient'
    },
    {
        'pregunta': 'Que significa PPL = 37 si el vocabulario tiene 37 caracteres?',
        'opciones': ['a) El modelo es excelente', 'b) Pierde lo mismo que repartir la
probabilidad por igual',
                     'c) El modelo tiene overfitting', 'd) El modelo necesita mas
capas'],
        'correcta': 'b',
        'explicacion': 'PPL = V es la del azar uniforme: el modelo no mejora a adivinar
entre V'
    },
    {
        'pregunta': 'Que optimizador usa weight decay desacoplado?',
        'opciones': ['a) SGD', 'b) RMSprop', 'c) Adam', 'd) AdamW'],
        'correcta': 'd',
        'explicacion': 'AdamW desacopla el weight decay del gradiente, corrigiendo un
error de Adam'
    },
    {
        'pregunta': 'Cual es la ventaja principal de Top-p sobre Top-k en muestreo?',
        'opciones': ['a) Es mas rapido', 'b) Se adapta a la forma de la distribucion',
                     'c) Siempre incluye mas tokens', 'd) No necesita temperatura'],
        'correcta': 'b',
```

```
        'explicacion': 'Top-p adapta el numero de tokens al nivel de concentracion de
la distribucion'
    },
    {
        'pregunta': 'Que es el Vanishing Gradient en RNNs?',
        'opciones': ['a) Los gradientes crecen sin control',
                     'b) Los gradientes se vuelven nulos al retropropagar muchos pasos',
                     'c) Los pesos convergen a cero', 'd) La perdida no baja'],
        'correcta': 'b',
        'explicacion': 'El gradiente se multiplica por valores < 1 en cada paso,
decayendo exponencialmente'
    },
]

def evaluar_modelo_simulado(benchmark, tasa_acierto=0.80, semilla=42):
    """
    Simula la evaluacion de un modelo en un benchmark de opcion multiple.
    En la practica, se le daria la pregunta+opciones al modelo y se
    miraria cual opcion asigna mayor probabilidad.
    """
    np.random.seed(semilla)
    resultados = []
    for i, q in enumerate(benchmark):
        # Simular respuesta del modelo
        acierta = np.random.random() < tasa_acierto
        opciones = [o[0] for o in q['opciones']]
        if acierta:
            respuesta = q['correcta']
        else:
            otras = [o for o in opciones if o != q['correcta']]
            respuesta = np.random.choice(otras)
        resultados.append({
            'correcto': acierta,
            'respuesta': respuesta,
            'esperada': q['correcta']
        })
    return resultados

print('Mini-benchmark de IA (5 preguntas):')
print()

modelos = [
    ('Modelo aleatorio',  0.25),
    ('Modelo basico',     0.60),
    ('Modelo avanzado',   0.80),
    ('Modelo experto',    1.00),
]

print(f"{'Modelo':>20}  {'Aciertos':>10}  {'Precision':>12}  Tasa simulada")
print('-' * 68)
for nombre, tasa in modelos:
    res = evaluar_modelo_simulado(BENCHMARK, tasa_acierto=tasa)
    n_ok = sum(r['correcto'] for r in res)
    prec = n_ok / len(BENCHMARK)
    print(f'{nombre:>20}  {n_ok:>4}/{len(BENCHMARK):>4}  {prec:>12.1%}  {tasa:.0%}')

print()
print('Con 5 preguntas el azar pesa mucho: el modelo que acierta el 25 % de las')
print('veces ha sacado 3 de 5. Para estimar una tasa hacen falta cientos de
preguntas.')
print()
print('En MMLU (57 materias; 14 042 preguntas de test), segun cada informe:')
print('  GPT-3.5 (2023): ~70%   GPT-4 (2023): ~86%   Claude 3 Opus (2024): ~87%')
print('  Experto humano (estimacion): ~90%   Humano no especialista: ~35%')
```

> Salida del programa:

```
=== BLEU SCORE: EJEMPLOS ===

  Traduccion perfecta:
    Candidato: "el modelo aprende de los datos"
    Referencia: "el modelo aprende de los datos"
    BLEU-2: 1.0000  (BLEU-1: 1.000, BLEU-2: 1.000, BP: 1.000)
```

```
  Traduccion buena:
    Candidato: "el modelo aprende usando los datos"
    Referencia: "el modelo aprende de los datos"
    BLEU-2: 0.7071  (BLEU-1: 0.833, BLEU-2: 0.600, BP: 1.000)

  Traduccion parcial:
    Candidato: "el sistema procesa informacion"
    Referencia: "el modelo aprende de los datos"
    BLEU-2: 0.0000  (BLEU-1: 0.250, BLEU-2: 0.000, BP: 0.607)

  Texto sin relacion:
    Candidato: "la tarde cae sobre el monte"
    Referencia: "el modelo aprende de los datos"
    BLEU-2: 0.0000  (BLEU-1: 0.167, BLEU-2: 0.000, BP: 1.000)

=== FRAMEWORK DE EVALUACION: OPCION MULTIPLE ===

Mini-benchmark de IA (5 preguntas):

               Modelo    Aciertos     Precision  Tasa simulada
------------------------------------------------------------------
    Modelo aleatorio     3/   5         60.0%  25%
       Modelo basico     3/   5         60.0%  60%
     Modelo avanzado     4/   5         80.0%  80%
      Modelo experto     5/   5        100.0%  100%

Con 5 preguntas el azar pesa mucho: el modelo que acierta el 25 % de las
veces ha sacado 3 de 5. Para estimar una tasa hacen falta cientos de preguntas.

En MMLU (57 materias; 14 042 preguntas de test), segun cada informe:
  GPT-3.5 (2023): ~70%   GPT-4 (2023): ~86%   Claude 3 Opus (2024): ~87%
  Experto humano (estimacion): ~90%   Humano no especialista: ~35%
```

18.5 De los benchmarks clásicos a los de frontera

Durante años, la capacidad de un LLM se resumía en unos pocos benchmarks públicos: MMLU (2020; 15 908 preguntas de opción múltiple de 57 materias, 14 042 de ellas de test), HumanEval (2021; 164 problemas de Python cuyo código debe pasar unos tests, medidos con pass@k), GSM8K (2021; 8500 problemas de matemáticas de varios pasos; en 15.4 viste por qué encadenar pasos es difícil para un modelo que responde de golpe), HellaSwag (2019; sentido común) y TruthfulQA (2022; preguntas trampa que miden la veracidad). Las cifras de MMLU que imprime el Programa EVAL-2 son de 2023. Desde entonces, los mejores modelos rondan o superan el 90 % en MMLU, a uno o dos puntos unos de otros, y en GSM8K y HumanEval están cerca del techo: estos benchmarks están saturados y ya no distinguen a los mejores modelos. Siguen siendo útiles para comparar modelos pequeños y para aprender cómo se evalúa.

Por eso la comunidad crea pruebas cada vez más difíciles. En 2026 las más citadas son GPQA Diamond (preguntas de ciencias de nivel de doctorado; los expertos humanos aciertan en torno al 70 % y los mejores modelos ya los superan), Humanity's Last Exam (unas 2500 preguntas en la frontera del conocimiento; los modelos pasaron de alrededor del 8 % en su lanzamiento, en enero de 2025, a más del 60 % a mediados de 2026, con ayuda de herramientas como la búsqueda web o la ejecución de código), SWE-bench (corregir errores reales en repositorios de GitHub) y ARC-AGI (razonamiento abstracto a partir de muy pocos ejemplos; su tercera versión, de 2026, sigue resistiendo a los modelos). El ciclo se repite: un benchmark nuevo parece imposible, se convierte en objetivo y en pocos años queda saturado. Es la Ley de Goodhart (18.1) en acción, y la razón por la que cualquier cifra de este apartado envejecerá pronto.

18.6 Calibración — ¿Sabe el modelo lo que no sabe?

Un aspecto crucial de la evaluación que a menudo se ignora: ¿es el modelo transparente sobre su incertidumbre? Un modelo bien calibrado, cuando dice 'estoy 90% seguro', debería estar en lo correcto el 90% de las veces. Un modelo mal calibrado puede estar muy seguro de respuestas incorrectas — uno de los ingredientes de lo que llamamos alucinaciones.

La calibración se mide con el Error de Calibración Esperado (ECE): se agrupan las predicciones por su confianza (del 0-10 %, del 10-20 %...), en cada grupo se compara la

confianza media con el porcentaje real de aciertos y se promedian las diferencias, ponderadas por el tamaño de cada grupo. Un ECE de 0 es una calibración perfecta. Una alucinación es el caso extremo de mala calibración: una respuesta falsa dada con total seguridad.

> ℹ️ Los LLMs modernos tienden a estar sobreconfiados: dicen las cosas de forma asertiva incluso cuando no están seguros. Curiosamente, el modelo base puede estar mejor calibrado que el asistente: en el informe técnico de GPT-4 (OpenAI, 2023), el modelo base estaba bien calibrado en una prueba con preguntas de MMLU, y el modelo ajustado con RLHF, bastante peor. Es una observación sobre ese modelo y esa prueba; una explicación plausible, que el informe no aísla, es que el ajuste premia las respuestas seguras y agradables. Enseñar a un modelo a decir «no lo sé» cuando corresponde es hoy un área de trabajo activa.

18.7 Las trampas de la evaluación

Conocer los benchmarks no es suficiente — también es necesario entender sus limitaciones y cómo se pueden 'manipular' inadvertidamente:

Contaminación de datos (data contamination)

Los benchmarks se publican con sus preguntas y respuestas. Si un LLM se entrena sobre texto de internet, es muy probable que ese texto incluya foros, artículos y discusiones que contengan exactamente esas preguntas y respuestas. El modelo puede haber 'memorizado' las respuestas durante el preentrenamiento, no aprenderlas durante la evaluación.

Este problema es real y documentado. El informe técnico de GPT-4 reconoce, por ejemplo, que parte del conjunto de entrenamiento de GSM8K estaba en sus datos. La solución —usar benchmarks privados y confidenciales— tiene sus propios problemas de reproducibilidad. En la sección 15.2 mediste el fenómeno a pequeña escala: el 81-100 % del texto que genera el Nano-LLM son copias literales de su corpus.

La brecha entre benchmarks y utilidad real

Un modelo puede puntuar muy alto en MMLU y aun así ser difícil de usar en la práctica: si genera texto verboso, no sigue instrucciones bien, o alucina con frecuencia. La evaluación con usuarios reales en tareas reales (con frecuencia más costosa pero más informativa) puede contar una historia muy diferente a los benchmarks académicos.

18.8 Evaluación humana — La prueba definitiva

Para conversación, redacción o resúmenes, la evaluación humana sigue siendo la referencia. El método más común es la comparación A/B: se presentan a evaluadores humanos dos respuestas (sin decirles cuál viene de cuál modelo) y se les pide que elijan la mejor. Este es exactamente el proceso que usa el RLHF que mencionamos en el Capítulo 17: las preferencias humanas se convierten en señal de entrenamiento.

La plataforma LMArena (arena.ai), nacida en Berkeley como Chatbot Arena y hoy una empresa independiente, permite a usuarios reales comparar respuestas de distintos LLMs y votar por la mejor. Con millones de comparaciones, produce un ranking de tipo Elo (como el del ajedrez) más cercano a la utilidad real que muchos benchmarks académicos. Sus cifras dependen de quién vota, qué pregunta y cuándo: al citar una posición o una puntuación, hay que dar la fecha y la categoría del ranking.

> 💡 La evaluación humana tiene sus propios sesgos: los evaluadores prefieren respuestas más largas, más seguras y más 'agradables'. Un modelo que siempre valida al usuario puede puntuar alto en evaluación humana aunque sea menos útil en la práctica. La evaluación rigurosa requiere diseñar las tareas cuidadosamente para evitar estos sesgos.

18.9 Guía práctica para evaluar tu propio modelo

Cuando entrenes tu propio modelo — ya sea el Nano-LLM o algo más ambicioso — aplica este proceso. El Programa RZ-1 del Capítulo 15 aplica los pasos 1, 2 y 4 al Nano-LLM, y los Programas RZ-2 a RZ-4, el paso 5:

```
📄 Protocolo de evaluacion rigurosa
PASO 1: DIVIDIR LOS DATOS ANTES DE EMPEZAR
  -> Train (80%), Validacion (10%), Test (10%)
  -> El test NO se toca hasta que el modelo esta completamente listo.
  -> Toda decision de hiperparametros usa solo train y validacion.

PASO 2: MONITORIZAR PPL EN VALIDACION DURANTE EL ENTRENAMIENTO
  -> Guardar el modelo con mejor PPL en validacion.
  -> Parar cuando PPL_val lleva N epocas sin mejorar (early stopping).
  -> Un cociente PPL_val / PPL_train que crece avisa de sobreajuste.

PASO 3: EVALUACION FINAL EN TEST (solo una vez)
  -> Calcular PPL_test con el mejor modelo guardado.
  -> Este numero es el rendimiento real del modelo.

PASO 4: EVALUACION CUALITATIVA
  -> Generar textos con distintas semillas y estrategias de muestreo.
  -> Revisar manualmente: coherencia, gramatica, relevancia.
  -> Identificar fallos sistematicos (temas que el modelo no maneja).

PASO 5: SI ES POSIBLE, TAREA ESPECIFICA
  -> Diseñar un mini-benchmark con preguntas representativas.
  -> Calcular precision en esas preguntas.
  -> Comparar con un baseline (modelo aleatorio, modelo bigrama).

SEÑALES DE ALARMA:
  PPL_test >> PPL_train  -> Overfitting: regularizar o mas datos.
  PPL_train alta          -> Underfitting: mas capacidad o mas entreno.
  PPL_test baja pero texto incoherente -> El modelo memorizo patrones
                          superficiales que no generaliza bien.
  Precision benchmark baja -> El modelo no aprende la tarea subyacente.
```

📌 RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO

- La perplejidad debe calcularse siempre sobre datos de test separados, nunca sobre el conjunto de entrenamiento.
- Los benchmarks (MMLU, HumanEval, GSM8K) miden capacidades concretas, pero son vulnerables a la Ley de Goodhart y se saturan en pocos años; hoy se usan otros más difíciles (GPQA, Humanity's Last Exam, SWE-bench).
- Un modelo bien calibrado es transparente sobre su incertidumbre; una alucinación dicha con seguridad es un síntoma de mala calibración (que, en sentido estricto, se mide sobre muchas predicciones, no sobre una).

✏️ EJERCICIOS — Capítulo 18 — Evaluación de Modelos

Evaluar un modelo exige tanto criterio como cálculo. Practica con estos ejercicios antes de continuar y compara tus resultados con los propuestos para entender qué mide realmente cada métrica.

Ejercicio 18.1 Calcular perplejidad de tres modelos hipotéticos

Dadas las pérdidas medias por token de tres modelos sobre el mismo conjunto de test: Modelo A (1.2), Modelo B (2.5) y Modelo C (0.3), calcula la perplejidad de cada uno e interpreta qué significa en términos de 'número efectivo de opciones' que maneja el modelo en cada predicción.

> 💡 💡 Pista: PPL = exp(pérdida). Recuerda interpretar el resultado: una PPL de 3 significa que el modelo, en promedio, 'duda' entre aproximadamente 3 opciones en cada posición.

✅ **Solución:**

📄 **ejercicio_18_1.py**

```
import numpy as np

modelos = {
    'Modelo A': 1.2,
    'Modelo B': 2.5,
    'Modelo C': 0.3,
}

print('=== INTERPRETACION DE PERPLEJIDAD ===\n')
print(f"{'Modelo':>10}  {'Perdida':>9}  {'PPL':>8}  Interpretacion")
print('-' * 60)

for nombre, perdida in modelos.items():
    ppl = np.exp(perdida)
    if ppl < 2:
        interp = 'Excelente: casi siempre acierta el token correcto'
    elif ppl < 5:
        interp = 'Muy bueno: duda entre pocas opciones razonables'
    elif ppl < 15:
        interp = 'Aceptable: bastante incertidumbre'
    else:
        interp = 'Pobre: el modelo apenas ha aprendido patrones'
    print(f'{nombre:>10}  {perdida:>9.2f}  {ppl:>8.2f}  {interp}')

print()
print('Comparacion relativa:')
ppls = {k: np.exp(v) for k,v in modelos.items()}
mejor = min(ppls, key=ppls.get)
peor  = max(ppls, key=ppls.get)
print(f'  {mejor} es {ppls[peor]/ppls[mejor]:.1f}x mejor que {peor}')
```

Ejercicio 18.2 Calcular BLEU con múltiples referencias

El BLEU del Programa EVAL-2 usa una sola referencia. En la práctica, las traducciones suelen tener varias referencias válidas. Modifica la función bleu() para aceptar una lista de referencias y usar, para cada n-grama, el máximo conteo entre todas las referencias (en lugar de una sola).

> 💡 💡 Pista: Cuando hay varias referencias, un n-grama del candidato 'cuenta como acierto' si aparece en CUALQUIERA de las referencias. Para el clipping, usa el máximo de las apariciones en cada referencia individual.

✅ **Solución:**

📄 ejercicio_18_2.py

```
import math
from collections import Counter

def ngramas(tokens, n):
    return [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

def bleu_multi_ref(candidato, referencias, n_max=4):
    '''BLEU con multiples referencias validas'''
    cand = candidato.lower().split()
    refs = [r.lower().split() for r in referencias]

    precisiones = []
    for n in range(1, n_max+1):
        cand_ng = Counter(ngramas(cand, n))
        # Para cada n-grama, tomar el MAXIMO conteo entre todas las referencias
        max_ref_ng = Counter()
        for ref in refs:
            ref_ng = Counter(ngramas(ref, n))
            for ng, cnt in ref_ng.items():
                max_ref_ng[ng] = max(max_ref_ng[ng], cnt)

        coincidencias = sum(min(cnt, max_ref_ng[ng]) for ng, cnt in cand_ng.items())
        total = max(len(cand) - n + 1, 0)
        precisiones.append(coincidencias/total if total > 0 else 0.0)

    if any(p==0 for p in precisiones):
        return 0.0
    log_avg = sum(math.log(p) for p in precisiones) / n_max
    # Brevity penalty: usar la referencia mas cercana en longitud
    len_ref_cercana = min(refs, key=lambda r: abs(len(r)-len(cand)))
    bp = math.exp(min(0, 1 - len(len_ref_cercana)/max(len(cand),1)))
    return bp * math.exp(log_avg)

candidato = 'el modelo aprende rapido de los datos'
referencias = [
    'el modelo aprende rapidamente de los datos',
    'el modelo aprende rapido con los datos',
    'la red aprende velozmente de la informacion',
]

score = bleu_multi_ref(candidato, referencias, n_max=2)
print(f'Candidato: "{candidato}"')
print(f'Referencias: {len(referencias)} variantes validas')
print(f'BLEU-2 (multi-referencia): {score:.4f}')
print()
print('Con multiples referencias, el BLEU suele ser MAS ALTO')
print('porque hay mas oportunidades de coincidir con alguna variante valida.')
```

Ejercicio 18.3 Detectar overfitting de benchmark con un experimento controlado

Simula el fenómeno de 'overfitting de benchmark': entrena (simuladamente) un modelo optimizando directamente para acertar las preguntas de un mini-benchmark de 20 preguntas, y evalúalo después en un conjunto de 20 preguntas NUEVAS pero del mismo estilo. Compara la precisión en ambos conjuntos.

> 💡 💡 Pista: Este ejercicio es conceptual: simula que 'memorizar' las respuestas del benchmark de entrenamiento da 100% en ese conjunto pero en torno al 25 % (el azar con cuatro opciones) en preguntas nuevas no vistas — exactamente como la contaminación de datos infla las métricas sin reflejar capacidad real.

✅ **Solución:**

📄 ejercicio_18_3.py

```
import numpy as np

np.random.seed(42)
```

```
def generar_benchmark(n, semilla):
    '''Genera un benchmark simulado de opcion multiple'''
    np.random.seed(semilla)
    return [{'id': i, 'respuesta_correcta': np.random.randint(4)} for i in range(n)]

benchmark_train = generar_benchmark(20, semilla=1)   # 'visto' en preentrenamiento
benchmark_nuevo = generar_benchmark(20, semilla=2)   # nunca visto

def modelo_memorizador(pregunta, benchmark_memorizado):
    '''Simula un modelo que memorizo las respuestas exactas'''
    for q in benchmark_memorizado:
        if q['id'] == pregunta['id']:
            return q['respuesta_correcta']  # responde perfecto si la vio
    return np.random.randint(4)  # si no la vio, responde al azar

def evaluar(modelo_fn, benchmark, memorizado):
    aciertos = sum(
        modelo_fn(q, memorizado) == q['respuesta_correcta']
        for q in benchmark
    )
    return aciertos / len(benchmark)

print('=== SIMULACION: OVERFITTING DE BENCHMARK ===\n')

acc_train = evaluar(modelo_memorizador, benchmark_train, benchmark_train)
acc_nuevo = evaluar(modelo_memorizador, benchmark_nuevo, benchmark_train)

print(f'Precision en benchmark CONTAMINADO (visto antes): {acc_train:.1%}')
print(f'Precision en benchmark NUEVO (nunca visto):       {acc_nuevo:.1%}')
print(f'Precision esperada al azar (4 opciones):           25.0%')
print()
print('El modelo "memorizador" parece perfecto (100%) en el benchmark')
print('contaminado, pero su rendimiento real (en preguntas nuevas)')
print('es indistinguible del azar. Esto es exactamente lo que ocurre')
print('cuando un LLM ha visto las respuestas del benchmark durante')
print('su preentrenamiento: la metrica miente sobre la capacidad real.')
```

ℹ️ Casos reales documentados: investigadores han encontrado fragmentos textuales de GSM8K y otros benchmarks en los datos de preentrenamiento de varios LLMs comerciales, lo que pone en duda parte de sus puntuaciones reportadas.

Capítulo 19: Ética y Sesgos en Inteligencia Artificial (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Explicar los cuatro tipos principales de sesgo en sistemas de IA.
> ✓ Detectar sesgo en un modelo que nunca usa la variable protegida directamente.
> ✓ Calcular e interpretar las principales métricas de equidad (fairness).
> ✓ Asumir con criterio la responsabilidad profesional del desarrollador de IA.

19.1 Una tecnología poderosa no es una tecnología neutral

Hasta aquí hemos aprendido cómo construir sistemas de IA: neuronas, redes, atención, transformers. Pero construir algo no es suficiente. El ingeniero que diseña un sistema asume también responsabilidad sobre su impacto en el mundo real.

La Inteligencia Artificial no es una tecnología neutral. Toma decisiones —quién recibe un crédito, quién es contratado, quién es considerado sospechoso, qué tratamiento médico recibe un paciente— a una escala y velocidad imposible para los humanos. Cuando esas decisiones son injustas, los efectos se amplifican de forma masiva.

Este capítulo no está pensado para desanimarte. Está pensado para que seas el tipo de desarrollador que la IA necesita: alguien que entiende la tecnología por dentro, como tú ya lo haces, y que además se hace las preguntas difíciles antes de desplegar un sistema.

19.2 ¿Qué es un sesgo en IA?

En estadística, el sesgo (bias) es la diferencia sistemática entre las predicciones de un modelo y la realidad. En el contexto ético, el sesgo es más específico: es cuando un sistema trata de forma injustamente diferente a personas o grupos en función de características protegidas como la raza, el género, la edad, el origen o la religión.

Es importante distinguir entre dos tipos de error en un modelo:

* Error aleatorio: el modelo se equivoca, pero de forma impredecible. Un buen entrenamiento reduce este tipo de error.
* Sesgo sistemático: el modelo se equivoca siempre en la misma dirección para ciertos grupos. Más datos o más entrenamiento no lo corrigen si el origen del sesgo está en los propios datos.

Los tipos de sesgo más comunes en IA son:

* Sesgo de representación: el conjunto de entrenamiento no refleja adecuadamente a todos los grupos. Un reconocedor facial entrenado principalmente con caras caucásicas fallará más con otras etnias.
* Sesgo histórico: los datos de entrenamiento reflejan decisiones humanas pasadas que eran discriminatorias. Un modelo entrenado para predecir 'candidatos exitosos' basándose en contrataciones históricas reproducirá los sesgos de quienes contrataban.
* Sesgo de medición: la variable que intentamos predecir no mide lo que creemos. Si usamos 'arrestos previos' como proxy de 'peligrosidad', incorporamos el sesgo de los sistemas policiales que patrullan más ciertos barrios.
* Bucle de retroalimentación (feedback loop): el modelo retroalimenta sus propias predicciones. Si el sistema predice que ciertos solicitantes son de alto riesgo y por eso se les deniega el crédito, en el futuro habrá menos datos positivos de ese grupo, confirmando la predicción original.

19.3 Casos reales — Cuando el algoritmo discrimina

COMPAS: predicción de reincidencia criminal

COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) es un sistema usado en EE. UU. para predecir la probabilidad de reincidencia de personas condenadas, influyendo en decisiones de fianza y libertad condicional. En 2016, ProPublica publicó un análisis demoledor: el sistema tenía el doble de probabilidad de etiquetar erróneamente como 'alto riesgo' a acusados negros que a acusados blancos, y el doble de probabilidad de etiquetar erróneamente como 'bajo riesgo' a acusados blancos. El algoritmo nunca usaba la raza como variable de entrada, pero podía reflejarla de forma indirecta: varias de las variables que sí usa (como los antecedentes o el entorno familiar y social del acusado) están correlacionadas con ella.

> ℹ️ La empresa creadora de COMPAS respondió que su modelo era igual de fiable para ambos grupos: a una misma puntuación de riesgo le correspondía la misma tasa de reincidencia en los dos. El debate que siguió reveló algo crucial: distintas métricas de 'justicia' son matemáticamente incompatibles entre sí. Kleinberg y otros (2016) y Chouldechova (2017) demostraron que, si la tasa real de reincidencia es distinta en dos grupos y el modelo no es perfecto, ningún modelo puede estar a la vez igual de bien calibrado para ambos (que un «riesgo del 60 %» signifique lo mismo en los dos) y tener en ambos las mismas tasas de falsos positivos y de falsos negativos. Es exactamente lo que ocurrió: la empresa defendía la calibración, y ProPublica medía los errores. La elección de qué optimizar es una decisión ética, no técnica.

Amazon: el algoritmo de contratación que discriminaba por género

Entre 2014 y 2017, Amazon desarrolló un sistema de IA para filtrar currículums de forma automatizada. El sistema fue entrenado sobre una década de solicitudes de empleo y decisiones de contratación previas. El problema: el sector tecnológico había estado dominado históricamente por hombres. El algoritmo aprendió a penalizar currículums que mencionaban la palabra «women's» («de mujeres», como en «capitana del club de ajedrez de mujeres») o que procedían de universidades femeninas. Amazon descartó el proyecto cuando detectó el problema, pero el caso ilustra cómo un objetivo aparentemente neutro ('encontrar candidatos como los que hemos contratado antes') puede codificar discriminación histórica.

Reconocimiento facial: el sesgo que puede costar la libertad

Múltiples estudios, incluyendo el influyente Gender Shades de Joy Buolamwini y Timnit Gebru (2018), demostraron que los sistemas comerciales de análisis facial, al clasificar el género de una cara, tenían tasas de error drásticamente diferentes según la combinación de género y tono de piel. Para mujeres de piel oscura, la tasa de error podía ser 34 puntos porcentuales más alta que para hombres de piel clara. Identificar a una persona es una tarea distinta, con fallos parecidos: en EE. UU. se han documentado varias detenciones de personas inocentes a partir de coincidencias de reconocimiento facial (The Washington Post contó al menos ocho en enero de 2025), casi todas de personas negras.

IA médica y los sesgos invisibles

Un estudio publicado en Science en 2019 analizó un algoritmo ampliamente usado en el sistema sanitario estadounidense para decidir qué pacientes necesitaban atención especial. El algoritmo usaba el coste sanitario histórico como proxy de la necesidad de atención. El problema: los pacientes negros históricamente habían tenido menos acceso al sistema, por lo que generaban menores costes. El algoritmo interpretaba esto como menor necesidad de atención, cuando en realidad reflejaba menor acceso. El sesgo resultante era que, a igual gravedad de la enfermedad, los pacientes negros eran identificados como prioritarios con mucha menos frecuencia: según los autores (Obermeyer et al.), corregirlo elevaría la proporción de pacientes negros seleccionados para el programa del 17.7 % al 46.5 %.

19.4 Cómo entran los sesgos en el pipeline de un modelo

Es tentador pensar que si no incluimos variables sensibles (raza, género, etc.) como features, el modelo será automáticamente justo. Esto es falso. Los sesgos pueden entrar en cualquier fase del desarrollo:

* Recogida de datos: ¿de dónde vienen los datos? ¿Representan a todos los grupos? ¿Quién decidió qué datos recoger y cuáles ignorar?
* Etiquetado: si humanos etiquetan los datos, traen sus propios sesgos. Estudios muestran que incluso con instrucciones claras, los anotadores difieren sistemáticamente según su propio perfil demográfico.
* Selección de features: variables aparentemente neutrales como el código postal, la universidad o el tipo de dispositivo móvil están fuertemente correlacionadas con factores socioeconómicos y étnicos.
* Elección de la función de pérdida: optimizar la precisión global puede ocultar disparidades grandes entre grupos minoritarios (que pesan poco en el promedio).
* Evaluación: si el conjunto de test no está estratificado por grupos, el buen rendimiento global puede esconder rendimiento pésimo en subgrupos.
* Despliegue y retroalimentación: los sistemas en producción generan nuevos datos sesgados por sus propias predicciones, creando ciclos de retroalimentación que amplifican el sesgo original.

19.5 Detectando el sesgo — Programa de análisis

El mejor antídoto contra el sesgo es medirlo explícitamente. El siguiente programa simula un escenario realista: un modelo de scoring crediticio entrenado sobre datos históricos sesgados. El modelo nunca usa el grupo como variable de entrada y aun así aprende el sesgo implícito en los datos.

PROGRAMA E-1 — Sesgo en acción: el modelo 'justo' que discrimina

📄 programa_E1_sesgo.py

```
import numpy as np

np.random.seed(42)

# ============================================================
# ESCENARIO: sistema de scoring crediticio
# Dos grupos con las MISMAS caracteristicas objetivas,
# pero el Grupo 0 tiene historicamente menos acceso al credito.
# Ese sesgo historico queda 'congelado' en los datos, DE DOS FORMAS:
# (1) menor probabilidad de aprobacion, y (2) un historial crediticio
# registrado peor (aunque la capacidad real de pago sea identica).
# ============================================================

def generar_dataset(n=1000):
    datos = []
    for _ in range(n):
        grupo     = np.random.randint(0, 2)
        ingresos  = np.clip(np.random.normal(0.5, 0.15), 0, 1)
        historial_real = np.clip(np.random.normal(0.5, 0.15), 0, 1)

        # Probabilidad justa basada solo en caracteristicas objetivas
        prob_justa = 0.4 * ingresos + 0.4 * historial_real + 0.05

        # SESGO HISTORICO: el grupo 0 recibio credito con 35% menos
        # de frecuencia que el grupo 1, con identicas caracteristicas.
        # Este sesgo esta 'grabado' en los datos de entrenamiento.
        if grupo == 0:
            prob = prob_justa * 0.65   # penalizacion injusta
        else:
            prob = prob_justa

        # VARIABLE PROXY: el historial REGISTRADO esta sesgado a la baja
        # para el grupo 0 (mismo poder adquisitivo real, peor historial
```

```
        # observable por falta historica de acceso al credito).
        historial_registrado = historial_real * 0.7 if grupo == 0 else historial_real

        aprobado = int(np.random.random() < np.clip(prob, 0, 1))
        datos.append([grupo, ingresos, historial_registrado, aprobado])
    return np.array(datos)


datos = generar_dataset(1000)
grupo    = datos[:, 0].astype(int)
X        = datos[:, 1:3]            # ingresos, historial (SIN grupo)
y        = datos[:, 3].astype(int)

# ================================================================
# ENTRENAMIENTO: regression logistica sin usar 'grupo' como feature
# El desarrollador cree que su modelo es justo porque no usa el grupo
# ================================================================
def sigmoid(z): return 1/(1+np.exp(-np.clip(z,-500,500)))

w = np.zeros(2); b = 0.0
for _ in range(3000):
    pred = sigmoid(X @ w + b)
    err  = pred - y
    w   -= 0.5 * (X.T @ err) / len(y)   # lr aumentado de 0.05: asegura convergencia
    b   -= 0.5 * err.mean()

pred_bin = (sigmoid(X @ w + b) > 0.5).astype(int)

# ================================================================
# ANALISIS DE SESGO: comparar resultados por grupo
# ================================================================
print('=== ANALISIS DE SESGO EN EL MODELO DE CREDITO ===\n')
print('El modelo NO usa el grupo como variable de entrada.')
print('Aun asi, aprende el sesgo historico de los datos.\n')

print(f"{'Grupo':>8}  {'Aprobados reales':>18}  {'Aprobados modelo':>18}
{'Diferencia':>12}")
print('-' * 62)
for g in [0, 1]:
    mask = grupo == g
    real    = y[mask].mean()
    modelo  = pred_bin[mask].mean()
    dif     = modelo - real
    print(f'{g:>8}  {real:>18.1%}  {modelo:>18.1%}  {dif:>+12.1%}')

# Paridad demografica
t0 = pred_bin[grupo==0].mean()
t1 = pred_bin[grupo==1].mean()
print(f'\nParidad demografica: diferencia del {abs(t0-t1):.1%}')
print(f'(0% = paridad perfecta, > 5% = señal de alerta)')
print(f'\nPrecision global: {(pred_bin==y).mean():.1%}')
print('La precision global oculta la disparidad entre grupos.')
print('Un desarrollador desatento pensaria que el modelo es justo.')
```

> Salida del programa:

```
=== ANALISIS DE SESGO EN EL MODELO DE CREDITO ===

El modelo NO usa el grupo como variable de entrada.
Aun asi, aprende el sesgo historico de los datos.

   Grupo    Aprobados reales    Aprobados modelo    Diferencia
--------------------------------------------------------------
       0               34.2%                1.6%       -32.5%
       1               45.8%               19.4%       -26.4%

Paridad demografica: diferencia del 17.7%
(0% = paridad perfecta, > 5% = señal de alerta)

Precision global: 61.4%
La precision global oculta la disparidad entre grupos.
Un desarrollador desatento pensaria que el modelo es justo.
```

ℹ️ El modelo tiene un 61% de precisión global, medida sobre los mismos datos con que se ajustó (para valorarla habría que compararla con una línea base, como predecir siempre la clase más frecuente), y nada parece alarmante a simple vista. La tasa de aprobación real ya difería entre grupos (34.2% vs 45.8%, reflejo del sesgo histórico de 0.65x), pero el modelo AMPLIFICA esa diferencia: aprueba al Grupo 0 apenas un 1.6% de las veces, frente al 19.4% del Grupo 1 — una brecha de 17.8 puntos porcentuales, frente a los 11.6 de los propios datos de partida. El modelo no discrimina intencionalmente ni usa "grupo" como variable: aprendió a penalizar el historial crediticio registrado, que está sesgado a la baja para el Grupo 0 aunque su capacidad real de pago sea la misma — exactamente como ocurre con variables proxy reales (código postal, nombre) en sistemas de scoring del mundo real.

19.6 Midiendo la equidad — Métricas de fairness

No existe una única definición matemática de 'justo'. Diferentes métricas de equidad capturan distintos aspectos de la justicia, y algunas son matemáticamente incompatibles entre sí cuando las tasas reales difieren entre grupos y el modelo comete errores. Conocerlas es el primer paso para elegir conscientemente cuál aplicar en cada contexto.

* Paridad demográfica (Statistical Parity): el modelo aprueba a todos los grupos con la misma tasa, independientemente de sus características. Fácil de medir, pero puede ser injusta si los grupos realmente difieren en la variable objetivo.
* Equal Opportunity (Igualdad de oportunidad): el modelo tiene la misma tasa de verdaderos positivos (TPR) para todos los grupos. Garantiza que quienes merecen la aprobación tienen la misma oportunidad, independientemente del grupo.
* Equalized Odds: el modelo tiene la misma TPR Y la misma FPR para todos los grupos. Más restrictivo que Equal Opportunity, asegura simetría en ambos tipos de error.
* Calibración: cuando el modelo predice un 70% de probabilidad, el 70% de esos casos realmente ocurren, para todos los grupos. Importante en medicina y finanzas.

PROGRAMA E-2 — Midiendo la equidad: métricas de fairness

📄 **programa_E2_fairness.py**

```
import numpy as np

def metricas_equidad(y_real, y_pred, grupo):
    '''Calcula las principales metricas de equidad por grupo.'''
    resultados = {}
    for g in np.unique(grupo):
        mask = grupo == g
        yr, yp = y_real[mask], y_pred[mask]
        vp = ((yp==1)&(yr==1)).sum()  # verdadero positivo
        fp = ((yp==1)&(yr==0)).sum()  # falso positivo
        vn = ((yp==0)&(yr==0)).sum()  # verdadero negativo
        fn = ((yp==0)&(yr==1)).sum()  # falso negativo
        resultados[g] = {
            'tasa_seleccion': yp.mean(),
            # sin casos en el denominador la métrica no está definida: nan, no 0
            'TPR (sensibil.)': vp/(vp+fn) if (vp+fn) else float('nan'),
            'FPR (1-espec.)':  fp/(fp+vn) if (fp+vn) else float('nan'),
            'precision_PPV':   vp/(vp+fp) if (vp+fp) else float('nan'),
            'exactitud':       (vp+vn)/len(yr),
        }
    return resultados


# Simulacion: sistema de seleccion de candidatos sesgado
np.random.seed(7)
n = 1000
grupo  = np.random.choice([0, 1], size=n, p=[0.4, 0.6])
y_real = np.random.binomial(1, 0.45, n)  # capacidad real similar
```

```
# Modelo sesgado: el grupo 0 necesita superar un umbral mas alto
y_pred = np.array([
    int(np.random.random() > (0.58 if grupo[i]==0 else 0.42))
    for i in range(n)
])

met = metricas_equidad(y_real, y_pred, grupo)

print('=== METRICAS DE EQUIDAD: SISTEMA DE SELECCION ===\n')
print(f"{'Metrica':>22}  {'Grupo 0':>10}  {'Grupo 1':>10}  {'Diferencia':>12}
{'Alerta?':>8}")
print('-' * 70)

UMBRAL_ALERTA = 0.05
metricas_mostrar = [
    'tasa_seleccion', 'TPR (sensibil.)', 'FPR (1-espec.)',
    'precision_PPV', 'exactitud'
]
for m in metricas_mostrar:
    v0  = met[0][m]
    v1  = met[1][m]
    dif = v0 - v1
    alerta = '!!! SI' if abs(dif) > UMBRAL_ALERTA else 'no'
    print(f'{m:>22}  {v0:>10.1%}  {v1:>10.1%}  {dif:>+12.1%}  {alerta:>8}')

print()
print('Interpretacion:')
print('  Paridad demografica: tasa_seleccion igual para todos los grupos')
print('  Equal Opportunity:   TPR igual (misma oportunidad si eres valido)')
print('  Equalized Odds:      TPR Y FPR iguales (simetria total)')
print()
print('Nota: estas metricas son INCOMPATIBLES entre si en muchos casos.')
print('Elegir cual garantizar es una decision etica, no tecnica.')
```

> Salida del programa:

```
=== METRICAS DE EQUIDAD: SISTEMA DE SELECCION ===

               Metrica     Grupo 0     Grupo 1   Diferencia   Alerta?
----------------------------------------------------------------------
        tasa_seleccion       39.5%       58.3%       -18.8%  !!! SI
       TPR (sensibil.)       37.8%       55.1%       -17.3%  !!! SI
       FPR (1-espec.)        40.8%       60.7%       -19.8%  !!! SI
         precision_PPV       43.5%       40.6%        +2.9%      no
             exactitud       49.5%       46.1%        +3.4%      no

Interpretacion:
  Paridad demografica: tasa_seleccion igual para todos los grupos
  Equal Opportunity:   TPR igual (misma oportunidad si eres valido)
  Equalized Odds:      TPR Y FPR iguales (simetria total)

Nota: estas metricas son INCOMPATIBLES entre si en muchos casos.
Elegir cual garantizar es una decision etica, no tecnica.
```

19.7 Técnicas para reducir el sesgo

Una vez detectado el sesgo, existen diversas técnicas para mitigarlo. Ninguna es perfecta — todas implican compensaciones — pero conocerlas permite tomar decisiones informadas:

En los datos

* Rebalanceo: asegúrate de que todos los grupos estén suficientemente representados en el conjunto de entrenamiento. Técnicas como SMOTE generan ejemplos sintéticos de los grupos subrepresentados.
* Reweighting: asigna mayor peso a los ejemplos de grupos subrepresentados durante el entrenamiento, para que el modelo les preste más atención.
* Auditoría de etiquetas: revisa las etiquetas con equipos diversos para detectar inconsistencias sistemáticas entre grupos.

En el modelo

* Regularización de equidad: añade a la función de pérdida un término que penaliza las diferencias en métricas de equidad entre grupos. El modelo aprende a optimizar rendimiento y equidad simultáneamente.
* Umbralización diferencial: usa umbrales de decisión distintos para cada grupo, calibrados para que las métricas de equidad deseadas se cumplan.
* Debiasing adversarial: entrena un segundo modelo que intenta predecir el grupo a partir de las representaciones internas del modelo principal. El modelo principal aprende a confundir a ese adversario, produciendo representaciones que no codifican información del grupo.

En la evaluación

* Evalúa siempre por subgrupos: reportar solo la métrica global puede esconder disparidades graves. Desglosa precisión, recall y otras métricas por cada grupo demográfico relevante.
* Red-teaming: antes del despliegue, organiza equipos que busquen activamente fallos y sesgos en el sistema, especialmente con casos límite de grupos minoritarios.

> 💡 Ninguna técnica elimina el sesgo completamente. La mejor práctica es combinar varias y evaluar continuamente en producción, porque los datos y la sociedad evolucionan y el sesgo puede reaparecer.

19.8 Transparencia y explicabilidad

Un modelo de IA puede ser muy preciso y aun así inaceptable si nadie puede entender cómo toma sus decisiones. La opacidad de los modelos de Deep Learning —a menudo llamados 'cajas negras'— plantea problemas serios en contextos de alto impacto.

Imagina que tu solicitud de hipoteca es denegada por un modelo de IA. ¿Tienes derecho a saber por qué? En la Unión Europea, el RGPD te da derecho a no ser objeto de decisiones basadas únicamente en un tratamiento automatizado que te afecten de forma significativa, salvo excepciones y con garantías (artículo 22), y a recibir información significativa sobre la lógica aplicada (artículos 13 a 15). En febrero de 2025, el Tribunal de Justicia de la UE precisó, en el asunto C-203/22, sobre una evaluación automatizada de solvencia, que esa información debe explicar de forma comprensible el procedimiento y los principios aplicados; hasta dónde llega ese 'derecho a una explicación' en cada caso sigue discutiéndose. Un modelo que no puede explicarse difícilmente puede cumplir este requisito.

El mecanismo de atención como ventana de explicabilidad

Una de las ventajas inesperadas del mecanismo de atención que hemos estudiado (recuerda el Diagrama 11.2) es que ofrece cierta interpretabilidad. Los pesos de atención muestran qué partes del input influyeron más en la predicción. Aunque esta interpretación debe tomarse con cautela —los pesos de atención no siempre corresponden directamente a la importancia causal—, son una herramienta valiosa para auditar comportamientos sospechosos.

> En la sección 15.3 leíste la atención de un modelo que suma y descubriste en ella el algoritmo de la suma en columnas: al escribir las decenas, una cabeza trae las unidades del segundo sumando para compararlas con la cifra de las unidades que el modelo acaba de escribir: ahí está la llevada. Es un ejemplo real de interpretabilidad, y también de su límite: los pesos dicen dónde mira el modelo, no qué calcula con lo que ve.

LIME y SHAP

Para modelos de caja negra, existen técnicas de explicabilidad local post-hoc. LIME (Local Interpretable Model-agnostic Explanations) entrena un modelo lineal simple alrededor de cada predicción individual, aproximando el comportamiento local. SHAP (SHapley Additive

exPlanations) usa teoría de juegos para asignar a cada feature una contribución justa a la predicción final. Ambas permiten preguntar: 'Para esta predicción concreta, ¿qué features fueron más importantes?'

Estas herramientas son especialmente útiles cuando el modelo produce resultados inesperados o discriminatorios: pueden revelar si el modelo está basándose en proxies injustos.

19.9 Regulación: el AI Act de la Unión Europea

La Unión Europea aprobó en 2024 el AI Act (Reglamento (UE) 2024/1689), la primera ley integral sobre Inteligencia Artificial del mundo. Entró en vigor el 1 de agosto de 2024 y se aplica por fases. Clasifica los sistemas de IA según su nivel de riesgo:

* Riesgo inaceptable (prohibidos): sistemas que manipulan subliminalmente a las personas, la puntuación social de las personas, el reconocimiento biométrico en tiempo real en espacios públicos con fines policiales (con excepciones tasadas).
* Alto riesgo: sistemas que afectan infraestructuras críticas, educación, empleo, servicios esenciales, migración, justicia. Requieren análisis de riesgos, datasets de alta calidad, transparencia, supervisión humana y registro ante autoridades.
* Riesgo limitado: chatbots y sistemas que interactúan con personas. Deben informar al usuario de que está interactuando con una IA.
* Riesgo mínimo: la gran mayoría de aplicaciones de IA. Sin requisitos específicos, aunque se promueven códigos de conducta voluntarios.
* Modelos de propósito general, como los LLM: documentación técnica, un resumen de los datos de entrenamiento y una política de respeto a los derechos de autor; los de riesgo sistémico (los más grandes) deben además evaluarse, notificar incidentes graves y garantizar su ciberseguridad.

El calendario, tal como estaba en octubre de 2026: las prohibiciones se aplican desde el 2 de febrero de 2025; las obligaciones de los modelos de propósito general, desde el 2 de agosto de 2025, y la mayoría de las demás disposiciones, desde el 2 de agosto de 2026. En 2026 la UE aprobó un paquete de simplificación, el «Ómnibus digital» (Reglamento (UE) 2026/1744, en vigor desde el 27 de julio de 2026), que retrasó las obligaciones de los sistemas de alto riesgo: hasta el 2 de diciembre de 2027 para los de usos como el empleo, la educación o el crédito, y hasta el 2 de agosto de 2028 para los integrados en productos ya regulados (máquinas, dispositivos médicos…). Que un sistema se use en uno de esos ámbitos no lo convierte automáticamente en «de alto riesgo»: el reglamento fija condiciones y excepciones. Antes de un proyecto real, comprueba el calendario vigente en la web de la Comisión Europea (consultada en noviembre de 2026).

Para los desarrolladores, el AI Act no es solo un requisito legal: es un marco que define lo que la sociedad considera aceptable. Entenderlo es parte de la formación profesional de cualquier ingeniero de IA en Europa.

> ℹ️ Aunque el AI Act es europeo, tiene efecto extraterritorial: aplica a cualquier sistema de IA cuyos outputs se usen en la UE, independientemente de dónde esté desarrollado. Es el equivalente al RGPD para los datos personales, pero para los sistemas de IA.

19.10 Model Cards y Datasheets: documentar para la transparencia

Dos herramientas prácticas se han convertido en estándar de la industria para la transparencia en IA:

Model Cards (Google, 2019)

Un Model Card es un documento breve que acompaña a cada modelo publicado, describiendo: el propósito del modelo, sus limitaciones, el rendimiento desglosado por subgrupos, consideraciones éticas y casos de uso no recomendados. Es el equivalente del

prospecto de un medicamento: te dice para qué sirve, sus efectos secundarios y cuándo no debe usarse.

Datasheets for Datasets (Microsoft Research, 2018)

Un Datasheet describe el dataset: cómo fue recopilado, quién lo recopiló, bajo qué condiciones, qué sesgos conocidos tiene, cómo debe y no debe usarse. Permite a quienes entrenan modelos tomar decisiones informadas sobre si un dataset es adecuado para su uso.

Incluir Model Cards y Datasheets en tus proyectos, aunque sean personales o académicos, es una práctica profesional que marca la diferencia y que cada vez más empleadores y publicaciones científicas exigen.

19.11 Tu responsabilidad como desarrollador

Terminemos este capítulo con algo personal. Has aprendido a construir sistemas de IA desde cero. Ese conocimiento te da poder. Y ese poder viene con responsabilidades concretas:

* Pregunta siempre: ¿quién se beneficia de este sistema y quién puede verse perjudicado? No des por sentado que la neutralidad técnica existe.
* Analiza los datos antes de entrenar: ¿representan bien a todos los grupos afectados? ¿Tienen sesgos históricos? ¿Quién los recopiló y con qué propósito?
* Documenta tus modelos: si alguien va a usar tu sistema, merece saber qué puede y no puede hacer, y dónde falla.
* Actúa si algo no te parece bien: la presión para desplegar rápido es real en la industria. Señalar problemas éticos antes del despliegue es más efectivo —y más ético— que lamentarlos después.
* Rodéate de diversidad: los equipos homogéneos no detectan los sesgos que afectan a quienes están fuera de su propia experiencia. La diversidad en los equipos de desarrollo es una medida de seguridad técnica.

La ética en IA no es una asignatura separada de la ingeniería. Es parte de hacer bien el trabajo. Un sistema que discrimina, aunque funcione perfectamente desde el punto de vista técnico, es un sistema mal diseñado.

> ℹ️ 'La tecnología no es buena ni mala; tampoco es neutral.' — Melvin Kranzberg (1986). Esta frase, aplicada a la IA, resume perfectamente por qué la ética no es opcional: los sistemas que construimos reflejan los valores de quienes los diseñan, quieran o no.

19.12 Para profundizar

* 'Weapons of Math Destruction' — Cathy O'Neil (2016). El libro más accesible sobre el impacto social de los algoritmos.
* 'The Alignment Problem' — Brian Christian (2020). Sobre los desafíos de hacer que la IA haga lo que realmente queremos.
* 'Data Feminism' — Catherine D'Ignazio y Lauren Klein (2020). Sobre el poder y los sesgos en la ciencia de datos.
* Fairlearn (fairlearn.org): librería de código abierto, iniciada por Microsoft y hoy mantenida por su comunidad, para medir y mitigar el sesgo en modelos de ML con Python.
* AI Fairness 360 (github.com/Trusted-AI/AIF360): librería de código abierto, iniciada por IBM, con decenas de métricas de equidad y 15 algoritmos para mitigar sesgos.
* Model Cards de Hugging Face (huggingface.co/docs/hub/model-cards): la plantilla y la guía con las que se documenta cada modelo publicado en el Hub.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - Ningún sistema de IA es neutral: los datos de entrenamiento pueden codificar y amplificar sesgos históricos.
> - Un modelo puede discriminar sin usar nunca la variable protegida directamente, a través de proxies correlacionados.
> - Las distintas métricas de equidad no pueden cumplirse todas a la vez: elegir cuál priorizar es una decisión ética.

Capítulo 20: Herramientas y próximos pasos (*)

> 🎯 **AL TERMINAR ESTE CAPÍTULO SABRÁS:**
> ✓ Conocer las herramientas que la industria usa hoy para trabajar con LLMs reales.
> ✓ Cargar y ejecutar un modelo preentrenado con Hugging Face Transformers en pocas líneas.
> ✓ Entender qué es RAG (Retrieval-Augmented Generation) y por qué resuelve un problema real.
> ✓ Explicar cómo CNN y Transformer se combinan en los modelos multimodales actuales.
> ✓ Diseñar tu propia hoja de ruta de aprendizaje según tu nivel actual.
> ✓ Conocer los recursos, cursos y comunidades más relevantes para continuar.

20.1 De la teoría a la caja de herramientas

Has construido cada componente de un LLM desde cero: neuronas, atención, Transformer, tokenización BPE, optimizadores. Ese conocimiento es exactamente lo que necesitas para entender —y no solo usar— el ecosistema de herramientas que la industria emplea cada día. Este capítulo es un puente corto entre lo que has aprendido y lo que encontrarás nada más empezar a trabajar con IA en el mundo real.

20.2 Hugging Face Transformers — Modelos preentrenados en 3 líneas

Hugging Face es la plataforma central del ecosistema open-source de IA: aloja más de un millón de modelos preentrenados, datasets y tokenizadores listos para usar. Su librería transformers es el estándar de facto para trabajar con Transformers en Python.

Lo notable es que, gracias a todo lo que ya entiendes, el código siguiente no es magia: es exactamente el mismo pipeline que construiste en el Capítulo 13, con un modelo ya entrenado en lugar del tuyo: el mismo GPT-2 que moviste con tu propio código en el Capítulo 16.

📄 **Ejemplo de uso real de Hugging Face Transformers**

```
from transformers import pipeline

# Cargar un modelo preentrenado de generación de texto
# (descarga los pesos la primera vez, despues los cachea localmente)
generador = pipeline('text-generation', model='gpt2')

resultado = generador(
    'La inteligencia artificial aprende',
    max_new_tokens=40,      # tokens nuevos, sin contar el prompt
    do_sample=True,         # muestrear (si no, elige siempre el mas probable)
    temperature=0.8,        # el mismo parametro del Capitulo 13
    top_p=0.9,              # el mismo Top-p del Capitulo 14
    num_return_sequences=1
)
print(resultado[0]['generated_text'])

# ------------------------------------------------------------
# Tokenizador BPE real, identico en espiritu al Programa BPE-1/2
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained('gpt2')
tokens = tok.encode('la inteligencia artificial aprende')
print(f'Tokens: {tokens}')
print(f'Decodificado: {tok.decode(tokens)}')
```

> ℹ️ Detrás de pipeline('text-generation') ocurre exactamente lo que implementaste en los Programas 8 y 8 bis: tokenización BPE, embeddings, bloques Transformer apilados, una capa de salida y muestreo con temperatura y Top-p. La diferencia es

> la escala: 124 millones de parámetros en vez de 60 000, entrenados sobre unos 40 GB de texto en vez de 93 frases.

20.3 Ejecutar LLMs localmente con Ollama

Ollama es una herramienta que simplifica radicalmente la ejecución de LLMs open-source en tu propio ordenador, sin depender de una API externa: con un modelo descargado y sin usar sus funciones en la nube, tus textos no salen del equipo. Permite descargar y ejecutar modelos como LLaMA, Mistral o Gemma con un solo comando.

📄 Uso típico de Ollama desde la terminal

```
# Instalar Ollama (una sola vez): https://ollama.com

# Descargar y ejecutar un modelo interactivamente
ollama run llama3

# Uso programatico desde Python
import requests

respuesta = requests.post('http://localhost:11434/api/generate', json={
    'model': 'llama3',
    'prompt': 'Explica que es el self-attention en una frase',
    'stream': False
})
print(respuesta.json()['response'])
```

Ejecutar modelos localmente tiene ventajas claras: privacidad (con un modelo descargado y sin funciones en la nube, los datos no salen de tu máquina), coste cero por token, y control completo sobre el modelo. La desventaja es que necesitas hardware capaz de cargar el modelo en memoria — un modelo de 7000 millones de parámetros en precisión reducida (int4) puede ejecutarse en una GPU de consumo con 8 GB de memoria, como vimos en los ejercicios del Capítulo 17.

20.4 RAG — Retrieval-Augmented Generation

Un LLM tiene dos limitaciones fundamentales: su conocimiento termina en la fecha de su entrenamiento, y no tiene acceso a documentos privados o específicos de una organización. RAG (Generación Aumentada por Recuperación) resuelve ambos problemas combinando un sistema de búsqueda con el LLM.

La idea, sorprendentemente, usa exactamente los embeddings y la similitud coseno que aprendiste en el Capítulo 8. El ejemplo siguiente es una simulación geométrica: los embeddings son números al azar y la «pregunta» se fabrica cerca del documento 2, así que solo muestra cómo se ordena por similitud coseno. Un RAG real calcula los embeddings de los textos y de la pregunta con el mismo modelo entrenado, y después llama al LLM:

DIAGRAMA 20.1 — RAG: RETRIEVAL-AUGMENTED GENERATION

Resumen visual del Capítulo 20 — El ecosistema práctico

Documentos → Embeddings → Vector Store → Recuperar → Inyectar → Respuesta

RAG soluciona dos limitaciones del LLM: fecha de corte y acceso a datos privados

EL ECOSISTEMA PRÁCTICO

De la teoría a la caja de herramientas

Dos limitaciones del LLM:

1. **FECHA DE CORTE:** su conocimiento termina en la fecha de entrenamiento.
   Pregunta: "¿Qué pasó ayer?" → *No sabe.*
2. **DATOS PRIVADOS:** no tiene acceso a documentos de tu empresa.
   Pregunta: "¿Qué dice el contrato X?" → *No sabe.*

Hugging Face — modelos en 3 líneas

```
from transformers import pipeline
gen = pipeline('text-generation',
               model='gpt2')
out = gen('Hola, me llamo')
print(out[0]['generated_text'])
```

Internamente: BPE → Transformer → softmax
→ exactamente lo que implementaste en el Programa 8

Ollama — LLMs locales

```
# Terminal:
ollama run llama3

# Python:
import ollama
r = ollama.chat('llama3',
      messages=[{'role':'user',
                 'content':'Hola'}])
```

Privacidad: modelo descargado, sin funciones en la nube

✓ Privacidad ✓ Sin coste API ✓ Offline

el pipeline RAG completo

EL PIPELINE RAG — PASO A PASO

Dos fases: INDEXACIÓN (offline) y CONSULTA (online)

① FASE DE INDEXACIÓN (se hace UNA vez)

- **A — DOCUMENTOS**
  Textos, PDFs, páginas web, contratos, manuales...
- **B — CHUNKING**
  Dividir en fragmentos de ~500 palabras con solapamiento para no perder contexto
  `chunk_size=500, overlap=50`
- **C — EMBEDDING**
  Cada chunk → vector de 768 dimensiones usando un modelo de embedding
  `vec = embed_model.encode(chunk)`
- **D — VECTOR STORE**
  Índice de vectores para búsqueda rápida
  FAISS, Chroma, Pinecone, Weaviate
  Almacena: (vector, texto_original, metadatos)

— — — — — tiempo después — — — — —

② FASE DE CONSULTA (en tiempo real)

- **E — PREGUNTA DEL USUARIO**
  "¿Qué dice el contrato sobre devoluciones?"
- **F — EMBEDDING DE LA PREGUNTA**
  La pregunta también → vector 768D
  `q_vec = embed_model.encode(pregunta)`
- **G — BÚSQUEDA POR SIMILITUD COSENO**
  Comparar q_vec con TODOS los vectores en el Vector Store
  `resultados = store.search(q_vec, k=3)`
- **H — TOP-K CHUNKS RECUPERADOS**
  Los 3 fragmentos más similares
  k=3 es configurable
- **I — CONTEXTO AUMENTADO → LLM**
  System prompt + chunks + pregunta

```
prompt = f'''Usa solo esta información:
{chunk_1}  {chunk_2}  {chunk_3}
Pregunta: {pregunta}'''
```

- **J — RESPUESTA FUNDAMENTADA**
  "Según la cláusula 7 del contrato, las devoluciones deben solicitarse en 30 días."
  cita la fuente, no alucina

por qué funciona (similitud coseno)

POR QUÉ FUNCIONA RAG

La similitud coseno del Capítulo 8 en acción

El puente entre capítulos:

En el Capítulo 8 aprendiste que:

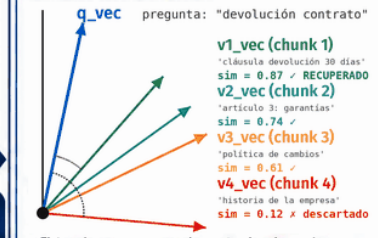
- Las palabras similares tienen vectores similares (alta sim. coseno)
- "rey" − "hombre" + "mujer" ≈ "reina"

RAG extiende esto a FRAGMENTOS completos:

- Chunk sobre "devoluciones" tiene vector similar a pregunta sobre "devoluciones"
- La similitud coseno los conecta

→ Semántica, no palabras clave exactas

Similitud coseno en acción:



El ángulo entre vectores determina la relevancia

Top-3 resultados del Vector Store:

Rank	Similitud	Fragmento
1	0.87	Cláusula 7: devoluciones...
2	0.74	Artículo 3: garantías...
3	0.61	Política de cambios...

Sin RAG — el LLM alucina:

"Las devoluciones están permitidas en los primeros 14 días." ← INVENTADO

Con RAG — respuesta fundamentada:

"Según la cláusula 7, las devoluciones se solicitan en 30 días." ← DEL DOCUMENTO

herramientas y multimodalidad

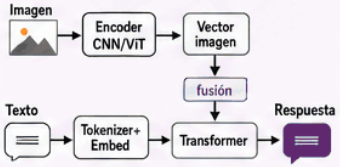
ECOSISTEMA Y MULTIMODALIDAD

El ecosistema de herramientas LLM:

Herramienta	Para qué	Nivel
HuggingFace	Modelos preentrenados	★★★
Ollama	LLMs locales	★★☆
LangChain	Orquestar pipelines RAG	★★★
FAISS	Vector Store local	★★☆
Chroma	Vector Store embebido	★☆☆
OpenAI API	GPT-4, embeddings	★★★

Modelos Multimodales

CNN + Transformer = visión + lenguaje



Modelo	Capacidades
GPT-4V	texto + imágenes
Gemini	texto + imágenes + audio
LLaMA-3.2V	texto + imágenes (open)
CLIP	imagen ↔ texto (embeddings)

Con los Caps. 12 y 20 y el Ap. G tienes las bases de TODOS estos modelos.

RAG vs Fine-tuning — ¿cuándo usar cada uno?

RAG	Fine-tuning
Datos cambian frecuentemente ✓	Datos estables ✓
Documentos privados ✓	Cambiar el estilo ✓
Sin re-entrenar ✓	Respuestas más fluidas ✓
Fuentes citables ✓	Sin necesitar retrieval ✓
Alucinaciones reducidas ✓	Caro en GPU ✗ si cambia

RAG primero — fine-tune solo si no es suficiente.

INDEXAR: docs → chunks → embed → vector store

CONSULTAR: query → embed → search → retrieve → inject → LLM

$$sim_coseno(q, doc) = (q \cdot doc) \;/\; (|q| \cdot |doc|)$$

HuggingFace + Ollama: modelos listos

Vector Store: búsqueda semántica

RAG: LLM + conocimiento externo

- Blue = Query / HuggingFace / entrada
- Teal = Embeddings / Vector Store
- Green = Documentos recuperados
- Orange = Contexto aumentado / LLM
- Purple = Respuesta final / multimodal
- Red = Limitaciones / alucinaciones

```
# (en produccion: Pinecone, Chroma, Weaviate, FAISS...)
documentos = [
    'El BPE fusiona pares de caracteres frecuentes iterativamente.',
    'El self-attention conecta cualquier par de tokens directamente.',
    'AdamW desacopla el weight decay del gradiente.',
]

# Embeddings simulados (en produccion: un modelo de embeddings real)
np.random.seed(1)
embeddings_docs = np.random.randn(len(documentos), 8)

def sim_coseno(u, v):
    return np.dot(u,v) / (np.linalg.norm(u)*np.linalg.norm(v) + 1e-8)

def buscar_relevantes(pregunta_emb, k=1):
    '''Recupera los k documentos mas similares (igual que en Cap. 8)'''
    similitudes = [sim_coseno(pregunta_emb, d) for d in embeddings_docs]
    indices = np.argsort(similitudes)[::-1][:k]
    return [documentos[i] for i in indices]

# Simular pregunta del usuario sobre AdamW
pregunta_emb = embeddings_docs[2] + np.random.randn(8)*0.1  # similar al doc 2
relevantes = buscar_relevantes(pregunta_emb, k=1)

print('Fragmento recuperado:', relevantes[0])
print()
print('Prompt final enviado al LLM:')
print(f'  Contexto: "{relevantes[0]}"')
print(f'  Pregunta: "Que hace AdamW de diferente a Adam?"')
print(f'  -> El LLM responde usando el contexto recuperado.')
```

> 💡 RAG es, en esencia, el ejercicio de similitud coseno del Capítulo 8 aplicado a millones de documentos con una base de datos vectorial optimizada para búsquedas de vecinos más cercanos. El LLM nunca 'aprende' los documentos nuevos: simplemente los lee como contexto en cada consulta, igual que tú leerías un libro abierto durante un examen.

20.5 Multimodalidad — Cuando CNN y Transformer se Unen

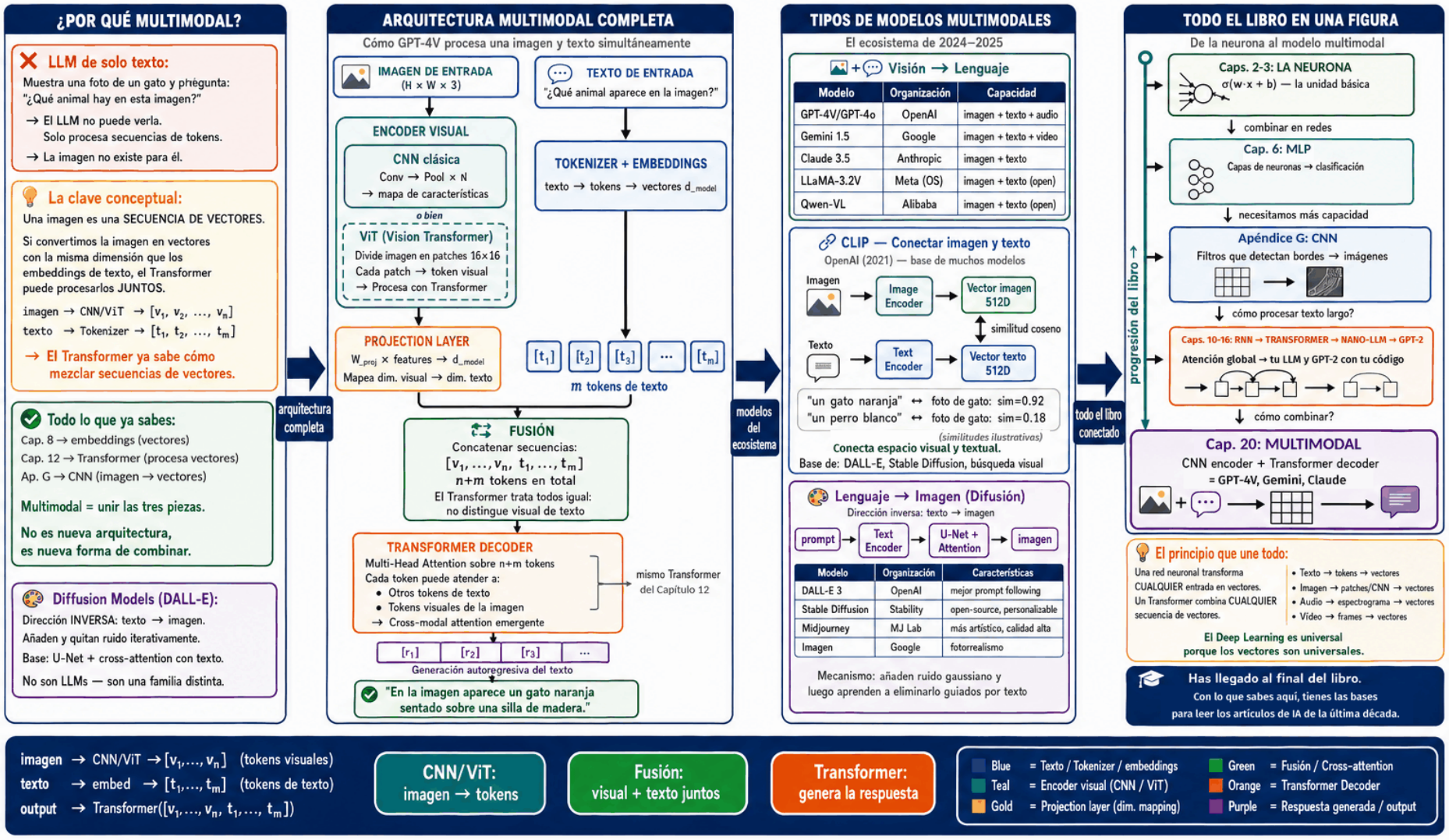
Los grandes modelos actuales —GPT, Gemini, Claude y muchos modelos abiertos— procesan texto e imágenes a la vez. La receta más extendida se entiende con lo que ya sabes: un encoder visual trocea la imagen en parches y convierte cada parche en un vector; una pequeña capa de proyección lleva esos vectores a la dimensión de los embeddings de texto, y el Transformer del Capítulo 12 los procesa junto con los tokens, como si fueran palabras. Durante años ese encoder fue una CNN como la del Apéndice G; hoy suele ser un Vision Transformer (ViT), que aplica la atención del Capítulo 11 a los parches. Algunos modelos, como Flamingo (DeepMind, 2022), usan en cambio atención cruzada (Apéndice F.3): el texto consulta a la imagen. El camino inverso, generar imágenes a partir de texto, suele usar otra familia: los modelos de difusión.

DIAGRAMA 20.2 — MODELOS MULTIMODALES

Resumen visual del Capítulo 20 — Cuando CNN y Transformer se unen

Imagen → Encoder Visual → Tokens Visuales ↔ Tokens de Texto → Transformer → Respuesta

GPT-4V, Gemini, LLaMA-3.2V: el mismo principio que aprendiste, a escala masiva



La idea central de un modelo de difusión es sorprendentemente elegante: se entrena una red (a menudo basada en componentes convolucionales tipo U-Net, con bloques de atención intercalados) para revertir un proceso de añadir ruido gaussiano progresivamente a una imagen. Durante el entrenamiento, el modelo aprende a predecir y eliminar el ruido en cada paso. Durante la generación, se parte de ruido puro y se aplica el proceso inverso paso a paso hasta obtener una imagen coherente.

📄 La idea de difusión, en pseudocódigo conceptual (simplificado)

```
# ENTRENAMIENTO: aprender a eliminar ruido
for imagen in dataset:
    nivel_ruido = random_nivel()
    imagen_ruidosa = imagen + ruido_gaussiano(nivel_ruido)
    ruido_predicho = modelo(imagen_ruidosa, nivel_ruido)
    loss = MSE(ruido_predicho, ruido_real)   # el modelo aprende a predecir EL RUIDO
    backward(loss)  # exactamente igual que en todo el libro

# GENERACION: partir de ruido puro y 'limpiar' iterativamente
imagen = ruido_puro_aleatorio()
for paso in range(1000, 0, -1):              # de mucho ruido a ninguno
    ruido_estimado = modelo(imagen, paso)
    imagen = imagen - factor(paso) * ruido_estimado   # eliminar un poco de ruido

# En los modelos reales la mezcla es sqrt(a)*imagen + sqrt(1-a)*ruido,
# y al generar se añade también un poco de ruido nuevo en cada paso.
# Tras 1000 pasos: una imagen coherente y nueva
```

Los modelos de texto a imagen como Stable Diffusion añaden el texto como condición: en cada paso de 'limpieza', el modelo recibe también el embedding del prompt de texto (procesado por un Transformer, como en este libro) y lo usa, vía cross-attention, para guiar qué imagen debe emerger del ruido. La conexión con todo lo que has aprendido es real: backpropagation, gradiente descendente, optimizadores como AdamW, y mecanismos de atención — la misma caja de herramientas, aplicada a un problema distinto.

> ℹ️ No hemos implementado un modelo de difusión en este libro porque su arquitectura central (U-Net convolucional con atención) y su objetivo de entrenamiento (predecir ruido en lugar de predecir el siguiente token) merecerían un libro propio. Pero ahora entiendes lo esencial: optimización por gradiente, redes profundas, y atención — los mismos tres pilares de todo lo que has construido aquí.

20.6 La IA no es un destino, es un camino

Llevas varios capítulos construyendo desde cero una neurona, una red, un mecanismo de atención y un modelo de lenguaje completo. Eso no lo hace la mayoría de las personas que usan herramientas de IA. Tú entiendes por dentro lo que ellas solo usan por fuera. Eso es una ventaja enorme.

La IA está en uno de los momentos más apasionantes de su historia. Los modelos se vuelven más capaces cada pocos meses. Las aplicaciones se multiplican en medicina, ciencia, arte, ingeniería, educación. Y el campo sigue necesitando personas que entiendan los fundamentos, que piensen críticamente sobre el diseño de los modelos, y que empujen los límites de lo posible.

20.7 Próximos pasos recomendados

Si este libro despertó tu curiosidad, aquí tienes una hoja de ruta para continuar:

Nivel 1: Consolidar los fundamentos (1-3 meses)

* Implementa todos los programas de este libro y modifícalos: cambia arquitecturas, hiperparámetros, datos de entrenamiento.
* Aprende PyTorch: el framework de IA más usado en investigación. Reimplementa el Nano-LLM en PyTorch y verás cuánto más sencillo es el entrenamiento con autograd, que calcula el backward por ti.
* Estudia el artículo original 'Attention is All You Need' (disponible gratis en arxiv.org/abs/1706.03762).

Nivel 2: Proyectos propios (3-12 meses)

* Entrena un modelo de caracteres sobre un libro completo de Gutenberg. Con 10 horas de entrenamiento en una GPU modesta, podrás generar prosa legible.
* Amplía el tokenizador BPE del Capítulo 9 a nivel de byte (9.4), como los que usan GPT y LLaMA, y entrénalo con un corpus mayor.
* Experimenta con fine-tuning de un modelo open-source pequeño (GPT-2 de 124M parámetros) sobre datos propios.
* Amplía el Capítulo 15 con tareas nuevas (restas, ordenar listas, encadenar condiciones) y comprueba, con datos apartados y varias semillas, si tu modelo memoriza o generaliza.

Nivel 3: Investigación y contribución (1-3 años)

* Estudia un Máster en Machine Learning, Data Science o Inteligencia Artificial.
* Contribuye a proyectos open-source: HuggingFace Transformers, llama.cpp, etc.
* Lee los artículos de investigación más influyentes: GPT, BERT, Scaling Laws, Constitutional AI.

20.8 Recursos para seguir aprendiendo

Estos son algunos de los mejores recursos gratuitos disponibles hoy:

Cursos online

* fast.ai — Practical Deep Learning for Coders (práctico, motivador).
* Deep Learning Specialization de Andrew Ng (Coursera) — base teórica sólida.
* CS231n y CS224n de Stanford — visión por computador y NLP, nivel universitario.
* Andrej Karpathy — canal de YouTube con vídeos sobre LLMs desde cero.

Libros

* 'Deep Learning' de Goodfellow, Bengio y Courville — la referencia académica.
* 'Hands-On Machine Learning' de Aurélien Géron — muy práctico con scikit-learn y TensorFlow.
* 'The Little Book of Deep Learning' de François Fleuret — conciso y riguroso.

Comunidad

* Hugging Face Papers (huggingface.co/papers) — artículos recientes con enlaces a su código, modelos y datasets. (Papers With Code, que ocupaba este lugar, fue cerrado por Meta en julio de 2025.)
* Hugging Face Hub (huggingface.co) — modelos, datasets y comunidad.
* Kaggle — competiciones de ML con datasets reales y premios.

20.9 Una reflexión final

La Inteligencia Artificial es probablemente la tecnología más transformadora que la humanidad haya desarrollado. Como toda tecnología poderosa, puede usarse para bien o para mal. La diferencia está en las personas que la diseñan, la implementan y la gobiernan.

Necesitamos ingenieros e investigadores que no solo sepan construir modelos de IA, sino que entiendan sus limitaciones, sus sesgos, sus implicaciones éticas y sociales. Necesitamos personas que hagan preguntas incómodas: ¿Este sistema es justo? ¿Es transparente? ¿Puede causar daño?

Si has llegado hasta aquí, tienes el conocimiento técnico. Cultiva también la curiosidad, el pensamiento crítico y la responsabilidad. El futuro de la IA depende de personas como tú.

> 💡 'El mejor momento para empezar a aprender sobre IA fue hace cinco años. El segundo mejor momento es ahora.' — Adaptado del proverbio chino sobre plantar árboles.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE CAPÍTULO**
>
> - Hugging Face Transformers permite usar modelos preentrenados con el mismo pipeline que construiste: tokenizar, codificar, generar.
> - Ollama facilita ejecutar LLMs open-source localmente, sin coste por token y, en modo local, sin que tus datos salgan del equipo.
> - RAG combina recuperación por similitud (Capítulo 8) con generación de texto para dar a los LLMs acceso a información actualizada y privada.
> - Los modelos multimodales unen un encoder visual (CNN/ViT) con un Transformer compartido; los modelos de difusión usan una arquitectura y objetivo distintos pero los mismos pilares de optimización.
> - **El camino sigue con tres niveles: consolidar los fundamentos, construir proyectos propios e investigar o contribuir. La técnica necesita, además, pensamiento crítico y responsabilidad.**

✏️ EJERCICIOS — Capítulo 20 — Herramientas y próximos pasos

Ya cerca del final del recorrido, estos ejercicios te invitan a dar tus propios pasos. Resuélvelos y compara tu solución con la propuesta: quizá descubras alternativas que no habías considerado.

Ejercicio 20.1 Construir un sistema RAG mínimo

Extiende el ejemplo de RAG del capítulo: crea una base de conocimiento de 6 documentos sobre distintos temas del libro (neuronas, atención, optimizadores, BPE, overfitting, ética). Implementa una función que, dada una pregunta, recupere los 2 documentos más relevantes usando similitud coseno, igual que en el Capítulo 8.

> 💡 💡 Pista: Reutiliza la función sim_coseno() del Capítulo 8 y del ejemplo de RAG de este capítulo. La clave es ordenar los documentos por similitud descendente y devolver los k primeros.

✅ Solución:

📄 ejercicio_20_1.py

```
import numpy as np

np.random.seed(7)

documentos = [
    'La neurona artificial calcula una suma ponderada y aplica una activacion.',
    'El self-attention conecta directamente cualquier par de tokens.',
    'Adam combina Momentum y RMSprop con correccion de sesgo.',
    'BPE fusiona iterativamente los pares de caracteres mas frecuentes.',
    'El overfitting ocurre cuando el modelo memoriza en vez de generalizar.',
    'Un modelo puede discriminar sin usar la variable protegida directamente.',
]
temas = ['neurona', 'atencion', 'optimizador', 'bpe', 'overfitting', 'etica']

# Embeddings simulados: en la practica vendrian de un modelo real
# Para la demo, generamos embeddings correlacionados con el tema
D = 16
embeddings_docs = np.random.randn(len(documentos), D) * 0.5

def sim_coseno(u, v):
    return np.dot(u,v) / (np.linalg.norm(u)*np.linalg.norm(v) + 1e-8)

def rag_buscar(pregunta_emb, k=2):
    sims = [sim_coseno(pregunta_emb, d) for d in embeddings_docs]
    idx  = np.argsort(sims)[::-1][:k]
    return [(documentos[i], temas[i], sims[i]) for i in idx]

# Simulamos una pregunta 'sobre optimizadores'
pregunta_emb = embeddings_docs[2] + np.random.randn(D)*0.15

print('=== SISTEMA RAG MINIMO ===\n')
print('Pregunta simulada: relacionada con optimizadores\n')
resultados = rag_buscar(pregunta_emb, k=2)
for doc, tema, sim in resultados:
    print(f'  [{tema:>12}] sim={sim:.3f}  "{doc}"')

print()
print('El LLM recibiria estos 2 fragmentos como contexto adicional')
print('antes de generar su respuesta a la pregunta del usuario.')
```

Ejercicio 20.2 Calcular el coste de tokens de una consulta con RAG

Una consulta RAG típica envía al LLM: el prompt del sistema (~200 tokens), los fragmentos recuperados (~150 tokens cada uno) y la pregunta del usuario (~30 tokens). Si recuperas

k=3, k=5 y k=10 fragmentos, calcula el total de tokens enviados y el coste aproximado si el modelo cobra $0.01 por cada 1000 tokens de entrada.

> 💡 💡 Pista: El coste de RAG crece linealmente con k (el número de fragmentos recuperados). Hay un compromiso entre dar más contexto al modelo (mejor respuesta) y el coste/latencia de procesar más tokens.

✅ Solución:

📄 **ejercicio_20_2.py**

```
def costo_consulta_rag(k, tokens_sistema=200, tokens_por_fragmento=150,
                       tokens_pregunta=30, precio_por_1k=0.01):
    total_tokens = tokens_sistema + (k * tokens_por_fragmento) + tokens_pregunta
    costo = total_tokens / 1000 * precio_por_1k
    return total_tokens, costo

print('=== COSTE DE TOKENS SEGUN K (fragmentos recuperados) ===\n')
print(f"{'k':>4}  {'Tokens totales':>15}  {'Costo (USD)':>12}")
print('-' * 36)

for k in [1, 3, 5, 10, 20]:
    tokens, costo = costo_consulta_rag(k)
    print(f'{k:>4}  {tokens:>15}  ${costo:>11.5f}')

print()
print('A 1000 consultas/dia con k=5:')
tokens5, costo5 = costo_consulta_rag(5)
print(f'  Costo diario: ${costo5*1000:.2f}')
print(f'  Costo mensual: ${costo5*1000*30:.2f}')
print()
print('Por eso en produccion se ajusta k cuidadosamente:')
print('mas contexto puede mejorar la respuesta, pero el costo')
print('y la latencia crecen linealmente con k.')
```

> ℹ️ En sistemas RAG de producción, técnicas como el re-ranking (ordenar de nuevo los fragmentos recuperados con un modelo más preciso pero más caro, aplicado solo a los k candidatos) ayudan a maximizar la calidad sin disparar el coste.

Apéndice A: Resumen de Todos los Programas

Esta sección lista todos los programas del libro organizados por capítulo, con su objetivo principal y los conceptos que ilustran. Los programas están diseñados para ejecutarse en orden — cada uno se apoya en los anteriores.

Capítulos 1-4: La neurona y cómo aprende

Programa 1A — Funciones de activación Archivo: `programa_1a_activaciones.py` Implementa y visualiza sigmoid, ReLU, tanh y softmax. Muestra en tabla numérica cómo cada función transforma los valores de entrada.

Programa 1B — La neurona artificial completa Archivo: `programa_1b_neurona.py` Define la clase Neurona con pesos, sesgo y función sigmoid. Prueba la neurona con distintas entradas y muestra la pre-activación y la salida.

Programa 2 — Neurona que aprende: puerta AND Archivo: `programa_2_neurona_aprende.py` Entrena una neurona con gradiente descendente manual para aprender la función lógica AND. Muestra cómo los pesos convergen con las épocas.

Capítulo 5: Optimizadores

Programa O-1 — SGD vs SGD+Momentum (Rosenbrock) Archivo: `programa_01_momentum.py` Compara SGD puro y SGD+Momentum en la función de Rosenbrock. Momentum llega al mínimo en unos 2000 pasos; SGD, tras 3000, sigue lejos.

Programa O-2 — Todos los optimizadores desde cero Archivo: `programa_02_optimizadores.py` Implementa SGD, SGDMomentum, RMSprop, Adam y AdamW con la misma interfaz paso(params, grads). Incluye la corrección de sesgo de Adam y el weight decay desacoplado de AdamW.

Programa O-3 — Comparativa de optimizadores en clasificación Archivo: `programa_03_comparativa.py` Entrena el mismo MLP en el problema de las dos lunas con cinco optimizadores distintos. Los cinco llegan al 100 % en validación; las diferencias están en la pérdida.

Programa O-4 — Learning Rate Scheduling Archivo: `programa_04_lr_scheduling.py` Implementa y visualiza cinco estrategias: constante, step decay, exponencial, coseno y warmup+coseno. Este último es el estándar en LLMs modernos.

Capítulo 6: Red Neuronal Multicapa

Programa 3 — Red Neuronal Multicapa desde cero Archivo: `programa_3_mlp.py` Implementa CapaDensa, ReLU, Sigmoid y RedNeuronal con forward y backward generales. Base reutilizable para todos los experimentos siguientes.

Programa 4 — MLP resuelve XOR Archivo: `programa_4_xor.py` Demuestra que una red multicapa puede resolver el problema XOR, que una sola neurona no puede. Muestra la pérdida bajando y las cuatro predicciones correctas.

Capítulo 7: Sobreajuste y Regularización

Programa R-1 — Demostración de overfitting con curvas de aprendizaje Archivo: `programa_R1_overfitting.py` Entrena tres redes (4, 16 y 256 neuronas) en el mismo dataset pequeño. Muestra numéricamente cómo el modelo grande memoriza (error de entrenamiento 0.032 frente a 0.091 en validación).

Programa R-2 — Regularización L2 y comparativa Archivo: `programa_R2_regularizacion.py` Compara el mismo modelo grande (256 neuronas) sin

regularización, con L2 moderada y L2 fuerte. Muestra cómo la suma de los cuadrados de los pesos cae de 608.5 a 32.5 con L2 moderada.

Programa R-3 — Dropout implementado desde cero Archivo: `programa_R3_dropout.py` Implementa la clase Dropout con máscara aleatoria y escalado 1/(1-p). Verifica que la esperanza de la salida es igual en entrenamiento e inferencia.

Programa R-4 — Early Stopping con paciencia Archivo: `programa_R4_early_stopping.py` Implementa EarlyStopping que guarda los mejores pesos y para el entrenamiento cuando la validación no mejora. Demuestra que el modelo óptimo era el de la época 3655, no el de la época 4656, en la que se detuvo.

Capítulos 8 y 9: Texto como Vectores y Tokenización

Programa 5 — Tokenizador y tabla de embeddings Archivo: `programa_5_embeddings.py` Implementa TokenizadorCaracter y TablaEmbeddings. Muestra cómo convertir texto en índices y recuperar vectores densos aprendibles.

Programa BPE-1 — Entrenador BPE desde cero Archivo: `programa_BPE1_entrenador.py` Implementa el algoritmo BPE completo: tokenización inicial, conteo de pares, fusión iterativa. Muestra cómo emergen subpalabras como 'apren', 'enci', 'tok' del corpus.

Programa BPE-2 — Codificador y decodificador BPE Archivo: `programa_BPE2_codificador.py` Aplica las reglas aprendidas para codificar texto nuevo. Demuestra eficiencia (entre 1.3 y 2.3 veces menos tokens que por caracteres; 1.48 en todo el corpus) y robustez ante palabras desconocidas.

Capítulo 10: Redes Recurrentes

Programa RNN-1 — RNN desde cero paso a paso Archivo: `programa_RNN1_rnn_basico.py` Implementa un RNN básico y visualiza el estado oculto evolucionando carácter a carácter. Muestra la diferencia fundamental con el Transformer: procesamiento secuencial vs. paralelo.

Programa RNN-2 — Vanishing gradient: demostración numérica Archivo: `programa_RNN2_vanishing_gradient.py` Simula el flujo del gradiente hacia atrás en un RNN. Muestra que la norma del gradiente cae a $7{\cdot}10^{-5}$ en 5 pasos y, con una escala mayor, explota, explicando por qué los RNNs no pueden aprender dependencias largas.

Capítulos 11 y 12: Atención y Transformer

Programa 6 — Mecanismo de Self-Attention Archivo: `programa_6_atencion.py` Implementa Q/K/V con máscara causal opcional. Visualiza los pesos de atención y compara la versión bidireccional y la causal.

Programa 7 — Bloque Transformer completo Archivo: `programa_7_transformer_bloque.py` Bloque de GPT-2 (Pre-LN): LayerNorm con γ y β antes de la self-attention causal y de la FFN con GELU, cada una con su suma residual. Cuenta sus parámetros y muestra que, recién iniciado, el bloque es casi la identidad.

Capítulos 13 y 14: El Nano-LLM y Muestreo

Programa 8 — Nano-LLM completo Archivo: programa_8_nano_llm.py GPT-2 en miniatura: modelo de lenguaje de nivel carácter con la arquitectura de GPT-2 (4 cabezas, Pre-LN, GELU, pesos compartidos) y 60 000 parámetros, backward analítico completo, mini-lotes, Adam con warm-up + coseno y recorte del gradiente. Guarda sus pesos para los capítulos siguientes.

Programa S-1 — Visualización de estrategias de muestreo Archivo: `programa_S1_visualizacion.py` Compara con barras ASCII cómo afectan temperatura, Top-k y Top-p a la misma distribución de probabilidad. Demuestra por qué Top-p es adaptativo y Top-k no.

Programa S-2 — Generación con todas las estrategias Archivo: `programa_S2_generacion_completa.py` Implementa generar_avanzado() con soporte para temperatura, Top-k, Top-p y repetition penalty combinables. Con el Nano-LLM, que ha memorizado su corpus, el greedy recita en vez de entrar en bucle; muestra cómo evitarlo.

Capítulo 15: ¿Memoriza o razona?

Programa RZ-1 — ¿El Nano-LLM memoriza o generaliza? Archivo: programa_RZ1_memoriza.py Aparta 10 de las 93 frases, compara con contadores de n-gramas y mide la copia literal del texto que genera el Programa 8. Veredicto: memoriza (validación 4.18 frente a 0.21 en entrenamiento).

Programa RZ-2 — El Nano-LLM aprende a sumar Archivo: programa_RZ2_suma.py El modelo del Programa 8, con 2 cabezas, aprende sumas de dos cifras escritas al revés y acierta el 100 % de 2000 sumas que nunca vio. Con 250 sumas, en cambio, memoriza buena parte: acierta el 58.8 % de las nuevas. Muestra a qué cifras atiende cada cabeza.

Programa RZ-3 — Razonar paso a paso con letras Archivo: programa_RZ3_deduce_letras.py Deducción encadenando hechos «a > b» que cambian en cada problema. Entrenado con 1 a 4 pasos, resuelve hasta 10 si escribe los pasos; respondiendo de golpe se queda en el azar.

Programa RZ-4 — Razonar con frases del corpus Archivo: programa_RZ4_deduce_frases.py Tokens de palabra, verbos que favorecen o perjudican y tres respuestas (SÍ, NO, NO SE PUEDE DEDUCIR). Entrenado con 1 a 4 pasos, generaliza hasta 8. Incluye un reinicio automático para las mesetas.

Capítulo 16: Tu código mueve GPT-2

Programa 8 bis — GPT-2 de verdad, con el forward del Programa 8 Archivo: programa_8bis_gpt2_real.py Lee a mano el archivo safetensors de GPT-2 small (124 439 808 parámetros) y lo ejecuta con las funciones del Programa 8. Tokenizador BPE real, predicciones, perplejidad, greedy y muestreo. Solo necesita NumPy y la carpeta gpt2/.

Programa G-1 — La lente de logits Archivo: programa_G1_lente_logits.py Aplica la LayerNorm final y wte^S a la salida de cada bloque para ver cómo se forma la predicción capa a capa.

Programa G-2 — Cabezas de inducción en GPT-2 real Archivo: programa_G2_induccion.py 50 tokens al azar repetidos dos veces: la pérdida se desploma en la segunda copia. Encuentra las cabezas de inducción y la cabeza «token anterior» y dibuja el Diagrama 16.2.

Programa G-3 — El KV-cache Archivo: programa_G3_kvcache.py Prefill y generación token a token con K y V guardadas; comprueba que da los mismos logits y mide el ahorro con contextos de 50, 200 y 800 tokens.

Programa G-4 — Ajuste fino de GPT-2 con tu backward Archivo: programa_G4_ajuste_fino.py Ajusta GPT-2 con las 83 frases de RZ-1 usando forward_backward() del Programa 8 sin cambios y compara la validación, en bits por carácter, con la del Nano-LLM.

Capítulo 18: Evaluación de Modelos de Lenguaje

Programa EVAL-1 — Perplejidad train vs. test Archivo: `programa_EVAL1_perplexity.py` Calcula la perplejidad correctamente sobre conjuntos separados. Presenta tres casos de diagnóstico: buen modelo, overfitting leve y memorización catastrófica.

Programa EVAL-2 — BLEU y framework de evaluación Archivo: `programa_EVAL2_bleu_benchmark.py` Implementa BLEU desde cero con n-gramas y brevedad. Construye un mini-benchmark de opción múltiple sobre IA con cuatro modelos de referencia y cita las cifras publicadas de MMLU de GPT-3.5 y GPT-4 (2023) y de Claude 3 Opus (2024).

Capítulo 19: Ética y Sesgos

Programa E-1 — Sesgo en acción: el modelo justo que discrimina Archivo: `programa_E1_sesgo.py` Simula un sistema de scoring crediticio sesgado. Demuestra que un modelo que nunca usa el grupo como variable puede aprender igualmente el sesgo histórico de los datos. Muestra la paridad demográfica como métrica de detección.

Programa E-2 — Métricas de equidad (fairness) Archivo: `programa_E2_fairness.py` Implementa tasa de selección, TPR, FPR, precisión y exactitud desglosadas por grupo. Muestra qué métricas revelan el sesgo y por qué no pueden cumplirse todas a la vez.

Apéndice D: Versiones PyTorch

Programa PT-1 — Tensores, Autograd y activaciones Archivo: `programa_PT1_tensores_activaciones.py` Demuestra la diferencia clave de PyTorch: loss.backward() calcula automáticamente todos los gradientes. Verifica que autograd produce el mismo resultado que el cálculo manual.

Programa PT-2 — Neurona AND con PyTorch Archivo: `programa_PT2_neurona_and.py` El mismo problema AND del Programa 2, pero con nn.Linear y optim.SGD. El bucle de entrenamiento se reduce a cuatro líneas.

Programa PT-3 — MLP con nn.Module y XOR Archivo: `programa_PT3_mlp_xor.py` Define la clase MLP(nn.Module) con nn.Sequential. Adam optimizer. Demuestra que la arquitectura es idéntica al Programa 3 NumPy, solo cambia el training loop.

Programa PT-4 — Embeddings con nn.Embedding Archivo: `programa_PT4_embeddings.py` nn.Embedding como tabla de búsqueda diferenciable. Muestra cómo se integra en un modelo y se actualiza automáticamente con el optimizador.

Programa PT-5 — Self-Attention con nn.Module Archivo: `programa_PT5_self_attention.py` Wq, Wk, Wv como nn.Linear con gradientes automáticos. Interfaz más limpia que la versión NumPy y totalmente compatible con el ecosistema PyTorch.

Programa PT-6 — Bloque Transformer en PyTorch Archivo: `programa_PT6_transformer_bloque.py` Usa nn.MultiheadAttention y nn.TransformerEncoderLayer nativos. Mismo diseño Pre-LN y misma cuenta de parámetros que el Programa 7 (3280), pero con 2 cabezas (el Programa 7 usa una). TransformerEncoderLayer con norm_first=True da la misma cuenta, aunque por defecto usa ReLU, no GELU.

Programa PT-7 — Nano-LLM completo en PyTorch Archivo: `programa_PT7_nano_llm.py` El Nano-LLM del Programa 8 como nn.Module, con exactamente la misma arquitectura (60 000 parámetros: la de GPT-2, con nn.LayerNorm y pesos compartidos). Mismo entrenamiento: Adam con warm-up y coseno y recorte del gradiente, con el lote de 8 ventanas como un tensor. El backward se reduce a loss.backward().

Apéndice G: Visión por Computador y CNNs

Programa 9A — Convolución manual y filtros Archivo: `programa_9a_convolucion.py` Aplica cuatro filtros clásicos (Sobel-X, Sobel-Y, Laplaciano, Gaussiano) a una imagen ASCII. Visualiza lo que detecta cada filtro sin necesitar librerías de imágenes.

Programa 9B — CNN entrenada para clasificar patrones Archivo: `programa_9b_cnn.py` CNN completa con backpropagation real: Conv(4 filtros 3×3) + ReLU + MaxPool(2×2) + Dense. Alcanza el 100 % clasificando barras horizontales/verticales y muestra los filtros aprendidos.

Apéndice B: Glosario

Términos clave del libro ordenados alfabéticamente.

Adam: Optimizador que combina Momentum (primer momento) y RMSprop (segundo momento) con corrección de sesgo para los primeros pasos. Es el optimizador por defecto en la mayoría de proyectos de Deep Learning. Hiperparámetros típicos: lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8.

AdamW: Variante de Adam que desacopla el weight decay del gradiente. A diferencia de Adam+L2, la regularización tiene la misma fuerza para todos los parámetros. Es el estándar para entrenar Transformers y LLMs.

Alucinación: Fenómeno por el que un LLM genera información incorrecta presentada con confianza como si fuera correcta. Cuando además se dice con seguridad, apunta a mala calibración (que se mide sobre muchas predicciones): el modelo no muestra que duda.

API: Application Programming Interface. Interfaz de programación que permite a una aplicación comunicarse con otra. Los LLMs como GPT-4 o Claude se usan habitualmente a través de una API: el usuario envía un prompt y recibe la respuesta generada sin acceso al modelo internamente.

Atención (Attention): Mecanismo que permite a cada posición de una secuencia consultar información de todas las demás posiciones, ponderada por relevancia. La base del Transformer.

Autorregresivo: Propiedad de un modelo de lenguaje que genera texto un token a la vez, usando como contexto todos los tokens generados anteriormente. GPT y casi todos los LLMs de texto actuales son autorregresivos. El entrenamiento sí se paraleliza (todas las posiciones a la vez, gracias a la máscara causal), pero la generación avanza token a token.

Backpropagation: Algoritmo que calcula los gradientes de todos los parámetros de una red neuronal aplicando la regla de la cadena, propagando el error desde la salida hacia la entrada.

Batch (lote): Subconjunto de los datos de entrenamiento procesado en una sola actualización de pesos. Los lotes permiten estimar el gradiente eficientemente y aprovechar el paralelismo de las GPUs.

Batching: Práctica de agrupar varios ejemplos de entrenamiento en un único lote (batch) para procesarlos juntos en una sola pasada por la red, en vez de uno a uno. Promedia el gradiente de todos los ejemplos del lote antes de actualizar los pesos, lo que suaviza el entrenamiento y aprovecha mucho mejor la GPU/CPU. El Programa 8 usa batching manual en NumPy; el Programa PT-7 (Apéndice D) delega el mismo proceso en PyTorch.

BatchNorm: Batch Normalization. Técnica de normalización que calcula media y varianza a través de la dimensión del batch para cada canal. Eficaz en CNNs (imágenes de tamaño fijo, lotes grandes), pero poco adecuada para LLMs: depende de estadísticas del lote, ruidosas con lotes pequeños y secuencias de longitud variable. En inferencia usa las medias acumuladas al entrenar, así que funciona también con batch=1. Ver Apéndice F.2.

BCE: Binary Cross-Entropy. Función de pérdida para clasificación binaria (dos clases). BCE = -(y·log(p) + (1-y)·log(1-p)). Penaliza con fuerza las predicciones muy seguras que resultan incorrectas. Equivalente a la Cross-Entropy cuando el vocabulario tiene solo dos elementos.

Beam Search: Estrategia de decodificación que mantiene las B mejores secuencias parciales en paralelo. Popular antes de Top-p; produce texto fluido pero genérico y poco creativo.

Benchmark: Conjunto de tareas y métricas estándar para comparar modelos. Ejemplos: MMLU, HumanEval, GSM8K. Sujetos a la Ley de Goodhart cuando se usan como objetivo de entrenamiento.

BERT: Bidirectional Encoder Representations from Transformers. Modelo Transformer de Google (2018) que usa atención bidireccional: ve el contexto anterior Y posterior a cada token simultáneamente. Excelente para clasificación y comprensión, pero no para generación. Entrenado con Masked Language Modeling y WordPiece. Precedió a los LLMs autorregresivos modernos.

BLEU: Bilingual Evaluation Understudy. Métrica que evalúa texto generado comparando n-gramas con textos de referencia. Ampliamente usada en traducción automática. Rango 0-1; mayor es mejor.

BPE — Byte Pair Encoding: Algoritmo de tokenización que parte de caracteres individuales y fusiona iterativamente los pares más frecuentes hasta alcanzar un vocabulario objetivo. Base de los tokenizadores de GPT-2/3/4, LLaMA y RoBERTa.

Byte-level BPE: Variante de BPE que parte de los 256 bytes posibles en lugar de caracteres. Puede tokenizar cualquier texto, incluidos emojis y caracteres de cualquier idioma, sin OOV.

Cabeza de inducción (induction head): Circuito de atención que, al ver un token, busca dónde apareció antes y copia lo que lo seguía. Es la base del aprendizaje en contexto (Olsson et al., 2022). En el Capítulo 15, emb_prev le da al modelo este mecanismo ya hecho.

Cadena de razonamiento (chain of thought): El modelo escribe los pasos intermedios antes de la respuesta. Así, un modelo poco profundo resuelve problemas que no podría resolver de golpe (15.4; Wei et al., 2022).

Calibración: Propiedad de un modelo por la que su confianza expresada coincide con su precisión real. Un modelo 90% seguro debería acertar el 90% de las veces. Medida con ECE (Expected Calibration Error).

Chat Template: Formato con tokens especiales (ej. <|im_start|>, <|im_end|>) que delimita los turnos de system/user/assistant en un LLM de chat durante el SFT y la inferencia. Cada familia de modelos define el suyo (ChatML, formato LLaMA-3, etc.). Ver Capítulo 17 (17.3).

CNN: Convolutional Neural Network. Red neuronal con capas convolucionales que aplican filtros locales deslizantes sobre la entrada. Estándar en visión por computador: detectan bordes, texturas y formas de forma jerárquica. Cada filtro comparte sus pesos en toda la imagen (parameter sharing), lo que las hace eficientes. Estudiadas en el Apéndice G del libro.

Contaminación de datos: Situación en que el texto del conjunto de evaluación (benchmark) aparece en los datos de preentrenamiento del modelo. Infla artificialmente las métricas sin reflejar capacidades reales.

Cosine Decay: Estrategia de scheduling del learning rate donde lr decrece siguiendo la mitad de un coseno, desde lr_max hasta lr_min. Produce convergencia más suave que el step decay.

Cross-Attention: Variante del mecanismo de atención donde Q proviene de una fuente (decoder) y K, V de otra (encoder, imagen o documento). La matriz de atención es (T_q × T_kv), no cuadrada. Usada en el Transformer original y en T5 (el decoder consulta al encoder) y en algunos modelos multimodales, como Flamingo. Ver Apéndice F.3.

Data Augmentation: Técnica de regularización que genera versiones modificadas de los datos existentes: rotaciones y recortes en imágenes, sustitución de sinónimos o back-translation en texto.

Data Leakage: Error en la preparación de datos donde información del conjunto de test o validación influye en el entrenamiento. Causa métricas optimistas que no reflejan el rendimiento real.

DeepSpeed: Framework de entrenamiento distribuido de Microsoft. ZeRO fragmenta parámetros, gradientes y estados del optimizador entre múltiples GPUs, permitiendo entrenar modelos de billones de parámetros. Ver Apéndice F.4.3.

Docker: Plataforma de contenedores que empaqueta una aplicación con todas sus dependencias en una imagen portable. Estándar en MLOps para garantizar reproducibilidad del entorno de entrenamiento e inferencia. Ver Apéndice F.4.3.

DPO: Direct Preference Optimization. Alternativa a RLHF que alinea el modelo con preferencias humanas sin entrenar un modelo de recompensa ni usar aprendizaje por refuerzo (PPO): optimiza directamente una función de pérdida sobre pares (respuesta preferida, respuesta rechazada) con descenso de gradiente ordinario. Propuesto por Rafailov et al. (2023). Ver Capítulo 17 (17.3).

Dropout: Técnica de regularización que desactiva aleatoriamente una fracción p de neuronas en cada paso de entrenamiento. Con el dropout invertido (Programa R-3), las activaciones que quedan se multiplican por $1/(1-p)$ durante el entrenamiento, y en inferencia no se hace nada. Propuesto por Hinton et al. (2012).

Early Stopping: Técnica de regularización que detiene el entrenamiento cuando el error de validación deja de mejorar durante N épocas consecutivas (paciencia N). Guarda los pesos del mejor momento observado.

ECE: Expected Calibration Error. Métrica que cuantifica la diferencia entre la confianza expresada por un modelo y su precisión real, promediada sobre intervalos de confianza.

Embedding: Vector denso de baja dimensión que representa un token. Se aprende durante el entrenamiento y captura similitudes semánticas entre tokens con contextos similares.

Época (epoch): Una pasada completa sobre todo el conjunto de datos de entrenamiento. Los modelos suelen entrenarse durante muchas épocas hasta que el error de validación converge.

Estado oculto: En un RNN, vector que resume toda la historia de la secuencia procesada hasta el paso actual. Pasa de un paso temporal al siguiente, actualizando la 'memoria' de la red.

Evaluación extrínseca: Medición del rendimiento de un modelo en tareas concretas y útiles (preguntas, código, traducción). Complementaria a la evaluación intrínseca (perplejidad). Más informativa sobre la utilidad real.

Evaluación intrínseca: Medición de qué bien un modelo predice el texto (perplejidad). No requiere tareas etiquetadas pero no mide directamente la utilidad del modelo.

Exploding Gradient: Situación en que los gradientes crecen exponencialmente al retropropagar a través de muchos pasos, causando actualizaciones de pesos descontroladas. Se combate con gradient clipping.

Fertilidad del token: Número de tokens BPE por palabra en un idioma dado. El inglés tiene fertilidad baja (~1.2 tokens/palabra) porque los tokenizadores se entrenaron principalmente en inglés. Con el tokenizador de GPT-2, el español gasta unas dos veces más tokens por palabra que el inglés (Ejercicio 16.1).

Few-shot / Zero-shot / One-shot: Paradigmas de uso de LLMs según los ejemplos incluidos en el prompt. Zero-shot: sin ejemplos, solo instrucción. One-shot: un ejemplo. Few-shot: varios ejemplos (típicamente 3-10). Los LLMs grandes pueden resolver tareas nuevas en estos modos sin actualizar sus pesos, simplemente imitando el patrón del prompt.

FFN: Feed-Forward Network. Subcapa densa del bloque Transformer que se aplica de forma independiente a cada posición de la secuencia. Consta de dos transformaciones lineales con una activación no lineal entre ellas (típicamente GELU). Su dimensión interna suele ser 4× la dimensión del modelo. Complementa la self-attention aportando capacidad de transformación local.

Fine-tuning: Proceso de continuar el entrenamiento de un modelo preentrenado sobre un conjunto más pequeño y específico para adaptar su comportamiento. Mucho más eficiente que entrenar desde cero.

Forward pass: El cálculo que convierte una entrada en una predicción, pasando por todas las capas de la red de izquierda a derecha (de entrada a salida).

FSDP: Fully Sharded Data Parallel. Técnica de entrenamiento distribuido de PyTorch (Meta) equivalente a DeepSpeed ZeRO-3. Fragmenta parámetros, gradientes y estados entre todos los procesos participantes. Ver Apéndice F.4.3.

Función de activación: Función no lineal aplicada a la salida de cada neurona. Sin ella, la red sería equivalente a una sola capa lineal. Ejemplos: ReLU, sigmoid, tanh, GELU.

Función de pérdida (loss): Medida numérica del error del modelo. El entrenamiento busca minimizarla. Ejemplos: MSE para regresión, BCE para clasificación binaria, Cross-Entropy para clasificación multi-clase y LLMs.

GELU: Gaussian Error Linear Unit. Función de activación $f(x) = x \cdot \Phi(x)$, donde Φ es la función de distribución acumulada gaussiana. Más suave que ReLU: no trunca abruptamente en 0 sino que atenúa gradualmente los valores negativos. Estándar en GPT, BERT y la mayoría de Transformers modernos. GPT-2, y nuestro Nano-LLM, usan su aproximación con tanh (12.2).

Generalización: Rendir bien en datos que el modelo no vio durante el entrenamiento. Es lo contrario de memorizar (7.1, 15.2).

Generalización por longitud: Resolver problemas más largos que todos los del entrenamiento, por ejemplo deducciones de 10 pasos tras entrenar con 1 a 4 (Programas RZ-3 y RZ-4).

Goodhart's Law: Principio planteado por el economista Charles Goodhart (1975); su formulación más citada es de Marilyn Strathern (1997): 'Cuando una medida se convierte en un objetivo, deja de ser una buena medida.' Explica por qué los benchmarks se saturan.

GPT: Generative Pre-trained Transformer. Familia de LLMs de OpenAI. GPT-1 (2018) demostró el potencial del preentrenamiento no supervisado. GPT-2 (2019) sorprendió por su calidad textual. GPT-3 (2020, 175B parámetros) estableció el few-shot learning como capacidad emergente. GPT-4 (2023) introdujo capacidades multimodales. Todos usan arquitectura Transformer autorregresiva y tokenización BPE.

GQA / MQA: Grouped-Query Attention / Multi-Query Attention. Variantes del Multi-Head Attention en las que varias cabezas de consulta (Q) comparten un único juego de claves y valores (K, V), en vez de tener uno propio cada una. Reduce el tamaño del KV-cache (proporcionalmente al número de cabezas agrupadas) con una pérdida de calidad mínima. MQA es el caso extremo con un solo grupo de K/V para todas las cabezas de Q. Usado por Mistral, LLaMA-2/3 y la mayoría de LLMs recientes. Ver Capítulo 17.

GPU: Graphics Processing Unit. Procesador gráfico con miles de núcleos pequeños optimizados para operaciones matriciales paralelas. Imprescindible para entrenar redes neuronales: una GPU puede ser 100-1000× más rápida que una CPU para multiplicaciones de matrices. Los LLMs grandes requieren clusters de cientos o miles de GPUs. NVIDIA domina el mercado con sus tarjetas A100, H100 y sucesoras.

Gradient Clipping: Técnica que limita la norma del vector de gradiente a un valor máximo antes de aplicar la actualización. Evita la explosión del gradiente sin cambiar la dirección del descenso.

Gradiente: Vector con las derivadas (parciales) de la función de pérdida respecto a cada parámetro (4.3). Indica la dirección de mayor crecimiento de la pérdida; el entrenamiento mueve los parámetros en la dirección opuesta.

Gradiente Descendente: Algoritmo de optimización que actualiza los parámetros restando el gradiente multiplicado por el learning rate. Base de todos los optimizadores modernos.

Greedy Decoding: Estrategia de generación que siempre elige el token con mayor probabilidad. Determinista y coherente pero propenso a bucles y repeticiones.

Hiperparámetros: Parámetros de diseño del entrenamiento fijados antes de empezar: learning rate, número de capas, tamaño del lote, dropout, etc. No se aprenden, se buscan manualmente o con métodos de optimización.

Hugging Face: Empresa y plataforma open-source con la librería Transformers (modelos preentrenados), el Hub (repositorio de modelos y datasets) y herramientas como PEFT y Accelerate. Referencia del ecosistema LLM open-source. Ver Capítulo 20.

HumanEval: Benchmark de generación de código: 164 problemas de Python donde el modelo debe escribir código que pase tests unitarios. La métrica principal es pass@k (% de problemas resueltos en k intentos).

Inferencia: Proceso de usar un modelo ya entrenado para generar predicciones sobre datos nuevos, sin actualizar sus pesos. Opuesto al entrenamiento. Durante la inferencia, el gradiente no se calcula (ahorra memoria y cómputo). En LLMs, la inferencia es autorregresiva: se genera un token a la vez.

Jacobiano: Matriz de derivadas parciales $\partial F_i/\partial x_j$ de una función vectorial F. En retropropagación, el Jacobiano de cada capa determina si el gradiente se desvanece o explota. La conexión residual añade la identidad I, que ofrece un canal directo al gradiente (salvo cancelaciones poco habituales, como un Jacobiano de F cercano a $-I$). Ver Capítulo 10 (10.4) y Apéndice F.1.

Kubernetes: Sistema de orquestación de contenedores Docker para despliegue, escalado y gestión de aplicaciones en producción. Estándar para servir modelos de ML a gran escala. Ver Apéndice F.4.3.

KV-cache: Memoria donde un modelo generativo guarda las claves (K) y los valores (V) de los tokens ya procesados para no recalcularlos en cada paso: cada token nuevo solo calcula los suyos. Su ahorro crece con la longitud del contexto (16.7); su precio es memoria. Ver Capítulo 17.

Layer Normalization: Normalización de activaciones por muestra (no entre muestras). Estabiliza el entrenamiento en redes profundas. Estándar en los bloques Transformer modernos.

Learning Rate Scheduling: Estrategia que varía el learning rate durante el entrenamiento. Los LLMs usan warmup lineal seguido de decaimiento en coseno para un entrenamiento estable y eficiente.

Lente de logits (logit lens): Técnica de interpretabilidad que aplica la LayerNorm final y la capa de salida a la vía residual después de cada bloque, para ver qué predeciría el modelo si terminara ahí (nostalgebraist, 2020; Programa G-1).

Línea base (baseline): Modelo simple con el que se compara uno complejo, por ejemplo un contador de n-gramas (15.2). Si el modelo complejo no la supera, no ha aprendido nada que el simple no sepa.

LLaMA: Large Language Model Meta AI. Familia de modelos open-source de Meta (7B, 13B, 70B, 405B parámetros). Arquitectura decoder-only con Pre-LN (RMSNorm), RoPE y SwiGLU. Referencia para fine-tuning open-source. Ver Capítulo 17.

LLM: Large Language Model. Modelo de lenguaje de gran escala basado en Transformer, preentrenado sobre enormes cantidades de texto para predecir el siguiente token.

LoRA: Low-Rank Adaptation. Técnica de fine-tuning eficiente que congela los pesos originales del modelo y añade matrices de bajo rango entrenables en paralelo. En lugar de actualizar W ($d\times d$), entrena A ($d\times r$) y B ($r\times d$) con $r \ll d$. Reduce los parámetros entrenables en 10-1000× sin pérdida significativa de rendimiento. Estándar para adaptar LLMs grandes con GPUs pequeñas.

Loss Masking: Técnica usada en SFT que excluye del cálculo de la pérdida los tokens del prompt (system/user), calculando gradiente solo sobre los tokens generados por el assistant. Evita que el modelo “aprenda” a imitar las preguntas en vez de a responderlas. Ver Capítulo 17 (17.3).

LSTM: Long Short-Term Memory. Variante de RNN propuesta por Hochreiter & Schmidhuber (1997) con puertas de olvido, entrada y salida que mitigan el vanishing gradient. Dominó el NLP hasta 2017.

Máscara causal: Mecanismo en el self-attention que impide que el token en la posición t vea tokens en posiciones futuras. Garantiza una generación autorregresiva coherente con el orden causal.

Memorización: El modelo reproduce sus datos de entrenamiento en lugar de extraer un patrón general; su pérdida en datos nuevos es mucho mayor que en los de entrenamiento (15.2).

Meseta (plateau): Tramo del entrenamiento en el que la pérdida apenas baja durante mucho tiempo; algunas inicializaciones no salen de ella (Programas RZ-2, RZ-3 y RZ-4).

Mistral: Familia de LLMs open-source de Mistral AI. El modelo Mistral 7B supera a LLaMA 2 13B usando Grouped Query Attention (GQA) y Sliding Window Attention. Ver Capítulo 17.

Mixture of Experts (MoE): Arquitectura en la que la capa feed-forward se sustituye por varias "expertas" paralelas, de las que un router activa solo unas pocas por token. Permite aumentar mucho el número total de parámetros sin aumentar el coste de cómputo por token. Usado por Mixtral, DeepSeek-V3 o Llama 4, entre otros.

MLOps: Machine Learning Operations. Prácticas y herramientas para desplegar, monitorizar y mantener modelos de ML en producción, incluyendo CI/CD, control de versiones de modelos y monitorización de drift. Ver Apéndice F.4.3.

MLP: Multi-Layer Perceptron. Red neuronal con al menos una capa oculta completamente conectada. Aproximador universal con función de activación no lineal.

MMLU: Massive Multitask Language Understanding. Benchmark de 15 908 preguntas (14 042 de test) de opción múltiple en 57 materias académicas. Estándar para medir capacidades generales de LLMs.

Momentum: Técnica que acumula un vector de velocidad como suma ponderada de los gradientes pasados. Reduce el zigzagueo de SGD en paisajes de pérdida difíciles. Hiperparámetro beta típico: 0.9.

MSE: Mean Squared Error. Error cuadrático medio. Función de pérdida para regresión: $MSE = (1/n)\cdot\Sigma(y - \hat{y})^2$. Penaliza más los errores grandes (por el cuadrado). Diferenciable en todas partes. La raíz del MSE (RMSE) tiene las mismas unidades que la variable predicha.

Multi-head Attention: Extensión del mecanismo de atención que usa H cabezas independientes en paralelo, cada una con sus propias matrices Q, K, V. Las salidas de las H cabezas se concatenan y se proyectan. Permite al modelo atender simultáneamente a diferentes tipos de relaciones (sintácticas, semánticas, posicionales) en distintas cabezas. El Transformer original usa 8 cabezas.

N-grama: Secuencia de n símbolos consecutivos. Un modelo de n-gramas predice el siguiente símbolo contando qué siguió a los $n - 1$ anteriores en el corpus (15.2).

NLP: Natural Language Processing. Procesamiento de Lenguaje Natural. Campo de la IA que estudia cómo los ordenadores pueden entender, generar y manipular lenguaje humano. Incluye tareas como traducción, resumen, respuesta a preguntas, clasificación de sentimientos y generación de texto. Los LLMs han revolucionado el NLP desde 2018.

NumPy: Librería de Python para computación numérica eficiente con arrays multidimensionales. Base de todos los programas de este libro excepto el Apéndice D.

OOV: Out of Vocabulary. Problema de tokenizaciones word-level: palabras no vistas durante el entrenamiento no pueden procesarse. El BPE por bytes lo elimina; el BPE por caracteres, mientras el texto no traiga caracteres que no estaban en su corpus.

Overfitting: Sobreajuste. Situación en que el modelo rinde bien en entrenamiento pero mal en datos nuevos porque ha memorizado los ejemplos de entrenamiento en lugar de aprender el patrón subyacente.

Paciencia: En Early Stopping, número de épocas sin mejora en validación antes de parar el entrenamiento. Valor típico: 10-50 épocas. Valores más altos dan más oportunidad de escapar de mínimos locales.

Parámetros: Pesos y sesgos de una red neuronal. Son los valores que se aprenden durante el entrenamiento. Un LLM moderno tiene entre miles de millones y billones de parámetros.

Paso de entrenamiento (step): Una actualización de los pesos con un mini-lote. Es distinto de una época: los 12 000 pasos del Programa 8 equivalen a unas 454 pasadas por su corpus (13.3.2).

Perceptrón: El modelo de neurona artificial original (Rosenblatt, 1958). Predecesor de todas las redes neuronales modernas.

Perplejidad (PPL): exp(pérdida_media_por_token). Mide cuánto le sorprende al modelo el texto evaluado. Adivinar al azar entre V tokens da PPL=V; PPL=1 es perfecta. Debe calcularse SIEMPRE sobre datos de test separados.

Positional Encoding: Vector sumado al embedding de cada token para informar al modelo de su posición en la secuencia. El Transformer original usa funciones seno/coseno de distintas frecuencias. GPT-2 y GPT-3 aprenden la posición con una tabla de parámetros (13.1), y la mayoría de los modelos modernos (LLaMA, Mistral, Qwen) usan Rotary Position Embedding (RoPE). Esencial porque el self-attention por sí solo no distingue el orden: si se permutan los tokens, sus salidas se permutan igual (es equivariante a las permutaciones).

Post-LN: Post-Layer Normalization. LayerNorm aplicado DESPUÉS de la suma residual: X = LayerNorm(X + Subcapa(X)). Presentado en Vaswani et al. (2017). Más inestable que Pre-LN en entrenamientos a gran escala. Ver Capítulo 12 (12.2).

Pre-LN: Pre-Layer Normalization. LayerNorm aplicado ANTES de la subcapa: X = X + Subcapa(LayerNorm(X)). Estándar en GPT-2, GPT-3, LLaMA y Mistral; es la que usan todos los modelos del libro desde el Programa 7: el Nano-LLM y los Programas RZ. Más estable, permite learning rates más altos. En PyTorch: norm_first=True. Ver Capítulo 12 (12.2) y Apéndice F.1.

Preentrenamiento: Primera fase del entrenamiento de un LLM: predecir el siguiente token en una cantidad enorme de texto general. Después se ajusta (fine-tuning) para una tarea o un estilo. En 16.8, GPT-2 preentrenado y ajustado con 83 frases generaliza mucho mejor que el Nano-LLM entrenado desde cero con las mismas.

Prompt: Texto de entrada que el usuario proporciona a un LLM para guiar su generación. Puede incluir instrucciones, ejemplos (few-shot), contexto y la pregunta o tarea concreta. El diseño cuidadoso de prompts (prompt engineering) tiene un impacto enorme en la calidad de la salida del modelo sin modificar sus pesos.

PyTorch: Framework de Deep Learning de Meta/Facebook. Usa diferenciación automática (autograd) para calcular gradientes sin implementarlos manualmente. Estándar en investigación académica.

Q / K / V: Query, Key, Value. Las tres proyecciones lineales del mecanismo de atención. Q (consulta): lo que el token actual busca. K (clave): lo que cada token ofrece. V (valor): la información que se recupera si la clave coincide. La atención calcula $\text{softmax}(QK^T/\sqrt{d})\cdot V$. Cada cabeza aprende sus propias matrices Q, K, V.

RAG: Retrieval-Augmented Generation. Técnica que combina un LLM con una base de conocimiento externa: antes de generar, el sistema recupera los fragmentos más relevantes de documentos y los incluye en el prompt como contexto. Reduce las alucinaciones y permite al modelo acceder a información actualizada sin reentrenar. Estándar en aplicaciones empresariales con LLMs.

Regularización: Conjunto de técnicas para prevenir el overfitting: L2 (weight decay), L1, Dropout, Early Stopping, Data Augmentation. Todas penalizan la complejidad excesiva del modelo.

ReLU: Rectified Linear Unit. Función de activación f(x) = max(0, x). Simple, eficiente y muy efectiva. Introduce no linealidad truncando los valores negativos a cero. Estándar en CNNs y MLPs. Problema: "neuronas muertas" (si x siempre negativo, el gradiente es cero). Variantes: Leaky ReLU, PReLU, ELU. En Transformers se prefiere GELU.

Repetition Penalty: Técnica de generación que reduce la probabilidad de tokens ya generados recientemente. Evita que el modelo entre en bucles repetitivos. Factor típico: 1.05-1.3.

Residual Connection: Conexión residual o skip connection. Suma la entrada de un bloque directamente a su salida: y = F(x) + x. Propuesta por He et al. (2015) para redes muy profundas. Permite al gradiente fluir directamente hacia atrás sin atravesar todas las capas, mitigando el vanishing gradient. Presente en todos los bloques del Transformer.

ResNet: Residual Network. Arquitectura CNN de He et al. (2015) que introdujo las conexiones residuales (skip connections), permitiendo entrenar redes de cientos de capas. El mismo principio se aplica en los bloques Transformer. Ver Apéndice F.1.

RLHF: Reinforcement Learning from Human Feedback. Técnica que usa comparaciones humanas para entrenar un modelo de recompensa y refinar el LLM. Mejora la utilidad y la seguridad, aunque suele empeorar la calibración (18.6).

RMSprop: Root Mean Square Propagation. Optimizador que adapta el learning rate por parámetro dividiendo por la raíz de la media de cuadrados de los gradientes recientes. Propuesto por Hinton (2012).

RNN: Red Neuronal Recurrente. Arquitectura que procesa secuencias paso a paso, manteniendo un estado oculto como 'memoria'. Precedió al Transformer en NLP. Limitada por el vanishing gradient y la falta de paralelismo.

RoPE: Rotary Position Embeddings. Codificación posicional que rota los vectores Q y K un ángulo proporcional a su posición antes de calcular la atención, en vez de sumar un vector de posición al embedding. Solo la distancia relativa entre posiciones sobrevive al producto escalar, lo que, con técnicas de reescalado, facilita ampliar la ventana de contexto. Usado por LLaMA, Mistral y la mayoría de LLMs modernos. Propuesto por Su et al. (2021). Ver Capítulo 12.

Safetensors: Formato de archivo para guardar pesos de redes neuronales: 8 bytes con la longitud de una cabecera JSON (nombre, tipo, forma y posición de cada tensor) seguidos de los números. Es seguro porque, a diferencia de los archivos pickle, no puede contener código ejecutable. El Programa 8 bis lo lee con NumPy.

Scaling Laws: Leyes de escalado. Relaciones empíricas que describen cómo mejora el rendimiento de un LLM al aumentar parámetros (N), datos (D) y cómputo (C). Kaplan et al. (2020, OpenAI) mostraron que la pérdida sigue una ley potencial con cada uno de estos factores. Chinchilla (2022) la refinó: con un cómputo de entrenamiento fijo, el reparto óptimo ronda los 20 tokens por parámetro (muchos modelos actuales usan bastantes más para abaratar la inferencia).

Self-attention: Variante del mecanismo de atención donde Q, K y V provienen de la misma secuencia. La secuencia se 'atiende a sí misma'. Base del Transformer.

Semilla aleatoria: Número que fija los valores aleatorios (pesos iniciales, lotes) para que un experimento sea reproducible. Repetir con varias semillas indica si un resultado es robusto (15.3).

SentencePiece: Librería de tokenización subpalabra que trabaja sobre el texto en bruto, sin separar antes las palabras. Implementa BPE (el que usan LLaMA y LLaMA-2) y Unigram (T5, ALBERT, XLNet), que es probabilístico: puede tokenizar el mismo texto de varias formas.

Sesgo (bias): 1) Parámetro adicional sumado a la salida de una neurona; permite desplazar la función de activación. 2) En evaluación: error sistemático de un modelo hacia ciertos grupos.

SFT: Supervised Fine-Tuning. Ajuste fino supervisado. Primera etapa del proceso RLHF: el LLM preentrenado se ajusta sobre un conjunto de pares (instrucción, respuesta deseable) etiquetados por humanos. Convierte el modelo de "predictor de texto" a "seguidor de instrucciones". Precede al entrenamiento del modelo de recompensa y al PPO.

SGD: Stochastic Gradient Descent. Algoritmo de optimización más básico: w := w - lr × grad. 'Estocástico' porque usa lotes aleatorios. Base de todos los optimizadores más sofisticados.

SiLU: Sigmoid Linear Unit. $f(x) = x \cdot \sigma(x)$. Su derivada solo se anula en un punto ($x \approx -1.278$): no tiene, como ReLU, toda una semirrecta plana, y eso evita las neuronas muertas. Función de activación estándar en LLaMA, Mistral y Phi-3. En PyTorch: F.silu(). Equivalente a Swish. Ver Capítulo 12 (12.2).

Softmax: Función que convierte un vector de logits en una distribución de probabilidad (todos positivos, suman 1). Usada en la capa de salida de clasificadores y en el mecanismo de atención.

Subpalabra: Unidad de tokenización BPE: fragmento de palabra aprendido estadísticamente del corpus. Más grande que un carácter, más pequeña que una palabra. Permite manejar morfología y palabras desconocidas.

SVD: Singular Value Decomposition (Descomposición en Valores Singulares). Factorización $M = U\Sigma V^T$. Base matemática de técnicas como LoRA (adaptación de bajo rango para fine-tuning eficiente). Ver Apéndice F.4.1.

Swish: Función de activación $f(x) = x \cdot \sigma(x)$ propuesta por Ramachandran et al. (2017). Equivalente a SiLU. Su derivada solo se anula en $x \approx -1.278$; al no tener una semirrecta plana como ReLU, evita las neuronas muertas. Usada en LLaMA, Mistral y Phi-3. Ver Capítulo 12 (12.2).

Temperatura: Parámetro de generación que divide los logits antes del softmax. T<1: distribución concentrada (determinista). T>1: distribución plana (creativa/aleatoria).

TGI: Text Generation Inference. Framework de Hugging Face para servir LLMs en producción. Implementa batching continuo, cuantización y paginación de KV-cache para maximizar el throughput. Ver Capítulo 20 y Apéndice F.4.3.

Token: Unidad mínima de texto procesada por un LLM. Puede ser un carácter, una subpalabra o una palabra entera, según el tokenizador.

Tokenizador: Sistema que convierte texto en secuencias de tokens (índices enteros). El tipo de tokenizador afecta directamente el vocabulario, la longitud de las secuencias y las capacidades del modelo.

Top-k: Estrategia de muestreo que conserva solo los k tokens más probables y asigna probabilidad cero al resto. Simple pero con el problema de que k es fijo independientemente de la distribución.

Top-p / Nucleus: Estrategia de muestreo que conserva el conjunto mínimo de tokens cuyas probabilidades suman p. Adaptativa: con distribuciones concentradas usa pocos tokens; con distribuciones planas, más. Estándar en LLMs modernos.

Transfer Learning: Aprendizaje por transferencia. Técnica que reutiliza un modelo preentrenado en una tarea grande (como predecir texto en internet) para resolver una tarea más específica con pocos datos (fine-tuning). En NLP, el preentrenamiento de LLMs es la forma más efectiva de transfer learning: los modelos aprenden representaciones generales del lenguaje reutilizables en cualquier tarea.

Transformer: Arquitectura de red neuronal propuesta en 2017 que usa self-attention en lugar de recurrencia. Base de todos los LLMs modernos. Permite procesamiento paralelo de secuencias completas.

Underfitting: Situación en que el modelo es demasiado simple para capturar el patrón real en los datos. El error es alto tanto en entrenamiento como en validación. Solución: más capacidad (neuronas/capas) o más entrenamiento.

Vanishing Gradient: Fenómeno en RNNs (y redes profundas sin residuales) donde el gradiente se multiplica por valores menores que 1 en cada capa o paso temporal, decreciendo exponencialmente. Impide aprender dependencias de largo alcance.

Ventana de contexto: Context window. Número máximo de tokens que el modelo puede procesar simultáneamente. Determina cuánto "recuerda" el modelo en una conversación. GPT-2: 1024 tokens. GPT-3: 2048. En 2026, los grandes modelos comerciales (GPT, Claude, Gemini) llegan a alrededor de 1 000 000, y algún modelo abierto, como Llama 4 Scout, anuncia hasta 10 millones. Limitada por la complejidad cuadrática $O(n^2)$ del self-attention. Los modelos recientes usan técnicas (Flash Attention, sliding window) para ampliarla eficientemente.

Vocabulario: Conjunto de todos los tokens distintos que el modelo conoce. En BPE, el vocabulario incluye los caracteres base más todos los tokens fusionados. Tamaño típico en LLMs: entre 32 000 y 256 000 tokens.

Warmup: Fase inicial del learning rate scheduling donde lr sube linealmente desde 0 hasta lr_max durante N pasos. Estabiliza el entrenamiento en los primeros pasos cuando los gradientes son inestables.

Weight Decay: Regularización L2 implementada como decaimiento directo de los pesos: w *= (1 - lr*wd) en cada paso. En AdamW está desacoplado del gradiente, lo que lo hace más efectivo.

Weights & Biases: Plataforma MLOps (wandb.ai) para monitorizar experimentos de ML. Registra métricas, hiperparámetros, artefactos y visualizaciones en tiempo real, facilitando la comparación de experimentos y la reproducibilidad. Ver Apéndice F.4.3.

WordPiece: Algoritmo de tokenización subpalabra similar a BPE pero que fusiona el par que más aumenta la verosimilitud del corpus. Usado por BERT y los modelos de Google.

Apéndice C: Referencias y Recursos

Artículos fundamentales

* Rosenblatt, F. (1958). 'The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain'. Psychological Review. — El primer modelo de neurona artificial.
* Rumelhart, D.E., Hinton, G.E., Williams, R.J. (1986). 'Learning representations by back-propagating errors'. Nature. — El algoritmo de backpropagation moderno.
* LeCun, Y. et al. (1989). 'Backpropagation Applied to Handwritten Zip Code Recognition'. Neural Computation. — La primera red convolucional entrenada con retropropagación (Apéndice G).
* Hochreiter, S. & Schmidhuber, J. (1997). 'Long Short-Term Memory'. Neural Computation. — Las LSTM que dominaron el NLP durante 20 años.
* Gage, P. (1994). 'A New Algorithm for Data Compression'. C Users Journal. — El algoritmo BPE original, antes de su uso en NLP.
* Mikolov, T. et al. (2013). 'Efficient Estimation of Word Representations in Vector Space' (word2vec). arXiv:1301.3781. — Los embeddings y la analogía rey − hombre + mujer ≈ reina (8.5).
* Sennrich, R., Haddow, B. & Birch, A. (2016). 'Neural Machine Translation of Rare Words with Subword Units'. ACL. — BPE aplicado por primera vez al lenguaje, en traducción automática neuronal.
* Vaswani, A. et al. (2017). 'Attention is All You Need'. arXiv:1706.03762. — El artículo que introduce el Transformer.
* Su, J. et al. (2021). 'RoFormer: Enhanced Transformer with Rotary Position Embedding' (RoPE). arXiv:2104.09864. — La codificación posicional que ha sustituido a la sinusoidal en la mayoría de LLMs modernos.
* Kingma, D.P. & Ba, J. (2014). 'Adam: A Method for Stochastic Optimization'. arXiv:1412.6980. — El optimizador Adam.
* Loshchilov, I. & Hutter, F. (2017). 'Decoupled Weight Decay Regularization'. arXiv:1711.05101. — AdamW y el weight decay desacoplado.
* Srivastava, N. et al. (2014). 'Dropout: A Simple Way to Prevent Neural Networks from Overfitting'. JMLR. — El artículo original del Dropout.
* Hinton, G.E. et al. (2012). 'Improving neural networks by preventing co-adaptation of feature detectors'. arXiv:1207.0580. — La primera propuesta del Dropout.
* Krizhevsky, A., Sutskever, I. & Hinton, G.E. (2012). 'ImageNet Classification with Deep Convolutional Neural Networks' (AlexNet). NeurIPS. — El resultado que puso en marcha el aprendizaje profundo moderno.
* He, K. et al. (2015). 'Deep Residual Learning for Image Recognition' (ResNet). arXiv:1512.03385. — Las conexiones residuales que hereda el Transformer.
* Ramachandran, P., Zoph, B. & Le, Q.V. (2017). 'Searching for Activation Functions' (Swish). arXiv:1710.05941.
* Brown, T. et al. (2020). 'Language Models are Few-Shot Learners' (GPT-3). arXiv:2005.14165. — Demuestra las capacidades emergentes de los LLMs a escala.
* Holtzman, A. et al. (2020). 'The Curious Case of Neural Text Degeneration'. arXiv:1904.09751. — Introduce el nucleus sampling (Top-p).
* Ouyang, L. et al. (2022). 'Training language models to follow instructions with human feedback' (RLHF). arXiv:2203.02155. — InstructGPT y la alineación con preferencias humanas.
* Rafailov, R. et al. (2023). 'Direct Preference Optimization: Your Language Model is Secretly a Reward Model' (DPO). arXiv:2305.18290. — Alternativa a RLHF que evita entrenar un modelo de recompensa separado y el bucle de PPO.

* Hendrycks, D. et al. (2020). 'Measuring Massive Multitask Language Understanding' (MMLU). arXiv:2009.03300. — El benchmark de referencia para capacidades generales.
* Chen, M. et al. (2021). 'Evaluating Large Language Models Trained on Code' (HumanEval). arXiv:2107.03374. — Benchmark de generación de código.
* Karpathy, A. (2022). 'nanoGPT'. GitHub: github.com/karpathy/nanoGPT. — Referencia directa para nuestro Nano-LLM. En noviembre de 2025 su autor lo declaró superado por nanochat (github.com/karpathy/nanochat), que cubre también el ajuste fino y la interfaz de chat.
* Radford, A. et al. (2019). 'Language Models are Unsupervised Multitask Learners' (GPT-2). Informe técnico de OpenAI. — El modelo que el Nano-LLM del Capítulo 13 reproduce en miniatura.
* Xiong, R. et al. (2020). 'On Layer Normalization in the Transformer Architecture'. arXiv:2002.04745. — Pre-LN frente a Post-LN.
* Kaplan, J. et al. (2020). 'Scaling Laws for Neural Language Models'. arXiv:2001.08361. — Las leyes de escalado del Capítulo 17.
* Buolamwini, J. & Gebru, T. (2018). 'Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification'. Proceedings of Machine Learning Research 81 (FAT* 2018). — El estudio de sesgos del Capítulo 19.
* Kleinberg, J., Mullainathan, S. & Raghavan, M. (2016). 'Inherent Trade-Offs in the Fair Determination of Risk Scores'. arXiv:1609.05807. — Calibración e igualdad de errores no pueden cumplirse a la vez (Capítulo 19).
* Chouldechova, A. (2017). 'Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments'. Big Data 5(2). — El mismo resultado, aplicado a COMPAS (Capítulo 19).
* Obermeyer, Z. et al. (2019). 'Dissecting racial bias in an algorithm used to manage the health of populations'. Science 366(6464). — El sesgo del algoritmo sanitario del Capítulo 19.
* nostalgebraist (2020). 'interpreting GPT: the logit lens'. LessWrong, lesswrong.com/posts/AcKRB8wDpdaN6v6ru. — La lente de logits (16.5).
* Elhage, N. et al. (2021). 'A Mathematical Framework for Transformer Circuits'. Transformer Circuits Thread, transformer-circuits.pub/2021/framework. — Primer estudio de las cabezas de inducción (16.6).
* Nye, M. et al. (2021). 'Show Your Work: Scratchpads for Intermediate Computation with Language Models'. arXiv:2112.00114. — Escribir los cálculos intermedios.
* Wei, J. et al. (2022). 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. arXiv:2201.11903. — Cadenas de razonamiento.
* Olsson, C. et al. (2022). 'In-context Learning and Induction Heads'. arXiv:2209.11895. — Cabezas de inducción.
* Power, A. et al. (2022). 'Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets'. arXiv:2201.02177. — Memorizar antes de generalizar en tareas aritméticas.
* Hoffmann, J. et al. (2022). 'Training Compute-Optimal Large Language Models' (Chinchilla). arXiv:2203.15556. — La ley de escalado del Ejercicio 17.2.
* Lee, N. et al. (2023). 'Teaching Arithmetic to Small Transformers'. arXiv:2307.03381 (ICLR 2024). — Formato invertido de la suma.
* Belrose, N. et al. (2023). 'Eliciting Latent Predictions from Transformers with the Tuned Lens'. arXiv:2303.08112. — La lente de logits refinada (16.5).
* Schaeffer, R. et al. (2023). 'Are Emergent Abilities of Large Language Models a Mirage?'. arXiv:2304.15004. — La emergencia depende de la métrica.
* Gu, A. & Dao, T. (2023). 'Mamba: Linear-Time Sequence Modeling with Selective State Spaces'. arXiv:2312.00752. — Una alternativa a la atención para secuencias muy largas.

* Papineni, K., Roukos, S., Ward, T. & Zhu, W.-J. (2002). 'BLEU: a Method for Automatic Evaluation of Machine Translation'. ACL. — La métrica BLEU del Capítulo 18.
* Ba, J.L., Kiros, J.R. & Hinton, G.E. (2016). 'Layer Normalization'. arXiv:1607.06450. — La LayerNorm del bloque Transformer.
* Hendrycks, D. & Gimpel, K. (2016). 'Gaussian Error Linear Units (GELUs)'. arXiv:1606.08415. — La activación GELU de GPT-2 y del Nano-LLM.
* Jain, S. & Wallace, B.C. (2019). 'Attention is not Explanation'. NAACL. arXiv:1902.10186. — Por qué los pesos de atención no bastan para explicar una predicción.
* Dao, T. et al. (2022). 'FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness'. arXiv:2205.14135. — Calcular la atención sin guardar entera la matriz T×T.
* Ainslie, J. et al. (2023). 'GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints'. arXiv:2305.13245. — Grouped-Query Attention.
* Kwon, W. et al. (2023). 'Efficient Memory Management for Large Language Model Serving with PagedAttention'. arXiv:2309.06180. — PagedAttention y vLLM.
* Benchmarks citados en el Capítulo 18: Zellers, R. et al. (2019), HellaSwag, arXiv:1905.07830; Cobbe, K. et al. (2021), GSM8K, arXiv:2110.14168; Lin, S. et al. (2022), TruthfulQA, arXiv:2109.07958; Rein, D. et al. (2023), GPQA, arXiv:2311.12022; Jimenez, C. E. et al. (2023), SWE-bench, arXiv:2310.06770; Phan, L. et al. (2025), Humanity's Last Exam, arXiv:2501.14249; Chollet, F. (2019), On the Measure of Intelligence (ARC), arXiv:1911.01547.

Recursos online gratuitos

* The Illustrated Transformer — Jay Alammar (jalammar.github.io). La mejor explicación visual del Transformer con animaciones paso a paso.
* Neural Networks: Zero to Hero — Andrej Karpathy (karpathy.ai/zero-to-hero.html). Serie completa construyendo modelos desde cero, incluyendo BPE y GPT.
* fast.ai — Practical Deep Learning for Coders (course.fast.ai). Enfoque top-down muy efectivo; cubre overfitting, regularización y técnicas avanzadas.
* Stanford CS224n: NLP with Deep Learning (web.stanford.edu/class/cs224n). Nivel universitario; materiales públicos con slides y vídeos.
* LMArena, antes Chatbot Arena (arena.ai; la antigua dirección lmarena.ai redirige allí). Ranking ELO de LLMs basado en millones de preferencias humanas reales.
* Hugging Face Papers (huggingface.co/papers). Artículos recientes con enlaces a código, modelos y datasets; sustituye a Papers With Code, cerrado en julio de 2025.
* Hugging Face Hub (huggingface.co). Modelos open-source, datasets, tokenizadores y comunidad.
* Tokenizer Playground (platform.openai.com/tokenizer). Herramienta de OpenAI para ver cómo GPT-4 tokeniza cualquier texto.

Libros recomendados

* Goodfellow, I., Bengio, Y., Courville, A. (2016). 'Deep Learning'. MIT Press. Disponible gratis en deeplearningbook.org. La referencia académica completa.
* Géron, A. (2022). 'Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow'. O'Reilly. Muy práctico, cubre overfitting, regularización y CNNs en profundidad. Su sucesor, «Hands-On Machine Learning with Scikit-Learn and PyTorch» (2025), sustituye Keras por PyTorch.

* Howard, J. & Gugger, S. (2020). 'Deep Learning for Coders with fastai and PyTorch'. O'Reilly. Orientado a la práctica; excelente sobre evaluación y técnicas modernas.
* Fleuret, F. (2023). 'The Little Book of Deep Learning'. Disponible en fleuret.org/public/lbdl.pdf. Conciso y matemáticamente riguroso.
* Raschka, S. (2024). 'Build a Large Language Model (From Scratch)'. Manning. Construye un GPT con PyTorch paso a paso y carga en él los pesos de GPT-2: el complemento natural de la Parte III de este libro.
* O'Neil, C. (2016). 'Weapons of Math Destruction'. Crown. El libro más accesible sobre el impacto social de los algoritmos y los sesgos en IA.

Libros recomendados en español

* Géron, Aurélien (2023). 'Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow' (3.ª ed.). Anaya Multimedia. La referencia práctica más recomendada en español. Cubre desde los fundamentos hasta redes profundas con proyectos reales. Traducción de la 3.ª edición inglesa.
* Chollet, François. 'Deep Learning con Python'. Anaya Multimedia (ISBN 978-84-415-4225-9). Explica las redes neuronales profundas hasta la implementación con Keras. Las ediciones inglesas posteriores (2.ª, 2021; 3.ª, 2025, con Matthew Watson) añaden los Transformers. Muy valorado por su claridad conceptual.
* Russell, Stuart y Norvig, Peter. 'Inteligencia Artificial: Un Enfoque Moderno'. Pearson (2.ª ed. en español, 2004; la 4.ª edición inglesa es de 2020). El manual universitario de referencia mundial, disponible en español. Cubre búsqueda, planificación, incertidumbre y aprendizaje. Exigente pero completo para quien busca una base teórica sólida.
* Severance, Charles (Dr. Chuck). 'Python para Todos'. Disponible gratis en es.py4e.com. El mejor punto de partida si aún estás aprendiendo Python. Con ejercicios y vídeos. Altamente valorado por su claridad y accesibilidad.
* Torres, Jordi (2020). 'Python Deep Learning: Introducción práctica con Keras y TensorFlow 2'. Marcombo. Uno de los pocos libros de aprendizaje profundo escritos originalmente en español, por un investigador del Barcelona Supercomputing Center.

Comunidades y plataformas

* Kaggle (kaggle.com). Competencias, datasets y notebooks. Excelente para practicar evaluación y comparación de modelos.
* Hugging Face (huggingface.co). Modelos open-source, datasets, tokenizadores BPE listos para usar con la librería 'tokenizers'.
* arXiv.org/cs.LG y cs.CL. Preprints de investigación en Machine Learning y Procesamiento de Lenguaje Natural.
* AI Fairness 360 (github.com/Trusted-AI/AIF360). Librería de código abierto, iniciada por IBM, con decenas de métricas de equidad y 15 algoritmos para mitigar sesgos.
* Fairlearn (fairlearn.org). Librería de código abierto, iniciada por Microsoft y hoy mantenida por su comunidad, para medir y mitigar sesgos en modelos ML con Python.

Apéndice D: Los Mismos Programas con PyTorch

D.1 ¿Por qué PyTorch?

A lo largo del libro hemos construido cada componente desde cero con NumPy: los gradientes, el backpropagation, las actualizaciones de pesos. Ese camino era intencionado — entender cada operación sin ningún velo es la mejor forma de comprender de verdad cómo funciona una red neuronal.

Ahora que tienes esa base, presentamos PyTorch: el framework de Deep Learning más usado en investigación académica y ampliamente adoptado en la industria. PyTorch no hace nada distinto de lo que hemos hecho nosotros. Lo que sí hace es automatizar las partes más tediosas y propensas a errores, especialmente el cálculo de gradientes.

La diferencia más importante es el sistema de diferenciación automática, llamado Autograd. Cuando defines operaciones con tensores de PyTorch, el framework construye internamente un grafo de computación. Cuando llamas a .backward(), recorre ese grafo hacia atrás aplicando la regla de la cadena automáticamente. Tú nunca tienes que calcular un solo gradiente a mano.

> 💡 Comparación directa: en el Programa 3 (MLP) necesitamos unas 40 líneas para implementar el backward pass manualmente. Con PyTorch, esas 40 líneas se reducen a tres: optimizador.zero_grad() / loss.backward() / optimizador.step(). La arquitectura del modelo es exactamente la misma.

D.2 Conceptos clave de PyTorch

Antes de los programas, un resumen de las abstracciones fundamentales de PyTorch que sustituyen a las operaciones NumPy que ya conoces:

* torch.Tensor: el equivalente a np.ndarray, pero con soporte para gradientes y GPU. La operación A @ B funciona igual que en NumPy.
* requires_grad=True: indica que queremos calcular gradientes respecto a este tensor. Los parámetros de nn.Linear y nn.Embedding lo tienen activado por defecto.
* nn.Module: clase base para todos los modelos. Al heredar de ella, los parámetros se registran automáticamente y son accesibles con modelo.parameters().
* nn.Linear(n_ent, n_sal): equivalente a nuestra clase CapaDensa. Crea automáticamente la matriz de pesos W y el vector de sesgos b, inicializados con valores razonables.
* nn.Embedding(V, D): la tabla de embeddings del Capítulo 8. Internamente es una matriz (V, D) diferenciable.
* optim.Adam y optim.SGD: implementaciones de los optimizadores. Reemplazan nuestro w -= lr * grad.
* loss.backward(): calcula los gradientes de todos los parámetros. Una sola línea reemplaza todas las funciones backward() que escribimos en NumPy.

D.3 Instalación de PyTorch

📄 **Instalación (elegir según el sistema)**

```
# Opción 1: CPU solamente (suficiente para este libro)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

# Opción 2: Con GPU NVIDIA: copia el comando que da pytorch.org/get-started/locally
# (elige allí tu versión de CUDA; el comando termina en .../whl/cuXXX)

# PyTorch ya no se distribuye por conda: usa siempre pip
# (también dentro de un entorno de conda)

# Verificar la instalación
python -c "import torch; print('PyTorch', torch.__version__, '- GPU:',
torch.cuda.is_available())"
```

ℹ️ Para los programas de este apéndice, la versión CPU es completamente suficiente. Todos los modelos son lo bastante pequeños para entrenar en segundos en cualquier ordenador moderno.

PROGRAMA PT-1 — Tensores, Autograd y Funciones de Activación

El primer programa muestra la abstracción más importante de PyTorch: el tensor diferenciable. Comparamos la sintaxis con NumPy y demostramos que .backward() calcula gradientes de forma automática.

📄 **programa_PT1_tensores_activaciones.py**

```
import torch
import torch.nn.functional as F

# ============================================================
# PARTE 1: Tensores y comparativa con NumPy
# ============================================================
import numpy as np

# NumPy
a_np = np.array([1.0, 2.0, 3.0])
# PyTorch — sintaxis casi identica
a_pt = torch.tensor([1.0, 2.0, 3.0])

print('NumPy: ', a_np)
print('PyTorch:', a_pt)
print('Suma NumPy: ', a_np.sum())
print('Suma PyTorch:', a_pt.sum())
print()

# ============================================================
# PARTE 2: Autograd — gradientes automáticos
# ============================================================
# Queremos calcular la derivada de L = (w*x - y)^2
# respecto a w, evaluada en w=2, x=3, y=10

x = torch.tensor(3.0)
y = torch.tensor(10.0)
w = torch.tensor(2.0, requires_grad=True)  # <-- activa autograd

# Forward: calcular la perdida
pred = w * x
L    = (pred - y) ** 2

print(f'w={w.item()}, x={x.item()}, y={y.item()}')
print(f'pred = w*x = {pred.item()}')
print(f'L = (pred-y)^2 = {L.item()}')

# Backward: PyTorch calcula dL/dw automáticamente
L.backward()

print(f'dL/dw (autograd): {w.grad.item()}')

# Verificación manual: dL/dw = 2*(w*x - y)*x = 2*(6-10)*3 = -24
dL_dw_manual = 2 * (w.item()*x.item() - y.item()) * x.item()
print(f'dL/dw (manual):   {dL_dw_manual}')
print(f'Coinciden: {abs(w.grad.item() - dL_dw_manual) < 1e-5}')

# ============================================================
# PARTE 3: Funciones de activación — torch vs implementación propia
# ============================================================
z = torch.tensor([-3.0, -1.0, 0.0, 1.0, 3.0])

print(f"{'z':>6}  {'sigmoid':>10}  {'ReLU':>8}  {'tanh':>8}")
print('-' * 38)
for zi, s, r, t in zip(z,
                       torch.sigmoid(z),
                       F.relu(z),
                       torch.tanh(z)):
    print(f'{zi.item():>6.1f}  {s.item():>10.4f}  {r.item():>8.2f}  {t.item():>8.4f}')
```

```
# Softmax: devuelve distribucion de probabilidad
logits = torch.tensor([2.0, 1.0, 0.5, -0.5])
probs  = F.softmax(logits, dim=0)
print(f'Softmax: {probs.numpy().round(4)}  suma={probs.sum().item():.6f}')
```

> Salida del programa:

```
NumPy:  [1. 2. 3.]
PyTorch: tensor([1., 2., 3.])
Suma NumPy:  6.0
Suma PyTorch: tensor(6.)

w=2.0, x=3.0, y=10.0
pred = w*x = 6.0
L = (pred-y)^2 = 16.0
dL/dw (autograd): -24.0
dL/dw (manual):   -24.0
Coinciden: True

     z     sigmoid      ReLU      tanh
-------------------------------------
  -3.0     0.0474      0.00   -0.9951
  -1.0     0.2689      0.00   -0.7616
   0.0     0.5000      0.00    0.0000
   1.0     0.7311      1.00    0.7616
   3.0     0.9526      3.00    0.9951
Softmax: [0.5977 0.2199 0.1334 0.0491]  suma=1.000000
```

PROGRAMA PT-2 — Neurona que Aprende: AND con Autograd

La misma puerta lógica AND del Programa 2, pero ahora PyTorch calcula todos los gradientes automáticamente. El bucle de entrenamiento se reduce a cuatro líneas. Fíjate en la diferencia respecto a las 15 líneas de backpropagation manual que necesitábamos en NumPy.

programa_PT2_neurona_and.py

```
import torch
import torch.nn as nn
import torch.optim as optim
torch.manual_seed(0)   # pesos iniciales reproducibles

# Datos: puerta logica AND
X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
y = torch.tensor([[0],[0],[0],[1]],         dtype=torch.float32)

# ============================================================
# EL MODELO: una sola neurona = nn.Linear(2, 1)
# PyTorch crea automaticamente los pesos W (2x1) y sesgo b (1)
# ============================================================
neurona = nn.Linear(2, 1)

# BCEWithLogitsLoss = sigmoid + BCE en una sola operacion
# (mas estable numericamente que aplicarlos por separado)
criterio    = nn.BCEWithLogitsLoss()
optimizador = optim.SGD(neurona.parameters(), lr=0.5)

print('=== NEURONA AND CON PYTORCH ===')
print(f'Parametros iniciales:')
print(f'  Pesos: {neurona.weight.data.numpy().round(3)}')
print(f'  Sesgo: {neurona.bias.data.item():.4f}')
print()

# ============================================================
# BUCLE DE ENTRENAMIENTO
# Comparacion con NumPy:
#   NumPy:   ~15 lineas de gradientes manuales
#   PyTorch: 4 lineas (zero_grad, forward, backward, step)
# ============================================================
for epoca in range(10_000):
    logits = neurona(X)          # forward
    loss   = criterio(logits, y) # perdida
    optimizador.zero_grad()      # limpiar gradientes
    loss.backward()              # backprop automatico
```

```
        optimizador.step()          # actualizar pesos

        if epoca % 2000 == 0:
            print(f'Epoca {epoca:5d}: perdida = {loss.item():.6f}')

    # Resultados finales
    print(f'\nPesos finales: {neurona.weight.data.numpy().round(3)}')
    print(f'Sesgo final:   {neurona.bias.data.item():.4f}')
    print('\nPrueba final:')
    with torch.no_grad():
        probs = torch.sigmoid(neurona(X))
    for xi, yi, pi in zip(X, y, probs):
        ok = 'v' if round(pi.item()) == int(yi.item()) else 'x'
        print(f'  AND{xi.int().tolist()} = {int(yi.item())}  pred={pi.item():.4f}  {ok}')
```

> Salida del programa:

```
=== NEURONA AND CON PYTORCH ===
Parametros iniciales:
  Pesos: [[-0.005  0.379]]
  Sesgo: -0.5820

Epoca     0: perdida = 0.571347
Epoca  2000: perdida = 0.017364
Epoca  4000: perdida = 0.008659
Epoca  6000: perdida = 0.005755
Epoca  8000: perdida = 0.004307

Pesos finales: [[10.678 10.678]]
Sesgo final:   -16.1863

Prueba final:
  AND[0, 0] = 0  pred=0.0000  v
  AND[0, 1] = 0  pred=0.0040  v
  AND[1, 0] = 0  pred=0.0040  v
  AND[1, 1] = 1  pred=0.9943  v
```

ℹ️ Los pesos finales (10.68 y 10.68, con sesgo −16.19) no son los del Programa 2 (7.28 y −11.01): PyTorch usa aquí otra pérdida (la entropía cruzada binaria en vez del MSE), actualiza con los cuatro ejemplos a la vez y parte de otros pesos iniciales. Pero la solución es la misma: dos pesos iguales y un sesgo negativo de 1.5 veces cada peso (16.19 / 10.68 ≈ 11.01 / 7.28 ≈ 1.5), mayor que un peso y menor que la suma de los dos. La arquitectura es la misma; lo único diferente es que PyTorch calculó el gradiente automáticamente.

PROGRAMA PT-3 — MLP con nn.Module y XOR

Reimplementamos la clase RedNeuronal usando la API de nn.Module de PyTorch. La arquitectura es idéntica al Programa 3+4 de NumPy, pero la definición es mucho más limpia y no necesitamos implementar el backward en absoluto.

📄 **programa_PT3_mlp_xor.py**

```
import torch
import torch.nn as nn
import torch.optim as optim

X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
y = torch.tensor([[0],[1],[1],[0]],         dtype=torch.float32)

# ============================================================
# CLASE MLP CON nn.Module
# Hereda de nn.Module para registrar los parametros
# automaticamente y ser compatible con todos los optimizadores
# ============================================================
class MLP(nn.Module):
    def __init__(self, n_ent, n_ocul, n_sal):
        super().__init__()
        # nn.Sequential encadena las capas en orden
        self.red = nn.Sequential(
            nn.Linear(n_ent, n_ocul),  # Z1 = X @ W1 + b1
            nn.ReLU(),                  # A1 = ReLU(Z1)
            nn.Linear(n_ocul, n_sal),  # Z2 = A1 @ W2 + b2
```

```
            nn.Sigmoid()              # y^ = sigmoid(Z2)
        )

    def forward(self, x):
        return self.red(x)


# Crear modelo, perdida y optimizador
torch.manual_seed(42)
modelo      = MLP(2, 8, 1)
criterio    = nn.BCELoss()
optimizador = optim.Adam(modelo.parameters(), lr=0.01)

n_params = sum(p.numel() for p in modelo.parameters())
print(f'Arquitectura: 2 -> 8 (ReLU) -> 1 (Sigmoid)')
print(f'Parametros:   {n_params}')
print(f'Estructura:')
for nombre, param in modelo.named_parameters():
    print(f'  {nombre:>20}: {list(param.shape)}')
print()

# ================================================================
# ENTRENAMIENTO: el bucle mas sencillo posible
# ================================================================
for epoca in range(5000):
    pred = modelo(X)
    loss = criterio(pred, y)
    optimizador.zero_grad()
    loss.backward()
    optimizador.step()
    if epoca % 1000 == 0:
        print(f'Epoca {epoca:5d}: perdida = {loss.item():.6f}')

# Evaluacion final
print('\nXOR resuelto con MLP PyTorch:')
with torch.no_grad():
    pred = modelo(X)
correcto = 0
for xi, yi, pi in zip(X, y, pred):
    ok = 'v' if round(pi.item()) == int(yi.item()) else 'x'
    correcto += round(pi.item()) == int(yi.item())
    print(f'  XOR{xi.int().tolist()} = {int(yi.item())}  pred={pi.item():.4f}  {ok}')
print(f'Precision: {correcto}/4')
```

PROGRAMA PT-4 — Embeddings con nn.Embedding

La tabla de embeddings del Capítulo 8 se implementa en PyTorch con nn.Embedding. La gran ventaja: al estar integrada en el modelo como un nn.Module, sus pesos se actualizan automáticamente durante el entrenamiento junto con el resto de parámetros.

programa_PT4_embeddings.py

```
import torch
import torch.nn as nn

# Tokenizador identico al Programa 5 NumPy
texto = 'hola mundo, aprendiendo inteligencia artificial'
vocab = sorted(set(texto))
V     = len(vocab)
c2i   = {c: i for i, c in enumerate(vocab)}
i2c   = dict(enumerate(vocab))

def codificar(t):   return torch.tensor([c2i[c] for c in t])
def decodificar(t): return ''.join(i2c[i.item()] for i in t)

# ================================================================
# nn.Embedding: tabla de busqueda diferenciable
# Es una matriz (V, DIM) cuyos valores se APRENDEN
# Se indexa igual que una lista: tabla[indice] -> vector
# ================================================================
DIM   = 8
tabla = nn.Embedding(V, DIM)
```

```
print(f'Vocabulario: {V} caracteres')
print(f'Tabla de embeddings: {tabla.weight.shape}')
print(f'Parametros aprendibles: {tabla.weight.numel()}')
print(f'  (= {V} tokens x {DIM} dim = {V * DIM})')
print()

# Uso: igual que indexar un array
tokens  = codificar('hola')
embs    = tabla(tokens)   # (4, 8)
print(f'Tokens de hola: {tokens.tolist()}')
print(f'Shape: {embs.shape}')
print(f'Embedding de h:')
print(f'  {embs[0].detach().numpy().round(3)}')
print()

# Similitud coseno entre dos embeddings
def sim_coseno(u, v):
    return (u @ v / (u.norm() * v.norm() + 1e-8)).item()

chars_test = ['h', 'o', 'l', 'a', ' ', 'i']
print('Similitudes coseno (aleatorias antes de entrenar):')
for c1, c2 in [('h','o'), ('h','h'), ('h',' ')]:
    e1 = tabla(torch.tensor(c2i[c1]))
    e2 = tabla(torch.tensor(c2i[c2]))
    print(f'  sim({repr(c1)}, {repr(c2)}) = {sim_coseno(e1, e2):.4f}')

print()
print('Tras el entrenamiento, caracteres en contextos')
print('similares tendran similitudes mas altas.')
print('Esto emerge de loss.backward() + optimizer.step().')
```

PROGRAMA PT-5 — Self-Attention con nn.Module

El mecanismo de self-attention del Programa 6, reimplementado como clase PyTorch. Las matrices Wq, Wk, Wv son ahora instancias de nn.Linear, lo que significa que sus gradientes se calculan automáticamente.

📄 **programa_PT5_self_attention.py**

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttentionCausal(nn.Module):
    '''
    Self-attention causal con PyTorch.
    Wq, Wk, Wv son nn.Linear -> gradientes automaticos.
    '''
    def __init__(self, d):
        super().__init__()
        self.d  = d
        self.Wq = nn.Linear(d, d, bias=False)
        self.Wk = nn.Linear(d, d, bias=False)
        self.Wv = nn.Linear(d, d, bias=False)
        self.Wo = nn.Linear(d, d, bias=False)

    def forward(self, X):
        # X: (T, d)
        T = X.shape[0]
        Q = self.Wq(X)   # (T, d)
        K = self.Wk(X)   # (T, d)
        V = self.Wv(X)   # (T, d)

        # Puntuaciones escaladas
        scores = (Q @ K.T) / (self.d ** 0.5)   # (T, T)

        # Mascara causal: el token t no puede ver t+1, t+2...
        mask   = torch.triu(torch.full((T, T), float('-inf')), diagonal=1)
        scores = scores + mask
```

```
        # Pesos de atencion y salida
        pesos  = F.softmax(scores, dim=-1)  # (T, T)
        return self.Wo(pesos @ V), pesos    # (T, d)


torch.manual_seed(3)
T, d = 5, 8
X    = torch.randn(T, d)

attn = SelfAttentionCausal(d)
with torch.no_grad():
    salida, pesos = attn(X)

print(f'=== SELF-ATTENTION PYTORCH ===')
print(f'Entrada: {X.shape}  ->  Salida: {salida.shape}')
print(f'Pesos de atencion ({T}x{T}):')
print(pesos.numpy().round(3))
print(f'Filas suman 1: {torch.allclose(pesos.sum(dim=-1), torch.ones(T))}')
n_p = sum(p.numel() for p in attn.parameters())
print(f'Parametros de la capa: {n_p}')

# Comparativa de codigo con NumPy:
# NumPy:   salida, pesos = self_attention(X, Wq, Wk, Wv)
#          -> requeria pasar matrices como argumentos
# PyTorch: salida, pesos = attn(X)
#          -> las matrices son parte del objeto, con gradientes
```

PROGRAMA PT-6 — Bloque Transformer

El bloque Transformer del Programa 7 (Pre-LN, como GPT-2) reimplementado con nn.MultiheadAttention (aquí con 2 cabezas; el Programa 7 usa una) y nn.TransformerEncoderLayer (que por defecto usa ReLU en lugar de GELU: misma estructura y cuenta de parámetros, no los mismos cálculos). PyTorch incluye estos módulos optimizados de fábrica, lo que hace el código notablemente más compacto.

📄 **programa_PT6_transformer_bloque.py**

```
import torch
import torch.nn as nn

class BloqueTransformer(nn.Module):
    '''
    Bloque Transformer usando modulos nativos de PyTorch.
    Mismo diseño que el Programa 7 NumPy (Pre-LN, como GPT-2):
      LayerNorm + Self-Attention causal + residual
      LayerNorm + Feed-Forward (GELU) + residual
    '''
    def __init__(self, d, d_ff, n_heads=2):
        super().__init__()
        # Multi-Head Attention nativo
        self.atencion = nn.MultiheadAttention(
            embed_dim=d,
            num_heads=n_heads,
            dropout=0.0,
            batch_first=False   # X tiene forma (T, B, d)
        )
        # Feed-Forward Network
        self.ffn = nn.Sequential(
            nn.Linear(d, d_ff), nn.GELU(approximate='tanh'),
            nn.Linear(d_ff, d)
        )
        # Dos capas de normalizacion
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

    def forward(self, X):
        # X: (T, d) -- sin dimension de batch
        T  = X.shape[0]

        # Mascara causal: impide ver el futuro
        mask = nn.Transformer.generate_square_subsequent_mask(T)
```

```
            # Sub-bloque 1: norm + Self-Attention + residual
            a = self.norm1(X).unsqueeze(1)   # (T, 1, d) para MultiheadAttention
            attn_out, _ = self.atencion(a, a, a,
                                        attn_mask=mask,
                                        is_causal=True)

            X = X + attn_out.squeeze(1)

            # Sub-bloque 2: norm + FFN + residual
            X = X + self.ffn(self.norm2(X))
            return X


    torch.manual_seed(5)
    d, d_ff, T = 16, 64, 6
    bloque = BloqueTransformer(d, d_ff, n_heads=2)
    X = torch.randn(T, d)

    with torch.no_grad():
        Y = bloque(X)

    n_p = sum(p.numel() for p in bloque.parameters())
    print(f'Bloque Transformer PyTorch: d={d}, d_ff={d_ff}, n_heads=2')
    print(f'Entrada: {X.shape}  ->  Salida: {Y.shape}')
    print(f'Parametros: {n_p:,}  (los mismos 3.280 del Programa 7)')
    print(f'Std entrada: {X.std().item():.4f}   Std salida: {Y.std().item():.4f}')

    # Alternativa aun mas compacta: nn.TransformerEncoderLayer
    capa_rapida = nn.TransformerEncoderLayer(
        d_model=d, nhead=2, dim_feedforward=d_ff,
        dropout=0.0, batch_first=True, norm_first=True
    )
    n_p2 = sum(p.numel() for p in capa_rapida.parameters())
    print(f'\nTransformerEncoderLayer: {n_p2:,} params (mismo numero de parametros)')
```

> Salida del programa (PyTorch 2.14, CPU):

```
Bloque Transformer PyTorch: d=16, d_ff=64, n_heads=2
Entrada: torch.Size([6, 16])  ->  Salida: torch.Size([6, 16])
Parametros: 3,280  (los mismos 3.280 del Programa 7)
Std entrada: 0.9249   Std salida: 0.9984

TransformerEncoderLayer: 3,280 params (mismo numero de parametros)
```

PROGRAMA PT-7 — Nano-LLM Completo con PyTorch

El programa más importante del apéndice: el Nano-LLM del Programa 8, el GPT-2 en miniatura, reimplementado con PyTorch. La arquitectura es exactamente la misma —4 cabezas con sesgo en Q, K, V y Wo, Pre-LN con nn.LayerNorm, FFN 48 → 192 → 48 con GELU, LayerNorm final y pesos compartidos entre la entrada y la salida— y por eso imprime el mismo número de parámetros: 60 000. Hemos comprobado que, cargando en él los pesos que guarda el Programa 8, da los mismos logits (con diferencias de 10^{-5}, la precisión de float32). El bloque está escrito a mano, con las mismas piezas, para que se lea igual que el de NumPy. Lo que desaparece es todo el código de gradientes manuales: el backward es una sola línea, loss.backward(). El entrenamiento es el del Programa 8 —Adam con warm-up y coseno y recorte del gradiente—, con el lote de 8 ventanas procesado como un único tensor. Usar AdamW (5.6) es cambiar una línea: torch.optim.AdamW en lugar de torch.optim.Adam.

📄 programa_PT7_nano_llm.py (parte 1: datos y modelo)

```
# ============================================================
# PROGRAMA PT-7 — Nano-LLM completo con PyTorch
# Apéndice D — El MISMO GPT-2 en miniatura del Programa 8,
```

```
# ahora con autograd: PyTorch calcula el backward por nosotros.
#
# Comprobado con PyTorch 2.14 (CPU): imprime
# «Total de parámetros: 60,000», igual que el Programa 8.
#
# Las piezas son las de GPT-2 y las del Programa 8:
#   - embeddings de token y de posición aprendidos
#   - bloques Pre-LN: nn.LayerNorm (con γ y β) ANTES de cada subcapa
#   - atención causal de 4 cabezas, con sesgo en Q, K, V y Wo
#   - FFN 48 -> 192 (GELU, aproximación tanh) -> 48
#   - LayerNorm final y pesos compartidos entre entrada y salida
# Cambia el backward (autograd) y nada más.
# ============================================================
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

# -- Corpus: el mismo TEXTO del Programa 8 ------------------
fuente = open('programa_8_nano_llm.py', encoding='utf-8').read()
TEXTO = fuente.split('TEXTO = """')[1].split('"""')[0]
VOCAB = sorted(set(TEXTO))
V = len(VOCAB)
C2I = {c: i for i, c in enumerate(VOCAB)}
I2C = dict(enumerate(VOCAB))
DATOS = torch.tensor([C2I[c] for c in TEXTO], dtype=torch.long)

# -- Hiperparámetros: idénticos al Programa 8 ----------------
CTX, D, H, D_FF, N_BLQ = 24, 48, 4, 192, 2
LR, PASOS, BATCH, CLIP = 3e-3, 12000, 8, 1.0
STD_RES = 0.02 / math.sqrt(2 * N_BLQ)
torch.manual_seed(42)


class Bloque(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1 = nn.LayerNorm(D)              # γ y β incluidos
        self.Wq, self.Wk, self.Wv = nn.Linear(D, D), nn.Linear(D, D), nn.Linear(D, D)
        self.Wo = nn.Linear(D, D)
        self.ln2 = nn.LayerNorm(D)
        self.W1 = nn.Linear(D, D_FF)
        self.W2 = nn.Linear(D_FF, D)

    def atencion(self, a):                      # a: (B, T, D)
        B, T, _ = a.shape
        separar = lambda x: x.view(B, T, H, D // H).transpose(1, 2)   # (B, H, T, DH)
        Q, K, Vm = separar(self.Wq(a)), separar(self.Wk(a)), separar(self.Wv(a))
        scores = Q @ K.transpose(-2, -1) / math.sqrt(D // H)
        futuro = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
        pesos = F.softmax(scores.masked_fill(futuro, float('-inf')), dim=-1)
        return self.Wo((pesos @ Vm).transpose(1, 2).reshape(B, T, D))

    def forward(self, X):
        X = X + self.atencion(self.ln1(X))                          # Pre-LN
        X = X + self.W2(F.gelu(self.W1(self.ln2(X)), approximate='tanh'))
        return X


class NanoLLM(nn.Module):
    def __init__(self):
        super().__init__()
        self.wte = nn.Embedding(V, D)
        self.wpe = nn.Embedding(CTX, D)
        self.bloques = nn.ModuleList([Bloque() for _ in range(N_BLQ)])
        self.lnf = nn.LayerNorm(D)
        for nombre, p in self.named_parameters():   # inicialización de GPT-2
            if nombre.endswith('bias'):
                nn.init.zeros_(p)
            elif p.dim() == 2:
                std = STD_RES if ('Wo' in nombre or 'W2' in nombre) else 0.02
                nn.init.normal_(p, std=std)

    def forward(self, idx):                     # idx: (B, T)
        T = idx.shape[1]
        X = self.wte(idx) + self.wpe(torch.arange(T))
        for b in self.bloques:
            X = b(X)
        return self.lnf(X) @ self.wte.weight.T   # pesos compartidos
```

📄 programa_PT7_nano_llm.py (parte 2: entrenamiento y generación)

```
modelo = NanoLLM()
print(f'Vocabulario: {V} caracteres')
```

```
print(f'Total de parámetros: {sum(p.numel() for p in modelo.parameters()):,}')

# -- Entrenamiento: Adam + warmup/coseno + recorte ------------
opt = torch.optim.Adam(modelo.parameters(), lr=LR, betas=(0.9, 0.999), eps=1e-8)


def lr_factor(t, t_warmup=150, lr_min=1e-4):
    if t < t_warmup:
        return (t + 1) / t_warmup
    c = 0.5 * (1 + math.cos(math.pi * (t - t_warmup) / (PASOS - t_warmup)))
    return (lr_min + (LR - lr_min) * c) / LR


plan = torch.optim.lr_scheduler.LambdaLR(opt, lr_factor)

for paso in range(PASOS):
    i = torch.randint(0, len(DATOS) - CTX - 1, (BATCH,))
    xb = torch.stack([DATOS[j:j + CTX] for j in i])
    yb = torch.stack([DATOS[j + 1:j + CTX + 1] for j in i])
    loss = F.cross_entropy(modelo(xb).reshape(-1, V), yb.reshape(-1))
    opt.zero_grad()
    loss.backward()                                # todo el backward en una línea
    torch.nn.utils.clip_grad_norm_(modelo.parameters(), CLIP)
    opt.step()
    plan.step()
    if paso % 1000 == 0 or paso == PASOS - 1:
        print(f'Paso {paso:5d}: pérdida = {loss.item():.4f}   '
              f'(bits/char = {loss.item() / math.log(2):.2f})')


# -- Generación: temperatura + top-p (como el Programa 8) ------
@torch.no_grad()
def generar(semilla, n_chars=120, temperatura=0.8, top_p=0.9):
    tokens = [C2I.get(c, 0) for c in semilla[-CTX:]]
    for _ in range(n_chars):
        logits = modelo(torch.tensor([tokens[-CTX:]]))[0, -1]
        probs = F.softmax(logits / temperatura, dim=-1)
        orden = torch.argsort(probs, descending=True)
        acumulada = torch.cumsum(probs[orden], 0)
        n = int((acumulada < top_p).sum()) + 1
        nucleo = torch.zeros_like(probs)
        nucleo[orden[:n]] = probs[orden[:n]]
        tokens.append(int(torch.multinomial(nucleo / nucleo.sum(), 1)))
    return semilla + ''.join(I2C[t] for t in tokens[len(semilla[-CTX:]):])


print('\n=== TEXTO GENERADO POR EL NANO-LLM (PyTorch) ===')
print(generar('la inteligencia'))
print()
print(generar('los modelos de'))
```

> Salida del programa (PyTorch 2.14, CPU):

```
Vocabulario: 46 caracteres
Total de parámetros: 60,000
Paso     0: pérdida = 3.8674   (bits/char = 5.58)
Paso  1000: pérdida = 1.6491   (bits/char = 2.38)
Paso  2000: pérdida = 0.8981   (bits/char = 1.30)
Paso  3000: pérdida = 0.7064   (bits/char = 1.02)
Paso  4000: pérdida = 0.4982   (bits/char = 0.72)
Paso  5000: pérdida = 0.4594   (bits/char = 0.66)
Paso  6000: pérdida = 0.4293   (bits/char = 0.62)
Paso  7000: pérdida = 0.3269   (bits/char = 0.47)
Paso  8000: pérdida = 0.3594   (bits/char = 0.52)
Paso  9000: pérdida = 0.2948   (bits/char = 0.43)
Paso 10000: pérdida = 0.2729   (bits/char = 0.39)
Paso 11000: pérdida = 0.2876   (bits/char = 0.41)
Paso 11999: pérdida = 0.2854   (bits/char = 0.41)

=== TEXTO GENERADO POR EL NANO-LLM (PyTorch) ===
la inteligencia artificial aprende de los datos.
los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas

los modelos de lenguaje predicen el siguiente token.
las redes neuronales tienen capas de neuronas.
el aprendizaje profundo usa muchas
```

La salida no coincide número a número con la del Programa 8: PyTorch usa otro generador de números aleatorios, así que los pesos iniciales y las ventanas de cada mini-lote son otros. Pero el comportamiento es el mismo: la pérdida baja de 3.87 a 0.29 en el último mini-lote y el texto generado recita el corpus exactamente igual que el del Programa 8. La primera

línea que imprime el programa es la comprobación clave: 60 000 parámetros, igual que en NumPy.

> **✅ PyTorch y NumPy, el mismo GPT-2**
>
> La comprobación más exigente del Programa 8 bis (16.1) se hizo precisamente con PyTorch. La librería transformers de Hugging Face trae una implementación de GPT-2, `GPT2LMHeadModel`, construida con las mismas piezas de este apéndice. Cargando en ella los mismos pesos, sus logits y los de nuestro `forward()` de NumPy coinciden con una diferencia máxima de $8 \cdot 10^{-5}$ en float32 (en cinco frases) y de $1.6 \cdot 10^{-12}$ en float64. Y su `loss.backward()` da los mismos gradientes que el `forward_backward()` del Programa 8: en float64, con un error relativo máximo de $3 \cdot 10^{-13}$.
>
> Los dos programas de la comprobación están en la carpeta `verificaciones` del repositorio del libro. Necesitan `pip install torch transformers` y la misma carpeta gpt2/ del Capítulo 16.

D.4 Tabla comparativa: NumPy vs PyTorch

Para consolidar lo aprendido, esta tabla muestra las equivalencias entre los conceptos que implementamos manualmente con NumPy y sus contrapartes en PyTorch:

```
  📄 Equivalencias NumPy / PyTorch
  CONCEPTO              NUMPY (manual)              PYTORCH
  ------------------------------------------------------------------------
  Array/Tensor          np.array([1,2,3])           torch.tensor([1,2,3])
  Producto matricial    A @ B                       A @ B  (identico)
  Capa lineal           W, b = randn(), zeros()     nn.Linear(n_ent, n_sal)
  Activacion sigmoid    1/(1+np.exp(-z))            torch.sigmoid(z)
  Activacion ReLU       np.maximum(0, z)            F.relu(z)
  Softmax               exp(z)/exp(z).sum()         F.softmax(z, dim=-1)
  Embeddings            tabla[indices]              nn.Embedding(V, D)
  LayerNorm             (x-mean)/std manualmente    nn.LayerNorm(d)
  Perdida BCE           -(y*log(p) + ...)           nn.BCELoss()
  Perdida CE multi      -log(probs[target])         F.cross_entropy()
  Gradientes            calcular a mano (~40 lin)   loss.backward()  (1 lin)
  Actualizar pesos      w -= lr * grad              optimizer.step()
  Optimizador SGD       w -= lr * grad              optim.SGD(params, lr=lr)
  Optimizador Adam      implementar manualmente     optim.Adam(params, lr=lr)
  Gradient clipping     np.clip(grad, -c, c)        clip_grad_norm_(params, c)
  Entrenamiento lotes   bucle manual por muestra    lote = stack([...]) auto
  Contar parametros     contar manualmente          sum(p.numel() for p in
m.parameters())
  Guardar modelo        np.save(pesos)              torch.save(modelo.state_dict())
  Cargar modelo         pesos = np.load()
modelo.load_state_dict(torch.load())
```

> 💡 La conclusión es clara: PyTorch no cambia los conceptos ni la arquitectura. Todo lo que aprendiste construyendo las redes desde cero con NumPy sigue siendo válido y directamente aplicable. PyTorch simplemente te libera del trabajo más mecánico (calcular gradientes a mano) para que puedas concentrarte en diseñar arquitecturas, preparar datos y mejorar los modelos.

D.5 Siguientes pasos con PyTorch

Con el conocimiento de este libro y este apéndice, estás preparado para dar los siguientes pasos en el ecosistema PyTorch:

* Hugging Face Transformers: la librería más usada para LLMs open-source. Ahora entiendes exactamente lo que hacen por dentro sus modelos GPT-2, LLaMA, BERT y docenas más.
* Fine-tuning: tomar un modelo preentrenado (por ejemplo GPT-2 de 124M parámetros) y continuarlo entrenando sobre datos propios. Con lo que sabes, el proceso es conceptualmente idéntico al entrenamiento del Nano-LLM.

* torch.compile: desde PyTorch 2.0, una sola línea puede acelerar el entrenamiento entre un 30% y un 300%. model = torch.compile(modelo).
* Datasets y DataLoader: la API estándar de PyTorch para cargar y preprocesar grandes conjuntos de datos de forma eficiente.
* Mixed Precision (torch.autocast): entrenar con float16 en lugar de float32 reduce mucho la memoria de las activaciones y puede acelerar bastante el cálculo en GPUs modernas (no todo baja a la mitad: los pesos maestros y los estados del optimizador suelen seguir en float32).

Apéndice E: Guía de Instalación — requirements.txt

Este apéndice es para ti si nunca has instalado librerías de Python o si quieres asegurarte de tener exactamente el entorno que usa el libro. No se asume ningún conocimiento previo de terminales ni de gestión de paquetes.

E.1 ¿Qué es requirements.txt?

Un archivo de texto con la lista de librerías que necesita el libro. En lugar de instalarlas una a una, le dices a Python: 'instala todo lo que hay en esta lista' con un único comando. Es el estándar profesional para compartir entornos de trabajo reproducibles.

E.2 Paso 1 — Verifica que tienes Python

Abre la terminal (en Windows, el Símbolo del sistema: busca CMD; en Mac, Terminal; en Linux, Ctrl+Alt+T) y escribe: python --version (en Mac y Linux, python3 --version)

Si ves 'Python 3.11.x' o superior, perfecto. Si no, descárgalo gratis desde https://python.org (elige la versión más reciente que te sugiera). En Windows, marca en la primera pantalla del instalador la casilla «Add python.exe to PATH» antes de pulsar «Install Now».

E.3 Paso 2 — Instala las librerías

Coloca el archivo requirements.txt en una carpeta y desde la terminal, navega hasta esa carpeta:

```
Windows:   cd C:\Users\TuNombre\Downloads
Mac/Linux: cd ~/Downloads
```

Luego crea el entorno virtual entorno_ia en tu carpeta de usuario (el mismo de la sección 1.4), actívalo e instala en él las librerías. En Linux y en macOS recientes pip no deja instalar fuera de un entorno virtual:

```
Windows (CMD):  python -m venv %USERPROFILE%\entorno_ia
                %USERPROFILE%\entorno_ia\Scripts\activate.bat
Mac/Linux:      python3 -m venv ~/entorno_ia
                source ~/entorno_ia/bin/activate
Solo en Linux, antes, PyTorch en versión CPU (si no, pip baja la de CUDA, de varios GB):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
Después, en los tres sistemas:
pip install -r requirements.txt
```

Verás cómo se descargan e instalan las librerías. Tarda unos minutos, según tu conexión; PyTorch es, con diferencia, la descarga más grande. Cuando vuelva a aparecer el cursor, habrá terminado.

E.4 Paso 3 — Verifica la instalación

Escribe esto en la terminal y pulsa Enter:

```
python -c "import numpy as np; print('Listo! NumPy', np.__version__)"
```

Si ves 'Listo! NumPy 2.x.x', el entorno está listo para empezar. Todas las salidas del libro se obtuvieron con NumPy 2 (versión 2.5.3) y se reproducen igual con la 2.4.6, la que pip instala con Python 3.11. Con NumPy 1 algunas cifras pueden cambiar en los últimos decimales, porque NumPy 2 cambió las reglas con las que combina float32 y float64 (lo verás en el Ejercicio 16.5). Lo mismo vale para otras versiones de PyTorch o para ejecutar en GPU: pueden cambiar los últimos decimales y, por acumulación, el texto que se genera, pero no lo que el libro explica. Para reproducirlas al decimal, el repositorio incluye requirements-probado.txt, con las versiones exactas usadas, y verificaciones/verificar_entorno.py, que comprueba el entorno y los archivos de GPT-2.

E.5 ¿Y si algo falla?

Los errores más comunes tienen solución sencilla:

- 'pip no se reconoce': prueba con python -m pip (o python3 -m pip) en lugar de pip

- 'python no se reconoce como un comando interno o externo' (Windows): reinstala Python y marca la casilla «Add python.exe to PATH» (paso 1)
- 'ensurepip is not available' (Ubuntu/Debian): instala el módulo de entornos virtuales con sudo apt install python3-venv y repite el paso 2
- 'externally-managed-environment' o 'Permission denied' (Mac/Linux): no uses sudo; activa el entorno virtual (paso 2) y repite el comando
- Cualquier otro error: copia el mensaje, pégalo en Google entre comillas y busca. La comunidad Python es enorme y casi cualquier problema tiene solución documentada.

E.6 Librerías incluidas

- numpy>=2.3 — computación numérica (imprescindible, Capítulos 1-20)
- matplotlib>=3.8 — gráficas y visualización (opcional)
- torch>=2.1, torchvision>=0.16 — PyTorch, para el Apéndice D y la carpeta verificaciones/ del repositorio (en un Mac con procesador Intel, usa el entorno aparte que crea su script; ver 1.4)
- transformers>=4.40, requests>=2.31 — Hugging Face y Ollama en el Capítulo 20 (Herramientas y próximos pasos), y la carpeta verificaciones/

No necesitas instalar nada más para ejecutar todos los programas del libro, salvo los pesos de GPT-2 del Capítulo 16 (E.7) y, si quieres probar el ejemplo de la sección 20.3, la aplicación Ollama (ollama.com).

> 📦 **Con esto estás listo. El resto lo aprenderás construyendo.**

E.7 Tiempos de ejecución aproximados

Medidos en un portátil, con NumPy y un solo núcleo de CPU. Tu equipo puede ser más rápido o más lento, y las proporciones pueden variar algo con la CPU y la biblioteca BLAS, pero el orden de magnitud se mantiene:

Programa	Tiempo aproximado
Programa 8 (Nano-LLM, 12 000 pasos)	2.5 minutos
RZ-1 (tras ejecutar el Programa 8)	2 minutos
RZ-2 (sumas)	3 minutos
RZ-3 (deducción con letras)	8 minutos
RZ-4 (deducción con frases)	35 minutos
PT-7 (PyTorch, CPU)	2 minutos

Los programas del Capítulo 16 necesitan además los pesos de GPT-2 small: 548 MB que se descargan una sola vez de huggingface.co/openai-community/gpt2: model.safetensors, vocab.json, merges.txt y config.json, en una carpeta gpt2 junto a los programas (16.1). Sus tiempos se midieron en el ordenador de pruebas del libro, de 2 núcleos; el Programa 8 bis tarda 1 minuto y 43 segundos con 2 hilos de cálculo y 1 minuto y 59 segundos con uno solo. La memoria es la máxima que usó cada programa:

Programa (Capítulo 16)	Tiempo	Memoria
Programa 8 bis (GPT-2 real)	2 minutos	1.1 GB
G-1 (lente de logits)	25 segundos	1.1 GB
G-2 (cabezas de inducción)	5 segundos	1.1 GB
G-3 (KV-cache)	2 minutos	2.1 GB
G-4 (ajuste fino, 200 pasos)	30 minutos (25 pasos: 6.5 minutos)	4.6 GB

Apéndice F — De la Academia a la Industria

Este apéndice se puede leer por separado. Está pensado para dos tipos de lector a la vez, así que cada tema empieza con una explicación en palabras normales, con analogías, y solo después llega la notación matemática formal — marcada aparte, en un recuadro, para quien la quiera o la necesite. Si la notación no es tu fuerte, puedes quedarte solo con la explicación y el código: es perfectamente suficiente para entender qué hace cada técnica y por qué importa. Se organiza en cinco apartados: F.1 explica las conexiones residuales, que permiten entrenar redes muy profundas; F.2 compara BatchNorm y LayerNorm; F.3 presenta la atención cruzada; F.4 ofrece una hoja de ruta profesional, y F.5 responde a la pregunta «¿Qué necesito saber para que me contraten?». El gradiente que se desvanece (Capítulo 10), la normalización Pre-LN (Capítulo 12) y las activaciones GELU y SiLU (Capítulo 12) se tratan en el cuerpo del libro.

F.1. Conexiones Residuales

La idea de las conexiones residuales es sorprendentemente simple: en vez de pedirle a cada capa que aprenda la transformación completa de la entrada a la salida, le pedimos que aprenda solo la diferencia — el 'residuo' — respecto a dejar la entrada tal cual está, y luego sumamos esa diferencia a la entrada original: salida = entrada + lo que aprende la capa.

Es como una autopista de circunvalación junto a las calles de una ciudad: si el tráfico por las calles (la transformación que aprende la capa) está colapsado, los coches (el gradiente) siempre tienen la autopista — el camino directo, sin modificar — para llegar igualmente a su destino. Ese camino directo nunca se atenúa, porque no multiplica nada: solo suma.

Esto resuelve el problema del gradiente que se desvanece (sección 10.4): aunque el 'camino de las calles' siga atenuando la señal capa a capa, el 'camino de la autopista' permite que, en la práctica, siga llegando gradiente a las primeras capas. He et al. (2015) lo demostraron entrenando ResNet, una red de 152 capas — algo impensable sin este truco.

> 🔬 En notación formal: sea X_l la salida del bloque l y F_l(X_l) la transformación que ese bloque aprende. Una conexión residual se define como X_{l+1} = X_l + F_l(X_l). El término ∂F_l/∂X_l es el Jacobiano de F_l (cómo cambia su salida al cambiar ligeramente su entrada), e I es la matriz identidad d×d (la que deja cualquier vector exactamente igual). Con conexiones residuales en todos los bloques, el gradiente que llega a la primera capa es: ∂L/∂X_1 = ∂L/∂X_L · ∏_{l=1}^{L} (I + ∂F_l/∂X_l). Aunque el Jacobiano ∂F_l/∂X_l ≈ 0 al inicio del entrenamiento, (I + 0) = I tiene autovalores = 1, así que, al menos al principio, el gradiente tiene un camino sin atenuación (más adelante podría haber cancelaciones, por ejemplo si ∂F_l/∂X_l se acercara a −I, pero no es lo habitual).

F.2. BatchNorm vs LayerNorm

Ambas técnicas hacen, en el fondo, lo mismo: reescalan un grupo de números para que tengan una media parecida a 0 y una dispersión parecida a 1 — piensa en 'curvar las notas de un examen' para que la media de la clase quede en un punto de referencia conocido. La diferencia está en QUÉ grupo de números se usa para calcular esa media y esa dispersión:

- BatchNorm: 'curva las notas' comparando cada característica entre todos los ejemplos del lote (batch) que se procesan a la vez. Necesita lotes grandes al entrenar para que la media signifique algo; en inferencia usa las medias que acumuló al entrenar, así que funciona con un solo ejemplo, pero en texto esas medias encajan mal con secuencias de longitud variable. Es el estándar en redes convolucionales con imágenes de tamaño fijo.
- LayerNorm: 'curva las notas' comparando las características DENTRO de un mismo ejemplo, no entre ejemplos distintos. Por eso funciona igual de bien con un lote de mil ejemplos que con uno solo — justo lo que necesita un LLM al generar texto token a token. Por eso los Transformers usan LayerNorm.

> 🔬 En notación formal: para cada característica d, sea x_d su valor original, μ la media y σ² la varianza (calculadas sobre el eje que corresponda, según BatchNorm

> o LayerNorm), y ε una constante pequeña que evita dividir entre 0. El valor normalizado es $\hat{x}_d = (x_d - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$. Como normalizar a media 0 y varianza 1 podría eliminar información útil, se añaden dos parámetros aprendibles — γ_d (escala) y β_d (desplazamiento) — que permiten a la red deshacer parte de la normalización si le conviene: $y_d = \gamma_d \cdot \hat{x}_d + \beta_d$.

```
import numpy as np

def layernorm(x, gamma, beta, eps=1e-5):
    """LayerNorm desde cero. x: (B, T, D)"""
    mu    = x.mean(axis=-1, keepdims=True)
    sigma = x.std(axis=-1, keepdims=True)
    return gamma * (x - mu) / (sigma + eps) + beta

x     = np.random.randn(2, 5, 64)
out   = layernorm(x, np.ones(64), np.zeros(64))
print(f"Media por token: {out[0,0].mean():.6f}  (debe ser ~0)")
print(f"Std  por token:  {out[0,0].std():.6f}   (debe ser ~1)")
```

F.3. Atención Cruzada (Cross-Attention)

En la Self-Attention que ya conoces del Capítulo 11, las tres piezas — Query, Key y Value — salen todas del mismo texto: cada palabra 'pregunta' a las demás palabras de su misma frase. En la Atención Cruzada, la Query sigue viniendo de un lado, pero la Key y el Value vienen de una fuente distinta:

- Query: la secuencia que está 'preguntando' — por ejemplo, el texto que se está generando.
- Key y Value: la fuente que se consulta — por ejemplo, una imagen, un audio o un documento de referencia.

La fórmula matemática es exactamente la misma que en la Self-Attention — un softmax de las puntuaciones entre Q y K, aplicado después a V —, pero ahora Q y K/V pueden tener longitudes distintas, así que la matriz de atención resultante ya no es cuadrada: cada palabra del texto puede 'mirar' un número distinto de fragmentos de la otra fuente. Es lo que ocurre, por ejemplo, en el modelo multimodal Flamingo (DeepMind, 2022) cuando le describes una foto: el texto que va generando 'consulta' constantemente a la imagen.

> 🔬 En notación formal: sea Q la matriz de Queries, K la de Keys, V la de Values, y d_k la dimensión de cada vector Key (el factor de escala que evita que el softmax se sature). La fórmula es $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$, igual que en la Self-Attention — pero aquí la matriz de puntuaciones $Q \cdot K^T$ tiene dimensión $(T_q \times T_{kv})$, el número de posiciones de la Query por el número de posiciones de Key/Value, en lugar de $(T \times T)$ como en la Self-Attention.

```
import numpy as np

def softmax(x, axis=-1):
    e_x = np.exp(x - x.max(axis=axis, keepdims=True))
    return e_x / e_x.sum(axis=axis, keepdims=True)

def cross_attention(Q, K, V):
    """Q: (T_q, d_k)  K,V: (T_kv, d_k)/(T_kv, d_v)"""
    d_k   = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)   # (T_q x T_kv) — NO cuadrada
    pesos  = softmax(scores)
    return pesos @ V                   # (T_q x d_v)

np.random.seed(42)
```

```
T_q, T_kv, d_k = 5, 12, 32   # 5 tokens texto, 12 tokens imagen
Q = np.random.randn(T_q, d_k)
K = np.random.randn(T_kv, d_k)
V = np.random.randn(T_kv, d_k)
salida = cross_attention(Q, K, V)
print(f"Q: {Q.shape}  K: {K.shape}  -> salida: {salida.shape}")
# Q: (5, 32)  K: (12, 32)  -> salida: (5, 32)
# El texto 'consulta' a la imagen: 5 tokens ven 12 tokens de imagen
```

F.4. Hoja de Ruta: Completar las Habilidades que Faltan

F.4.1. Matemáticas

Álgebra lineal: matrices, autovalores, SVD. 3Blue1Brown — Essence of Linear Algebra (YouTube). 'Linear Algebra Done Right', Axler.

Cálculo multivariable: gradientes, Jacobianos, Hessianos. Khan Academy — Multivariable Calculus.

Probabilidad: distribuciones, Bayes, entropía. StatQuest with Josh Starmer (YouTube).

F.4.2. PyTorch y Deep Learning

- Tutorial oficial de PyTorch: pytorch.org/tutorials
- fast.ai (course.fast.ai): top-down, modelos grandes desde el principio.
- Stanford CS224n (NLP) y CS231n (Visión): disponibles en YouTube.
- 'Annotated Transformer' (Harvard NLP): implementación línea a línea.

F.4.3. Infraestructura y MLOps (gran parte del trabajo real)

Docker y Kubernetes: contenerizar y orquestar entornos de entrenamiento.

Weights & Biases: monitorizar experimentos y visualizar gradientes (wandb.ai).

Hugging Face Transformers: leer el código fuente; entender pipeline internamente.

vLLM / TGI: despliegue con paginación KV-cache y batching dinámico.

DeepSpeed / PyTorch FSDP: entrenamiento distribuido a gran escala.

F.5. ¿Qué Necesito Saber para que me Contraten?

F.5.1. Los roles del mercado

ML Engineer / LLM Engineer: PyTorch avanzado, HuggingFace Transformers, fine-tuning con LoRA/QLoRA, vLLM.

Research Scientist: Matemática sólida (los Capítulos 4, 10 y 12 y este apéndice son el mínimo), leer papers, reproducir resultados.

Data Scientist / AI Engineer: RAG, prompt engineering, evaluación, criterio para fine-tuning vs RAG vs prompt.

MLOps Engineer: Docker, Kubernetes, Weights & Biases, CI/CD para modelos (ver F.4.3).

F.5.2. Las preguntas de entrevista más frecuentes

Pregunta: ¿Por qué se divide por √d_k en la atención?

> Respuesta: Para mantener la varianza de los scores en ~1 y evitar que el softmax se sature. Con d_k=64: dividir por 8.

Pregunta: ¿Por qué LLaMA usa Pre-LN en lugar de Post-LN?

> Respuesta: Pre-LN mantiene la varianza del gradiente estable en las primeras capas, permite learning rates más altos y reduce divergencia (loss=NaN).

Pregunta: ¿Por qué no se usa BatchNorm en Transformers?

> Respuesta: BatchNorm necesita lotes grandes al entrenar y, en inferencia, depende de medias acumuladas que encajan mal con secuencias de longitud variable. LayerNorm normaliza cada token por separado, sin depender del lote.

Pregunta: Tu modelo sobreajusta. ¿Qué haces?

Respuesta: Aumentar dropout, añadir weight decay, reducir tamaño, early stopping, más datos.

F.5.3. Lo que ningún proceso de selección pasa por alto

- Proyectos propios en GitHub: código limpio, documentado y ejecutable.
- Capacidad de explicar en voz alta qué hace cada línea de código.
- Experiencia en al menos un área concreta: NLP, visión, audio o código.

Cuando hayas completado los pasos de F.4, tendrás buena parte de la base práctica de un profesional. El conocimiento crece cuando se comparte: enseña, contribuye al open-source y escribe sobre tus experimentos.

Apéndice G: Visión por Computador y Redes Convolucionales

> 🎯 **AL TERMINAR ESTE APÉNDICE SABRÁS:**
> ✓ Explicar por qué el MLP no escala bien a datos con estructura espacial.
> ✓ Aplicar manualmente filtros de convolución y entender qué detecta cada uno.
> ✓ Implementar una CNN completa con backpropagation desde cero.
> ✓ Relacionar la arquitectura CNN con el Transformer: el ADN compartido.

> ℹ️ Este apéndice se sale del hilo principal del libro, que es el texto, para presentar la otra gran familia del Deep Learning: las redes convolucionales para imágenes. Es opcional. Te servirá para entender la multimodalidad (sección 20.5) y usa las mismas piezas que ya conoces: neuronas, ReLU, softmax, entropía cruzada y descenso por gradiente.

G.1 Una imagen es una matriz de números

Hasta ahora hemos trabajado con datos tabulares: vectores de números donde cada posición tiene un significado bien definido. El lenguaje también puede representarse como secuencias de números. Pero ¿qué ocurre con las imágenes?

Una imagen digital en escala de grises es, simplemente, una matriz de números enteros entre 0 y 255. Cada número representa la intensidad luminosa de un píxel: 0 es negro puro, 255 es blanco puro. Una imagen a color tiene tres matrices de este tipo, una por cada canal (rojo, verde, azul) — el famoso sistema RGB.

📄 **Una imagen pequeña representada como matriz numérica**

```
# Imagen de 8x8 píxeles en escala de grises
# Representa la letra 'T' de forma simplificada

imagen = [
  [255, 255, 255, 255, 255, 255, 255, 255],   # fila blanca
  [255,   0,   0,   0,   0,   0,   0, 255],   # borde superior
  [255,   0, 255, 255, 255, 255,   0, 255],   # T: barra horizontal
  [255,   0,   0, 255, 255,   0,   0, 255],   # T: palo vertical
  [255,   0,   0, 255, 255,   0,   0, 255],
  [255,   0,   0, 255, 255,   0,   0, 255],
  [255,   0,   0,   0,   0,   0,   0, 255],   # borde inferior
  [255, 255, 255, 255, 255, 255, 255, 255],   # fila blanca
]

# Una imagen de 224x224 píxeles RGB contiene:
# 224 x 224 x 3 = 150 528 números
# Una imagen Full HD (1920x1080 RGB): 6 220 800 números
```

G.2 Por qué el MLP no es la herramienta adecuada

Nuestro MLP del capítulo 6 funciona con vectores planos. Para usarlo con imágenes, deberíamos 'aplanar' la matriz en un vector largo y pasarlo como entrada. Esto tiene dos problemas graves:

* Explosión de parámetros: una imagen modesta de 64 × 64 píxeles tiene 4096 entradas. Si la primera capa oculta tiene 256 neuronas, necesitamos 4096 × 256 = más de un millón de pesos. Solo en la primera capa. La red se vuelve inmanejable.
* Estructura espacial oculta: al aplanar la imagen no se pierde ningún dato (el orden de los píxeles es fijo), pero el MLP trata cada píxel como una entrada independiente: no sabe cuáles son vecinos y tendría que aprenderlo de los datos. No aprovecha de entrada que los objetos son estructuras locales y coherentes, ni que un patrón puede aparecer en cualquier posición.

Las Redes Convolucionales (CNNs, Convolutional Neural Networks), propuestas por Yann LeCun en 1989 y popularizadas a partir de AlexNet en 2012, resuelven ambos problemas con un enfoque elegante: en lugar de conectar cada neurona a todos los píxeles, cada neurona solo observa una pequeña región local de la imagen. Y la misma 'plantilla de observación' se reutiliza en todas las posiciones de la imagen.

> ℹ️ La intuición detrás de las CNNs viene de la neurociencia: las células del córtex visual de los mamíferos responden a patrones locales (bordes, contrastes) en campos receptivos pequeños. LeCun simplemente formalizó esta idea en una operación matemática.

G.3 La operación de convolución

El corazón de una CNN es la convolución 2D. Un filtro (también llamado kernel) es una pequeña matriz de pesos, típicamente de 3 × 3 o 5 × 5. Este filtro se desliza por toda la imagen, posición por posición, y en cada posición calcula el producto punto entre los valores del filtro y los píxeles de la región que cubre. El resultado es un único número que va a parar al mapa de características (feature map). (En rigor, esta operación es una correlación cruzada, porque el filtro no se invierte; en Deep Learning se la llama convolución de todos modos.)

El proceso completo, para una imagen de tamaño H × W con un filtro de tamaño fH × fW, produce un mapa de características de tamaño (H − fH + 1) × (W − fW + 1). Cada valor en ese mapa responde a la presencia del patrón del filtro en esa posición de la imagen.

📄 **La convolución paso a paso (3x3 sobre 5x5)**

```
# Imagen 5x5 (simplificada):
# [ 1  2  3  0  1 ]
# [ 0  1  2  3  0 ]
# [ 1  0  1  2  3 ]
# [ 2  1  0  1  2 ]
# [ 0  2  1  0  1 ]

# Filtro 3x3 (detector de patrón diagonal):
# [ 1  0  0 ]
# [ 0  1  0 ]
# [ 0  0  1 ]

# Posición (0,0): cubre la submatriz superior izquierda 3x3
# Producto punto: 1*1 + 0*2 + 0*3 + 0*0 + 1*1 + 0*2 + 0*1 + 0*0 + 1*1 = 3

# Posición (0,1): desplazamos el filtro 1 columna a la derecha
# Producto punto: 1*2 + 0*3 + 0*0 + 0*1 + 1*2 + 0*3 + 0*0 + 0*1 + 1*2 = 6

# ...y así para todas las posiciones
# Mapa resultante: (5-3+1) x (5-3+1) = 3x3
```

G.4 Los filtros y lo que detectan

Aquí está la parte más fascinante: filtros distintos detectan características distintas. Los filtros clásicos del procesado de imagen (Sobel, Laplaciano, Gaussiano) se diseñaban a mano. En una CNN, los filtros se aprenden automáticamente mediante backpropagation, y el modelo descubre qué patrones son útiles para la tarea.

Los filtros más comunes que las CNNs aprenden en sus primeras capas son detectores de bordes (horizontales, verticales, diagonales), esquinas y manchas de color. En capas más profundas aprenden patrones más complejos: texturas, partes de objetos, y finalmente conceptos enteros en las capas más altas.

PROGRAMA 9A — Convolución manual y visualización de filtros

📄 **programa_9a_convolucion.py**

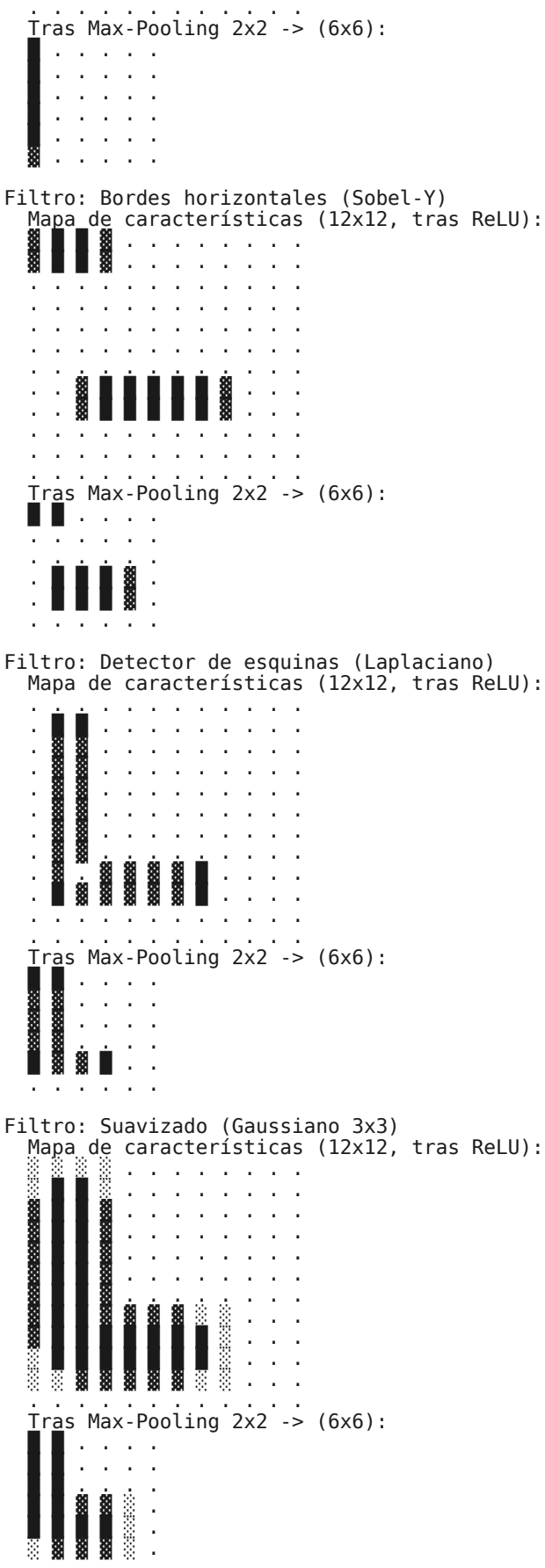
```
import numpy as np

# ============================================================
# CONVOLUCIÓN 2D MANUAL CON VISUALIZACIÓN ASCII
```

```
# Veremos cómo distintos filtros 'ven' la misma imagen
# ============================================================

def convolucion_2d(imagen, filtro, relleno=1):
    """
    Aplica un filtro a una imagen 2D.
    imagen:   (H, W)  — valores 0..1
    filtro:   (fH, fW)
    relleno:  píxeles de ceros añadidos al borde (para mantener tamaño)
    Retorna:  (H, W)  — mapa de características
    """
    H, W   = imagen.shape
    fH, fW = filtro.shape
    if relleno:
        imagen = np.pad(imagen, relleno, mode='constant', constant_values=0)
    H_r, W_r = imagen.shape
    H_out = H_r - fH + 1
    W_out = W_r - fW + 1
    resultado = np.zeros((H_out, W_out))
    for i in range(H_out):
        for j in range(W_out):
            region = imagen[i:i+fH, j:j+fW]
            resultado[i, j] = np.sum(region * filtro)
    return resultado

def max_pooling(feat_map, tamano=2):
    """
    Reduce el mapa de características tomando el máximo
    en cada ventana de tamano x tamano.
    """
    H, W = feat_map.shape
    H_out = H // tamano
    W_out = W // tamano
    resultado = np.zeros((H_out, W_out))
    for i in range(H_out):
        for j in range(W_out):
            ventana = feat_map[i*tamano:(i+1)*tamano,
                               j*tamano:(j+1)*tamano]
            resultado[i, j] = ventana.max()
    return resultado

def relu(x):
    return np.maximum(0, x)

def a_ascii(matriz, umbral=0.3):
    """Convierte una matriz en arte ASCII para visualización"""
    filas = []
    vmax = np.abs(matriz).max() + 1e-8
    for fila in matriz:
        chars = []
        for v in fila:
            v_norm = v / vmax
            if v_norm > 0.5:  chars.append('█')
            elif v_norm > 0.2: chars.append('▓')
            elif v_norm > 0.0: chars.append('░')
            elif v_norm < -0.5: chars.append('v')
            elif v_norm < -0.2: chars.append('▽')
            else:              chars.append('.')
        filas.append('  ' + ' '.join(chars))
    return chr(10).join(filas)   # CORREGIDO: salto de linea, no espacio


# -- Definir una imagen de 12x12: una 'L' mayúscula ----
# (1 = blanco/activado, 0 = negro/apagado)
imagen = np.array([
  [0,0,0,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
  [0,1,1,0,0,0,0,0,0,0,0,0],
```

```
    [0,1,1,1,1,1,1,1,1,0,0,0,0],
    [0,1,1,1,1,1,1,1,1,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0,0,0,0],
], dtype=float)

# -- Cuatro filtros clásicos de 3x3 --------------------
filtros = {
    'Bordes verticales (Sobel-X)': np.array([
        [-1, 0, 1],
        [-2, 0, 2],
        [-1, 0, 1]], dtype=float),

    'Bordes horizontales (Sobel-Y)': np.array([
        [-1,-2,-1],
        [ 0, 0, 0],
        [ 1, 2, 1]], dtype=float),

    'Detector de esquinas (Laplaciano)': np.array([
        [ 0,-1, 0],
        [-1, 4,-1],
        [ 0,-1, 0]], dtype=float),

    'Suavizado (Gaussiano 3x3)': np.array([
        [1/16, 2/16, 1/16],
        [2/16, 4/16, 2/16],
        [1/16, 2/16, 1/16]], dtype=float),
}

print('=== CONVOLUCIÓN 2D: DISTINTOS FILTROS, MISMA IMAGEN ===\n')
print('Imagen original (12x12 — letra L):',)
print(a_ascii(imagen))
print()

for nombre, filtro in filtros.items():
    mapa = convolucion_2d(imagen, filtro, relleno=1)
    mapa_relu = relu(mapa)                    # solo activaciones positivas
    mapa_pool = max_pooling(mapa_relu, 2)    # reducir a la mitad
    print(f'Filtro: {nombre}')
    print(f'  Mapa de características (12x12, tras ReLU):')
    print(a_ascii(mapa_relu))
    print(f'  Tras Max-Pooling 2x2 -> (6x6):')
    print(a_ascii(mapa_pool))
    print()
```

> Salida del programa:

```
=== CONVOLUCIÓN 2D: DISTINTOS FILTROS, MISMA IMAGEN ===

Imagen original (12x12 — letra L):
  . . . . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ . . . . . . . . .
  . ▒ ▒ ▒ ▒ ▒ ▒ . . . . .
  . ▒ ▒ ▒ ▒ ▒ ▒ . . . . .
  . . . . . . . . . . . .
  . . . . . . . . . . . .

Filtro: Bordes verticales (Sobel-X)
  Mapa de características (12x12, tras ReLU):
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
  ▒ ▒ . . . . . . . . . .
```



Observa cómo el filtro Sobel-X detecta exactamente los bordes verticales de la L (el palo vertical izquierdo y el borde derecho de la barra horizontal), mientras que Sobel-Y detecta los bordes horizontales (la unión entre el palo vertical y la barra). El Laplaciano detecta todas las esquinas. Cada filtro 've' una característica diferente de la misma imagen.

> 💡 En una CNN entrenada, los filtros de las primeras capas aprenden automáticamente detectores similares a Sobel y Laplaciano — sin que nadie se los programe explícitamente. Las capas más profundas combinan esos bordes en formas más complejas: arcos, círculos, texturas... hasta llegar a 'rueda', 'cara' o 'gato' en las capas finales.

G.5 Max Pooling: reducir sin olvidar lo importante

Después de la convolución, el mapa de características suele ser tan grande como la imagen original (con relleno) o algo menor. El Max Pooling reduce su tamaño tomando el valor máximo en pequeñas ventanas (típicamente 2 × 2). El máximo conserva la información de si el patrón apareció en esa región, independientemente de en qué píxel exacto dentro de la ventana.

El pooling tiene tres ventajas: reduce el número de parámetros de las capas siguientes, aporta cierta tolerancia a pequeños desplazamientos (si el borde se mueve un píxel dentro de la misma ventana, el máximo no cambia; si pasa a la ventana vecina, sí), y amplía el campo receptivo efectivo de las capas posteriores.

G.6 Arquitectura CNN completa

Una CNN típica apila alternativamente capas de convolución y pooling, y termina con una o más capas densas (como nuestro MLP) que hacen la clasificación final:

📄 **Arquitectura CNN estándar**

```
Entrada (imagen HxWxC)
    v
Conv2D(F1 filtros, 3x3) + ReLU    -> (HxWxF1)   # extrae características básicas
    v
MaxPool2D(2x2)                    -> (H/2xW/2xF1) # reduce dimensión
    v
Conv2D(F2 filtros, 3x3) + ReLU    -> (H/2xW/2xF2) # características más complejas
    v
MaxPool2D(2x2)                    -> (H/4xW/4xF2) # reduce más
    v
Flatten()                         -> (H/4 * W/4 * F2,)  # aplanar
    v
Dense + ReLU                      -> clasificación de alto nivel
    v
Dense + Softmax                   -> probabilidades de clase

# Ejemplos reales:
# LeNet-5 (1998):  2 conv + 2 pool + 3 dense — reconocía dígitos manuscritos
# AlexNet (2012):  5 conv + 3 pool + 3 dense — ganó ImageNet con 60M parámetros
# ResNet-50 (2015):50 capas conv con conexiones residuales — 25M parámetros
```

G.7 Una CNN que aprende sola

Vamos a construir una CNN desde cero que aprenda a distinguir dos tipos de patrones en imágenes de 10 × 10: imágenes con una barra horizontal y imágenes con una barra vertical. Es un ejemplo sencillo pero muestra el mecanismo completo: generación de datos, forward pass convolucional, backpropagation y evaluación.

PROGRAMA 9B — CNN entrenada para clasificar patrones

📄 **programa_9b_cnn.py (parte 1: datos y operaciones)**

```
import numpy as np

# ============================================================
# CNN ENTRENADA DESDE CERO
# Tarea: ¿barra horizontal o barra vertical?
# Arquitectura: Conv(4 filtros 3x3) -> ReLU -> MaxPool(2x2)
#               -> Flatten -> Dense(1) -> Sigmoid
# ============================================================

np.random.seed(42)
```

```
# -- Generador de datos sintéticos ---------------------
def generar_dataset(n_por_clase=150):
    """
    Clase 0: imagen 10x10 con una barra HORIZONTAL de 2 px de alto
    Clase 1: imagen 10x10 con una barra VERTICAL de 2 px de ancho
    Añadimos ruido gaussiano para hacerlo más realista.
    """
    imagenes, etiquetas = [], []
    for _ in range(n_por_clase):
        # Clase 0: barra horizontal
        img = np.zeros((10, 10))
        fila = np.random.randint(2, 7)        # fila aleatoria (evita bordes)
        img[fila:fila+2, 1:9] = 1.0           # barra de 2 px de alto
        img += np.random.randn(10, 10) * 0.15 # ruido
        imagenes.append(np.clip(img, 0, 1))
        etiquetas.append(0)
        # Clase 1: barra vertical
        img = np.zeros((10, 10))
        col = np.random.randint(2, 7)          # columna aleatoria
        img[1:9, col:col+2] = 1.0              # barra de 2 px de ancho
        img += np.random.randn(10, 10) * 0.15
        imagenes.append(np.clip(img, 0, 1))
        etiquetas.append(1)
    idx = np.random.permutation(len(imagenes))
    X = np.array(imagenes)[idx]     # (N, 10, 10)
    y = np.array(etiquetas)[idx]    # (N,)
    return X, y

X, y = generar_dataset(150)
# División entrenamiento / validación
corte = int(0.8 * len(X))
X_tr, y_tr = X[:corte], y[:corte]
X_val, y_val = X[corte:], y[corte:]
print(f'Dataset: {len(X_tr)} entrenamiento, {len(X_val)} validación')

# -- Operaciones base ----------------------------------
def conv_forward(img, filtros, sesgos, relleno=1):
    """
    img:     (H, W)  — imagen de un canal
    filtros: (F, fH, fW) — F filtros
    sesgos:  (F,)
    retorna: (F, H, W)  — F mapas de características
    """
    H, W   = img.shape
    F, fH, fW = filtros.shape
    if relleno:
        img_p = np.pad(img, relleno)
    else:
        img_p = img
    H_r, W_r = img_p.shape
    H_o = H_r - fH + 1
    W_o = W_r - fW + 1
    out = np.zeros((F, H_o, W_o))
    for f in range(F):
        for i in range(H_o):
            for j in range(W_o):
                out[f,i,j] = np.sum(img_p[i:i+fH, j:j+fW] * filtros[f]) + sesgos[f]
    return out

def max_pool_forward(feat, tam=2):
    """feat: (F, H, W) -> (F, H//tam, W//tam)"""
    F, H, W = feat.shape
    H_o, W_o = H//tam, W//tam
    out  = np.zeros((F, H_o, W_o))
    mask = np.zeros_like(feat, dtype=bool)   # guardamos posición del máximo
    for f in range(F):
        for i in range(H_o):
            for j in range(W_o):
                ventana = feat[f, i*tam:(i+1)*tam, j*tam:(j+1)*tam]
                out[f,i,j] = ventana.max()
                # Máscara: ¿dónde está el máximo?
                pos = np.unravel_index(ventana.argmax(), ventana.shape)
                mask[f, i*tam+pos[0], j*tam+pos[1]] = True
```

```
    return out, mask

def max_pool_backward(dout, mask, tam=2):
    """Propaga el gradiente solo al píxel que fue el máximo"""
    F, H_o, W_o = dout.shape
    dfeat = np.zeros_like(mask, dtype=float)
    for f in range(F):
        for i in range(H_o):
            for j in range(W_o):
                dfeat[f, i*tam:(i+1)*tam, j*tam:(j+1)*tam] += (
                    mask[f, i*tam:(i+1)*tam, j*tam:(j+1)*tam] * dout[f,i,j]
                )
    return dfeat

def sigmoid(x): return 1/(1+np.exp(-np.clip(x,-500,500)))
```

📄 **programa_9b_cnn.py (parte 2: forward, backward y entrenamiento)**

```
# -- Parámetros de la CNN ------------------------------
F  = 4     # número de filtros convolucionales
fS = 3     # tamaño del filtro: 3x3

# Inicialización He para los filtros
filtros = np.random.randn(F, fS, fS) * np.sqrt(2.0/(fS*fS))
sesgos  = np.zeros(F)

# Tras conv(relleno=1): (F, 10, 10)
# Tras maxpool(2):       (F, 5, 5)
# Tras flatten:          F*5*5 = 100 entradas
n_flat = F * 5 * 5

# Capas densas: 100 -> 16 -> 1
W1 = np.random.randn(n_flat, 16) * np.sqrt(2.0/n_flat)
b1 = np.zeros(16)
W2 = np.random.randn(16, 1) * np.sqrt(2.0/16)
b2 = np.zeros(1)

LR = 0.01

def forward_completo(img):
    """Pasa una imagen por toda la CNN"""
    # 1. Convolución + ReLU
    conv   = conv_forward(img, filtros, sesgos, relleno=1)
    relu_c = np.maximum(0, conv)              # (F, 10, 10)
    # 2. Max Pooling
    pool, mask = max_pool_forward(relu_c, tam=2)  # (F, 5, 5)
    # 3. Aplanar
    flat = pool.flatten()                     # (100,)
    # 4. Capa densa 1
    z1   = flat @ W1 + b1                     # (16,)
    a1   = np.maximum(0, z1)                  # ReLU
    # 5. Capa densa 2 -> salida
    z2   = a1 @ W2 + b2                       # (1,)
    pred = sigmoid(z2)                     # CORRECCION: array de 1 elemento (no
escalar)
    cache = (img, conv, relu_c, mask, pool, flat, z1, a1, z2)
    return pred, cache

def backward_completo(pred, target, cache):
    global filtros, sesgos, W1, b1, W2, b2
    img, conv, relu_c, mask, pool, flat, z1, a1, z2 = cache
    # -- Pérdida BCE y su gradiente ----------------------
    # BCE calculada desde el logit (estable): log(1 + e^z) - target·z
    loss = np.logaddexp(0, z2) - target * z2
    # BCE + sigmoide: dL/dz2 = pred - target (la derivada de la sigmoide se cancela)
    # -- Capa 2 ------------------------------------------
    dz2  = pred - target                      # gradiente respecto al logit z2
    dW2  = a1[:, None] * dz2[None, :]
    db2  = dz2
    da1  = dz2 @ W2.T                         # (16,)
    # -- Capa 1 ------------------------------------------
    dz1  = da1 * (z1 > 0)                     # ReLU
    dW1  = flat[:, None] * dz1[None, :]
    db1  = dz1
```

```
    dflat = dz1 @ W1.T                        # (100,)
    # -- Unflatten -> Max Pool backward -----------------
    dpool = dflat.reshape(F, 5, 5)
    drelu_c = max_pool_backward(dpool, mask, tam=2)  # (F,10,10)
    # -- ReLU backward ----------------------------------
    dconv = drelu_c * (conv > 0)              # (F,10,10)
    # -- Convolución backward (gradiente de filtros) ----
    img_p = np.pad(img, 1)                    # con relleno
    dF = np.zeros_like(filtros)
    db_conv = np.zeros_like(sesgos)
    H_o, W_o = dconv.shape[1], dconv.shape[2]
    for f in range(F):
        db_conv[f] = dconv[f].sum()
        for i in range(H_o):
            for j in range(W_o):
                dF[f] += img_p[i:i+fS, j:j+fS] * dconv[f,i,j]
    # -- Actualizar parámetros --------------------------
    filtros -= LR * dF
    sesgos  -= LR * db_conv
    W1 -= LR * dW1; b1 -= LR * db1
    W2 -= LR * dW2; b2 -= LR * db2
    return float(loss[0])  # CORRECCION: extraer escalar del array de 1 elemento

# -- Bucle de entrenamiento -----------------------------
print('\nEntrenando CNN...')
print(f'Parámetros: filtros={filtros.size}+{sesgos.size} sesgos,
dense={W1.size+b1.size+W2.size+b2.size}, '
      f'total={filtros.size+sesgos.size+W1.size+b1.size+W2.size+b2.size}')
print()
EPOCAS = 15
for epoca in range(EPOCAS):
    perm = np.random.permutation(len(X_tr))
    loss_total = 0.0
    for i in perm:
        pred, cache = forward_completo(X_tr[i])
        loss_total += backward_completo(pred, float(y_tr[i]), cache)
    # Validación
    correctos = sum(
        round(float(forward_completo(X_val[i])[0][0])) == y_val[i]
        for i in range(len(X_val)))
    acc_val = correctos / len(X_val)
    print(f'Época {epoca+1:2d}: pérdida={loss_total/len(X_tr):.4f}  '
          f'precisión validación={acc_val*100:.1f}%')

# -- Prueba final con imágenes concretas ----------------
print('\n=== PRUEBA VISUAL ===')
print('Imagen 1: barra horizontal')
img_h = np.zeros((10,10)); img_h[4:6, 1:9] = 1.0
pred_h, _ = forward_completo(img_h)
pred_h = float(pred_h[0])  # CORRECCION: extraer escalar
print(f'  Predicción: {pred_h:.4f}  '
      f'-> {"HORIZONTAL" if pred_h < 0.5 else "VERTICAL"} ✓' if pred_h < 0.5 else '✗')

print('Imagen 2: barra vertical')
img_v = np.zeros((10,10)); img_v[1:9, 4:6] = 1.0
pred_v, _ = forward_completo(img_v)
pred_v = float(pred_v[0])  # CORRECCION: extraer escalar
print(f'  Predicción: {pred_v:.4f}  '
      f'-> {"VERTICAL" if pred_v > 0.5 else "HORIZONTAL"} ✓' if pred_v > 0.5 else '✗')

# -- Visualizar filtros aprendidos ----------------------
print('\n=== FILTROS APRENDIDOS (3x3) ===')
print('(valores positivos █, neutros ., negativos ▽)')
for f in range(F):
    filt = filtros[f]
    vmax = np.abs(filt).max() + 1e-8
    print(f'  Filtro {f+1}:', end=' ')
    for fila in filt:
        row = ''.join('█' if v/vmax > 0.3 else '▽' if v/vmax < -0.3 else '.' for v in
fila)
        print(row, end='  ')
    print()
```

```
  > Salida del programa:
Dataset: 240 entrenamiento, 60 validación

Entrenando CNN...
Parámetros: filtros=36+4 sesgos, dense=1633, total=1673

Época  1: pérdida=0.2088  precisión validación=100.0%
Época  2: pérdida=0.0466  precisión validación=100.0%
Época  3: pérdida=0.0207  precisión validación=100.0%
Época  4: pérdida=0.0119  precisión validación=100.0%
Época  5: pérdida=0.0081  precisión validación=100.0%
Época  6: pérdida=0.0059  precisión validación=100.0%
Época  7: pérdida=0.0047  precisión validación=100.0%
Época  8: pérdida=0.0038  precisión validación=100.0%
Época  9: pérdida=0.0032  precisión validación=100.0%
Época 10: pérdida=0.0027  precisión validación=100.0%
Época 11: pérdida=0.0024  precisión validación=100.0%
Época 12: pérdida=0.0021  precisión validación=100.0%
Época 13: pérdida=0.0019  precisión validación=100.0%
Época 14: pérdida=0.0017  precisión validación=100.0%
Época 15: pérdida=0.0015  precisión validación=100.0%

=== PRUEBA VISUAL ===
Imagen 1: barra horizontal
  Predicción: 0.0002  -> HORIZONTAL ✓
Imagen 2: barra vertical
  Predicción: 1.0000  -> VERTICAL ✓

=== FILTROS APRENDIDOS (3x3) ===
(valores positivos █, neutros ., negativos ▽)
  Filtro 1: ...  ▽..  █▽.
  Filtro 2: .██  .██  ...
  Filtro 3: ...  ███  █..
  Filtro 4: ▽█.  ▽▽▽  ...
```

G.8 Lo que acabamos de construir

Con 1673 parámetros (apenas 40 en los filtros convolucionales, 36 pesos y 4 sesgos, y el resto en las capas densas), nuestra CNN alcanza el 100% de precisión distinguiendo barras horizontales de verticales con ruido, desde la primera época. Es un problema sencillo para esta arquitectura. Veamos qué aprendieron los filtros:

* Los cuatro filtros aprendidos no forman los detectores de bordes perfectamente limpios de un libro de texto, pero cada uno codifica una asimetría distinta entre filas y columnas (más activación arriba que abajo, o a la izquierda que a la derecha, etc.) — suficiente para que la capa densa combine esas señales y distinga barras horizontales de verticales con precisión perfecta.
* Esto ilustra algo importante: la red no necesita converger a los filtros "canónicos" que un ingeniero diseñaría a mano (un Sobel-X limpio, por ejemplo) para resolver la tarea — cualquier combinación de filtros que capture la asimetría relevante del problema sirve igual de bien, y el descenso de gradiente encuentra una de las muchas soluciones válidas.
*

El modelo descubrió por sí solo, a partir del gradiente, unos filtros que capturan la asimetría horizontal/vertical del problema — sin que nadie se los programara explícitamente.

> ℹ️ AlexNet (2012), la CNN que revolucionó la visión por computador ganando ImageNet, usa exactamente el mismo principio que hemos implementado, pero con 5 capas convolucionales, 60 millones de parámetros y entrenada sobre 1.2 millones de imágenes en dos GPUs durante una semana. El mecanismo es idéntico; la escala, diferente.

G.9 CNN y LLM: el mismo ADN

Puede parecer que las CNNs (para imágenes) y los Transformers (para texto) son tecnologías muy diferentes. Sin embargo, comparten el mismo ADN conceptual:

* Ambas usan operaciones locales que se reutilizan en distintas posiciones: la convolución desliza el mismo filtro por la imagen; la atención aplica las mismas matrices W_Q, W_K, W_V a todos los tokens.
* Ambas apilan capas que aprenden representaciones progresivamente más abstractas: de píxeles a bordes a objetos en la CNN; de caracteres a palabras a conceptos en el Transformer.
* Ambas se entrenan con backpropagation y gradiente descendente.
* Los modelos multimodales modernos combinan un encoder visual (una CNN o, cada vez más, un Vision Transformer) con un Transformer para texto, procesando ambas modalidades en el mismo espacio vectorial.

Con este apéndice, tu repertorio cubre ya los dos paradigmas dominantes del Deep Learning moderno. Tienes las herramientas conceptuales para entender casi cualquier arquitectura que encuentres en la literatura científica actual.

> 📌 **RESUMEN — LO ESENCIAL DE ESTE APÉNDICE**
>
> - La convolución aplica el mismo filtro en todas las posiciones de la imagen, explotando la estructura espacial local.
> - Max Pooling reduce dimensión y aporta cierta tolerancia (no una invarianza exacta) a pequeños desplazamientos.
> - CNN y Transformer comparten el mismo ADN: operaciones locales reutilizadas, apiladas en representaciones progresivas.

DIAGRAMA G.1 — CNN: ARQUITECTURA Y FLUJO DE DATOS

Resumen visual del Apéndice G — Redes Convolucionales

Filtro → Conv → ReLU → MaxPool → Flatten → Dense → Predicción

El mismo ADN que el Transformer: operaciones locales reutilizadas, apiladas en profundidad

ARQUITECTURA CNN COMPLETA

Flujo de una imagen de entrada a la predicción

1. **Imagen de entrada** — (H × W × C); 10 × 10 × 1 (escala de grises)
2. **Capa Convolucional** — F filtros de tamaño f×f → F mapas de características; parámetros: F × f² + F
   `salida = conv2d(imagen, filtros)`
3. **Activación ReLU** — `max(0, mapa)` — Suprime respuestas negativas
4. **Max Pooling 2×2** — Toma el máximo de cada bloque 2×2; H×W → H/2 × W/2 — Reduce dimensión, gana invarianza
5. Repetir Conv → ReLU → Pool × N bloques — Cada bloque aprende patrones más abstractos
6. **Flatten → vector 1D** — Capa Densa (fully connected) — Combina todas las características
7. **Predicción final** — sigmoid → P(vertical) — Ej: barra vertical → P(vertical) = 1.0000

```
filtros 4×3×3 + 4 sesgos:   40 params
capa densa:                 1633 params
-------------------------------------
TOTAL CNN:                  1673 params
```

vs Transformer: misma idea, diferente dominio

→ cómo funciona la convolución

LA CONVOLUCIÓN — CÓMO FUNCIONA

Un filtro 3×3 desliza sobre la imagen calculando productos escalares

Imagen (6×6)

0	0	1	0	0	0
0	0	1	0	0	0
0	0	1	0	0	0
0	1	1	0	0	0
0	1	1	1	0	0
0	0	0	0	0	0

Filtro Sobel-X

−1	0	+1
−2	0	+2
−1	0	+1

detector de bordes verticales

Mapa de características (4×4)

4	0	−4	0
4	−1	−4	0
4	−2	−4	−1
3	−1	−3	−2

sin relleno: 6×6 → 4×4 (con relleno=1, como el Programa 9A, sería 6×6)

Cálculo en posición (1,1):

```
suma = Σ imagen[i,j] × filtro[i,j]
     = 0×(−1) + 0×0 + 1×(+1)
       + 0×(−2) + 0×0 + 1×(+2)
       + 0×(−1) + 0×0 + 1×(+1) = 4
resultado = 4 → borde vertical detectado
```

Parameter Sharing — la clave de las CNNs:

El MISMO filtro 3×3 se aplica en TODAS las posiciones.
Una imagen 10×10 tiene 100 posiciones pero solo 9 parámetros.
→ El modelo aprende UN detector de borde válido para toda la imagen.
→ Si aprende 'borde vertical' en esquina, lo reconoce en el centro.

Esto se llama *invarianza translacional*.

$$output[i,j] = \sum_m \sum_n imagen[i+m, j+n] \times filtro[m,n] + sesgo$$

Complejidad: $O(H \times W \times f^2 \times F)$ por capa convolucional

→ los filtros clásicos

LOS FILTROS CLÁSICOS (PROGRAMA 9A)

Diseñados a mano; una CNN aprende detectores parecidos

Filtro 1 — Bordes HORIZONTALES (Sobel-Y)

−1	−2	−1
0	0	0
+1	+2	+1

activa en cambios de arriba a abajo

Filtro 2 — Bordes VERTICALES (Sobel-X)

−1	0	+1
−2	0	+2
−1	0	+1

activa en cambios de izquierda a derecha

Filtro 3 — Detector de ESQUINAS (Laplaciano)

0	−1	0
−1	+4	−1
0	−1	0

activa en cruces y esquinas

Filtro 4 — SUAVIZADO (Gaussiano 1/16)

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

elimina ruido, borra bordes finos

¿Y qué aprende la CNN?

La CNN NO recibe instrucciones sobre bordes.
Solo ve: imagen → etiqueta (horizontal/vertical).
El backpropagation ajusta los 9 valores de cada filtro hasta que separan las dos clases.

En redes grandes: detectores parecidos a Sobel.

→ conexión con el Transformer

CNN ↔ TRANSFORMER: EL MISMO ADN

Dos arquitecturas distintas, un principio compartido

Analogía estructural:

Concepto	CNN	Transformer
Operación base	Convolución 3×3	Self-Attention
Dónde opera	Ventana local f×f	Todos los tokens
Reutilización	Mismo filtro en toda la img	Mismas W_Q, W_K, W_V por bloque
Apila	Conv → ReLU → Pool	LN → MHA → LN → FFN
Aprende	Detectores de borde	Relaciones entre tokens
Comparte pesos	Sí (espacial)	Sí (por bloque)

CNN sobre imagen — Misma operación 3×3 aplicada N veces → detecta patrón LOCAL

misma idea

Transformer sobre texto — Misma Q/K/V aplicada a cada token → captura relación GLOBAL

El principio unificador del Deep Learning:

1. Define una operación PARAMETRIZADA pequeña (un filtro 3×3 / una cabeza de atención)
2. REUTILÍZALA en toda la secuencia/imagen (parameter sharing)
3. APILA múltiples capas (jerarquía de representaciones)
4. Entrena con backpropagation

Misma receta, distinto dominio: imágenes (CNN) y secuencias (Transformer)

El futuro: modelos multimodales

GPT-4V, Gemini, LLaMA-3.2V combinan:
✓ Un encoder CNN (o ViT) para imágenes
✓ Un Transformer para texto
✓ Un mecanismo de fusión entre ambos

Con el Capítulo 12 y el Apéndice G tienes las bases de AMBOS paradigmas del Deep Learning moderno.

$$output[i,j] = \sum imagen[i+m,j+n] \times filtro[m,n] + b$$

$$ReLU(x) = max(0, x) \quad MaxPool = max\ de\ bloque\ 2\times2$$

$$params = F\cdot(f^2+1) + 16\cdot(n_flat+1) + (16+1)$$

Convolución: filtro deslizante | **Pooling: invarianza + reducción** | **ADN común con el Transformer**

Blue = Imagen de entrada / píxeles
Teal = Filtros convolucionales / mapas
Green = ReLU / respuestas positivas
Orange = Max Pooling / reducción
Purple = Capas densas / predicción
Gold = Pesos del filtro / parámetros

✏️ EJERCICIOS — Apéndice G — Visión y CNNs

Para cerrar el apéndice, pon a prueba lo aprendido sobre visión por computador. Resuelve los ejercicios y compara con la solución: los filtros y las convoluciones se entienden mejor practicando.

Ejercicio G.1 Diseñar un filtro detector de bordes diagonales

Diseña (a mano) un filtro 3 × 3 que detecte bordes diagonales en la dirección \ (diagonal principal). Aplícalo a una imagen 10 × 10 que contenga una línea diagonal (la matriz identidad multiplicada por 1) y a una imagen con una barra horizontal. El filtro debe activarse fuertemente sobre la diagonal y poco sobre la barra.

> 💡 💡 Pista: Un detector de diagonal \ debería tener valores positivos en la diagonal principal del filtro y negativos fuera de ella. Piensa en él como: '¿hay más brillo en la diagonal que en el resto de la ventana 3 × 3?'

✅ Solución:

📄 ejercicio_G_1.py

```
import numpy as np

def convolucion_2d(img, filtro, relleno=1):
    H, W = img.shape; fH, fW = filtro.shape
    if relleno: img = np.pad(img, relleno)
    H_r, W_r = img.shape
    out = np.zeros((H_r-fH+1, W_r-fW+1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i,j] = np.sum(img[i:i+fH, j:j+fW] * filtro)
    return out

# Filtro diagonal principal \
filtro_diag = np.array([
    [ 2, -1, -1],
    [-1,  2, -1],
    [-1, -1,  2]], dtype=float)

# Imagen 1: diagonal (np.eye)
img_diag = np.eye(10)
# Imagen 2: barra horizontal
img_hor  = np.zeros((10,10)); img_hor[4:6, :] = 1.0

for nombre, img in [('diagonal', img_diag), ('barra horiz.', img_hor)]:
    resultado = np.maximum(0, convolucion_2d(img, filtro_diag, 1))
    activacion_total = resultado.sum()
    print(f'Imagen {nombre}: activación total = {activacion_total:.2f}')

print()
print('El filtro debe activarse mucho más sobre la diagonal')
print('que sobre la barra horizontal.')
print()
print('Variante: filtro diagonal secundaria /')
filtro_anti = np.array([[-1,-1,2],[-1,2,-1],[2,-1,-1]], dtype=float)
print('  filtro_anti =', filtro_anti.tolist())
```

Ejercicio G.2 El campo receptivo crece con la profundidad

Implementa una función que calcule el campo receptivo de una CNN capa por capa: cada convolución 3×3 añade 2 «saltos» al campo y cada MaxPool 2×2 añade uno y duplica el salto para las capas siguientes (el salto es la distancia, en píxeles de la imagen, entre dos posiciones vecinas de la capa). Calcula para nuestras CNNs del Programa 9B y una arquitectura de 5 capas convolucionales.

> 💡 💡 Pista: El campo receptivo es el tamaño de la región de la imagen original que 've' una neurona en la capa actual. Con Conv(3×3) + Pool(2×2) + Conv(3×3): 1 → 3 (conv) → 4 (pool; desde aquí el salto es 2) → 4 + 2·2 = 8 (conv). Cuantas más capas, mayor la región que procesa cada neurona.

✅ Solución:

📄 ejercicio_G_2.py

```
def campo_receptivo(capas):
    """
    capas: lista de 'conv3' o 'pool2'
    Retorna el campo receptivo y el historial paso a paso.
    """
    campo, salto = 1, 1   # salto: píxeles de la imagen entre dos posiciones vecinas
    hist  = [('entrada', campo)]
    for capa in capas:
        if capa == 'conv3':
            campo += 2 * salto      # el filtro 3x3 abarca un vecino más a cada lado
            hist.append(('conv 3x3', campo))
        elif capa == 'pool2':
            campo += salto          # la ventana 2x2 abarca una posición más...
            salto *= 2              # ...y a partir de aquí las posiciones van de 2 en
2
            hist.append(('maxpool 2x2', campo))
    return campo, hist

arquitecturas = {
    'Nuestra CNN simple':   ['conv3', 'pool2'],
    'CNN con 2 pools':      ['conv3', 'pool2', 'conv3', 'pool2'],
    'VGG-style (5 conv)':   ['conv3','conv3','pool2','conv3','conv3','pool2','conv3'],
    'Muy profunda (8 conv)':['conv3','conv3','conv3','conv3',
                             'pool2','conv3','conv3','conv3','conv3'],
}

for nombre, arq in arquitecturas.items():
    campo_final, hist = campo_receptivo(arq)
    print(f'  {nombre}:')
    for nombre_capa, c in hist:
        print(f'    {nombre_capa:>15}: campo = {c}x{c}')
    print(f'    -> Una neurona profunda ve {campo_final}x{campo_final} px\n')
```

Ejercicio G.3 Adaptar la CNN para 3 clases

Modifica la arquitectura del Programa 9B para clasificar 3 tipos de patrones: horizontal, vertical y diagonal. El cambio principal está en la capa de salida: de Dense(16->1)+Sigmoid a Dense(16->3)+Softmax. Genera el dataset con las 3 clases y explica cómo cambia la función de pérdida.

> 💡💡 Pista: Para clasificación multiclase, la capa de salida produce 3 logits (uno por clase). Softmax los convierte en probabilidades. La pérdida es la entropía cruzada multiclase: -log(P(clase_correcta)).

✅ Solución orientativa (los datos y los cambios principales; entrenarla y evaluarla queda como ampliación):

📄 ejercicio_G_3.py

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max()); return e / e.sum()

def generar_3_clases(n=80):
    X, y = [], []
    np.random.seed(1)
    for _ in range(n):
        for clase in range(3):
            img = np.zeros((10,10))
            ruido = np.random.randn(10,10)*0.1
            if clase == 0:   # horizontal
                f = np.random.randint(2, 7)
                img[f:f+2, 1:9] = 1.0
            elif clase == 1: # vertical
                c = np.random.randint(2, 7)
                img[1:9, c:c+2] = 1.0
```

```
            else:            # diagonal
                for i in range(8): img[i,i] = 1.0
            X.append(np.clip(img+ruido, 0, 1))
            y.append(clase)
    idx = np.random.permutation(len(X))
    return np.array(X)[idx], np.array(y)[idx]

X, y = generar_3_clases()
print(f'Dataset: {len(X)} imágenes, 3 clases')
print(f'Clase 0 (horiz): {sum(y==0)}  Clase 1 (vert): {sum(y==1)}  Clase 2 (diag):
{sum(y==2)}')
print()
print('Cambios respecto al Programa 9B (clasificación binaria):')
print('  ANTES: Dense(16->1) + Sigmoid + BCE loss')
print('  AHORA: Dense(16->3) + Softmax + CE multiclase')
print()
print('Función de pérdida multiclase:')
print('  logits = red.forward(img)  # (3,)')
print('  probs  = softmax(logits)   # (3,) suman 1')
print('  loss   = -log(probs[clase_correcta])')
print()
logits_ej = np.array([2.0, 0.5, -0.3])
probs_ej  = softmax(logits_ej)
for i, (nombre, p) in enumerate(zip(['horiz','vert','diag'], probs_ej)):
    print(f'  P({nombre}) = {p:.4f}  {"<- clase predicha" if i==np.argmax(probs_ej)
else ""}')
```

ℹ️ La pérdida mínima posible es 0 (predicción perfecta) y la de un modelo que reparte la probabilidad por igual entre las 3 clases es ln(3) ≈ 1.099 (no es un máximo: si se equivoca con seguridad, la pérdida puede ser mucho mayor). Con 3 clases en lugar de 2 puede hacer falta algo más de capacidad en las capas convolucionales (más filtros o más bloques) para obtener la misma precisión; conviene comprobarlo antes de añadirla.

Índice alfabético de materias

Términos del libro ordenados alfabéticamente. Los números remiten a las páginas en las que cada término se explica o se usa de forma destacada (hasta ocho por término). Las definiciones breves están en el Glosario (Apéndice B).

A

Activación, funciones de: 16-18, 23, 25, 58, 341
Adam: 41, 44-45, 49, 53-54, 153, 160
AdamW: 41, 45, 49, 52-53, 55, 179, 347
AI Act (Reglamento europeo de IA): 308
Ajuste fino (fine-tuning): 267-268, 280, 282, 284, 317, 350, 356
Alucinación: 259, 295-297
Atención (self-attention): 18, 124-125, 136, 139, 307, 345, 355
Atención causal: 129, 149, 151, 215
Atención cruzada (cross-attention): 314, 316, 354-355
Autograd: 267, 317, 340-342
Autorregresivo, modelo: 149, 182, 234, 279, 282, 284

B

Backpropagation (retropropagación): 9, 32, 58, 149, 154, 182, 267, 340
Banco y río (ambigüedad): 130-131, 134
BatchNorm: 354, 356
Beam search: 210
Benchmarks: 221, 288, 295-297, 299
BERT: 106, 139, 317, 350
BLEU: 288, 291-292, 298
BPE (Byte Pair Encoding): 89, 97, 99, 102, 104, 106-108
Byte-level BPE: 104

C

Cabezas de inducción: 234, 251, 262, 264, 271, 274, 353
Calibración: 202, 210, 288, 295, 297, 302, 305, 307
Capacidades emergentes: 11, 51, 279, 284
Chain of thought (razonar paso a paso): 228, 234, 282
CNN (red convolucional): 291, 311, 314, 358-359, 363, 367-368
Codificación posicional: 139-140, 143, 146, 149, 182
COMPAS: 302
Conexión residual: 139-141, 143, 150, 260, 262, 354
Contaminación de datos: 221, 288, 296, 299
Convolución: 316, 354, 358-359, 363, 367-368, 370
Corpus: 97, 108, 151, 177, 179, 209, 234, 280
Cross-entropy (entropía cruzada): 31, 151-152, 154, 182, 184, 186, 280

D

Data augmentation: 82
Decaimiento coseno: 53, 149, 153, 179, 182, 192, 248, 270
Decoder-only: 279, 284
DeepSeek: 11, 242, 282
Descenso por gradiente: 9, 26, 31-35, 41
Diferencias finitas: 149, 182
Difusión, modelos de: 314, 316, 318
DPO: 279, 281, 284

Dropout: 65, 75, 77, 79, 83, 85, 87

E

Early stopping: 65, 79, 82-83, 85, 87, 179, 244
Embeddings: 83, 90-91, 93, 149-150, 182, 344
Época (epoch): 10, 38-39, 41, 49, 70, 82, 152
Equidad (fairness): 301, 305, 307, 309-310
Escalado de la atención (√d_k): 130, 134, 355-356
Escalado, leyes de: 286, 317
Evaluación de modelos: 288, 291, 296

F

Feed-forward (FFN): 140-141, 143, 145, 149-151, 154
Fertilidad del token: 97, 104, 106, 108
Forward pass: 152, 154, 158, 251-252, 256, 265, 282
Función de pérdida: 31, 72, 79, 154, 280-281, 303, 340

G

GELU: 141, 149-151, 155, 158, 182, 199
Generalización: 45, 65, 70, 74, 85, 120, 149, 234
Glitch tokens (SolidGoldMagikarp): 105
Goodhart, ley de: 288, 295, 297
GPT-2: 90, 104, 150-151, 177, 251, 267, 270
GPT-3: 92, 104-106, 143, 153, 177, 279
GPU: 10, 12, 120, 139, 251, 280, 282, 286
GQA (Grouped-Query Attention): 140, 151, 283
Gradient clipping (recorte del gradiente): 51, 53, 153, 182, 197, 199, 215, 267
Gradiente: 9, 31, 41, 43-45, 114, 153
Gradiente que desaparece (vanishing gradient): 110, 114, 119-120, 354
Greedy, decodificación: 106, 161, 202, 206, 209-210, 258-259

H

Hiperparámetros: 72, 74, 156, 176, 179, 187, 216, 237
Hugging Face: 11, 256, 258, 271, 311, 317, 350, 356
Humanity's Last Exam: 295, 297

I

Inicialización de pesos: 23, 25-26, 64, 70, 121, 198, 227

K

KV-cache: 119, 151, 251, 265, 275, 282-283

L

LayerNorm: 139, 141, 143, 145, 150, 155, 182, 354
Learning rate (tasa de aprendizaje): 33, 38, 41, 43, 45, 49, 153, 182
Lente de logits (logit lens): 251, 260, 262, 271, 273, 353
Línea base (baseline): 215-216, 221, 243, 305
LLaMA: 11, 106, 139-140, 151, 153, 280, 282
Logits: 14, 112, 150, 155, 202, 259-260, 271
LoRA: 356
LSTM: 117, 119-121

M

Máscara causal: 124, 129, 131, 134, 138, 251, 265, 280
Max pooling: 363, 368
Memorización: 65, 86, 177, 215-216, 221, 242-243
Meseta (plateau): 70, 82, 86, 226-228, 242, 248
Mini-lote (batch): 41, 149, 152, 154, 160, 162, 177, 182
Mixture of Experts (MoE): 151
MLP (perceptrón multicapa): 9, 57-59, 62-63, 110, 358
MMLU: 291, 295-297
Model Cards: 308-309
Momentum: 41-44, 49, 53-54
MSE (error cuadrático medio): 31-32, 343
Multi-head attention: 137, 139-140, 143, 150, 215, 283, 346
Multimodalidad: 311, 314, 318, 355, 358, 368

N

N-gramas: 215-216, 219, 292, 298
Nano-LLM: 14, 105, 149-150, 177, 216, 267, 279
Neurona artificial: 9-10, 16, 18, 20, 24
NumPy: 10, 12, 23, 251, 276, 340, 350, 352

O

Ollama: 312, 318, 353
One-hot: 90-91
Optimizadores: 9, 41, 45, 47, 49, 52, 54, 340
Overfitting (sobreajuste): 9, 65, 67, 70, 72, 86, 177, 234

P

Parámetros, recuento de: 63, 145, 150, 279, 286, 347, 363
Perceptrón: 10, 16, 57
Perplejidad: 152, 177, 221, 257, 275, 288-289, 298
Pesos compartidos (weight tying): 149-150, 155, 182, 215, 230, 347
Post-LN / Pre-LN: 141, 143, 149-150, 155, 182, 215, 356
Preentrenamiento: 120, 267, 270-271, 280, 284, 296, 300
Prompt: 247, 264, 281, 316, 319, 356
PyTorch: 11-12, 317, 340, 342, 347, 350, 356

Q

Query, Key, Value: 93, 124-125, 130, 134, 138, 283, 355

R

RAG (Retrieval-Augmented Generation): 93, 311-312, 318-319, 356
Razonamiento, modelos de: 151, 242, 282
Regla de la cadena: 32, 35, 58, 149, 152, 154, 182, 340
Regularización L2 (weight decay): 45, 49, 53, 55, 72, 74, 179, 200
ReLU: 16-18, 20, 23, 28, 59, 141
Repetition penalty: 202, 206, 209, 211, 259
RLHF: 279, 281-282, 284, 296
RMSNorm: 140, 151
RMSprop: 41, 43-44, 49, 53
RNN (red recurrente): 51, 110, 112, 114, 119-122
RoPE: 140, 151

S

Safetensors: 251-252, 271, 353
Semilla aleatoria: 20, 64, 70, 227-228, 235, 242, 245
Sesgo (en los datos y en los modelos): 17, 21, 70, 301-303, 306, 309
SFT (ajuste supervisado): 279-281, 284
SGD: 41-45, 49, 53-54
Sigmoide: 16-18, 20-21, 23, 25, 32
SiLU / Swish: 140, 354
Similitud coseno: 90, 92-93, 95, 146, 312, 319
Softmax: 14, 18, 134, 136, 154, 202, 355, 371
SwiGLU: 140, 151

T

Tanh: 16-18, 114, 121, 141, 155
Temperatura: 14, 149, 182, 184, 202, 212-213, 275
Tokenización: 89-90, 97, 99, 104-105, 151, 256
Tokens: 11, 90-91, 104-105, 202, 274, 280
Top-k: 161, 202-203, 206, 209-211
Top-p (nucleus sampling): 149, 161, 182, 202-203, 206, 210, 212
Train / validación / test: 47, 65, 70, 72, 178-179, 216, 298
Transformer: 10, 110, 119-120, 139-141, 311

U

Underfitting: 55, 65, 70, 74

V

Ventana de contexto: 140, 178, 282
Vision Transformer (ViT): 314, 318, 368
Vocabulario: 90, 94, 104, 106, 149-150, 183, 185

W

Warm-up: 41, 49, 52-53, 149, 153, 182, 192
wte / wpe (tablas de embeddings): 140, 150-151, 155, 230, 252, 267, 270

X

XOR: 30, 41, 57, 60-64

Recursos oficiales

Proyecto IA — De la Neurona al LLM

Descripción	URL	QR
Versión digital *Repositorio en GitHub*	github.com/csoria39/proyecto-ia-libro	
Edición impresa *Disponible en breve en Amazon*	www.amazon.com	
Donaciones *Ko-fi (importe libre, con tarjeta)*	ko-fi.com/delaneuronaalllm	
Contacto *Correo electrónico*	de.la.neurona.al.llm@gmail.com	